



DS Prep

THE COMPANION GUIDE

Data Science Interview Foundations

Concepts, mental models, interview tactics and cheat sheets for the beginner data scientist

18

CHAPTERS

121

TOPICS

850+

QUESTIONS ANSWERED

12

CHEAT SHEETS

Rahul Kapil

Beginner edition



DS Prep

Data Science Interview Foundations

The Companion Guide

Created by **Rahul Kapil**

DS Prep Companion Guide: Data Science Interview Foundations. Beginner edition.
Created by Rahul Kapil.

This book is the print and e-book companion to the DS Prep web app. The two share one structure: Chapter 7 here is Module 7 there. Company names, tools and products are mentioned for teaching purposes only and belong to their owners. Interview processes differ between companies and change over time, so treat every process description as typical, not guaranteed.

Built from DS Prep content version 2026-10-04.

About this book

Most people preparing for a data science interview do not lack material. They lack a map. There are thousands of videos, question lists and courses, and it is hard to tell which ideas are the load-bearing ones and which are decoration. This book is the map.

It does three jobs. First, it gives you the **foundations**: for every area an interviewer can test, a short essay explains the mental model that the details hang from. If you understand why a window function keeps rows while GROUP BY collapses them, you do not need to memorise forty queries. Second, it gives you the **fundamentals topic by topic**, in the same order as the DS Prep app, each with a plain-language explanation, the facts to know, a model answer you could say aloud, and questions to test yourself. Third, it prepares you for the **interview itself**: tactics that experienced candidates use, the questions that come up again and again with model answers, and a two-page cheat sheet at the end of each technical chapter.

Who it is for

You are a student, a recent graduate or a working professional moving into data science or analytics. You can write a little code, or you are willing to learn. You do not need a statistics degree. Every chapter starts from the beginning, uses everyday language first, and introduces a formula only after the idea is clear.

What "beginner level" means here

It does not mean shallow. It means that nothing is assumed, that the order is deliberate, and that the goal is the level interviewers actually test for entry and junior roles: explain a concept simply, reason about why it works, and apply it to a small realistic problem. Chapter 0 calls these the three levels of knowledge, and the whole book is written to get you to the third.

How the book is organised

Chapters 0 to 17 mirror the eighteen modules of the app. Chapters 1 to 12 are the technical core, from Python and SQL through statistics, machine learning and deployment. Chapters 13 to 15 cover business cases, projects, your resume and the behavioural round. Chapter 16 is a rapid-fire question bank and Chapter 17 turns everything into a plan. The appendices collect study plans, formulas, decision tables and checklists, and the glossary defines every term in plain words.

Every chapter has the same five parts, so you always know where you are:

1. **Foundations.** The conceptual framework for the whole area.
2. **Topic by topic.** One short section per topic: "In plain words", a **worked example, step by step**, a "**Your turn**" exercise with its solution, "The fundamentals", "Say it in 60 seconds", key terms, hands-on practice, common mistakes and "Check yourself".
3. **Interview tips, tricks and hacks.** What to do in the room.
4. **Frequently asked questions.** Real interview questions with model answers.
5. **Cheat sheet** (Chapters 1 to 12). Two dense pages for last-minute revision.

How to use this book with the app

The book and the app are designed to be used together, and each does something the other cannot.

Use the book for	Use the app for
Reading away from a screen, with a pen	Worked examples with runnable code and verified outputs
The big picture of each area before you start it	Interactive demos: joins, the CLT, sample size, bias and variance, attention
Model spoken answers and frequently asked questions	Hide-and-reveal practice with self-rating, and the review drill
Cheat sheets on paper before an interview	A study planner that puts topics on real dates and tracks progress
Worked examples you can follow with a pencil	The same examples on screen, plus code you can run in your practice lab (topic 0.3)
The glossary and quick reference at the back	The same glossary and reference pages, searchable, from the Library

A routine that works

1. **Before a module:** read the chapter's Foundations essay. Ten minutes, no notes. You are building the frame.
2. **During the module:** study each topic in the app. Afterwards, read the same topic here and say the "Say it in 60 seconds" answer aloud in your own words. If you cannot, you have found tomorrow's revision.
3. **After the module:** answer the "Check yourself" questions with the answers covered, then work through the chapter's frequently asked questions aloud. Mark every one you miss.
4. **The week before an interview:** read only three things per chapter: the tips, your marked questions and the cheat sheet.

How to read a topic

Every topic follows the same teaching order: understand, see it done, do it, then explain it. Read "In plain words" to get the idea. Work through the **worked example** with a pencil, checking each step yourself; the small numbers are chosen so every step can be done by hand. Then attempt **"Your turn"** before reading its solution, even if you get it wrong: the attempt is what makes it stick. Only then read the fundamentals and the model answer.

The green "Say it in 60 seconds" boxes are written as speech, in the first person, because interviews are spoken. Do not memorise them. Read one, close the book, and explain the idea to an imaginary friend who is not in tech. The grey "Common mistakes" line lists what interviewers see candidates get wrong most often. The "Check yourself" answers are printed directly under each question, so cover them with a sheet of paper as you go.

If you are short of time

Chapters 2 (SQL), 5 (statistics), 7 (machine learning) and 14 (projects) carry the highest priority, and Chapters 1, 3, 6, 8, 13 and 15 come next. With four weeks, read those ten, use the cheat sheets for the rest, and spend most of your time practising aloud. Appendix A has twelve-week, eight-week and four-week plans.

Contents

About this book	3
How to use this book with the app	4
0 Orientation	8
The typical interview pipeline · What different company types emphasise · How to study, and the three levels of knowledge	
1 Python Programming	18
Core Python language checklist · Complexity cheat sheet for Python structures · Python questions I: the data model · Python questions II: runtime and style · Coding patterns: strings · Coding patterns: arrays and hash maps · Coding patterns: the classics · Implement from scratch I: statistics and metrics · Implement from scratch II: regression, k-means, KNN · Practice targets and self-check · Cheat sheet (p. 59)	
2 SQL	61
SQL topic checklist · Logical order of execution · ROW_NUMBER vs RANK vs DENSE_RANK · Practice schema used in every problem · SQL problems, level 1: basics · SQL problems, level 2: ranking and top-N · SQL problems, level 3: running totals, growth, LAG/LEAD · SQL problems, level 4a: streaks, retention, cohorts · SQL problems, level 4b: funnels, churn windows, sessions · SQL problems, level 5a: median, pivot, share of total · SQL problems, level 5b: churn, NTILE, RFM, recursion · Conceptual SQL questions · Common SQL traps interviewers watch for · Practice platforms and self-check · Cheat sheet (p. 101)	
3 NumPy, pandas, Visualization	103
NumPy essentials and broadcasting · pandas checklist · SQL to pandas translation table · Worked pandas problems · Conceptual pandas questions · Visualization: choosing the right chart · The EDA template and self-check · Cheat sheet (p. 138)	
4 Data Structures and Algorithms (light)	140
DSA topics to cover · Pattern templates: sliding window to BFS · Targets and self-check · Cheat sheet (p. 153)	
5 Statistics and Probability	155
Descriptive statistics · Probability fundamentals · Bayes' theorem: the worked example · Classic probability puzzles · Distributions you must know · Central Limit Theorem and the Law of Large Numbers · Hypothesis testing: the core definitions · Which test do I use? · Multiple testing and Python recipes · Other essentials: sampling, MLE, Bayesian, linear algebra, calculus · Stats Q&A and self-check · Cheat sheet (p. 184)	
6 A/B Testing and Experimentation	186
The experiment lifecycle · Sample size: the back-of-envelope method · A/B testing pitfalls · Advanced experimentation ideas · Practice case questions and self-check · Cheat sheet (p. 202)	
7 Machine Learning Fundamentals and Algorithms	204
Core ML concepts · Why L1 gives sparsity · Data leakage: classic examples · Linear regression · Logistic regression · Decision trees · Random forest (bagging) · Gradient boosting: GBM, XGBoost, LightGBM, CatBoost · Support vector machines · K-nearest neighbours · Naive Bayes · Ensembling and the algorithm selection guide · Optimisation: gradient descent and friends · Hyperparameter tuning and sklearn pipelines · ML Q&A and self-check · Cheat sheet (p. 240)	
8 Feature Engineering, Metrics, Imbalanced Data	242
Data preprocessing: missing values, outliers, encoding, scaling · Feature engineering and feature selection · Dimensionality reduction: PCA, t-SNE, UMAP, LDA · Classification metrics · Choosing the metric from business cost · Regression, ranking and clustering metrics · Imbalanced data playbook and self-check · Cheat sheet (p. 262)	
9 Time Series, Clustering, Recommenders, Anomaly Detection	264
Time series forecasting · Clustering · Recommender systems · Anomaly detection · Cheat sheet (p. 276)	

10	Deep Learning, NLP, GenAI/LLMs	278
	Deep learning fundamentals · NLP: classical pipeline, TF-IDF and embeddings · Transformers and attention · LLM fundamentals and prompting · RAG end to end · RAG vs fine-tuning, agents and project ideas · Computer vision and self-check · Cheat sheet (p. 299)	
11	Data Engineering and Tooling	301
	Data engineering concepts · PySpark · Databases and cloud platforms · Developer tools: Git, Linux, environments, Docker · BI tools, Excel and self-check · Cheat sheet (p. 316)	
12	MLOps and Deployment Basics	318
	The ML lifecycle · MLOps key concepts: serving, drift, monitoring · MLOps Q&A · Cheat sheet (p. 327)	
13	Business Sense, Product Metrics, Case Studies	329
	Metric vocabulary · Framework: diagnosing a metric drop · Framework: measuring the success of a feature · ML problem framing: from business ask to DS problem · Guesstimates · Communication rules for data scientists	
14	Projects: What to Build and How to Present	347
	What a strong project demonstrates · Recommended portfolio and project ideas · Project template and workflow checklist · Turning a project into resume bullets · The project deep-dive questions · Red flags to avoid	
15	Resume, Behavioral and HR Round	365
	Resume checklist · "Tell me about yourself" · Behavioural questions and STAR stories · HR and logistics questions, including salary · Interview-day tips	
16	Master Question Bank with Answers	379
	Question bank: statistics and probability · Question bank: machine learning I · Question bank: machine learning II · Question bank: SQL and Python rapid-fire · Question bank: case and product · Self-test: answer aloud in under two minutes	
17	12-Week Study Plan, Checklists, Resources	395
	The 12-week plan · Master readiness checklist · Company-wise emphasis cheat sheet · Resources and final advice	
	Appendices: study plans, formulas, decision tables, checklists	408
	Glossary of terms	415

0

CHAPTER 0 · MODULE 0 IN THE APP

Orientation

What a data science interview actually is, who is hiring, and how to study so that the effort turns into offers.

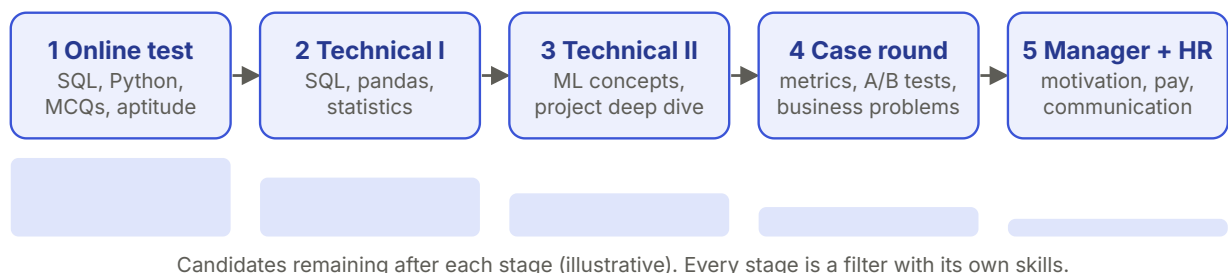
3 topics · about 28 min in the app · 2 questions answered

- 0.1** The typical interview pipeline

0.2 What different company types emphasise
- 0.3** How to study, and the three levels of knowledge

Foundations

A data science interview is not one exam. It is a sequence of filters, each run by a different person who is checking a different thing. Candidates who treat it as a single test of "how much I know" prepare badly: they read widely, feel busy, and then fail the first stage because they never practised writing SQL against a timer. The first idea in this book is therefore structural. Know the stages, know what each one measures, and prepare for the stage in front of you.



Candidates remaining after each stage (illustrative). Every stage is a filter with its own skills.

Figure 1. A typical process has five stages. The early ones test fluency, the later ones test judgement.

What each stage is really asking

The **online test** asks one question: can this person write correct SQL and Python without help? It is graded by a machine, so partial understanding earns nothing. The **first technical round** asks whether you can do the daily work of an analyst or junior data scientist: query data, reshape it, and reason about averages, samples and significance. The **second technical round** asks whether you understand machine learning beyond calling a library, and whether the projects on your resume are really yours. The **case round** asks how you think when the problem is vague, which is what the job is like. The **final round** asks whether people want to work with you, and what it will cost to hire you.

Because the stages differ, your preparation should have a different texture for each. Fluency comes from timed repetition. Understanding comes from explaining aloud. Judgement comes from practising structured answers to open questions. Reading alone builds none of the three.

Companies differ, the core does not

An analytics consulting firm, a bank, a product startup and a global capability centre all call the role "data scientist", yet they weight the skills differently. Consulting firms care about SQL, statistics and clear communication with clients. Product companies ask hard SQL and want you to design experiments and reason about product metrics. Banks want you to explain logistic regression and risk models carefully. Large technology centres add a coding round on data structures. What they share is a core of four subjects: **SQL, Python, statistics and machine learning fundamentals**. Build that core first. Add the extras for the companies you are actually interviewing with, in the order of your interview dates.

The three levels of knowledge

Interviewers probe every topic at three depths, usually in this order.

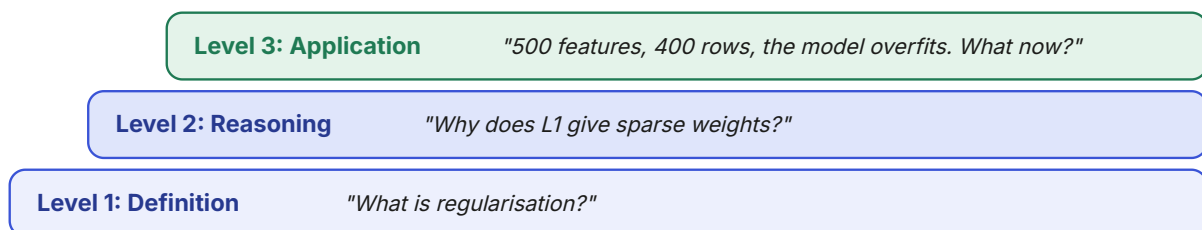


Figure 2. Most rejected candidates can answer level 1. Offers are decided at level 3.

A definition shows you have met the idea. Reasoning shows you understand the mechanism. Application shows you can use it when the situation is messy and nobody tells you which tool to pick. A useful test for any topic in this book: can you define it in one sentence, say why it works, and then say what you would do in a concrete situation? If the third part is weak, the topic is not finished.

How learning actually sticks

Four habits separate people who improve quickly from people who stay busy. **Explain aloud:** if you cannot say it in sixty seconds to a friend outside tech, you do not own it yet. **Do, do not watch:** type every query and every snippet yourself, because recognition feels like knowledge and is not. **Keep a mistakes log:** write down each question you got wrong, what you said and what was right, and revisit it on a schedule. **Rehearse the real thing:** do full mock interviews, timed and spoken, before the ones that count.

The rest of this book is organised to make those habits easy. Each topic gives you a plain explanation to check your understanding, a model spoken answer to compare against, and questions to answer with the page covered.

Topic by topic

Each topic gives the idea in plain words, the fundamentals to know, a model spoken answer and three questions to test yourself. Open the same topic in the app for worked examples, runnable code and interactive demos.

0.1 The typical interview pipeline

IN PLAIN WORDS

Getting a data science job is like a video game with five levels, and each level checks a different skill. Level 1 is a timed online test (mostly SQL and Python). Levels 2 and 3 are conversations with engineers about code, statistics, machine learning and the projects you built. Level 4 is a "business puzzle" where you show you can think about a company's problem. Level 5 is a chat with a manager or HR about how you work with people and what you want.

If you lose at level 1 you never meet the others, so the early, boring skills (SQL, Python) matter most.

Why it matters: Knowing what each round tests tells you where to spend your hours.

Takeaway: Practise the round you will face next, not the one you enjoy most.

WORKED EXAMPLE, STEP BY STEP

Plan backwards from a real date

Riya has an online test in 9 days at a food-delivery startup, and expects a technical round about two weeks after that. She can study 2 hours a day.

- 1. Name the next filter.** The online test comes first. It is machine-graded SQL and Python, plus multiple-choice questions.
- 2. Count the hours.** 9 days $\times$ 2 hours = 18 hours before the test.
- 3. Split by what the filter measures.** SQL timed sets: 9 hours. Python timed sets: 6 hours. Definitions in statistics and ML for the multiple-choice part: 3 hours. Projects and deep ML theory: 0 hours this week.
- 4. Make each day concrete.** Day 1 is "two SQL problems against a 20-minute timer each, then two easy Python problems". No day says "study SQL".
- 5. Switch after the test.** The two weeks before the technical round go to statistics explanations, ML reasoning and rehearsing one project aloud.
- 6. Keep one habit through both phases:** 15 minutes a day on the mistakes log.

What this shows. You do not study "data science". You prepare for the next filter, with hours counted and tasks named.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Your hiring-manager round is in 3 days and you have 6 hours in total. The earlier rounds are passed. Split the 6 hours.

Solution. This round tests motivation, communication and pay, so technical revision gets very little. A sensible split: 2 hours rehearsing the 90-second introduction and four STAR stories aloud; 1.5 hours on a two-minute walk-through of your best project with its business impact; 1 hour researching the company and one data problem it has, and writing three questions to ask; 1 hour on salary research and your notice-period answer; 30 minutes checking logistics. The test is whether every hour produces something you can say out loud.

THE FUNDAMENTALS

Round	Format	What is tested
1. Online assessment	60-120 min, HackerRank/HackerEarth/Codility/Mettl	SQL, Python, MCQs on stats/ML, aptitude
2. Technical R1	45-60 min, live	SQL + pandas + stats fundamentals
3. Technical R2	45-60 min	ML concepts, project deep-dive
4. Case / Product round	45-60 min	Metrics, A/B testing, business problem solving
5. Hiring manager / HR	30-45 min	Communication, motivation, compensation

SAY IT IN 60 SECONDS

"A data science hiring process usually has five stages. First an online test, mostly SQL and Python, graded by hidden test cases, so it is a filter. Then one or two live technical rounds: SQL, pandas, statistics, then a deep dive into your projects. Then a case round, where you get a vague business problem like 'orders dropped ten per cent' and show structured thinking. Finally a hiring manager or HR chat about motivation, salary and notice period. The catch is that each stage tests something different, and failing the first one means you never see the rest. So I prepare for the stage I am about to face, and SQL and Python get daily practice because they decide the early filter."

KEY TERMS online assessment · technical round · case round · hiring manager · HR round · shortlisting

HANDS-ON PRACTICE Write down the five rounds for a company you are targeting and one concrete preparation task for each.

COMMON MISTAKES Preparing only ML theory and then failing the SQL online assessment · Treating the HR round as a formality

CHECK YOURSELF**1. Your first online test is in five days. Which topics get your time, and which do not?**

Timed SQL and Python sets every day, plus a quick pass over statistics and ML definitions for the MCQs. Deep ML theory and projects wait: the online test is a filter and failing it ends the process.

2. Why is the case round hard to prepare for by reading definitions?

It tests structured reasoning on a vague prompt, not recall. You prepare with reusable frameworks (verify, decompose, segment; metrics and experiment design) and by practising them aloud on fresh prompts.

3. A friend says "the HR round is a formality". What risk are they ignoring?

Salary, notice period, motivation and communication are all judged there, and offers are lost or under-negotiated in that round. It deserves rehearsed answers like any technical round.

0.2 What different company types emphasise**IN PLAIN WORDS**

Different kinds of companies are like different kinds of restaurants. A fast-food place (IT services) wants you to cook the standard menu reliably: Python, SQL and basic machine learning. A fancy restaurant (product company) asks you to invent a new dish: hard SQL, experiments and product sense. A catering firm (analytics consulting) cares how well you explain the food to customers. A bank (BFSI) wants you to be very careful and to be able to justify every ingredient.

Same job title, very different tests.

Why it matters: The same hours of practice pay off differently depending on where you apply.

Takeaway: Look at who you are applying to, then decide what to study first.

WORKED EXAMPLE, STEP BY STEP**Decode a job description in two minutes**

A posting reads: *"Data Scientist. Partner with product managers to define success metrics, design and analyse A/B tests, and build models for delivery-time prediction. Strong SQL required. Python, statistics. Experience with experimentation a plus."*

- 1. Underline the verbs.** "Define metrics", "design and analyse A/B tests", "build models".
- 2. Spot the type.** Product managers, metrics and experiments point to a product company, not a consulting firm or a bank.
- 3. Rank the skills by how often they appear.** SQL is "required" and listed first. Experimentation is named twice. Modelling is one line.
- 4. Translate into preparation.** First: hard SQL (cohorts, funnels, window functions). Second: A/B testing and metric design. Third: ML fundamentals with one forecasting or regression project.
- 5. Find the company's data problem.** "Delivery-time prediction" is stated. Prepare two sentences on how you would frame and evaluate it.

6. Leave out what is absent. No mention of data structures, deep learning or Spark, so they get revision time only.

What this shows. The posting tells you the weights. Read it the way you would read an exam syllabus.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Decode this line from a bank's posting and name the top three things to prepare: *"Build and validate credit risk scorecards; strong logistic regression, SQL/SAS; explain model decisions to risk and compliance teams."*

Solution. It is a BFSI risk role, where being able to explain the model matters more than raw accuracy. Prepare: (1) logistic regression in depth, including coefficients as odds ratios, and scorecard vocabulary such as weight of evidence, KS and Gini; (2) SQL for aggregation and joins on account-level data; (3) plain-language explanation of a model decision to a non-technical reviewer, with reason codes. Deep learning and product experimentation are low priority here.

THE FUNDAMENTALS

Company type	Examples	Emphasis
Analytics / consulting	Fractal, Tiger Analytics, Mu Sigma, LatentView, Tredence, ZS, Deloitte, EY, Accenture	SQL, stats, case studies, communication, business framing
IT services	TCS, Infosys, Wipro, Cognizant, HCL, Capgemini	Python fundamentals, SQL, basic ML, project discussion
Product / startups	Flipkart, Swiggy, Zomato, Meesho, PhonePe, Razorpay, CRED, Dream11, Ola, Paytm	Hard SQL, product sense, experimentation, ML depth
GCCs / global captives	Amazon, Walmart, Microsoft IDC, Google, Goldman Sachs, Amex, Target, Adobe	DSA + SQL + ML + stats, strict bar
Fintech / BFSI	HDFC, ICICI, Bajaj Finserv, Jio, Paytm	Credit risk modeling, logistic regression, scorecards, SAS/SQL

SAY IT IN 60 SECONDS

"Different companies hire for different things, so I tune my preparation. Analytics and consulting firms care most about SQL, statistics, case studies and how well I communicate. IT services want solid Python, SQL and basic machine learning plus projects. Product companies and startups set the highest bar on SQL, A/B testing and product sense. Global capability centres add data structures and algorithms. Banks and fintechs want logistic regression, scorecards and risk vocabulary. The catch is that the shared core is the same everywhere: SQL, Python, statistics and core ML. So I start there, then add the extras in the order of my interview dates."

KEY TERMS analytics consulting · IT services · product companies · GCC · BFSI · AI-first startups

HANDS-ON PRACTICE Pick three target companies, classify each by type, and list the top three skills each will weight most.

COMMON MISTAKES Using the same generic preparation for every company type · Ignoring the domain (for example credit risk concepts for BFSI)

CHECK YOURSELF**1. You are applying to a bank and to a food-delivery startup. What is shared and what differs in your preparation?**

Shared: SQL, Python, statistics, ML core. Bank extras: scorecards, WoE/IV, KS and Gini, risk concepts. Startup extras: hard SQL, A/B testing, metrics and product cases.

2. Why does a GCC interview often include a coding round that an analytics firm skips?

GCCs run a structured, uniform bar that includes data structures and algorithms alongside SQL and ML; analytics firms weight cases and communication more.

3. How do you answer "why us?" so it does not sound generic?

Name a specific data problem the company has (ETA prediction, fraud, demand forecasting) and say how your skills apply to it.

0.3 How to study, and the three levels of knowledge**IN PLAIN WORDS**

Learning for an interview is like learning to ride a bike: watching videos helps a little, but you only get good by actually riding and falling. So do these four things: say every idea out loud as if teaching a friend; type every example yourself; write down each thing you got wrong so you do not repeat it; and practise whole mock interviews.

Interviewers also ask in three steps. First "what is it?", then "why does it work?", then "what would you do in this messy situation?". Be ready for step three.

Why it matters: Most rejected candidates can define things but freeze on "why" and "what would you do".

Takeaway: Explain aloud, code it yourself, log mistakes, and prepare every topic to level three.

WORKED EXAMPLE, STEP BY STEP**Set up your practice lab in 20 minutes**

You need a place to run Python and SQL, and a place to record mistakes. Nothing has to be installed.

1. Python, no installation. Open Google Colab (colab.research.google.com) and create a notebook. Type `print(2 + 2)` and press Shift+Enter. pandas, NumPy and scikit-learn are already there.

2. SQL, no installation. In the same notebook, SQLite comes with Python:

```
import sqlite3, pandas as pd
con = sqlite3.connect("practice.db")
con.executescript(open("sample.sql").read()) # the setup script from topic 2.4
pd.read_sql("SELECT * FROM employees LIMIT 3", con)
```

Copy the setup script from the "Practice schema" topic into a file called `sample.sql` first. A browser playground such as DB Fiddle or SQLite Online also works: paste the script, then run queries. **3. Check it works.** `pd.read_sql("SELECT COUNT(*) FROM orders", con)` should return 16. **4. Create the mistakes log.** A spreadsheet with seven columns: date, topic,

question, what I said, what was right, fix, revisit on. 5. **Make the first entry today.** Attempt any question in the app, get it wrong, and log it. An empty log means you are reading, not practising. 6. **Fix a daily slot.** The same 45 minutes every day for timed problems. Put it in your calendar now.

What this shows. The barrier to practising is a setup you can finish in one sitting. After this, every example in the course can be typed and run.

YOUR TURN (COVER THE SOLUTION FIRST)

1. You were asked "What is the difference between WHERE and HAVING?" and answered "They both filter, HAVING is for GROUP BY". Write the mistakes-log line, including a fix and a revisit date.

Solution. A good entry: *topic* SQL basics; *question* WHERE vs HAVING; *what I said* "both filter, HAVING is for GROUP BY"; *what was right* WHERE filters rows before grouping, HAVING filters groups after aggregation, so an aggregate such as COUNT(*) can only appear in HAVING; *fix* write one query that uses both, and explain the execution order aloud; *revisit on* two days from today, then one week later. The answer you gave was not wrong, it was incomplete, and the log should capture exactly what was missing.

THE FUNDAMENTALS

0.3 How to use this course

1. Do modules in order the first time. SQL, Python, Stats, and ML are non-negotiable.
2. For every concept, **explain it aloud** in 60 seconds as if to a non-technical person.
3. Code every example yourself. Do not just read.
4. Keep a "mistakes log" of questions you got wrong.
5. Do at least 3 mock interviews before real ones (friends, Pramp, Interviewing.io, or record yourself).

0.4 The three levels of knowledge interviewers probe

1. **Definition:** "What is regularization?"
2. **Reasoning:** "Why does L1 give sparse solutions?"
3. **Application:** "Your model overfits on a small dataset with 500 features; what do you do?"

Prepare every topic to level 3.

SAY IT IN 60 SECONDS

"To prepare for interviews I do four things. I explain every concept aloud in sixty seconds as if to a friend, because that exposes gaps. I type every example myself instead of reading it. I keep a mistakes log with the question, what I said and what was right, and I close each gap within two days. And I do mock interviews. Interviewers also probe in three levels: what is it, why does it work, and what would you do in this messy situation. Most people stop at the definition. The catch is that the third level, application, is where offers are decided, so I prepare every topic to that depth."

KEY TERMS active recall · explain aloud · mistakes log · mock interviews · definition reasoning application

HANDS-ON PRACTICE Pick one concept you think you know and explain it aloud in 60 seconds to a non-technical listener; note where you stumble.

COMMON MISTAKES Reading without coding or explaining aloud · Stopping preparation at definitions · Not keeping a record of mistakes

CHECK YOURSELF

1. Give the three levels of knowledge for "cross-validation".

Definition: rotate which fold validates and average. Reasoning: it uses all data for both fitting and validation, giving a steadier estimate. Application: use stratified, group or time-series splits depending on the data, never plain k-fold on temporal data.

2. Why does a mistakes log beat re-reading notes?

It targets exactly what you forget, and revisiting it on a schedule is spaced repetition. Rereading mostly rehearses what you already know.

3. How can you tell if you really understand something?

You can explain it simply in sixty seconds with an example, answer "why" and "what if", and apply it to a new scenario without notes.

Interview tips, tricks and hacks

- 1. Find out the process before you prepare.** Ask the recruiter how many rounds there are, what each covers and which tools are allowed. It is a normal question, and it lets you spend your hours on the next filter instead of guessing.
- 2. Protect the daily forty-five minutes.** Timed SQL and Python practice every day beats a long weekend session. Fluency decays quickly and the first two stages test nothing else.
- 3. Prepare to level 3 for the core, level 1 for the fringe.** SQL, statistics and ML fundamentals deserve application-level depth. For PySpark or Docker, a clear definition and one honest example of use is enough for most entry roles.
- 4. Run a mistakes log from day one.** One line per mistake: the question, your answer, the right answer, the reason. Review it for fifteen minutes a day. It becomes your personal question bank for the final week.
- 5. Say "it depends" only with the next sentence ready.** "It depends on whether missing a positive is costlier than a false alarm; if so I would..." shows judgement. A bare "it depends" sounds like avoidance.
- 6. Research the company type, then name one real problem of theirs.** For a food-delivery firm: delivery-time prediction, restaurant ranking, demand forecasting. It turns "why us?" from a platitude into evidence.
- 7. Do at least three mock interviews.** Record yourself if you have no partner. Most people discover that they ramble, skip the example, or never state their assumptions. These are easy to fix once you hear them.
- 8. Treat every real interview as data.** Straight after each one, write down every question you were asked. Patterns appear within three interviews.

Frequently asked questions

2 questions interviewers ask on this chapter, with model answers. Cover the answer, say yours aloud, then compare.

THE TYPICAL INTERVIEW PIPELINE

Q1 You have an OA in five days and a technical round in two weeks. How do you split your time?

Spend the first five days almost entirely on timed SQL and Python sets (for example two SQL problems and two coding problems every day, under a timer), plus a quick pass over statistics and ML definitions for the MCQs. After the OA, shift to depth: statistics reasoning, ML algorithms, and rehearsing the project deep-dive aloud. The principle is to optimise for the next filter, because failing it makes everything else irrelevant.

HOW TO STUDY, AND THE THREE LEVELS OF KNOWLEDGE

Q2 What makes a good 60-second explanation of a concept?

Open with the one-sentence idea in plain words, give one concrete everyday example, name one situation where it breaks or is commonly misunderstood, and finish with why it matters for decisions. No jargon without an analogy. If you cannot do it in 60 seconds, you probably understand the definition but not the concept yet.

1

CHAPTER 1 · HIGH PRIORITY · MODULE 1 IN THE APP

Python Programming

Python is the working language of data science. Interviewers want clean, correct code written quickly, and a clear account of how the language behaves.

10 topics · about 3 h 49 min in the app · 62 questions answered · two-page cheat sheet at the end

- | | |
|---|--|
| 1.1 Core Python language checklist | 1.6 Coding patterns: arrays and hash maps |
| 1.2 Complexity cheat sheet for Python structures | 1.7 Coding patterns: the classics |
| 1.3 Python questions I: the data model | 1.8 Implement from scratch I: statistics and metrics |
| 1.4 Python questions II: runtime and style | 1.9 Implement from scratch II: regression, k-means, KNN |
| 1.5 Coding patterns: strings | 1.10 Practice targets and self-check |

Foundations

You will write Python in the online test, in live coding rounds and for the rest of your career, so two different things are being tested. One is **fluency**: can you turn a small problem into working code in a few minutes? The other is **understanding**: can you explain why a piece of code behaves as it does? Both come from one mental model, and the model is simple.

Names point to objects

In Python, a variable is not a box that holds a value. It is a name tied to an object that lives somewhere in memory. Assignment never copies anything; it ties one more name to the same object.

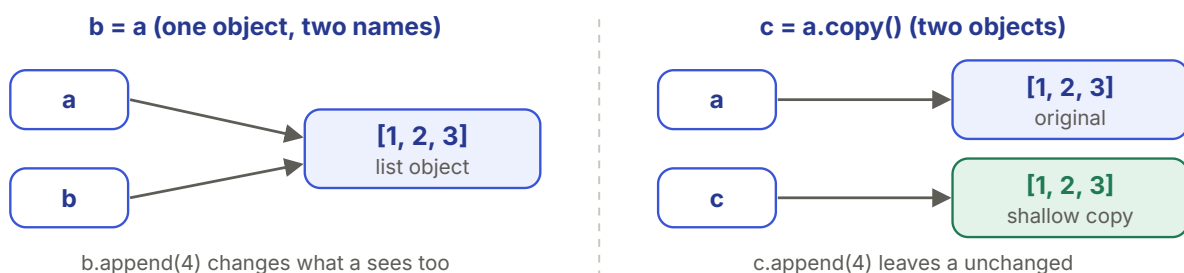


Figure 3. Assignment adds a second name for one object. A copy creates a second object.

Almost every "tricky" Python question is this picture in disguise. Why did changing one list change another? Two names, one object. Why does a default argument such as `def f(x, acc=[])` remember values between calls? The list is created once, when the function is defined, and every call uses that same object. Why does `a == b` differ from `a is b`? The first compares values, the second asks whether two names point at the same object.

Mutable and immutable

Some objects can be changed in place: lists, dictionaries and sets. These are **mutable**. Others cannot: numbers, strings, tuples and frozensets. These are **immutable**, and any "change" really builds a new object. The distinction has a practical consequence. Dictionary keys and set members must be **hashable**, which in practice means immutable, because the dictionary finds a key by computing a number from its contents; if the contents could change, the key would be lost. That is why a tuple can be a key and a list cannot.

The four containers and what each is for

A **list** is an ordered sequence you can grow; use it when order matters. A **tuple** is a fixed record. A **dictionary** maps keys to values and finds a key in constant time on average. A **set** stores unique items and answers "have I seen this?" in constant time. Choosing the right container is the single most common way to make a slow solution fast: replacing "search the list for a match" with "look it up in a dictionary" turns a quadratic algorithm into a linear one. Chapter 4 builds on exactly this.

Complexity is part of the answer

For every solution, an interviewer will ask how its running time grows with the input. You need a small set of facts by heart: indexing a list and appending to it are constant time; inserting at the front or searching a list is linear; dictionary and set lookups are constant on average; sorting is $n \log n$. State the time and the extra memory your solution uses without being asked.

Functions, iteration and laziness

Functions are objects too: they can be passed around, returned, and wrapped. A **decorator** is simply a function that takes a function and returns a new one with extra behaviour. A **generator** produces values one at a time instead of building a whole list, so it uses almost no memory and can represent a stream. A **comprehension** is a compact, readable loop that builds a list, set or dictionary. Idiomatic Python uses these tools along with `enumerate`, `zip`, `sorted` with a key, and the `collections` module, and interviewers notice whether you reach for them.

From scratch, on purpose

Data science interviews often ask you to implement something you would normally import: a mean and standard deviation, a train-test split, k-nearest neighbours, a gradient descent loop. The purpose is not to check memory of formulas. It is to see whether you understand the algorithm well enough to write its dozen lines, handle the edge cases (an empty list, a division by zero) and explain the cost. Practise a handful until you can write them on a blank page.

Topic by topic

Each topic gives the idea in plain words, the fundamentals to know, a model spoken answer and three questions to test yourself. Open the same topic in the app for worked examples, runnable code and interactive demos.

1.1 Core Python language checklist

IN PLAIN WORDS

Python is like a kitchen with labelled jars. A list is a jar you can keep adding to and swapping things in. A tuple is a sealed jar. A dict is a cupboard where you look things up by label. A set is a bag that never holds two of the same thing. A generator is a chef who cooks one dish at a time when you ask, instead of cooking the whole menu first.

Most interview questions are just "do you know which jar to use and what happens when you change what is inside it".

Why it matters: Interviewers use small Python puzzles to check you know the language, not only the libraries.

Takeaway: Know the core types, what can change, and how to loop and slice cleanly.

WORKED EXAMPLE, STEP BY STEP

Read a small program the way the computer does

Before writing code, practise *tracing* it: keep a table of every variable and update it line by line.

```
orders = [("asha", 250), ("ravi", 120), ("asha", 90),
          ("meera", 300), ("ravi", 60)]
totals = {}
for name, amount in orders:
    if amount ≥ 100:
        totals[name] = totals.get(name, 0) + amount
print(totals)
```

1. Before the loop: `totals` is an empty dictionary `{}`.

2. Trace each pass:

Pass	name, amount	amount ≥ 100?	totals after the pass
1	asha, 250	yes	{asha: 250}
2	ravi, 120	yes	{asha: 250, ravi: 120}
3	asha, 90	no, skipped	unchanged
4	meera, 300	yes	{asha: 250, ravi: 120, meera: 300}
5	ravi, 60	no, skipped	unchanged

3. Read the tricky line slowly. `totals.get(name, 0)` means "the current total for this name, or 0 if the name is not there yet". Adding `amount` and storing it back is how a running total per key is built.

4. Output: `{'asha': 250, 'ravi': 120, 'meera': 300}`.

What this shows. A loop, a condition and a dictionary are enough for "total per group", the most common task in data work. If you can fill in this table, you can debug almost anything.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Without running it, write the output of each line:

```
xs = [3, 8, 1, 6]
print(xs[1:3])
print(xs[-1])
print([x * 2 for x in xs if x > 2][::-1])
```

Solution. `[8, 1]` (positions 1 and 2; the stop position 3 is excluded). `6` (the last item). For the third line, work inside out: keep the items greater than 2, giving `3, 8, 6`; double them, giving `[6, 16, 12]`; then `[::-1]` reverses the list: `[12, 16, 6]`.

2. Three short ones, without running them first. (a) Using a loop with `break` and `else`, print the first number greater than 10 in `[3, 8, 12, 5, 20]`, or "none found". (b) Flatten `[[1, 2], [3], [4, 5, 6]]` and keep only the odd numbers, in one comprehension. (c) What do `bool("0")`, `int("3.7")` and `-7 // 2` give?

Solution. (a)

```
for n in [3, 8, 12, 5, 20]:
    if n > 10:
        print("first > 10:", n)      # first > 10: 12
        break
else:
    print("none found")              # only when the loop ends without a break
```

(b) `[x for sub in [[1, 2], [3], [4, 5, 6]] for x in sub if x % 2 == 1]` gives `[1, 3, 5]`. The `for`s read left to right, and the filter goes last.

(c) `bool("0")` is `True` (any non-empty string is truthy). `int("3.7")` raises `ValueError`; use `int(float("3.7"))`, which gives `3`. `-7 // 2` is `-4`, because floor division rounds down, not towards zero.

THE FUNDAMENTALS

- ☐ Data types: `int`, `float`, `str`, `bool`, `None`, `list`, `tuple`, `set`, `dict`, `frozenset`
- ☐ Mutable vs immutable (`list`, `dict`, `set` vs `tuple`, `str`, `int`, `frozenset`)
- ☐ Indexing, slicing (`a[::-1]`, `a[1:5:2]`), negative indices
- ☐ String methods, f-strings, `split/join/strip/replace`, regex basics (`re` module)
- ☐ Comprehensions (`list`, `dict`, `set`), generator expressions
- ☐ Functions: default args, `*args`, `**kwargs`, keyword-only args, `lambda`, recursion
- ☐ Higher-order functions: `map`, `filter`, `reduce`, `sorted(key=...)`, `zip`, `enumerate`, `any`, `all`
- ☐ Iterators and generators (`yield`), `itertools`, `collections`
- ☐ Decorators and closures (basic)
- ☐ OOP: classes, `__init__`, `self`, inheritance, `super()`, `@staticmethod`, `@classmethod`, `@property`, dunder methods (`__str__`, `__repr__`, `__len__`)
- ☐ Exceptions: `try/except/else/finally`, raising, custom exceptions
- ☐ File I/O, `with` statement, JSON and CSV handling
- ☐ Modules, packages, virtual environments (`venv`, `conda`), `pip`, `requirements.txt`

- ☐ Type hints (basic), docstrings, PEP8

Numbers, conversions and truthiness

```
7 / 2, 7 // 2, 7 % 2      # (3.5, 3, 1)  true division, floor division, remainder
# (-4, 1)                floor rounds DOWN, so the remainder takes the divisor's sign
-7 // 2, -7 % 2
2 ** 10, divmod(17, 5)    # (1024, (3, 2))  power; quotient and remainder in one call

0.1 + 0.2 == 0.3          # False: binary floats cannot store 0.1 exactly
import math
math.isclose(0.1 + 0.2, 0.3)  # True: compare floats with a tolerance, never with ==
# (2, 4)  ties go to the nearest EVEN number ("banker's rounding")
round(2.5), round(3.5)

int("42"), float("3.7"), int(3.99)  # (42, 3.7, 3)  int() truncates, it does not round
# ValueError: go through float first → int(float("3.7")) = 3
int("3.7")
```

Every object is either "truthy" or "falsy" when used in an `if`. The falsy ones are `None`, `False`, zero (`0`, `0.0`) and anything empty (`""`, `[]`, `{}`, `set()`). Everything else is truthy:

```
bool(""), bool([]), bool(0.0)  # (False, False, False)
# (True, True)  a non-empty string or list is truthy, whatever it holds
bool("False"), bool([0])

name = None
# 'guest'      `or` returns the first truthy value: a handy default
name or "guest"
0 or "guest"    # 'guest'  careful: a legitimate 0 is also replaced
```

`bool("False")` IS TRUE

Text read from a file or a form is a non-empty string, so it is always truthy. Compare the text itself: `flag = text.strip().lower() == "true"`. The same care applies to `x or default`: when `0` or an empty string is a valid value, test `x is None` instead.

Cleaning text and formatting numbers

Cleaning and checking text, the first step of most data work:

```
# 'Sales_2024_Q1.csv'  strip removes spaces and newlines at both ends
f = " Sales_2024_Q1.csv \n".strip()
f.lower().startswith("sales")          # True
f.endswith((".csv", ".xlsx"))          # True  a tuple means "any of these"
# ('Sales', '_', '2024_Q1.csv')  split once, at the first separator
f.partition("_")
# ['Sales_2024_Q1', 'csv']  split once, from the right
f.rsplit(".", 1)
# (6, -1)  position, or -1 when absent (str.index raises instead)
f.find("2024"), f.find("zzz")
# (['a', 'b', '', 'c'], ['a', 'b'])  no argument: split on any whitespace
"a,b,c".split(","), " a b".split()
# (True, False)  isdigit does not accept a sign
"42".isdigit(), "-42".isdigit()
```

The part after the colon in an f-string is a **format spec**. These are the ones you use in reports:

```
f"{1234567.891:,.2f}"      # '1,234,567.89'   thousands separator, 2 decimals
f"{0.4567:.1%}"           # '45.7%'         multiply by 100 and add %
# 'left / right/ mid /'   align in 8 characters
f"{'left':<8}|{'right':>8}|{'mid':^8}|"
f"{42:05d}", f"{1e6:.2e}"   # ('00042', '1.00e+06') zero-pad; scientific notation
count, val = 3, 2.71828
# 'count=3 val=2.72'       the = shows name and value: quick debugging
f"{count=} {val=:.2f}"
```

Tuples, lists, dicts and sets: the everyday methods

Tuples and unpacking. A tuple is a fixed record. Unpacking assigns its parts to names in one line, and a starred name collects "the rest".

```
point = (3, 4)
x, y = point          # x = 3, y = 4
first, *rest = [1, 2, 3, 4] # first = 1, rest = [2, 3, 4]
head, *mid, last = [1, 2, 3, 4, 5] # head = 1, mid = [2, 3, 4], last = 5
a, b = 1, 2
a, b = b, a           # swap without a temporary variable → a = 2, b = 1
# ((5,), 5)          the comma makes the tuple, not the brackets
(5,), (5)
```

Lists.

```
xs = [3, 1, 2]
# [9, 3, 1, 2]   insert at a position (O(n): everything shifts)
xs.insert(0, 9)
# (2, 9)         remove and return the last / the item at an index → xs = [3, 1]
xs.pop(), xs.pop(0)
# [3]           remove the first matching VALUE (ValueError if absent)
xs.remove(1)
# (1, 2)        first position; number of occurrences
[1, 2, 2, 3].index(2), [1, 2, 2, 3].count(2)

sorted([3, 1, 2], reverse=True) # [3, 2, 1]   returns a NEW list
# result is None: .sort() works in place and returns nothing
result = [3, 1, 2].sort()

rows = [("b", 2), ("a", 2), ("c", 1)]
sorted(rows, key=lambda r: r[1])             # [('c', 1), ('b', 2), ('a', 2)]
```

Python's sort is **stable**: rows with equal keys keep their original order (b stays before a above). That is why you can sort by a secondary key first and by the primary key second.

Dictionaries.

```

d = {"name": "Asha", "city": "Pune"}
d["gpa"]                # KeyError: the key is missing
d.get("gpa"), d.get("gpa", 0.0) # (None, 0.0) safe lookup with an optional default
d.setdefault("gpa", 3.8)    # inserts ONLY if the key is missing → 3.8
d.setdefault("gpa", 1.0)    # key exists, so nothing changes → 3.8
d.update({"city": "Delhi", "age": 29}) # overwrite or add several keys at once
# (29, None) remove and return; the default avoids a KeyError
d.pop("age"), d.pop("zzz", None)
del d["gpa"]             # remove without returning
"name" in d, "Asha" in d # (True, False) `in` tests KEYS, not values

for key, value in d.items(): # the standard way to loop over a dict
    print(key, value)
# {'a': 1, 'b': 3} merge; the right side wins (Python 3.9+)
{"a": 1, "b": 2} | {"b": 3}

```

`d.keys()`, `d.values()` and `d.items()` are live **views**: they reflect later changes to the dict, and they keep insertion order.

Sets.

```

A, B = {1, 2, 3, 4}, {3, 4, 5}
A | B, A & B      # ({1, 2, 3, 4, 5}, {3, 4}) union, intersection
# ({1, 2}, {1, 2, 5}) in A but not B; in exactly one of them
A - B, A ^ B
{3, 4} ≤ A        # True subset test
s = {1, 2}
s.add(3)
s.discard(99)     # no error if the value is absent
s.remove(99)      # KeyError: use discard when "absent" is fine
# [3, 1, 2] remove duplicates but KEEP the order (a set would not)
list(dict.fromkeys([3, 1, 3, 2, 1]))

```

Set arithmetic answers many analytics questions directly: users active in January but not February is `jan - feb`; users in both is `jan & feb`.

Conditions and loops

```

score = 72
if score ≥ 85:
    grade = "A"
elif score ≥ 70:
    grade = "B"
else:
    grade = "C"

# the same as a conditional expression → 'B'
grade = "A" if score ≥ 85 else "B" if score ≥ 70 else "C"
18 ≤ 25 < 65 # True: comparisons chain, as in maths

```

```

for i in range(2, 10, 3):      # 2, 5, 8   range(start, stop, step); stop is excluded
    ...
# unpack nested pairs in the loop header
for i, (name, sc) in enumerate(["a", 1], ("b", 2), start=1):
    print(i, name, sc)        # 1 a 1, then 2 b 2
for name in reversed(["a", "b", "c"]): # c, b, a   without building a reversed copy
    ...

value, years = 1000, 0
# `while` repeats until a condition fails: 1000 growing at 7% a year
while value ≤ 2000:
    value *= 1.07
    years += 1
# 11   (the doubling time; the "rule of 72" predicts 72 / 7 = 10.3)
years

```

`break` leaves the loop, `continue` skips to the next item, and `pass` is a placeholder that does nothing. A loop's `else` block runs only when the loop finished **without** hitting `break`, which makes "search, and report if nothing was found" tidy:

```

for n in [3, 8, 12, 5, 20]:
    if n > 10:
        print("first > 10:", n)    # first > 10: 12
        break
    else:
        print("none found")        # runs only if the loop never breaks

```

The loop that every iterative algorithm uses (gradient descent, k-means, Newton's method) is "repeat until the change is tiny, with a cap on the number of rounds":

```

x = 10.0                                # find the square root of 2 by Newton's method
for i in range(100):                    # the cap prevents an infinite loop
    new = 0.5 * (x + 2 / x)
    if abs(new - x) < 1e-12:             # converged: stop early
        break
    x = new
i, x                                    # (7, 1.4142135623730954)

```

More comprehension patterns

A few more shapes that come up constantly:

```

m = [[1, 2], [3], [4, 5, 6]]
# [1, 2, 3, 4, 5, 6]   flatten: read the `for`s left to right
[x for row in m for x in row]
# [1, 3, 5]           flatten and filter
[x for row in m for x in row if x % 2]
# ['even', 'odd', 'even']   if/else goes BEFORE the for
["even" if x % 2 == 0 else "odd" for x in range(3)]
# [[0, 0, 0], [0, 1, 2]]   a nested comprehension builds a grid
[[r * c for c in range(3)] for r in range(2)]

prices = {"tea": 10, "coffee": 25, "juice": 40}
# {10: 'tea', 25: 'coffee', 40: 'juice'}   invert a dict
{v: k for k, v in prices.items()}
# {'coffee': 25, 'juice': 40}               filter a dict
{k: v for k, v in prices.items() if v > 20}

```

A filter (`if` with no `else`) goes at the end; a choice between two values (`a if cond else b`) goes at the front. If a comprehension needs more than two `for`s or becomes hard to read aloud, write a normal loop.

zip, unzip and key functions

`zip` stops silently at the shortest input, which can hide a data bug. Ask it to check, and use the star to "unzip":

```
# [('sql', 3), ('python', 6)] 'spark' is dropped without warning
list(zip(names, [3, 6]))
# ValueError: zip() argument 2 is shorter than argument 1 (Python 3.10+)
list(zip(names, [3, 6], strict=True))

pairs = [("sql", 3), ("python", 6)]
# ('sql', 'python') and (3, 6): unzip into two tuples
words, lengths = zip(*pairs)
# {'sql': 3, 'python': 6} two lists → a lookup table
dict(zip(words, lengths))

max(names, key=len) # 'python' max, min and sorted all accept key=
# [('a', 3), ('b', 2)] sort a dict by value
sorted({'b': 2, 'a': 3}.items(), key=lambda kv: -kv[1])
```

More of itertools

More of `itertools`. Each of these is lazy: it produces items on demand instead of building a list.

```
from itertools import chain, product, permutations, groupby, islice, count

list(chain([1, 2], [3], [4, 5])) # [1, 2, 3, 4, 5] glue iterables end to end
list(product(["S", "M"], ["red", "blue"]))
# [('S', 'red'), ('S', 'blue'), ('M', 'red'), ('M', 'blue')] every pairing: a hyper-
parameter grid
# 6 ordered pairs: AB AC BA BC CA CB (combinations gave 3: order ignored)
list(permutations("ABC", 2))
# [10, 15, 20, 25] take the first 4 items of an endless counter
list(islice(count(10, 5), 4))
# ['b', 'd', 'f'] slice anything iterable, even a generator
list(islice("abcdefg", 1, 6, 2))
```

`itertools.groupby` ONLY GROUPS NEIGHBOURS

Unlike SQL's `GROUP BY`, it starts a new group every time the key changes, so unsorted data gives the same key several times:

```
data = [("fruit", "apple"), ("veg", "okra"), ("fruit", "kiwi")]
key = lambda r: r[0]
[(k, [v for _, v in g]) for k, g in groupby(data, key)]
# [('fruit', ['apple']), ('veg', ['okra']), ('fruit', ['kiwi'])] 'fruit' appears
twice
[(k, [v for _, v in g]) for k, g in groupby(sorted(data, key=key), key)]
# [('fruit', ['apple', 'kiwi']), ('veg', ['okra'])] sort by the same
key first
```

For unsorted data a `defaultdict(list)` is simpler and needs no sort.

Raising your own errors

Raise your own errors too. Checking inputs at the top of a function (a "guard clause") turns a confusing failure deep inside into a clear message at the door:

```
def mean(xs):
    if not xs:
        # fail early, with a message that names the problem
        raise ValueError("mean of an empty list")
    return sum(xs) / len(xs)

try:
    mean([])
except ValueError as e:
    print("caught:", e)                                # caught: mean of an empty list
```

Use `ValueError` for a bad value, `TypeError` for the wrong type, `KeyError` for a missing key and `FileNotFoundError` for a missing file, so callers can catch exactly the failure they can handle.

Several files in one with statement

One `with` statement can manage several resources, and every one of them is closed when the block ends:

```
with open("notes.txt") as src, open("copy.txt", "w") as dst:
    dst.write(src.read().upper())
src.closed          # True
```

Paths and dates: `pathlib` and `datetime`

`pathlib.Path` treats a file path as an object instead of a string, so the same code works on Windows, macOS and Linux:

```
from pathlib import Path

p = Path("data") / "raw" / "sales_2024.csv"    # the / operator joins path parts
p.name, p.stem, p.suffix                      # ('sales_2024.csv', 'sales_2024', '.csv')
p.parent                                      # data/raw
p.with_suffix(".parquet")                     # data/raw/sales_2024.parquet

out = Path("output")
# create the folder (and parents); no error if it exists
out.mkdir(parents=True, exist_ok=True)
(out / "a.txt").write_text("line one\nline two\n")
(out / "a.txt").exists()                       # True
(out / "a.txt").read_text().splitlines()       # ['line one', 'line two']
# ['a.txt'] every file matching a pattern
sorted(f.name for f in out.glob("*.txt"))
```

Dates arrive as text. `strptime` parses text into a datetime, `strftime` formats a datetime back into text, and subtracting two dates gives a `timedelta`:

```

from datetime import datetime, date, timedelta

# datetime(2026, 3, 15, 14, 30)
d = datetime.strptime("2026-03-15 14:30", "%Y-%m-%d %H:%M")
d.strftime("%d %b %Y, %A")           # '15 Mar 2026, Sunday'
d + timedelta(days=30)               # 2026-04-14 14:30:00
(date(2026, 12, 31) - date(2026, 3, 15)).days # 291
d.weekday()                         # 6 Monday is 0, Sunday is 6
# '2026-03-15T14:30:00' the unambiguous format to store and exchange
d.isoformat()

```

Common format codes: `%Y` four-digit year, `%m` month, `%d` day, `%H:%M:%S` time, `%b` short month name, `%A` weekday name. In pandas the same codes are used by `pd.to_datetime(..., format=...)` and `.dt.strftime`.

SAY IT IN 60 SECONDS

"Python has a handful of core types and the interview question is usually which one to use and what happens when you change it. Lists, dicts and sets are mutable, meaning they change in place. Tuples, strings and numbers are immutable. A name is just a label stuck on an object, so assigning one list to another name does not copy it, both names see the same changes. Comprehensions build lists compactly, generators produce values one at a time to save memory, and slicing with a step, like `a[::-1]`, reverses a sequence. The catch is mutability: it explains the copy bug, the dictionary-key rule and the default-argument trap. If I get that right, most Python questions are easy."

KEY TERMS mutable vs immutable · slicing · comprehensions · generators · args and kwargs · decorators · OOP · exceptions · virtual environments

HANDS-ON PRACTICE Re-implement each snippet from memory in a notebook, changing the data, until you can predict the output without running it.

COMMON MISTAKES Confusing identity with equality · Modifying a list while iterating over it · Using a bare `except`

CHECK YOURSELF

1. `a = [1, 2]; b = a; b.append(3)` . What is `a` , and why?

`[1, 2, 3]` : assignment never copies; both names point to the same list.

2. Why can a tuple be a dictionary key but a list cannot?

Keys must be hashable and stable; mutable lists could change after insertion. A tuple of hashable items cannot change.

3. You need the sum of squares of a million numbers. List comprehension or generator expression, and why?

Generator expression inside `sum(...)` : it produces one value at a time, so memory stays constant instead of building a million-item list.

1.2 Complexity cheat sheet for Python structures

IN PLAIN WORDS

Imagine finding a friend's name in a phone book. If the names are in a messy pile you check them one by one, which takes longer as the pile grows. If the book is organised like a cupboard with labelled drawers (a dict or set), you open the right drawer straight away, no matter how many drawers there are.

Big-O just describes how the work grows when the pile grows. Lists are "check one by one" for searching. Sets and dicts are "open the drawer".

Why it matters: Replacing a list with a set is the most common one-line speed-up interviewers look for.

Takeaway: For lookups, use a set or dict. For a queue, use a deque.

WORKED EXAMPLE, STEP BY STEP

Count the steps: finding duplicates two ways

Task: does a list of customer ids contain any duplicate?

- 1. The obvious way.** Compare every id with every other id, using two nested loops.
- 2. Count the comparisons for $n = 1,000$.** Each pair is compared once: $1,000 \times 999 / 2 = 499,500$ comparisons.
- 3. Now $n = 1,000,000$.** About 500,000,000,000 comparisons. At roughly 10 million simple steps a second in Python, that is about 14 hours.
- 4. The better way.** Walk the list once, keeping a set of ids already seen. For each id ask "is it in the set?" (one step on average), then add it.
- 5. Count again.** $n = 1,000$ needs about 1,000 steps. $n = 1,000,000$ needs about 1,000,000 steps: a tenth of a second.
- 6. Name the growth.** The first method grows with n^2 , so it is $O(n^2)$. The second grows with n , so it is $O(n)$, at the price of $O(n)$ extra memory for the set.

n	Nested loops	One pass with a set
1,000	499,500	1,000
1,000,000	5×10^{11} (about 14 hours)	1,000,000 (about 0.1 s)

What this shows. Big-O is a way of counting steps before you run anything. The idea, "trade memory for time with a set or dictionary", is the most reused trick in coding interviews.

YOUR TURN (COVER THE SOLUTION FIRST)

1. What is the time complexity of this code if `users` has n items and `blocked` is a **list** with m items? How do you make it faster?

```
clean = []
for u in users:
    if u not in blocked:
        clean.append(u)
```

Solution. `u not in blocked` scans the list, which is up to m steps, and it runs n times: $O(n \times m)$. With $n = 200,000$ and $m = 50,000$ that is about 10 billion steps. Convert once with `blocked = set(blocked)` (m steps); each check is then one step on average, so the total is $O(n + m)$. The append is $O(1)$ each time and does not change the answer.

THE FUNDAMENTALS

Operation	list	dict / set	deque
Access by index / key	$O(1)$	$O(1)$ avg	$O(n)$ middle
Search (<code>in</code>)	$O(n)$	$O(1)$ avg	$O(n)$
Append	$O(1)$ amortized	$O(1)$	$O(1)$
Insert/pop at front	$O(n)$	-	$O(1)$
Sort	$O(n \log n)$	-	-

SAY IT IN 60 SECONDS

"Big-O describes how the work grows as the data grows. Looking something up in a list means checking items one by one, so it takes longer as the list grows, linear time. Looking something up in a set or dictionary uses a hash table, so it jumps straight to the spot, constant time on average. Appending to a list is also constant on average, though occasionally Python has to copy it into a bigger block. Sorting is $n \log n$. The catch that interviewers love: doing a list lookup inside a loop quietly makes the whole thing quadratic. Turning the list into a set first fixes it in one line."

KEY TERMS big-O · amortized · hash table · deque · sorting · list vs set membership

BUILDS ON 1.1 Core Python language checklist

HANDS-ON PRACTICE Time "x in list" vs "x in set" for 10^5 items with `timeit`, and explain the gap.

COMMON MISTAKES Using a list for membership tests inside a loop (accidental $O(n^2)$) · Building strings with `+=` in a loop · Using `pop(0)` on a list as a queue

CHECK YOURSELF

1. **You check 200,000 ids against a blacklist stored as a 50,000-item list. Complexity and fix?**

About 10^{10} operations (n times m). Convert the blacklist to a set once; each check becomes $O(1)$, total $O(n + m)$.

2. **Why is `list.pop(0)` a poor way to build a queue?**

It shifts every remaining element left: $O(n)$ per pop. `collections.deque.popleft()` is $O(1)$.

3. **What does "amortised $O(1)$ " mean for `append`?**

Occasional expensive resizes are spread across many cheap appends, so the average cost per append is constant.

1.3 Python questions I: the data model

IN PLAIN WORDS

Everything in Python is an object, and a variable name is just a sticky note on an object. Two sticky notes can point at the same object (that is why changing a list through one name changes it for the other). A copy makes a new object; a "deep" copy also copies everything inside.

A function's default value is made once, when the function is defined, like a cookie-cutter cut once that every call reuses, so a default list keeps growing between calls. A generator hands you one value at a time instead of a full basket. A decorator wraps a function in a little extra behaviour, like putting a timer around it.

Why it matters: These are the questions that separate people who memorised syntax from people who understand Python.

Takeaway: Names point to objects; defaults are created once; generators are lazy; decorators wrap functions.

WORKED EXAMPLE, STEP BY STEP

Draw the sticky notes

Trace which names point at which objects. Draw a box for each object and a label for each name.

```
a = [1, 2]
b = a
c = a[:]
b.append(3)
c.append(4)
a = a + [5]
```

1. `a = [1, 2]` creates list object L1 and sticks the label `a` on it.
2. `b = a` sticks a second label, `b`, on the same L1. Nothing is copied.
3. `c = a[:]` a slice builds a *new* list L2 with the same items. `c` points at L2.
4. `b.append(3)` changes L1 in place. L1 is now `[1, 2, 3]`, and `a` sees it too.
5. `c.append(4)` changes L2 only. L2 is `[1, 2, 4]`.
6. `a = a + [5]` `+` builds a *new* list L3 = `[1, 2, 3, 5]` and moves the label `a` to it. `b` still points at L1.

Name	Points at	Value
a	L3	[1, 2, 3, 5]
b	L1	[1, 2, 3]
c	L2	[1, 2, 4]

What this shows. Two operations look alike but differ: `append` changes an object in place, so every name on it sees the change, while `+` and slicing create new objects. Most "why did my list change?" bugs are step 2.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Predict the two printed lines.

```
grid = [[0, 0], [0, 0]]
copy1 = list(grid)
copy1[0][0] = 9
copy1.append([7, 7])
print(grid)
print(copy1)
```

Solution. `list(grid)` is a shallow copy: a new outer list whose items are the *same* inner lists. Changing `copy1[0][0]` changes the shared inner list, so `grid` sees it. Appending to `copy1` changes only the new outer list. Output: `[[9, 0], [0, 0]]` and then `[[9, 0], [0, 0], [7, 7]]`. Use `copy.deepcopy(grid)` when you need full independence.

2. Write a generator `evens(limit)` that yields the even numbers below `limit`, then add up the first five of them using `islice`. Why is `sum(evens(10**12))` a bad idea even though the generator uses almost no memory?

Solution.

```
from itertools import islice

def evens(limit):
    for n in range(0, limit, 2):
        yield n

sum(islice(evens(100), 5))      # 0 + 2 + 4 + 6 + 8 = 20
```

`islice` stops after five items, so the generator does only five steps of work, whatever the limit. `sum(evens(10**12))` would use little memory but still run 500 billion loop iterations: laziness saves **memory**, not **time**. For this sum the closed form is instant: the even numbers below L add up to $k(k-1)$ with $k = L/2$.

THE FUNDAMENTALS

Iterators, endless generators and batches

```
g = (x * 2 for x in range(3))
# ([0, 2, 4], []) the second pass finds it empty
list(g), list(g)
# 0 give min/max a default when the stream may be empty
max((x for x in []), default=0)
```

What a `for` loop really does: `iter()` and `next()`. `iter(obj)` asks an object for an iterator, and `next(it)` asks the iterator for one more item. When nothing is left, `next` raises `StopIteration`, and that is the signal a `for` loop uses to stop.

```

it = iter([10, 20])
next(it), next(it)           # (10, 20)
next(it)                     # StopIteration
next(it, "done")             # 'done' a default instead of the exception
# peek at the first row of anything iterable (None if it is empty)
next(iter(rows), None)

```

Generators can be endless, because nothing is computed until asked. Take what you need with `islice`:

```

from itertools import islice

def fibonacci():
    a, b = 0, 1
    # an infinite stream: safe only because it is lazy
    while True:
        yield a
        a, b = b, a + b

list(islice(fibonacci(), 10))           # [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

```

Batches with bounded memory. The pattern behind chunked file reading, mini-batch training and bulk database inserts: collect items until the batch is full, hand it over, start again, and do not forget the last partial batch.

```

def batches(items, size):
    batch = []
    for x in items:
        batch.append(x)
        if len(batch) == size:
            yield batch
            batch = []
    if batch:
        # the leftover, smaller batch
        yield batch

[ len(b) for b in batches(range(25), 10) ] # [10, 10, 5]

```

Python 3.12 added `itertools.batched(iterable, n)`, which does the same and yields tuples. For a binary file, the two-argument form `iter(callable, sentinel)` calls a function repeatedly until it returns the sentinel:

```

with open("data.bin", "rb") as f:           # a 10-byte file
    # stop at the empty read
    sizes = [len(chunk) for chunk in iter(lambda: f.read(4), b"")]
    sizes                                     # [4, 4, 2]

```

SAY IT IN 60 SECONDS

"Python names are labels on objects. If two names point at the same list, changing it through one changes it for the other. A shallow copy duplicates only the outer container, so nested lists are still shared; a deep copy duplicates everything. Equality with double equals compares values, while 'is' checks it is the very same object. Default arguments are evaluated once when the function is defined, so a default empty list is shared across calls and keeps growing; the fix is None and creating the list inside. A generator uses yield to hand back one value at a time, and a decorator is a function that wraps another to add behaviour. The catch: mutability and shared references explain almost all of these surprises."

KEY TERMS mutability · shallow vs deep copy · is vs == · mutable default argument · generator · decorator · args kwargs

BUILDS ON 1.1 Core Python language checklist

HANDS-ON PRACTICE Write a decorator `@retry(times=3)` that retries a function on exception, and test it.

COMMON MISTAKES Saying that "Python passes by value" or "by reference" instead of explaining object references · Using `is` to compare strings or numbers · Forgetting `functools.wraps` in a decorator

CHECK YOURSELF

1. What does `def f(x, L=[]): L.append(x); return L` return for `f(1)` then `f(2)`, and what is the fix?

`[1]` then `[1, 2]`: the default list is created once. Use `L=None` and set `L = []` inside when None.

2. `a = [[1, 2], [3]]`; `b = list(a)`; `b[0].append(9)`. Does `a` change?

Yes: `list(a)` is a shallow copy; the inner lists are shared. Use `copy.deepcopy` for independence.

3. Why use `@functools.wraps` in a decorator?

It preserves the wrapped function's name and docstring; otherwise everything reports itself as `wrapper`, which breaks debugging and introspection.

1.4 Python questions II: runtime and style

IN PLAIN WORDS

Picture a kitchen with one stove that only one cook can use at a time. That single stove is the GIL. Extra cooks (threads) help when cooks spend their time waiting for deliveries (reading files, calling a website), but they do not cook faster, because they all queue for the one stove. To cook more at once you need more kitchens (separate processes).

Python also tidies up after you: it counts how many names point to each object and throws the object away when the count hits zero, with a cleaner for tricky loops.

Why it matters: "Why doesn't threading speed this up?" is a classic question.

Takeaway: Threads for waiting (I/O), processes for heavy computing (CPU), and NumPy already does the heavy part in C.

WORKED EXAMPLE, STEP BY STEP**Threads or processes? Work out the time on paper**

You have 4 tasks. Estimate the total time under each approach, for two different kinds of task.

Case A: each task waits 2 seconds for a website to reply (I/O-bound).

1. **One after another:** $4 \times 2 = 8$ seconds.
2. **Four threads:** while a thread waits for the network it releases the GIL, so all four wait at the same time. Total: about 2 seconds.

Case B: each task does 2 seconds of pure Python arithmetic (CPU-bound).

3. **One after another:** 8 seconds.
4. **Four threads:** only one thread can run Python code at a time, so they take turns. Total: still about 8 seconds, sometimes slightly more.
5. **Four processes on a 4-core machine:** each process has its own interpreter and its own GIL, so they truly run together. Total: about 2 seconds, plus a little start-up cost.

	Sequential	4 threads	4 processes
Waiting on the network	8 s	about 2 s	about 2 s (wasteful)
Heavy Python computation	8 s	about 8 s	about 2 s

What this shows. Ask "is the program waiting or computing?" Waiting: threads or asyncio. Computing: processes, or better, NumPy, which does the heavy loop in C.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Pick the tool for each job: (a) download 300 files from a server; (b) compute a statistic over each of 8 large Python lists; (c) add two arrays of 10 million numbers.

Solution. (a) Threads or asyncio: the time is spent waiting on the network, and waiting threads do not block one another. (b) Multiprocessing (a process pool): it is CPU-bound Python, so threads would take turns under the GIL. (c) NumPy: `a + b` runs in compiled code in a fraction of a second; no threads or processes needed.

THE FUNDAMENTALS**Writing a context manager with a generator**

The shorter way to write a context manager is a generator with one `yield`, decorated with `contextlib.contextmanager`. The code before `yield` is the setup, the code after it is the cleanup, and `try / finally` guarantees the cleanup even if the block raises:

```
import time
from contextlib import contextmanager

@contextmanager
def timer(label):
    start = time.perf_counter()
    try:
        yield                                # the body of the `with` block runs here
    finally:
        print(f"{label}: {time.perf_counter() - start:.3f}s")

with timer("sum"):
    sum(range(10**6))                       # prints something like sum: 0.012s
```

`contextlib.closing(obj)` wraps any object that has a `.close()` method, such as a `sqlite3` connection, so that it is closed when the block ends. The habit to show in interviews: any resource you open (a file, a connection, a lock) is opened in a `with`.

SAY IT IN 60 SECONDS

"CPython has a Global Interpreter Lock, so only one thread runs Python bytecode at a time. That means threads do not speed up CPU-heavy pure Python, but they do help when the program is mostly waiting on files or networks. For CPU-bound work I use multiprocessing, or libraries like NumPy that do the heavy loops in C. Memory is managed with reference counting, plus a cyclic garbage collector for reference loops. For classes, `repr` is for developers and `str` is for readable output. And 'with' guarantees cleanup even if an error occurs. The catch: choosing threads for CPU-bound work is a classic mistake; match the tool to whether the bottleneck is waiting or computing."

KEY TERMS GIL · multiprocessing · reference counting · garbage collection · dunder methods · context manager · module vs package · exceptions

BUILDS ON 1.3 Python questions I: the data model

HANDS-ON PRACTICE Use `multiprocessing.Pool` to parallelise a CPU-heavy function and compare the time with a `ThreadPool`.

COMMON MISTAKES Saying threads make CPU-bound Python code faster · Bare `except` clauses · Mixing up `append` and `extend`

CHECK YOURSELF

1. You must call 500 web APIs and also crunch a huge array. What do you use for each?

Threads or `asyncio` for the 500 API calls (I/O-bound); NumPy or multiprocessing for the array work (CPU-bound).

2. What is the difference between `__str__` and `__repr__`, and which should you write first?

`__repr__` is unambiguous for developers, `__str__` readable for users. Write `__repr__` first: `str()` falls back to it.

3. Why is `except:` with no exception type dangerous?

It also catches `KeyboardInterrupt` and hides real bugs. Catch the specific exception you expect.

1.5 Coding patterns: strings

IN PLAIN WORDS

Most string puzzles are the same three moves in different costumes: count the letters (a tally sheet), walk from both ends toward the middle (two fingers), or remember what you saw already (a notebook). A palindrome check is two fingers. An anagram check is comparing two tally sheets. "First letter that appears once" is a tally sheet and then one more pass.

Why it matters: These are warm-up questions; nailing them fast and cleanly earns you time for the harder problem.

Takeaway: Count with a Counter, scan with two pointers, and always say the complexity.

WORKED EXAMPLE, STEP BY STEP

First non-repeating character, traced

Find the first character in "swiss" that appears only once.

- 1. Plan in words.** A character's count tells us whether it repeats. So: count everything first, then walk the string again and return the first character whose count is 1.
- 2. Pass 1, build the tally.** Read s, w, i, s, s.

Character	Count
s	3
w	1
i	1

- 3. Pass 2, walk the string in order.** Position 0 is s with count 3: skip. Position 1 is w with count 1: stop.
- 4. Answer:** w, at index 1.
- 5. Cost.** Two passes over n characters: $O(n)$ time. The tally holds at most one entry per distinct character: $O(k)$ space.
- 6. Edge cases to say aloud.** Empty string: return -1. Every character repeats ("aabb"): return -1. Upper and lower case: ask whether A and a are the same.

What this shows. "Count first, then scan" is one of three moves that solve most string questions. The other two are two pointers from both ends, and a set of what you have already seen.

YOUR TURN (COVER THE SOLUTION FIRST)

- Are "listen" and "silent" anagrams? Show the check by counting, and give the time complexity.

Solution. Tally each word. listen: l1, i1, s1, t1, e1, n1. silent: s1, i1, l1, e1, n1, t1. The two tallies are identical, so they are anagrams. In code: `Counter(a) == Counter(b)`. One pass over each word, so $O(n)$ time, compared with $O(n \log n)$ for `sorted(a) == sorted(b)`. A quick first check: if the lengths differ, answer no immediately.

SAY IT IN 60 SECONDS

"Most string problems reduce to three moves. Count characters with a Counter, scan from both ends with two pointers, or remember what I have seen with a dict or stack. A palindrome is two pointers moving inward after cleaning the text. Anagrams are equal if their Counters match, which is linear, versus sorting which is $n \log n$. The first non-repeating character is one pass to count and one pass to find it. Valid parentheses uses a stack. The catch: I always clarify case, punctuation and empty input first, test on a tiny example, and state time and space complexity. Building a string with plus-equals in a loop is quadratic, so I join a list instead."

KEY TERMS palindrome · anagram · Counter · two pointers · stack · string building

HANDS-ON PRACTICE Solve 10 easy string problems on LeetCode and say the complexity out loud for each.

COMMON MISTAKES Forgetting to normalise case or strip non-alphanumerics · Sorting when a Counter is enough · Not stating time and space complexity

CHECK YOURSELF**1. Which data structure checks balanced brackets, and why?**

A stack: push openers; each closer must match the most recent unclosed opener on top.

2. Palindrome in $O(1)$ extra space: how?

Two pointers from both ends moving inward, skipping non-alphanumerics and comparing lowercased characters.

3. Why is `Counter(a) == Counter(b)` better than sorting for an anagram test?

$O(n)$ time versus $O(n \log n)$, and it handles any hashable symbol.

1.6 Coding patterns: arrays and hash maps

IN PLAIN WORDS

A hash map is a notebook where you can look up anything instantly by its name. Many array puzzles are solved by reading the array once and writing each thing you see into the notebook, so later items can ask "have I already seen what I need?". For two-sum you ask, for each number, "have I already seen the partner that adds up to the target?". Grouping anagrams is sorting the letters of each word to make a label, then filing all words with the same label in the same folder.

Why it matters: "Do it in one pass with a dict" is the key upgrade from a slow loop-in-a-loop.

Takeaway: When you catch yourself writing a nested loop over one array, ask whether a dict or set can replace the inner loop.

WORKED EXAMPLE, STEP BY STEP**Best day to buy and sell, traced**

Prices over six days: `[7, 1, 5, 3, 6, 4]`. Buy once, sell later. What is the best profit?

- 1. Brute force first.** Try every pair of buy day and later sell day: 15 pairs for 6 days, $O(n^2)$ in general.

2. The better idea. On any day, the best sale you can make today is today's price minus the *lowest price seen so far*. So carry two numbers through one pass: `lowest` and `best`.

3. Trace it.

Day price	lowest so far	profit if sold today	best so far
7	7	0	0
1	1	0	0
5	1	4	4
3	1	2	4
6	1	5	5
4	1	3	5

4. Answer: 5 (buy at 1, sell at 6).

5. Cost. One pass, two variables: $O(n)$ time, $O(1)$ extra space.

6. Edge case. Prices that only fall, such as `[7, 6, 4]`: `best` never rises above 0, which is the right answer (do not trade).

What this shows. Many array problems become one pass once you ask "what small piece of information about the past do I need to carry forward?" Here it is one number.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Find the largest sum of a contiguous run in `[2, -3, 4, -1, 2]`. Carry two numbers: `cur` (best sum ending at this position) and `best`. Show the trace.

Solution. At each number, either extend the current run or restart at this number: `cur = max(x, cur + x)`. Start with `cur = best = 2`. Then `-3`: `cur = max(-3, -1) = -1`, `best = 2`. Then `4`: `cur = max(4, 3) = 4`, `best = 4`. Then `-1`: `cur = 3`, `best = 4`. Then `2`: `cur = 5`, `best = 5`. Answer: 5, from the run `[4, -1, 2]`. This is Kadane's algorithm: $O(n)$ time, $O(1)$ space.

SAY IT IN 60 SECONDS

"The key upgrade in array problems is replacing a nested loop with a hash map. For two-sum, I read the array once and for each number ask whether I have already seen its partner, target minus the number; I store each value after checking. That is linear time and linear space instead of quadratic. Grouping anagrams uses the sorted letters as a dictionary key. Merging sorted lists uses two pointers. Kadane's algorithm tracks the best subarray ending here. The catch: I check duplicates and empty input, and I only store after checking so an element does not pair with itself."

KEY TERMS hash map · two sum · complement lookup · deduplicate · merge two pointers · Kadane · defaultdict

HANDS-ON PRACTICE Do 10 easy array/hash-map problems and one medium each day for a week; write the pattern name next to each.

COMMON MISTAKES Using a nested loop when a dict gives $O(n)$ · Mutating the input unintentionally · Not handling duplicates or empty input

CHECK YOURSELF**1. Why store `seen[n] = i` after the check in two-sum?**

Otherwise a number could pair with itself (for example target 8 and value 4 appearing once).

2. Maximum subarray in $O(n)$: what do you track?

The best sum ending at the current element (extend or restart at x) and the overall best (Kadane).

3. How do you group anagrams?

Use the sorted letters of each word as a key in a `defaultdict(list)` and append each word.

1.7 Coding patterns: the classics

IN PLAIN WORDS

These three are the warm-up questions. FizzBuzz checks you can put conditions in the right order (check the most specific one, "divisible by both", first). Fibonacci checks you know that adding the last two numbers as you go is far faster than asking the question again from scratch each time. A prime test checks you know you only need to look for divisors up to the square root, because if a number has a small factor it has a matching big one.

Why it matters: They are quick signals of basic care, and a slow answer here costs you confidence.

Takeaway: Order conditions from specific to general, avoid repeating work, and stop early when you can.

WORKED EXAMPLE, STEP BY STEP**Why the simple Fibonacci explodes: count the calls**

The recursive definition is two lines: `fib(n) = fib(n-1) + fib(n-2)`, with `fib(0) = 0` and `fib(1) = 1`.

1. Draw the calls for fib(5). It calls `fib(4)` and `fib(3)`. `fib(4)` calls `fib(3)` and `fib(2)`. Already `fib(3)` is being computed twice.

2. Count every call. `Calls(n) = Calls(n-1) + Calls(n-2) + 1`, and a single call for $n = 0$ or 1 .

n	2	3	4	5	10	30
Calls	3	5	9	15	177	2,692,537

3. See the waste. For `fib(5)`, `fib(3)` is computed 2 times, `fib(2)` 3 times and `fib(1)` 5 times. The same answers are worked out again and again.

4. Fix 1: remember answers. Store each `fib(k)` the first time it is computed. Each value is then computed once: about n steps.

5. Fix 2: build upward. Keep only the last two numbers: $(0, 1) \rightarrow (1, 1) \rightarrow (1, 2) \rightarrow (2, 3) \rightarrow (3, 5) \rightarrow (5, 8)$. After n updates the first number is `fib(n)`. For $n = 30$ that is 30 steps instead of 2.7 million calls.

What this shows. When a problem's sub-problems repeat, store them. This is the whole idea of dynamic programming, and it is why interviewers ask for Fibonacci.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Is 91 prime? Show the divisors you test and say why you can stop where you do.

Solution. You only need to test divisors up to the square root of 91, which is about 9.5: if $91 = a \times b$ with $a \leq b$, then a cannot be larger than the square root. Test 2: no (91 is odd). 3: no ($9 + 1 = 10$ is not a multiple of 3). 5: no. 7: yes, $91 = 7 \times 13$. So 91 is not prime, and the check needed four divisions, not ninety.

SAY IT IN 60 SECONDS

"Three classics test basic care. FizzBuzz: check divisible by fifteen first, or build the string from Fizz and Buzz pieces, otherwise fifteen prints only Fizz. Fibonacci: the naive recursion is exponential because it recomputes the same values; an iterative loop keeping the last two numbers is linear time and constant space, and memoisation also works. Primes: if a number has a factor, it has one at most its square root, so I test divisors only up to the square root. The catch: state complexity and test edge cases like zero, one and two. These problems are small, but a slow or sloppy answer costs confidence."

KEY TERMS FizzBuzz · Fibonacci · prime test · loop invariants · iterative vs recursive

HANDS-ON PRACTICE Write each from memory, then add tests for edge cases ($n = 0, 1, 2$, negative).

COMMON MISTAKES Checking divisibility by 3 and 5 before checking 15 · Recursing without memoisation · Looping to n instead of the square root of n

CHECK YOURSELF

1. Why is naive recursive Fibonacci $O(2^n)$?

Each call spawns two more and the same subproblems are recomputed repeatedly; memoisation or a loop makes it $O(n)$.

2. Why only test divisors up to the square root of n ?

If $n = a \times b$ with $a \leq b$ then $a \leq \sqrt{n}$, so any composite number has a divisor within that range.

3. How can you test "n is a power of two" without a loop?

$n > 0$ and $n \& (n - 1) == 0$: a power of two has exactly one set bit.

1.8 Implement from scratch I: statistics and metrics

IN PLAIN WORDS

"From scratch" questions check that you understand the recipe, not just the button to press. The mean is fair-share, the variance is the average squared distance from the mean, and the median is the middle person when everyone stands in order of height. A train-test split is like hiding some exam questions to test a student later. Precision asks "of everything I flagged, how much was right?" and recall asks "of everything that was really there, how much did I find?".

Why it matters: Writing these proves you know what the library is doing for you.

Takeaway: Know the formula, say why, handle the edge cases (empty input, divide by zero), then compare with the library.

WORKED EXAMPLE, STEP BY STEP**Mean, median, variance and standard deviation by hand**

Delivery times in minutes for five orders: 20, 30, 25, 45, 30.

- 1. Mean.** Add and divide: $(20 + 30 + 25 + 45 + 30) / 5 = 150 / 5 = 30$.
- 2. Median.** Sort: 20, 25, 30, 30, 45. The middle value is **30**.
- 3. Distances from the mean.** Subtract 30 from each value, then square.

Value	Distance from 30	Squared
20	-10	100
30	0	0
25	-5	25
45	+15	225
30	0	0
	sum = 0	sum = 350

- 4. Variance.** Average the squares. Dividing by $n = 5$ gives 70 (population variance).
Dividing by $n - 1 = 4$ gives **87.5** (sample variance, used when the data is a sample).
- 5. Standard deviation.** The square root, back in minutes: $\sqrt{87.5} \approx 9.4$ minutes.
- 6. Read it.** "A typical delivery takes 30 minutes, give or take about 9."

What this shows. The distances always sum to zero, which is why we square them before averaging. Variance is in squared units; the standard deviation brings it back to the units of the data.

YOUR TURN (COVER THE SOLUTION FIRST)

- For the values 3, 7, 7, 19, compute the mean, the median, the sample variance and the sample standard deviation. Which describes the "typical" value better here, mean or median?

Solution. Mean = $36 / 4 = 9$. Median = $(7 + 7) / 2 = 7$. Distances from 9: -6, -2, -2, +10; squares 36, 4, 4, 100; sum 144. Sample variance = $144 / 3 = 48$; standard deviation = $\sqrt{48} \approx 6.9$. The value 19 pulls the mean above three of the four values, so the median (7) describes a typical value better.

SAY IT IN 60 SECONDS

"From-scratch questions test that I know the recipe. Mean is the sum over n . Variance is the average squared distance from the mean, divided by n for a population or n minus one for a sample, because the sample mean was estimated from the same data. Standard deviation is its square root. The median is the middle value of the sorted data, or the average of the two middle ones. A train-test split shuffles row indices with a fixed seed and cuts at the training fraction. From the confusion matrix, precision is true positives over predicted positives, recall is true positives over actual positives, F1 is their harmonic mean. The catch: guard divide-by-zero, and say which variance you used."

KEY TERMS variance · Bessel correction · median · train-test split · confusion matrix · precision · recall · F1

HANDS-ON PRACTICE Implement each function, then compare with numpy/sklearn on random data to confirm they agree.

COMMON MISTAKES Dividing by n instead of $n-1$ for a sample variance without saying so · Splitting without shuffling ordered data · Dividing by zero in precision or recall

CHECK YOURSELF

1. What does `ddof=1` do and what is NumPy's default?

It divides by $n - 1$ (sample variance). NumPy defaults to 0 (population), pandas `.var()` to 1.

2. TP=30, FP=10, FN=20. Precision and recall?

Precision $30/40 = 0.75$; recall $30/50 = 0.60$.

3. Why shuffle before a train-test split, and when must you not?

Shuffling avoids sorted files putting one class or period in the test set. Do not shuffle time series: split by time.

1.9 Implement from scratch II: regression, k-means, KNN

IN PLAIN WORDS

Training a model by gradient descent is like walking down a foggy hill: you cannot see the bottom, but you can feel which way is downhill under your feet, so you take a small step that way and repeat. The size of each step is the learning rate. For k-means, imagine dropping k flags on a playground and asking every kid to stand at the nearest flag; then you move each flag to the middle of its kids, and repeat until nobody wants to switch. KNN is even simpler: to guess what a new kid likes, ask the k kids who live closest.

Why it matters: Interviewers ask you to write these to see whether you understand the loop and the gradients, not just the library call.

Takeaway: Loss, gradient, step. Assign, update, repeat. Look at neighbours and vote.

WORKED EXAMPLE, STEP BY STEP

One step of gradient descent, by hand

Three points lie exactly on the line $y = 2x$: (1, 2), (2, 4), (3, 6). Fit `prediction = w * x` starting from $w = 0$, with learning rate 0.1.

- 1. Predict.** With $w = 0$ every prediction is 0.
- 2. Errors (prediction – actual):** -2, -4, -6.
- 3. Loss.** Mean squared error = $(4 + 16 + 36) / 3 = 18.67$.
- 4. Gradient.** The slope of the loss with respect to w is $(2/n) \times \sum(\text{error} \times x) = (2/3) \times (-2 \times 1 + -4 \times 2 + -6 \times 3) = (2/3) \times (-28) = -18.67$. Negative means "increase w to reduce the loss".
- 5. Update.** $w = 0 - 0.1 \times (-18.67) = 1.867$.
- 6. Check.** New predictions 1.87, 3.73, 5.60; errors -0.13, -0.27, -0.40; loss = **0.083**. One step cut the loss from 18.67 to 0.08.
- 7. Second step.** Gradient = $(2/3) \times (-0.133 - 0.533 - 1.2) = -1.244$, so $w = 1.867 + 0.124 = 1.991$, very close to the true value 2.

What this shows. Training is this loop: predict, measure the error, find which direction reduces it, take a small step. Try learning rate 0.3 on paper: w jumps to 5.6, overshoots, and the loss gets worse. That is what "learning rate too high" means.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Run k-means by hand with $k = 2$ on the numbers 1, 2, 3, 10, 11, 12, starting from centres 1 and 2. Do two rounds of "assign, then move the centres".

Solution. Round 1, assign each point to the nearer centre: 1 goes to centre 1; the points 2, 3, 10, 11, 12 are all nearer to centre 2. Move the centres to the means: centre A = 1, centre B = $(2 + 3 + 10 + 11 + 12) / 5 = 7.6$. Round 2, assign again: 1, 2, 3 are nearer to 1 than to 7.6; 10, 11, 12 are nearer to 7.6. Move: centre A = 2, centre B = 11. A third round changes nothing, so it has converged on the two obvious groups, despite the poor starting centres.

SAY IT IN 60 SECONDS

"Gradient descent trains a model by repeating three steps: compute predictions, compute the gradient of the loss, and move the weights a small step against it. For linear regression with squared error the gradient is two over n times X transpose times the errors. For logistic regression with cross-entropy it is one over n times X transpose times prediction minus label, the same shape. K-means alternates two steps: assign each point to its nearest centroid, then move each centroid to the mean of its points, until nothing changes. KNN stores the data and votes among the k nearest points. The catch: validate on synthetic data with known weights, scale features, and remember k-means only finds a local optimum."

KEY TERMS gradient descent · mean squared error · cross-entropy gradient · sigmoid · k-means · Lloyd algorithm · KNN · vectorization

BUILDS ON 1.8 Implement from scratch I: statistics and metrics

HANDS-ON PRACTICE Generate synthetic data, train your implementation, and compare weights with sklearn to 2 decimals.

COMMON MISTAKES Not scaling features, so gradient descent diverges or crawls · Forgetting the bias term · Empty clusters in k-means

CHECK YOURSELF

1. **Your gradient descent loss becomes NaN. First suspects?**

Learning rate too high or unscaled features; also check for $\log(0)$ or bad data.

2. **How do you check your from-scratch logistic regression is right?**

Generate data from known weights and compare with sklearn's `LogisticRegression` (very large C) to two decimals.

3. **Why does k-means give different results across runs, and what fixes it?**

Random initialisation reaches different local optima. Use `k-means++`, several restarts and a fixed seed.

1.10 Practice targets and self-check

IN PLAIN WORDS

Coding interviews reuse about eight ideas in new costumes. Need to know if you have seen something? Use a set or dict. Pairs in a sorted list? Two pointers. Longest or shortest stretch with a rule? A sliding window. Totals over ranges? Prefix sums. Nested brackets or undo? A stack. Shortest path? BFS. Top K? A heap. Overlapping meetings? Sort and merge. After you solve a problem, write the idea in one line, so next time you recognise it in seconds.

Why it matters: Speed in a live round comes from recognising the pattern, not from inventing a solution.

Takeaway: Learn the eight patterns, solve against a timer, and say your plan aloud.

WORKED EXAMPLE, STEP BY STEP

A live coding round, narrated

The question: "Given a sentence, return the most frequent word." Here is what a strong candidate says and does, in order.

- 1. Restate.** "So I take a string, split it into words, and return the word that appears most often."
- 2. Clarify.** "Should 'The' and 'the' count as the same word? What about punctuation? If two words tie, which do I return?" (Suppose: ignore case, no punctuation, any of the tied words.)
- 3. Example by hand.** "For 'the cat and the hat' the counts are the: 2, cat: 1, and: 1, hat: 1, so the answer is 'the'."
- 4. Plan and cost, before typing.** "I will lower-case and split, count with a dictionary in one pass, then take the key with the largest count. That is $O(n)$ time and $O(k)$ space for k distinct words."
- 5. Code, speaking as you go.**

```
from collections import Counter
def most_frequent(sentence):
    words = sentence.lower().split()
    if not words:
        return None
    return Counter(words).most_common(1)[0][0]
```

- 6. Test aloud.** The example gives 'the'. An empty string returns None. A single word returns that word.
- 7. Close.** "Time $O(n)$, space $O(k)$. If the text were huge I would read it as a stream and keep only the counts."

What this shows. The code is five lines. The marks are in steps 1 to 4 and 6 to 7. Practise the narration, not just the solution.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Narrate steps 1 to 4 for this question: "Given two lists, return True if they share at least one element."

Solution. Restate: "I return True if any value appears in both lists." Clarify: "Can the lists be empty? Are the items hashable, such as numbers or strings? Do duplicates matter?" Example: [1, 2, 3] and [9, 3] share 3, so True; [1, 2] and [5] gives False. Plan: brute force compares every pair, $O(n \times m)$. Better: put the first list into a set (n steps), then check each item of the second (m steps), stopping at the first hit: $O(n + m)$ time, $O(n)$ space. In code: `return not set(a).isdisjoint(b)` or `any(x in s for x in b)`.

THE FUNDAMENTALS**1.5 Practice list (do all)**

Difficulty	Count	Source
Easy	40	LeetCode (arrays, strings, hash maps)
Medium	20	LeetCode (two pointers, sliding window, intervals)
Python-specific	15	HackerRank Python track

1.6 Self-check (Module 1)

- ☐ Can you explain mutable default arguments, shallow vs deep copy, generators, and decorators without notes?
- ☐ Can you write `two_sum`, `group_anagrams`, and `kmeans` from memory?
- ☐ Can you solve an easy LeetCode problem in under 15 minutes?

SAY IT IN 60 SECONDS

"For coding rounds I follow the same script. Restate the problem and ask about edge cases. Work a small example by hand. State a brute-force solution and its complexity, then improve it, usually by swapping a loop for a hash map or two pointers. Write clean code while explaining. Test with the example and an edge case. Finally state time and space complexity. To get fast I practise with a timer, fifteen minutes for easy and thirty for medium, I look only at a hint after ten minutes, and I write the pattern in one line afterwards. The catch: silence hurts, so I think aloud."

KEY TERMS timed practice · pattern recognition · LeetCode · HackerRank · STAR for coding · communication

HANDS-ON PRACTICE Do two timed problems today (20 min each) and write a one-line pattern note for each.

COMMON MISTAKES Reading solutions too early · Practising without a timer · Not reviewing failed problems

CHECK YOURSELF

1. You see "longest substring with at most 2 distinct characters". Which pattern?

Sliding window with a dict of counts; shrink from the left when there are more than two distinct characters.

2. "Number of subarrays that sum to k": which pattern?

Prefix sums with a hash map counting earlier prefix values (`count[cur - k]`).

3. After 10 minutes stuck, what should you do in practice and in the interview?

Practice: read only a hint, then retry. Interview: say what you know, propose an approach and ask a targeted question rather than going silent.

Interview tips, tricks and hacks

- 1. Talk before you type.** Restate the problem, give one example input and output, name the approach and its complexity, and only then write code. Silent coding is the most common reason strong candidates are marked down.
- 2. Start with the brute force, say its cost, then improve.** "Two loops, $O(n^2)$. A dictionary of seen values makes it one pass, $O(n)$ time and $O(n)$ space." You get credit for the reasoning even if the optimal code is not perfect.
- 3. Reach for a dictionary or set first.** Counting, grouping, "have I seen it", "find the complement": a hash map solves most easy and medium data-science coding questions.
- 4. Know your one-liners.** `collections.Counter`, `defaultdict(list)`, `sorted(xs, key=...)`, `enumerate`, `zip`, slicing with `[::-1]`, `''.join(parts)`. They save minutes and signal fluency.
- 5. Test with three inputs aloud.** A normal case, an empty input and a case with duplicates or negatives. Finding your own bug is a positive signal, not a failure.
- 6. Explain behaviour with the names-and-objects picture.** For any "what does this print?" question, draw which names point at which objects. It resolves mutable defaults, aliasing, shallow copies and `is` versus `==`.
- 7. Never use a mutable default argument.** Write `def f(x, acc=None)` and create the list inside. If asked why, give the one-sentence reason: defaults are evaluated once, at definition time.
- 8. Prefer clear over clever.** Readable names and a simple loop beat a dense one-liner you cannot debug under pressure.
- 9. When you are stuck, shrink the problem.** Solve it for a list of two elements by hand, then generalise. Say that this is what you are doing.

Frequently asked questions

62 questions interviewers ask on this chapter, with model answers. Cover the answer, say yours aloud, then compare.

CORE PYTHON LANGUAGE CHECKLIST

Q1 What is the output? `x = [[0] * 3] * 2; x[0][0] = 1; print(x)`

`[[0] * 3] * 2` repeats a **reference** to the same inner list twice, so changing `x[0][0]` changes both rows. The output is `[[1, 0, 0], [1, 0, 0]]`. Build independent rows with a comprehension: `[[0] * 3 for _ in range(2)]`.

Q2 Why is modifying a list while iterating over it dangerous, and what do you do instead?

When you remove an element, everything after it shifts left, but the loop's internal index still advances, so one item is skipped. Safer patterns: build a new list with a comprehension (`[x for x in xs if keep(x)]`), iterate over a copy (`for x in xs[:]`), or iterate backwards when removing by index.

COMPLEXITY CHEAT SHEET FOR PYTHON STRUCTURES**Q3 You loop over 200,000 user IDs and, for each, check whether it is in a blacklist stored as a list of 50,000 IDs. What is the complexity, and how do you fix it?**

Each `in` on a list is $O(m)$ with $m = 50,000$, repeated $n = 200,000$ times, giving about 10^{10} operations. Convert the blacklist to a `set` once ($O(m)$); each lookup is then $O(1)$ on average, so the total is about $O(n + m)$. Mention the trade-off: the set uses more memory and loses order, which is usually fine for membership tests.

PYTHON QUESTIONS I: THE DATA MODEL**Q4 List vs tuple?**

List is mutable, tuple is immutable, hashable (if elements are hashable), slightly faster and smaller in memory. Tuples can be dict keys.

Q5 Shallow vs deep copy?

Shallow copy (`copy.copy` , `list.copy()` , `a[:]`) copies the outer container but shares references to nested objects. Deep copy (`copy.deepcopy`) recursively copies everything.

```
import copy
a = [[1, 2], [3, 4]]
b = copy.copy(a)
b[0][0] = 99
print(a)    # [[99, 2], [3, 4]] → nested object shared
c = copy.deepcopy(a)
c[0][0] = 0
print(a)    # unchanged
```

Q6 `is` vs `==`?

`==` compares values; `is` compares identity (same object in memory).

Q7 Mutable default argument trap?

```
def add(x, lst=[]):      # BAD: list created once at function definition
    lst.append(x)
    return lst
add(1); add(2)          # [1, 2] - surprise!

def add(x, lst=None):   # GOOD
    if lst is None:
        lst = []
    lst.append(x)
    return lst
```

Q8 What is a generator and why use it?

A function with `yield` that produces values lazily, one at a time, using $O(1)$ memory. Useful for large files/streams.

```
def read_large(path):
    with open(path) as f:
        for line in f:
            yield line.strip()
```

Q9 What is a decorator?

A function that takes a function and returns a modified function.

```
import time
from functools import wraps

def timer(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        t = time.time()
        out = fn(*args, **kwargs)
        print(f"{fn.__name__} took {time.time()-t:.4f}s")
        return out
    return wrapper

@timer
def slow():
    time.sleep(0.5)
```

Q10 *args and **kwargs ?

`*args` collects extra positional args into a tuple; `**kwargs` collects extra keyword args into a dict.

Q11 What does this print? `def f(a, L=[]): L.append(a); return L` followed by `print(f(1), f(2), f(3, []))`

`f(1)` and `f(2)` share the default list, so after the second call that single list is `[1, 2]`. Because `print` evaluates all three calls first and then prints, the first two results are the *same list object* and both show `[1, 2]`. `f(3, [])` uses a fresh list: `[3]`. Output: `[1, 2] [1, 2] [3]`.

Q12 Why can a tuple be a dictionary key but a list cannot?

A dictionary finds a key by its hash, so the hash must never change while the key is in the dict. Lists are mutable, so their contents (and any hash based on them) could change; Python refuses with `TypeError: unhashable type`. A tuple of hashable items is immutable and hashable. Note that a tuple containing a list is still unhashable.

PYTHON QUESTIONS II: RUNTIME AND STYLE**Q13 What is the GIL?**

Global Interpreter Lock in CPython: only one thread executes Python bytecode at a time. Threads don't speed up CPU-bound work; use `multiprocessing` (or NumPy/C extensions) for CPU-bound and `threads/asyncio` for I/O-bound work.

Q14 How does Python manage memory?

Reference counting plus a cyclic garbage collector. Memory is managed in private heap.

Q15 `append` vs `extend` vs `+` ?

`append(x)` adds `x` as one element; `extend(iterable)` adds each element; `+` creates a new list.

Q16 What are `__init__` and `__str__` vs `__repr__` ?

`__init__` initializes an instance; `__str__` is the readable string for users; `__repr__` is the unambiguous developer representation.

Q17 List comprehension vs generator expression?

`[x*x for x in r]` builds the whole list in memory; `(x*x for x in r)` is lazy.

Q18 How do you handle exceptions properly?

Catch specific exceptions, avoid bare `except:`, use `finally` for cleanup or use `with`.

Q19 Explain `lambda`, `map`, `filter`, `reduce`.

```
from functools import reduce
nums = [1, 2, 3, 4]
list(map(lambda x: x**2, nums))      # [1, 4, 9, 16]
list(filter(lambda x: x % 2 == 0, nums)) # [2, 4]
reduce(lambda a, b: a + b, nums)    # 10
```

Q20 What is the difference between a module and a package?

Module = a `.py` file; package = a directory of modules (with `__init__.py` traditionally).

Q21 What is the difference between `@staticmethod` and `@classmethod`?

A `@classmethod` gets the class as its first argument (`cls`), so it works with subclasses and is the standard way to write alternative constructors (`Account.from_string(...)`). A `@staticmethod` gets neither `self` nor `cls`; it is an ordinary function placed inside the class namespace for organisation. An instance method gets `self` and can use instance state.

Q22 Why does `with open(...)` as `f:` matter?

`with` calls the object's `__enter__` on entry and `__exit__` on exit, even when an exception is raised. For files that means the handle is always closed and buffers flushed, avoiding leaks and corrupted writes. The same pattern works for locks, database connections and temporary directories.

CODING PATTERNS: STRINGS**Q23 Implement: Palindrome**

```
def is_palindrome(s):
    s = ''.join(c.lower() for c in s if c.isalnum())
    return s == s[::-1]
```

Q24 Implement: Anagram

```
from collections import Counter
def is_anagram(a, b):
    return Counter(a) == Counter(b)
```

Q25 Implement: First non-repeating character

```
from collections import Counter

def first_unique(s):
    cnt = Counter(s)
    for i, ch in enumerate(s):
        if cnt[ch] == 1:
            return i
    return -1
```

Q26 Implement: Reverse words

```
def reverse_words(s):
    return ' '.join(s.split()[::-1])
```

Q27 Implement: Character frequency sorted by count desc, then alphabetical

```
from collections import Counter

def freq(s):
    return sorted(Counter(s).items(), key=lambda x: (-x[1], x[0]))
```

Q28 Implement: valid parentheses. Given a string of `()[]{}` characters, return whether the brackets are balanced and correctly nested.

```
def is_valid(s: str) → bool:
    match = {'(': ')', '[': ']', '{': '}'
    stack = []
    for ch in s:
        if ch in '([{':
            stack.append(ch)
        elif ch in match:
            if not stack or stack.pop() != match[ch]:
                return False
    return not stack          # leftovers mean unclosed openers

is_valid("{}[]")    # True
is_valid("[]")      # False
is_valid("(")       # False
```

$O(n)$ time, $O(n)$ space. The stack captures "the most recent unclosed opener", which is exactly what must be closed next.

Q29 Implement: run-length encoding. Compress `"aaabccddd"` into `"a3bc2d4"` (omit counts of 1).

```
def compress(s: str) → str:
    out, i = [], 0
    while i < len(s):
        j = i
        while j < len(s) and s[j] == s[i]:
            j += 1
        out.append(s[i] + (str(j - i) if j - i > 1 else ""))
        i = j
    return "".join(out)

compress("aaabccddd")    # 'a3bc2d4'
```

$O(n)$ time; collecting pieces in a list and joining once avoids the $O(n^2)$ cost of repeated string concatenation.

Q30 Implement: is `s` a subsequence of `t` ? (`"ace"` is a subsequence of `"abcde"` ; `"aec"` is not.)

```
def is_subsequence(s: str, t: str) → bool:
    it = iter(t)
    return all(ch in it for ch in s)    # `in` consumes the iterator up to the match

is_subsequence("ace", "abcde")    # True
is_subsequence("aec", "abcde")    # False
```

The trick: `ch in it` advances the shared iterator, so each character of `s` must be found *after* the previous one. $O(\text{len}(t))$ time.

Q31 Implement: longest common prefix of a list of words.

```
def longest_common_prefix(words):
    if not words:
        return ""
    prefix = words[0]
    for w in words[1:]:
        while not w.startswith(prefix):
            prefix = prefix[:-1]
    return prefix

longest_common_prefix(["flower", "flow", "flight"]) # 'fl'
longest_common_prefix(["dog", "car"])              # ''
```

Worst case $O(\text{total characters})$. Mention `os.path.commonprefix` exists but that interviewers want the logic.

CODING PATTERNS: ARRAYS AND HASH MAPS**Q32 Implement: Two sum: $O(n)$**

```
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        if target - n in seen:
            return [seen[target - n], i]
        seen[n] = i
```

Q33 Implement: Remove duplicates, preserve order

```
def dedupe(lst):
    return list(dict.fromkeys(lst))
```

Q34 Implement: Second largest without sorting

```
def second_largest(nums):
    first = second = float('-inf')
    for n in nums:
        if n > first:
            first, second = n, first
        elif first > n > second:
            second = n
    return second
```

Q35 Implement: Rotate array right by k

```
def rotate(nums, k):
    k %= len(nums)
    return nums[-k:] + nums[:-k]
```

Q36 Implement: Merge two sorted lists

```
def merge_sorted(a, b):
    i = j = 0; out = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]: out.append(a[i]); i += 1
        else:           out.append(b[j]); j += 1
    return out + a[i:] + b[j:]
```


Q37 Implement: Group anagrams

```
from collections import defaultdict
def group_anagrams(words):
    d = defaultdict(list)
    for w in words:
        d[''.join(sorted(w))].append(w)
    return list(d.values())
```

Q38 Implement: Flatten nested list

```
def flatten(lst):
    for x in lst:
        if isinstance(x, list):
            yield from flatten(x)
        else:
            yield x
```

Q39 Implement: move all zeros to the end while keeping the order of the other elements, in place.

```
def move_zeros(nums):
    j = 0 # next position for a non-zero
    for i, x in enumerate(nums):
        if x != 0:
            nums[j], nums[i] = nums[i], nums[j]
            j += 1
    return nums

move_zeros([0, 1, 0, 3, 12]) # [1, 3, 12, 0, 0]
```

One pass, $O(n)$ time, $O(1)$ extra space. This "reader and writer pointers" pattern also solves remove-duplicates-in-place.

Q40 Implement: best time to buy and sell a stock once. Given daily prices, return the maximum profit (0 if none).

```
def max_profit(prices):
    lowest, best = float("inf"), 0
    for p in prices:
        lowest = min(lowest, p) # cheapest buy so far
        best = max(best, p - lowest) # best sale today
    return best

max_profit([7, 1, 5, 3, 6, 4]) # 5 (buy at 1, sell at 6)
max_profit([7, 6, 4, 3, 1]) # 0
```

$O(n)$ time, $O(1)$ space. Interview follow-up: allow many transactions (sum every positive day-to-day rise).

Q41 Implement: maximum sum of a contiguous subarray (Kadane's algorithm).

```
def max_subarray(nums):
    cur = best = nums[0]
    for x in nums[1:]:
        cur = max(x, cur + x) # extend, or restart at x if the running sum hurts
        best = max(best, cur)
    return best

max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4]) # 6 (subarray [4, -1, 2, 1])
```

$O(n)$ time, $O(1)$ space. Initialising with `nums[0]` (not 0) handles all-negative arrays correctly.

Q42 Implement: intersection of two arrays, keeping duplicates ([1,2,2,1] and [2,2] give [2,2]).

```
from collections import Counter

def intersect(a, b):
    counts, out = Counter(a), []
    for x in b:
        if counts[x] > 0:
            out.append(x)
            counts[x] -= 1
    return out

intersect([1, 2, 2, 1], [2, 2]) # [2, 2]
```

$O(n + m)$ time. Use `set(a)` & `set(b)` when duplicates do not matter.

Q43 Implement: product of array except self, without division.

```
def product_except_self(nums):
    n = len(nums)
    out = [1] * n
    left = 1
    for i in range(n):
        out[i] = left # product of everything to the left of i
        left *= nums[i]
    right = 1
    for i in range(n - 1, -1, -1):
        out[i] *= right # times everything to the right of i
        right *= nums[i]
    return out

product_except_self([1, 2, 3, 4]) # [24, 12, 8, 6]
```

$O(n)$ time, $O(1)$ extra space besides the output, and no division so zeros are safe.

CODING PATTERNS: THE CLASSICS**Q44 Implement: FizzBuzz**

```
def fizzbuzz(n):
    for i in range(1, n+1):
        print("FizzBuzz" if i%15==0 else "Fizz" if i%3==0 else "Buzz" if i%5==0 else i)
```

Q45 Implement: The nth Fibonacci number (iterative)

```
def fib(n): # iterative  $O(n)$ ,  $O(1)$  space
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

Q46 Implement: Primality test

```
def is_prime(n):
    if n < 2: return False
    i = 2
    while i * i ≤ n:
        if n % i == 0: return False
        i += 1
    return True
```

Q47 Implement: greatest common divisor (Euclid's algorithm).

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

gcd(48, 18)    # 6
```

Runs in $O(\log \min(a, b))$. In real code use `math.gcd`.

Q48 Implement: find the missing number in $0..n$ given a list of n distinct numbers from that range.

```
def missing_number(nums):
    n = len(nums)
    return n * (n + 1) // 2 - sum(nums)

missing_number([3, 0, 1])    # 2
```

$O(n)$ time, $O(1)$ space. An alternative that avoids big-number overflow concerns in other languages is XOR of all indices and values.

Q49 Implement: is n a power of two? (One line, no loops.)

```
def is_power_of_two(n):
    return n > 0 and n & (n - 1) == 0

is_power_of_two(16)    # True    (10000 & 01111 = 0)
is_power_of_two(18)    # False
```

Subtracting 1 flips the lowest set bit and everything below it, so AND-ing removes exactly that bit.

IMPLEMENT FROM SCRATCH I: STATISTICS AND METRICS**Q50 Implement from scratch: Mean, variance, std**

```
def mean(x): return sum(x) / len(x)
def var(x, ddof=0):
    m = mean(x)
    return sum((v - m) ** 2 for v in x) / (len(x) - ddof)
def std(x, ddof=0): return var(x, ddof) ** 0.5
```

Q51 Implement from scratch: Median

```
def median(x):
    s = sorted(x); n = len(s)
    return s[n//2] if n % 2 else (s[n//2 - 1] + s[n//2]) / 2
```

Q52 Implement from scratch: Train-test split

```
import numpy as np

def train_test_split(X, y, test_size=0.2, seed=42):
    rng = np.random.default_rng(seed)
    idx = rng.permutation(len(X))
    cut = int(len(X) * (1 - test_size))
    tr, te = idx[:cut], idx[cut:]
    return X[tr], X[te], y[tr], y[te]
```

Q53 Implement from scratch: Confusion matrix metrics (binary)

```
import numpy as np

def metrics(y_true, y_pred):
    y_true, y_pred = np.array(y_true), np.array(y_pred)
    tp = np.sum((y_true == 1) & (y_pred == 1))
    tn = np.sum((y_true == 0) & (y_pred == 0))
    fp = np.sum((y_true == 0) & (y_pred == 1))
    fn = np.sum((y_true == 1) & (y_pred == 0))
    precision = tp / (tp + fp) if tp + fp else 0
    recall = tp / (tp + fn) if tp + fn else 0
    f1 = 2 * precision * recall / (precision + recall) if precision + recall else 0
    accuracy = (tp + tn) / len(y_true)
    return dict(tp=tp, tn=tn, fp=fp, fn=fn, precision=precision, recall=recall, f1=f1,
                accuracy=accuracy)
```

Q54 Implement: Pearson correlation from scratch, and say what it measures.

```
def pearson(x, y):
    mx, my = sum(x) / len(x), sum(y) / len(y)
    cov = sum((a - mx) * (b - my) for a, b in zip(x, y))
    sx = sum((a - mx) ** 2 for a in x) ** 0.5
    sy = sum((b - my) ** 2 for b in y) ** 0.5
    return cov / (sx * sy)

pearson([1, 2, 3, 4], [2, 4, 6, 8])          # 1.0
pearson([-2, -1, 0, 1, 2], [4, 1, 0, 1, 4]) # 0.0 (perfect y = x^2 relationship, zero linear correlation)
```

The second example is the classic interview point: zero correlation does not mean independence.

Q55 Implement: standardise a list to z-scores, and explain when you need it.

```
def zscore(x):
    m = mean(x)
    s = std(x)          # population std; use ddof=1 if you prefer the sample version
    return [(v - m) / s for v in x]
```

Standardising puts features on a common scale so no feature dominates distances (KNN, k-means), regularised linear models treat them fairly, and gradient descent converges faster. Fit the mean and std on the **training** data only and reuse them on the test data.

IMPLEMENT FROM SCRATCH II: REGRESSION, K-MEANS, KNN**Q56 Implement from scratch: Linear regression with gradient descent**

```
import numpy as np

def linreg_gd(X, y, lr=0.01, epochs=1000):
    n, d = X.shape
    w, b = np.zeros(d), 0.0
    for _ in range(epochs):
        y_hat = X @ w + b
        err = y_hat - y
        w -= lr * (2 / n) * (X.T @ err)
        b -= lr * (2 / n) * err.sum()
    return w, b
```

Q57 Implement from scratch: Logistic regression with gradient descent

```
import numpy as np

def sigmoid(z): return 1 / (1 + np.exp(-z))
def logreg_gd(X, y, lr=0.1, epochs=2000):
    n, d = X.shape
    w, b = np.zeros(d), 0.0
    for _ in range(epochs):
        p = sigmoid(X @ w + b)
        w -= lr * (X.T @ (p - y)) / n
        b -= lr * (p - y).mean()
    return w, b
```

Q58 Implement from scratch: K-Means

```
import numpy as np

def kmeans(X, k, iters=100, seed=0):
    rng = np.random.default_rng(seed)
    C = X[rng.choice(len(X), k, replace=False)]
    for _ in range(iters):
        d = np.linalg.norm(X[:, None] - C[None], axis=2) # (n, k)
        labels = d.argmin(axis=1)
        newC = np.array([X[labels == j].mean(axis=0) if np.any(labels == j) else C[j] for j in
range(k)])
        if np.allclose(C, newC): break
        C = newC
    return labels, C
```

Q59 Implement from scratch: KNN classifier

```
import numpy as np

def knn_predict(Xtr, ytr, x, k=3):
    d = np.linalg.norm(Xtr - x, axis=1)
    nearest = ytr[np.argsort(d)[:k]]
    return np.bincount(nearest).argmax()
```

Q60 Implement: linear regression using the normal equation, and say when you would not use it.

```
def linreg_normal(X, y):
    Xb = np.c_[X, np.ones(len(X))] # add a bias column
    return np.linalg.solve(Xb.T @ Xb, Xb.T @ y) # last entry is the intercept
```

Use `solve` or `lstsq` rather than an explicit inverse (more stable). It costs $O(d^3)$ in the number of features, fails when $X^T X$ is singular (perfect multicollinearity), and cannot be used for models without a closed form (logistic regression). Gradient descent scales to huge data through mini-batches.

Q61 How would you turn batch gradient descent into mini-batch SGD?

```
def linreg_sgd(X, y, lr=0.05, epochs=50, batch=32, seed=0):
    rng = np.random.default_rng(seed)
    n, d = X.shape
    w, b = np.zeros(d), 0.0
    for _ in range(epochs):
        order = rng.permutation(n)          # reshuffle each epoch
        for s in range(0, n, batch):
            idx = order[s:s + batch]
            err = X[idx] @ w + b - y[idx]
            w -= lr * (2 / len(idx)) * (X[idx].T @ err)
            b -= lr * (2 / len(idx)) * err.sum()
    return w, b
```

Each update is noisier but far cheaper, and the noise can help escape shallow local minima in non-convex problems. Mini-batches also fit GPU memory.

PRACTICE TARGETS AND SELF-CHECK**Q62 You submit a solution and it fails one hidden test. What is your debugging routine?**

Re-read the statement for a missed constraint; list the edge cases (empty, single element, duplicates, negatives, very large values); hand-trace the smallest failing-looking input; add prints or assertions for the invariants you believe are true; and check integer division, off-by-one boundaries and whether you mutated the input. Explain aloud as you go: interviewers watch the process.

TYPES AND MUTABILITY

Type	Mutable	Hashable	Note
int float bool str bytes	no	yes	bool is an int
tuple	no	if items are	fixed record
frozenset	no	yes	immutable set
list	yes	no	ordered, dynamic array
dict	yes	no	insertion-ordered (3.7+)
set	yes	no	unique, unordered

- Dict keys and set members must be hashable. `None` is a singleton: test with `is None`.
- Truthiness: `0`, `0.0`, `''`, `[]`, `{}`, `set()`, `None` are falsy. `[] == False` is `False`; `bool([])` is `False`.
- `==` compares values, `is` compares identity. Small ints and interned strings may share objects: never rely on `is` for values.

NUMBERS, TRUTHINESS, FORMATTING

<code>7 // 2, -7 // 2, -7 % 2</code>	<code># 3, -4, 1 (floors)</code>
<code>0.1 + 0.2 == 0.3</code>	<code># False</code>
<code>math.isclose(0.1 + 0.2, 0.3)</code>	<code># True</code>
<code>int("3.7")</code>	<code># ValueError</code>
<code>int(float("3.7")), int(3.99)</code>	<code># 3, 3 (truncates)</code>
<code>bool("False"), bool([0])</code>	<code># True, True</code>
<code>f"{0.4567:.1%} {42:05d}"</code>	<code># '45.7% 00042'</code>
<code>f"{'a':<4} {'b':>4} {n:}"</code>	<code># align; debug</code>

- Compare floats with a tolerance. `round(2.5)` is `2`: ties go to even.
- Text from a file is truthy even when it says "False": compare `s.lower() == "true"`. Use `x is None`, not `x or d`, when `0` is valid.

COMPLEXITY OF BUILT-INS

Operation	Cost
<code>list[i]</code> , <code>append</code> , <code>pop()</code>	$O(1)$
<code>list.insert(0, x)</code> , <code>pop(0)</code> , <code>x in list</code>	$O(n)$
<code>dict/set</code> <code>get</code> , <code>set</code> , <code>in</code> , <code>delete</code>	$O(1)$ avg
<code>sorted</code> , <code>list.sort</code>	$O(n \log n)$, stable
<code>deque</code> <code>append/pop</code> both ends	$O(1)$
<code>heapq</code> <code>push / pop</code> smallest	$O(\log n)$
<code>bisect</code> <code>search</code> in sorted list	$O(\log n)$
<code>s += t</code> in a loop	$O(n^2)$: use <code>''.join(parts)</code>
<code>slice a[i:j]</code>	$O(j - i)$ copy

- Amortised $O(1)$ `append`: the array over-allocates and doubles.

SYNTAX YOU MUST KNOW COLD

<code>a, *mid, z = [1, 2, 3, 4]</code>	<code># unpacking</code>
<code>a, b = b, a</code>	<code># swap</code>
<code>x[::-1]; x[1:-1]; x[::2]</code>	<code># slices</code>
<code>[f(i) for i in r if i % 2]</code>	<code># list comp</code>
<code>{k: v for k, v in pairs}</code>	<code># dict comp</code>
<code>(i * i for i in r)</code>	<code># generator</code>
<code>for i, v in enumerate(xs, 1): ...</code>	
<code>for a, b in zip(xs, ys): ...</code>	<code># shortest</code>
<code>sorted(xs, key=lambda t: (-t[1], t[0]))</code>	
<code>max(xs, key=len); any(...); all(...)</code>	
<code>y = a if cond else b</code>	
<code>if (n := len(xs)) > 3: ...</code>	<code># walrus</code>
<code>f"{x:.2f} {n:,} {s:r>8}"</code>	<code># f-string</code>

- `sorted` returns a new list; `.sort()` sorts in place and returns `None`.
- `dict.get(k, d)`, `setdefault`, `d | e` merge (3.9+), `{**a, **b}`.

METHODS WORTH REMEMBERING

- list**: `append` `extend` `insert` `remove` `pop` `index` `count` `sort` `reverse` `clear`; `xs + ys` copies; `xs * 3` repeats.
- dict**: `keys` `values` `items` `get` `pop` `popitem` `update` `setdefault`; iterate `for k, v in d.items()`.
- set**: `add` `discard` `remove` `union` `|` `intersection` `&` `difference` `-` `symmetric_difference` `^`, `issubset`; `x in s` is $O(1)$.
- Numbers**: `divmod(a, b)`, `round(x, 2)` (banker's rounding), `abs` `pow` `sum` `min` `max`, `int("12")`, `float`, `math.sqrt` `floor` `ceil` `log` `gcd` `inf`.
- `range(a, b, step)` is lazy; `list(range(5)).reversed()`, `sorted(reverse=True)`, `zip(*matrix)` transposes.

- Chained comparison: `0 < x < 10`. and/or return an operand: `x or default`.

FUNCTIONS AND SCOPE

```
def f(a, b=2, *args, c, **kw): ...
def f(x, /, y, *, z): ...      # pos-only / kw-only
lambda x: x * 2
nonlocal n                    # rebind enclosing variable
```

- LEGB** lookup: Local, Enclosing, Global, Built-in.
- Arguments are passed by object reference: mutating a passed list changes the caller's list; rebinding the name does not.

TRAP Mutable default argument: `def f(x, acc=[])` shares one list across calls. Use `acc=None` then `acc = acc or []`. **Late binding closures:** `[lambda: i for i in range(3)]` all return `2`. Bind with `lambda i=i: i`.

- Decorator = function that takes a function and returns a function; use `functools.wraps`.
- Generator = function with `yield`; lazy, one pass, constant memory. `yield` from delegates.

LOOPS AND ITERATORS

```
for x in xs:
    if ok(x): break
else: ...
it = iter(xs); next(it, None) # default: no error
next(iter(rows), None)      # peek at first item
list(islice(gen(), 10))     # first 10 of a stream
zip(a, b, strict=True)      # error if lengths differ
ks, vs = zip(*pairs)        # unzip
for blk in iter(lambda: f.read(4096), b''):
```

- A `for` loop calls `next()` until `StopIteration`. A generator is spent after one pass: `list(g)` twice gives `[]` the second time.
- Laziness saves memory, not time. Batch pattern: fill a list to size `n`, `yield` it, `reset`, then `yield` the leftover.

COPYING AND EQUALITY

- Assignment never copies. `b = a` is the same object.
- Shallow copy (`a.copy()`, `a[:]`, `list(a)`, `copy.copy`) copies the container, shares the items. `copy.deepcopy` copies recursively.
- `[[0] * 3] * 3` makes three references to one row. Use `[[0] * 3 for _ in range(3)]`.
- Strings are immutable; `s[0] = 'a'` raises `TypeError`.

OOP AND SPECIAL METHODS

- `__init__` initialises; `__new__` creates. `__repr__` for developers (unambiguous), `__str__` for users.
- Operators: `__eq__` `__lt__` `__hash__` `__len__` `__getitem__` `__iter__` `__next__` `__contains__` `__call__` `__enter__` `__exit__`.
- Defining `__eq__` without `__hash__` makes the class unhashable.
- @property computed attribute; @classmethod gets `cls` (alt constructors); @staticmethod gets nothing.
- Inheritance: `super().__init__()`; method resolution order via `C3` (`Cls.__mro__`).
- @dataclass generates `__init__`, `__repr__`, `__eq__`; `frozen=True` for immutable. `__slots__` saves memory.
- Duck typing: "if it quacks"; use ABCs or `Protocol` for contracts.

ERRORS, CONTEXT MANAGERS

```
try:
    risky()
except (KeyError, ValueError) as e:
    handle(e)
else:
    ok()                # no exception
finally:
    cleanup()           # always
with open(p) as f: ...  # closes on exit
raise ValueError("msg") from e
```

Catch specific exceptions, never bare `except:`. Custom errors: subclass `Exception`.

COLLECTIONS

```
from collections import (Counter, defaultdict,
                        deque, namedtuple)
Counter("aab").most_common(1) # [('a', 2)]
d = defaultdict(list); d[k].append(v)
q = deque(maxlen=3); q.appendleft(x)
q.popleft() # 0(1) queue
P = namedtuple("P", "x y")
```

ITERTOOLS, FUNCTOOLS, HEAPQ, BISECT

```
from itertools import (permutations, combinations,
                      product, accumulate, groupby, chain, islice)
list(combinations("abc", 2)) # ab ac bc
list(accumulate([1, 2, 3])) # [1, 3, 6]
import heapq
h = []; heapq.heappush(h, x); heapq.heappop(h)
heapq.nlargest(3, xs) # top-k
heapq.heappush(h, -x) # max-heap trick
import bisect; bisect.bisect_left(xs, x)
from functools import lru_cache, partial, reduce
@lru_cache(maxsize=None) # memoise
```

groupby needs sorted input (it groups neighbours only). heapq is a min-heap only. chain(a, b) joins iterables; product(a, b) gives every pairing (a parameter grid); islice slices any iterator.

STRINGS

- split(), ' '.join(xs), strip lstrip rstrip, lower upper title, startswith endswith, find (-1) vs index (error), replace, count, isalpha isdigit isalnum, zfill, partition.
- ord('a') = 97, chr(97). Reverse s[::-1]. Palindrome: s == s[::-1].
- Anagram: sorted(a) == sorted(b) or Counter(a) == Counter(b).
- Regex: re.findall(r"\d+", s), re.sub(r"\s+", " ", s), re.match anchors at start, re.search anywhere; groups (...), lazy .*?.

CLASSIC SNIPPETS

```
# two-sum in O(n)
seen = {}
for i, x in enumerate(nums):
    if t - x in seen: return seen[t - x], i
    seen[x] = i
# dedupe, keep order
list(dict.fromkeys(xs))
# first non-repeating char
c = Counter(s)
next((ch for ch in s if c[ch] == 1), None)
# max subarray (Kadane)
best = cur = a[0]
for x in a[1:]:
    cur = max(x, cur + x); best = max(best, cur)
# flatten one level
[y for x in xs for y in x]
# word frequency
Counter(text.lower().split())
```

FROM SCRATCH: STATS AND ML

```
import math
def mean(x): return sum(x) / len(x)
def var(x, ddof=1):
    m = mean(x)
    return sum((v - m) ** 2 for v in x) / (len(x) - ddof)
def std(x): return math.sqrt(var(x))
def sigmoid(z): return 1 / (1 + math.exp(-z))
def softmax(z):
    m = max(z); e = [math.exp(v - m) for v in z]
    return [v / sum(e) for v in e]
def dist(a, b):
    return math.sqrt(sum((p - q) ** 2 for p, q in zip(a, b)))
```

Sample variance divides by n - 1. Subtract the max in softmax for numerical stability.

FROM SCRATCH: GRADIENT DESCENT

```
# fit y = w*x + b by gradient descent
w = b = 0.0; lr = 0.01
for _ in range(1000):
    gw = gb = 0.0
    for x, y in data:
        err = (w * x + b) - y
        gw += 2 * err * x / len(data)
        gb += 2 * err / len(data)
    w -= lr * gw; b -= lr * gb
```

K-means: assign each point to the nearest centre, move each centre to its group's mean, repeat. KNN: sort by distance, majority vote of the k nearest.

FILES, JSON, DATES

```
from pathlib import Path; import json
txt = Path("a.txt").read_text()
data = json.loads(s); json.dumps(d, indent=2)
from datetime import datetime, timedelta
datetime.strptime("2025-01-31", "%Y-%m-%d")
d + timedelta(days=7); d.strftime("%b %d")
(d2 - d1).days; d.weekday() # Monday = 0
p = Path("data") / "raw" / "a.csv"
p.name, p.stem, p.suffix, p.parent
p.with_suffix(".parquet"); p.exists()
Path("out").mkdir(parents=True, exist_ok=True)
with open(a) as src, open(b, "w") as dst: ...
```

```
from contextlib import contextmanager
@contextmanager
def timer():
    t = time.perf_counter()
    try: yield # with-body runs here
    finally: print(time.perf_counter() - t)
```

Use with open(...); csv.DictReader for CSV; pd.read_csv for real work.

CONCURRENCY AND MEMORY

- GIL: one thread runs Python bytecode at a time. Threads help I/O-bound work; use multiprocessing / ProcessPoolExecutor for CPU-bound; async for many network waits.
- Memory: reference counting plus a cycle collector. Generators and iterators avoid building big lists.
- Iterator protocol: __iter__ returns self, __next__ raises StopIteration.
- Profiling: timeit, cProfile; vectorise with NumPy before optimising loops.

GOTCHAS

TRAP sum of floats: 0.1 + 0.2 ≠ 0.3; use math.isclose. Modifying a list while iterating over it skips items; iterate over a copy. Integer division // floors toward -inf: -7 // 2 = -4. % has the sign of the divisor. dict order is insertion order; set order is arbitrary. is on ints/strings; except Exception hiding bugs; = on floats.

ERROR MESSAGES -> CAUSE

Message	Usual cause
TypeError: unhashable type: 'list'	list used as dict key or set item
KeyError / IndexError	missing key / index out of range
UnboundLocalError	assigned in a function before use, needs nonlocal/global
RecursionError	no base case (default limit ~1000)
ValueError: too many values to unpack	length mismatch in unpacking
AttributeError: 'NoneType'...	a function returned None (e.g. .sort())

STYLE AND TOOLING

- PEP 8, type hints (def f(x: int) → str), docstrings, small pure functions, f-strings.
- venv + requirements.txt; pytest for tests (assert f(2) == 4); black/ruff for formatting.
- if __name__ == "__main__": guards script code.

SAY IT IN 20 SECONDS

- **list vs tuple**: mutable vs immutable, tuples are hashable and slightly faster.
- **Shallow vs deep copy**: shared inner objects vs fully independent.
- **Generator vs list**: lazy and one-pass vs materialised in memory.
- **GIL**: one bytecode thread at a time; threads for I/O, processes for CPU.
- **Decorator**: a function that wraps a function to add behaviour.
- "State complexity for every solution."

2

CHAPTER 2 · HIGHEST PRIORITY · MODULE 2 IN THE APP

SQL

SQL is the biggest single filter in data science hiring. A small set of ideas, practised until automatic, covers almost every question you will be asked.

14 topics · about 5 h 45 min in the app · 63 questions answered · two-page cheat sheet at the end

- | | |
|---|--|
| 2.1 SQL topic checklist | 2.9 SQL problems, level 4b: funnels, churn windows, sessions |
| 2.2 Logical order of execution | 2.10 SQL problems, level 5a: median, pivot, share of total |
| 2.3 ROW_NUMBER vs RANK vs DENSE_RANK | 2.11 SQL problems, level 5b: churn, NTILE, RFM, recursion |
| 2.4 Practice schema used in every problem | 2.12 Conceptual SQL questions |
| 2.5 SQL problems, level 1: basics | 2.13 Common SQL traps interviewers watch for |
| 2.6 SQL problems, level 2: ranking and top-N | 2.14 Practice platforms and self-check |
| 2.7 SQL problems, level 3: running totals, growth, LAG/LEAD | |
| 2.8 SQL problems, level 4a: streaks, retention, cohorts | |

Foundations

SQL decides more interview outcomes than any other subject, for a simple reason: every data role uses it daily, and it is easy to test objectively. The good news is that SQL interviews are not broad. They return again and again to a short list of ideas. If you hold four mental models firmly, the syntax follows.

Model 1: a query is a pipeline, and it does not run in the order you write it

You write `SELECT` first, but the database evaluates it nearly last.

You write



The database runs



SELECT runs fifth: its aliases and window functions do not exist yet when WHERE runs.

Figure 4. The logical order of execution explains most SQL error messages.

This one picture answers a whole family of questions. An alias defined in `SELECT` cannot be used in `WHERE`, because `WHERE` has already run. `WHERE` filters rows and `HAVING` filters groups, because one runs before grouping and the other after. A window function cannot appear in `WHERE`, because window functions are computed during `SELECT`; you compute it in an inner query and filter outside.

Model 2: every table has a grain

The **grain** is what one row represents: one customer, one order, one item within an order, one login event. Before you join two tables, say the grain of each out loud. If one order has three items, joining orders to items gives three rows for that order, and any sum of the order amount is now three times too big. This is called **fan-out**, it produces plausible-looking wrong numbers, and it is the most common mistake in SQL interviews. The cure is a habit: state the grain, aggregate to the grain you need before joining, and check row counts after every join.

Model 3: joins answer "which rows do I keep?"

An inner join keeps only rows that match on both sides. A left join keeps every row of the left table and fills the right side with NULLs where nothing matches. That NULL is useful: "customers who never ordered" is a left join followed by a filter for NULL on the right, known as an anti-join. One trap follows directly from the pipeline model: a condition on the right-hand table placed in `WHERE` removes the NULL rows and quietly turns your left join into an inner join. Put such conditions in the `ON` clause.

Model 4: NULL means "unknown"

NULL is not zero and not an empty string. It is the absence of a value, and any comparison with it is neither true nor false but unknown. So `NULL = NULL` is not true; you test with `IS NULL`. Aggregates skip NULLs, which is why `COUNT(*)` and `COUNT(column)` can differ. And `NOT IN` with a list that contains a NULL returns no rows at all, a classic trap whose safe replacement is `NOT EXISTS`.

Grouping versus windows

`GROUP BY` collapses many rows into one row per group. A **window function** computes across a group of rows but keeps every row. That single difference is why window functions dominate interviews: questions such as "the top three earners in each department", "each day's running total", "the change from the previous month" all need a per-row answer that depends on neighbouring rows. Learn the pattern `function() OVER (PARTITION BY group ORDER BY something)` and the three ranking functions, which differ only in how they treat ties.

Ten patterns cover the field

Missing rows (anti-join). Duplicates (group and count). Top N per group (rank in an inner query, filter outside). Running totals and moving averages (windowed sums). Change from the previous row (`LAG`). Streaks of consecutive days (date minus row number). Retention and cohorts (first date per user, then a left join). Funnels (count distinct users per step). Rows to columns (conditional sums). Hierarchies (recursive queries). When a new question arrives, your first job is to recognise which of these it is. The topics that follow take them in order of difficulty.

Topic by topic

Each topic gives the idea in plain words, the fundamentals to know, a model spoken answer and three questions to test yourself. Open the same topic in the app for worked examples, runnable code and interactive demos.

2.1 SQL topic checklist

IN PLAIN WORDS

SQL is how you ask a big spreadsheet-like database questions. You say which columns you want (SELECT), from which table (FROM), keeping only some rows (WHERE), then you can group rows into piles (GROUP BY) and summarise each pile (COUNT, SUM, AVG). A join is stapling two tables together where they share an id. The one weird rule to remember: a missing value (NULL) means "I don't know", so even "I don't know equals I don't know" is not true, it is also "I don't know".

Why it matters: SQL is the biggest single filter in data science interviews, and most mistakes come from NULLs and joins.

Takeaway: Filter, group, join, and respect NULL.

WORKED EXAMPLE, STEP BY STEP

A LEFT JOIN and COUNT, traced on real rows

Using the practice data (topic 2.4): how many orders does each of users 6, 7 and 8 have, including users with none?

```
SELECT u.user_id, COUNT(*) AS rows_seen, COUNT(o.order_id) AS orders
FROM users u
LEFT JOIN orders o ON o.user_id = u.user_id
WHERE u.user_id ≥ 6
GROUP BY u.user_id;
```

- 1. FROM users:** start with users 6, 7 and 8.
- 2. LEFT JOIN orders:** keep every user. Users with orders get one row per order; user 8 has no orders, so he gets one row with NULLs on the right.

user_id	order_id	amount
6	10	700
6	15	500
7	16	1400
8	NULL	NULL

- 3. GROUP BY user_id:** three piles: user 6 (2 rows), user 7 (1 row), user 8 (1 row).
- 4. Count two ways.** COUNT(*) counts rows: 2, 1, 1. COUNT(o.order_id) skips NULLs: 2, 1, 0.
- 5. Result:** user 6 has 2 orders, user 7 has 1, user 8 has 0, which is the right answer only with COUNT(o.order_id).

What this shows. A LEFT JOIN keeps unmatched rows by filling NULLs. `COUNT(*)` would report one order for a user who never ordered. Count a column from the right-hand table.

YOUR TURN (COVER THE SOLUTION FIRST)

1. The employees table has 9 rows; only Asha has `manager_id = NULL` (she is the top boss).

Predict: (a) `SELECT COUNT(*), COUNT(manager_id) FROM employees`; (b) how many rows `WHERE manager_id <> 1` returns. Asha (1) manages Ravi, Meera, Isha and Omar.

Solution. (a) 9 and 8: `COUNT(manager_id)` skips Asha's NULL. (b) 4 rows: Karan, Dev, Nina and Zoya. The four people managed by Asha fail the test, and Asha's own row is dropped too, because `NULL <> 1` is unknown, not true. If you expected 5, you have met the NULL rule: use `manager_id <> 1 OR manager_id IS NULL` when the NULL row should be kept.

THE FUNDAMENTALS

Fundamentals

- ☐ SELECT, WHERE, ORDER BY, LIMIT/TOP, DISTINCT
- ☐ LIKE, IN, BETWEEN, IS NULL, CASE WHEN
- ☐ Aggregates: COUNT, SUM, AVG, MIN, MAX; `COUNT(*)` vs `COUNT(col)` vs `COUNT(DISTINCT col)`
- ☐ GROUP BY, HAVING (WHERE vs HAVING)
- ☐ NULL behavior: `NULL = NULL` is unknown; aggregates ignore NULLs; `COALESCE`, `NULLIF`

Joins and set operations

- ☐ INNER, LEFT, RIGHT, FULL OUTER, CROSS, SELF
- ☐ Anti-join (NOT EXISTS / LEFT JOIN ... IS NULL) and semi-join (EXISTS)
- ☐ UNION vs UNION ALL, INTERSECT, EXCEPT/MINUS
- ☐ Join on multiple keys, non-equi joins, fan-out/duplication pitfalls

Advanced

- ☐ Subqueries (scalar, correlated, in FROM)
- ☐ CTEs, multiple CTEs, recursive CTEs (basic)
- ☐ Window functions: `ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE`, `LAG`, `LEAD`, `FIRST_VALUE`, `LAST_VALUE`, `SUM()` OVER, `AVG()` OVER, frames (`ROWS BETWEEN ...`)
- ☐ Date/time: `DATE_TRUNC`, `EXTRACT`, `DATEDIFF` / `DATE_PART`, interval arithmetic
- ☐ String functions: `SUBSTRING`, `CONCAT`, `SPLIT_PART`, `REGEXP`
- ☐ Pivoting with `CASE WHEN`, conditional aggregation
- ☐ Indexes, primary/foreign keys, normalization (1NF-3NF), ACID, OLTP vs OLAP

SAY IT IN 60 SECONDS

"SQL asks a database questions. SELECT chooses columns, FROM the table, WHERE keeps rows, GROUP BY makes piles and aggregates like COUNT, SUM and AVG summarise each pile, HAVING filters piles. Joins combine tables on a shared key: inner keeps matches, left keeps every left row and fills NULLs. The rule that trips people up is NULL: it means unknown, so NULL equals NULL is not true, aggregates skip NULLs, and NOT IN with a NULL in the list returns nothing. COUNT star counts rows, COUNT of a column counts non-NULL values. The catch: check the grain of each table before joining, or sums get inflated."

KEY TERMS SELECT · WHERE · GROUP BY · HAVING · NULL · JOIN · CTE · window functions · subquery · UNION

HANDS-ON PRACTICE Work through every bullet of the checklist on the practice schema and write one query per bullet.

COMMON MISTAKES Treating NULL as a value (`col = NULL`) · Using NOT IN with a nullable subquery · Counting rows when you need distinct users

CHECK YOURSELF

- 1. A table has values 1, 2, NULL. What do COUNT(*), COUNT(col) and AVG(col) return?**
3, 2 and 1.5: NULLs are skipped by COUNT(col) and AVG.
- 2. Why does `WHERE id NOT IN (SELECT manager_id ...)` return nothing when manager_id has a NULL?**
`id <> NULL` is unknown for every row, so no row passes. Use NOT EXISTS or filter the NULLs out.
- 3. WHERE versus HAVING?**
WHERE filters rows before grouping; HAVING filters groups after aggregation.

2.2 Logical order of execution

IN PLAIN WORDS

Think of making a fruit salad. First you pick the bowls of fruit you will use (FROM and JOIN). Then you throw away the bad fruit (WHERE). Then you sort the rest into piles (GROUP BY) and maybe throw away small piles (HAVING). Only then do you decide how to label and present each serving (SELECT), tidy up duplicates, arrange them in order (ORDER BY) and take just the first few (LIMIT). You write the recipe starting with "SELECT", but the kitchen works in the other order.

Why it matters: "Why does my alias not work here?" becomes easy once you know the kitchen's order.

Takeaway: FROM, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY, LIMIT.

WORKED EXAMPLE, STEP BY STEP**Follow one query through the pipeline**

Question: among users with at least two delivered orders before March, who spent the most?

```
SELECT user_id, COUNT(*) AS n, SUM(amount) AS spend
FROM orders
WHERE status = 'delivered' AND order_date < '2025-03-01'
GROUP BY user_id
HAVING COUNT(*) ≥ 2
ORDER BY spend DESC
LIMIT 1;
```

- 1. FROM orders:** all 16 orders.
- 2. WHERE:** keep delivered orders before March. 9 rows survive (order 4 is cancelled; orders 11 to 16 are in March).
- 3. GROUP BY user_id:** six piles.

user	orders	spend
1	3	2,900
2	2	3,400
3	1	1,200
4	1	3,000
5	1	1,800
6	1	700

- 4. HAVING COUNT(*) ≥ 2:** only users 1 and 2 remain. User 4 spent more than user 1 but has one order, so the group is removed here.
- 5. SELECT:** compute `n` and `spend` for the remaining groups.
- 6. ORDER BY spend DESC:** user 2 (3,400), then user 1 (2,900). The alias `spend` works here because ORDER BY runs after SELECT.
- 7. LIMIT 1:** user 2.

What this shows. Write the query top to bottom, but read it in this order. If you had written `WHERE spend > 1000`, it would fail: WHERE runs at step 2, before `spend` exists.

YOUR TURN (COVER THE SOLUTION FIRST)

- Why does this query raise an error, and how do you fix it?

```
SELECT user_id, SUM(amount) AS spend
FROM orders
WHERE spend > 2000
GROUP BY user_id;
```

Solution. WHERE runs before GROUP BY and before SELECT, so neither the sum nor the alias `spend` exists yet. A condition on an aggregate belongs in HAVING, which runs after grouping: `... GROUP BY user_id HAVING SUM(amount) > 2000`. (Some databases accept the alias in HAVING; repeating `SUM(amount)` works everywhere.)

THE FUNDAMENTALS

```
FROM / JOIN → WHERE → GROUP BY → HAVING → SELECT (incl. window functions)
→ DISTINCT → ORDER BY → LIMIT
```

Consequences: you cannot use a SELECT alias in WHERE (in most DBs); window functions cannot be used directly in WHERE (wrap in CTE/subquery).

SAY IT IN 60 SECONDS

"SQL is written in one order and run in another. Logically it goes FROM and joins, then WHERE, then GROUP BY, then HAVING, then SELECT, including window functions, then DISTINCT, ORDER BY and LIMIT. That explains the common errors. A SELECT alias cannot be used in WHERE because WHERE runs first. A window function cannot be in WHERE for the same reason, so I compute it in a CTE and filter outside. ORDER BY can use an alias because it runs after SELECT. WHERE filters rows early and cheaply; HAVING filters aggregated groups. The catch: this is the logical order; the optimiser may physically reorder work as long as the result is identical."

KEY TERMS logical query processing · FROM · WHERE · GROUP BY · HAVING · SELECT · window functions · ORDER BY · alias scope

BUILDS ON 2.1 SQL topic checklist

HANDS-ON PRACTICE Take any query you wrote and annotate each clause with its logical step number, then explain which names are visible at each step.

COMMON MISTAKES Writing queries in the order they are typed and assuming that is the order they run · Filtering on a window function directly · Referring to an alias in HAVING or WHERE

CHECK YOURSELF

1. How do you keep only the highest-paid person per department using ROW_NUMBER?

Compute `ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC)` in a CTE, then `WHERE rn = 1` outside, since windows cannot be filtered in WHERE.

2. Why is `ORDER BY annual` allowed but `WHERE annual > 9` not?

ORDER BY runs after SELECT created the alias; WHERE runs before.

3. When should a filter go in WHERE rather than HAVING?

Whenever it concerns raw rows, not aggregates: it shrinks the data before grouping.

2.3 ROW_NUMBER vs RANK vs DENSE_RANK

IN PLAIN WORDS

Imagine a race where two runners tie for second place. "Position in the list" (ROW_NUMBER) still gives everyone a different number: 1, 2, 3, 4. "Olympic ranking" (RANK) gives both silver medals a 2 and skips bronze, so the next runner is 4. "Dense ranking" (DENSE_RANK) gives both a 2 and the next runner gets 3, no skipping. A window function is just "look at my neighbours in this ordered list and tell me something", without squashing the list into one row.

Why it matters: Top-N per group, running totals and "compare to the previous row" are the most-asked SQL patterns, and they all use windows.

Takeaway: ROW_NUMBER never ties, RANK skips after ties, DENSE_RANK does not.

WORKED EXAMPLE, STEP BY STEP**Three ranking functions on one department**

Sales salaries in the practice data: Asha 90,000; Ravi 70,000; Meera 70,000; Karan 50,000. Rank them from highest to lowest within the department.

```
SELECT name, salary,
       ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn,
       RANK()          OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS drnk
FROM employees WHERE dept_id = 10;
```

- 1. PARTITION BY dept_id** says "rank inside each department separately". **ORDER BY salary DESC** says "highest first".
- 2. Line them up:** 90k, 70k, 70k, 50k.
- 3. Fill the table.**

name	salary	ROW_NUMBER	RANK	DENSE_RANK
Asha	90,000	1	1	1
Ravi	70,000	2	2	2
Meera	70,000	3	2	2
Karan	50,000	4	4	3

- 4. Read the difference at the tie.** ROW_NUMBER gives Ravi and Meera different numbers; which one gets 2 is arbitrary. RANK gives both 2 and then *skips* 3. DENSE_RANK gives both 2 and continues with 3.
- 5. Choose by the question.** "Exactly one row per person in order": ROW_NUMBER (add a tie-breaker such as `emp_id`). "The second-highest salary": DENSE_RANK = 2, which gives 70,000. "Competition-style positions": RANK.

What this shows. The three functions differ only at ties, so the first question to ask about any ranking problem is "what should happen to ties?"

YOUR TURN (COVER THE SOLUTION FIRST)

- The Data department has Isha 80,000, Dev 60,000 and Nina 60,000. Give ROW_NUMBER, RANK and DENSE_RANK for each, and say which rows `WHERE DENSE_RANK = 2` returns.

Solution. Isha: 1, 1, 1. Dev and Nina: ROW_NUMBER 2 and 3 (in an arbitrary order), RANK 2 and 2, DENSE_RANK 2 and 2. `DENSE_RANK = 2` returns both Dev and Nina: the "second-highest salary" is 60,000, earned by two people. With ROW_NUMBER = 2 you would get just one of them, chosen arbitrarily.

THE FUNDAMENTALS

Value	ROW_NUMBER	RANK	DENSE_RANK
100	1	1	1
100	2	1	1
90	3	3	2
80	4	4	3

SAY IT IN 60 SECONDS

"A window function computes a value for each row using nearby rows without collapsing them. The syntax is function OVER with optional PARTITION BY for groups, ORDER BY, and a frame. For ranking: ROW_NUMBER gives unique numbers even for ties, RANK gives ties the same number and leaves gaps after, DENSE_RANK gives ties the same number with no gaps. So with salaries 100, 100, 90 the ranks are 1, 1, 3 and dense ranks 1, 1, 2. LAG and LEAD look at the previous or next row, and SUM OVER with ORDER BY makes a running total. The catch: decide how ties behave, and note that LAST_VALUE needs an explicit frame because the default ends at the current row."

KEY TERMS ROW_NUMBER · RANK · DENSE_RANK · NTILE · LAG · LEAD · PARTITION BY · frame · ROWS vs RANGE

BUILDS ON 2.2 Logical order of execution

HANDS-ON PRACTICE Rank the practice employees by salary three ways and explain the differences on the tied rows.

COMMON MISTAKES Using RANK when you need exactly N rows per group · Forgetting ORDER BY inside OVER · Relying on ROW_NUMBER order for ties without a tie-breaker

CHECK YOURSELF

1. Salaries 90, 70, 70, 50. Give ROW_NUMBER, RANK, DENSE_RANK for 50.

4, 4 and 3.

2. Which function gives "the Nth highest salary" robustly, and why?

DENSE_RANK = N: ties do not hide lower values, unlike RANK (gaps) or ROW_NUMBER (arbitrary).

3. How does PARTITION BY differ from GROUP BY?

Both split into groups, but PARTITION BY keeps every row; GROUP BY collapses each group to one row.

2.4 Practice schema used in every problem

IN PLAIN WORDS

Before you can practise cooking you need a kitchen with some food in it. This page gives you a small, pretend company: employees and departments, customers who place orders, the items inside each order, the products, and a log of who logged in or clicked on what day. The data is tiny on purpose so you can work out every answer by hand, then check it by running the query.

Why it matters: Interview SQL is always "given these tables...", so knowing what each row means (its grain) is half the answer.

Takeaway: Always ask "one row equals what?" before joining two tables.

WORKED EXAMPLE, STEP BY STEP**See fan-out with your own eyes**

Orders 1 and 3 together are worth $1,500 + 2,500 = 4,000$. Order 3 contains two products. Watch what a join does.

- 1. The grain of each table.** `orders` : one row per order. `order_items` : one row per product inside an order.
- 2. Join them.**

order_id	orders.amount	product_id	line value (qty × price)
1	1,500	1	1,500
3	2,500	2	1,000
3	2,500	1	1,500

- 3. Sum the order amount after the join:** $1,500 + 2,500 + 2,500 = 6,500$. Order 3 was counted twice because it has two item rows.
- 4. Sum the line values instead:** $1,500 + 1,000 + 1,500 = 4,000$. Correct, because each line is counted once at its own grain.
- 5. Across the whole dataset** the wrong sum is 28,800 against a true total of 21,500.
- 6. The habit.** Before every join, say "one row of A is ..., one row of B is ...". If B has many rows per A row, either aggregate B first or sum only B's columns.

What this shows. Fan-out produces believable numbers that are simply too big. It is invisible unless you check the grain and the row counts.

YOUR TURN (COVER THE SOLUTION FIRST)

- Orders 11 (amount 2,200) and 14 (amount 2,600) each have two item rows. If you join `orders` to `order_items` for just these two orders, how many rows do you get, and what do `SUM(o.amount)` and `SUM(i.qty * i.price)` return?

Solution. 4 rows (two per order). `SUM(o.amount)` = $2,200 + 2,200 + 2,600 + 2,600 = 9,600$, which double-counts. `SUM(i.qty * i.price)` = $(1,500 + 700) + (1,800 + 800) = 4,800$, which equals the true total $2,200 + 2,600$. The item lines add up to the order amounts, so summing at the item grain is safe.

THE FUNDAMENTALS

```
employees(emp_id, name, dept_id, salary, manager_id, hire_date)
departments(dept_id, dept_name)
users(user_id, signup_date, city, device)
orders(order_id, user_id, order_date, amount, status)
order_items(order_id, product_id, qty, price)
products(product_id, category, product_name)
logins(user_id, login_date)
events(user_id, event_time, event_name)    -- e.g. 'view', 'cart', 'purchase'
```

SAY IT IN 60 SECONDS

"Before joining tables I ask what one row means, the grain. Orders has one row per order, order items one per product line, so joining them multiplies order rows, which inflates any sum of order amounts. On the sample data, summing order amounts after the join gives 28,800 instead of the true 21,500. So I either aggregate before joining or sum only item-level columns. I also check keys, NULLs and duplicates, like repeated login rows. The catch: wrong grain gives plausible but wrong numbers, and it is the most common SQL mistake in interviews."

HANDS-ON PRACTICE Load the sample data into any SQL engine and run the first three problems before opening the solutions.

COMMON MISTAKES Not checking the grain of a table before joining · Copying a Postgres query into MySQL or SQLite without checking functions

CHECK YOURSELF

1. Why does `SUM(orders.amount)` double-count after joining `order_items`?

Each order repeats once per item row (fan-out). Aggregate first or `sum qty * price` instead.

2. What is the grain of the `logins` table and how can it mislead counts?

One row per login event, possibly duplicated per user-day, so `COUNT(*)` overcounts activity; use `DISTINCT user_id, date`.

3. Which sanity check catches a bad join early?

Compare row counts before and after the join and check that totals still reconcile with the source.

2.5 SQL problems, level 1: basics

IN PLAIN WORDS

Level 1 is about asking clear questions of one or two tables: "who is missing?", "which groups are big enough?", "which rows are copies?". Finding people who never ordered is like comparing the class list with the list of people who handed in homework: anyone on the first list but not the second is missing. Finding duplicates is grouping identical emails into piles and looking for piles with more than one item.

Why it matters: These are warm-ups, and clean answers here set the tone for the round.

Takeaway: Missing means `LEFT JOIN + IS NULL` (or `NOT EXISTS`); duplicates mean `GROUP BY + HAVING COUNT(*) > 1`.

WORKED EXAMPLE, STEP BY STEP

Find duplicate emails, then keep one

The `customers` table has 6 rows. Which emails appear more than once, and how do you remove the extra rows?

1. Group by the thing that should be unique.

```
SELECT email, COUNT(*) AS n, MIN(id) AS keep_id
FROM customers
GROUP BY email
HAVING COUNT(*) > 1;
```

2. Trace the groups.

email	ids in the group	n	keep_id
a@x.com	1, 3	2	1
b@x.com	2, 5	2	2
c@x.com	4	1	

email	ids in the group	n	keep_id
d@x.com	6	1	

3. HAVING COUNT(*) > 1 keeps a@x.com and b@x.com. That answers "which emails are duplicated".

4. Decide which row survives. A common rule is the lowest id (the first sign-up). The rows to delete are ids 3 and 5.

5. Test the delete as a SELECT first.

```
SELECT * FROM customers c
WHERE id > (SELECT MIN(id) FROM customers c2 WHERE c2.email = c.email);
```

It returns ids 3 and 5. Only then change `SELECT *` to `DELETE`.

What this shows. "Group by the key, count, filter the groups" finds duplicates. Always preview a DELETE as a SELECT.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Write a query for "users who have never placed an order" on the practice data, and say which user it returns.

Solution.

```
SELECT u.user_id
FROM users u
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.user_id);
```

It returns user 8, the only user with no row in `orders`. The equivalent anti-join is `LEFT JOIN orders o ON o.user_id = u.user_id WHERE o.order_id IS NULL`. Avoid `NOT IN (SELECT user_id FROM orders)`: if any `user_id` in `orders` were NULL, it would return nothing.

SAY IT IN 60 SECONDS

"Level one is about missing and duplicate rows. To find customers who never ordered, I LEFT JOIN users to orders and keep rows where the order id is NULL, or use NOT EXISTS, which is safest. To find duplicates, GROUP BY the key and HAVING COUNT star greater than one. To delete duplicates keeping one, keep the minimum id per key, or number rows with ROW_NUMBER partitioned by the key and delete where the number is above one. Group filters go in HAVING, row filters in WHERE. The catch: NOT IN breaks with NULLs, and I test a DELETE as a SELECT first."

KEY TERMS anti-join · HAVING · duplicates · DELETE duplicates · NOT EXISTS · scalar subquery

BUILDS ON 2.4 Practice schema used in every problem

HANDS-ON PRACTICE Attempt each problem before revealing its answer, then run the solution on the sample data and compare.

COMMON MISTAKES NOT IN with NULLs · Deleting with a subquery on the same table in MySQL without a derived table · Filtering after the aggregate with WHERE

CHECK YOURSELF**1. Users who signed up in January 2025: write the date filter safely.**

`signup_date ≥ '2025-01-01' AND signup_date < '2025-02-01'` : a half-open range that works for timestamps too.

2. How do you list emails that appear more than once?

`SELECT email, COUNT(*) FROM customers GROUP BY email HAVING COUNT(*) > 1.`

3. Users with 2 or more delivered orders: where does each condition go?

`status = 'delivered'` in WHERE (rows), `COUNT(*) ≥ 2` in HAVING (groups).

2.6 SQL problems, level 2: ranking and top-N

IN PLAIN WORDS

Level 2 asks "who is the best in each group?". The trick is to give every row a rank inside its own group (like ranking students in each class separately) and then keep only ranks you want. The only real decision is what to do with ties: if two students tie for 2nd place, do you want both to count as "top 3"? Say your tie rule out loud; interviewers love it.

Why it matters: Top-N per group appears in almost every SQL round.

Takeaway: Rank inside each group with `PARTITION BY`, filter in an outer query, and state the tie rule.

WORKED EXAMPLE, STEP BY STEP**Top two salaries per department, traced**

Return the two highest salaries in each department, keeping ties.

1. Rank inside each department with `DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS r`.**2. The ranked rows.**

dept	name	salary	r
Sales	Asha	90,000	1
Sales	Ravi	70,000	2
Sales	Meera	70,000	2
Sales	Karan	50,000	3
Data	Isha	80,000	1
Data	Dev	60,000	2
Data	Nina	60,000	2
Ops	Zoya	95,000	1
Ops	Omar	55,000	2

3. Filter outside: window functions cannot be used in WHERE, so wrap the query: `SELECT * FROM (...) t WHERE r ≤ 2`.

- 4. Result:** 8 rows. Sales returns 3 people (the tie at 70,000), Data returns 3, Ops returns 2.
- 5. Compare with ROW_NUMBER:** exactly 2 rows per department, 6 in total, and one of the tied people is dropped at random. State which behaviour you want before writing the query.

What this shows. Top-N per group is always "rank in an inner query, filter in the outer query". The only real decision is how ties are handled.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Which employees earn more than their own manager in the practice data? Write the query and give the answer. (Zoya's manager is Omar; Isha's manager is Asha.)

Solution.

```
SELECT e.name, e.salary, m.name AS manager, m.salary AS manager_salary
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

Only Zoya: 95,000 against Omar's 55,000. Isha earns 80,000 and her manager Asha earns 90,000. This is a self-join: the same table appears twice with two aliases, one in the role of employee and one in the role of manager.

SAY IT IN 60 SECONDS

"Top-N per group is a ranking problem. I rank rows inside each group with a window function partitioned by the group and ordered by the measure, then filter on the rank in an outer query or CTE. The decision is ties. DENSE_RANK with rank at most three keeps everyone tied, so a department can return four rows. ROW_NUMBER returns exactly three but picks among ties arbitrarily unless I add a tie-breaker. RANK can skip the third place. Employees earning more than their manager is a self join. The catch: I say my tie rule out loud, and I compute revenue from order items, not from the order total after a join."

KEY TERMS DENSE_RANK · top-N per group · self join · CTE · ROW_NUMBER · tie handling

BUILDS ON 2.3 ROW_NUMBER vs RANK vs DENSE_RANK · 2.5 SQL problems, level 1: basics

HANDS-ON PRACTICE Solve P5 to P8 without looking, then change the tie rule (RANK instead of DENSE_RANK) and predict how the output changes.

COMMON MISTAKES Using LIMIT/OFFSET for Nth highest with duplicates · Using ROW_NUMBER when ties should be included · Filtering on the window function in WHERE

CHECK YOURSELF**1. Top 3 salaries per department: when does DENSE_RANK return more than 3 rows per department?**

When salaries tie at a rank, for example 90, 70, 70, 50 returns four rows because 50 has dense rank 3.

2. Latest order per user, exactly one row each: which function?

`ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY order_date DESC)` with `rn = 1` (add a tie-breaker).

3. Employees earning more than their manager: which technique?

Self join: employee `e` joined to manager `m` on `e.manager_id = m.emp_id`, then compare salaries.

2.7 SQL problems, level 3: running totals, growth, LAG/LEAD

IN PLAIN WORDS

Level 3 is about time: "how much so far?", "what is the average of the last few days?", "how much did it grow compared with last month?". A running total is a bank balance that grows with each deposit. A moving average smooths out bumpy days by looking at a few neighbours. Month-over-month growth compares this month with the previous row using LAG, which means "peek at the row just above me".

Why it matters: Growth and trend questions are everywhere in product and analytics interviews.

Takeaway: `SUM()` OVER for running totals, `AVG()` OVER with a frame for moving averages, LAG for comparisons.

WORKED EXAMPLE, STEP BY STEP**Running total and month-over-month change, traced**

Delivered revenue by month in the practice data: January 7,800; February 5,200; March 8,100.

1. First reduce to one row per month (aggregate before the window):

```
WITH m AS (
  SELECT DATE_TRUNC('month', order_date) AS month, SUM(amount) AS rev
  FROM orders WHERE status = 'delivered' GROUP BY 1)
SELECT month, rev,
  SUM(rev) OVER (ORDER BY month) AS running,
  LAG(rev) OVER (ORDER BY month) AS prev,
  100.0 * (rev - LAG(rev) OVER (ORDER BY month))
    / NULLIF(LAG(rev) OVER (ORDER BY month), 0) AS pct
FROM m;
```

2. Trace each window column.

month	rev	running	prev (LAG)	change	% change
Jan	7,800	7,800	NULL	NULL	NULL
Feb	5,200	13,000	7,800	-2,600	-33.3%

month	rev	running	prev (LAG)	change	% change
Mar	8,100	21,100	5,200	+2,900	+55.8%

3. Running total: each row adds its revenue to everything before it in month order.

4. LAG: looks one row back. January has no previous row, so it is NULL, and so is its change. That is correct, not an error.

5. Percent change: $(8,100 - 5,200) / 5,200 = 55.8\%$. The `100.0` makes the division decimal, and `NULLIF` avoids dividing by zero.

What this shows. Window functions answer "compared with before" questions while keeping one row per month. Aggregate to the grain you want first, then apply the window.

YOUR TURN (COVER THE SOLUTION FIRST)

1. User 1's orders are dated 5 January, 20 January, 10 February and 2 March. Using `order_date - LAG(order_date) OVER (PARTITION BY user_id ORDER BY order_date)`, what are the gaps in days, and what is the average gap?

Solution. First order: NULL (nothing before it). Then 15 days (5 to 20 January), 21 days (20 January to 10 February), and 20 days (10 February to 2 March). The average of the three gaps is $56 / 3 \approx 18.7$ days. `PARTITION BY user_id` makes LAG look back only within the same user; without it, the previous row could belong to someone else.

SAY IT IN 60 SECONDS

"Time-based questions use window functions over ordered rows. A running total is `SUM OVER` ordered by date; when I aggregate by day first it is `SUM of SUM OVER`. A moving average uses `AVG` with a frame such as six rows preceding to current row, but that counts rows, so missing days need a calendar table. Previous value comparisons use `LAG`, partitioned by user or category. Month-over-month growth is current minus previous, divided by previous, with `NULLIF` to avoid dividing by zero and a decimal multiplier to avoid integer division. The catch: first rows have no previous value and return NULL, and I must partition LAG per entity."

KEY TERMS running total · moving average · LAG · month-over-month growth · NULLIF · frame clause

BUILDS ON 2.6 SQL problems, level 2: ranking and top-N

HANDS-ON PRACTICE Reproduce P9 to P12 on the sample data and check each number by hand.

COMMON MISTAKES Computing a moving average over rows when the requirement is over days · Forgetting NULLIF and dividing by zero · Using integer division for percentages

CHECK YOURSELF

1. Why can a "7-day moving average" using `ROWS BETWEEN 6 PRECEDING` be wrong?

It averages 7 rows, not 7 days; days with no data are missing. Build a calendar with zero-filled days first.

2. Days between consecutive orders per user: what is the expression?

`order_date - LAG(order_date) OVER (PARTITION BY user_id ORDER BY order_date)`.

3. Revenue 8,200 then 5,200. Month-over-month growth?

$(5,200 - 8,200) / 8,200 = -36.6\%$.

2.8 SQL problems, level 4a: streaks, retention, cohorts

IN PLAIN WORDS

A streak is "days in a row". Here is the trick: number the days 1, 2, 3 for each user, and subtract that number from each date. Dates that are consecutive move together with the counter, so the subtraction gives the same answer for the whole streak; a gap makes the answer jump. That "same answer" becomes a label for the streak, so you can group by it and count. Retention just asks: of the people who joined on a day, how many came back exactly 1 day (or 7 days) later?

Why it matters: Retention and streak questions are favourites at product companies because they mirror real growth metrics.

Takeaway: Streaks: date minus row number. Retention: cohort size as the denominator, return on exactly day N.

WORKED EXAMPLE, STEP BY STEP

Find login streaks with "date minus row number"

User 1 logged in on 2, 3, 3 (twice), 4, 6 and 7 January. What are the streaks of consecutive days?

- 1. Remove duplicates first:** 2, 3, 4, 6, 7 January. A duplicate row would break the trick.
- 2. Number the days in order** with `ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date)`.
- 3. Subtract the row number from the date.** Consecutive dates rise by 1 and so does the row number, so the difference stays constant within a streak.

login_date	row number	date – row number
2 Jan	1	1 Jan
3 Jan	2	1 Jan
4 Jan	3	1 Jan
6 Jan	4	2 Jan
7 Jan	5	2 Jan

- 4. Group by that constant.** Group "1 Jan" covers 2 to 4 January: a streak of 3 days. Group "2 Jan" covers 6 to 7 January: 2 days.
- 5. Answer:** user 1's longest streak is 3 days. The gap on 5 January changed the constant, which is exactly how a break is detected.

What this shows. The constant is not a real date. It is a label that stays the same while the days are consecutive. Group by it to turn rows into streaks.

YOUR TURN (COVER THE SOLUTION FIRST)

1. User 2 logged in on 3 January and then on 5, 6, 7, 8, 9 and 10 January. Apply the same trick: what labels do you get, and what is the longest streak?

Solution. Row numbers 1 to 7. 3 Jan – 1 = 2 Jan. Then 5 Jan – 2 = 3 Jan, 6 – 3 = 3 Jan, and so on to 10 Jan – 7 = 3 Jan. Two groups: "2 Jan" (just 3 January, length 1) and "3 Jan" (5 to 10 January, length 6). The longest streak is 6 days.

SAY IT IN 60 SECONDS

"For streaks I use gaps and islands. After keeping one row per user per day, I number each user's days 1, 2, 3 and subtract that number from the date. Consecutive days produce the same result, so that value labels a streak; I group by it and use the count as the length. For retention, I define the cohort as each user's first day, then count users who return exactly one or seven days later, divided by cohort size, using a left join so non-returners stay in. A monthly cohort table is the same idea by month. The catch: duplicates break the streak arithmetic, and I name which retention definition I used."

KEY TERMS gaps and islands · streak · row number trick · day-N retention · cohort table · LEAD · DISTINCT

BUILDS ON 2.7 SQL problems, level 3: running totals, growth, LAG/LEAD

HANDS-ON PRACTICE Write the streak query and the D1/D7 query from memory on the sample data and compare with the results here.

COMMON MISTAKES Forgetting DISTINCT so duplicate logins break the streak arithmetic · Dividing by activity rows instead of cohort size · Counting a user as retained on any later day instead of exactly day N

CHECK YOURSELF

1. **Why must you de-duplicate login rows before the row-number trick?**

A duplicate day gets its own row number, shifting the subtraction and splitting a real streak.

2. **D7 retention for 2 users joined on Jan 3; one returns on Jan 10. Value and denominator?**

50%: one of the two (cohort size is the denominator, not login rows), returning exactly on day 7.

3. **Why LEFT JOIN logins to the cohort?**

To keep users who never returned; an inner join would drop them and inflate retention.

2.9 SQL problems, level 4b: funnels, churn windows, sessions

IN PLAIN WORDS

A funnel is a slide with three sections: lots of people look at a product, fewer add it to the cart, even fewer buy. You count how many different people reached each section. To make sure they did the steps in the right order, you check the time of each step. A session is one visit to an app: if someone goes quiet for more than 30 minutes, the next click starts a new visit. You mark "new visit" whenever the gap from the previous click is big, then add up the marks to number the visits.

Why it matters: Funnels and sessions are the bread and butter of product analytics SQL.

Takeaway: Count distinct users per step; enforce order with per-step first timestamps; sessionise with LAG plus a running SUM.

WORKED EXAMPLE, STEP BY STEP**A funnel, with and without step order**

The events table has 11 clicks from 5 users. How many users got to each step?

- 1. Count distinct users per step, not events.** User 4 viewed three times but is one viewer.

step	users	as % of viewers
view	5 (users 1 to 5)	100%
cart	2 (users 1, 2)	40%
purchase	2 (users 1, 5)	40%

- 2. Notice the oddity.** Purchases equal carts, yet the buyers are not the same people. User 5 viewed and then purchased without a cart event.

- 3. Ask the clarifying question.** "Should a purchase count only if a cart came first?" A strict funnel requires each step to follow the previous one.

- 4. Strict version.** Take each user's first time at each step and require view < cart < purchase. Only user 1 qualifies: viewed 10:00, cart 10:05, purchased 10:20.

- 5. Report both.** Loose funnel: 5 → 2 → 2. Strict funnel: 5 → 2 → 1. The gap tells the product team that some users buy without the cart, perhaps through a "buy now" button.

What this shows. Funnel numbers depend on definitions: users versus events, and whether order is enforced. Say your definition before giving the numbers.

YOUR TURN (COVER THE SOLUTION FIRST)

1. User 4 has events at 13:00, 13:02 and 13:40. With a new session starting after a gap of more than 30 minutes, how many sessions does user 4 have? Show the gap column and the session number.

Solution. Gaps from LAG: 13:00 → NULL (first event, so it starts session 1); 13:02 → 2 minutes (same session); 13:40 → 38 minutes, more than 30, so it starts a new one. Flag new sessions as 1, 0, 1. The running sum of the flags gives session numbers 1, 1, 2. User 4 has 2 sessions.

SAY IT IN 60 SECONDS

"A funnel counts distinct users at each step: viewers, carters, buyers, using COUNT DISTINCT with CASE WHEN. For a strict funnel I take each user's first timestamp per step and require view before cart before purchase, so someone who bought without carting does not count. For users who bought in January but not February, I use half-open date ranges and NOT EXISTS. For sessions, I use LAG to get the previous event time, flag a new session when there is none or the gap exceeds thirty minutes, and take a running SUM of the flags to number sessions. The catch: count users, not events, and enforce step order."

KEY TERMS funnel · conditional aggregation · ordered funnel · anti-join · sessionization · LAG · cumulative SUM

BUILDS ON 2.8 SQL problems, level 4a: streaks, retention, cohorts

HANDS-ON PRACTICE Take the sample events and compute both funnel versions, then sessionize user 4 by hand before running the query.

COMMON MISTAKES Counting events instead of distinct users in a funnel · Not enforcing step order · Using timestamp BETWEEN with inclusive end dates

CHECK YOURSELF**1. Why can a simple funnel and an ordered funnel differ?**

The simple version counts a user who purchased without a cart view as a buyer; the ordered version requires the steps in sequence.

2. How do you number sessions with a 30-minute gap rule?

Flag rows where the gap from `LAG(event_time)` exceeds 30 minutes or is NULL, then `SUM(flag) OVER (PARTITION BY user ORDER BY time)`.

3. Why prefer `>= start AND < next_start` to BETWEEN for dates?

BETWEEN includes only midnight of the end date for timestamps and would miss most of the last day.

2.10 SQL problems, level 5a: median, pivot, share of total**IN PLAIN WORDS**

The median is the middle person when everyone stands in a line by height, which is better than the average when one giant (or a billionaire) pulls the average up. A pivot rotates a table: instead of one row per category and month you get one column per category, like turning a list into a little spreadsheet. Percent of total asks "how big is my slice of the pie?", which needs the size of the whole pie next to each slice.

Why it matters: Reporting questions ("median delivery time", "revenue by category per month", "share of total") come up constantly.

Takeaway: Median via percentile or middle row numbers; pivot with `SUM(CASE WHEN)`; share with `SUM() OVER ()`.

WORKED EXAMPLE, STEP BY STEP**Pivot revenue by category, traced**

Turn "one row per month and category" into "one row per month, one column per category" (delivered orders, revenue from order items).

1. Start long. The grouped data has one row per month and category:

month	category	revenue
Jan	Electronics	7,000
Jan	Fashion	800
Feb	Fashion	3,900
Feb	Grocery	1,300
Mar	Electronics	4,000
Mar	Fashion	2,600
Mar	Grocery	1,500

2. Write one conditional sum per column.

```
SUM(CASE WHEN category = 'Electronics' THEN revenue ELSE 0 END) AS electronics,
SUM(CASE WHEN category = 'Fashion'      THEN revenue ELSE 0 END) AS fashion,
SUM(CASE WHEN category = 'Grocery'      THEN revenue ELSE 0 END) AS grocery
... GROUP BY month
```

3. How it works for January. The Electronics column adds 7,000 (the Electronics row) + 0 (the Fashion row) = 7,000.

4. The wide result.

month	electronics	fashion	grocery
Jan	7,000	800	0
Feb	0	3,900	1,300
Mar	4,000	2,600	1,500

5. Check the total: 7,800 + 5,200 + 8,100 = 21,100, the same as the delivered revenue by month. A pivot rearranges numbers; it must never change the total.

What this shows. `SUM(CASE WHEN ...)` is a pivot you can write in any database. With `ELSE 0`, a missing combination shows as 0 instead of NULL.

YOUR TURN (COVER THE SOLUTION FIRST)

1. The 16 order amounts sorted are: 300, 400, 500, 600, 700, 800, 900, 1,100, 1,200, 1,400, 1,500, 1,800, 2,200, 2,500, 2,600, 3,000. What is the median, and why might it be better than the mean (1,343.75) to describe a typical order?

Solution. With 16 values the median is the average of the 8th and 9th: $(1,100 + 1,200) / 2 = 1,150$. The mean is higher because a few large orders (2,500, 2,600, 3,000) pull it up; the median shows what a typical order looks like and is not moved by those few. In SQL: `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount)` in PostgreSQL, or number the rows and average the middle two elsewhere.

SAY IT IN 60 SECONDS

"The median is the middle value after sorting, and it resists outliers. In Postgres I use `PERCENTILE_CONT` of 0.5; portably I number rows, count rows, and average the middle one or two rows. A pivot turns rows into columns using conditional aggregation: `SUM of CASE WHEN category equals X then revenue else zero`, grouped by month. Percent of total divides each group's sum by the grand total using `SUM of SUM over an empty window`, with a decimal multiplier so it is not truncated. The catch: pivot columns are fixed in the query, so I mention that dynamic pivots need other tools."

KEY TERMS median · `PERCENTILE_CONT` · pivot · conditional aggregation · percent of total · window over aggregate

BUILDS ON 2.7 SQL problems, level 3: running totals, growth, LAG/LEAD

HANDS-ON PRACTICE Compute the overall median order amount by hand from the sorted values, then write the portable SQL and check.

COMMON MISTAKES Using `AVG` when the median is asked · Hard-coding pivot columns without telling the interviewer they are static · Computing percent of total with an integer division

CHECK YOURSELF**1. Sorted salaries 55k and 95k in one department. Median?**

75k, the average of the two middle values.

2. How do you write a pivot with one column per category?

`SUM(CASE WHEN category = 'Fashion' THEN revenue ELSE 0 END) AS fashion` per category, grouped by month.

3. Why `100.0 * x / y` and not `100 * x / y`?

Integer division truncates; the decimal literal forces a decimal result.

2.11 SQL problems, level 5b: churn, NTILE, RFM, recursion**IN PLAIN WORDS**

Churn candidates are customers who have gone quiet: their last visit was a long time ago. RFM scores every customer on three simple questions: how Recently did they buy, how Frequently, and how much Money did they spend? NTILE is a fair pile-maker: it sorts customers and deals them into equal-sized piles (top 20%, next 20%, and so on). A recursive query is like asking "who works for my boss?", then "who works for those people?", repeating until you reach the bottom of the company tree.

Why it matters: Marketing, CRM and org-structure questions use exactly these patterns.

Takeaway: Use an as-of date, NTILE for buckets, RFM as three NTILEs, and a recursive CTE with an anchor plus a step.

WORKED EXAMPLE, STEP BY STEP**Walk an org chart with a recursive query**

Who reports to whom, and at what level? The practice data has Asha at the top.

```
WITH RECURSIVE chain AS (
  SELECT emp_id, name, 1 AS lvl FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.emp_id, e.name, c.lvl + 1
  FROM employees e JOIN chain c ON e.manager_id = c.emp_id)
SELECT * FROM chain;
```

- 1. Anchor (runs once):** people with no manager. Result: Asha, level 1.
- 2. Round 1:** join employees whose manager is in the current result (Asha). Adds Ravi, Meera, Isha and Omar at level 2.
- 3. Round 2:** whose manager is among those four? Karan (Ravi), Dev and Nina (Isha), Zoya (Omar), at level 3.
- 4. Round 3:** whose manager is among Karan, Dev, Nina or Zoya? Nobody. An empty round ends the recursion.
- 5. Result:** 9 rows: 1 person at level 1, 4 at level 2, 4 at level 3.

What this shows. A recursive query is a loop written in SQL: start from the anchor, keep adding the next layer, and stop when a round adds nothing. If the data had a cycle, such as A managing B and B managing A, it would never stop, so production queries add a depth limit.

YOUR TURN (COVER THE SOLUTION FIRST)

1. "Churn candidates": as of 31 March 2025, which users' last delivered order is more than 30 days old? Last delivered orders: user 1 on 2 March, user 2 on 5 March, user 3 on 3 February, user 4 on 15 March, user 5 on 9 March, user 6 on 18 March, user 7 on 20 March.

Solution. The cut-off is 1 March 2025 (31 March minus 30 days). Only user 3 (3 February) is earlier, so user 3 is the churn candidate. In SQL: `SELECT user_id FROM orders WHERE status = 'delivered' GROUP BY user_id HAVING MAX(order_date) < DATE '2025-03-31' - 30`. Use a fixed as-of date rather than `CURRENT_DATE` so the result does not change each day. User 8 has never ordered, which is a different question (anti-join).

SAY IT IN 60 SECONDS

"Churn candidates are customers whose last order is older than a window, so I group by user, take the maximum order date, and compare with an as-of date, a fixed date rather than today so results are reproducible. NTILE deals ordered rows into equal buckets, but ties can straddle boundaries. RFM computes recency, frequency and monetary per customer then NTILE scores each. A recursive CTE has an anchor, such as the root employee, and a recursive step joining children to what we have, adding one to the level, until nothing new appears. 'Has done X in every month' is a group with COUNT DISTINCT equal to the number of months. The catch: guard against cycles and be explicit about the as-of date."

KEY TERMS churn · NTILE · RFM · recursive CTE · hierarchy · relational division · as-of date

BUILDS ON 2.10 SQL problems, level 5a: median, pivot, share of total

HANDS-ON PRACTICE Fix an as-of date of 2025-03-31 and recompute churn candidates, RFM scores and the org tree on the sample data.

COMMON MISTAKES Using `CURRENT_DATE` in a question that needs a fixed as-of date for reproducibility · Reading `NTILE(5)` as "top 20%" without checking ties and sizes · Forgetting the termination condition in recursion

CHECK YOURSELF

1. **Why use a fixed as-of date instead of `CURRENT_DATE`?**

Results stay reproducible and testable; `CURRENT_DATE` changes every day.

2. **Two parts of a recursive CTE?**

An anchor query (the root rows) and a recursive step that joins the table to the CTE's previous output, combined with `UNION ALL`.

3. **Find users who ordered in all 3 months of data.**

`GROUP BY user_id HAVING COUNT(DISTINCT month) = 3` (relational division).

2.12 Conceptual SQL questions

IN PLAIN WORDS

An index is like the index at the back of a textbook: instead of reading every page to find "photosynthesis", you jump straight to the right page. It makes finding things faster, but every time the book changes you must also update the index, so writing gets a little slower. A transaction is like transferring money: either both the subtraction and the addition happen, or neither does.

Why it matters: These are the short "explain it" questions that follow every SQL coding round.

Takeaway: Give a one-line definition, a cost or trade-off, and a tiny example for each.

WORKED EXAMPLE, STEP BY STEP

Why an index makes a lookup fast: count the reads

A table of 1,000,000 orders. Find the order with `order_id = 742,113`.

- 1. Without an index:** the database reads rows until it finds the match. On average half the table, 500,000 rows; if the id is missing, all 1,000,000.
- 2. With a B-tree index:** the ids are kept sorted in a tree. Each node holds about 100 keys and points to the right child for the next range.
- 3. Count the levels.** 100 keys per node gives 100 entries after one level, $100^2 = 10,000$ after two and $100^3 = 1,000,000$ after three. So **3 node reads** reach the right entry, plus 1 read to fetch the row.
- 4. Compare:** about 4 reads against about 500,000. That is why a lookup on an indexed column feels instant.
- 5. The cost.** Every insert or update must also update the index, and the index takes disk space. So index the columns you filter, join and sort on, not every column.
- 6. When the index cannot help.** `WHERE YEAR(order_date) = 2025` applies a function to every row's date before comparing, so the sorted order of the index is useless. A range on the raw column, `order_date ≥ '2025-01-01' AND order_date < '2026-01-01'`, uses it.

What this shows. An index is a sorted shortcut, like the index at the back of a book. It makes reads fast and writes slower.

YOUR TURN (COVER THE SOLUTION FIRST)

- There is an index on `orders(user_id)`. Which of these can use it, and why? (a) `WHERE user_id = 42`; (b) `WHERE user_id + 0 = 42`; (c) `WHERE user_id IN (3, 7, 9)`; (d) `WHERE CAST(user_id AS TEXT) = '42'`.

Solution. (a) Yes: a direct comparison on the indexed column. (c) Yes: three quick lookups. (b) and (d) No (in most databases): the expression changes the column before comparing, so the database cannot use the sorted index and scans every row. The rule is to keep the indexed column alone on one side of the comparison, which is called being "sargable".

SAY IT IN 60 SECONDS

"An index is a sorted structure, usually a B-tree, that makes lookups and range scans fast at the cost of slower writes and more storage. A clustered index is the table's physical order, one per table; non-clustered are separate structures pointing to rows. DELETE removes chosen rows and can be rolled back, TRUNCATE empties the table quickly, DROP removes the table. UNION removes duplicates, UNION ALL keeps them and is faster. ACID means atomic, consistent, isolated, durable: a money transfer either fully happens or not at all. OLTP is many small transactions, OLAP is large analytical scans. To optimise a slow query I read the plan, index join and filter columns, filter early, and avoid functions on indexed columns. The catch: indexes are not free."

KEY TERMS UNION ALL · index · B-tree · clustered index · CTE vs temp table · DELETE vs TRUNCATE · normalisation · ACID · OLAP · EXPLAIN

HANDS-ON PRACTICE Explain each row aloud in 30 seconds with one example, then run EXPLAIN on a query from the sample data.

COMMON MISTAKES Saying indexes are free · Confusing clustered and non-clustered indexes · Answering optimisation without mentioning the query plan

CHECK YOURSELF**1. Why can `WHERE YEAR(order_date) = 2025` be slow even with an index on `order_date`?**

The function on the column prevents index use (not sargable); use a range: `order_date ≥ '2025-01-01' AND order_date < '2026-01-01'`.

2. Explain ACID with a bank transfer.

Atomic: debit and credit both happen or neither. Consistent: total money preserved. Isolated: others never see half the transfer. Durable: once committed it survives a crash.

3. When would you pick a temp table over a CTE?

When an expensive intermediate result is reused across statements or needs an index.

2.13 Common SQL traps interviewers watch for

IN PLAIN WORDS

Most wrong SQL answers are not wrong because of difficult ideas; they are wrong because of small traps: counting the same order several times after a join, treating "unknown" as if it were a value, dividing two whole numbers and getting zero, or filtering in the wrong place. Learn the seven traps below and you will catch most mistakes before the interviewer does.

Why it matters: Interviewers deliberately set data where these traps bite, then watch whether you notice.

Takeaway: Check the grain before joining, respect NULLs, cast before dividing, and put conditions where they belong.

WORKED EXAMPLE, STEP BY STEP**The LEFT JOIN that silently became an INNER JOIN**

Goal: every user from 6 to 8, with their orders over 600 if they have any.

1. Version A puts the condition in WHERE.

```
SELECT u.user_id, o.amount FROM users u
LEFT JOIN orders o ON o.user_id = u.user_id
WHERE u.user_id ≥ 6 AND o.amount > 600;
```

2. Trace it. The join produces (6, 700), (6, 500), (7, 1,400) and (8, NULL). Now WHERE runs: 500 fails. `NULL > 600` is unknown, so user 8's row fails too. Result: (6, 700) and (7, 1,400). **User 8 has vanished.**

3. Version B moves the condition into ON.

```
SELECT u.user_id, o.amount FROM users u
LEFT JOIN orders o ON o.user_id = u.user_id AND o.amount > 600
WHERE u.user_id ≥ 6;
```

4. Trace it. The join only matches orders over 600, and the LEFT JOIN still keeps every user: (6, 700), (7, 1,400) and (8, NULL). User 8 stays, with nothing matched.

5. The rule. Conditions on the *right-hand* table of a LEFT JOIN go in ON. Conditions on the left-hand table (here `u.user_id ≥ 6`) can stay in WHERE.

What this shows. Both queries run without errors and both results look plausible. Only tracing the NULL row reveals that one of them answers a different question.

YOUR TURN (COVER THE SOLUTION FIRST)

1. 15 of the 16 orders are delivered. In PostgreSQL, what do `SELECT 100 * 15 / 16` and `SELECT 100.0 * 15 / 16` return, and which one belongs in a report?

Solution. `100 * 15 / 16` is all integers, so the division truncates: $1,500 / 16 = 93$. `100.0 * 15 / 16` is decimal: 93.75. The report needs 93.75%. The same trap appears as `COUNT(...) / COUNT(*)`, which returns 0 for any share below 100% when both counts are integers. Multiply by 1.0 or 100.0 first.

THE FUNDAMENTALS

1. Joining before aggregating, causing **row fan-out** and inflated sums.
2. Using `NOT IN` with NULLs.
3. Forgetting to `DISTINCT` users when counting unique users.
4. Filtering on the right table of a LEFT JOIN in `WHERE` (turns it into an INNER join). Put the condition in `ON` instead.
5. Integer division: `100 * a / b` with integers truncates; cast to float or multiply by `1.0`.
6. Date boundaries: use `≥ start AND < next_day` instead of `BETWEEN` for timestamps.
7. Ties in ranking: choose RANK/DENSE_RANK/ROW_NUMBER deliberately and state why.

SAY IT IN 60 SECONDS

"Most wrong SQL answers come from seven traps. Row fan-out: joining a one-to-many table before summing inflates totals. NOT IN with NULLs returns nothing. Counting rows instead of distinct users. Putting a right-table filter in WHERE after a left join turns it into an inner join; the condition belongs in ON. Integer division truncating percentages. Using BETWEEN on timestamps. And ties in ranking functions. In the sample data fan-out gave 28,800 instead of 21,500. The catch: all of these produce plausible numbers, so I sanity check row counts and totals after every join and say my assumptions aloud."

KEY TERMS fan-out · NOT IN NULL · LEFT JOIN WHERE · integer division · BETWEEN timestamps · ties · DISTINCT users

BUILDS ON 2.6 SQL problems, level 2: ranking and top-N

HANDS-ON PRACTICE For each trap, write a query that exhibits the bug and another that fixes it on the sample data.

COMMON MISTAKES Joining before aggregating · Forgetting DISTINCT when counting users · Putting right-table filters in WHERE on a LEFT JOIN

CHECK YOURSELF

1. **After** `users LEFT JOIN orders ... WHERE o.status = 'delivered'`, **users with no orders vanish. Why and fix?**

NULL = 'delivered' is unknown, so those rows are filtered, making it an inner join. Put the condition in the ON clause.

2. `SELECT 100 * 3 / 8` **returns 37 in Postgres. Why?**

Integer division truncates. Use `100.0 * 3 / 8` for 37.5.

3. **"Customers who ordered in March": which two mistakes does a join to order_items plus COUNT(*) make?**

It fans out to item rows and counts rows rather than distinct customers; use COUNT(DISTINCT user_id) on orders alone.

2.14 Practice platforms and self-check

IN PLAIN WORDS

SQL interview questions come from about ten patterns. Missing rows: LEFT JOIN with IS NULL. Best per group: rank inside each group. Running totals: SUM over ordered rows. Compared to last time: LAG. Streaks: date minus row number. Retention and funnels: count distinct users per step. Pivot and share of total: conditional sums and window totals. Org trees: recursive queries. Learn these and new questions look like old friends. In the interview, say what each row means, plan in steps, and test on two or three rows by hand.

Why it matters: SQL is the biggest single filter, and calm structure impresses as much as the query.

Takeaway: Ten patterns, a step-by-step script, and a hand check on a tiny example.

WORKED EXAMPLE, STEP BY STEP**From question to query in five moves**

Question: *"For each city, which user has the highest total delivered spend?"*

- 1. Output grain:** one row per city, with the top user and their spend.
- 2. Inputs and grain:** `users` (one row per user, has `city`) and `orders` (one row per order, has `amount` and `status`). The join is user to orders, one to many, which is fine because we sum order amounts at order grain.
- 3. Plan in steps aloud:** (a) total delivered spend per user, with their city; (b) rank users within each city; (c) keep rank 1.

```
WITH spend AS (
  SELECT u.city, u.user_id, SUM(o.amount) AS total
  FROM users u JOIN orders o ON o.user_id = u.user_id
  WHERE o.status = 'delivered'
  GROUP BY u.city, u.user_id),
ranked AS (
  SELECT *, RANK() OVER (PARTITION BY city ORDER BY total DESC) AS r
  FROM spend)
SELECT city, user_id, total FROM ranked WHERE r = 1;
```

- 4. Check by hand.** Bengaluru: user 4 has 5,600 against user 1's 5,100, so user 4. Mumbai: user 2 (3,700) beats user 7 (1,400). Pune: only user 5 (2,900). Delhi: users 3 and 6 both have 1,200, a tie.
- 5. Decide the tie, and say so.** RANK returns both Delhi users (5 rows in total). If the business wants one per city, use ROW_NUMBER with a tie-breaker, for example the earlier sign-up.

What this shows. Grain, plan, write, check by hand, decide the ties. The hand check found the tie, which a query that merely runs would never reveal.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Name the pattern and outline the steps (no full SQL needed) for: *"For each month, what share of orders came from customers who had ordered before?"*

Solution. Pattern: first-order flag, then conditional aggregation by month. Steps: (1) for each order, find the customer's first order date, with `MIN(order_date) OVER (PARTITION BY user_id)` or a CTE; (2) flag the order as repeat if its date is after that first date; (3) group by month and compute `AVG(CASE WHEN is_repeat THEN 1.0 ELSE 0 END)`; (4) check one month by hand and decide whether cancelled orders count. State the definition of "repeat" before writing anything.

THE FUNDAMENTALS**2.8 Practice platforms and targets**

Platform	Target
LeetCode SQL 50 (study plan)	Complete all
DataLemur	40+ problems incl. hard
StrataScratch	30+ from Indian product companies (Amazon, Uber, etc.)
HackerRank SQL	Advanced certificate

Platform	Target
Mode SQL tutorial	Window functions section

2.9 Self-check (Module 2)

- ☐ Can you write the retention query (P14) and streak query (P13) from scratch?
- ☐ Can you explain why a LEFT JOIN with a WHERE on the right table becomes an INNER JOIN?
- ☐ Can you solve a medium problem in 15 minutes and a hard problem in 25?

SAY IT IN 60 SECONDS

"For a live SQL round I restate the question and the output grain, such as one row per user. I inspect the tables for keys, NULLs, duplicates and date types, and ask about ties and cancelled orders. I plan in steps aloud: a CTE for this, rank that, filter. I write it incrementally, then check on two or three rows by hand and confirm no fan-out by comparing row counts. Finally I mention performance, like indexes and filtering early. The patterns I rely on are anti-join, rank per group, running totals, LAG, gaps and islands, cohort and funnel counts, pivots and recursion. The catch: never write the whole query before agreeing on the definition."

KEY TERMS timed SQL practice · SQL interview script · pattern list · self-check

HANDS-ON PRACTICE Run a 60-minute timed mock: two medium problems and one hard problem, using a fresh dataset.

COMMON MISTAKES Writing the full query before clarifying the grain and the output · Not testing with a tiny example · Practising only easy problems

CHECK YOURSELF

1. The interviewer says "write the retention rate". What do you ask first?

Which definition (day-N, rolling, monthly), cohort basis, what counts as active, denominator, window and edge cases such as duplicates and test accounts.

2. Name the pattern: "users who never bought in the last 90 days but bought before".

Anti-join on a recent window (NOT EXISTS) combined with a prior-purchase condition, relative to an as-of date.

3. How do you validate your query is right?

Compute a tiny case by hand, check row counts and totals reconcile, and test edge cases like ties and NULLs.

Interview tips, tricks and hacks

- 1. Say the output grain before writing anything.** "One row per user per month, with their order count." It prevents half of all wrong answers and shows the interviewer you think like an analyst.
- 2. Ask about ties, NULLs and duplicates.** "If two people share the third-highest salary, do you want both?" The question itself earns marks, and the answer decides between `ROW_NUMBER`, `RANK` and `DENSE_RANK`.
- 3. Build with CTEs, one step each.** Name each step (`first_orders`, `ranked`) and narrate it. If you run out of time, a clearly correct first two steps score better than a tangled full query.

4. **Check row counts after every join.** Say it aloud: "orders has one row per order, items has several, so this join multiplies rows and I must not sum the order total here."
5. **Use half-open date ranges.** `d ≥ '2025-01-01' AND d < '2025-02-01'` is correct for dates and timestamps. `BETWEEN` on a timestamp silently drops most of the last day.
6. **Multiply by 1.0 for percentages.** Integer division truncates in many databases: `3 / 8` is 0. Write `100.0 * a / b` and wrap the divisor in `NULLIF(b, 0)`.
7. **Count users, not rows.** In retention and funnel questions the answer is almost always `COUNT(DISTINCT user_id)`.
8. **Test on three rows by hand.** Pick a tiny example, compute the answer in your head, and trace your query against it. This catches off-by-one windows and wrong join types.
9. **Finish with one sentence on performance.** "On a large table I would filter early and make sure the join keys are indexed." You rarely need more than that at entry level.
10. **Memorise the ten patterns, not a hundred queries.** When you see a new question, name the pattern first. It turns a blank page into a template.

Frequently asked questions

63 questions interviewers ask on this chapter, with model answers. Cover the answer, say yours aloud, then compare.

SQL TOPIC CHECKLIST

Q1 A table `t(a)` contains the values 1, 2, NULL. What do `SELECT COUNT(*) FROM t`, `COUNT(a)`, `AVG(a)` and `SELECT * FROM t WHERE a < 1` return?

`COUNT(*)` = 3 rows; `COUNT(a)` = 2 (NULL skipped); `AVG(a)` = $(1 + 2) / 2 = 1.5$ (NULL ignored, not treated as 0); `WHERE a < 1` returns only 2, because `NULL < 1` is unknown and unknown rows are filtered out. To include the NULL row, write `WHERE a < 1 OR a IS NULL`.

LOGICAL ORDER OF EXECUTION

Q2 Why is `ORDER BY annual` allowed when `WHERE annual > ...` is not?

Logical order: `FROM`, `WHERE`, `GROUP BY`, `HAVING`, `SELECT` (aliases and window functions created), `DISTINCT`, `ORDER BY`, `LIMIT`. The alias is defined in the `SELECT` step, so any clause that runs after it (`ORDER BY`, `LIMIT`) may use it; clauses before it (`WHERE`, `GROUP BY`, `HAVING` in most engines) may not. This is a logical order: the optimiser may physically reorder work as long as the result is identical.

ROW_NUMBER VS RANK VS DENSE_RANK

Q3 Write a query that returns the second-highest salary in each department, correctly handling ties for first place.

```
WITH r AS (
  SELECT dept_id, name, salary,
         DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rk
  FROM employees
)
SELECT dept_id, name, salary FROM r WHERE rk = 2;
```

If two people tie for the top salary, `DENSE_RANK` still gives the next distinct salary rank 2; `RANK` would skip to 3 and `ROW_NUMBER` would wrongly treat the second tied person as "second". Decide and state the tie semantics the interviewer wants.

PRACTICE SCHEMA USED IN EVERY PROBLEM**Q4 Before joining `orders` to `order_items`, what must you check, and what goes wrong if you skip it?**

`orders` has one row per order; `order_items` has one row per product line. A join produces one row per item, so `orders.amount` repeats once per item. On the sample data, `SUM(o.amount)` after the join returns 28,800 instead of the true 21,500. The fix is to aggregate each table to the level you need *before* joining (or sum only item-level columns such as `qty * price`).

SQL PROBLEMS, LEVEL 1: BASICS**Q5 Customers who never placed an order.**

```
SELECT u.user_id
FROM users u
LEFT JOIN orders o ON o.user_id = u.user_id
WHERE o.order_id IS NULL;

-- or
SELECT user_id FROM users u
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.user_id);
```

Pitfall: `NOT IN` returns nothing if the subquery contains NULL.

Q6 Department-wise average salary, only departments with avg > 50k.

```
SELECT d.dept_name, AVG(e.salary) AS avg_sal
FROM employees e JOIN departments d USING (dept_id)
GROUP BY d.dept_name
HAVING AVG(e.salary) > 50000;
```

Q7 Find duplicate emails / duplicate rows.

```
SELECT email, COUNT(*) FROM customers GROUP BY email HAVING COUNT(*) > 1;
```

Q8 Delete duplicates keeping the lowest id.

```
DELETE FROM customers
WHERE id NOT IN (SELECT MIN(id) FROM customers GROUP BY email);
-- MySQL requires wrapping the subquery in a derived table.
-- Window version (Postgres):
DELETE FROM customers WHERE id IN (
  SELECT id FROM (
    SELECT id, ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) rn FROM customers
  ) t WHERE rn > 1);
```

Q9 Users and order count and total spend for delivered orders, only for users with at least 2 delivered orders.

```
SELECT user_id, COUNT(*) AS n_orders, SUM(amount) AS total, ROUND(AVG(amount), 1) AS avg_amt
FROM orders
WHERE status = 'delivered'           -- row filter first
GROUP BY user_id
HAVING COUNT(*) ≥ 2;                 -- then the group filter
```

user_id	n_orders	total	avg_amt
1	4	5100	1275.0
2	3	3700	1233.3
4	2	5600	2800.0
5	2	2900	1450.0
6	2	1200	600.0

User 3 is excluded because one of their two orders was cancelled.

Q10 List users who signed up in January 2025.

```
SELECT user_id, signup_date
FROM users
WHERE signup_date ≥ '2025-01-01' AND signup_date < '2025-02-01';
```

Returns users 1, 2, 3 and 4. Prefer `≥ start AND < next_start` over `BETWEEN`: it is correct for timestamps too (`BETWEEN '2025-01-01' AND '2025-01-31'` would miss anything on the 31st after midnight).

Q11 Which orders are above the overall average order amount?

```
SELECT order_id, amount FROM orders
WHERE amount > (SELECT AVG(amount) FROM orders)  -- average = 1,343.75
ORDER BY amount DESC;
```

Orders 6, 14, 3, 11, 9, 1 and 16 qualify. The subquery runs once and returns a single value.

SQL PROBLEMS, LEVEL 2: RANKING AND TOP-N**Q12 Nth highest salary (overall).**

```
SELECT DISTINCT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) r FROM employees
) t WHERE r = 2;  -- change 2 to N
```

Q13 Top 3 earners in each department.

```
WITH ranked AS (
  SELECT e.*, DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) r
  FROM employees e
)
SELECT * FROM ranked WHERE r ≤ 3;
```

Q14 Employees earning more than their manager.

```
SELECT e.name
FROM employees e JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

Q15 Top product category by revenue per month.

```
WITH monthly AS (
  SELECT DATE_TRUNC('month', o.order_date) AS mth, p.category,
         SUM(oi.qty * oi.price) AS rev
  FROM orders o
  JOIN order_items oi USING (order_id)
  JOIN products p USING (product_id)
  GROUP BY 1, 2
), ranked AS (
  SELECT *, RANK() OVER (PARTITION BY mth ORDER BY rev DESC) r FROM monthly
)
SELECT mth, category, rev FROM ranked WHERE r = 1;
```

Q16 Return each user's most recent order (exactly one row per user).

```
SELECT user_id, order_id, order_date
FROM (
  SELECT o.*, ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY order_date DESC) AS rn
  FROM orders o
) t
WHERE rn = 1;
```

Here `ROW_NUMBER` is right because you want exactly one row. If two orders could share a timestamp, add a tie-breaker: `ORDER BY order_date DESC, order_id DESC`.

Q17 Top 2 products by revenue within each category.

```
WITH rev AS (
  SELECT p.category, p.product_name, SUM(oi.qty * oi.price) AS revenue
  FROM order_items oi JOIN products p USING (product_id)
  GROUP BY p.category, p.product_name
), ranked AS (
  SELECT *, DENSE_RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS rk FROM rev
)
SELECT * FROM ranked WHERE rk ≤ 2;
```

category	product_name	revenue	rk
Electronics	Earbuds	8900	1
Electronics	Power bank	2100	2
Fashion	Sneakers	4800	1
Fashion	T-shirt	2500	2
Grocery	Basmati rice	1800	1
Grocery	Tea	1400	2

Q18 List employees who earn more than the average salary of their own department.

```
SELECT name, dept_id, salary
FROM (SELECT e.*, AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg FROM employees e) t
WHERE salary > dept_avg;
```

Returns Asha (Sales), Isha (Data) and Zoya (Ops). The window average is available on every row without a join, which is why this pattern beats a `JOIN` to a grouped subquery for readability.

SQL PROBLEMS, LEVEL 3: RUNNING TOTALS, GROWTH, LAG/LEAD**Q19 Running total of daily revenue.**

```
SELECT order_date,
       SUM(amount) AS daily_rev,
       SUM(SUM(amount)) OVER (ORDER BY order_date) AS running_total
FROM orders
GROUP BY order_date;
```

Q20 7-day moving average.

```
SELECT order_date, daily_rev,
       AVG(daily_rev) OVER (ORDER BY order_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS
ma7
FROM (SELECT order_date, SUM(amount) AS daily_rev FROM orders GROUP BY order_date) t;
```

Q21 Month-over-month growth %.

```
WITH m AS (
  SELECT DATE_TRUNC('month', order_date) mth, SUM(amount) rev
  FROM orders GROUP BY 1
)
SELECT mth, rev,
       LAG(rev) OVER (ORDER BY mth) AS prev_rev,
       ROUND(100.0 * (rev - LAG(rev) OVER (ORDER BY mth)) / NULLIF(LAG(rev) OVER (ORDER BY
mth),0), 2) AS mom_pct
FROM m;
```

Q22 Days between a user's consecutive orders.

```
SELECT user_id, order_date,
       order_date - LAG(order_date) OVER (PARTITION BY user_id ORDER BY order_date) AS gap_days
FROM orders;
```

Q23 For each user, show the gap in days between their first and second order.

```

WITH o AS (
  SELECT user_id, order_date,
         ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY order_date) AS rn,
         LAG(order_date) OVER (PARTITION BY user_id ORDER BY order_date) AS prev
  FROM orders
)
SELECT user_id, prev AS first_order, order_date AS second_order,
       order_date - prev AS days_to_second      -- Postgres; use DATEDIFF / julianday
       elsewhere
FROM o WHERE rn = 2;

```

user_id	first_order	second_order	days_to_second
1	2025-01-05	2025-01-20	15
2	2025-01-06	2025-02-14	39
3	2025-01-15	2025-02-03	19
4	2025-01-12	2025-03-15	62
5	2025-02-05	2025-03-09	32
6	2025-02-20	2025-03-18	26

Users with only one order (user 7) simply do not appear. This measure (time to repeat purchase) is a common retention metric.

Q24 Cumulative spend per user over time, restarting for each user.

```

SELECT user_id, order_date, amount,
       SUM(amount) OVER (PARTITION BY user_id ORDER BY order_date) AS cumulative
FROM orders;

```

For user 1 the cumulative column reads 1500, 2300, 2900, 5100. Without `PARTITION BY` the total would run across all users.

Q25 What share of its month's revenue does each order contribute?

```

SELECT order_id, amount,
       ROUND(100.0 * amount / SUM(amount) OVER (PARTITION BY DATE_TRUNC('month', order_date)),
       1) AS pct_of_month
FROM orders;

```

For January (total 8,200): order 6 contributes 36.6%, order 3 contributes 30.5%, order 1 contributes 18.3%, and so on. No join or subquery is needed because the window supplies the group total on every row.

SQL PROBLEMS, LEVEL 4A: STREAKS, RETENTION, COHORTS**Q26 Users with logins on 3 or more consecutive days (gaps-and-islands).**

```

WITH d AS (
  SELECT DISTINCT user_id, login_date FROM logins
), grp AS (
  SELECT user_id, login_date,
         login_date - (ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date))::int AS g
  FROM d
)
SELECT user_id, MIN(login_date) AS streak_start, MAX(login_date) AS streak_end, COUNT(*) AS
streak_len
FROM grp
GROUP BY user_id, g
HAVING COUNT(*) ≥ 3;

```

Idea: subtracting row number from a consecutive date gives a constant per streak.

Alternative using LEAD:

```

SELECT DISTINCT user_id FROM (
  SELECT user_id, login_date,
         LEAD(login_date, 1) OVER (PARTITION BY user_id ORDER BY login_date) d1,
         LEAD(login_date, 2) OVER (PARTITION BY user_id ORDER BY login_date) d2
  FROM (SELECT DISTINCT user_id, login_date FROM logins) x
) t WHERE d1 = login_date + 1 AND d2 = login_date + 2;

```

Q27 Day-N retention (D1, D7) by signup cohort.

```

WITH first_day AS (
  SELECT user_id, MIN(login_date) AS d0 FROM logins GROUP BY user_id
)
SELECT f.d0 AS cohort_date,
       COUNT(DISTINCT f.user_id) AS cohort_size,
       COUNT(DISTINCT CASE WHEN l.login_date = f.d0 + 1 THEN l.user_id END) AS d1_users,
       ROUND(100.0 * COUNT(DISTINCT CASE WHEN l.login_date = f.d0 + 1 THEN l.user_id END)
             / COUNT(DISTINCT f.user_id), 2) AS d1_retention_pct,
       COUNT(DISTINCT CASE WHEN l.login_date = f.d0 + 7 THEN l.user_id END) AS d7_users,
       ROUND(100.0 * COUNT(DISTINCT CASE WHEN l.login_date = f.d0 + 7 THEN l.user_id END)
             / COUNT(DISTINCT f.user_id), 2) AS d7_retention_pct
FROM first_day f
LEFT JOIN logins l ON l.user_id = f.user_id
GROUP BY f.d0
ORDER BY f.d0;

```

Q28 Monthly cohort retention table.

```

WITH cohort AS (
  SELECT user_id, DATE_TRUNC('month', MIN(order_date)) AS cohort_month
  FROM orders GROUP BY user_id
), activity AS (
  SELECT DISTINCT user_id, DATE_TRUNC('month', order_date) AS act_month FROM orders
)
SELECT c.cohort_month,
       (EXTRACT(YEAR FROM a.act_month) - EXTRACT(YEAR FROM c.cohort_month)) * 12
       + (EXTRACT(MONTH FROM a.act_month) - EXTRACT(MONTH FROM c.cohort_month)) AS
month_number,
       COUNT(DISTINCT a.user_id) AS active_users
FROM cohort c JOIN activity a USING (user_id)
GROUP BY 1, 2
ORDER BY 1, 2;

```

Q29 Find each user's longest login streak.

```
WITH d AS (SELECT DISTINCT user_id, login_date FROM logins),
grp AS (
  SELECT user_id, login_date,
         login_date - (ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date))::int AS g
  FROM d
),
streaks AS (SELECT user_id, g, COUNT(*) AS len FROM grp GROUP BY user_id, g)
SELECT user_id, MAX(len) AS longest_streak FROM streaks GROUP BY user_id;
```

user_id	longest_streak
1	3
2	6
3	2
4	4
5	1

Two levels of grouping: first by island, then by user.

Q30 For each month, how many buyers are new (first ever order that month) and how many are returning?

```
WITH first_m AS (
  SELECT user_id, DATE_TRUNC('month', MIN(order_date)) AS fm FROM orders GROUP BY user_id
)
SELECT DATE_TRUNC('month', o.order_date) AS month,
       COUNT(DISTINCT CASE WHEN f.fm = DATE_TRUNC('month', o.order_date) THEN o.user_id END)
AS new_buyers,
       COUNT(DISTINCT CASE WHEN f.fm < DATE_TRUNC('month', o.order_date) THEN o.user_id END)
AS returning_buyers
FROM orders o JOIN first_m f USING (user_id)
GROUP BY 1 ORDER BY 1;
```

month	new_buyers	returning_buyers
2025-01	4	0
2025-02	2	3
2025-03	1	5

Conditional `COUNT(DISTINCT CASE WHEN ...)` is the workhorse of product analytics SQL.

SQL PROBLEMS, LEVEL 4B: FUNNELS, CHURN WINDOWS, SESSIONS**Q31 Funnel: view → cart → purchase conversion.**

```
SELECT
  COUNT(DISTINCT CASE WHEN event_name = 'view' THEN user_id END) AS viewers,
  COUNT(DISTINCT CASE WHEN event_name = 'cart' THEN user_id END) AS carters,
  COUNT(DISTINCT CASE WHEN event_name = 'purchase' THEN user_id END) AS buyers
FROM events;
```

Stricter version (ordered funnel: user must have done the steps in sequence) uses self-joins or `MIN(CASE WHEN ... THEN event_time END)` per step and checks ordering.

Q32 Users who bought in Jan but not in Feb.

```
SELECT DISTINCT user_id FROM orders
WHERE order_date ≥ '2025-01-01' AND order_date < '2025-02-01'
AND user_id NOT IN (
  SELECT user_id FROM orders WHERE order_date ≥ '2025-02-01' AND order_date < '2025-03-01');
```

Q33 Sessionization: new session if gap > 30 min.

```
WITH flagged AS (
  SELECT user_id, event_time,
         CASE WHEN event_time - LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time)
              > INTERVAL '30 minutes'
              OR LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time) IS NULL
              THEN 1 ELSE 0 END AS new_sess
  FROM events
)
SELECT *, SUM(new_sess) OVER (PARTITION BY user_id ORDER BY event_time) AS session_id
FROM flagged;
```

Q34 Show the funnel split by device.

```
SELECT u.device,
       COUNT(DISTINCT CASE WHEN e.event_name = 'view'      THEN e.user_id END) AS viewers,
       COUNT(DISTINCT CASE WHEN e.event_name = 'purchase' THEN e.user_id END) AS buyers
FROM events e JOIN users u USING (user_id)
GROUP BY u.device;
```

device	viewers	buyers
android	2	1
ios	2	1
web	1	0

Segmenting a funnel by device, country or app version is how you localise a conversion drop. Remember small counts make percentages noisy.

Q35 How many sessions does each user have, using a 30-minute inactivity rule?

Reuse the flag step and aggregate:

```
WITH flagged AS (
  SELECT user_id, event_time,
         CASE WHEN LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time) IS NULL
              OR event_time - LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time)
              > INTERVAL '30 minutes'
              THEN 1 ELSE 0 END AS new_sess
  FROM events
)
SELECT user_id, SUM(new_sess) AS sessions FROM flagged GROUP BY user_id;
```

Only user 4 has two sessions; the others have one. You can now compute sessions per user per day, average session length (MAX - MIN of time per session) and events per session.

SQL PROBLEMS, LEVEL 5A: MEDIAN, PIVOT, SHARE OF TOTAL**Q36 Median salary per department (Postgres).**

```
SELECT dept_id, PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) AS median_sal
FROM employees GROUP BY dept_id;
-- Portable: use ROW_NUMBER and COUNT window, average the middle one or two rows.
```

Q37 Pivot: revenue by category per month as columns.

```
SELECT DATE_TRUNC('month', o.order_date) mth,
       SUM(CASE WHEN p.category = 'Electronics' THEN oi.qty*oi.price ELSE 0 END) AS
electronics,
       SUM(CASE WHEN p.category = 'Fashion'      THEN oi.qty*oi.price ELSE 0 END) AS fashion
FROM orders o JOIN order_items oi USING(order_id) JOIN products p USING(product_id)
GROUP BY 1;
```

Q38 Percentage of total (share of revenue by category).

```
SELECT category, SUM(rev) AS rev,
       ROUND(100.0 * SUM(rev) / SUM(SUM(rev)) OVER (), 2) AS pct_share
FROM cat_revenue GROUP BY category;
```

Q39 What is the median order amount across all orders?

```
WITH r AS (
  SELECT amount, ROW_NUMBER() OVER (ORDER BY amount) AS rn, COUNT(*) OVER () AS n FROM orders
)
SELECT AVG(amount) AS median_amount FROM r WHERE rn IN ((n + 1) / 2, (n + 2) / 2);
```

The 16 orders sorted are 300, 400, 500, 600, 700, 800, 900, 1100, 1200, 1400, ...; the 8th and 9th are 1,100 and 1,200, so the median is **1,150**, below the mean of 1,343.75 (a right-skewed distribution: a few big orders pull the mean up).

Q40 Give each category's share of revenue within each month.

```
WITH m AS (
  SELECT DATE_TRUNC('month', o.order_date) AS month, p.category, SUM(oi.qty * oi.price) AS rev
  FROM orders o JOIN order_items oi USING (order_id) JOIN products p USING (product_id)
  GROUP BY 1, 2
)
SELECT month, category, rev,
       ROUND(100.0 * rev / SUM(rev) OVER (PARTITION BY month), 1) AS pct_of_month
FROM m;
```

For January that gives Electronics 85.4%, Fashion 9.8%, Grocery 4.9% (of 8,200). The `PARTITION BY month` makes the "whole pie" each month's own total.

SQL PROBLEMS, LEVEL 5B: CHURN, NTILE, RFM, RECURSION**Q41 Customers whose latest order was > 90 days ago (churn candidates).**

```
SELECT user_id, MAX(order_date) AS last_order
FROM orders GROUP BY user_id
HAVING MAX(order_date) < CURRENT_DATE - INTERVAL '90 days';
```

Q42 Top 20% customers by spend (NTILE).

```
WITH s AS (SELECT user_id, SUM(amount) spend FROM orders GROUP BY user_id),
     b AS (SELECT *, NTILE(5) OVER (ORDER BY spend DESC) AS quintile FROM s)
SELECT * FROM b WHERE quintile = 1;
```

Q43 RFM segmentation (recency, frequency, monetary).

```
WITH rfm AS (
  SELECT user_id,
         CURRENT_DATE - MAX(order_date) AS recency,
         COUNT(*) AS frequency,
         SUM(amount) AS monetary
  FROM orders GROUP BY user_id
)
SELECT *,
       NTILE(4) OVER (ORDER BY recency ASC) AS r_score, -- low recency days = better
       NTILE(4) OVER (ORDER BY frequency DESC) AS f_score,
       NTILE(4) OVER (ORDER BY monetary DESC) AS m_score
FROM rfm;
```

Q44 Recursive CTE: employee hierarchy.

```
WITH RECURSIVE org AS (
  SELECT emp_id, name, manager_id, 1 AS level FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.emp_id, e.name, e.manager_id, o.level + 1
  FROM employees e JOIN org o ON e.manager_id = o.emp_id
)
SELECT * FROM org;
```

Q45 Show each employee's management chain as text (for example `Asha > Omar > Zoya`).

```
WITH RECURSIVE org AS (
  SELECT emp_id, name, 1 AS level, name AS path FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.emp_id, e.name, o.level + 1, o.path || ' > ' || e.name
  FROM employees e JOIN org o ON e.manager_id = o.emp_id
)
SELECT name, level, path FROM org ORDER BY path;
```

Rows include `Isha | 2 | Asha > Isha`, `Dev | 3 | Asha > Isha > Dev` and `Zoya | 3 | Asha > Omar > Zoya`. Use `CONCAT` in MySQL. The same carry-a-value idea computes subtree totals or depth.

Q46 Find users who ordered in every month of the data (January, February and March).

```
SELECT user_id
FROM orders
GROUP BY user_id
HAVING COUNT(DISTINCT DATE_TRUNC('month', order_date)) = 3;
```

Users 1 and 2. The pattern "has all of the required items" is called relational division; the number on the right comes from `(SELECT COUNT(DISTINCT month) FROM orders)` when it is not fixed.

CONCEPTUAL SQL QUESTIONS**Q47 WHERE vs HAVING?**

WHERE filters rows before grouping; HAVING filters groups after aggregation.

Q48 UNION vs UNION ALL?

UNION removes duplicates (extra sort/hash cost); UNION ALL keeps all rows.

Q49 INNER vs LEFT join?

INNER returns matches only; LEFT keeps all left rows, NULLs for non-matches.

Q50 RANK vs DENSE_RANK?

RANK leaves gaps after ties; DENSE_RANK does not.

Q51 CTE vs subquery vs temp table?

CTE is readable/reusable in one statement; temp table is materialized, can be indexed, persists in session.

Q52 What is an index?

A data structure (usually B-tree) that speeds reads at the cost of writes and storage.

Q53 Clustered vs non-clustered?

Clustered defines physical row order (one per table); non-clustered is a separate structure pointing to rows.

Q54 Primary vs unique key?

Primary: unique + not null, one per table. Unique: allows one NULL (DB dependent), many per table.

Q55 DELETE vs TRUNCATE vs DROP?

DELETE removes rows (filterable, logged, rollback); TRUNCATE removes all rows fast, resets identity; DROP removes the table.

Q56 Normalization?

Organizing tables to reduce redundancy: 1NF atomic values, 2NF no partial dependency, 3NF no transitive dependency.

Q57 OLTP vs OLAP?

OLTP: many small transactions, normalized. OLAP: analytical queries over large data, denormalized (star schema).

Q58 How to optimize a slow query?

Check the EXPLAIN plan, add indexes on join/filter columns, avoid `SELECT *`, filter early, avoid functions on indexed columns, reduce joins fan-out, use partitioning.

Q59 COUNT(*) vs COUNT(col)?

COUNT(*) counts rows; COUNT(col) counts non-NULL values.

Q60 What is a self join?

Joining a table to itself, e.g., employee-manager.

Q61 What are ACID properties?

Atomicity, Consistency, Isolation, Durability.

COMMON SQL TRAPS INTERVIEWERS WATCH FOR

Q62 "Count the customers who ordered in March." A candidate writes `SELECT COUNT(*) FROM orders o JOIN order_items oi USING (order_id) WHERE order_date ≥ '2025-03-01'`. What is wrong?

Two problems: (1) the join to `order_items` fans out the rows, and (2) `COUNT(*)` counts rows rather than distinct customers. The question only needs `orders`: `SELECT COUNT(DISTINCT user_id) FROM orders WHERE order_date ≥ '2025-03-01' AND order_date < '2025-04-01'`, which returns 6 for March (users 1, 2, 4, 5, 6, 7). Also clarify whether cancelled orders count.

PRACTICE PLATFORMS AND SELF-CHECK

Q63 The interviewer says "write a query for the retention rate" with no other detail. What do you ask before writing SQL?

Clarify: (1) **Definition**: day-N retention (exactly day N), rolling retention, or monthly? (2) **Cohort**: by signup date, first purchase, or first app open? (3) **Activity**: what counts as "active" (any login, a purchase)? (4) **Denominator**: all users in the cohort? (5) **Time window and granularity**: which dates, day/week/month? (6) **Edge cases**: duplicates, test accounts, time zones. Then restate the definition, agree it, and write the query. Asking these questions is itself part of the signal.

LOGICAL EXECUTION ORDER

- FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT (aliases, window functions) → DISTINCT → ORDER BY → LIMIT/OFFSET
- SELECT aliases are **not** visible in WHERE/HAVING (they are in ORDER BY).
 - Window functions cannot be in WHERE: compute in a CTE or subquery, filter outside.
 - WHERE filters rows before grouping; HAVING filters groups after aggregation.
 - This is the logical order; the optimiser may execute differently but never changes the result.

QUERY TEMPLATE

```
WITH x AS (SELECT ... FROM t WHERE ...)
SELECT a, COUNT(*) AS n, AVG(v) AS m
FROM x JOIN y ON x.id = y.id
WHERE d ≥ '2025-01-01' AND d < '2025-02-01'
GROUP BY a
HAVING COUNT(*) > 5
ORDER BY n DESC
LIMIT 10;
```

Dates: use half-open ranges ≥ start AND < next, never BETWEEN on timestamps (it misses most of the last day).

JOINS

Join	Keeps
INNER	matches only
LEFT	all left rows; NULLs on the right if no match
RIGHT / FULL	all right / all rows of both
CROSS	every pair (m × n rows)
SELF	table joined to itself (manager, previous row)

- Anti-join** (no match): LEFT JOIN ... WHERE r.id IS NULL, or NOT EXISTS.
- Semi-join** (has a match, no duplication): EXISTS.
- Right-table filter in WHERE after a LEFT JOIN turns it into an inner join: put it in ON.
- Duplicate keys on either side multiply rows (**fan-out**). Check the grain of both tables first.
- USING (id) merges equal-named key columns; NATURAL JOIN is risky, avoid.

NULL RULES

- NULL = NULL is unknown, not true. Use IS NULL / IS NOT NULL.
- COUNT(*) counts rows; COUNT(col) skips NULLs; COUNT(DISTINCT col) unique non-NULLs.
- SUM/AVG/MIN/MAX ignore NULLs: AVG of 1, 2, NULL is 1.5.
- NULL + 1 is NULL. COALESCE(x, 0) = first non-NULL. NULLIF(a, 0) = NULL if equal (safe divide).
- ORDER BY NULL placement differs by engine; use NULLS FIRST/LAST (PG).
- NULL in WHERE a <> 1 is filtered out: NULL rows vanish silently.
- TRAP** NOT IN (subquery) returns nothing when the subquery contains a NULL. Use NOT EXISTS.

AGGREGATION AND SETS

- Aggregates: COUNT SUM AVG MIN MAX, STRING_AGG / GROUP_CONCAT, PERCENTILE_CONT(.5) WITHIN GROUP (ORDER BY x).
- Every non-aggregated SELECT column must be in GROUP BY.
- Conditional counts: SUM(CASE WHEN s = 'x' THEN 1 ELSE 0 END) or COUNT(*) FILTER (WHERE s = 'x') (PG).
- GROUP BY multiple columns; ROLLUP/CUBE for subtotals; GROUPING SETS.
- UNION stacks and removes duplicates; UNION ALL keeps them (faster); INTERSECT, EXCEPT. Same column count and types.
- IN (list) vs EXISTS: EXISTS stops at the first match and is NULL-safe.
- Subquery types: scalar (one value), row/table, correlated (refers to the outer row, runs per row).

- CTE (WITH) = named, readable subquery. Recursive CTE = anchor UNION ALL recursive step.

WINDOW FUNCTIONS

```
fn(x) OVER (
  PARTITION BY g
  ORDER BY d
  ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)
```

- Keeps every row (unlike GROUP BY). PARTITION BY = groups; ORDER BY = order within the group.
- Default frame when ORDER BY is present: RANGE UNBOUNDED PRECEDING ... CURRENT ROW. LAST_VALUE needs ... UNBOUNDED FOLLOWING.
- Frames count **rows**, so missing days need a calendar table.

Function	For 90, 70, 70, 50
ROW_NUMBER	1, 2, 3, 4 (ties arbitrary)
RANK	1, 2, 2, 4 (gaps)
DENSE_RANK	1, 2, 2, 3 (no gaps)
NTILE(4)	quartile bucket 1-4

- LAG(x, n) / LEAD(x, n): n rows back / forward. FIRST_VALUE, NTH_VALUE. SUM() OVER (ORDER BY d) = running total. PERCENT_RANK, CUME_DIST for percentiles.
- Window functions run after GROUP BY: SUM(SUM(x)) OVER () is valid.

HANDY FUNCTIONS

- Strings: LOWER TRIM SUBSTR LENGTH CONCAT / ||, LIKE 'a%', REPLACE, SPLIT_PART, REGEXP_REPLACE, POSITION.
- Dates (PG): DATE_TRUNC('month', d), EXTRACT(dow FROM d), d + INTERVAL '7 day', AGE(a, b), CURRENT_DATE.
- MySQL: DATE_FORMAT, DATEDIFF(a, b), DATE_ADD. BigQuery: DATE_DIFF, TIMESTAMP_TRUNC. SQL Server: DATEDIFF(day, a, b), TOP n.
- Cast: CAST(x AS INT), x::numeric (PG). Integer division truncates: write 100.0 * a / b.
- CASE WHEN ... THEN ... ELSE ... END works in SELECT, ORDER BY and inside aggregates.
- Limit rows: LIMIT n OFFSET k (PG/MySQL), FETCH FIRST n ROWS, TOP n.
- DISTINCT ON (k) (PG): first row per k by ORDER BY.

DATE RECIPES (POSTGRESQL)

- Month start: DATE_TRUNC('month', d); next month: + INTERVAL '1 month'.
- Last 7 days: d ≥ CURRENT_DATE - 7.
- Days between: b - a (dates) or EXTRACT(EPOCH FROM b - a) / 86400.
- Weekday (0 = Sunday): EXTRACT(dow FROM d); ISO week: EXTRACT(week FROM d).
- Hour bucket: DATE_TRUNC('hour', ts).
- Time zones: store UTC, convert with ts AT TIME ZONE 'Asia/Kolkata'.
- Calendar table: generate_series(d1, d2, '1 day') fills missing days before a moving average.

INTERVIEW QUESTION -> TOOL

Asked	Reach for
Nth highest, top N per group	DENSE_RANK / ROW_NUMBER in a CTE
Never did X / missing rows	anti-join, NOT EXISTS
Duplicates	GROUP BY ... HAVING COUNT(*) > 1
Change vs previous	LAG
Running or moving total	SUM/AVG OVER + frame
Consecutive days	date - ROW_NUMBER
Retention, cohorts	first-date CTE + LEFT JOIN
Funnel	COUNT(DISTINCT CASE ...)
Rows to columns	SUM(CASE ...) pivot
Hierarchy, org chart	recursive CTE
Median	PERCENTILE_CONT or row-number trick

TOP-N PER GROUP, NTH HIGHEST

```
SELECT * FROM (
  SELECT *, DENSE_RANK() OVER (
    PARTITION BY dept ORDER BY sal DESC) r
  FROM emp) t
WHERE r ≤ 3;
-- second highest without windows
SELECT MAX(sal) FROM emp
WHERE sal < (SELECT MAX(sal) FROM emp);
```

Say the tie rule: DENSE_RANK keeps all ties, ROW_NUMBER returns exactly N (add a tie-breaker). Employees paid more than their manager: self-join e.boss = m.id.

DEDUPE, ANTI-JOIN, DUPLICATES

```
-- latest row per user
SELECT * FROM (SELECT *, ROW_NUMBER() OVER
  (PARTITION BY user_id ORDER BY ts DESC) rn
  FROM ev) t WHERE rn = 1;
-- emails that appear twice or more
SELECT email, COUNT(*) FROM u
GROUP BY email HAVING COUNT(*) > 1;
-- users who never ordered
SELECT u.* FROM u WHERE NOT EXISTS
  (SELECT 1 FROM o WHERE o.uid = u.id);
```

Test a DELETE as a SELECT first.

RUNNING TOTAL, MOVING AVERAGE, GROWTH

```
SUM(rev) OVER (ORDER BY d)
AVG(rev) OVER (ORDER BY d ROWS 6 PRECEDING)
rev - LAG(rev) OVER (ORDER BY m)
100.0 * (rev - LAG(rev) OVER (ORDER BY m))
  / NULLIF(LAG(rev) OVER (ORDER BY m), 0)
d - LAG(d) OVER (PARTITION BY uid ORDER BY d)
```

Lines: running total, 7-row average, change, % growth, days between orders. Aggregate to one row per day first, then window. Partition LAG by entity; the first row has NULL.

GAPS AND ISLANDS (STREAKS)

```
-- one row per user per day first (DISTINCT)
SELECT uid, MIN(d) AS start, COUNT(*) AS len
FROM (SELECT uid, d, d - (ROW_NUMBER() OVER
  (PARTITION BY uid ORDER BY d))::int AS grp
  FROM days) t
GROUP BY uid, grp
HAVING COUNT(*) ≥ 3;
```

Consecutive days minus a running counter give a constant: the streak id (PG: date - int is a date). Duplicate days break it.

RETENTION AND COHORTS

```
WITH c AS (SELECT uid, MIN(d) AS d0
  FROM logins GROUP BY uid)
SELECT c.d0, COUNT(DISTINCT c.uid) AS n,
  COUNT(DISTINCT l.uid) * 1.0
  / COUNT(DISTINCT c.uid) AS d7
FROM c LEFT JOIN logins l
  ON l.uid = c.uid AND l.d = c.d0 + 7
GROUP BY c.d0;
```

Cohort = first activity date. Denominator = cohort size, not login rows. LEFT JOIN keeps non-returns. Monthly cohorts: truncate dates to month and join on month offset.

FUNNEL AND SESSIONS

```
SELECT
  COUNT(DISTINCT uid) FILTER (WHERE e='view') v,
  COUNT(DISTINCT uid) FILTER (WHERE e='cart') c,
  COUNT(DISTINCT uid) FILTER (WHERE e='buy') b
FROM ev;
-- flag a new session after a 30 minute gap
WITH g AS (SELECT *, ts - LAG(ts) OVER
  (PARTITION BY uid ORDER BY ts) AS gap FROM ev)
SELECT *, SUM(CASE WHEN gap IS NULL
  OR gap > INTERVAL '30 min' THEN 1 ELSE 0 END)
  OVER (PARTITION BY uid ORDER BY ts) AS sess
FROM g;
```

Count users, not events (portable form: COUNT(DISTINCT CASE WHEN e='buy' THEN uid END)). A strict funnel needs step order: first timestamps must increase view < cart < buy.

PIVOT, SHARE OF TOTAL, MEDIAN

```
SELECT m,
  SUM(CASE WHEN cat='A' THEN rev ELSE 0 END) a,
  SUM(CASE WHEN cat='B' THEN rev ELSE 0 END) b
FROM s GROUP BY m;
100.0 * SUM(rev) / SUM(SUM(rev)) OVER ()
PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY x)
```

Lines: pivot, percent of total, median (PG). Portable median: number rows, take the middle one or average the two middle ones. Pivot columns are fixed in the query.

RECURSIVE CTE, "IN EVERY MONTH"

```
WITH RECURSIVE t AS (
  SELECT id, boss, 1 AS lvl
  FROM emp WHERE boss IS NULL
  UNION ALL
  SELECT e.id, e.boss, t.lvl + 1
  FROM emp e JOIN t ON e.boss = t.id)
SELECT * FROM t;
```

"Did X in all 3 months": GROUP BY uid HAVING COUNT(DISTINCT month) = 3. Churn candidates: last order older than a fixed as-of date. Guard against cycles.

TRAPS CHECKLIST

- **Fan-out**: summing an order-level column after joining items inflates totals. Aggregate before joining.
- **Right-table filter** in WHERE after LEFT JOIN drops unmatched rows. Use ON.
- **Integer division** truncates; BETWEEN on timestamps; COUNT(*) where you need COUNT(DISTINCT user).
- **Ties** in ranking; NOT IN with NULLs; LAST_VALUE default frame; GROUP BY column missing.
- Sanity check after every join: row count and totals reconcile with the source.

SPEED, INDEXES, CONCEPTS

- **Index** = sorted structure (B-tree) for lookups, ranges and joins; costs write speed and space. Composite index: leftmost column first.
- **Sargable**: no function on an indexed column (WHERE YEAR(d) = 2025 is bad, a date range is good). Select only needed columns, filter early, read EXPLAIN.
- DELETE removes rows (rollback ok); TRUNCATE empties fast; DROP removes the table.
- **ACID**: Atomic, Consistent, Isolated, Durable. **OLTP** many small writes; **OLAP** big scans (columnar warehouse).
- Keys: primary (unique, not null), foreign (references). Normalisation cuts redundancy; analytics often uses a star schema.
- Isolation anomalies: dirty read, non-repeatable read, phantom read.
- Clustered index = physical order (one per table); non-clustered = separate pointer structure.
- Large joins: filter and aggregate first; temp table if an expensive result is reused.

LIVE-ROUND ROUTINE

1. Restate the question and the **output grain** (one row per what?).
2. Inspect keys, NULLs, duplicates, date types; ask about ties and cancelled rows.
3. Plan in steps aloud: CTE 1, CTE 2, final select.
4. Write incrementally; check on 2-3 rows by hand.
5. Reconcile row counts after joins; mention indexes and performance.

SAY IT IN 20 SECONDS

- "Before joining, I check the grain of each table."
- "Top N per group: rank in a CTE, filter outside."
- "Retention: cohort by first day, divide by cohort size."
- "WHERE filters rows, HAVING filters groups."
- "I test on two or three rows by hand."

3

CHAPTER 3 · HIGH PRIORITY · MODULE 3 IN THE APP

NumPy, pandas, Visualization

NumPy and pandas are how you actually touch data in Python. The goal is fluent manipulation and exploration, and the ability to translate any SQL idea into pandas and back.

7 topics · about 3 h in the app · 30 questions answered · two-page cheat sheet at the end

3.1 NumPy essentials and broadcasting

3.2 pandas checklist

3.3 SQL to pandas translation table

3.4 Worked pandas problems

3.5 Conceptual pandas questions

3.6 Visualization: choosing the right chart

3.7 The EDA template and self-check

Foundations

If SQL is how you ask a database for data, pandas is how you work with it once it is in your hands. Interviews in this area are practical: reshape this table, compute that rate per group, find and fix the problems in this dataset. Three ideas make the library feel small.

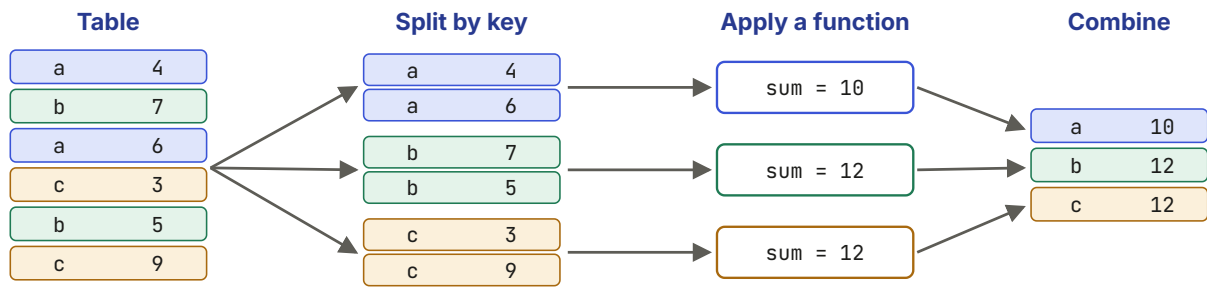
Idea 1: think in whole columns

NumPy stores numbers in one typed block of memory and applies an operation to the entire block at once, in fast compiled code. This is called **vectorisation**. `prices * 1.1` multiplies a million prices without a Python loop. The habit to build is to ask "what operation on the whole column gives me this?" instead of "how do I loop over the rows?". Code written this way is shorter, faster by orders of magnitude, and what interviewers expect.

Broadcasting extends the idea to arrays of different shapes. Line up the shapes from the right; two dimensions are compatible if they are equal or one of them is 1, and the 1 is stretched to fit. So a table with four columns minus a row of four column means works, giving each column its own centre. Subtracting row means needs the means as a column, which is what `keepdims=True` provides.

Idea 2: split, apply, combine

Most analysis questions have the same shape: for each group, compute something.



`GROUP BY` in SQL and `groupby().agg()` in pandas are the same three steps.

Figure 5. The pattern behind every "per customer", "per month" or "per category" question.

In pandas this is `groupby`. Two verbs follow it. `agg` shrinks each group to a single row, like `GROUP BY` in SQL. `transform` computes a group value and hands it back to every original row, like a window function, which is how you add "the department's average salary" next to each employee. Knowing which of the two you need is most of the skill.

Idea 3: a DataFrame is a table with an index

A **Series** is one labelled column and a **DataFrame** is a set of Series that share an index. You select by label with `loc` and by position with `iloc`. You filter with a column of true and false values (a boolean mask), combining conditions with `&` and `|` inside brackets. You join tables with `merge`, stack them with `concat`, and reshape between wide and long with `pivot_table` and `melt`. Every one of these has a direct SQL equivalent, and translating between the two is a favourite interview exercise: `WHERE` is a mask, `JOIN` is `merge`, `LAG` is `groupby().shift()`, `ROW_NUMBER` is `groupby().cumcount()`.

Exploration is a routine, not an art

Exploratory data analysis sounds open-ended, but good analysts run the same checks in the same order: shape and types, missing values, duplicates and impossible values, the distribution of each variable, the relationship of each feature with the target, then segments and time trends. For anything strange, ask whether it is a data error, a real effect or a change in how the data was recorded. Two disciplines matter in interviews. First, a correlation found during exploration is a lead, not a conclusion. Second, any decision about cleaning or features must be made using the training data only, or information leaks from the test set into your model.

Charts are arguments

A chart exists to make one point. Choose it from the question: bars to compare categories, a line for change over time, a histogram or box plot for a distribution, a scatter plot for a relationship between two numbers. Start bar charts at zero, label axes with units, and put the conclusion in the title. If the audience needs a minute to decode the chart, it has failed, however clever it is.

Topic by topic

Each topic gives the idea in plain words, the fundamentals to know, a model spoken answer and three questions to test yourself. Open the same topic in the app for worked examples, runnable code and interactive demos.

3.1 NumPy essentials and broadcasting

IN PLAIN WORDS

A NumPy array is a neat grid of numbers stored in one block of memory, like a spreadsheet that does maths on the whole grid at once. Instead of telling Python "take the first number, add 1, then the second..." you just say "add 1 to everything" and it happens in fast compiled code. Broadcasting is the polite rule that lets a small grid stretch to match a bigger one: adding a row of three numbers to a table with three columns adds it to every row.

Why it matters: Everything in pandas, scikit-learn and deep learning sits on NumPy; vectorised thinking is expected in interviews.

Takeaway: Think in whole arrays, know which axis collapses, and remember broadcasting compares shapes from the right.

WORKED EXAMPLE, STEP BY STEP

Broadcasting, traced on a small score table

Three students (rows) and three subjects (columns):

	Maths	Physics	Chemistry
Asha	60	70	80
Ravi	80	90	70
Meera	70	80	90

```
S = np.array([[60, 70, 80], [80, 90, 70], [70, 80, 90]])
```

- 1. Column means** (`axis=0` collapses the rows, leaving one number per subject):
`S.mean(axis=0) = [70, 80, 80]`. Shape (3,).
- 2. Subtract them.** `S - S.mean(axis=0)` lines up shapes from the right: (3, 3) against (3,).
 The row of three means is copied down every row, so each column is centred on its own subject: `[[-10, -10, 0], [10, 10, -10], [0, 0, 10]]`. Ravi is 10 above the class in Maths.
- 3. Row means** (`axis=1` collapses the columns): Asha 70, Ravi 80, Meera 80, shape (3,).
- 4. The trap.** `S - S.mean(axis=1)` *also* runs, because the shape (3,) still lines up with the last axis. But it subtracts Asha's mean from the Maths column, Ravi's from Physics and so on. Wrong, silently.
- 5. The fix.** `S.mean(axis=1, keepdims=True)` has shape (3, 1): one number per row, standing as a column. Broadcasting stretches it across: `[[-10, 0, 10], [0, 10, -10], [-10, 0, 10]]`. Each student is now compared with their own average.

What this shows. Read shapes from the right; a 1 stretches. With a square table, the wrong axis does not even raise an error, so always ask "one number per row, or per column?"

YOUR TURN (COVER THE SOLUTION FIRST)

1. `X` has shape (4, 3). What happens with (a) `X - X.mean(axis=0)`, (b) `X - X.mean(axis=1)`, (c) `X - X.mean(axis=1, keepdims=True)`? Give the shape of the mean in each case.

Solution. (a) The mean has shape (3,), which matches the last axis of (4, 3), so it works: each column is centred. (b) The mean has shape (4,), which does not match 3: broadcasting raises a `ValueError`. (c) The mean has shape (4, 1), which stretches across the 3 columns, so each row is centred. Unlike the square example, a non-square shape makes the mistake in (b) fail loudly.

2. Without a loop: (a) from the 5×5 array `g = np.arange(1, 26).reshape(5, 5)`, extract the inner 3×3 block and the main diagonal; (b) in `arr = np.array([3, -1, 4, -1, 5, -9])`, replace every negative number with the mean of the non-negative ones.

Solution.

```
g = np.arange(1, 26).reshape(5, 5)
g[1:4, 1:4]          # [[ 7  8  9] [12 13 14] [17 18 19]]  rows 1 to 3, columns 1
                    # to 3
np.diag(g)           # [ 1  7 13 19 25]

arr = np.array([3, -1, 4, -1, 5, -9])
np.where(arr < 0, arr[arr ≥ 0].mean(), arr)  # [3. 4. 4. 4. 5. 4.]
```

`arr[arr ≥ 0]` is `[3, 4, 5]`, whose mean is 4.0. `np.where(condition, a, b)` then picks 4.0 where the value is negative and the original value elsewhere. The result is a float array because the mean is a float.

THE FUNDAMENTALS

- ☐ `ndarray` creation: `array`, `zeros`, `ones`, `arange`, `linspace`, `eye`, `random`
- ☐ Shape, dtype, `reshape`, `ravel`, `flatten`, `transpose`, `newaxis`
- ☐ Indexing, slicing, boolean masking, fancy indexing
- ☐ **Broadcasting rules** (dimensions compared from the right; each pair must be equal or one must be 1)
- ☐ Vectorization vs Python loops (and why it is faster)
- ☐ Axis operations: `sum(axis=0)` collapses rows (column-wise result), `axis=1` collapses columns
- ☐ `np.where`, `np.argmax`, `np.argsort`, `np.unique`, `np.percentile`, `np.corrcoef`
- ☐ Linear algebra: `@`, `dot`, `np.linalg.inv`, `eig`, `norm`

```
import numpy as np
X = np.array([[1, 2, 3], [4, 5, 6]])
X.mean(axis=0)          # [2.5, 3.5, 4.5] per column
X - X.mean(axis=0)      # broadcasting: center columns
X[X > 2]                # boolean mask
np.where(X > 3, 1, 0)    # conditional
```

More on creating and reshaping arrays

Random numbers, reproducibly. Create one generator with a seed and draw everything from it. The same seed gives the same numbers on every run, which is what makes an experiment repeatable.

```
rng = np.random.default_rng(seed=42)
# [ 0.305 -1.04  0.75 ] draws from N(0, 1)
rng.standard_normal(3)
rng.uniform(10, 20, size=3)           # floats in [10, 20)
# whole numbers in [0, 10): the upper end is excluded
rng.integers(0, 10, size=5)
# sample from a list (p=[...] for unequal chances)
rng.choice(["red", "green", "blue"], size=4)

x = np.arange(10)
rng.permutation(x)                    # a shuffled COPY; x itself is unchanged
rng.shuffle(x)                        # shuffles x IN PLACE and returns None
```

How much memory an array takes is simply elements × bytes per element, which is why the dtype matters:

```
m = np.arange(12, dtype=np.float64).reshape(3, 4)
m.size, m.itemsize, m.nbytes          # (12, 8, 96) 12 numbers x 8 bytes
# 48 half the memory, about 7 significant digits
m.astype(np.float32).nbytes
np.full((2, 2), 7.5)                  # a constant-filled array
```

Prefer `linspace` to `arange` for decimal steps: `np.arange(0, 1, 0.25)` excludes the stop, and with steps like 0.1 floating-point error can add or drop the last point.

Reshaping: which calls share memory. `ravel` returns a view when it can, `flatten` always copies. Editing a view edits the original.

```
m = np.arange(6).reshape(2, 3)
r, f = m.ravel(), m.flatten()
r[0] = 100                          # m[0, 0] is now 100: r is a view of m
# m[0, 1] is still 1: f is an independent copy
f[1] = 555
# (True, False) ask NumPy when in doubt
np.shares_memory(m, r), np.shares_memory(m, f)

v = np.array([1, 2, 3])              # shape (3,)
# (1, 3) a row; v[:, np.newaxis] gives the (3, 1) column
np.expand_dims(v, axis=0).shape
np.squeeze(v[:, np.newaxis]).shape   # (3,) drop the axes of length 1
# (3, 2, 4) reorder the axes explicitly
np.zeros((2, 3, 4)).transpose(1, 0, 2).shape
```

Joining and splitting arrays.


```

A = np.array([[1, 2], [3, 4]]); B = np.array([[5, 6], [7, 8]])
# (4, 2)  stack as extra rows:  concatenate(..., axis=0)
np.vstack([A, B]).shape
# [[1 2 5 6] [3 4 7 8]]  side by side:  concatenate(..., axis=1)
np.hstack([A, B])
# [[1 4] [2 5] [3 6]]  1-D arrays become COLUMNS (a feature matrix)
np.column_stack([[1, 2, 3], [4, 5, 6]])
# (2, 2, 2)  stack creates a NEW axis; the others reuse an existing one
np.stack([A, B]).shape

big = np.arange(12).reshape(4, 3)
# [(2, 3), (2, 3)]  two equal parts along axis 0
[p.shape for p in np.split(big, 2)]
# [(1, 3), (2, 3), (1, 3)]  cut BEFORE rows 1 and 3
[p.shape for p in np.split(big, [1, 3])]

```

Indexing a marks table

A slightly larger example makes the patterns easier to see. `exam` holds the marks of 6 students (rows) in 4 subjects (columns), and the sections below reuse it:

```

exam = np.array([[78, 85, 62, 90],
                 [55, 70, 88, 64],
                 [92, 95, 89, 97],
                 [41, 58, 63, 49],
                 [67, 72, 80, 75],
                 [88, 79, 91, 84]])

# [[85 62] [70 88] [95 89]]  rows 0 to 2, columns 1 and 2
exam[0:3, 1:3]
# [78 92 67]  every second row, first column
exam[::2, 0]
exam[:, -1]  # [90 64 97 49 75 84]  the last column
exam[[1, 3]]  # rows 1 and 3 (fancy indexing: a copy)
# [62 64 63 67]  combine masks with & | ~ and PARENTHESES (not `and` / `or`)
exam[(exam > 60) & (exam < 70)]

capped = exam.copy()
capped[capped < 50] = 50  # assign THROUGH a mask: every mark below 50 becomes 50
capped.min()  # 50

```

To get **where** a condition holds instead of the values:

```

cond = exam > 94
# (array([2, 2]), array([1, 3]))  row indices and column indices, separately
np.where(cond)
# [[2 1] [2 3]]  one [row, column] pair per hit: easier to read
np.argwhere(cond)

```

`np.where(cond)` with one argument returns positions (the same as `np.nonzero`); with three arguments, `np.where(cond, a, b)`, it is the vectorised if/else shown above.

Aggregations along an axis, and missing values

On the `exam` matrix the two directions have plain meanings: `axis=0` gives one number per **subject**, `axis=1` one number per **student**.


```
# 1812                                no axis: the whole array
exam.sum()
exam.sum(axis=0)                      # [421 459 473 459]          per subject
exam.sum(axis=1)                      # [315 277 373 211 294 342]      per student
# (6, 1) keep the collapsed axis, so the result broadcasts against exam
exam.sum(axis=1, keepdims=True).shape
exam.sum(axis=1).argmax()              # 2          the student with the highest total
exam.mean(axis=0).argmin()            # 0          the subject with the lowest average

np.median(exam, axis=0)                # [72.5 75.5 84.  79.5]
# weighted mean per student: [78.1 67.  92.8 51.3 71.9 85.5]
np.average(exam, axis=1, weights=[0.4, 0.3, 0.2, 0.1])
np.cumsum(exam[0])                    # [ 78 163 225 315]      running total
# [51 37 29 48]          range = max - min ("peak to peak")
np.ptp(exam, axis=0)
# [63.75 88.25]          quantile takes 0 to 1, percentile takes 0 to 100
np.quantile(exam, [0.25, 0.75])
```

NUMPY'S `std` AND PANDAS' `std` USE DIFFERENT DEFAULTS

`np.std` divides by `n` (the **population** formula, `ddof=0`); `pandas' .std()` divides by `n - 1` (the **sample** formula, `ddof=1`). On the first subject, `exam.std(axis=0)` gives 18.01 but `exam.std(axis=0, ddof=1)` gives 19.73. When your number differs slightly from a colleague's, check `ddof` first, and pass it explicitly when it matters.

Missing values. NumPy stores a missing float as `np.nan`, and any arithmetic that touches a NaN returns NaN. The `nan*` functions skip them:

```
d = np.array([4.0, np.nan, 7.0, 1.0, np.nan])
np.mean(d), np.sum(d)                  # (nan, nan)      one NaN poisons the result
np.nanmean(d), np.nansum(d)           # (4.0, 12.0)     ignore the NaNs
np.nanmedian(d), np.nanmax(d)         # (4.0, 7.0)
np.isnan(d).sum()                     # 2              count the missing values
# False                               NaN is not equal to anything, even itself: test with np.isnan
np.nan == np.nan
```

Element-wise functions, branching and sorting

Element-wise maths functions ("universal functions") apply to every element at once:

```
x = np.array([1.0, 4.0, 9.0, 16.0])
np.sqrt(x)                            # [1.  2.  3.  4.]
np.log(x), np.exp([0, 1])             # natural log; e to the power
# log(1 + x): safe at x = 0 and accurate for tiny x; the usual transform for skewed counts
np.log1p(x)
# [0 0 5 6] force values into a range (capping outliers, bounding probabilities)
np.clip([-5, 0, 5, 10], 0, 6)
```

Branching, testing and membership without a loop:

```
v = np.array([-3, 0, 5, -1, 8])
np.where(v > 0, "pos", "non-pos")      # two-way choice
# [-1  0  1 -1  1] many-way choice: the first true condition wins
np.select([v < 0, v == 0, v > 0], [-1, 0, 1])

a = np.array([1.0, np.nan, np.inf, -2.0])
# [ True False False  True] neither NaN nor infinite: check before modelling
np.isfinite(a)
np.any(a > 0), np.all(a > 0)          # (True, False)

labels = np.array(["cat", "dog", "cat", "bird", "dog", "cat"])
# [ True False  True  True False  True] element-wise "is it in this list?"
np.isin(labels, ["cat", "bird"])
```

Sorting whole rows by one column uses `argsort`, the indices that would sort that column:

```
s = np.array([30, 10, 20])
np.sort(s), np.argsort(s)              # ([10 20 30], [1 2 0])
# [41 55 67 78 88 92] rows reordered by the first subject
exam[np.argsort(exam[:, 0])[:, 0]]
# the two students with the highest totals (negate to sort descending)
exam[np.argsort(-exam.sum(axis=1))[:, 2]]
```

More linear algebra

* IS NOT MATRIX MULTIPLICATION

`A * B` multiplies element by element; `A @ B` (or `np.matmul`, or `np.dot` for 2-D arrays) is the matrix product. With `A = [[4, 2], [1, 3]]` and `B = [[1, 0], [2, 1]]`, `A * B` is `[[4, 0], [2, 3]]` but `A @ B` is `[[8, 2], [7, 3]]`. Both run without error on square matrices, so the mistake is silent.

A few more calls, and the reason each one matters:

```
A = np.array([[4., 2.], [1., 3.]])
np.linalg.inv(A)                       # [[ 0.3 -0.2] [-0.1  0.4]]
# True compare float arrays with allclose, never with ==
np.allclose(A @ np.linalg.inv(A), np.eye(2))
# 10.0 a zero determinant means "no inverse" (perfectly collinear columns)
np.linalg.det(A)
np.linalg.matrix_rank(A)                # 2 the number of independent columns
# S = [5.117 1.954]; U @ np.diag(S) @ Vt rebuilds A (the engine under PCA)
U, S, Vt = np.linalg.svd(A)
# 7.0 the L1 norm: sum of absolute values (L2 above gave 5.0)
np.linalg.norm([3, -4], ord=1)
```

Linear regression in three lines. Least squares solves the "normal equations" $(X.T @ X) \beta = X.T @ y$. Simulate data with a known answer (intercept 3, slope 2) and recover it:

```
rng = np.random.default_rng(0)
x = rng.uniform(0, 10, 50)
y = 3 + 2 * x + rng.normal(0, 1, 50)    # the truth, plus noise

# a column of 1s gives the model its intercept
X = np.column_stack([np.ones_like(x), x])
beta = np.linalg.solve(X.T @ X, X.T @ y) # [2.757 2.052] close to (3, 2)
np.allclose(beta, np.linalg.lstsq(X, y, rcond=None)[0]) # True
```

`np.linalg.lstsq` gives the same answer and is the safer call in practice: it still works when columns are nearly collinear, where forming $X.T @ X$ loses precision.

SAY IT IN 60 SECONDS

"NumPy stores numbers in one typed block of memory and applies operations to the whole array at once, which is why it is fast: no Python loop per element. Broadcasting is the rule that lets arrays of different shapes combine. Compare the shapes from the right; two dimensions match if they are equal or one of them is 1, and the 1 is stretched. So a 3 by 4 array minus a length-4 row of column means works, one value per column. To subtract row means I need shape 3 by 1, so I use `keepdims` or `reshape`. Slices are views, not copies, so editing one edits the original. The catch: axis zero is down the rows, collapsing rows, and axis one is across columns."

KEY TERMS ndarray · shape · dtype · broadcasting · vectorization · boolean mask · axis · reshape

BUILDS ON 1.1 Core Python language checklist

HANDS-ON PRACTICE Standardise each column of a random matrix using broadcasting, without loops, and verify the means are about 0.

COMMON MISTAKES Confusing `axis=0` and `axis=1` · Mutating a view and expecting a copy · Using a Python loop over a NumPy array

CHECK YOURSELF

1. **X has shape (5, 3). How do you centre each column, and what shape is the mean?**

`X - X.mean(axis=0)` : the mean has shape (3,), broadcast across all 5 rows.

2. **Why does `X - X.mean(axis=1)` fail for shape (5, 3), and how do you fix it?**

The mean has shape (5,), which cannot align with the last axis of size 3. Use `keepdims=True` to get (5, 1).

3. **You slice `b = a[:2]` and set `b[0] = 99`. What happens to `a`?**

`a[0]` also becomes 99: a basic slice is a view of the same memory. Use `.copy()` to avoid it.

3.2 pandas checklist

IN PLAIN WORDS

pandas is a spreadsheet you control with code. A DataFrame is a table with labelled columns; a Series is one column. You pick rows and columns (`loc` and `iloc`), filter them with yes/no tests, add new columns, group rows into piles to summarise them (`groupby`), glue tables together (`merge`), and rotate tables (`pivot`). If you already know SQL, almost every idea has a twin in pandas.

Why it matters: pandas questions are heavily tested in technical rounds, often as "translate this SQL" or "clean this data".

Takeaway: Select, filter, transform, group, join, reshape, and prefer vectorised methods over loops.

WORKED EXAMPLE, STEP BY STEP**agg versus transform, traced**

Five orders:

row	user	amount
0	a	100
1	b	50
2	a	300
3	c	80
4	b	150

- 1. Split** by user: $a = \{100, 300\}$, $b = \{50, 150\}$, $c = \{80\}$.
- 2. agg collapses each group to one row.** `df.groupby("user")["amount"].agg("sum")` gives a 400, b 200, c 80: three rows, one per user. Use it for "total per user".
- 3. transform hands the group value back to every row.** `df.groupby("user")["amount"].transform("sum")` gives 400, 200, 400, 80, 200: five rows, aligned with the original table.
- 4. Use transform for per-row comparisons.** `df["share"] = df["amount"] / df.groupby("user")["amount"].transform("sum")` gives 0.25, 0.25, 0.75, 1.00, 0.75. Each order's share of its user's total, with no merge needed.
- 5. The SQL equivalents.** agg is `GROUP BY`; transform is `SUM(amount) OVER (PARTITION BY user)`.

What this shows. Decide the output shape first. One row per group: agg. Every original row plus a group value: transform.

YOUR TURN (COVER THE SOLUTION FIRST)

- Using the same five orders, write one line that adds a column `is_biggest` that is True for each user's largest order, and give its values.

Solution. `df["is_biggest"] = df["amount"] == df.groupby("user")["amount"].transform("max")`. The group maxima per row are 300, 150, 300, 80, 150, so the values are False, False, True, True, True. User c's only order is its largest. With a tie, both tied rows would be True.

- A column arrives as text: `pd.Series(["1,200", "3,450", "n/a"])`. Turn it into numbers, with NaN for anything invalid. Then say what you would check before using the result.

Solution.

```
s = pd.Series(["1,200", "3,450", "n/a"])
pd.to_numeric(s.str.replace(",", "", regex=False), errors="coerce")    # [1200.0,
3450.0, NaN]
```

Remove the thousands separator first, because "1,200" is not a valid number. `errors="coerce"` turns "n/a" into NaN instead of raising. Before using it, count the NaNs and look at the raw values behind them: `s[converted.isna()].value_counts()`. If they are all genuine placeholders ("n/a", "-", ""), carry on; if some are real numbers in an unexpected format ("1.200,50", "₹1200"), fix the parsing instead of losing those rows.

I/O and inspection

- ☐ `read_csv` (`dtype`, `parse_dates`, `usecols`, `chunksize`), `read_excel`, `read_json`, `read_parquet`, `read_sql`
- ☐ `head`, `info`, `describe`, `dtypes`, `shape`, `isna().sum()`, `nunique`, `value_counts`

Selection

- ☐ `loc` (label-based, inclusive) vs `iloc` (position-based, exclusive end)
- ☐ Boolean filtering, `query`, `isin`, `between`, `str.contains`
- ☐ Avoid chained assignment; understand `SettingWithCopyWarning` and `.copy()`

Transformation

- ☐ `assign`, `apply` (row/column), `map`, `applymap` / `map` on `DataFrame`, vectorized alternatives
- ☐ `astype`, `to_datetime`, `.dt` accessor, `.str` accessor
- ☐ `cut` vs `qcut`, `np.select`, `np.where`
- ☐ `fillna`, `dropna`, `drop_duplicates`, `duplicated`, `replace`, `clip`

Aggregation and reshaping

- ☐ `groupby` + `agg` (named aggregation), `transform`, `filter`, `apply`
- ☐ `merge` (`how=inner/left/right/outer`, `on`, `left_on/right_on`, `suffixes`, `indicator`), `join`, `concat`
- ☐ `pivot`, `pivot_table`, `melt`, `stack/unstack`, `crosstab`
- ☐ `sort_values`, `nlargest`, `rank`, `cumsum`, `cumcount`, `shift`, `diff`, `pct_change`
- ☐ `rolling`, `expanding`, `ewm`, `resample`

Performance

- ☐ Use categorical dtype, downcast numerics, read in chunks, avoid `iterrows`, prefer vectorization
- ☐ Parquet over CSV; know Polars/DuckDB exist

Building a DataFrame, and the first edits

A **Series** is one labelled column; a **DataFrame** is a set of Series that share an index. You can build a frame from a dict of columns or from a list of row dicts, and the result is the same:

```
s = pd.Series([10, 20, 30], index=["a", "b", "c"], name="demo")
# (20, 20.0) look up by label; aggregate like an array
s["b"], s.mean()

# dict of columns
df1 = pd.DataFrame({"city": ["Delhi", "Pune"], "temp": [35, 28]})
# list of rows (JSON, API results)
df2 = pd.DataFrame([{"city": "Delhi", "temp": 35}, {"city": "Pune", "temp": 28}])
df1.equals(df2) # True
```

Some later examples use a second small table, eight employees in three departments:

```
emp = pd.DataFrame({
    "id":      [101, 102, 103, 104, 105, 106, 107, 108],
    "name":    ["Asha", "Ben", "Chen", "Dana", "Esha", "Farid", "Gita", "Hari"],
    "dept":    ["Eng", "Eng", "Eng", "Sales", "Sales", "Sales", "HR", "HR"],
    "gender":  ["F", "M", "M", "F", "F", "M", "F", "M"],
    "tenure":  [2.0, 7.5, 4.0, 1.0, 6.0, 3.0, 9.0, 2.5],          # years
    "salary":  [88000, 104000, 93000, 58000, 71000, 64000, 67000, 52000],
})
```

```
# .copy(): an independent frame that is safe to modify
d = emp[["id", "name", "salary"]].head(3).copy()
d["bonus"] = d["salary"] * 0.1                # a new column from existing ones
d = d.rename(columns={"salary": "base"})        # rename with a dict {old: new}
# drop columns (drop(index=[...]) drops rows)
d = d.drop(columns=["bonus"])

# dept A to Z, highest salary first within each
emp.sort_values(["dept", "salary"], ascending=[True, False])
# Ben 104000, Chen 93000 top-N without a full sort
emp.nlargest(2, "salary")[["name", "salary"]]
# [3, 1, 2, 7, 4, 6, 5, 8] 1 = highest; ties share the lower rank (SQL RANK)
emp["salary"].rank(ascending=False, method="min")
```

Profiling a table, and SQL into pandas

More ways to look before you compute:

```
# random rows: head() can mislead when a file is sorted
emp.sample(3, random_state=42)
# choose the percentiles: 5% = 54100, 95% = 100150
emp["salary"].describe(percentiles=[0.05, 0.95])
# text columns too: count, unique, top (most common), freq
emp.describe(include="all")
# size in KB; deep=True counts the text inside string columns
emp.memory_usage(deep=True).sum() / 1024
emp[["tenure", "salary"]].corr()                # Pearson correlation: 0.34
# rank correlation: 0.48 (robust to outliers and curves)
emp[["tenure", "salary"]].corr(method="spearman")
```

SQL straight into a DataFrame. `read_sql` runs a query over a database connection and returns the result as a frame; `to_sql` writes a frame to a table. Python's built-in `sqlite3` needs no setup, so it is a good way to practise both languages on the same data:

```
import sqlite3
from contextlib import closing

# closing(): the connection is closed when the block ends
with closing(sqlite3.connect("company.db")) as conn:
    emp.to_sql("employees", conn, if_exists="replace", index=False)
    result = pd.read_sql("""
        SELECT dept, COUNT(*) AS headcount, ROUND(AVG(salary)) AS avg_salary
        FROM employees GROUP BY dept ORDER BY avg_salary DESC
    """, conn)

#      dept  headcount  avg_salary
# 0    Eng           3    95000.0
# 1  Sales           3    64333.0
# 2     HR           2    59500.0
```

For other databases you pass a SQLAlchemy engine instead of the `sqlite3` connection. Let the database do the filtering and aggregation, and pull only the result into pandas.

Filtering and the index

Two more filtering tools, and the index itself:

```
threshold = 90000
# Ben and Chen; @ reads a Python variable inside query
emp.query("tenure ≥ 3 and salary > @threshold")
# ~ negates a mask: the 5 rows NOT in Eng
emp[~emp["dept"].isin(["Eng"])]

e = emp.set_index("id")                    # make a column the row labels
e.loc[101, "salary"]                      # 88000 a single cell, by label
# 3 rows: label slices include the end
e.loc[101:103, ["name", "dept"]]

top = emp.nlargest(3, "salary")
# [1, 2, 0] filtered and sorted frames KEEP their old labels
top.index.tolist()
# [0, 1, 2] renumber; drop=True discards the old labels
top.reset_index(drop=True).index.tolist()
# conform to a given list of labels: the unknown 999 becomes a row of NaN
e.reindex([101, 102, 999])
```

`reindex` is how you make a result line up with a fixed list (every date in a calendar, every category in a report), so that missing entries show up as NaN instead of silently disappearing.

Mapping, encoding and cleaning

```
# [1.0, 0.0, 1.0, NaN] a value missing from the dict becomes NaN
pd.Series(["yes", "no", "yes", "maybe"]).map({"yes": 1, "no": 0})
# '$88,000', '$104,000', ... map with a function
emp["salary"].map(lambda v: f"${v:,.0f}")
# 'Asha (Eng)', ... row-wise apply: flexible but slow
emp.apply(lambda r: f"{r['name']} ({r['dept']})", axis=1)
# [0, 2, 1, 0, 2, 1, 2, 1] integer bin codes instead of labels
pd.cut(emp["tenure"], bins=[0, 2, 5, 10], labels=False)
```

`cut` bins are open on the left and closed on the right by default, so `[0, 2, 5, 10]` means (0, 2], (2, 5], (5, 10]: a tenure of exactly 2.0 falls in the first bin.

One-hot encoding turns a category into 0/1 columns for a model:

```
pd.get_dummies(emp[["id", "dept"]], columns=["dept"], drop_first=True, dtype=int).head(4)
#   id  dept_HR  dept_Sales
# 0  101        0          0    ← Eng is the dropped baseline: both columns are 0
# 1  102        0          0
# 2  103        0          0
# 3  104        0          1
```

`drop_first=True` removes one level, because k categories need only $k - 1$ columns; keeping all k makes them perfectly collinear with the intercept in a linear model (the "dummy variable trap").

Cleaning messy text, numbers and dates

Raw files arrive as text. A deliberately dirty table:

```

raw = pd.DataFrame({
    "id":      ["1", "2", "3", "4", "5"],
    "price":   ["$12.50", "8", "N/A", "15.25", "bad"],
    "qty":     ["3", "x", "5", "", "7"],
    "ts":      ["2024-01-05 10:30:00", "2024-01-06 11:45:00", "not a date", "2024-01-08
09:00:00", "2024-01-09 18:15:00"],
})
# ValueError: could not convert string to float: '$12.50'
raw["price"].astype(float)

clean = raw.copy()
# a strict cast, for columns you trust
clean["id"] = clean["id"].astype("int32")
clean["price"] = pd.to_numeric(clean["price"].str.replace("$", "", regex=False),
errors="coerce")
# anything invalid becomes NaN
clean["qty"] = pd.to_numeric(clean["qty"], errors="coerce")
# invalid becomes NaT
clean["ts"] = pd.to_datetime(clean["ts"], format="%Y-%m-%d %H:%M:%S", errors="coerce")
#
#   id  price  qty      ts
# 0   1  12.50   3.0 2024-01-05 10:30:00
# 1   2   8.00  NaN 2024-01-06 11:45:00
# 2   3   NaN   5.0      NaT
# 3   4  15.25  NaN 2024-01-08 09:00:00
# 4   5   NaN   7.0 2024-01-09 18:15:00

```

`astype` fails on the first bad value; `errors="coerce"` converts what it can and marks the rest as missing. Always count the new NaNs afterwards (`clean.isna().sum()`): each one is a row the source system got wrong, and silently losing them is a data-quality bug.

The **`.str` accessor** applies a string method to a whole column and passes missing values through safely:

```

people = pd.DataFrame({
    "raw":      [" ALICE Smith, 34, Sales ", "bob JONES, 45, HR", "Carla Diaz, 29,
Engineering "],
    "sku":      ["AB-1234", "XY-99", "AB-5"],
    "notes":    ["Great fit", None, "needs TRAINING"],
})
# 'alice smith, 34, sales', ... methods chain
people["raw"].str.strip().str.lower()
# each capture group becomes a column: AB | 1234, XY | 99, AB | 5
people["sku"].str.extract(r"([A-Z]+)-(\d+)")
# EVERY match: 12, 15, 7 (one row per match)
pd.Series(["order 12 and 15", "order 7"]).str.extractall(r"(\d+)")

parts = people["raw"].str.split(" ", expand=True) # expand=True: one column per piece
parts.columns = ["name", "age", "dept"]
parts = parts.apply(lambda col: col.str.strip()) # ALICE Smith | 34 | Sales ...
# 'Alice Smith', 'Bob Jones', 'Carla Diaz'
parts["name"].str.title()

# [True, False, False] na=False: a missing note is "no match"
people["notes"].str.contains("great", case=False, na=False)

```

Without `na=False`, older pandas returns NaN for the missing note, and a mask containing NaN cannot be used to filter.

The **`.dt` accessor** does the same for dates, and is where most time features come from:


```
t = pd.Series(pd.date_range("2024-02-28 22:00", periods=3, freq="9h"))
# 2024-02-28 22:00, 2024-02-29 07:00, 2024-02-29 16:00
t.dt.year, t.dt.month, t.dt.hour, t.dt.quarter      # parts of the date
t.dt.dayofweek                                     # [2, 3, 3]    Monday = 0
# ['Wednesday', 'Thursday', 'Thursday']
t.dt.day_name()
# [False, True, True]    2024 is a leap year
t.dt.is_month_end
# ['2024-02', '2024-02', '2024-02']    back to text
t.dt.strftime("%Y-%m")

hired = pd.to_datetime(pd.Series(["2024-03-01", "2018-09-15", "2022-03-10"]))
# [671, 2665, 1393]    subtracting dates gives timedeltas; .dt.days makes them numbers
(pd.Timestamp("2026-01-01") - hired).dt.days
```

Time zones take two steps: first *label* the zone the timestamps are in, then *convert*.

```
utc = t.dt.tz_localize("UTC")                    # declare: these naive times are UTC
# 2024-02-29 03:30+05:30, 12:30+05:30, 21:30+05:30
utc.dt.tz_convert("Asia/Kolkata")
```

The first event is on 28 February in UTC but on 29 February in India. Daily counts depend on the time zone, so agree on one before you group by day.

Missing values and duplicates in practice

Measure the problem before you fix it, and know the options for dropping:

```
# PERCENT missing per column (isna().sum() gives counts)
df.isna().mean() * 100
# drop a row if ANY of these columns is missing (the default)
df.dropna(subset=["a", "b"], how="any")
df.dropna(subset=["a", "b"], how="all")    # drop a row only if ALL of them are missing
# keep only COLUMNS with at least 50% real values
df.dropna(axis=1, thresh=int(0.5 * len(df)))
```

For ordered data (time series, sensor readings) you can fill from the neighbours:

```
s = pd.Series([1.0, np.nan, np.nan, 4.0, np.nan, 6.0])
pd.DataFrame({"raw": s, "ffill": s.ffmpeg(), "bfill": s.bfill(), "linear": s.interpolate()})
#   raw  ffill  bfill  linear
# 0  1.0    1.0    1.0    1.0
# 1  NaN    1.0    4.0    2.0
# 2  NaN    1.0    4.0    3.0
# 3  4.0    4.0    4.0    4.0
# 4  NaN    4.0    6.0    5.0
# 5  6.0    6.0    6.0    6.0
```

`ffill` carries the last known value forward, `bfill` pulls the next one back, and `interpolate` draws a straight line between the known points. On a date index with uneven gaps, `interpolate(method="time")` spaces the line by the real time elapsed instead of by row count. In a forecasting or modelling setting use `ffill` only: `bfill` and interpolation use future values, which is leakage.

Duplicates are defined by the columns you choose:

```

dups = pd.DataFrame({"id": [1, 2, 2, 3, 3, 3], "score": [88, 72, 72, 90, 90, 95]})
# [False, False, True, False, True, False]   whole-row repeats
dups.duplicated().tolist()
# [False, False, True, False, True, True]     same id seen before
dups.duplicated(subset=["id"]).tolist()
# one row per id, keeping the LAST: scores 88, 72, 95
dups.drop_duplicates(subset=["id"], keep="last")

```

Sort first (for example by an `updated_at` column) so that "last" really is the most recent record.

Split, apply, combine

The idea behind all three is **split, apply, combine**: split the rows into groups, compute something for each group, and combine the results. A few more forms on the `emp` table:

```

# several statistics at once
emp.groupby("dept")["salary"].agg(["mean", "median", "std"]).round(0)
#           mean  median    std
# Eng    95000.0  93000.0  8185.0
# HR     59500.0  59500.0 10607.0
# Sales  64333.0  64000.0  6506.0

# two keys; as_index=False keeps them as normal columns
emp.groupby(["dept", "gender"], as_index=False)["salary"].mean()

emp["dept_z"] = emp.groupby("dept")["salary"].transform(lambda s: (s - s.mean()) / s.std())
# Asha -0.86, Ben 1.10, Chen -0.24, ...   each salary compared with ITS OWN department

# ['Eng', 'Sales']: HR's rows are dropped
emp.groupby("dept").filter(lambda g: g["salary"].mean() > 60000)["dept"].unique()

```

A z-score within the group answers "is this person paid unusually for their department?", which the raw salary cannot.

Which rows a join keeps

Two tiny tables show exactly which rows each join keeps. Dana (user 4) has no orders, and order 104 belongs to a user (5) who does not exist:

```

users = pd.DataFrame({"user_id": [1, 2, 3, 4], "name": ["Asha", "Ben", "Chen", "Dana"]})
orders2 = pd.DataFrame({"order_id": [101, 102, 103, 104, 105], "uid": [1, 1, 3, 5, 2],
                        "amount": [250.0, 80.5, 120.0, 60.0, 300.0]})

for how in ["inner", "left", "right", "outer"]:
    print(how, len(users.merge(orders2, left_on="user_id", right_on="uid", how=how)))
# inner 4      only matched pairs
# left  5      + Dana, with NaN order columns
# right 5      + order 104, with NaN user columns
# outer 6      both unmatched rows

```

`validate=` makes the join check its own assumption and fail loudly instead of silently multiplying rows:

```

# passes: each order has at most one user
orders2.merge(users, left_on="uid", right_on="user_id", how="left", validate="many_to_one")
orders2.merge(orders2, on="uid", validate="one_to_one")
# MergeError: Merge keys are not unique in either left or right dataset; not a one-to-one merge.

```

If a key is repeated on both sides, every left row pairs with every matching right row (2 rows $\times$ 2 rows = 4), which inflates sums. Check `len()` before and after any join. `df.join` matches a column of the left table to the **index** of the right one:

`orders2.join(users.set_index("user_id"), on="uid", how="left")`. `pd.concat([...], ignore_index=True)` stacks rows and rennumbers them; with `axis=1` it aligns on the index, so reset or check the index first.

Pivot, melt, stack and crosstab

`pivot_table` groups, aggregates and reshapes in one call, and `margins=True` adds the totals:

```
sales = pd.DataFrame({"region": ["North", "North", "South", "South", "North", "South"],
                      "quarter": ["Q1", "Q2", "Q1", "Q2", "Q1", "Q2"],
                      "revenue": [100.0, 130.0, 170.0, 95.0, 120.0, 200.0]})
sales.pivot_table(index="region", columns="quarter", values="revenue", aggfunc="sum",
                  fill_value=0, margins=True)
# quarter      Q1      Q2    All
# North      220.0  130.0  350.0
# South      170.0  295.0  465.0
# All        390.0  425.0  815.0
```

`aggfunc` also accepts a dict, for example `{"units": "sum", "revenue": "mean"}`, to aggregate each value column differently.

Long and wide are the same data in two layouts. Long has one row per observation (date, ticker, close); wide has one column per ticker. `pivot` goes long to wide without aggregating, `melt` goes back:

```
prices = pd.DataFrame({"date": pd.to_datetime(["2024-01-01"] * 2 + ["2024-01-02"] * 2),
                      "ticker": ["AAA", "BBB"] * 2, "close": [100.0, 50.0, 102.0, 51.0]})
wide = prices.pivot(index="date", columns="ticker", values="close")
# ticker      AAA    BBB
# 2024-01-01  100.0   50.0
# 2024-01-02  102.0   51.0
# the original four rows again
wide.reset_index().melt(id_vars="date", var_name="ticker", value_name="close")
```

`pivot` raises `ValueError: Index contains duplicate entries` if a (date, ticker) pair appears twice; that is the moment to use `pivot_table` with an explicit `aggfunc`. Long is the layout for `groupby` and plotting libraries; wide is the layout for correlation matrices and comparing series side by side.

`unstack` moves the inner level of a grouped result into columns, `stack` moves it back, and `crosstab` counts combinations of two columns:

```
# a Series with a two-level index
mi = emp.groupby(["dept", "gender"])["salary"].mean()
mi.unstack()
# gender      F      M
# Eng      88000.0  98500.0
# HR       67000.0  52000.0
# Sales    64500.0  64000.0

pd.crosstab(emp["dept"], emp["gender"], normalize="index", margins=True).round(2)
# gender      F      M
# Eng      0.33  0.67      ← each ROW sums to 1: the gender mix within a department
# HR       0.50  0.50
# Sales    0.67  0.33
# All      0.50  0.50
```

Windows, lags and returns

The window types side by side, on six daily closing prices:

```
px = pd.Series([100.0, 102.0, 101.0, 105.0, 107.0, 104.0], index=pd.date_range("2024-01-01", periods=6, freq="D"))
pd.DataFrame({
    "close": px,
    # the last 3 days; NaN until the window is full
    "ma3": px.rolling(3).mean(),
    # exponentially weighted: recent days count more
    "ewm": px.ewm(span=3, adjust=False).mean(),
    "run_max": px.cummax(),
    "exp_mean": px.expanding().mean(),
}).round(2)
#      close      ma3      ewm  run_max  exp_mean
# 2024-01-01  100.0    NaN  100.0   100.0   100.00
# 2024-01-02  102.0    NaN  101.0   102.0   101.00
# 2024-01-03  101.0  101.00  101.0   102.0   101.00
# 2024-01-04  105.0  102.67  103.0   105.0   102.00
# 2024-01-05  107.0  104.33  105.0   107.0   103.00
# 2024-01-06  104.0  105.33  104.5   107.0   103.17
```

`rolling(3, min_periods=1)` fills the first rows with the average of whatever is available instead of NaN. Lags, leads and returns:

```
px.shift(1)      # yesterday's price (LAG): NaN, 100, 102, 101, 105, 107
px.shift(-1)     # tomorrow's price (LEAD): 102, 101, 105, 107, 104, NaN
px.diff()        # change from the previous row: NaN, 2, -1, 4, 2, -3
px.pct_change()  # return: NaN, 0.0200, -0.0098, 0.0396, 0.0190, -0.0280
```

A LEAD IS THE FUTURE

`shift(-1)` puts tomorrow's value on today's row. It is fine for building a *target* ("did the price rise the next day?"), but used as a *feature* it leaks the answer into the model. Lags (`shift(1)` , rolling over past rows) are safe; anything that looks forward is not.

SHIFT WITHIN EACH GROUP

When several series are stacked in one table, a plain `shift` carries the last row of one series into the first row of the next:

```
pr = pd.DataFrame({"ticker": ["AAA"] * 3 + ["BBB"] * 3, "close": [100.0, 102.0, 101.0,
50.0, 51.0, 49.0]})
# BBB's first row gets 101.0: AAA's last price
pr["wrong"] = pr["close"].shift(1)
# BBB's first row is NaN, as it should be
pr["prev"] = pr.groupby("ticker")["close"].shift(1)
# 0.0200, -0.0098 / 0.0200, -0.0392
pr["ret"] = pr.groupby("ticker")["close"].pct_change()
```

Sort by group and time, then `groupby(...).shift()`, `.diff()` or `.pct_change()`.

Memory and speed, measured

How much the right dtypes save, measured. A 200,000-row table with an id, an age, a score and two short text columns (country: 5 values; device: 3 values):

```
def optimize_dtypes(df, cat_threshold=0.5):
    """Return a smaller copy: downcast numbers, and store low-cardinality text as
    category."""
    out = df.copy()
    for col in out.columns:
        s = out[col]
        if pd.api.types.is_float_dtype(s):
            out[col] = pd.to_numeric(s, downcast="float")           # float64 → float32
        elif pd.api.types.is_integer_dtype(s):
            # int64 → the smallest int that fits
            out[col] = pd.to_numeric(s, downcast="integer")
        elif pd.api.types.is_string_dtype(s) and s.nunique() / max(len(s), 1) <
cat_threshold:
            # few distinct values → category
            out[col] = s.astype("category")
    return out
```

Step	Memory	What changed
As loaded	24.5 MB	int64, int64, float64, text, text
Numbers downcast	21.6 MB	user_id int32, age int8, score float32
Text as <code>category</code>	2.1 MB	each distinct string stored once; rows hold small integer codes

The numeric downcast saves little here; the text columns were the cost, and `category` removes 91% of the total. Rule of thumb: use `category` when the number of distinct values is well below the number of rows (a unique ratio under about 50%). Do not use it for near-unique columns such as names or ids, where it adds overhead. `float32` keeps about 7 significant digits, which is fine for features and wrong for money totals.

The speed ladder, measured. One task done six ways on 50,000 rows: `a * b if a > b else a + b`.

Method	Time	Relative
<code>np.where</code> on NumPy arrays	0.06 ms	1×

Method	Time	Relative
pandas vectorised (<code>.where</code>)	0.26 ms	about 4×
list comprehension over <code>zip(df.a, df.b)</code>	5.8 ms	about 90×
<code>df.itertuples()</code>	15 ms	about 250×
<code>df.apply(..., axis=1)</code>	207 ms	about 3,300×
<code>df.iterrows()</code>	526 ms	about 8,500×

The exact numbers depend on the machine, but the order does not. Work down the ladder only as far as you must: a vectorised expression first, then `np.where` / `np.select`, then `.map`, and only then row-wise code. Notice that when you really do need a Python loop, a comprehension over `zip` of the columns beats `apply(axis=1)` by a wide margin.

Method chaining writes a transformation as one top-to-bottom recipe, with no half-finished intermediate variables to get out of sync:

```
report = (
    emp
    .query("tenure > 1")
    .assign(salary_k=lambda d: d["salary"] / 1000)
    .groupby("dept", as_index=False)
    .agg(avg_salary_k=("salary_k", "mean"), n=("id", "count"))
    .sort_values("avg_salary_k", ascending=False)
    .round(1)
)
#      dept  avg_salary_k  n
# 0    Eng           95.0  3
# 2  Sales           67.5  2
# 1     HR           59.5  2

def add_flag(df, col, threshold):
    return df.assign(**{f"{col}_flag": df[col] > threshold})

# 2  pipe plugs your own function into a chain
emp.pipe(add_flag, "salary", 90000)["salary_flag"].sum()
```

Each step returns a new DataFrame, so the raw data stays untouched and the whole block can be rerun safely. Seed your random generators and transform copies, never the raw frame: those two habits make an analysis reproducible.

SAY IT IN 60 SECONDS

"In pandas I follow a fixed routine. Load, then look at shape, dtypes, head, and missing counts. Select with loc by label or iloc by position, and filter with boolean masks; combine conditions with ampersand and brackets. Create columns by assignment, aggregate with groupby then agg, combine tables with merge, and reshape with pivot or melt. The two groupby tools are agg, which shrinks each group to one row, and transform, which returns a value per original row, like a group mean next to each row. The catch: chained assignment can silently modify a copy, so I use loc to set values, and I always check dtypes, because numbers stored as strings break everything."

KEY TERMS DataFrame · loc · iloc · boolean filtering · groupby · merge · pivot · rolling · resample · to_datetime

BUILDS ON 3.1 NumPy essentials and broadcasting

HANDS-ON PRACTICE Load any CSV and answer five business questions using only groupby, merge and pivot_table, without loops.

COMMON MISTAKES Chained assignment · Using apply row by row when a vectorised method exists · Forgetting to sort before shift or diff

CHECK YOURSELF

1. How do you add each row's department average salary as a new column?

```
df['dept_avg'] = df.groupby('dept')['salary'].transform('mean').
```

2. Why write `df.loc[mask, 'col'] = 0` instead of `df[mask]['col'] = 0`?

The chained version may modify a temporary copy and warn; loc assigns to the original in one step.

3. What does merge do by default, and what do you check afterwards?

An inner join on common columns. Check row counts and use `validate='one_to_one'` or similar to catch duplicate-key fan-out.

3.3 SQL to pandas translation table

IN PLAIN WORDS

If you already know SQL, pandas is mostly a change of spelling. "WHERE" becomes a yes/no filter on the table, "GROUP BY" becomes groupby, "JOIN" becomes merge, and window functions (the tricky part) become groupby followed by transform, shift, cumsum or cumcount. The big idea: SQL keeps every row when you use a window function; pandas does the same with transform.

Why it matters: Interviewers often give a SQL-style task in pandas, or ask you to rewrite one in the other.

Takeaway: Learn the twelve pairs in the table and you can move between SQL and pandas in either direction.

WORKED EXAMPLE, STEP BY STEP

Translate a query clause by clause

SQL from topic 2.2: users with at least two delivered orders before March, by total spend.

```
SELECT user_id, COUNT(*) AS n, SUM(amount) AS spend
FROM orders
WHERE status = 'delivered' AND order_date < '2025-03-01'
GROUP BY user_id HAVING COUNT(*) ≥ 2
ORDER BY spend DESC;
```

1. FROM: the DataFrame `orders`.

2. WHERE becomes a boolean mask (note the brackets and `&`): `d =`

```
orders[(orders.status == "delivered") & (orders.order_date < "2025-03-01")]
```

3. GROUP BY plus the aggregates becomes groupby plus named agg: `g =`

```
d.groupby("user_id").agg(n=("amount", "size"), spend=("amount", "sum"))
```

4. HAVING becomes a filter on the grouped result: `g = g[g.n ≥ 2]`

5. ORDER BY becomes sort_values: `g.sort_values("spend", ascending=False)`

6. Result: user 2 (n 2, spend 3,400), then user 1 (n 3, spend 2,900), exactly as in SQL.

SQL	pandas
WHERE	<code>mask df[(cond1) & (cond2)]</code>
GROUP BY ... COUNT, SUM	<code>groupby().agg(name=(col, func))</code>
HAVING	filter the grouped frame
ORDER BY ... DESC	<code>sort_values(..., ascending=False)</code>

What this shows. Both languages run the same pipeline: filter rows, group, aggregate, filter groups, sort. Translate one clause at a time.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Translate to pandas: `SELECT user_id, order_date, amount - LAG(amount) OVER (PARTITION BY user_id ORDER BY order_date) AS change FROM orders.`

Solution.

```
o = orders.sort_values(["user_id", "order_date"])
o["change"] = o["amount"] - o.groupby("user_id")["amount"].shift(1)
```

`sort_values` provides the ORDER BY inside each partition; `groupby("user_id")` is the PARTITION BY; `shift(1)` is LAG. The first order of each user gets NaN, just as LAG gives NULL.

THE FUNDAMENTALS

SQL	pandas
<code>SELECT col1, col2</code>	<code>df[['col1', 'col2']]</code>
<code>WHERE a > 5 AND b = 'x'</code>	<code>df[(df.a > 5) & (df.b == 'x')]</code>
<code>GROUP BY k + SUM(v)</code>	<code>df.groupby('k')['v'].sum()</code>
<code>COUNT(DISTINCT u)</code>	<code>df['u'].nunique()</code>
<code>ORDER BY v DESC LIMIT 5</code>	<code>df.nlargest(5, 'v')</code> or <code>sort_values('v', ascending=False).head(5)</code>
<code>LEFT JOIN</code>	<code>a.merge(b, on='k', how='left')</code>
<code>ROW_NUMBER() OVER (PARTITION BY g ORDER BY v DESC)</code>	<code>df['rn'] = df.sort_values('v', ascending=False).groupby('g').cumcount() + 1</code>
<code>LAG(v) OVER (PARTITION BY g ORDER BY t)</code>	<code>df.sort_values('t').groupby('g')['v'].shift(1)</code>
<code>SUM(v) OVER (PARTITION BY g)</code>	<code>df.groupby('g')['v'].transform('sum')</code>
<code>CASE WHEN</code>	<code>np.select([cond1, cond2], [v1, v2], default=v3)</code>
<code>HAVING COUNT(*) > 1</code>	<code>df.groupby('k').filter(lambda g: len(g) > 1)</code>

SAY IT IN 60 SECONDS

"Most pandas interview questions are SQL ideas in different syntax. WHERE becomes a boolean mask. SELECT becomes a column list. GROUP BY with an aggregate becomes groupby then agg. JOIN becomes merge with how equal to inner or left. ORDER BY and LIMIT become sort_values and head. A window function like ROW_NUMBER over partition becomes groupby then cumcount or rank, and LAG becomes groupby then shift. COUNT DISTINCT becomes nunique. HAVING becomes filtering the grouped result. So I translate the SQL in my head first. The catch: pandas count skips NaN like SQL, and a left merge keeps unmatched rows with NaN, so I check for missing after merging."

KEY TERMS SQL to pandas · window functions in pandas · transform · cumcount · shift · np.select

BUILDS ON 3.2 pandas checklist · 2.7 SQL problems, level 3: running totals, growth, LAG/LEAD

HANDS-ON PRACTICE Take five SQL queries from Module 2 and translate each to pandas, then compare the outputs on the same data.

COMMON MISTAKES Forgetting to sort before cumcount or shift · Using apply with a lambda where transform is enough · Filtering with and/or instead of & and |

CHECK YOURSELF

- 1. Translate** `SELECT dept, AVG(salary) FROM emp GROUP BY dept HAVING COUNT(*) > 5.`

```
g = emp.groupby('dept').agg(avg=('salary', 'mean'), n=('salary', 'size')); g[g.n > 5]
['avg'].
```
- 2. Pandas equivalent of** `LAG(amount) OVER (PARTITION BY user ORDER BY date) ?`
Sort by date, then `df.groupby('user')['amount'].shift(1)`.
- 3. Users with no orders: SQL anti-join in pandas?**
`merge(..., how='left', indicator=True)` then keep `_merge == 'left_only'`.

3.4 Worked pandas problems

IN PLAIN WORDS

These ten problems are the "greatest hits" of pandas interviews: find the best items per group, see month-on-month growth, smooth a noisy series, fill missing values sensibly, compute retention, catch outliers, unpack lists stored in one cell, and check whether two tables really match. Each one is a small recipe: group, aggregate, sort, and shape the answer.

Why it matters: They test whether you can turn a business question into two or three chained operations.

Takeaway: Think in steps: group, compute, sort, reshape; verify with a tiny example.

WORKED EXAMPLE, STEP BY STEP**Group consecutive runs with "compare to the previous row"**

A sensor reports a status every minute: A, A, B, B, B, A. Label each run of identical values (the pandas cousin of SQL's gaps and islands).

1. Compare each value with the previous one: `s != s.shift()`.

minute	status	previous	changed?
0	A	NaN	True
1	A	A	False
2	B	A	True
3	B	B	False
4	B	B	False
5	A	B	True

2. A running count of the changes is the run id: `(s != s.shift()).cumsum()` gives 1, 1, 2, 2, 2, 3.**3. Group by the run id** to get each run's length and status: run 1 = A × 2, run 2 = B × 3, run 3 = A × 1.**4. Answer the question.** "Longest time in state B": 3 minutes (run 2).**5. Per entity**, add the entity to the comparison, for example `(df.status != df.groupby("machine").status.shift()).cumsum()`, so a new machine starts a new run.

What this shows. "Flag where something changes, then take a running sum" turns a row-by-row problem into a single vectorised line.

YOUR TURN (COVER THE SOLUTION FIRST)

1. `df` has columns `user`, `ts` and `amount`, one row per order. Write the pandas for "the latest order of each user" and say why the order of the operations matters.

Solution. `df.sort_values("ts").drop_duplicates("user", keep="last")`. Sorting by time first puts each user's latest order last, so `keep="last"` keeps it. Without the sort, "last" just means "last in the file", which is arbitrary. An alternative that keeps ties is `df[df.ts == df.groupby("user").ts.transform("max")]`.

THE FUNDAMENTALS**A capstone: a salary equity review, end to end**

This problem strings the whole toolkit together on the eight-person `emp` table from the pandas checklist (id, name, dept, gender, tenure, salary). The business questions: is there a gender pay gap, and which departments are over budget?

1. The raw gap.

```
raw = emp.groupby("gender")["salary"].mean()          # F 71000, M 78250
# -0.093: women earn 9.3% less on average
raw["F"] / raw["M"] - 1
```

2. Compare like with like. Departments pay very differently (Eng averages 95,000, HR 59,500), and two of the three engineers are men. Divide each salary by the median of its own department, then compare:

```
emp["vs_dept"] = emp["salary"] / emp.groupby("dept")["salary"].transform("median")
emp.groupby("gender")["vs_dept"].mean().round(3)      # F 1.022, M 0.998
```

Relative to their own department's median, women are paid slightly *more*. The raw gap came from who works where, not from unequal pay for the same work.

3. Control for several things at once with a regression: salary on tenure, department and gender.

```
X = pd.get_dummies(emp[["tenure", "dept", "gender"]], columns=["dept", "gender"],
drop_first=True, dtype=float)
X.insert(0, "intercept", 1.0)
beta = np.linalg.lstsq(X.to_numpy(), emp["salary"].to_numpy(), rcond=None)[0]
pd.Series(beta, index=X.columns).round(0)
# intercept      82586.0
# tenure         2599.0      each extra year is worth about 2,600
# dept_HR        -38568.0    compared with Eng, the baseline
# dept_Sales     -27275.0
# gender_M       1081.0      M versus F, holding department and tenure fixed
```

The `gender_M` coefficient is about 1,000, tiny beside salaries that differ by tens of thousands. With eight people this is an illustration, not evidence: a real review would report a confidence interval (for example with `statsmodels`) before concluding anything.

4. Payroll against budget: aggregate, merge, classify.

```
budget = pd.DataFrame({"dept": ["Eng", "Sales", "HR"], "budget": [280000, 200000, 150000]})
payroll = emp.groupby("dept", as_index=False).agg(headcount=("id", "count"), payroll=
("salary", "sum"))
check = payroll.merge(budget, on="dept", how="left", validate="one_to_one")
check["utilisation"] = (check["payroll"] / check["budget"]).round(2)
check["status"] = np.select([check["utilisation"] > 1, check["utilisation"] < 0.8],
["over budget", "under-used"], default="on track")
#   dept headcount payroll budget utilisation status
# 0   Eng         3   285000  280000      1.02 over budget
# 1   HR         2   119000  150000      0.79 under-used
# 2  Sales         3   193000  200000      0.96   on track
```

What to say in the interview. "The raw gap was 9.3%, but it is explained by department mix; within departments and controlling for tenure the difference is about 1,000 and not distinguishable from zero on this sample." A raw difference between two groups is a starting point. Always ask what else differs between the groups before you call it an effect.

SAY IT IN 60 SECONDS

"For a pandas problem I do three things. First I state the output shape: one row per user, per day, or per group. Second I pick the tool: `groupby` for aggregation, `transform` for a per-row group value, `merge` to combine, `pivot_table` to reshape, `rank` for top N. Third I build in small steps and print after each one. The classic ones are top N per group using `sort` then `groupby head`, de-duplicating while keeping the latest row with `sort` then `drop_duplicates`, a rolling average with `rolling`, and a streak using the `cumsum-on-change` trick. The catch: vectorised code beats `apply` with a Python function, and I must handle NaN and ties explicitly."

KEY TERMS `groupby` `head` · `resample` · `rolling` · `transform` median · named aggregation · cohort retention · IQR outliers · `explode` · `merge` indicator

BUILDS ON 3.3 SQL to pandas translation table

HANDS-ON PRACTICE Re-solve each problem on your own data, then vary the requirement (top 2, 14-day window, median instead of mean).

COMMON MISTAKES Forgetting `as_index=False`, leaving group keys in the index · Using `apply` with `lambda` where `transform` works · Rolling over unsorted data

CHECK YOURSELF

1. Top 2 salaries per department in pandas?

```
df.sort_values('salary', ascending=False).groupby('dept').head(2).
```

2. Keep only each user's latest row.

```
df.sort_values('ts').drop_duplicates('user', keep='last').
```

3. Why avoid `df.apply(f, axis=1)` on a million rows?

It loops in Python row by row; vectorised column operations run in compiled code and are orders of magnitude faster.

3.5 Conceptual pandas questions

IN PLAIN WORDS

These are the "explain it" questions for pandas. The short version: `loc` picks by name and `iloc` by position; `merge` joins on columns while `join` works on the row labels; `agg` squeezes each group to one number while `transform` keeps every row; and very large files are read in pieces or converted to a faster format so your computer does not run out of memory.

Why it matters: Interviewers ask these to check you understand pandas, not only type it.

Takeaway: Know the one-line difference and a tiny example for each.

WORKED EXAMPLE, STEP BY STEP

`loc` and `iloc`, with an index that is not 0, 1, 2

```
d = pd.DataFrame({"x": [1, 2, 3, 4]}, index=[10, 20, 30, 40])
```

1. `d.loc[10:30]` selects by label, and label slices *include* the end: labels 10, 20, 30, so `x = 1, 2, 3` (three rows).

2. `d.iloc[0:2]` selects by position, and position slices *exclude* the end: positions 0 and 1, so `x = 1, 2` (two rows).

3. **Now shuffle the index:** `d2 = pd.DataFrame({"x": [1, 2, 3]}, index=[2, 0, 1])`.

4. `d2.loc[0]` finds the row whose *label* is 0: `x = 2`.

5. `d2.iloc[0]` takes the *first* row: `x = 1`.

6. **Why it matters.** After filtering or sorting, labels and positions stop matching. Code that mixes them up gives wrong rows with no error.

What this shows. `loc` asks "which label?", `iloc` asks "which position?". When in doubt, look at `df.index` before slicing.

YOUR TURN (COVER THE SOLUTION FIRST)

1. What is wrong with `df[df.amount > 100]["flag"] = 1`, what might happen, and what is the correct line?

Solution. It is chained indexing: `df[df.amount > 100]` may return a *copy*, and then `["flag"] = 1` sets the column on that temporary copy. pandas warns (SettingWithCopyWarning, or under copy-on-write simply leaves `df` unchanged), and `df` does not get the flag. Correct: `df.loc[df.amount > 100, "flag"] = 1`, one operation on the original frame.

SAY IT IN 60 SECONDS

"The conceptual pandas questions test whether you understand what the library is doing. A Series is one labelled column, a DataFrame a table of Series sharing an index. `loc` selects by label and includes the end of a slice, `iloc` by integer position and excludes it. `apply` runs a Python function, so it is slow; vectorised operations are faster. `merge` joins on columns, `join` on the index, `concat` stacks. A view shares memory with the original and a copy does not, which is the source of the SettingWithCopy warning. Missing values are NaN, and `dropna`, `fillna` and `isna` handle them. The catch: category dtype saves memory for low-cardinality strings, and downcasting numerics helps big data."

KEY TERMS `loc` vs `iloc` · `apply` vs `map` · `merge` vs `join` vs `concat` · `transform` vs `agg` · `pivot` vs `pivot_table` · `chunksize` · `iterrows` · `view` vs `copy`

BUILDS ON 3.2 pandas checklist

HANDS-ON PRACTICE Explain each row in 30 seconds with a code example.

COMMON MISTAKES Reading a whole huge CSV into memory · Using `apply` for arithmetic that vectorises · Mixing up `join` and `merge`

CHECK YOURSELF

1. `df.loc[0:2]` versus `df.iloc[0:2]` : how many rows?

`loc` returns 3 rows (label slices include the end); `iloc` returns 2 (positional, end excluded).

2. Name two ways to cut the memory of a large DataFrame.

Convert low-cardinality strings to `category` and downcast numeric columns (`int64` to `int32`, `float64` to `float32`); also read only needed columns.

3. Difference between `merge`, `join`, `concat` ?

`merge`: SQL-style on columns; `join`: merge on index; `concat`: stack along an axis without matching keys.

3.6 Visualization: choosing the right chart

IN PLAIN WORDS

A chart is a sentence drawn with shapes. Pick the shape that matches the sentence you want to say: "how are these spread out?" is a histogram or box plot; "do these two things move together?" is a scatter plot; "which is biggest?" is a sorted bar chart; "how did it change over time?" is a line. A good chart has a title that says the answer ("Orders fell 10% after the price change"), labelled axes and as little decoration as possible.

Why it matters: Your charts are how non-technical people judge your analysis.

Takeaway: Match the chart to the question, label everything, never mislead, and put the takeaway in the title.

WORKED EXAMPLE, STEP BY STEP

Pick the chart from the question

One dataset: daily orders for a year, by city and by device. Four questions, four charts.

1. **"Did orders grow this year?"** Change over time: a **line chart** of weekly orders. A weekly sum smooths out the weekday wobble.
2. **"Which city orders the most?"** Comparing categories: a **horizontal bar chart**, sorted from largest to smallest, starting at zero.
3. **"Are order sizes skewed?"** Distribution of one variable: a **histogram** of order value, perhaps on a log axis if a few large orders stretch it.
4. **"Do bigger baskets come with longer delivery times?"** Relationship between two numbers: a **scatter plot** of basket size against delivery time, with a trend line.
5. **Write the title as the finding**, not the topic. Not "Orders by week", but "Orders grew 40% this year, mostly after the July launch".
6. **Remove what does not help:** 3-D effects, a legend when a direct label would do, colours with no meaning.

What this shows. The question chooses the chart. If you cannot say the question in one sentence, you are not ready to choose.

YOUR TURN (COVER THE SOLUTION FIRST)

1. A slide shows two bars: "Old checkout: 95% success" and "New checkout: 97% success". The y-axis runs from 94% to 98%, so the second bar looks twice as tall. What is wrong, and how would you fix it?

Solution. A bar's length represents its value, so the axis must start at zero; truncating it makes a 2-point difference look like a doubling. Either start the axis at 0% (the bars then look nearly equal, which is honest), or, if the 2-point gain is the message, show it directly: a dot plot, or a label "Success rate up from 95% to 97% (+2 points)", ideally with the confidence interval of the difference.

THE FUNDAMENTALS

Library choices: Matplotlib (control), Seaborn (statistical, quick), Plotly (interactive), Altair (declarative).

Goal	Chart
Distribution of one numeric variable	Histogram, KDE, box plot, violin
Relationship between two numerics	Scatter, hexbin, regression plot
Correlation among many	Heatmap of correlation matrix, pair plot
Category comparison	Bar chart (sorted), lollipop
Trend over time	Line chart
Part-to-whole	Stacked bar (prefer over pie)

Goal	Chart
Categorical vs numeric	Box/violin per category
Classification performance	Confusion matrix, ROC, PR curve, calibration plot

Good-chart rules: label axes and units, start bar charts at zero, limit colors, avoid 3D and dual axes, annotate the insight, and always title with the takeaway.

Be able to answer: What does a box plot show? How do you read a QQ plot? What does skewness look like in a histogram? What does a correlation heatmap not tell you (non-linearity, causation)?

SAY IT IN 60 SECONDS

"Choose the chart by the question. Comparing categories: bar chart. Change over time: line chart. Distribution of one variable: histogram or box plot. Relationship between two numbers: scatter plot. Many numeric relationships at once: correlation heatmap. Parts of a whole: a stacked bar, rarely a pie. Rules: start bar charts at zero, label axes with units, use one colour unless colour carries meaning, and avoid dual axes that mislead. Always state the takeaway in the title, like 'Orders fell 12 per cent after the price change'. The catch: a chart's job is a decision or insight, so I remove everything that does not support the message."

KEY TERMS histogram · box plot · scatter · heatmap · QQ plot · bar chart · line chart · chart choice · skewness

HANDS-ON PRACTICE Take one dataset and draw the right chart for each row of the chart-choice table, each with a takeaway title.

COMMON MISTAKES Pie charts with many slices · Truncated bar-chart axes · Dual axes that mislead

CHECK YOURSELF

1. Show how revenue split across four regions changed over twelve months. Chart?

A line chart with one line per region, or a stacked area if the total matters; not twelve pie charts.

2. Why must bar charts start at zero?

Bar length encodes value; truncating the axis exaggerates small differences and misleads.

3. Which chart for a skewed distribution with outliers, compared across 3 groups?

Side-by-side box plots (or violin plots), possibly on a log scale.

3.7 The EDA template and self-check

IN PLAIN WORDS

Exploring a dataset is like moving into a new house: before decorating you walk through every room. What is one row exactly? Are any rooms empty (missing values)? Is anything strange (a door that is 10 metres tall, an outlier)? Do the rooms connect the way you expect (joins and relationships)? Only after the walk-through do you decide what to build. Good analysts write down what they found and the questions it raised.

Why it matters: Case rounds and take-home tasks judge your process as much as your result.

Takeaway: Understand the grain, check quality, look at each variable alone and in pairs, segment, and note hypotheses.

WORKED EXAMPLE, STEP BY STEP**A first look at six rows**

A sample from a customer file:

id	age	income	city	signup
1	34	52,000	Pune	2024-03-01
2	-1	48,000	pune	2024-03-04
3	29		Delhi	2024-03-04
3	29		Delhi	2024-03-04
4	41	9,900,000	Mumbai	04/03/2024
5	23	31,000	Delhi	2024-03-09

- 1. Shape and grain.** Is one row one customer? id 3 appears twice with identical values: a **duplicate** row. `df.duplicated().sum()` = 1.
- 2. Types.** `signup` mixes two formats (2024-03-01 and 04/03/2024), so pandas reads it as text. Convert with `pd.to_datetime` and check what fails.
- 3. Impossible values.** Age -1 is not a real age: probably a placeholder for "unknown". Treat it as missing, and ask the data owner.
- 4. Missing values.** id 3 has no income. Ask why before filling: did people refuse, or does the form skip the question for some users?
- 5. Outliers.** An income of 9,900,000 may be a typo (an extra zero, or monthly versus yearly) or genuinely high. Check before deciding; do not just delete it.
- 6. Inconsistent categories.** "Pune" and "pune" are the same city. Standardise with `.str.strip().str.title()`.
- 7. Write down what you found** and what you did. That note is the first slide of any analysis.

What this shows. Six rows contained six different problems. EDA is a checklist you run every time, not a set of nice plots.

YOUR TURN (COVER THE SOLUTION FIRST)

- `df.describe()` shows a `quantity` column with min -3, median 1, max 500, and `df.isna().mean()` shows 40% missing for `discount`. List three things you would check next and why.

Solution. (1) Negative quantity: are these returns or refunds recorded as negative sales? If so, separate them; if not, they are errors. (2) Quantity 500 against a median of 1: a bulk or business order, or a typo? Look at the rows and decide whether they belong in the analysis. (3) 40% missing discount: does missing mean "no discount" (then fill with 0 and keep a flag), or "not recorded"? Check whether missingness depends on date or channel, which would suggest a logging change.

THE FUNDAMENTALS**3.7 EDA template (memorize this workflow)**

- 1. Understand the problem and data dictionary;** know the grain (one row = what?).

2. **Shape, dtypes, memory, duplicates.**
3. **Missingness:** how much, which columns, MCAR/MAR/MNAR, patterns.
4. **Univariate:** distributions, skew, outliers, cardinality.
5. **Bivariate:** feature vs target, feature vs feature (correlation, VIF).
6. **Segment analysis:** by time, geography, cohort.
7. **Data quality checks:** impossible values, unit mismatches, leakage candidates.
8. **Hypotheses:** what to engineer, what to test.

3.8 Self-check (Module 3)

- ☐ Translate 5 SQL window queries into pandas without help.
- ☐ Explain broadcasting with an example.
- ☐ Do a full EDA on a new Kaggle dataset in 45 minutes and summarize 5 insights.

SAY IT IN 60 SECONDS

"EDA is a short, repeatable routine before modelling. One: shape, dtypes, a few rows. Two: missing values, by column and by pattern. Three: duplicates and impossible values, like negative ages. Four: distributions of each variable, noting skew and outliers. Five: relationships, such as target against each feature and correlations. Six: segment checks, like time trends and group differences. Seven: write down the findings and next steps. For each odd finding I ask whether it is a data error, a real effect or a logging change. The catch: correlation in EDA is a lead for hypotheses, not proof, and I never look at the test set when deciding what to engineer."

KEY TERMS exploratory data analysis · grain · missingness · MCAR MAR MNAR · outliers · leakage · segment analysis · data quality

BUILDS ON 3.2 pandas checklist

HANDS-ON PRACTICE Run the eight steps on a new Kaggle dataset in 45 minutes and write five insights with one chart each.

COMMON MISTAKES Jumping to modelling before understanding the grain · Plotting everything without a question · Dropping outliers or missing rows without understanding why they exist

CHECK YOURSELF

1. EDA reveals 12 per cent missing in income. What do you do before imputing?

Check why: random, or related to other variables or the target (for example high earners skip)? The mechanism decides between imputing, adding a missing flag, or fixing collection.

2. Why should EDA decisions use only training data?

Using the test set to choose features or transformations leaks information and makes the test score optimistic.

3. Name three data-quality checks that take under a minute each.

Duplicate rows or keys, impossible values (negative, future dates), and null counts per column.

Interview tips, tricks and hacks

1. **State the output shape first.** "One row per user with their latest order." It tells you whether you need `agg`, `transform`, a sort with `drop_duplicates`, or a pivot.

2. **Translate from SQL in your head.** If you know how you would write it in SQL, say so and translate clause by clause. Interviewers like hearing the mapping.
3. **Use `.loc` for every assignment.** `df.loc[mask, 'col'] = value` always works. Chained indexing such as `df[mask]['col'] = value` may silently modify a copy.
4. **Avoid `apply` with `axis=1` unless you must.** It is a Python loop in disguise. Look for a vectorised form: arithmetic on columns, `np.where`, `.str` and `.dt` methods, `map` with a dictionary.
5. **After every `merge`, check the row count.** Use `validate='one_to_many'` or `indicator=True` when it matters. Duplicate keys multiply rows exactly as in SQL.
6. **Look at the data before computing on it.** `df.info()`, `df.describe()`, `df.isna().mean()` and a few rows. A numeric column stored as text explains many "mysterious" errors.
7. **Know the three top-N recipes.** Sort then `groupby().head(n)`; `groupby().rank()`; `nlargest`. Say which one handles ties the way the question wants.
8. **Narrate your EDA as a checklist.** In an open-ended exploration task, saying "first shape and types, then missing values, then distributions" shows structure even before you find anything.
9. **Give every chart a sentence.** If asked to plot, end with what the chart shows and what you would do next. The insight is the deliverable, not the figure.

Frequently asked questions

30 questions interviewers ask on this chapter, with model answers. Cover the answer, say yours aloud, then compare.

NUMPY ESSENTIALS AND BROADCASTING

Q1 What is the shape of `np.ones((4, 1, 3)) + np.ones((5, 1))`, or does it fail?

Write the shapes right-aligned, padding the shorter with 1s on the left: `(4, 1, 3)` and `(1, 5, 1)`. Compare each pair: 3 and 1 give 3; 1 and 5 give 5; 4 and 1 give 4. The result shape is `(4, 5, 3)`. It would fail if a pair were different and neither were 1 (for example 3 and 2).

Q2 Standardise every column of a matrix `X` (mean 0, standard deviation 1) in one line.

```
Z = (X - X.mean(axis=0)) / X.std(axis=0)
```

`X.mean(axis=0)` has shape `(d,)`; broadcasting subtracts it from every row. Fit the mean and std on the training set only, and reuse them on the test set. For a constant column, `std = 0` would divide by zero, so guard with `np.where(std == 0, 1, std)`.

PANDAS CHECKLIST

Q3 How many rows does `orders.loc[2:4]` return versus `orders.iloc[2:4]` on a default index?

`loc` slices by label and includes both ends, so labels 2, 3 and 4 give **3 rows**. `iloc` slices by position like Python lists and excludes the end, so positions 2 and 3 give **2 rows**. This is a favourite trick question; with a non-default index the two can select completely different rows.

Q4 After `df = pd.DataFrame({"a": [0, 2, 3], "b": [1, 1, 1]})`, what does `df.loc[df.a > 1, "b"] = 0` produce, and why is `df[df.a > 1]["b"] = 0` wrong?

The `.loc` form performs one assignment on the original frame, giving `b = [1, 0, 0]`. The chained form first evaluates `df[df.a > 1]` (which may return a copy), then assigns into that temporary object, so `df` is not updated (and pandas warns or, with Copy-on-Write, does nothing). Always combine the row selector and column selector in a single `.loc`.

SQL TO PANDAS TRANSLATION TABLE

Q5 Translate to pandas: users with more than 2 orders, showing the count.

```
counts = orders.groupby("user_id").size()           # COUNT(*) per user
counts[counts > 2]                                # HAVING COUNT(*) > 2
# user_id
# 1      4
# 2      3
```

or keep the original rows of those users: `orders.groupby("user_id").filter(lambda g: len(g) > 2)`. The first returns one row per qualifying user; the second returns all their order rows.

WORKED PANDAS PROBLEMS

Q6 pandas: Top 3 products by revenue within each category

```
(df.groupby(['category', 'product'], as_index=False)['revenue'].sum()
 .sort_values(['category', 'revenue'], ascending=[True, False])
 .groupby('category').head(3))
```

Q7 pandas: Month-over-month growth

```
m = (df.set_index('order_date')
     .resample('MS')['amount'].sum()
     .to_frame('rev'))
m['mom_pct'] = m['rev'].pct_change() * 100
```

Q8 pandas: 7-day rolling average per user

```
df = df.sort_values(['user_id', 'date'])
df['ma7'] = df.groupby('user_id')['value'].transform(lambda s: s.rolling(7,
min_periods=1).mean())
```

Q9 pandas: Fill missing values with group median

```
df['salary'] = df['salary'].fillna(df.groupby('dept')['salary'].transform('median'))
```

Q10 pandas: Named aggregation

```
df.groupby('city').agg(orders=('order_id', 'nunique'),
                       revenue=('amount', 'sum'),
                       aov=('amount', 'mean'))
```

Q11 pandas: Days since last order per user

```
import pandas as pd, numpy as np

last = df.groupby('user_id')['order_date'].max()
recency = (pd.Timestamp.today() - last).dt.days
```

Q12 pandas: Retention matrix

```
df['cohort'] = df.groupby('user_id')['order_date'].transform('min').dt.to_period('M')
df['period'] = df['order_date'].dt.to_period('M')
df['n'] = (df['period'] - df['cohort']).apply(lambda x: x.n)
cohort = df.groupby(['cohort', 'n'])['user_id'].nunique().unstack(fill_value=0)
retention = cohort.div(cohort[0], axis=0)
```

Q13 pandas: Outliers via IQR

```
q1, q3 = df['x'].quantile([.25, .75]); iqr = q3 - q1
outliers = df[(df['x'] < q1 - 1.5 * iqr) | (df['x'] > q3 + 1.5 * iqr)]
```

Q14 pandas: Explode a list column / split strings

```
df['tag'] = df['tags'].str.split(',')
df = df.explode('tag')
```

Q15 pandas: Merge diagnostics

```
out = a.merge(b, on='id', how='outer', indicator=True)
out['_merge'].value_counts()
```

Q16 pandas: for each user, keep only their latest order.

```
latest = (orders.sort_values(["user_id", "order_date"])
          .groupby("user_id").tail(1))
# equivalent:
latest = orders.sort_values("order_date").drop_duplicates("user_id", keep="last")
# equivalent with idxmax:
latest = orders.loc[orders.groupby("user_id")["order_date"].idxmax()]
```

All three give one row per user (the SQL `ROW_NUMBER() ... DESC = 1` pattern). `idxmax` fails if a group has only NaN dates.

Q17 pandas: share of each user's spend in the total, as a percentage.

```
spend = orders.groupby("user_id")["amount"].sum()
share = (spend / spend.sum() * 100).round(1).sort_values(ascending=False)
# user_id 4: 26.0, 1: 23.7, 2: 17.2 ...
```

Dividing a Series by a scalar total is the pandas version of SQL's `SUM(x) / SUM(SUM(x)) OVER ()`.

Q18 pandas: users who ordered in January but not in February.

```
jan = set(orders.loc[orders.order_date.between("2025-01-01", "2025-01-31"), "user_id"])
feb = set(orders.loc[orders.order_date.between("2025-02-01", "2025-02-28"), "user_id"])
jan - feb          # {4}
```

With sets the logic is obvious; for a large frame use `Series.isin` and boolean masks instead. Note `between` is inclusive, so for timestamps prefer `(d ≥ start) & (d < next_start)`.

CONCEPTUAL PANDAS QUESTIONS**Q19 loc vs iloc ?**

`loc` label-based and inclusive of end label; `iloc` integer-position-based, end exclusive.

Q20 apply vs map vs applymap ?

`map` element-wise on Series; `apply` along axis of DataFrame or on Series; `applymap` (now `DataFrame.map`) element-wise on DataFrame. Vectorized ops beat all three.

Q21 merge vs join vs concat ?

`merge` : SQL-style join on columns; `join` : join on index; `concat` : stack along axis.

Q22 transform vs agg ?

`agg` reduces to one row per group; `transform` returns same-length result aligned to original rows.

Q23 pivot vs pivot_table ?

`pivot` needs unique index/column pairs; `pivot_table` aggregates duplicates.

Q24 How to handle a 10 GB CSV?

`chunksize`, `usecols`, `dtype`, convert to Parquet, use Dask/Polars/DuckDB/PySpark.

Q25 Why is `iterrows` slow?

Python-level loop creating a Series per row; vectorize or use `itertuples`.

Q26 `copy` vs `view`?

Slicing may return a view or a copy; use `.copy()` explicitly; avoid chained indexing.

Q27 You run `df.groupby("user_id").apply(lambda g: g["amount"].sum())` on 10 million rows and it is slow. What do you change?

`apply` with a lambda calls Python once per group and cannot be optimised. The built-in aggregation (`sum`, `mean`, `agg`) runs in optimised Cython, so `df.groupby("user_id")["amount"].sum()` is typically dramatically faster. Reach for `apply` only when no built-in or vectorised method expresses the logic.

VISUALIZATION: CHOOSING THE RIGHT CHART**Q28 A histogram of delivery time shows a long right tail. What does that imply for mean vs median and for what you report?**

A long right tail means a few very long deliveries pull the mean above the median. The median better represents the typical delivery; the P90 or P95 captures the bad experiences that customers remember. Report both, and consider a log scale for the plot. The same logic applies to income, order value and session length.

Q29 Two variables have a correlation of 0. Can you conclude they are unrelated?

Pearson correlation captures **linear** relationships only. If $y = x^2$ with x symmetric around zero, the correlation is 0 even though y is fully determined by x . Plot the data, try Spearman correlation for monotonic relationships, and consider mutual information for general dependence.

THE EDA TEMPLATE AND SELF-CHECK**Q30 You discover 30% of `income` is missing in a loan dataset. What do you do?**

First ask why: did a form field allow skipping it, or do certain applicants (self-employed, high earners) tend to omit it? Check whether missing rows differ in other columns and in the target (default rate). If it looks random, simple median imputation inside a training-only pipeline may be fine; if it depends on observed variables, use group-wise or model-based imputation; if it depends on the value itself (MNAR), the missingness is informative, so add an `income_missing` flag and avoid over-trusting imputed values. Fit imputers on training data only to avoid leakage, and monitor the missing rate in production.

NUMPY ARRAYS

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]]) # shape (2, 3)
np.zeros((2, 3)); np.ones(3); np.eye(3)
np.arange(0, 10, 2); np.linspace(0, 1, 5)
a.shape, a.ndim, a.dtype, a.size
a.reshape(3, 2); a.reshape(-1); a.T
a[0, 1]; a[:, 1]; a[a > 2] # index, column, mask
np.where(a > 2, 1, 0) # vectorised if/else
```

- One typed block of memory; operations are vectorised in C, avoid Python loops.
- Basic slices are **views** (edit the original); fancy and boolean indexing return **copies**. `a.copy()` to be safe.
- `np.random.default_rng(42).normal(size=5)`: always seed.
- `ravel()` is a view when possible, `flatten()` always a copy; check with `np.shares_memory(a, b)`.
- `a[:, None]` / `np.expand_dims` add an axis, `np.squeeze` drops length-1 axes. `np.stack` makes a new axis; `vstack/hstack/column_stack` reuse one. `np.split(a, 2)`.
- `np.argwhere(a > 2)`: one [row, col] per hit. `np.isin(a, [...])`.

AXIS AND AGGREGATION

- `axis=0` collapses rows (one result per column); `axis=1` collapses columns (one per row).
- `a.sum(axis=0)`, `.mean`, `.std(ddof=1)`, `.min/.max`, `.argmax`, `.cumsum`, `np.percentile(a, 90)`, `np.unique(a, return_counts=True)`.
- `keepdims=True` keeps the axis so the result broadcasts back.
- Sorting: `np.sort`, `np.argsort` (indices); `stack`: `np.concatenate`, `np.vstack`, `np.hstack`.
- Linear algebra: `a @ b` (matmul), `np.dot`, `np.linalg.inv`, `.solve`, `.eig`, `.norm`.
- Regression: `np.linalg.lstsq(X, y, rcond=None)[0]` with a ones column in X; safer than solving `X.T @ X`.

TRAP `a * b` is element-wise, `a @ b` is the matrix product: both run on square arrays. `np.std` divides by `n` (`ddof=0`); pandas `.std()` by `n - 1`. Pass `ddof` explicitly.

NUMPY EXTRAS

- Floats: `np.isclose(a, b)`, `np.nan` is a float; `np.nanmean`, `np.nansum`, `np.isnan(a)`; `np.inf`.
- `np.clip(a, lo, hi)`, `np.diff(a)`, `np.cumsum/cumprod`, `np.corrcoef(x, y)`, `np.histogram(a, bins=10)`, `np.searchsorted`.
- Dtypes: ints overflow silently (`int8`), `astype(float)`, `np.float32` halves memory. Mixed lists become object.
- Vectorise, then `np.vectorize` only as a convenience (it still loops). `np.select([c1, c2], [v1, v2], default)` for many branches.
- Random: `rng = np.random.default_rng(0)`; `rng.integers`, `rng.choice(a, size, replace=False)`, `rng.permutation`.

BROADCASTING RULES

Compare shapes from the **right**. Two dimensions are compatible if equal or one is 1 (the 1 is stretched). Missing leading dimensions count as 1.

Shapes	Result
(3, 4) with (4,)	(3, 4) subtract column means
(3, 4) with (3, 1)	(3, 4) subtract row means (use <code>keepdims</code>)
(3, 1) with (1, 4)	(3, 4) outer-style grid
(3, 4) with (3,)	error

Pairwise distances: `np.linalg.norm(x[:, None] - y[None], axis=2)`.

PANDAS: LOOK, SELECT, FILTER

```
import pandas as pd
df = pd.read_csv("f.csv", parse_dates=["d"])
df.shape; df.dtypes; df.head(); df.info()
df.describe(include="all"); df.isna().sum()
df["a"]; df[["a", "b"]] # Series / DataFrame
df.loc[rows, cols] # by label (end included)
df.iloc[0:3, 1:4] # by position (end excluded)
df[(df.a > 1) & (df.b == "x")] # & | ~ with brackets
df.query("a > 1 and b == 'x'")
df.loc[df.a > 1, "b"] = 0 # safe assignment
```

- Never chain `df[mask]["b"] = 0`: it may write to a copy (`SettingWithCopy`). Use `.loc`.
- `df.assign(c=df.a * 2)`, `df.rename(columns=...)`, `df.drop(columns=...)`, `df.astype({"a": "int32"})`.

GROUPBY, AGG, TRANSFORM

```
df.groupby("g")["x"].mean()
df.groupby("g").agg(n=("x", "size"),
                    avg=("x", "mean"))
df.groupby(["g", "h"], as_index=False).sum()
df["share"] = df.x / df.groupby("g")["x"].transform("sum")
df.groupby("g").head(2) # first rows per group
df.groupby("g")["x"].rank(ascending=False)
df.groupby("g")["x"].shift(1) # previous in group
df.groupby("g")["x"].cumsum()
```

- `agg` shrinks each group to one row; `transform` returns one value per original row (same index).
- `size` counts rows, `count` counts non-NaN, `nunique` distinct.

COMBINE AND RESHAPE

```
pd.merge(a, b, on="id", how="left")
a.merge(b, left_on="x", right_on="y")
pd.concat([a, b], axis=0, ignore_index=True)
a.join(b, on="k") # join on index
df.pivot_table(index="r", columns="c",
               values="v", aggfunc="sum")
df.melt(id_vars="id", var_name="k", value_name="v")
df.explode("tags"); df.stack(); df.unstack()
```

- `merge(..., validate="one_to_one")` catches key duplicates; `indicator=True` adds `_merge` (anti-join: `left_only`).
- `how`: `inner` (default), `left`, `right`, `outer`.

MISSING, DUPLICATES, TYPES

```
df.isna().mean() # share missing
df.dropna(subset=["a"]); df.dropna(how="all")
df["a"].fillna(df["a"].median())
df.fillna({"a": 0, "b": "NA"}); df.ffill()
df.duplicated(subset=["id"]).sum()
df.drop_duplicates("id", keep="last")
pd.to_datetime(df.d); pd.to_numeric(s, errors="coerce")
df["c"] = df["c"].astype("category")
```

- `NaN` $\neq$ `NaN`; use `isna()`. Mean skips NaN by default (`skipna=True`).
- Sort before `drop_duplicates(keep="last")` to keep the latest.
- Memory: `category` for low-cardinality strings, `downcast` numerics, read only needed columns (`usecols`), `chunks`.

STRINGS AND DATES

```
df.s.str.lower().str.strip().str.contains("x")
df.s.str.split(",", expand=True); .str.extract(r"(\d+)")
df.d.dt.year; .dt.month; .dt.dayofweek; .dt.floor("D")
df.set_index("d").resample("W").sum()
df.x.rolling(7).mean(); df.x.expanding().max()
df.x.pct_change(); df.x.diff(); df.x.shift(7)
pd.cut(df.x, bins=[0, 18, 65, 120]) # fixed bins
pd.qcut(df.x, 4) # quantile bins
```


SQL -> PANDAS

SQL	pandas
WHERE	boolean mask / query
SELECT a, b	df[["a", "b"]]
GROUP BY ... AVG	groupby().agg()
HAVING	filter the grouped result
JOIN	merge(how=...)
ORDER BY ... LIMIT	sort_values().head()
COUNT(DISTINCT x)	nunique()
ROW_NUMBER	groupby().cumcount() + 1
RANK	groupby().rank(method="min")
LAG	groupby().shift(1)
SUM() OVER	groupby().transform("sum")
CASE WHEN	np.where / np.select
UNION ALL	pd.concat
DISTINCT	drop_duplicates()
anti-join	merge(indicator=True) then left_only

WORKED PATTERNS

```
# top 2 per group
df.sort_values("x", ascending=False).groupby("g").head(2)
# latest row per user
df.sort_values("ts").drop_duplicates("u", keep="last")
# conversion by group
df.groupby("g").agg(rate=("conv", "mean"))
# streak id: new id when the value changes
s = (df.v != df.v.shift()).cumsum()
# cohort retention table
df["m"] = df.d.dt.to_period("M")
first = df.groupby("u").m.transform("min")
df["age"] = (df.m - first).apply(lambda p: p.n)
df.pivot_table(index=first, columns="age",
                values="u", aggfunc="nunique")
```

State the output grain first; build in steps and print after each.

INDEX, APPLY, WINDOW HELPERS

- df.set_index("k"), reset_index(), sort_index(); label slicing needs a sorted index. df.loc["a":"c"] includes "c".
- Series.map({...}) / .map(func) elementwise; df.apply(func, axis=0/1) per column / row; applymap/DataFrame.map per cell.
- df.nlargest(3, "x"), df.sample(frac=.2, random_state=0), df.clip(lower, upper), df.where(cond, other).
- Correlation: df.corr(numeric_only=True) (Pearson), df.corr(method="spearman"), df.cov().
- Counts: s.value_counts(normalize=True), pd.crosstab(a, b), s.nunique().
- Split data: from sklearn.model_selection import train_test_split; train_test_split(X, y, test_size=.2, stratify=y, random_state=0).
- Display: pd.set_option("display.max_columns", 50); save with df.to_csv("o.csv", index=False), to_parquet.

CLEAN, ENCODE, RESHAPE: EXTRAS

```
pd.to_datetime(s, format="%Y-%m-%d", errors="coerce")
s.str.extract(r"([A-Z]+)-(\d+)") # groups → columns
s.str.contains("x", case=False, na=False)
s.dt.tz_localize("UTC").dt.tz_convert("Asia/Kolkata")
pd.get_dummies(df, columns=["c"], drop_first=True)
s.ffill(); s.bfill(); s.interpolate() # ordered data
df.dropna(axis=1, thresh=int(.5 * len(df)))
df.reindex(labels) # missing labels → NaN
df.query("a > @limit") # @ reads a variable
pd.read_sql(query, conn); df.to_sql("t", conn)
pd.crosstab(a, b, normalize="index", margins=True)
s.ewm(span=7).mean(); s.cummax(); s.shift(-1) # lead
df.pipe(func, arg) # own step in a chain
df.memory_usage(deep=True).sum()
```

- errors="coerce" turns bad values into NaN / NaT: count them afterwards. astype fails on the first bad value.
- pivot only reshapes (duplicate pairs raise); pivot_table aggregates; melt goes wide to long.
- Stacked series: groupby("g")["x"].shift(1), never a plain shift. A lead (shift(-1)) is a target, not a feature: leakage. bfill and interpolate also look ahead.
- Speed, fastest first: NumPy arrays, pandas vectorised, np.where/np.select, comprehension over zip(df.a, df.b), itertuples, apply(axis=1), iterrows (thousands of times slower).
- category pays when distinct values are far fewer than rows (a 200k-row table: 24.5 MB to 2.1 MB).

SPEED AND GOTCHAS

- Prefer vectorised column ops; apply(axis=1) is a Python loop (slow). np.where, .str, .dt, map(dict) are faster.
 - iterrows is slow; itertuples is faster; vectorise if you can.
 - inplace=True rarely saves memory; assign back instead.
 - = on a column of strings with NaN gives False for NaN. df.a.isin([...]) for membership.
 - df.copy() before modifying a slice you keep.
- TRAP** After a merge, compare row counts: duplicate keys fan out rows. groupby drops NaN keys by default (dropna=False to keep). Integer columns with NaN become float.

CHART CHOICE

Question	Chart
Compare categories	bar (start at zero)
Trend over time	line
Distribution of one variable	histogram, box, violin
Relationship of two numbers	scatter (+ trend line)
Many numeric pairs	correlation heatmap, pair plot
Part of a whole	stacked bar (pie only for 2-3 parts)
Ranking	sorted horizontal bar

```
import matplotlib.pyplot as plt, seaborn as sns
fig, ax = plt.subplots(figsize=(6, 3))
sns.histplot(df.x, bins=30, ax=ax)
sns.boxplot(data=df, x="g", y="v")
sns.heatmap(df.corr(numeric_only=True), annot=True)
ax.set(title="Takeaway as title", xlabel="x (unit)")

fig, axs = plt.subplots(1, 2, figsize=(8, 3))
sns.scatterplot(data=df, x="a", y="b", hue="g")
ax.set_yscale("log"); ax.tick_params(axis="x", rotation=45)
fig.tight_layout(); fig.savefig("f.png", dpi=150)
```

Label axes with units, one colour unless colour means something, avoid dual axes and 3-D, put the takeaway in the title, log scale for skewed data.

EDA ROUTINE

- Shape, dtypes, a few rows, memory.
- Missing values by column and by pattern: why missing?
- Duplicates and impossible values (negative age, future date).
- Each variable's distribution: skew, outliers, rare categories.
- Target versus each feature; correlations (leads, not proof).
- Segments and time trends; check data drift.
- Write findings and next steps. Decide on training data only.

SAY IT IN 20 SECONDS

- "transform keeps the rows, agg collapses them."
- "Broadcasting aligns shapes from the right; 1 stretches."
- "loc is label-based and inclusive, iloc positional and exclusive."
- "Slices are views, boolean masks are copies."

4

CHAPTER 4 · MEDIUM PRIORITY · MODULE 4 IN THE APP

Data Structures and Algorithms (light)

A light, practical dose of data structures and algorithms: enough to pass the coding filter with calm, pattern-based solutions.

3 topics · about 1 h 5 min in the app · 15 questions answered · two-page cheat sheet at the end

4.1 DSA topics to cover

4.3 Targets and self-check

4.2 Pattern templates: sliding window to BFS

Foundations

Data scientists are not expected to be competitive programmers, yet many companies, especially large technology centres, include one coding round. The bar is easy to medium problems, solved in ten to twenty minutes, with a clear explanation. That bar is reachable with a small toolkit, provided you study the right thing. The right thing is not hundreds of problems. It is one idea about cost, and about eight patterns.

The idea of cost

An algorithm's cost is described by how the number of steps grows as the input grows, written in **Big-O** notation. We ignore constants and keep the dominant term, because for large inputs only the growth rate matters.

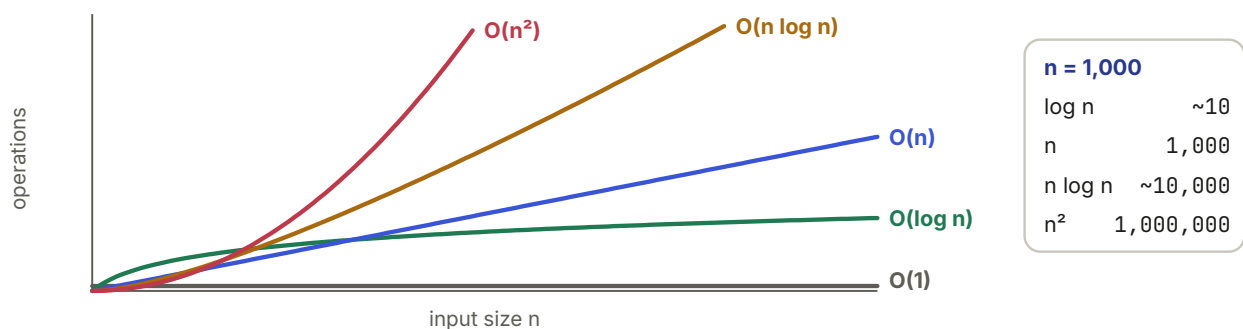


Figure 6. For a thousand items, a quadratic algorithm does a hundred times the work of an $n \log n$ one.

In practice you need to recognise five classes. Constant, $O(1)$: looking up a key in a dictionary. Logarithmic, $O(\log n)$: binary search, which halves the problem each step. Linear, $O(n)$: one pass over the data. $O(n \log n)$: sorting. Quadratic, $O(n^2)$: a loop inside a loop. Most interview improvements are a move from quadratic to linear or $n \log n$, and almost always the move is made the same way: trade memory for time by storing what you have seen in a hash map, or sort first so that structure does the work.

Structures are promises about cost

A data structure is a way of storing data that makes certain operations cheap. An **array** (a Python list) gives instant access by position. A **hash map** (a dictionary) and a **hash set** give instant lookup by key. A **stack** returns the most recent item first and a **queue** the oldest first. A **heap** always gives you the smallest item quickly, which makes it the tool for "top k". **Trees** and **graphs** model hierarchy and connection, and you explore them depth first with recursion or breadth first with a queue. You rarely build these yourself; you choose among them.

Patterns, and the words that reveal them

Problems announce their pattern through their wording, and learning these cues is the most efficient preparation there is.

- "Find a pair", "have we seen", "count occurrences": a **hash map**.
- "Sorted array", "pair that sums to": **two pointers** moving inward.
- "Longest or shortest substring or subarray such that": a **sliding window** that grows on the right and shrinks on the left.
- "Sorted", or "the minimum value that works": **binary search**.
- "Matching brackets", "next greater element": a **stack**.
- "Fewest steps", "shortest path", "level by level": **breadth-first search**.
- "All paths", "connected groups", anything on a tree: **depth-first search**.
- "Top k", "k-th largest": a **heap**.
- "Number of ways", "minimum cost with choices": **dynamic programming**, which is recursion that remembers answers to sub-problems.

What "prepared" looks like

You are prepared when you can read a new problem, name its pattern from the wording, write the template from memory, adapt it, and state the time and space cost. Fifty to eighty problems solved and then re-solved from a blank page a week later will get you there. Three hundred problems glanced at once will not, because recognising a solution is far easier than producing one.

Topic by topic

Each topic gives the idea in plain words, the fundamentals to know, a model spoken answer and three questions to test yourself. Open the same topic in the app for worked examples, runnable code and interactive demos.

4.1 DSA topics to cover

IN PLAIN WORDS

Data structures are containers and algorithms are recipes for using them. A hash map is a labelled cupboard for instant lookups. A stack is a pile of plates (last in, first out). A queue is a line at the canteen (first in, first out). A heap is a pile that always keeps the biggest (or smallest) thing on top. A tree is a family tree, and a graph is a map of friends. Most interview problems are old recipes in new costumes, so learn the patterns.

Why it matters: Bigger companies (Amazon, Microsoft, Walmart, Goldman, Adobe) filter on DSA even for data roles; analytics firms and IT services ask much less.

Takeaway: Know a dozen patterns well, and always state time and space complexity.

WORKED EXAMPLE, STEP BY STEP

Keep the top 3 scores from a stream with a heap

Scores arrive one at a time: 5, 1, 9, 3, 7, 8. Keep the three largest seen so far, using little memory.

- 1. Choose the structure.** A **min-heap of size 3**: it always shows its smallest item first, and that item is the one to throw out when something better arrives.
- 2. Trace it** (push the score; if the heap has more than 3 items, pop the smallest):

score	heap after the step	thrown out
5	[5]	
1	[1, 5]	
9	[1, 5, 9]	
3	[3, 5, 9]	1
7	[5, 7, 9]	3
8	[7, 8, 9]	5

- 3. Answer:** the top three are 7, 8 and 9, and the smallest of them (7) sits at the front of the heap.
- 4. Cost.** Each push or pop is $O(\log k)$ with $k = 3$, so the stream costs $O(n \log k)$, and memory stays at k items however long the stream is.
- 5. Compare.** Sorting everything is $O(n \log n)$ and needs all n items in memory, which is impossible for an endless stream.

```
import heapq
h = []
for x in scores:
    heapq.heappush(h, x)
    if len(h) > 3:
        heapq.heappop(h)
```

What this shows. Pick a data structure by the operation you need to be cheap. Here it is "remove the smallest of my current best".

YOUR TURN (COVER THE SOLUTION FIRST)

1. Pick a data structure for each and give a one-line reason: (a) an editor's undo button; (b) a printer queue served in arrival order; (c) "have I seen this user id before?" across a million ids; (d) the 10 most frequent words in a book.

Solution. (a) A stack: the last action is undone first. (b) A queue (`collections.deque`): first in, first out, with $O(1)$ at both ends. (c) A set: $O(1)$ average membership checks. (d) A dictionary of counts (`Counter`), then a heap or `most_common(10)` to pick the top 10: $O(n)$ to count plus $O(m \log 10)$ to select.

THE FUNDAMENTALS

- ☐ Big-O time and space analysis
- ☐ Arrays/strings, hash maps/sets (most important)
- ☐ Two pointers, sliding window, prefix sums
- ☐ Sorting (merge, quick conceptually), binary search (and on answer)
- ☐ Stack, queue, deque, heap/priority queue (`heapq`)
- ☐ Linked list basics (reverse, detect cycle)
- ☐ Trees: traversal, BFS/DFS, height; BST basics
- ☐ Graphs: BFS/DFS, connected components
- ☐ Recursion, basic dynamic programming (climbing stairs, coin change, LCS)
- ☐ Intervals (merge intervals), top-K problems

SAY IT IN 60 SECONDS

"For data science interviews, DSA means the easy-to-medium core. Arrays and strings with hash maps for counting and lookup. Two pointers and sliding window for subarray problems. Stacks and queues, sorting and binary search. Trees and graphs with DFS and BFS. Basic dynamic programming and heaps for top K. I do not need advanced graph theory or competitive tricks. Hash maps matter most because they turn many nested loops into one pass, going from quadratic to linear time. The catch: I state time and space complexity out loud for every solution, since interviewers always ask."

KEY TERMS big-O · hash map · two pointers · sliding window · prefix sums · binary search · stack · queue · heap · recursion · dynamic programming · BFS · DFS

HANDS-ON PRACTICE For each topic in the list, write one sentence on when to use it and one tiny example.

COMMON MISTAKES Memorising solutions instead of patterns · Ignoring time and space complexity · Skipping the easy/medium basics for hard problems

CHECK YOURSELF**1. Two-sum in $O(n)$: how?**

One pass with a hash map from value to index: for each x , check whether $\text{target} - x$ is already stored.

2. Which structure gives top K of a stream efficiently?

A min-heap of size K : $O(n \log K)$.

3. Why does a hash map usually improve a brute-force solution?

Lookup is $O(1)$ on average, replacing an inner loop that searches for a match.

4.2 Pattern templates: sliding window to BFS

IN PLAIN WORDS

Each template is a reusable trick. A **sliding window** is like a magnifying glass moving along a line of letters: stretch the right edge to include more, and squeeze the left edge when the rule is broken. **Binary search** is the guess-a-number game: always check the middle and throw away half. **Merge intervals**: sort meetings by start time and glue together the ones that overlap. **BFS** spreads out like ripples in a pond, visiting everything one step away, then two steps away. **Prefix sums** keep a running total so any stretch's total is a subtraction.

Why it matters: Recognising the pattern is 80% of solving the problem.

Takeaway: Learn the template, then practise spotting which problems fit it.

WORKED EXAMPLE, STEP BY STEP**A sliding window, traced**

Longest substring without a repeated character in "abcabcbb".

- 1. The idea.** Keep a window `[left, right]` with no repeats. Move `right` one step at a time; when the new character already appears inside the window, jump `left` just past its last position.
- 2. Remember the last position of each character** in a dictionary.
- 3. Trace it.**

right	char	left after the step	window	best
0	a	0	a	1
1	b	0	ab	2
2	c	0	abc	3
3	a	1	bca	3
4	b	2	cab	3
5	c	3	abc	3
6	b	5	cb	3
7	b	7	b	3

4. Answer: 3 (for example "abc").

5. Cost. Each pointer only moves forward, so $O(n)$ time, against $O(n^2)$ or worse for checking every substring.

What this shows. Words such as "longest", "shortest", "substring" or "subarray" with a condition point to a sliding window: grow on the right, shrink on the left, never move backwards.

YOUR TURN (COVER THE SOLUTION FIRST)

1. In the sorted list `[1, 3, 4, 6, 8, 11]`, find two numbers that add to 10 using two pointers. Show each step.

Solution. Start with the pointers at the ends: $1 + 11 = 12$, too big, so move the right pointer left. $1 + 8 = 9$, too small, so move the left pointer right. $3 + 8 = 11$, too big, move right. $3 + 6 = 9$, too small, move left. $4 + 6 = 10$: found. Each step discards one number for good, so it is $O(n)$, and it works only because the list is sorted.

SAY IT IN 60 SECONDS

"Most easy and medium problems fit a handful of templates. Sliding window: keep a window over a sequence, expanding right and shrinking left while a condition fails, for longest substring or subarray questions. Two pointers: from both ends of a sorted array, for pair sums. Hash map: counts, seen sets, complements. Stack: matching brackets and next greater element. Binary search: on a sorted array or on an answer range. BFS: shortest path in an unweighted graph using a queue. DFS: explore all paths or components by recursion. The catch: I first name the pattern from the wording, then code it, then test on an empty input, a single element and duplicates."

KEY TERMS sliding window · binary search · heap · Counter · merge intervals · BFS · prefix sum · hash map of prefixes

BUILDS ON 4.1 DSA topics to cover · 1.6 Coding patterns: arrays and hash maps

HANDS-ON PRACTICE Solve each template from memory, then solve one new problem of the same pattern within 25 minutes.

COMMON MISTAKES Not handling empty input or single-element cases · Off-by-one errors in binary search bounds · Forgetting to sort intervals first

CHECK YOURSELF

1. **Longest substring without repeating characters: which pattern and key idea?**

Sliding window with a set or last-seen map; move the left edge past the duplicate when it appears. $O(n)$.

2. **Why BFS rather than DFS for shortest path in an unweighted graph?**

BFS visits nodes in increasing distance, so the first time it reaches the target is the shortest path.

3. **Valid brackets "([])": which structure and result?**

A stack; push openers, and each closer must match the top. "`([])`" fails because `)` meets `[`.

4.3 Targets and self-check

IN PLAIN WORDS

DSA for data science is a small, learnable target. You are not training for competitive programming; you want to solve easy and medium problems calmly, in about twenty minutes, and explain your reasoning. That means recognising roughly eight patterns, writing clean code in your main language, and always stating time and space cost. A realistic aim is around fifty to eighty well-chosen problems solved and then re-solved from memory, not hundreds seen once. Check yourself by solving a fresh problem without hints while talking aloud.

Why it matters: DSA is usually a pass-or-fail filter; calm, pattern-based solutions beat clever ones.

Takeaway: Learn the patterns, practise aloud, and state complexity every time.

WORKED EXAMPLE, STEP BY STEP

A four-week DSA plan that sticks

Target: solve easy problems in about 10 minutes and medium ones in about 20, out loud.

Time available: 45 minutes a day.

1. **Week 1, hash maps and two pointers.** Two easy problems a day. Write the pattern name next to each solution.
2. **Week 2, sliding window, stacks, binary search.** One easy and one medium a day.
3. **Week 3, BFS, DFS, heaps, intervals.** One medium a day, plus re-solving.
4. **Week 4, mixed and timed.** Random problems against the clock, explaining aloud, plus re-solving.
5. **The re-solve rule.** Any problem you needed help with goes on a list. Re-solve it from a blank page 3 days later and again 10 days later. If it fails, it stays on the list.
6. **Track two numbers each week:** problems solved without help, and minutes per medium problem.

Week	New problems	Re-solves	Goal by Sunday
1	14 easy	0	name the pattern in under a minute
2	7 easy, 7 medium	about 5	medium in 30 minutes
3	7 medium	about 8	medium in 25 minutes
4	7 mixed	about 10	medium in 20 minutes, spoken

What this shows. About 50 problems, each solved twice, beats 200 seen once. The re-solve list is where the learning happens.

YOUR TURN (COVER THE SOLUTION FIRST)

1. On Monday you solved "valid parentheses" only after reading a hint, and "two sum" on your own. On Thursday you re-solve "valid parentheses" cleanly in 8 minutes. What goes on your schedule next?

Solution. "Two sum" needs no re-solve, though it can appear in a mixed timed set later. "Valid parentheses" passed its 3-day re-solve, so schedule the 10-day re-solve (about 10 days after Monday). If that also passes, it leaves the list. Log the hint you needed ("stack: each closer must match the top") in your pattern notes.

THE FUNDAMENTALS**4.3 Target**

Level	Count
LeetCode Easy	50
LeetCode Medium	30 (focus: arrays, strings, hash maps, two pointers, sliding window, trees, BFS/DFS)

4.4 Self-check

- ☐ Explain the time complexity of every solution you write.
- ☐ Solve a medium hash-map or sliding-window problem in 25 minutes.

SAY IT IN 60 SECONDS

"My DSA target is modest. I want to solve easy problems in about ten minutes and medium ones in about twenty, while explaining as I go. That needs the core patterns, a clean solution in one language, and the habit of stating time and space complexity. I measure myself by solving unseen problems without hints, aloud, and by re-solving old ones from memory a week later. Fifty to eighty well-understood problems beat three hundred skimmed ones. The catch: if I cannot explain why my solution works and what it costs, I have not learned it yet, so I stop collecting problems and review."

KEY TERMS LeetCode targets · complexity explanation · time-boxing

HANDS-ON PRACTICE Solve one medium hash-map or sliding-window problem in 25 minutes, and explain the complexity aloud.

COMMON MISTAKES Solving without explaining complexity · Skipping review of failed problems

CHECK YOURSELF**1. How do you know a DSA problem is truly learned?**

You can re-solve it from a blank page a week later, explain why it works, and state its complexity.

2. What should you do in an interview before coding a DSA problem?

Clarify the input and edge cases, propose a brute force, then the optimised approach with its complexity, and only then code.

3. Why is volume of problems a poor target?

Seeing many problems once builds recognition without recall; re-solving fewer problems builds the pattern skill the interview tests.

Interview tips, tricks and hacks

- 1. Follow a fixed script.** Clarify inputs and edge cases, describe a brute force and its cost, propose the better approach and its cost, code it, trace an example, state the final complexity. The script keeps you calm and earns marks at every step.
- 2. Name the pattern out loud.** "This is a longest-substring-with-a-condition problem, so I will use a sliding window." The interviewer now knows you are not guessing.
- 3. Ask about input size.** If n can be a hundred thousand, quadratic will not pass and you need $O(n)$ or $O(n \log n)$. If n is at most twenty, exploring all subsets is fine.
- 4. Reach for a hash map before anything else.** It solves two-sum, anagram grouping, first non-repeating character, subarray-sum counts and many more.
- 5. Write the edge cases on the board first.** Empty input, a single element, all equal elements, negatives. Test them at the end.
- 6. Learn binary search once, precisely.** Decide on one template (for example "first index where the condition is true") and use it every time. Most bugs come from mixing variants.
- 7. In breadth-first search, mark visited when you add to the queue.** Marking when you remove causes duplicates and time-outs.
- 8. If you cannot find the optimal solution, code the brute force cleanly.** A correct slow solution with a discussion of how to speed it up beats an incorrect clever one.
- 9. Re-solve, do not re-read.** Schedule every problem you needed help with for a blank-page attempt a week later.

Frequently asked questions

15 questions interviewers ask on this chapter, with model answers. Cover the answer, say yours aloud, then compare.

DSA TOPICS TO COVER

Q1 You need the 10 largest values from a stream of 10 million numbers that does not fit in memory. What do you use?

Keep a **min-heap** of the 10 best seen so far. For each new number, if the heap has fewer than 10 items push it; otherwise compare with the heap's smallest (the root) and replace it only if the new number is larger. Each step is $O(\log 10)$, so the whole pass is $O(n \log k)$ time and $O(k)$ memory. Sorting the whole stream would be $O(n \log n)$ time and need all the data in memory.

PATTERN TEMPLATES: SLIDING WINDOW TO BFS

Q2 Implement: Sliding window: longest substring without repeating characters

```
def longest_unique(s):
    seen, l, best = {}, 0, 0
    for r, ch in enumerate(s):
        if ch in seen and seen[ch] ≥ l:
            l = seen[ch] + 1
        seen[ch] = r
        best = max(best, r - l + 1)
    return best
```


Q3 Implement: Binary search

```
def bsearch(a, x):
    lo, hi = 0, len(a) - 1
    while lo ≤ hi:
        mid = (lo + hi) // 2
        if a[mid] == x: return mid
        if a[mid] < x: lo = mid + 1
        else: hi = mid - 1
    return -1
```

Q4 Implement: Top-K frequent elements

```
import heapq
from collections import Counter
def top_k(nums, k):
    return [x for x, _ in Counter(nums).most_common(k)]
```

Q5 Implement: Merge intervals

```
def merge_intervals(iv):
    iv.sort()
    out = [iv[0]]
    for s, e in iv[1:]:
        if s ≤ out[-1][1]: out[-1][1] = max(out[-1][1], e)
        else: out.append([s, e])
    return out
```

Q6 Implement: BFS on a graph

```
from collections import deque
def bfs(graph, start):
    q, seen = deque([start]), {start}
    while q:
        node = q.popleft()
        for nb in graph[node]:
            if nb not in seen:
                seen.add(nb); q.append(nb)
    return seen
```

Q7 Implement: Prefix sum: subarray sum equals k

```
def subarray_sum(nums, k):
    cnt, cur, res = {0: 1}, 0, 0
    for n in nums:
        cur += n
        res += cnt.get(cur - k, 0)
        cnt[cur] = cnt.get(cur, 0) + 1
    return res
```

Q8 Implement: reverse a singly linked list.

```
class Node:
    def __init__(self, val, next=None):
        self.val, self.next = val, next

def reverse_list(head):
    prev = None
    while head:
        head.next, prev, head = prev, head, head.next    # rewire, then advance
    return prev                                           # the old tail is the new head
```

$O(n)$ time, $O(1)$ space. The tuple assignment evaluates the right side first, so `head.next` is saved before being overwritten. `1 → 2 → 3` becomes `3 → 2 → 1`.

Q9 Implement: detect a cycle in a linked list.

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next      # tortoise moves 1, hare moves 2
        if slow is fast:
            return True
    return False
```

$O(n)$ time, $O(1)$ space (a `set` of visited nodes works too but uses $O(n)$ memory). If there is a loop the hare eventually laps the tortoise.

Q10 Implement: level-order (BFS) traversal of a binary tree, returning a list of levels.

```
from collections import deque

def level_order(root):
    if not root:
        return []
    out, q = [], deque([root])
    while q:
        level = []
        for _ in range(len(q)):                # the number of nodes in the current level
            node = q.popleft()
            level.append(node.v)
            if node.l: q.append(node.l)
            if node.r: q.append(node.r)
        out.append(level)
    return out

# tree: 3 with children 9 and 20; 20 has children 15 and 7 → [[3], [9, 20], [15, 7]]
```

$O(n)$ time, $O(\text{width})$ space. The tree height is a one-line recursion: `0 if not n else 1 + max(height(n.l), height(n.r))` (3 for this tree).

Q11 Implement: number of distinct ways to climb `n` stairs taking 1 or 2 steps at a time (dynamic programming).

```
def climb(n):
    a, b = 1, 1                # ways(0), ways(1)
    for _ in range(n - 1):
        a, b = b, a + b
    return b

[climb(i) for i in range(1, 7)]    # [1, 2, 3, 5, 8, 13]
```

To reach step `n` you came from step `n-1` (one step) or `n-2` (two steps), so the counts add. $O(n)$ time, $O(1)$ space. The DP recipe: define the state, write the recurrence, set base cases, choose the order.

Q12 Implement: minimum number of coins to make an amount (coin change), or -1 if impossible.

```
def coin_change(coins, amount):
    INF = float("inf")
    dp = [0] + [INF] * amount          # dp[a] = fewest coins for amount a
    for a in range(1, amount + 1):
        for c in coins:
            if c ≤ a:
                dp[a] = min(dp[a], dp[a - c] + 1)
    return dp[amount] if dp[amount] ≠ INF else -1

coin_change([1, 2, 5], 11)    # 3    (5 + 5 + 1)
coin_change([2], 3)          # -1
```

$O(\text{amount} \times \text{coins})$ time, $O(\text{amount})$ space. A greedy "largest coin first" approach fails for some coin systems, which is why DP is needed.

Q13 Implement: the k-th largest element of an array using a heap.

```
import heapq

def kth_largest(nums, k):
    h = []
    for x in nums:
        heapq.heappush(h, x)
        if len(h) > k:
            heapq.heappop(h)          # drop the smallest, keeping the k largest
    return h[0]

kth_largest([3, 2, 1, 5, 6, 4], 2)  # 5
```

$O(n \log k)$ time, $O(k)$ space. `heapq.nlargest(k, nums)[-1]` is the library shortcut; quickselect gives $O(n)$ average.

Q14 Implement: count the islands in a grid of "1" (land) and "0" (water) with DFS.

```
def num_islands(grid):
    R, C, seen, count = len(grid), len(grid[0]), set(), 0

    def dfs(r, c):
        if not (0 ≤ r < R and 0 ≤ c < C) or grid[r][c] ≠ "1" or (r, c) in seen:
            return
        seen.add((r, c))
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            dfs(r + dr, c + dc)

    for r in range(R):
        for c in range(C):
            if grid[r][c] == "1" and (r, c) not in seen:
                count += 1
                dfs(r, c)
    return count

num_islands(["11000", "11000", "00100", "00011"])  # 3
```

$O(R \cdot C)$ time. Python's recursion limit (about 1000) can be hit on big grids; use an explicit stack or BFS when the grid is large.

TARGETS AND SELF-CHECK

Q15 You can solve a problem in $O(n^2)$ time with $O(1)$ space or in $O(n)$ time with $O(n)$ space. Which do you choose?

It depends on the constraints, so ask. For n around 10^5 or larger the $O(n^2)$ approach is too slow, so take $O(n)$ time with $O(n)$ extra memory. If memory is very tight or n is small, the $O(1)$ -space version may be better. State both trade-offs, pick with a reason, and mention that you would measure if it mattered in practice.

BIG-O QUICK REFERENCE

Class	Name	Typical example
O(1)	constant	dict lookup, array index
O(log n)	logarithmic	binary search, heap push/pop
O(n)	linear	one pass, hash-map solutions
O(n log n)	linearithmic	sorting, merge sort, heap of n
O(n²)	quadratic	nested loops, bubble sort
O(2^n), O(n!)	exponential, factorial	subsets, permutations

- n = 10^5 needs O(n log n) or better; n = 10^3 allows O(n²); n <= 20 allows O(2^n).
- Drop constants and lower-order terms; state **time and space** separately; recursion uses stack space O(depth).
- Amortised O(1): dynamic array append, hash-table insert.

DATA STRUCTURES: WHAT EACH IS GOOD AT

Structure	Access	Search	Insert/delete	Use for
array / list	O(1)	O(n)	O(n) middle, O(1) end	indexed data
hash map / set		O(1) avg	O(1) avg	counting, lookup, dedupe
linked list	O(n)	O(n)	O(1) at node	fast splice, queues
stack (list)	top O(1)		push/pop O(1)	undo, parsing, DFS
queue (deque)			O(1) both ends	BFS, windows
heap (heapq)	min O(1)		O(log n)	top-k, scheduling
BST balanced		O(log n)	O(log n)	ordered data
trie		O(len)	O(len)	prefix search
graph (adj. list)				networks, dependencies

- Sorting: Python `sorted` is Timsort, O(n log n), stable. Merge sort O(n log n) stable; quicksort O(n log n) average, O(n²) worst; counting sort O(n + k).

CLUE -> PATTERN

If the problem says	Think
pair / complement / seen before / count	hash map or set
sorted array, pair or triplet sum	two pointers
longest / shortest substring or subarray with a rule	sliding window
sum of a range, many queries	prefix sums
sorted, or "minimum value such that"	binary search (on answer)
matching brackets, next greater element	stack, monotonic stack
shortest path, levels, fewest steps	BFS
all paths, components, tree recursion	DFS
top k, k-th largest, merge k sorted	heap
overlapping intervals	sort by start, then merge
count ways, min cost, choices at each step	dynamic programming
all subsets, permutations, combinations	backtracking
connected groups, cycle in undirected graph	union-find

INTERVIEW ROUTINE

1. Restate; ask about input size, duplicates, negatives, empty input, sorted?
 2. Say a brute force and its cost; then improve (hash map? sort? pointers?).
 3. State the plan; code; **trace** on a small example.
 4. Edge cases: empty, one element, all same, duplicates, negatives, large n.
 5. State time and space. Target: easy in ~10 minutes, medium in ~20.
- Learn ~8 patterns well; re-solve problems from a blank page a week later.

PYTHON HELPERS FOR DSA

```
from collections import Counter, defaultdict, deque
import heapq, bisect, math
float("inf"); math.inf; sys.setrecursionlimit(10**6)
sorted(xs, key=lambda x: (-x[1], x[0]))
bisect.bisect_left(xs, x) # lower bound
divmod(a, b); a // b; a % b # floor division
```

heapq is a min-heap: push -x for a max-heap, tuples (priority, item) for ordering. deque.popleft() is O(1); list.pop(0) is O(n).

HASH MAP: TWO-SUM, COUNTING

```
def two_sum(nums, t):
    seen = {}
    for i, x in enumerate(nums):
        if t - x in seen:
            return [seen[t - x], i]
    seen[x] = i
# subarray sum == k (prefix sums + map)
cnt = Counter({0: 1}); s = res = 0
for x in nums:
    s += x
    res += cnt[s - k]
    cnt[s] += 1
```

TWO POINTERS (SORTED ARRAY)

```
l, r = 0, len(a) - 1
while l < r:
    s = a[l] + a[r]
    if s == t: return l, r
    if s < t: l += 1
    else: r -= 1
```

Also: remove duplicates in place (slow/fast pointer), reverse, merge two sorted lists, container with most water, 3-sum (sort, fix one, two pointers).

SLIDING WINDOW

```
# longest substring without repeats
last = {}; l = best = 0
for r, ch in enumerate(s):
    if ch in last and last[ch] >= l:
        l = last[ch] + 1
    last[ch] = r
    best = max(best, r - l + 1)
```

Fixed size k: add a[r], drop a[r - k]. Variable: expand right, shrink left while the rule is broken.

BINARY SEARCH

```
def bs(a, x):
    lo, hi = 0, len(a)
    while lo < hi:
        mid = (lo + hi) // 2
        if a[mid] < x: lo = mid + 1
        else: hi = mid
    return lo
```

On the answer: binary search `lo..hi` with `ok(mid)` monotonic (min capacity, min speed). Avoid `(lo + hi)` overflow in other languages.

STACK AND MONOTONIC STACK

```
def valid(s):
    pair = {"(": ")", "[": "]", "{": "}"
    st = []
    for c in s:
        if c in pair:
            if not st or st.pop() != pair[c]: return False
            else: st.append(c)
    return not st
# next greater element
res = [-1] * n; st = []
for i, x in enumerate(a):
    while st and a[st[-1]] < x: res[st.pop()] = x
    st.append(i)
```

BFS AND DFS (GRID / GRAPH)

```
def bfs(start, adj):
    seen = {start}; q = deque([(start, 0)])
    while q:
        node, d = q.popleft()
        for nb in adj[node]:
            if nb not in seen:
                seen.add(nb); q.append((nb, d + 1))
def dfs(node, adj, seen):
    seen.add(node)
    for nb in adj[node]:
        if nb not in seen: dfs(nb, adj, seen)
```

Grid neighbours: for dr, dc in `((1,0), (-1,0), (0,1), (0,-1))`, check bounds. BFS = shortest path in an unweighted graph; mark visited when enqueued. Cycle in a directed graph: DFS with colours or topological sort (Kahn: in-degrees).

HEAP: TOP-K, MERGE INTERVALS

```
heapq.nlargest(k, xs)
h = []
for x in xs:
    heapq.heappush(h, x)
    if len(h) > k: heapq.heappop(h)
# merge intervals
out = []
for s, e in sorted(iv):
    if out and s <= out[-1][1]:
        out[-1][1] = max(out[-1][1], e)
    else: out.append([s, e])
```

DYNAMIC PROGRAMMING

```
from functools import lru_cache
@lru_cache(None)
def fib(n): return n if n < 2 else fib(n-1) + fib(n-2)
# coin change: fewest coins
dp = [0] + [inf] * amt
for a in range(1, amt + 1):
    for c in coins:
        if c <= a: dp[a] = min(dp[a], dp[a - c] + 1)
```

Recipe: define the state, the recurrence, the base case, the order. Top-down (memoised recursion) or bottom-up table. Classics: climbing stairs, knapsack, longest increasing subsequence ($O(n \log n)$) with bisect), edit distance, longest common subsequence.

BACKTRACKING, LINKED LIST, TREE

```
def subsets(a):
    res = []
    def go(i, cur):
        if i == len(a): res.append(cur[:]); return
        go(i + 1, cur)
        cur.append(a[i]); go(i + 1, cur); cur.pop()
    go(0, []); return res
# reverse linked list
prev = None
while head:
    head.next, prev, head = prev, head, head.next
# tree height
def h(t): return 0 if not t else 1 + max(h(t.left), h(t.right))
```

Inorder of a BST is sorted. Level order = BFS with a queue.

GRAPH EXTRAS: DIJKSTRA, TOPOLOGICAL SORT, UNION-FIND

```
# Dijkstra (non-negative weights),  $O((V+E) \log V)$ 
dist = {src: 0}; h = [(0, src)]
while h:
    d, u = heapq.heappop(h)
    if d > dist.get(u, inf): continue
    for v, w in adj[u]:
        if d + w < dist.get(v, inf):
            dist[v] = d + w; heapq.heappush(h, (d + w, v))
# union-find
def find(x):
    while p[x] != x: p[x] = p[p[x]]; x = p[x]
    return x
def union(a, b): p[find(a)] = find(b)
```

Topological order (Kahn): queue nodes with in-degree 0, pop, decrement neighbours; if fewer than V nodes output, there is a cycle. Weighted graph with negative edges: Bellman-Ford.

BITS, MATH, STRINGS

- $x \& 1$ odd; $x >> 1$ half; $x \& (x - 1)$ clears lowest set bit; $x \wedge x == 0$ (single number); $1 \ll k$.
- Primes: sieve of Eratosthenes $O(n \log \log n)$. `math.gcd`, `pow(a, b, mod)`.
- Palindrome expand-around-centre $O(n^2)$; anagram by sorted key or Counter; strings are immutable, build with a list and `join`.
- Matrix: transpose `zip(*m)`, rotate 90 clockwise `[list(r)[::-1] for r in zip(*m)]`; prefix sums per row for 2-D range sums.

COMMON MISTAKES

TRAP Off-by-one in loops and binary search; infinite loop when `lo/hi` do not move. Mutating a list while iterating; appending `cur` instead of `cur[:]` in backtracking. Forgetting to mark visited before enqueueing (BFS) or the base case (recursion). Using `list.pop(0)` as a queue, `+=` on strings in a loop, or a default mutable argument.

- Test: empty input, one element, duplicates, negative numbers, a sorted and a reverse-sorted input.

SAY IT IN 20 SECONDS

- "Brute force is $O(n^2)$; a hash map makes it $O(n)$ time, $O(n)$ space."
- "Sliding window: the window is valid or I shrink it."
- "BFS finds the shortest path in an unweighted graph."
- "Dynamic programming = state, recurrence, base case."

5

CHAPTER 5 · HIGHEST PRIORITY · MODULE 5 IN THE APP

Statistics and Probability

Statistics is the reasoning engine of data science. Interviews test whether you can explain each idea simply, choose the right test, and avoid the classic misreadings.

11 topics · about 5 h 30 min in the app · 44 questions answered · two-page cheat sheet at the end

- | | |
|---|---|
| 5.1 Descriptive statistics | 5.7 Hypothesis testing: the core definitions |
| 5.2 Probability fundamentals | 5.8 Which test do I use? |
| 5.3 Bayes' theorem: the worked example | 5.9 Multiple testing and Python recipes |
| 5.4 Classic probability puzzles | 5.10 Other essentials: sampling, MLE, Bayesian, linear algebra, calculus |
| 5.5 Distributions you must know | 5.11 Stats Q&A and self-check |
| 5.6 Central Limit Theorem and the Law of Large Numbers | |

Foundations

Statistics exists to answer one question: **what can we conclude about the world from limited, noisy data?** Every topic in this chapter is a tool for some part of that question, and they connect in a chain. If you see the chain, the formulas become labels on ideas you already understand.

Step 1: describe what you have

Before any inference, summarise the data: where its centre is and how spread out it is. The **mean** is the balance point and is dragged by extreme values. The **median** is the middle value and is not. When the two differ a lot, the data is skewed, and the median is usually the honest summary, which is why incomes and house prices are reported as medians. Spread is measured by the **standard deviation**, roughly the typical distance from the mean. **Correlation** measures how closely two variables move together along a straight line, from minus one to plus one. It says nothing about which causes which, and nothing about curved relationships.

Step 2: probability is the language of uncertainty

Probability assigns a number between zero and one to how likely an event is. Two ideas carry most of the weight. **Conditional probability** is the chance of one thing given that another is known to have happened; it shrinks the world to the cases where the condition holds. **Bayes' theorem** turns a conditional probability around: from "how likely is a positive test if you are ill" to "how likely are you to be ill given a positive test". The famous lesson is that the answer depends heavily on how common the illness is. With a rare disease, most positive results come from the large healthy population, so a positive test from a ninety-nine per cent accurate test can still mean only a one-in-six chance of disease. Forgetting the base rate is the most common probability error in interviews and in life.

Step 3: distributions describe how data behaves

A **distribution** is a description of which values occur and how often. A handful appear constantly. The **binomial** counts successes in a fixed number of yes-or-no trials. The **Poisson** counts events in a fixed interval. The **normal**, the bell curve, appears wherever many small independent effects add up.

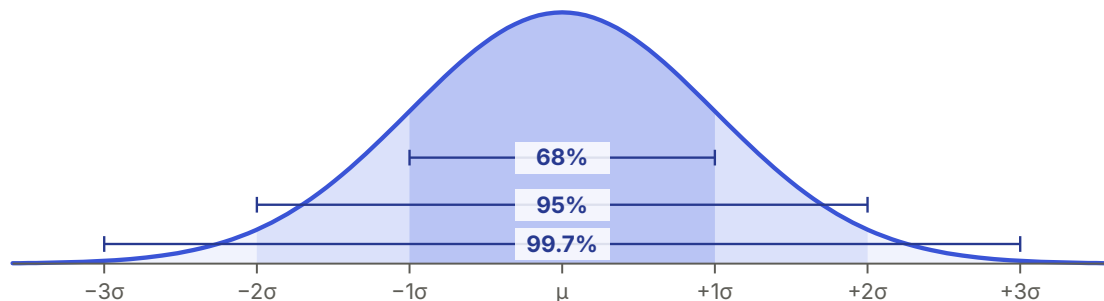


Figure 7. In a normal distribution about 68, 95 and 99.7 per cent of values lie within one, two and three standard deviations of the mean.

Step 4: the bridge from sample to population

We almost never see the whole population. We see a sample, compute its mean, and want to know how far that might be from the truth. The **Central Limit Theorem** is the bridge. It says that the average of many independent observations is approximately normally distributed around the true mean, whatever the shape of the original data, and that its spread, called the **standard error**, is the standard deviation divided by the square root of the sample size. Two consequences are worth saying aloud in any interview. Averages are predictable even when individual values are not. And precision is expensive: to halve the error you need four times the data.

Step 5: estimate, then test

From the standard error come the two tools of inference. A **confidence interval** is a range around your estimate, built so that the method captures the true value in, say, 95 per cent of repeated samples. A **hypothesis test** asks whether an observed effect could plausibly be chance. You assume there is no effect (the null hypothesis), and compute the **p-value**: the probability of seeing data at least as extreme as yours if that assumption were true. A small p-value means the data would be surprising under "no effect", so you reject it.

Two errors are possible. A **Type I error** is a false alarm: declaring an effect that is not there. A **Type II error** is a miss. **Power** is the probability of detecting a real effect of a given size, and it rises with sample size. Much confusion disappears once you can state what a p-value is *not*: it is not the probability that the null hypothesis is true, and it says nothing about how large or important the effect is.

Step 6: choose the test from the data

Choosing a test is not memorisation. Ask what type of outcome you have (a number or a category), how many groups, and whether the groups are independent or paired. Two independent means: a t-test. The same people before and after: a paired t-test. Three or more

groups: ANOVA. Two proportions or two categorical variables: a z-test or chi-square. Small or clearly non-normal samples: a rank-based test. And if you run many tests, some will "succeed" by chance, so you correct for it.

Topic by topic

Each topic gives the idea in plain words, the fundamentals to know, a model spoken answer and three questions to test yourself. Open the same topic in the app for worked examples, runnable code and interactive demos.

5.1 Descriptive statistics

IN PLAIN WORDS

Descriptive statistics are the few numbers that describe a messy pile of data. The average (mean) is the fair share; the median is the person in the exact middle of the line; the spread tells you whether everyone is similar or wildly different. One billionaire walking into a village drags the average income up but does not change the median at all, which is why we report medians for things like income and delivery time. Correlation asks whether two things rise and fall together, but two things moving together does not mean one causes the other.

Why it matters: These are the first questions in any stats screening round, and the traps (skew, Simpson's paradox, correlation vs causation) are favourites.

Takeaway: Pick the summary that fits the shape of the data, and never read causation into correlation.

WORKED EXAMPLE, STEP BY STEP

One extreme value against the mean, median and IQR

Monthly salaries (in thousands) at a six-person startup: 30, 35, 40, 45, 50 and the founder's 300.

- 1. Mean:** $(30 + 35 + 40 + 45 + 50 + 300) / 6 = 500 / 6 \approx 83.3$. Five of the six people earn less than this "average".
- 2. Median:** sort and take the middle. With six values, average the 3rd and 4th: $(40 + 45) / 2 = 42.5$. That is a typical salary.
- 3. Standard deviation:** about **106**, larger than every salary except one. A single value inflated it, so here it does not describe the "typical spread".
- 4. A robust spread, the IQR.** The quartiles are $Q1 \approx 36.25$ and $Q3 \approx 48.75$, so $IQR = 12.5$: the middle half of the team sits within 12.5 thousand of each other.
- 5. Flag the outlier with the IQR fences.** Upper fence = $Q3 + 1.5 \times IQR = 48.75 + 18.75 = 67.5$. The value 300 is far above it, so it is flagged. The lower fence is $36.25 - 18.75 = 17.5$, and nobody is below it.
- 6. Report it honestly:** "The median salary is 42.5 thousand; one founder salary of 300 thousand raises the mean to 83."

What this shows. The mean and standard deviation follow extreme values; the median and IQR resist them. With skewed data, lead with the robust pair.

YOUR TURN (COVER THE SOLUTION FIRST)

1. x takes the values $-2, -1, 0, 1, 2$ and $y = x^2$. Without a calculator, what is the correlation between x and y , and what does that teach you?

Solution. y is 4, 1, 0, 1, 4. As x goes from -2 to 0, y falls; from 0 to 2, y rises by exactly the same amounts. The ups and downs cancel, so the correlation is exactly 0, even though y is completely determined by x . Correlation measures only straight-line association, so a correlation of 0 does not mean "no relationship". Always plot the data.

THE FUNDAMENTALS

Concept	Key points
Mean, median, mode	Mean is sensitive to outliers; median robust. Right-skewed: mean > median > mode.
Variance / SD	Population divides by N ; sample divides by $N-1$ (Bessel's correction, because the sample mean is fitted from data).
Range, IQR	IQR = $Q3 - Q1$; robust spread measure.
Skewness	Asymmetry. Positive = long right tail. Fix with log/sqrt/Box-Cox.
Kurtosis	Tail heaviness.
Z-score	$(x - \text{mean})/\text{SD}$; number of SDs from the mean.
Covariance	Direction of joint variation; scale-dependent.
Correlation	Standardized covariance in $[-1, 1]$. Pearson = linear; Spearman = monotonic (rank-based); Kendall = rank concordance.
Percentile / quantile	Value below which $p\%$ of data fall.
Coefficient of variation	SD / mean; compares spread across scales.

Interview traps: correlation $\neq$ causation; zero correlation does not mean independence (e.g., $y = x^2$); Simpson's paradox (aggregate trend reverses within subgroups).

SAY IT IN 60 SECONDS

"Descriptive statistics summarise data with a centre, a spread and a shape. The mean is the average and is pulled by outliers; the median is the middle value and resists them; the mode is the most common. For spread, variance is the average squared distance from the mean, standard deviation is its square root in the original units, and the interquartile range is the middle fifty per cent. Skew says which tail is long: income is right-skewed, so mean exceeds median. Correlation measures linear association from minus one to plus one, and says nothing about causation. The catch: sample variance divides by n minus one, not n , to correct for estimating the mean from the same data."

KEY TERMS mean · median · mode · variance · standard deviation · Bessel's correction · IQR · skewness · z-score · covariance · correlation · Simpson's paradox

HANDS-ON PRACTICE Take a skewed variable (income, delivery time), compute mean, median, IQR and skewness, and explain which you would report and why.

COMMON MISTAKES Reporting the mean of a heavily skewed variable · Saying correlation implies causation · Computing population variance when a sample variance is needed

CHECK YOURSELF**1. Salaries 40, 45, 50, 55, 400 (k). Mean and median, and which do you report?**

Mean 118k, median 50k. Report the median; the outlier drags the mean.

2. Why divide by $n - 1$ for sample variance?

The sample mean is fitted to the same data, which makes deviations too small; dividing by $n - 1$ removes that bias (Bessel's correction).

3. Correlation is 0.0 between x and y . Does that mean they are unrelated?

No: it only rules out a linear relationship. $y = x^2$ has strong dependence and correlation near zero.

5.2 Probability fundamentals

IN PLAIN WORDS

Probability is a number between 0 (never) and 1 (always) that says how likely something is. To find it, count the ways you want to happen and divide by all the ways that could happen, as long as every way is equally likely. "And" usually means multiply (two coin flips both heads: $1/2 \times 1/2$). "Or" usually means add, but subtract the overlap so you do not count it twice. When something is hard to count directly, count its opposite and subtract from 1.

Why it matters: Probability puzzles and Bayes questions appear in almost every data science interview.

Takeaway: List the sample space, use complements, and do not confuse "cannot happen together" with "do not affect each other".

WORKED EXAMPLE, STEP BY STEP**Two dice, three questions**

Roll two fair dice. There are $6 \times 6 = 36$ equally likely outcomes.

1. $P(\text{sum} = 7)$. Count the ways: (1,6), (2,5), (3,4), (4,3), (5,2), (6,1). That is 6 of 36 = **$1/6$** .

2. $P(\text{at least one six})$. Counting directly is easy to get wrong, because (6,6) is easy to count twice. Use the complement: $P(\text{no six}) = (5/6) \times (5/6) = 25/36$, so $P(\text{at least one six}) = 1 - 25/36 = 11/36 \approx 0.31$.

3. Check by counting: 6 outcomes with a six on the first die, 6 with a six on the second, minus (6,6) counted twice: $6 + 6 - 1 = 11$. That is the addition rule, $P(A \text{ or } B) = P(A) + P(B) - P(A \text{ and } B)$.

4. $P(\text{sum} = 8, \text{ given the first die shows } 3)$. The condition shrinks the world to six outcomes: (3,1) to (3,6). Only (3,5) sums to 8, so the answer is **$1/6$** .

5. Independence. Knowing that the first die is 3 did not change the chance of a 5 on the second die. That is what independent means.

What this shows. Count equally likely outcomes; for "at least one", use one minus "none"; for "given that", shrink the sample space before counting.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Draw two cards from a shuffled 52-card deck without putting the first back. What is the probability that both are red? Why is it not $(1/2) \times (1/2)$?

Solution. $P(\text{first red}) = 26/52$. Given that, 25 red cards remain among 51, so $P(\text{second red} \mid \text{first red}) = 25/51$. Multiply: $(26/52) \times (25/51) \approx 0.245$. It is not 0.25 because the draws are not independent: taking a red card out leaves slightly fewer reds. With replacement, the answer would be exactly 0.25.

THE FUNDAMENTALS

- Sample space, events, union/intersection, complement
- Addition rule: $P(A \text{ or } B) = P(A) + P(B) - P(A \text{ and } B)$
- Multiplication: $P(A \text{ and } B) = P(A) P(B|A)$
- Independence: $P(A \text{ and } B) = P(A)P(B)$; mutually exclusive is NOT the same as independent
- Conditional probability: $P(A|B) = P(A \text{ and } B) / P(B)$
- **Law of total probability:** $P(B) = \text{sum over } i \text{ of } P(B|A_i) P(A_i)$
- **Bayes' theorem:** $P(A|B) = P(B|A) P(A) / P(B)$
- Permutations (order matters) $nPr = n!/(n-r)!$; combinations $nCr = n!/(r!(n-r)!)$
- Expected value $E[X] = \text{sum } x P(x)$; linearity of expectation; variance $\text{Var}(X) = E[X^2] - (E[X])^2$

SAY IT IN 60 SECONDS

"Probability is a number from zero to one for how likely an event is. For equally likely outcomes it is favourable divided by total. The addition rule: $P(A \text{ or } B)$ equals $P(A)$ plus $P(B)$ minus $P(A \text{ and } B)$. The multiplication rule: $P(A \text{ and } B)$ equals $P(A)$ times $P(B \text{ given } A)$. If events are independent, knowing one tells nothing about the other, so the product is simple. Conditional probability $P(A \text{ given } B)$ restricts attention to the cases where B happened. 'At least one' problems are easiest as one minus the chance of none. The catch: independent is not the same as mutually exclusive; exclusive events are strongly dependent, since one rules the other out."

KEY TERMS sample space · conditional probability · independence · mutually exclusive · law of total probability · combinations · expected value · linearity of expectation

HANDS-ON PRACTICE Solve ten small probability questions by writing the sample space, then check with a short simulation.

COMMON MISTAKES Treating mutually exclusive as independent · Multiplying probabilities that are not independent · Forgetting the complement trick

CHECK YOURSELF

1. **Chance of at least one six in four rolls of a die?**
 $1 - (5/6)^4 \approx 0.518$.
2. **Can two events with positive probability be both mutually exclusive and independent?**
No. If exclusive, $P(A \text{ and } B) = 0$, but independence requires $P(A)P(B) > 0$.
3. **Two cards drawn without replacement: probability both are aces?**
 $4/52 \times 3/51 = 1/221 \approx 0.45\%$.

5.3 Bayes' theorem: the worked example

IN PLAIN WORDS

Imagine 10,000 people. Only 100 are really sick. The test catches nearly all of them (99 positives). But among the 9,900 healthy people, 5% get a false alarm: 495 people. So there are $99 + 495 = 594$ positives, and only 99 of them are actually sick. If you test positive, your chance of being sick is 99 out of 594, about one in six. The test is good, but the disease is rare, so false alarms from the big healthy group swamp the real cases.

Why it matters: It is the most common probability question in interviews, and the same logic explains why fraud alerts and spam filters raise so many false alarms.

Takeaway: Rare things plus imperfect tests mean most alarms are false. Count people, not percentages.

WORKED EXAMPLE, STEP BY STEP

A fraud alert, with natural frequencies

0.5% of transactions are fraud. The alert catches 90% of fraud and wrongly flags 2% of genuine transactions. An alert fires: how likely is it to be fraud?

- 1. Imagine 10,000 transactions.** 0.5% are fraud: **50** fraud and **9,950** genuine.
- 2. Alerts on fraud:** 90% of 50 = **45** true alerts.
- 3. Alerts on genuine transactions:** 2% of 9,950 = **199** false alarms.
- 4. All alerts:** $45 + 199 = 244$.
- 5. Of the alerts, the fraud share:** $45 / 244 \approx 18\%$.
- 6. Same answer from the formula:** $P(\text{fraud} \mid \text{alert}) = (0.9 \times 0.005) / (0.9 \times 0.005 + 0.02 \times 0.995) = 0.0045 / 0.0244 \approx 0.184$.
- 7. Read it.** Although the alert is "90% accurate" on fraud, about 4 alerts in 5 are false, because genuine transactions vastly outnumber fraud.

What this shows. When the thing you look for is rare, false alarms from the large group dominate. Turn percentages into counts out of 10,000 and the answer becomes obvious.

YOUR TURN (COVER THE SOLUTION FIRST)

1. A flagged transaction (fraud probability now 18.4%) goes through a second, independent check with the same 90% catch rate and 2% false-alarm rate, and is flagged again. What is the probability of fraud now?

Solution. Use the old posterior as the new prior: $P = (0.9 \times 0.184) / (0.9 \times 0.184 + 0.02 \times 0.816) = 0.1656 / (0.1656 + 0.0163) \approx 0.91$. Two independent alerts take it from 0.5% to about 91%. This is Bayesian updating: each piece of evidence multiplies the odds by the likelihood ratio, here $0.9 / 0.02 = 45$.

THE FUNDAMENTALS

A disease has 1% prevalence. A test has 99% sensitivity and 5% false-positive rate. You test positive. What is $P(\text{disease} \mid \text{positive})$?

$$\begin{aligned}
 P(+|D) &= 0.99, \quad P(D) = 0.01, \quad P(+|\text{not } D) = 0.05 \\
 P(+) &= 0.99 \times 0.01 + 0.05 \times 0.99 = 0.0099 + 0.0495 = 0.0594 \\
 P(D|+) &= 0.0099 / 0.0594 = 16.7\%
 \end{aligned}$$

Insight: with a low base rate, most positives are false positives. The same logic applies to fraud detection alerts and spam filters.

SAY IT IN 60 SECONDS

"Bayes' theorem turns P of evidence given hypothesis into P of hypothesis given evidence: posterior equals likelihood times prior, divided by the total probability of the evidence. The classic lesson is the rare disease test. If one per cent have the disease, the test catches ninety-nine per cent of cases and false-alarms five per cent of the time, then among a thousand people about ten are sick and nine or ten test positive, but about fifty healthy people also test positive. So a positive result means roughly sixteen per cent, not ninety-nine. The catch: the prior matters, and people forget it, which is called base-rate neglect."

KEY TERMS Bayes' theorem · prior · likelihood · posterior · sensitivity · false positive rate · base rate fallacy · natural frequencies

BUILDS ON 5.2 Probability fundamentals

HANDS-ON PRACTICE Change the prevalence to 10% and recompute; then explain the change to a non-technical person.

COMMON MISTAKES Confusing $P(\text{positive} \mid \text{disease})$ with $P(\text{disease} \mid \text{positive})$ · Ignoring the base rate · Forgetting $P(\text{positive})$ is the sum over both groups

CHECK YOURSELF

1. **Prevalence 1%, sensitivity 99%, false-positive rate 5%. $P(\text{disease} \mid \text{positive})$?**

$$0.0099 / (0.0099 + 0.0495) \approx 16.7\%.$$

2. **Why is the posterior so much lower than the test's accuracy?**

The disease is rare, so false positives from the large healthy group outnumber true positives (base-rate neglect).

3. **Which two numbers describe a test's quality independently of prevalence?**

Sensitivity (true positive rate) and specificity (true negative rate).

5.4 Classic probability puzzles

IN PLAIN WORDS

Probability puzzles are small stories where your gut is often wrong, which is the point. The trick is always the same: list all the equally-likely possibilities, cross out the ones that do not fit what you were told, and count what is left. In Monty Hall, your first pick is right only one time in three, and that does not change when the host opens a door, so the remaining door carries the other two-thirds.

Why it matters: Interviewers use these to see how you reason under pressure, not whether you memorised answers.

Takeaway: List the possibilities, apply the information, count, then check with a simulation.

WORKED EXAMPLE, STEP BY STEP**Monty Hall by listing every case**

A car is behind one of three doors. You pick door 1. The host, who knows where the car is, opens another door that has a goat, then offers a switch.

- 1. List the equally likely positions of the car** (each has probability $1/3$), and what happens in each:

Car behind	Host opens	Stay with door 1	Switch
Door 1	2 or 3	win	lose
Door 2	3 (must avoid the car)	lose	win
Door 3	2 (must avoid the car)	lose	win

- 2. Count:** staying wins in 1 of 3 cases; switching wins in 2 of 3.
- 3. Why intuition fails.** It feels like 50/50 between two closed doors, but the host's choice is not random: he is forced to avoid the car. In two of the three worlds, he points you to it.
- 4. A sanity check with 100 doors.** You pick one; the host opens 98 goat doors. Would you keep your 1-in-100 pick? Switching wins 99 times in 100.
- 5. The assumption.** This works because the host always opens a goat door and always offers the switch. Say so in an interview.

What this shows. When a probability feels wrong, list the equally likely cases in a table. The table cannot be fooled.

YOUR TURN (COVER THE SOLUTION FIRST)

- Five people are in a room. What is the probability that at least two share a birthday (ignore leap years)? Set it up with "one minus none".

Solution. $P(\text{all different}) = (365/365) \times (364/365) \times (363/365) \times (362/365) \times (361/365) \approx 0.973$. So $P(\text{at least one shared birthday}) \approx 1 - 0.973 = 0.027$, about 2.7%. With 23 people the same calculation passes 50%, because the number of pairs grows quickly: 23 people form 253 pairs.

SAY IT IN 60 SECONDS

"Probability puzzles reward a clean method. Define the sample space, count favourable outcomes, and check by simulation if unsure. Birthday problem: with 23 people the chance of a shared birthday passes fifty per cent, because we compare all pairs, about 253 of them. Monty Hall: switching wins two thirds of the time, since your first pick had one third and the host's reveal moves the other two thirds to the remaining door. Expected value problems use linearity: add up the expected parts, even when they are dependent. The catch: say the assumptions out loud, such as the host always opening a goat door, because the answer depends on them."

KEY TERMS dice · geometric distribution · Monty Hall · birthday problem · biased coin · expected value · Von Neumann trick

BUILDS ON 5.2 Probability fundamentals

HANDS-ON PRACTICE Solve each puzzle aloud in under two minutes, then verify by simulation in Python.

COMMON MISTAKES Answering $1/2$ for the two-children problem without listing outcomes · Not stating assumptions (equally likely outcomes) · Giving the answer without a reason

CHECK YOURSELF

1. Monty Hall: why is switching better?

Your first pick wins $1/3$. The host always reveals a goat from the other two, so the remaining closed door carries their combined $2/3$.

2. Why does the birthday paradox feel surprising?

We think of 23 people vs one date, but there are 253 pairs, each a chance of a match.

3. Expected number of rolls of a fair die until the first six?

6 (geometric distribution, mean $1/p$).

5.5 Distributions you must know

IN PLAIN WORDS

A distribution is the "shape" of how likely each value is. Flip a coin once: Bernoulli. Count heads in 10 flips: binomial. Count how many customers walk in during one hour: Poisson. How long until the next customer: exponential. Heights and measurement errors pile up in a bell shape: normal. The t, chi-square and F shapes are the special ones statisticians use when they test things, and you only need to know where they appear.

Why it matters: Choosing the right distribution is the first step in modelling counts, waiting times and test statistics.

Takeaway: Match the story (yes/no, count of successes, count in an interval, waiting time, sum of many small effects) to the distribution.

WORKED EXAMPLE, STEP BY STEP

Turn business questions into distributions

A small café receives on average 3 online orders per hour, independently of each other.

- Counts in a fixed time point to the Poisson.** Orders per hour $\sim \text{Poisson}(\lambda = 3)$, with mean 3 and variance 3.
- P(no orders in an hour)** $= e^{-3} \approx 0.050$. One hour in twenty is quiet.
- P(at least one order)** $= 1 - 0.050 = 0.95$.
- The full shape:** $P(0) \approx 0.05$, $P(1) \approx 0.15$, $P(2) \approx 0.22$, $P(3) \approx 0.22$. The most likely counts are 2 and 3.
- A different question, a different distribution.** Of 10 visitors to the menu page, each orders with probability 20%. The number who order $\sim \text{Binomial}(n = 10, p = 0.2)$, mean 2.
- P(nobody orders)** $= 0.8^{10} \approx 0.11$. **P(3 or more order)** ≈ 0.32 .

Question shape	Distribution	Here
Events per interval	$\text{Poisson}(\lambda)$	orders per hour, $\lambda = 3$
Successes out of n tries	$\text{Binomial}(n, p)$	buyers out of 10 visitors

Question shape	Distribution	Here
Waiting time between events	Exponential	minutes until the next order

What this shows. You choose a distribution from how the numbers are generated (counts in time, successes out of tries, waiting times), not from the shape of a histogram.

YOUR TURN (COVER THE SOLUTION FIRST)

- Orders arrive on average every 20 minutes (a Poisson process). What is the probability that you wait more than 30 minutes for the next one? Name the distribution you use.

Solution. The waiting time between Poisson events is exponential with rate $\lambda = 1/20$ per minute. $P(\text{wait} > t) = e^{(-\lambda t)} = e^{(-30/20)} = e^{(-1.5)} \approx 0.22$. So about 22% of the time you wait more than half an hour. The exponential is memoryless: having already waited 10 minutes does not shorten the remaining wait.

THE FUNDAMENTALS

Distribution	Type	Use case	Mean	Variance
Bernoulli(p)	Discrete	Single yes/no trial	p	$p(1-p)$
Binomial(n,p)	Discrete	# successes in n trials (conversions out of visitors)	np	$np(1-p)$
Poisson(lambda)	Discrete	Count of events in an interval (orders per hour)	lambda	lambda
Geometric(p)	Discrete	Trials until first success	1/p	$(1-p)/p^2$
Uniform(a,b)	Continuous	Equal likelihood	$(a+b)/2$	$(b-a)^2/12$
Normal(mu, sigma^2)	Continuous	Heights, errors, sums of many small effects	mu	sigma^2
Exponential(lambda)	Continuous	Time between Poisson events	1/lambda	1/lambda^2
t-distribution	Continuous	Small-sample mean inference (unknown variance)	0	$df/(df-2)$
Chi-square	Continuous	Variance and categorical tests	k	2k
F	Continuous	Ratio of variances (ANOVA)	-	-

Normal rule: 68-95-99.7 within 1, 2, 3 SDs. 95% interval uses $z = 1.96$.

SAY IT IN 60 SECONDS

"A distribution says how likely each value is. Bernoulli: one yes or no trial. Binomial: number of successes in n independent trials. Poisson: count of events in a fixed interval when events are rare and independent, mean equal to variance. Normal: the bell curve, defined by mean and standard deviation, with about 68, 95 and 99.7 per cent within one, two and three standard deviations. Uniform: all values equally likely. Exponential: waiting time between Poisson events. Long-tailed ones like lognormal fit incomes and latencies. The catch: I pick a distribution from how the data is generated, such as counts or waiting times, and not just from how a histogram looks."

KEY TERMS Bernoulli · binomial · Poisson · geometric · uniform · normal · exponential · t distribution · chi-square · F · 68-95-99.7 · z-score

BUILDS ON 5.2 Probability fundamentals

HANDS-ON PRACTICE For five business quantities (conversions, orders per minute, time between arrivals, heights, revenue), name the distribution and its parameters.

COMMON MISTAKES Using a normal for counts near zero · Confusing Poisson rate and exponential rate · Forgetting the continuity of continuous distributions ($P(X = x) = 0$)

CHECK YOURSELF

1. Which distribution for the number of support tickets per hour?

Poisson (counts of independent events in an interval), check for overdispersion.

2. Heights are $N(170, 10)$. What share lies between 160 and 180?

About 68% (within one standard deviation).

3. Binomial versus Poisson: when does one approximate the other?

Binomial with large n and small p approximates Poisson with $\lambda = np$.

5.6 Central Limit Theorem and the Law of Large Numbers

IN PLAIN WORDS

Imagine a huge jar of jellybeans of every size. Grab a handful, work out the average size, and write it down. Do it again and again. Even if the jar is messy, your list of averages forms a neat bell shape, and it gets tighter the more beans you grab each time. The bell is centred on the true average size of all beans. This is why we can use a small sample to learn about a whole population, and why so many tests rely on the bell curve even when the original data is lopsided.

Why it matters: It is the reason z - and t -tests and confidence intervals work on non-normal data, including A/B tests on revenue.

Takeaway: Averages of big enough samples behave nicely (bell-shaped), with a spread that shrinks like $1/\sqrt{n}$.

WORKED EXAMPLE, STEP BY STEP

Watch averages become predictable

One fair die: the values 1 to 6 are equally likely (a flat distribution, nothing like a bell). Mean 3.5, standard deviation about 1.71.

- 1. One roll** can land anywhere from 1 to 6 with equal chance.
- 2. The average of 4 rolls.** It is still centred on 3.5, but its spread shrinks to $1.71 / \sqrt{4} \approx 0.85$. Averages of 1 or 6 are now rare, because all four rolls would have to be extreme.
- 3. The average of 100 rolls.** Spread (standard error) = $1.71 / \sqrt{100} \approx 0.17$.
- 4. The shape is now a bell.** By the Central Limit Theorem, the 100-roll average is approximately normal, so about 95% of such averages land within $3.5 \pm 1.96 \times 0.17$, that is between about **3.17 and 3.83**.
- 5. The cost of precision.** To halve the standard error you need four times the rolls: 400 rolls give $1.71 / 20 \approx 0.085$.

Rolls averaged	Standard error	95% of averages fall within
1	1.71	(no bell shape yet)
4	0.85	roughly 1.8 to 5.2
100	0.17	3.17 to 3.83

What this shows. Individual values can be wild, but averages are predictable, and their spread shrinks with $\sqrt{n}$. That is why a sample can tell us about a population.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Delivery times have a standard deviation of 12 minutes (and a skewed shape). You average 36 deliveries. What is the standard error of the average, and within what range around the true mean do about 95% of such averages fall?

Solution. Standard error = $12 / \sqrt{36} = 2$ minutes. By the CLT the average of 36 is approximately normal even though single deliveries are skewed, so about 95% of averages fall within the true mean $\pm 1.96 \times 2 \approx \pm 3.9$ minutes. To get ± 2 minutes you need four times the data: 144 deliveries.

THE FUNDAMENTALS

For i.i.d. samples with finite variance, the distribution of the sample mean approaches Normal(μ , σ^2/n) as n grows, regardless of the population's shape. **Standard error = $\sigma / \sqrt{n}$** . This is why we can run z/t-tests on non-normal data when n is large enough (rule of thumb $n \geq 30$, more for heavy skew).

Law of Large Numbers: the sample mean converges to the true mean as n increases. (CLT is about the *shape* of the distribution; LLN about *convergence*.)

SAY IT IN 60 SECONDS

"The Central Limit Theorem says that the average of many independent samples is approximately normal, whatever the shape of the original data, provided the variance is finite. The spread of that average, the standard error, is $\sigma / \sqrt{n}$, so quadrupling the sample halves the error. This is why we can build confidence intervals and t-tests for means even on skewed data. The Law of Large Numbers is the companion: as n grows, the sample mean converges to the true mean. The catch: CLT is about means, not about the raw data becoming normal, and it needs a large enough n , more so for heavy skew."

KEY TERMS central limit theorem · standard error · law of large numbers · sampling distribution · bootstrap · heavy tails

BUILDS ON 5.5 Distributions you must know

HANDS-ON PRACTICE Simulate sample means from an exponential distribution for $n = 1, 5, 30, 200$ and watch the shape change.

COMMON MISTAKES Saying the data becomes normal (it is the sample mean that does) · Applying rule-of-thumb $n \geq 30$ to heavy-tailed data · Confusing standard deviation with standard error

CHECK YOURSELF

1. **Std dev of a metric is 20, $n = 100$. Standard error of the mean?**

$20 / 10 = 2$.

2. **How much more data to halve the standard error?**

Four times as much, since error scales with $1/\sqrt{n}$.

3. **LLN versus CLT in one line each?**

LLN: the sample mean approaches the true mean. CLT: the sample mean's distribution around it is approximately normal with SD $\sigma/\sqrt{n}$.

5.7 Hypothesis testing: the core definitions

IN PLAIN WORDS

A hypothesis test is a courtroom for an idea. We start by assuming the boring explanation is true ("the new button changed nothing", the null hypothesis). Then we look at the data and ask: if the boring explanation were true, how surprising would results this extreme be? If they would be very surprising (a small p-value, say below 5%), we stop believing the boring explanation. A small p-value does not say the new idea is definitely right; it says the data is hard to explain by luck alone. And a big p-value does not prove nothing is happening; it may just mean we did not collect enough evidence.

Why it matters: Every A/B test and most analytical claims rely on this logic, and interviewers love testing the misconceptions.

Takeaway: p-value = how surprising the data is if nothing is going on. Report the size of the effect too.

WORKED EXAMPLE, STEP BY STEP**Build a p-value from a coin**

A friend's coin lands heads 15 times in 20 flips. Is it fair?

1. **Assume the boring explanation (the null hypothesis).** The coin is fair: $P(\text{heads}) = 0.5$.
2. **Ask how surprising the data is under that assumption.** $P(15 \text{ or more heads in } 20 \text{ flips} \mid \text{fair}) = \text{sum of the binomial probabilities for } 15, 16, \dots, 20 \approx \mathbf{0.021}$.
3. **Count both directions.** 15 tails would be just as suspicious, so the two-sided p-value doubles it: $\approx \mathbf{0.041}$.
4. **Compare with your threshold.** Using $\alpha = 0.05$, $0.041 < 0.05$, so you reject "fair". Data this lopsided would happen about 4 times in 100 if the coin were fair.
5. **Say what it is not.** It is not "a 4% chance the coin is fair". It is the chance of results this extreme *if* it were fair.
6. **Remember the two ways to be wrong.** If the coin really is fair, you have made a Type I error (a false alarm), which happens 5% of the time with $\alpha = 0.05$. With only 20 flips, a slightly biased coin (say 55% heads) would usually go undetected: a Type II error, caused by low power.

What this shows. Every hypothesis test has this shape: assume no effect, compute how surprising the data would be, and reject the assumption if it is surprising enough.

YOUR TURN (COVER THE SOLUTION FIRST)

1. A coin lands heads 60 times in 100 flips. Using the normal approximation (under the null the number of heads has mean 50 and standard deviation $\sqrt{100 \times 0.5 \times 0.5} = 5$), compute z and the approximate two-sided p -value. Is it significant at 5%?

Solution. $z = (60 - 50) / 5 = 2$. The two-sided p -value for $z = 2$ is about 0.046 (since 95% of a normal lies within ± 1.96). That is just under 0.05, so it is borderline significant. The exact binomial calculation gives about 0.057, just over 0.05. The lesson: a result this close to the threshold is weak evidence either way, so report the estimate (60%) with its interval rather than a verdict.

THE FUNDAMENTALS

Key definitions

- **Null hypothesis (H0):** default, no effect. **Alternative (H1):** what we want evidence for.
- **p-value:** probability of observing data at least as extreme as ours, assuming H0 is true. It is NOT the probability that H0 is true, and NOT the probability the result is due to chance.
- **Significance level alpha:** tolerated Type I error rate (usually 0.05).
- **Type I error (false positive):** reject H0 when true; probability alpha.
- **Type II error (false negative):** fail to reject H0 when false; probability beta.
- **Power = 1 - beta:** probability of detecting a true effect (target 80%).
- **Confidence interval (95%):** if we repeated the procedure many times, about 95% of such intervals would contain the true parameter. It is NOT "95% probability the parameter lies in this particular interval" (frequentist reading).
- **Effect size:** magnitude (Cohen's d , difference in proportions, odds ratio). Statistical significance does not imply practical significance.

Explain p-value to a non-technical person

"If our new feature actually did nothing, how surprising would it be to see a result this big just from random variation? A p -value of 0.03 means that would happen only about 3% of the time, so we have reasonable evidence something real is going on."

SAY IT IN 60 SECONDS

"Hypothesis testing asks whether data is surprising under a default assumption. The null says there is no effect; the alternative says there is. I pick a significance level, usually five per cent. The p -value is the probability of data at least this extreme if the null were true. A small p -value means the data is hard to explain by chance, so we reject the null. Type one error is a false alarm, with probability alpha. Type two is a miss, and power is one minus its probability. A confidence interval gives the plausible range for the effect. The catch: the p -value is not the probability that the null is true, and statistical significance does not mean practical importance."

KEY TERMS null hypothesis · p -value · significance level · Type I error · Type II error · power · confidence interval · effect size · Cohen's d

BUILDS ON 5.6 Central Limit Theorem and the Law of Large Numbers

HANDS-ON PRACTICE Take a two-group dataset, run a Welch t-test, and write the interpretation in three sentences: effect size with interval, p-value meaning, practical significance.

COMMON MISTAKES Interpreting the p-value as the probability that H_0 is true · Saying "we accept the null" after a non-significant result · Reporting a p-value without an effect size or interval

CHECK YOURSELF

1. Explain $p = 0.03$ in one sentence to a product manager.

If there were truly no difference, we would see a gap this large or larger only about 3% of the time.

2. What is power and how do you raise it?

Probability of detecting a real effect: raise it with more samples, a larger effect, lower variance, or a higher alpha.

3. Is $p = 0.03$ evidence the null is 3% likely to be true?

No. It is $P(\text{data} \mid \text{null})$, not $P(\text{null} \mid \text{data})$; that needs a prior (Bayes).

5.8 Which test do I use?

IN PLAIN WORDS

Choosing a test is like choosing a tool from a toolbox by asking three questions. What kind of data do I have (numbers like time, or categories like yes/no)? How many groups am I comparing (one, two, or more)? Are the groups separate people or the same people measured twice? Answer those and the right test appears. If the numbers are very lopsided and the sample is small, switch to the "rank-based" cousin of the test that does not need a bell curve.

Why it matters: "Which test would you use here?" is among the most common stats questions.

Takeaway: Type of outcome, number of groups, paired or independent, and whether normality is reasonable.

WORKED EXAMPLE, STEP BY STEP

Choose the test with three questions

For each scenario ask: (1) Is the outcome a number or a category? (2) How many groups? (3) Are the groups independent or paired?

- 1. "Did the new checkout raise the conversion rate?"** Outcome: converted yes or no (a category, summarised as a proportion). Two independent groups. **Two-proportion z-test** (or chi-square).
- 2. "Is average delivery time different between two cities?"** A number, two independent groups. **Welch's t-test**. Delivery times are skewed, but with hundreds of orders per city the CLT protects the test of means.
- 3. "Did a training course raise the same employees' scores?"** A number, the same people measured twice: **paired t-test** on the differences.
- 4. "Do three ad designs differ in average time on page?"** A number, three groups: **ANOVA**, then pairwise comparisons with a correction.

5. "Is device type related to whether a user churns?" Two categorical variables: **chi-square test of independence**.
6. "Eight customers rated two menus on a 1-to-5 scale." Small, ordinal and paired: **Wilcoxon signed-rank test**.

What this shows. You do not memorise a list of tests. Three questions about the data lead to the test, and saying them aloud is what earns the marks.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Name the test: (a) the share of users who click, for two subject lines; (b) average order value for the same 50 customers before and after a loyalty programme; (c) whether the city (4 cities) and the preferred payment method (3 methods) are related.

Solution. (a) Two independent proportions: a two-proportion z-test or chi-square. (b) A number, paired (the same customers): a paired t-test on the per-customer differences; with heavy outliers, the Wilcoxon signed-rank test. (c) Two categorical variables in a 4×3 table: a chi-square test of independence.

THE FUNDAMENTALS

Scenario	Test
Mean of one sample vs known value, sigma known, large n	One-sample z-test
Mean vs known value, sigma unknown	One-sample t-test
Compare means of two independent groups	Two-sample (Welch's) t-test
Before/after on the same subjects	Paired t-test
Compare means of 3+ groups	One-way ANOVA (then post-hoc Tukey)
Compare two proportions (conversion rates)	Two-proportion z-test or chi-square
Association between two categorical variables	Chi-square test of independence
Goodness of fit to a distribution	Chi-square GoF, Kolmogorov-Smirnov
Non-normal, small samples, two groups	Mann-Whitney U (independent), Wilcoxon signed-rank (paired)
Non-normal, 3+ groups	Kruskal-Wallis
Test normality	Shapiro-Wilk, QQ plot
Equality of variances	Levene's test, F-test
Correlation significance	Pearson/Spearman test

Assumptions to know for t-test: independence, approximate normality of the mean (or large n), (for Student's version) equal variance. Prefer Welch's.

SAY IT IN 60 SECONDS

"Choosing a test comes down to three questions. What type is the outcome? How many groups, and are they independent or paired? Are assumptions met? Comparing means of two independent groups: t-test, or Welch's if variances differ. Paired before and after: paired t-test. Three or more groups: ANOVA. Comparing proportions: z-test or chi-square. Two categorical variables: chi-square test of independence. Non-normal or ordinal data with small samples: Mann-Whitney or Wilcoxon. Correlation: Pearson or Spearman. The catch: with a large n the CLT makes the t-test fine on skewed data, but heavy-tailed metrics like revenue may still need a bootstrap or a transform."

KEY TERMS z-test · t-test · Welch · paired t-test · ANOVA · chi-square · Mann-Whitney · Wilcoxon · Kruskal-Wallis · Shapiro-Wilk · Levene

BUILDS ON 5.7 Hypothesis testing: the core definitions

HANDS-ON PRACTICE Work through the scenario questions below without looking at the answers, then check and note which assumption you missed.

COMMON MISTAKES Using an independent test on paired data · Running multiple t-tests instead of ANOVA · Testing normality with a huge n and overreacting to tiny deviations

CHECK YOURSELF**1. Conversion in control 10% versus treatment 11%, large samples. Which test?**

Two-proportion z-test (or chi-square on the 2×2 table).

2. Same users measured before and after a change. Why paired?

Pairing removes between-user variance, so it is more powerful and respects the dependence.

3. Three variants of a page, mean time on page: first step?

ANOVA (or Kruskal-Wallis), then post-hoc pairwise tests with correction if significant.

5.9 Multiple testing and Python recipes

IN PLAIN WORDS

If you flip a coin ten times and look for one lucky streak, you will probably find one. Testing one idea at the 5% level is fair, but testing twenty ideas means you will probably "discover" something that is just luck. Corrections make the bar higher when you ask many questions: Bonferroni divides the allowed error among all tests (strict), while Benjamini-Hochberg controls the proportion of false discoveries among the things you call discoveries (more forgiving).

Why it matters: Dashboards that slice experiments by many segments and metrics create false wins unless you correct or label them exploratory.

Takeaway: More looks mean more false alarms. Correct, or treat extra findings as hypotheses.

WORKED EXAMPLE, STEP BY STEP**Twenty metrics and the false alarm you are bound to find**

An experiment changes nothing, but you test 20 metrics at $\alpha = 0.05$.

- 1. Expected false alarms:** $20 \times 0.05 = 1$. You should expect one "significant" metric by pure chance.
- 2. Chance of at least one:** $1 - 0.95^{20} \approx 0.64$. Most such experiments will show a "winner".
- 3. Bonferroni:** require $p < 0.05 / 20 = 0.0025$ for each metric. Strict, but it keeps the chance of any false alarm under 5%.
- 4. Benjamini-Hochberg, on five p-values:** 0.001, 0.012, 0.039, 0.041, 0.6 (already sorted). Compare the i -th smallest with $(i / 5) \times 0.05$:

rank i	p-value	threshold $(i/5) \times 0.05$	passes?
1	0.001	0.01	yes
2	0.012	0.02	yes
3	0.039	0.03	no
4	0.041	0.04	no
5	0.6	0.05	no

- 5. Read the table.** The largest rank that passes is 2, so BH keeps the first two findings. Bonferroni (threshold $0.05 / 5 = 0.01$) keeps only the first. BH controls the share of false discoveries instead of the chance of any, so it keeps more.
- 6. Best fix of all:** decide one primary metric before the test starts.

What this shows. Test many things and something will look significant. Correct for it, or treat it as a lead for the next test.

YOUR TURN (COVER THE SOLUTION FIRST)

1. A team slices an A/B test into 12 segments and finds one with $p = 0.03$. Using Bonferroni, is it significant at an overall 5% level? What should they do?

Solution. The Bonferroni threshold is $0.05 / 12 \approx 0.0042$, and 0.03 is far above it, so it is not significant once you account for 12 looks. With 12 tries, the chance of at least one p below 0.05 by luck is $1 - 0.95^{12} \approx 46\%$. Treat the segment as a hypothesis: write it down and test it in a fresh experiment designed for that segment.

THE FUNDAMENTALS

Multiple testing

Running m tests inflates false positives: $P(\text{at least one FP}) = 1 - (1 - \alpha)^m$. Fixes: Bonferroni (α/m ; conservative), Holm, Benjamini-Hochberg (controls False Discovery Rate).

Python snippets

```

from scipy import stats
import numpy as np

# Two-sample Welch t-test
t, p = stats.ttest_ind(a, b, equal_var=False)

# Paired t-test
t, p = stats.ttest_rel(before, after)

# Chi-square test of independence
chi2, p, dof, exp = stats.chi2_contingency(pd.crosstab(df.g, df.converted))

# Two-proportion z-test
from statsmodels.stats.proportion import proportions_ztest
z, p = proportions_ztest([conv_a, conv_b], [n_a, n_b])

# Bootstrap CI for the mean
rng = np.random.default_rng(0)
boot = [rng.choice(x, size=len(x), replace=True).mean() for _ in range(10_000)]
ci = np.percentile(boot, [2.5, 97.5])

# Normality
stats.shapiro(x)

```

SAY IT IN 60 SECONDS

"If I run many tests, some will come out significant by chance. At five per cent, twenty tests give about a sixty-four per cent chance of at least one false positive. Bonferroni divides alpha by the number of tests, simple but strict. Benjamini-Hochberg controls the false discovery rate, the expected share of false findings, and is better when I run hundreds of tests. Better still is to decide the primary metric before the experiment. In Python, scipy gives the tests and statsmodels gives multipletests. The catch: slicing results into many segments after the fact is the same problem, so such findings are hypotheses to test again."

KEY TERMS multiple comparisons · Bonferroni · Holm · Benjamini-Hochberg · false discovery rate · bootstrap · permutation test

BUILDS ON 5.8 Which test do I use?

HANDS-ON PRACTICE Simulate 1,000 tests with no real effect at alpha 0.05 and count the false positives before and after correction.

COMMON MISTAKES Testing many segments without correcting · Using Bonferroni when hundreds of tests make it too conservative · Bootstrapping dependent data naively

CHECK YOURSELF

1. You test 20 independent metrics at $\alpha = 0.05$ with no true effects. Chance of at least one 'significant'?

$1 - 0.95^{20} \approx 64\%$.

2. Bonferroni versus Benjamini-Hochberg?

Bonferroni controls the chance of any false positive (strict); BH controls the false discovery rate (more power when testing many).

3. An A/B test shows no overall effect, but one of 12 segments is significant ($p = 0.04$). Ship it?

No: with 12 slices a hit is expected by chance; treat it as a hypothesis and confirm in a new test.

5.10 Other essentials: sampling, MLE, Bayesian, linear algebra, calculus

IN PLAIN WORDS

Four toolbox ideas sit underneath everything else. **Sampling**: how you pick people decides whether your data is fair. **Maximum likelihood**: choose the settings that make the data you actually saw the least surprising. **Bayesian thinking**: start with a belief, see data, update the belief. **Maths for ML**: vectors are lists of numbers with direction, matrices transform them, and the derivative tells you which way is downhill, which is how models learn.

Why it matters: These ideas explain why models are trained the way they are, and interviewers probe them for depth.

Takeaway: Sample fairly, fit by likelihood, update beliefs with data, and use derivatives to descend.

WORKED EXAMPLE, STEP BY STEP

The maths you need, in plain words

Five tools appear throughout data science. Each one, with a number you can check.

- 1. Σ (sigma) means "add them up".** Σx_i for $x = 2, 4, 6$ is $2 + 4 + 6 = 12$. The mean is $(1/n) \Sigma x_i = 12 / 3 = 4$.
- 2. A logarithm undoes a power.** $\log_{10}(1,000) = 3$ because $10^3 = 1,000$. The natural log uses $e \approx 2.718$: $\ln(e^2) = 2$. Logs turn multiplication into addition and squash long tails, which is why we take $\log(\text{income})$.
- 3. A derivative is a slope.** For $f(x) = x^2$, the slope at $x = 3$ is $2 \times 3 = 6$. Check it numerically: $(3.01^2 - 3^2) / 0.01 = 6.01$. Gradient descent uses slopes to know which way is downhill.
- 4. A dot product adds up products.** $[1, 2] \cdot [3, 4] = 1 \times 3 + 2 \times 4 = 11$. A linear model's prediction is a dot product of weights and features, plus a bias.
- 5. Maximum likelihood picks the most believable parameter.** A coin gives 7 heads in 10. The likelihood of that result is $p^7(1 - p)^3$. At $p = 0.5$ it is 0.00098; at $p = 0.6$ it is 0.00179; at $p = 0.7$ it is 0.00222, the highest. The best estimate is $p = 7/10$, the observed share.

What this shows. The notation is shorthand for simple arithmetic. When a formula looks frightening, put in three small numbers and compute it by hand.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Compute: (a) $\Sigma (x_i - 4)^2$ for $x = 2, 4, 6$; (b) the slope of $f(x) = 3x^2$ at $x = 2$; (c) the dot product of the weights $[0.5, -1]$ with the features $[4, 3]$, plus a bias of 2.

Solution. (a) $(-2)^2 + 0^2 + 2^2 = 8$. Divided by $n - 1 = 2$, it is the sample variance, 4. (b) The slope of $3x^2$ is $6x$, so at $x = 2$ it is 12. Check: $(3 \times 2.01^2 - 3 \times 4) / 0.01 \approx 12.03$. (c) $0.5 \times 4 + (-1) \times 3 + 2 = 2 - 3 + 2 = 1$. That is exactly how a linear model makes a prediction.

THE FUNDAMENTALS

- **Sampling methods**: simple random, stratified, cluster, systematic; bias types (selection, survivorship, non-response, confirmation).

- **Bias vs variance of an estimator; MLE (conceptual):** choose parameters that maximize the likelihood of observed data. Linear regression with Gaussian noise equals MLE; cross-entropy loss equals negative log-likelihood of Bernoulli.
- **Bayesian vs frequentist:** frequentist treats parameters as fixed, probability as long-run frequency; Bayesian treats parameters as random with priors updated to posteriors. Bayesian A/B testing gives $P(B > A)$ directly.
- **Bootstrapping:** resample with replacement to estimate sampling distributions when formulas are hard.
- **Linear algebra you must know:** vectors, dot product, matrix multiplication, transpose, inverse, rank, eigenvalues/eigenvectors (PCA), SVD (conceptual), norms (L1, L2), cosine similarity.
- **Calculus you must know:** derivative as slope, partial derivatives, gradient, chain rule (backprop), gradient descent update: $\theta := \theta - \eta r * \text{grad}$.

SAY IT IN 60 SECONDS

"A few more ideas come up often. Sampling: simple random, stratified to guarantee each group is represented, and cluster. Selection and survivorship bias distort samples. Maximum likelihood chooses the parameters that make the observed data most probable; for a normal it gives the sample mean. Bayesian thinking updates a prior with data into a posterior, giving probability statements about parameters. Linear algebra: a matrix transforms vectors, eigenvectors keep direction under that transform, and PCA uses them of the covariance matrix. Calculus: the gradient points uphill, so gradient descent steps the opposite way. The catch: frequentist intervals and Bayesian credible intervals answer slightly different questions."

KEY TERMS sampling methods · bias · MLE · Bayesian vs frequentist · prior · posterior · bootstrap · linear algebra · eigenvectors · cosine similarity · gradient descent

HANDS-ON PRACTICE Derive the MLE of a coin's bias by hand, then update a Beta prior with the same data and compare.

COMMON MISTAKES Treating a convenience sample as representative · Saying MLE and MAP are the same · Memorising linear algebra terms without intuition

CHECK YOURSELF**1. When is stratified sampling better than simple random sampling?**

When key subgroups are small or differ a lot; it guarantees representation and reduces variance.

2. What does the gradient tell gradient descent?

The direction of steepest increase of the loss; the algorithm steps in the opposite direction.

3. How do a 95% confidence interval and a 95% credible interval differ?

Confidence: 95% of intervals built this way would cover the true value over repeats. Credible: given the data and prior, there is 95% probability the parameter lies inside.

5.11 Stats Q&A and self-check

IN PLAIN WORDS

This is where you test yourself on statistics. The questions repeat a few ideas: what a p-value means, why the mean can mislead, how sample size shrinks uncertainty, which test fits which data, and what goes wrong when you test too much. A good answer has three parts: a plain definition, a tiny example with numbers, and one trap to avoid. If you can say those in under a minute without notes, you own the topic. If you stumble, return to the matching section and say it again out loud.

Why it matters: Interviewers probe statistics by asking you to explain, not compute.

Takeaway: Definition, example, trap, in under a minute.

THE FUNDAMENTALS

5.6 Stats Q&A (short answers)

5.7 Self-check (Module 5)

- ☐ Explain CLT, p-value, and confidence interval to a layperson in under a minute each.
- ☐ Solve the Bayes example without notes.
- ☐ Given a scenario, choose the correct test and state assumptions.

SAY IT IN 60 SECONDS

"When asked a statistics question I answer in three beats. First a plain definition in one sentence, with no jargon. Second a small example with real numbers, such as a coin or a conversion rate. Third the trap, the usual misunderstanding. For instance, for the p-value: the chance of data this extreme if there were no effect; for example, p equals 0.03 on a ten per cent versus eleven per cent conversion test; the trap is that it is not the chance the null is true. Then I stop and invite follow-up. The catch: formulas help, but interviewers listen for whether I can state assumptions and limits."

KEY TERMS Bessel's correction · non-normal data · p-value misconceptions · statistical vs practical significance · power · outliers · correlation vs causation

BUILDS ON 5.7 Hypothesis testing: the core definitions

HANDS-ON PRACTICE Answer each question aloud in under 60 seconds, then reveal and compare.

COMMON MISTAKES Giving one-line answers with no example · Forgetting to mention assumptions · Not distinguishing statistical and practical significance

CHECK YOURSELF

1. Give the three-beat answer for "what is a confidence interval?"

Definition: a range built so that 95% of such ranges cover the true value. Example: conversion 10% $\pm$ 1% from 10k users. Trap: not a 95% probability that this particular interval holds the truth.

2. How do you handle a statistics question you are unsure of?

State what you know, derive from first principles with a simple case, flag the assumption, and say how you would verify it. Never bluff.

3. Which five stats ideas should you be able to explain from memory?

p-value, confidence interval, Type I/II error and power, CLT, and Bayes' theorem with base rates.

Interview tips, tricks and hacks

- 1. Answer in three beats: definition, tiny example, caution.** "A p-value is the chance of data this extreme if there were no effect. For example, 0.03 means a gap this large would appear about 3 times in 100 no-effect experiments. It is not the chance the null is true."
- 2. Explain to a non-technical listener.** Interviewers often say "explain it to a product manager". Have an everyday version of the p-value, confidence interval, CLT and power ready, with no formulas.
- 3. Always mention the base rate in Bayes questions.** Use natural frequencies: "of 10,000 people, 100 are ill and 99 test positive; of 9,900 healthy, 495 test positive; so 99 out of 594." It is faster and harder to get wrong than the formula.
- 4. For puzzles, define the sample space first.** List outcomes, count, and say your assumptions. If unsure, describe how you would simulate it. A clear method with a small slip beats a confident wrong number.
- 5. Say "at least one" as one minus none.** It turns most hard-looking probability questions into a single multiplication.
- 6. Pair every significance claim with an effect size.** "Significant, and the lift is 1.2 percentage points, with an interval from 0.4 to 2.0." It shows you know that significance and importance differ.
- 7. Choose tests by asking three questions aloud.** Outcome type, number of groups, independent or paired. The interviewer hears the reasoning, which is what is being graded.
- 8. Know the four-times rule.** Halving the margin of error needs four times the sample. It is the quickest way to sanity-check a sample-size answer.
- 9. Never say "accept the null hypothesis".** You "fail to reject" it. A non-significant result with a small sample is absence of evidence, not evidence of absence.
- 10. When you forget a formula, rebuild it.** Standard error is spread over root n ; a test statistic is an estimate divided by its standard error. Deriving in front of the interviewer is a strength.

Frequently asked questions

44 questions interviewers ask on this chapter, with model answers. Cover the answer, say yours aloud, then compare.

DESCRIPTIVE STATISTICS

Q1 The mean delivery time is 38 minutes and the median is 31. What does this tell you, and what do you report to operations?

The mean exceeding the median indicates right skew: most deliveries are faster than 38 minutes, but a tail of slow ones inflates the average. Report the **median** for the typical experience and a **high percentile (P90 or P95)** for the bad tail, because operations cares about the late deliveries customers complain about. If cost is driven by total minutes, the mean still matters.

PROBABILITY FUNDAMENTALS

Q2 A test has 99% sensitivity. A patient tests positive. Is there a 99% chance they are sick?

The 99% is $P(\text{positive} \mid \text{sick})$. The question asks the reverse, $P(\text{sick} \mid \text{positive})$, which also needs the prevalence and the false-positive rate (Bayes' theorem). With 1% prevalence and a 5% false-positive rate it is only about 16.7%. Confusing the two conditional probabilities is the **base-rate fallacy**; see the Bayes topic.

Q3 Two independent systems each fail with probability 0.1. What is the probability that at least one fails?

Use the complement: both work with probability $0.9 \times 0.9 = 0.81$, so at least one fails with $1 - 0.81 = 0.19$. Using $P(A) + P(B) = 0.2$ would overcount the case where both fail (0.01); the addition rule's subtraction term fixes it: $0.1 + 0.1 - 0.01 = 0.19$.

BAYES' THEOREM: THE WORKED EXAMPLE

Q4 Redo the example with prevalence 10% instead of 1%. What is $P(\text{disease} \mid \text{positive})$?

$P(+) = 0.99 \times 0.10 + 0.05 \times 0.90 = 0.099 + 0.045 = 0.144$. $P(D \mid +) = 0.099 / 0.144 = 68.75\%$. A tenfold higher base rate lifts the posterior from 16.7% to 68.8%, with the same test. The test did not change; the population did.

Q5 A fraud model has 95% recall and a 1% false-positive rate. Fraud is 0.2% of transactions. Of the transactions it flags, what fraction is really fraud?

Per 100,000 transactions: 200 fraud, 99,800 legitimate. True positives = $0.95 \times 200 = 190$. False positives = $0.01 \times 99,800 = 998$. Precision = $190 / (190 + 998) = 16\%$. Even a model that looks strong on recall and false-positive rate produces mostly false alarms when the positive class is rare, so the review workload and the threshold must be chosen with the base rate in mind.

CLASSIC PROBABILITY PUZZLES

Q6 Two dice rolled: $P(\text{sum} = 7)$?

$6/36 = 1/6$

Q7 Family has two children, at least one is a girl: $P(\text{both girls})$?

$1/3$

Q8 Expected number of rolls of a fair die to get a 6?

6, geometric

Q9 Birthday problem: how many people make a shared birthday about 50% likely?

23 people give roughly a 50% chance that two share a birthday.

Q10 Monty Hall: should you switch doors, and what is the win probability if you do?

Yes. Switching wins with probability $2/3$ (staying wins only $1/3$).

Q11 You flip a fair coin until first head. Expected flips?

2

Q12 Drawing 2 cards from a deck: $P(\text{both aces})$?

$4/52 \times 3/51 = 1/221$

Q13 A biased coin with $P(H)=0.7$: how do you make a fair decision?

Flip twice: HT vs TH, discard HH/TT.

Q14 Expected value of a game: pay Rs 10, roll a die, win Rs 3 x face value. What is the expected value?

$E = 3 \times 3.5 = 10.5$; fair-plus

Q15 What is the expected number of coin flips to get two heads in a row (HH)?

Use states. Let E be the expected flips from the start, and H the expected flips when the last flip was a head. From the start: flip once; with probability 1/2 you are in state H (cost 1 flip + H), with probability 1/2 you stay at the start (1 flip + E). So $E = 1 + \frac{1}{2}H + \frac{1}{2}E$. From state H: flip again; with probability 1/2 you finish (1 flip), with probability 1/2 you fall back to the start (1 flip + E). So $H = 1 + \frac{1}{2}E$. Substituting: $E = 1 + \frac{1}{2}(1 + \frac{1}{2}E) + \frac{1}{2}E = 1.5 + \frac{3}{4}E$, so $E = 6$. (For HT the answer is 4, which is surprising: different patterns have different expected waiting times.)

Q16 Three cards: one is red on both sides, one is white on both sides, one is red on one side and white on the other. You draw one at random and see a red face. What is the probability the other side is also red?

There are six equally likely faces: R1, R2 (card 1), W1, W2 (card 2), R3, W3 (card 3). You saw a red face, which is R1, R2 or R3. For R1 and R2 the other side is red; for R3 it is white. So 2 of 3 equally likely red faces have a red back: **2/3**. The mistake is treating the two cards that *could* show red as equally likely; the double-red card has two ways to show red.

Q17 You roll a fair die up to four times hoping for a six (Chevalier de Méré's problem). Is it better than even odds?

$P(\text{at least one six in 4 rolls}) = 1 - (5/6)^4 = 1 - 0.482 = \mathbf{51.8\%}$, slightly better than even. The famous companion bet (at least one double six in 24 rolls of two dice) is $1 - (35/36)^{24} = 49.1\%$, slightly worse than even, which is how gamblers first discovered that intuition about small probabilities is unreliable.

DISTRIBUTIONS YOU MUST KNOW**Q18 Which distribution would you use for: (a) number of support tickets per day, (b) time until the next ticket arrives, (c) whether a single email is opened, (d) number of opened emails out of 500 sent?**

(a) Tickets per day are counts in a fixed interval: **Poisson**(λ = average per day). (b) The waiting time until the next ticket is **exponential** (mean $1/\lambda$), memoryless. (c) One yes/no outcome: **Bernoulli**(p). (d) Number of successes in $n = 500$ independent trials: **binomial**(500, p), approximately normal for large n . State the assumptions (independence, constant rate) and mention overdispersion if real counts vary more than a Poisson predicts.

CENTRAL LIMIT THEOREM AND THE LAW OF LARGE NUMBERS**Q19 A survey of 100 people gives a mean satisfaction of 7.2 with standard deviation 1.5. What is the standard error, and roughly what is the 95% confidence interval for the true mean?**

$SE = \sigma / \sqrt{n} = 1.5 / 10 = \mathbf{0.15}$. A 95% confidence interval is about mean $\pm 1.96 \times SE = 7.2 \pm 0.29 = \mathbf{(6.91, 7.49)}$. With $n = 100$ the t critical value is about 1.98, which gives nearly the same interval. To narrow the interval to ± 0.15 you would need four times the sample size ($n = 400$).

Q20 What is the difference between the standard deviation and the standard error?

The **standard deviation** describes how far individual observations are from the mean; it does not shrink as you collect more data. The **standard error** of the mean, $\sigma/\sqrt{n}$, describes how far the *sample mean* is from the true mean; it shrinks as n grows. Error bars on a chart should say which one they show.

HYPOTHESIS TESTING: THE CORE DEFINITIONS**Q21 A test of a new feature returns $p = 0.048$, and a colleague tests 20 other metrics on the same experiment and finds one with $p = 0.03$. Which result do you trust more, and why?**

When you test many metrics, the chance of at least one false positive rises (with 20 independent tests at $\alpha = 0.05$ it is $1 - 0.95^{20} \approx 64\%$). A result found by searching through metrics is **exploratory**; a **pre-specified primary metric** is **confirmatory**. Trust the primary metric, treat the other finding as a hypothesis to test in a new experiment, or apply a multiple-comparison correction (Bonferroni, Holm or Benjamini-Hochberg). Also check effect size and whether $p = 0.048$ is borderline enough that a small analysis choice would change it.

WHICH TEST DO I USE?**Q22 Does the average order value differ between Android and iOS users (each user contributes one average; 5,000 users per group, somewhat skewed)?**

Two independent numeric groups with large n : **Welch's two-sample t-test** (the CLT handles moderate skew). If the metric is very heavy-tailed, cap outliers or use a bootstrap for the difference in means, or **Mann-Whitney U** if you want a rank-based comparison. State that you compare means, which suits revenue totals.

Q23 The same 200 customers are surveyed for satisfaction before and after a support redesign. Which test?

The observations are **paired** (same people twice), so analyse the within-person differences: a **paired t-test** on the differences (Wilcoxon signed-rank if the differences are skewed or ordinal). An independent test would ignore the pairing and lose power.

Q24 Is the choice of payment method (UPI, card, COD) related to whether an order is cancelled?

Both variables are categorical, so use the **chi-square test of independence** on the payment × cancelled table (expected counts should be at least about 5 per cell; otherwise use Fisher's exact test or merge categories). A significant result says they are associated; follow up with cell residuals or cancellation rates per method to say *how*.

Q25 You run an A/B/C test on three checkout designs with conversion as a binary outcome. What do you do?

First an **omnibus chi-square test** of independence for the 3 × 2 table (is conversion related to design?). If significant, run pairwise **two-proportion z-tests** with a correction for three comparisons (Holm or Bonferroni). Report each lift with a confidence interval, and decide the winner considering effect size and guardrail metrics.

Q26 Delivery-time samples from 10 orders in each of two cities are strongly right-skewed. Which test?

With $n = 10$ per group and strong skew, the t-test's normality assumption is doubtful and the CLT cannot rescue you. Use **Mann-Whitney U**, or log-transform the times (often roughly normal) and run Welch's t-test, or a **permutation/bootstrap test** on the median difference. Mention that the rank-based test compares distributions (stochastic dominance), not strictly means.

Q27 You want to know whether a model's prediction errors have the same variance in two customer segments.

Use **Levene's test** for equality of variances (robust to non-normality); the **F-test** is sensitive to non-normality. This is the usual check behind choosing equal- vs unequal-variance t-tests, though Welch's test makes it largely unnecessary.

MULTIPLE TESTING AND PYTHON RECIPES

Q28 You ran an experiment and look at conversion in 12 segments (city, device and so on); two segments show $p < 0.05$. What do you conclude?

At $\alpha = 0.05$, 12 independent looks give a 46% chance of at least one false positive and about 0.6 expected false positives. So two "significant" segments are weak evidence. Apply Holm or BH across the 12, check that the effect is large and plausible (and was hypothesised before), and confirm in a follow-up experiment. Also be wary of **subgroup** analysis when the overall effect is flat.

OTHER ESSENTIALS: SAMPLING, MLE, BAYESIAN, LINEAR ALGEBRA, CALCULUS

Q29 You estimate the conversion rate from 1 success in 5 trials. What is the MLE, and why might you not trust it?

The MLE is the sample proportion, $1/5 = 0.2$. With five trials the estimate is extremely uncertain (a 95% interval spans roughly 0.01 to 0.72). A Bayesian estimate with a sensible prior (for example Beta(2, 18) reflecting a typical 10% rate) shrinks it toward prior knowledge: the posterior mean is $(2 + 1) / (2 + 18 + 5) = 0.12$. This "shrinkage" is also what regularisation does.

STATS Q&A AND SELF-CHECK

Q30 Why N-1 in sample variance?

The sample mean is estimated from the same data, so deviations are slightly too small; dividing by N-1 gives an unbiased estimate.

Q31 What if data is not normal?

With large n CLT helps; otherwise use non-parametric tests, transformations, or bootstrap.

Q32 Does a large p-value prove H_0 ?

No; it means insufficient evidence against H_0 .

Q33 Statistical vs practical significance?

With huge n , tiny effects become "significant"; always report effect size and CI.

Q34 What affects power?

Effect size, sample size, alpha, variance (and one- vs two-tailed).

Q35 How to handle outliers?

Investigate first; options are cap/winsorize, transform, robust models, or remove if an error.

Q36 Correlation vs causation?

Confounders, reverse causality, coincidence; establish causation via randomized experiments or causal inference methods.

Q37 Mean vs median when to use?

Median for skewed data (income, delivery time); mean when symmetric or when totals matter.

Q38 What is a confidence interval for a proportion?

$p_{\text{hat}} \pm z * \sqrt{p_{\text{hat}}(1-p_{\text{hat}})/n}$.

Q39 What is the odds ratio?

Ratio of odds between groups; in logistic regression $\exp(\text{coefficient})$.

Q40 What is heteroscedasticity?

Non-constant error variance; affects standard errors, fix via log transform, robust SEs, WLS.

Q41 Difference between parametric and non-parametric?

Parametric assume a distribution form (more power if true); non-parametric make fewer assumptions.

Q42 Why do we use the median for income but the mean for total revenue?

For a skewed variable the median describes the typical unit; for totals the mean is essential because $\text{total} = \text{mean} \times n$, and the median loses that relationship. Choose by the decision: "what does a typical customer look like?" (median) versus "how much revenue should we plan for?" (mean).

Q43 What is the difference between a confidence interval and a prediction interval?

A confidence interval quantifies uncertainty in an **estimated parameter** (the average order value) and shrinks as n grows. A **prediction interval** covers where a **single future observation** will fall; it includes both parameter uncertainty and the individual-level noise, so it stays wide however much data you have.

Q44 What happens to a confidence interval if you double the sample size?

The width is proportional to $\sigma/\sqrt{n}$, so doubling n divides the width by $\sqrt{2} \approx 1.41$, a 29% reduction. To halve the interval you need four times the data, which is why precision gets expensive.

DESCRIPTIVE STATISTICS

- **Mean** = sum/n (pulled by outliers). **Median** = middle value (robust). **Mode** = most frequent.
- **Variance** (sample) = $\sum(x - \text{mean})^2 / (n - 1)$; population divides by n. **SD** = $\sqrt{\text{variance}}$, same units as x.
- **IQR** = $Q3 - Q1$. Outlier rule: below $Q1 - 1.5 \cdot \text{IQR}$ or above $Q3 + 1.5 \cdot \text{IQR}$.
- **z-score** = $(x - \text{mean}) / \text{SD}$. Skew: right-skewed (income) means mean > median.
- **Covariance** = $\sum(x - \text{mx})(y - \text{my}) / (n - 1)$. **Correlation r** = $\text{cov} / (\text{sx} \cdot \text{sy})$, from -1 to 1: linear association only, units-free. $r = 0$ does not mean independent.
- Dividing by $n - 1$ (Bessel) corrects for estimating the mean from the same data.
- Robust spread: **MAD** = median $|x - \text{median}|$.

TRAP Correlation is not causation; confounders and reverse causation. Simpson's paradox: a trend reverses when groups are combined.

PROBABILITY RULES

- $0 \leq P(A) \leq 1$; $P(\text{not } A) = 1 - P(A)$. "At least one" = $1 - P(\text{none})$.
- Addition: $P(A \text{ or } B) = P(A) + P(B) - P(A \text{ and } B)$.
- Multiplication: $P(A \text{ and } B) = P(A) P(B | A)$. Independent: $P(A \text{ and } B) = P(A) P(B)$.
- Conditional: $P(A | B) = P(A \text{ and } B) / P(B)$.
- Total probability: $P(B) = \sum P(B | A_i) P(A_i)$.
- **Mutually exclusive** (cannot both happen) is not **independent**; exclusive events with $P > 0$ are dependent.
- Counting: $n!$ orderings; $nPr = n! / (n - r)!$; $nCr = n! / (r!(n - r)!)$.
- Expectation: $E[X + Y] = E[X] + E[Y]$ always (linearity). $\text{Var}(aX + b) = a^2 \text{Var}(X)$; $\text{Var}(X + Y) = \text{Var } X + \text{Var } Y$ if independent.

HANDY FACTS AND INEQUALITIES

- Markov: $P(X \geq a) \leq E[X] / a$ ($X \geq 0$). Chebyshev: $P(|X - \mu| \geq k\sigma) \leq 1/k^2$, any distribution (75% within 2σ).
- Percentiles: median = 50th, quartiles 25/75th; for the normal, the 97.5th percentile is $\mu + 1.96\sigma$.
- Skewness: sign gives the long tail; kurtosis: heavy tails. Log-transform right-skewed positives.
- Regression slope = $r(\text{sy}/\text{sx})$. Correlation is symmetric; regression is not.
- Law of total expectation: $E[X] = \sum E[X | A] P(A)$. Variance of a sum of n iid = $n\sigma^2$.
- Independence of events: $P(A | B) = P(A)$. Conditional independence differs from marginal independence.

- Expected value of a game = $\sum \text{value} \times \text{probability}$; decide with EV, mention variance.

BAYES' THEOREM

$P(H | E) = P(E | H) P(H) / P(E)$, posterior = likelihood $\times$ prior / evidence.

- Worked example: prevalence 1%, sensitivity 99%, false-positive 5%. $P(+)$ = $0.99 \cdot 0.01 + 0.05 \cdot 0.99 = 0.0594$. $P(\text{disease} | +)$ = $0.0099 / 0.0594 = 16.7\%$.
- Natural frequencies: of 10,000 people, 100 sick (99 test positive), 9,900 healthy (495 test positive): 99 of 594.
- Base-rate neglect: ignoring a small prior. Sensitivity = $TP / (TP + FN)$; specificity = $TN / (TN + FP)$.
- Odds form: posterior odds = prior odds $\times$ likelihood ratio.

CLASSIC PUZZLES

Puzzle	Answer
Two dice sum to 7	$6/36 = 1/6$
Two children, at least one girl, both girls	$1/3$
Birthday: 23 people share a birthday	> 50% (253 pairs)
Monty Hall	switch wins 2/3
Flips to first head	$E = 2$
Flips to HH / HT	$6 / 4$
Rolls to first six	6
Coupon collector, n types	$n(1 + 1/2 + \dots + 1/n)$

Method: define the sample space, count, then check by simulation.

DISTRIBUTIONS

Distribution	Mean	Variance	Use
Bernoulli(p)	p	$p(1 - p)$	one yes/no trial
Binomial(n, p)	np	$np(1 - p)$	successes in n trials
Geometric(p)	$1/p$	$(1 - p)/p^2$	trials to first success
Poisson(λ)	λ	λ	events per interval, rare and independent
Uniform(a, b)	$(a + b)/2$	$(b - a)^2/12$	equally likely
Normal(μ, σ^2)	μ	σ^2	sums and averages, errors
Exponential(λ)	$1/\lambda$	$1/\lambda^2$	waiting time, memoryless
Beta(α, β)	$\alpha/(\alpha + \beta)$		probability of a probability

- Binomial with large n, small p $\rightarrow$ Poisson(np). Normal approx when np and $n(1 - p) > 10$.
- Normal: 68% within 1σ , 95% within 2σ (1.96σ), 99.7% within 3σ . z crit: 1.645 (90%), 1.96 (95%), 2.576 (99%).
- t has heavier tails than normal, approaches it as df grows. Chi-square = sum of squared normals. F = ratio of variances.
- Log-normal / power law for incomes, latencies: report medians and percentiles.

PROBABILITY RECIPES

- Binomial: $P(X = k) = C(n, k) p^k (1 - p)^{n-k}$. Poisson: $P(X = k) = e^{-\lambda} \lambda^k / k!$.
- Draw without replacement (cards): multiply conditional probabilities, or use combinations: $P(2 \text{ aces}) = C(4, 2) / C(52, 2) = 1/221$.
- At least one success in n tries: $1 - (1 - p)^n$ (four rolls, a six: $1 - (5/6)^4 = 51.8\%$).
- Expected value of a die = 3.5; of a sum = sum of expectations. Gambler's game: $EV = \sum \text{payout} \times \text{probability}$.
- Conditional on a total: "given sum = 7, $P(\text{first die} = 3) = 1/6$ by symmetry.
- Random variable transformations: $E[aX + b] = a E[X] + b$. Sum of independent normals is normal.
- Simulate when stuck: `np.random.binomial`, then count frequencies.

CLT AND LLN

- **LLN**: the sample mean converges to the true mean as n grows.
- **CLT**: the mean of n independent samples is approximately normal with mean μ and **standard error SE** = $\sigma / \sqrt{n}$, whatever the data's shape (finite variance), usually $n \geq 30$, more for heavy skew.
- $4 \times$ the data halves the error. It is about **means**, not about the data becoming normal.
- SE of a proportion = $\sqrt{p(1 - p)/n}$. SE of a difference = $\sqrt{SE1^2 + SE2^2}$.
- Law of rare events, bootstrap and the normal approximation all lean on the CLT.

CONFIDENCE INTERVALS

- Mean: sample mean $\pm t \cdot s / \sqrt{n}$ (z when n is large or σ known). Proportion: $p \pm 1.96 \cdot \sqrt{p(1-p)/n}$.
- 95% CI meaning:** if we repeated the sampling and the method many times, 95% of the intervals would cover the true value. Not "95% probability this interval holds it".
- Wider with more variance, higher confidence, smaller n. Margin of error = critical value $\times$ SE.
- Report the interval, not just the p-value: it shows effect size and uncertainty.

HYPOTHESIS TESTING

- State H_0 (no effect) and H_1 .
 - Choose alpha (0.05) and the test.
 - Compute the statistic and **p-value**.
 - Decide and interpret.
- p-value** = P(data this extreme or more | H_0 true). Not P(H_0 true), not the effect size.
 - Type I** (false positive, rate α): reject a true H_0 . **Type II** (miss, rate β): fail to reject a false H_0 . **Power** = $1 - \beta$, usually 80%.
 - Power rises with larger n, larger effect, lower variance, higher α . One-sided tests need a prior direction.
 - Statistical vs practical significance: huge n makes tiny effects significant. Report effect size (difference, Cohen's d, odds ratio).
 - Not significant $\neq$ no effect: low power gives wide intervals.

TRAP Peeking at results and stopping at the first $p < 0.05$ inflates false positives.

WHICH TEST?

Data and question	Test
Mean vs known value	one-sample t
Two independent means	Welch t-test (unequal variances)
Same units before/after	paired t-test
3+ group means	ANOVA, then post-hoc (Tukey)
Two proportions	two-proportion z-test / chi-square
Two categoricals	chi-square of independence; Fisher if counts < 5
Fit to a distribution	chi-square goodness of fit, KS
Two samples, non-normal / ordinal	Mann-Whitney U; paired: Wilcoxon
3+ groups, non-normal	Kruskal-Wallis
Linear association	Pearson r; rank-based: Spearman
Two distributions differ	two-sample KS

- t statistic = (sample mean - μ_0) / (s / $\sqrt{n}$). Two proportions: $z = (p_1 - p_2) / \sqrt{p(1-p)(1/n_1 + 1/n_2)}$ with pooled p.
- Assumptions: independence, roughly normal means (CLT), equal variances for classic t/ANOVA. Heavy tails: bootstrap or transform.

SAMPLE SIZE (PER GROUP)

- Means: $n \sim 16\sigma^2/\delta^2$ (80% power, $\alpha = 0.05$). Proportions: $n \sim 16p(1-p)/\delta^2$.
- Example: baseline 10%, detect +1pp: $16 \cdot 0.09/0.0001 \sim 14,400$ per group.
- Halving the detectable effect quadruples n. Duration = total n / daily eligible traffic, in whole weeks.

MULTIPLE TESTING

- m tests at $\alpha = 0.05$: P(at least one false positive) = $1 - 0.95^m$ (20 tests: 64%).
- Bonferroni:** use α/m (controls family-wise error; strict). **Benjamini-Hochberg:** controls the false discovery rate; rank p-values, keep $p(i) \leq (i/m) \cdot q$.
- Pre-register one primary metric; treat post-hoc segment hits as hypotheses.

```
from scipy import stats
stats.ttest_ind(a, b, equal_var=False)
stats.ttest_rel(a, b)
stats.chi2_contingency(table)
stats.mannwhitneyu(a, b);
stats.ks_2samp(a, b)
stats.pearsonr(x, y);
stats.norm.ppf(0.975)
import statsmodels.stats.multitest as mt
mt.multipletests(p, method="fdr_bh")
from statsmodels.stats import power
power.TTestIndPower().solve_power(
    effect_size=0.2, power=0.8,
    alpha=0.05)
```

EFFECT SIZES AND ASSUMPTIONS

Quantity	Formula / use
Absolute lift	$p_{\text{treat}} - p_{\text{control}}$
Relative lift	$(p_{\text{treat}} - p_{\text{control}}) / p_{\text{control}}$
Cohen's d	difference in means / pooled SD (0.2 small, 0.5 medium, 0.8 large)
Odds ratio	$[p/(1-p)]_{\text{treat}} \div \text{control}$
R^2	$1 - SS_{\text{res}}/SS_{\text{total}}$

- Check normality with a histogram or Q-Q plot, equal variances with Levene's test, independence from the design (clustered users break it).
- Violations: transform (log), use Welch, use non-parametric tests, or bootstrap.
- Paired data: analyse differences; clustered data: aggregate to the randomisation unit.

OTHER ESSENTIALS

- Sampling:** simple random, stratified (represent subgroups), cluster. Biases: selection, survivorship, non-response.
- Bootstrap:** resample with replacement, recompute the statistic, take percentiles for a CI. **Permutation test:** shuffle labels.
- MLE:** choose parameters maximising the likelihood; for a normal it gives the sample mean and variance (divides by n). **MAP** adds a prior.
- Bayesian vs frequentist:** posterior = prior $\times$ likelihood gives probability statements about parameters; credible interval is a probability statement, a confidence interval is a procedure property.
- Linear algebra:** eigenvector keeps direction under a matrix; PCA = eigenvectors of the covariance matrix; SVD = $U \Sigma V^T$.
- Calculus:** the gradient points uphill; gradient descent steps against it; chain rule powers backprop.
- Regression: R^2 = explained variance share; coefficient t-test = estimate/SE.

SAY IT IN 20 SECONDS

- p-value:** "how surprising this data is if nothing is going on".
- CI:** "a range from a procedure that covers the truth 95% of the time".
- CLT:** "averages are roughly normal with SD $\sigma/\sqrt{n}$ ".
- Power:** "the chance to detect a real effect of this size".
- Bayes:** "update the prior with the likelihood; never forget the base rate".
- Definition, tiny example, one caution.

6

CHAPTER 6 · HIGH PRIORITY · MODULE 6 IN THE APP

A/B Testing and Experimentation

An A/B test is the cleanest way to learn whether a change causes an effect. Interviewers ask you to design one, size it, read it and spot what could go wrong.

5 topics · about 2 h 35 min in the app · 11 questions answered · two-page cheat sheet at the end

6.1 The experiment lifecycle

6.2 Sample size: the back-of-envelope method

6.3 A/B testing pitfalls

6.4 Advanced experimentation ideas

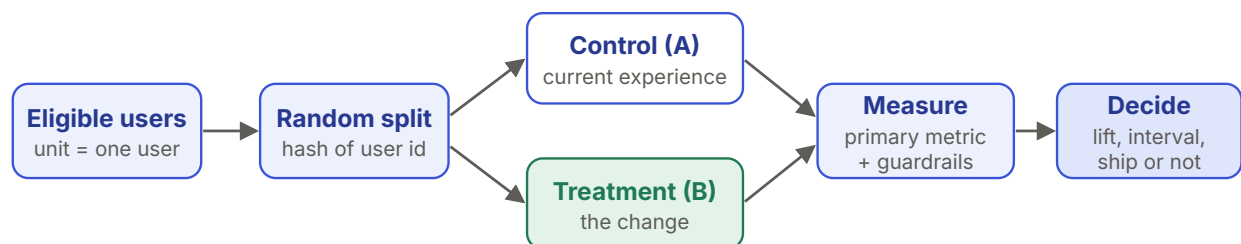
6.5 Practice case questions and self-check

Foundations

Suppose a product team changes the checkout button and sales rise the following week. Did the button cause it? Perhaps there was a holiday, a marketing campaign, or simply a good week. Observing a change after an action never proves the action caused it. An **A/B test**, also called a randomised controlled experiment, is how companies get a trustworthy answer, and it is the most direct application of Chapter 5.

Why randomisation is the whole trick

Split users **at random** into two groups. One sees the current version (control), the other sees the change (treatment). Because the split is random, the two groups are alike, on average, in every respect: age, device, loyalty, mood, the weather that week. The only systematic difference is the change itself. So a difference in the outcome, beyond what chance alone would produce, can be attributed to the change.



Randomisation makes the groups alike in everything except the change, so the difference is causal.

Figure 8. Random assignment, one change, one decision metric, and a decision rule written in advance.

Design before data

A sound experiment is decided on paper before a single user is assigned. You need five things.

1. A **hypothesis** tied to a decision: what will we do differently if this wins?
2. One **primary metric** that defines success, plus **guardrail metrics** that must not get worse, such as page speed, refund rate or unsubscribes.

3. A **unit of randomisation**, usually the user, and the same unit for analysis.
4. A **sample size** and duration, computed from the baseline, the smallest effect worth detecting, the significance level and the power.
5. A **decision rule**: what result leads to ship, iterate or stop.

Deciding these in advance is what prevents you from fooling yourself afterwards.

Sample size in one line

The smaller the effect you want to detect, the more users you need, and the relationship is quadratic. A workable rule of thumb for 5 per cent significance and 80 per cent power is: users per group is about sixteen times the variance divided by the square of the difference you want to detect. For a 10 per cent conversion rate and a one-percentage-point lift, that is roughly 14,400 per group. Halve the detectable lift and you need four times as many. Divide the total by daily traffic to get the duration, and round up to whole weeks so that weekdays and weekends are represented equally.

Reading the result

Check the plumbing first: if you planned a 50/50 split and got something clearly different, a bug in assignment or logging has broken the experiment (a **sample ratio mismatch**) and no analysis can rescue it. Then report the estimated effect with a **confidence interval**, not just a p-value, and compare the interval with the minimum effect that would justify the cost. Check the guardrails. Treat any segment-level finding you did not plan for as a hypothesis for the next test, because slicing data many ways will always turn up something "significant" by chance.

How experiments go wrong

Most failures are procedural, not mathematical. **Peeking** at results daily and stopping the moment they look significant inflates false positives enormously. **Novelty effects** make a new feature look better in its first days than it will be. **Interference** breaks the logic of the test when users affect one another, as in marketplaces where riders in both groups compete for the same drivers; the remedy is to randomise whole cities or time periods. An **underpowered** test that finds nothing has not shown that nothing is there.

Beyond the basic test

You should know, at the level of a sentence each, what exists when the standard design does not fit: variance reduction such as CUPED, which uses pre-experiment behaviour to detect smaller effects; sequential methods that permit valid early looks; bandits that shift traffic to the better option as evidence arrives; and quasi-experiments such as difference-in-differences when randomisation is impossible.

Topic by topic

Each topic gives the idea in plain words, the fundamentals to know, a model spoken answer and three questions to test yourself. Open the same topic in the app for worked examples, runnable code and interactive demos.

6.1 The experiment lifecycle

IN PLAIN WORDS

An A/B test is a fair race between two versions. You flip a coin for each visitor to decide whether they see the old page (A) or the new page (B), wait until enough people have visited, and compare one agreed measure (like "did they buy?"). The coin flip is what makes it fair: the two groups are alike in every way except the page, so any difference can be blamed on the page. Before you start you decide what counts as winning, how many people you need, and what must not get worse (like page speed or refunds).

Why it matters: Near-certain in product-company interviews: "design an A/B test for X".

Takeaway: Hypothesis, metrics, unit, sample size, run, analyse, decide, with the decision rule written before you start.

WORKED EXAMPLE, STEP BY STEP

Write the one-page plan before any user sees the change

A food-delivery app wants to lower the free-delivery threshold from 299 to 249 rupees.

- 1. Hypothesis and decision.** "A lower threshold raises orders per user enough to pay for the extra free deliveries. If it does, we roll it out in all cities."
- 2. Primary metric.** Contribution margin per user over 14 days. Orders alone would reward giving away delivery.
- 3. Guardrails.** Average order value (it may fall as people stop adding items to reach 299), delivery-partner wait time, and cancellation rate.
- 4. Unit.** Randomise by user, not by order, so the same person always sees the same threshold.
- 5. Size and duration.** Work out the sample size from the baseline and the smallest worthwhile lift (topic 6.2), then round the duration up to whole weeks, say 2.
- 6. Checks during the test.** Sample ratio mismatch on day 1; no decisions from peeking.
- 7. Decision rule, written now.** Ship if margin per user rises with the confidence interval above zero and the guardrails are flat; iterate if orders rise but margin falls; stop otherwise.

What this shows. Everything is decided before the data exists, which is what stops you from finding a convenient story afterwards.

YOUR TURN (COVER THE SOLUTION FIRST)

1. A team tests a new daily push notification and plans to judge it on "app opens per user". Name the main risk with that metric and three guardrails you would add.

Solution. Notifications almost always increase app opens in the short term, so the primary metric is easy to move without creating value, and opens are a weak proxy for value. Guardrails: notification opt-out or uninstall rate, orders or revenue per user (does value follow the opens?), and longer-term retention, for example 28-day active users. A good primary metric would be orders per user or retained users, with opens as a secondary metric.

THE FUNDAMENTALS

1. **Define the goal and hypothesis:** "Changing the checkout button color will increase conversion."
2. **Choose metrics**
 - **Primary (success) metric:** one, sensitive, tied to the goal (e.g., checkout conversion rate).
 - **Secondary metrics:** diagnostic (add-to-cart rate, AOV).
 - **Guardrail metrics:** must not degrade (latency, crash rate, refund rate, unsubscribe rate).
3. **Choose the randomization unit:** user (most common), session, device, cookie, store/city (for marketplaces, use geo or switchback to avoid interference).
4. **Estimate sample size** (see below) and **duration** (cover at least 1-2 full weekly cycles; avoid holidays/sales skew).
5. **Run:** check for **Sample Ratio Mismatch (SRM)** with a chi-square test on group sizes (if you planned 50/50 and got 50.6/49.4 with large n, something is broken).
6. **Analyze:** pre-planned test, CI for the difference, segment cuts only as hypothesis generators.
7. **Decide:** ship / iterate / kill, considering effect size, cost, guardrails, novelty.

SAY IT IN 60 SECONDS

"An A/B test is a controlled experiment. The lifecycle has seven steps. One: state the hypothesis and the decision it feeds. Two: pick a primary metric, a few guardrail metrics that must not get worse, and the unit of randomisation, usually the user. Three: compute the sample size and run time from the baseline, the minimum effect worth detecting, alpha and power. Four: randomise and check the setup with an A/A test or a sample ratio mismatch check. Five: run for the full planned duration, including whole weeks. Six: analyse with the pre-planned test. Seven: decide, document and monitor after launch. The catch: peeking at results early and stopping on the first significant day inflates false positives."

KEY TERMS hypothesis · primary metric · guardrail metric · randomization unit · sample ratio mismatch · duration · decision rule

BUILDS ON 5.7 Hypothesis testing: the core definitions

HANDS-ON PRACTICE Write a one-page design for an A/B test of a new delivery-fee message, covering all seven steps.

COMMON MISTAKES Choosing several primary metrics · Randomising by session but analysing by user · Ending the test as soon as it looks significant

CHECK YOURSELF**1. Why run an experiment for full weeks even if significance appears on day 3?**

Behaviour varies by weekday and early results are noisy; stopping at the first significant peek inflates the false-positive rate.

2. What is a guardrail metric? Give an example.

A metric that must not degrade while we optimise the primary one, such as page load time or unsubscribe rate when testing a new notification.

3. Expected 50/50 split but you see 52/48 on a million users. What does it signal?

A sample ratio mismatch: a chi-square test will flag it as impossible by chance, meaning a bug in assignment or logging; do not trust the results.

6.2 Sample size: the back-of-envelope method**IN PLAIN WORDS**

To notice a small difference between two groups you need lots of people, like needing a very precise scale to tell whether two apples differ by one gram. Smaller differences need more people; noisier measurements need more people; wanting to be more certain needs more people. There is a handy rule of thumb: people needed in each group is about $16 \times (\text{how noisy the measure is}) \div (\text{the smallest difference you care about})^2$. Halve the difference you care about and you need four times as many people.

Why it matters: Interviewers ask you to do this on the spot, in your head.

Takeaway: $n \text{ per group} \approx 16\sigma^2/\delta^2$; for a proportion, $16p(1-p)/\delta^2$. Halving δ quadruples n .

WORKED EXAMPLE, STEP BY STEP**Size a test with the rule of thumb**

Baseline conversion is 8%. The team cares about a lift of 1 percentage point (to 9%). There are 3,000 eligible users a day.

- 1. Variance of a yes/no outcome:** $p(1 - p) = 0.08 \times 0.92 = 0.0736$.
- 2. The rule of thumb** for 5% significance and 80% power: $n \text{ per group} \approx 16 \times \text{variance} / \delta^2$, where δ is the difference to detect.
- 3. Plug in:** $16 \times 0.0736 / 0.01^2 = 16 \times 0.0736 / 0.0001 = \mathbf{11,776 \text{ users per group}}$.
- 4. Total and duration:** $2 \times 11,776 = 23,552 \text{ users} \div 3,000 \text{ a day} \approx 7.9 \text{ days}$. Round up to **2 full weeks** so that every weekday is covered at least once and novelty has time to fade.
- 5. What if the team wants to detect 0.5 points?** δ halves, so n quadruples: 47,104 per group, about a month of traffic. Is a 0.5-point lift worth a month? That is a business question.
- 6. If the traffic is not there,** raise the minimum detectable effect, use a more sensitive metric, reduce variance with CUPED, or accept a longer run.

What this shows. Sample size follows from four inputs: baseline, minimum effect, significance and power. Halving the effect you want to detect costs four times the users.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Average order value has a standard deviation of 400 rupees. How many users per group do you need to detect a 20-rupee change, using the same rule of thumb?

Solution. For a mean, the variance is $\sigma^2 = 400^2 = 160,000$. $n \approx 16 \times 160,000 / 20^2 = 2,560,000 / 400 = 6,400$ users per group. If order value is heavily skewed, the real requirement can be higher; trimming extreme values or using CUPED reduces the variance and therefore n .

THE FUNDAMENTALS

For a two-sided test with $\alpha = 0.05$ and power = 0.8, per-group sample size:

- **Continuous metric:** $n \sim 16 * \sigma^2 / \delta^2$
- **Proportion:** $n \sim 16 * p(1 - p) / \delta^2$

where δ is the minimum detectable effect (MDE) in absolute terms.

Example: baseline conversion 10%, want to detect +1 pp (10% to 11%). $n \sim 16 \times 0.1 \times 0.9 / 0.01^2 = 16 \times 0.09 / 0.0001 = 14,400$ per group.

```
from statsmodels.stats.power import NormalIndPower
from statsmodels.stats.proportion import proportion_effectsize
es = proportion_effectsize(0.11, 0.10)
n = NormalIndPower().solve_power(effect_size=es, alpha=0.05, power=0.8, ratio=1)
```

Levers to reduce required sample size: larger MDE, lower-variance metric, CUPED/variance reduction, one-tailed (rarely justified), accept lower power.

SAY IT IN 60 SECONDS

"Sample size depends on four things: the baseline rate or variance, the smallest effect I care about, the significance level and the power. The back-of-envelope formula for each group is about sixteen times variance divided by delta squared, for five per cent alpha and eighty per cent power. For a conversion of ten per cent and a target lift of one percentage point, variance is point one times point nine, about 0.09, so n is sixteen times 0.09 over 0.0001, roughly fourteen thousand per group. Halving the detectable effect quadruples the sample. The catch: I divide the total by daily traffic to get run time, and a smaller effect than planned means an underpowered test."

KEY TERMS minimum detectable effect · power · alpha · sample size formula · baseline conversion · variance · test duration

BUILDS ON 6.1 The experiment lifecycle

HANDS-ON PRACTICE Compute the sample size for three MDEs at a 10% baseline and convert each into days of traffic.

COMMON MISTAKES Using relative lift as if it were absolute · Forgetting that the formula is per group · Planning a test that cannot reach the required sample in a sensible time

CHECK YOURSELF**1. Baseline conversion 10%, detect +1pp. Rough n per group?**

$16 \times 0.09 / 0.01^2 \approx 14,400$ per group.

2. What happens to the required sample if the minimum detectable effect is halved?

It quadruples, since n scales with $1/\delta^2$.

3. Traffic is 2,000 eligible users a day split in two groups, need 14,400 per group. Duration?

$28,800 / 2,000 = \text{about } 14.4 \text{ days}$; round up to 3 full weeks.

6.3 A/B testing pitfalls

IN PLAIN WORDS

The biggest trap in A/B testing is looking too often. If you check the results every day and stop as soon as you see a win, you will often stop on a lucky streak, like rolling dice until you get a six and declaring the dice magic. Other traps are just as sneaky: a new design is exciting at first then fades, treatment users affect control users (like drivers shared between groups), or you count visits as if they were different people. A good experimenter knows these traps and designs around them.

Why it matters: "What could go wrong with this experiment?" is a standard follow-up question.

Takeaway: Fix the sample size in advance, randomise and analyse by the same unit, run full weeks, and check for SRM.

WORKED EXAMPLE, STEP BY STEP**Check the split before reading the results**

You planned a 50/50 test. After a week: 50,600 users in control and 49,400 in treatment. Close enough?

- 1. Expected counts if the split is working:** 50,000 and 50,000.
- 2. Chi-square statistic:** $\sum (\text{observed} - \text{expected})^2 / \text{expected} = 600^2 / 50,000 + 600^2 / 50,000 = 7.2 + 7.2 = \mathbf{14.4}$.
- 3. p-value** for chi-square 14.4 with 1 degree of freedom $\approx \mathbf{0.00015}$. A random 50/50 split would produce an imbalance this large about 1.5 times in 10,000.
- 4. Conclusion: sample ratio mismatch.** Something is removing users unevenly: perhaps the new page crashes on some phones before the user is logged, or a bot filter treats the two arms differently.
- 5. Do not read the conversion numbers.** The groups are no longer comparable, so any difference may come from *who* is missing rather than from the change.
- 6. Fix and rerun.** Find where the users are lost (by device, browser or day), fix the logging or assignment, and start again.

What this shows. A 1.2% imbalance sounds small, but with 100,000 users it is almost impossible by chance. Check the split first; it is the cheapest bug detector in experimentation.

YOUR TURN (COVER THE SOLUTION FIRST)

1. A new home page shows lifts in click-through of +8% in week 1, +4% in week 2, +1.5% in week 3 and +1% in week 4. What is happening, and what would you report?

Solution. A novelty effect: users click more because the page is new, and it wears off. The stable effect is roughly the later weeks, about +1%. Report the week-by-week trend, base the decision on the settled effect with its confidence interval, and, if the effect matters long term, keep a small holdout group to measure it after launch. A three-day test would have reported +8%.

THE FUNDAMENTALS

Pitfall	Explanation / Fix
Peeking	Checking p-values repeatedly and stopping when significant inflates false positives. Fix: fixed horizon, or sequential testing (alpha spending, mSPRT).
SRM	Unequal split vs design signals logging/assignment bugs. Invalidate the test.
Novelty / primacy effect	Users react to change itself; effect decays/grows. Run longer, look at new vs returning users.
Simpson's paradox	Aggregate direction differs from segment direction due to unbalanced mix.
Multiple comparisons	Many metrics/segments cause false positives. Pre-register, correct (BH), or label exploratory.
Network effects / interference	In marketplaces/social, treatment leaks to control. Use cluster/geo randomization, switchback.
Wrong unit of analysis	Randomize by user but analyze by session inflates significance (pseudo-replication). Use delta method or user-level aggregation.
Seasonality / day-of-week	Run full weeks.
Outliers / heavy tails	Cap or winsorize revenue, use robust metrics or bootstrap.
Underpowered test	Non-significant does not mean no effect.
Selection bias in ramp-up	Keep randomization consistent; do not let users switch variants.

SAY IT IN 60 SECONDS

"Most A/B tests fail through process, not maths. Peeking: checking repeatedly and stopping at significance. Multiple metrics or segments: some will look significant by chance. Novelty and primacy effects: users react to newness, so early lifts fade. Sample ratio mismatch: the split is not what was planned. Interference: users affect each other, as in marketplaces or social networks, so control is contaminated. Wrong unit: randomising by session but analysing by user. Underpowered tests: a non-significant result does not prove no effect. The catch: for each pitfall I know the fix, such as fixed horizons, corrections, longer runs, cluster randomisation or holdouts."

KEY TERMS peeking · novelty effect · network effects · interference · Simpson's paradox · pseudo-replication · winsorizing · multiple comparisons

BUILDS ON 6.1 The experiment lifecycle

HANDS-ON PRACTICE For each pitfall in the table, write how you would detect it and how you would fix it.

COMMON MISTAKES Stopping a test the moment it is significant · Analysing sessions as if they were independent users · Declaring "no effect" from an underpowered test

CHECK YOURSELF

1. A new design lifts clicks 8% in week one and 1% by week four. What explains it?

A novelty effect: curiosity inflates early engagement; run long enough to see the settled effect.

2. Why can a ride-sharing price test with randomised riders be biased?

Riders compete for the same drivers, so treatment changes supply for control (interference).
Randomise by city or time block instead.

3. Test shows $p = 0.2$ with low power. Conclude no effect?

No: absence of evidence is not evidence of absence; report the confidence interval and the power or detectable effect.

6.4 Advanced experimentation ideas

IN PLAIN WORDS

These are the power tools of experimentation. CUPED is like handicapping in golf: if you know how each person did last week, you compare improvements rather than raw scores, which makes real differences easier to spot. Bayesian testing answers "what is the chance B is better?" directly. Bandits shift traffic toward the winner while learning, good for short-lived choices like banners. And when you cannot randomise at all (a price change in one city) you use clever comparisons like difference-in-differences.

Why it matters: Senior interviewers often ask "how would you do this better or with less data?"

Takeaway: Know the name, the idea and the trade-off of each method.

WORKED EXAMPLE, STEP BY STEP

Difference-in-differences when you cannot randomise

A food-delivery company launches express delivery in City A only. Orders per day:

	Before	After	Change
City A (launched)	100	120	+20
City B (not launched)	90	100	+10

- The naive estimate** compares A before and after: +20 orders. But things change over time for everyone: season, marketing, salaries.
- Use B to measure that background change.** B rose by 10 with no launch, so +10 is the "would have happened anyway" trend.
- Difference-in-differences:** $(120 - 100) - (100 - 90) = 20 - 10 = \mathbf{+10 \text{ orders a day}}$ attributed to express delivery.
- The key assumption: parallel trends.** Without the launch, A would have changed by the same amount as B. Check it by plotting both cities for several weeks *before* the launch; the lines should move together.

- 5. What breaks it.** Something else happened only in A (a festival, or a competitor leaving), or B was affected indirectly (customers ordering across city lines).

What this shows. When you cannot randomise, find a comparison group, subtract its change, and state the assumption that makes the subtraction fair.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Your metric has variance 100, and the same metric measured in the month before the experiment correlates 0.8 with it. By how much can CUPED reduce the variance, and what does that mean for sample size?

Solution. CUPED removes the part of the variance explained by the pre-period metric: the new variance $\approx 100 \times (1 - 0.8^2) = 100 \times 0.36 = 36$. Sample size is proportional to variance, so you need about 36% as many users for the same minimum detectable effect, nearly three times fewer. Equivalently, with the same users you can detect an effect about 0.6 times as large ($\sqrt{0.36} = 0.6$).

THE FUNDAMENTALS

- **CUPED:** use pre-experiment covariates to reduce variance.
- **Stratified / blocked randomization:** ensure balance.
- **Sequential testing, always-valid p-values.**
- **Bayesian A/B testing:** report $P(B > A)$ and expected loss.
- **Multi-armed bandits:** trade off exploration vs exploitation; good for short-lived decisions (banners), less for clean inference.
- **Diff-in-diff, synthetic control, propensity score matching, instrumental variables:** quasi-experimental methods when randomization is impossible (e.g., city-level policy, price change).
- **Uplift modeling:** predicts incremental effect of a treatment per user.

SAY IT IN 60 SECONDS

"Beyond the basics there are a few techniques worth naming. CUPED uses a pre-experiment covariate, like last month's spend, to reduce variance, so I can detect smaller effects with the same traffic. Sequential testing allows valid peeking by adjusting the thresholds. Bayesian A/B testing reports the probability that B beats A and the expected loss. Multi-armed bandits shift traffic towards winners to cut regret, good for short-lived choices but weaker for clean inference. Network effects need cluster or switchback designs. When randomising is impossible, I use difference-in-differences or synthetic control. The catch: each method buys something and costs something, and I should say which."

KEY TERMS CUPED · stratification · sequential testing · Bayesian A/B testing · multi-armed bandit · difference-in-differences · propensity score · uplift modelling

BUILDS ON 6.3 A/B testing pitfalls

HANDS-ON PRACTICE Simulate CUPED on synthetic data and report the variance reduction you get for two correlation strengths.

COMMON MISTAKES Using CUPED with post-treatment covariates · Treating $P(B > A)$ as a p-value · Using bandits when you need clean inference

CHECK YOURSELF**1. How does CUPED reduce variance?**

It subtracts the part of the metric predictable from pre-experiment behaviour, leaving less noise, so the same effect is detectable with fewer users.

2. When is a bandit better than a fixed A/B test?

When opportunity cost of a bad arm is high and the decision is short-lived, such as headline testing; for a clean causal estimate on a long-lived feature, use A/B.

3. You cannot randomise a policy change by user. What can you do?

A quasi-experiment: difference-in-differences with a comparable untreated group, synthetic control, or a regional rollout with matched controls.

6.5 Practice case questions and self-check**IN PLAIN WORDS**

Case questions test whether you can think in a clear order under pressure. Whatever the scenario, walk through the same steps: restate the goal, choose the main measure and the "do not hurt" measures, decide who or what gets randomised, say how many people you need and for how long, name the risks, describe the analysis and state the rule for deciding. Practise until this order is automatic.

Why it matters: This is the core of the product-company case round.

Takeaway: Structure beats cleverness: restate, metrics, unit, size, risks, analysis, decision.

WORKED EXAMPLE, STEP BY STEP**A complete case answer, in order**

Prompt: *"We built a new ranking for restaurants on the home screen. How would you test it?"*

- 1. Clarify the goal.** "Is the aim more orders, better matches (fewer cancellations or bad ratings), or a fairer spread of orders across restaurants?" Suppose: more completed orders without hurting satisfaction.
- 2. Metrics.** Primary: completed orders per user over 14 days. Guardrails: cancellation rate, delivery time (popular restaurants may get overloaded), rating of delivered orders, and the share of orders going to new restaurants.
- 3. Unit and design.** Randomise by user. Also note a marketplace risk: if the new ranking sends everyone to the same few kitchens, wait times rise for control users too. Watch restaurant-side load, and consider testing by city if that effect is large.
- 4. Size and duration.** From the baseline order rate and the smallest worthwhile lift, compute n ; run at least two full weeks.
- 5. Analysis plan.** Lift with a confidence interval, guardrail checks, a sample ratio mismatch check, and segments (new versus returning users) treated as exploratory.
- 6. Decision rule.** Ship if orders rise and the guardrails hold; if orders rise but delivery time worsens, try capping exposure for busy restaurants.

What this shows. Goal, metrics, design, size, analysis, decision: the same six steps answer almost every experiment case. Naming the marketplace risk is what separates a strong answer.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Results come back: orders per user +1.8% (95% CI +0.6% to +3.0%), but average delivery time rose from 31 to 34 minutes and cancellations rose slightly. What do you recommend?

Solution. The order lift is real (the interval is above zero), but a guardrail is clearly worse: a 3-minute increase in delivery time is likely to hurt retention later, and cancellations confirm the strain. Do not ship as is. Investigate whether the ranking concentrates orders on busy kitchens, add a load or wait-time penalty to the ranking, and re-test. Present the trade-off in numbers so the product team can decide.

THE FUNDAMENTALS

6.5 Practice case questions

Answer template: restate goal → define metrics (primary, guardrail) → unit of randomization → sample size/duration → risks (peeking, novelty, SRM) → analysis plan → decision rule.

6.6 Self-check

- ☐ Compute a required sample size by hand.
- ☐ List five reasons an experiment result may be untrustworthy.
- ☐ Explain the peeking problem and one remedy.

SAY IT IN 60 SECONDS

"For an experiment case I follow a script. Clarify the goal and the decision. Choose the primary metric and guardrails. Define the unit, the groups and any exclusions. Estimate sample size and duration. Name the risks: novelty, interference, seasonality. Say how I will analyse it, and what I will do for each outcome: ship, iterate or stop. For example, for a new checkout button, the primary metric is conversion to purchase, guardrails are refund rate and revenue per user, I randomise by user, run two full weeks, and ship only if the lift beats the minimum worth it. The catch: I always tie the answer back to a business decision."

KEY TERMS case answers · metrics trade-off · segments · marketplace pricing · heavy tails · SRM

BUILDS ON 6.3 A/B testing pitfalls

HANDS-ON PRACTICE Answer each question aloud using the template (goal, metrics, unit, sizing, risks, analysis, decision), then reveal the model answer.

COMMON MISTAKES Giving an answer without a structure · Ignoring guardrails and segments · Skipping the decision rule

CHECK YOURSELF**1. Test a recommendation carousel on the homepage. Primary metric and guardrails?**

Primary: click-through to purchase or revenue per user. Guardrails: page latency, bounce rate, diversity of items, returns.

2. Result: conversion +0.3% with a wide interval including 0, cost to maintain is high.**Decision?**

Do not ship on this evidence; the effect is uncertain and may be below the minimum worth the cost. Extend the test or iterate.

3. What is the first question to ask in any experiment case?

What decision will this result change, and what size of effect would justify it?

Interview tips, tricks and hacks

- 1. Use one skeleton for every experiment question.** Goal and decision, primary metric and guardrails, unit and population, sample size and duration, risks, analysis plan, decision rule. Say the headings, then fill them in.
- 2. Ask "what decision does this inform?" first.** It is the best opening question in any experimentation case and often reshapes the whole answer.
- 3. Always name a guardrail.** For any change that raises engagement, ask what it might damage: speed, returns, complaints, long-term retention.
- 4. Do the sample-size arithmetic aloud.** "Baseline 10 per cent, variance 0.09, detecting one point: 16 times 0.09 over 0.0001, about 14,400 per group, so two weeks at our traffic." Interviewers rarely expect more precision than this.
- 5. Say "whole weeks".** It shows you know that behaviour varies by weekday and that novelty fades.
- 6. Check for sample ratio mismatch before anything else.** Mentioning SRM unprompted marks you as someone who has run real experiments.
- 7. Report interval and business meaning together.** "Between +0.4 and +2.0 points, and we need +0.5 to cover the cost, so it is likely worth shipping."
- 8. Have an answer for "we cannot randomise".** Staged rollout with a holdout, difference-in-differences against a comparable group, or a before-and-after comparison with its weaknesses stated.
- 9. For marketplaces and social products, raise interference yourself.** Then propose randomising by city, cluster or time block.
- 10. Never call a non-significant result "no effect".** Say what size of effect the test could and could not have detected.

Frequently asked questions

11 questions interviewers ask on this chapter, with model answers. Cover the answer, say yours aloud, then compare.

THE EXPERIMENT LIFECYCLE

Q1 After two weeks the test shows a statistically significant +2% conversion, but the page load time guardrail got 300 ms slower. Do you ship?

Guardrails exist precisely for this situation. Check whether the slowdown exceeds the pre-agreed tolerance; estimate the downstream cost (latency usually hurts conversion, retention and SEO over time); look for a cause (heavy new component) and whether it can be fixed without losing the gain; and consider a follow-up test with the optimised build. If the tolerance is breached the default is **do not ship as is**; ship only with explicit sign-off that the gain outweighs the cost.

SAMPLE SIZE: THE BACK-OF-ENVELOPE METHOD

Q2 Your baseline conversion is 5% and you want to detect a 10% relative lift (5% to 5.5%). Estimate the sample size per group.

The absolute MDE is 0.5 percentage points ($\delta = 0.005$). $n \approx 16 \times p(1 - p) / \delta^2 = 16 \times 0.0475 / 0.000025 = 30,400$ per group (about 61,000 total). This is why low-conversion products need huge traffic to detect small relative lifts, and why teams focus on bigger changes or higher-funnel, higher-rate metrics.

A/B TESTING PITFALLS

Q3 The experiment has run 3 days, is significant at $p = 0.03$, and the PM wants to ship now. What do you say?

Explain: (1) **Peeking** inflates the false-positive rate; the test's error guarantees only hold at the pre-planned sample size. (2) **Three days** misses weekday/weekend behaviour and possible novelty effects. (3) Early estimates of effect size are inflated (the winner's curse). Offer a path: finish the planned duration (or use a pre-agreed sequential design that allows early stopping with valid error control), and monitor guardrails meanwhile. If the decision is cheap and reversible, agree an explicit risk threshold instead.

Q4 The test shows +2% on mobile and -2% on desktop, and the overall effect is flat. Interpret.

Opposite effects cancel in the aggregate. First check whether the segment results are **credible**: was the split pre-planned, are the sample sizes large enough for the confidence intervals to exclude zero, and does a mechanism explain it (a layout optimised for small screens)? If real, the decision is to **ship to mobile only** and fix or drop the desktop variant. If the segments were examined post hoc, treat it as a hypothesis and validate it with a follow-up experiment, because slicing creates spurious subgroup effects.

ADVANCED EXPERIMENTATION IDEAS

Q5 You cannot run an A/B test on a new pricing policy rolled out to Mumbai only. How do you estimate its effect?

Choose control cities that resemble Mumbai before the change (or build a **synthetic control** from several), then compare Mumbai's change in the metric with the control's change (**difference-in-differences**). Validate by confirming the pre-period trends were parallel, check for other simultaneous changes (promotions, weather, festivals), and run a placebo test on pre-period dates or on unaffected metrics. State the limits: it relies on untestable assumptions and gives a weaker causal claim than randomisation.

PRACTICE CASE QUESTIONS AND SELF-CHECK

Q6 You ran an A/B test for a new recommendation widget; conversion is +0.3% ($p = 0.04$) but average order value dropped. What do you do?

Restate: conversion went up (+0.3% absolute, $p = 0.04$) but average order value (AOV) fell, so the two metrics pull in opposite directions.

What I would do: (1) Evaluate the metric that combines them, **revenue per visitor** (conversion $\times$ AOV), with a confidence interval; a lift in conversion with a fall in AOV may net positive or negative. (2) Check **why** AOV fell: the widget may be recommending cheaper items, pulling users toward low-value purchases (a mix shift), or converting marginal users who buy small baskets (which is fine). Segment by new vs returning users and by category. (3) Check guardrails such as returns and margin, not just revenue. (4) Consider the evidence strength: $p = 0.04$ on a +0.3% effect is fragile; check power and the confidence interval, and whether the conversion gain survives a longer run.

Decision: If revenue per visitor (or contribution margin per visitor) is significantly positive and guardrails hold, ship; if it is flat or negative, iterate on the widget (for example recommend complementary higher-value items) and retest.

Numbers. If conversion rises from 10.0% to 10.03% (+0.3% relative) while order value falls 3%, revenue per visitor falls about 2.7%. Statistical significance on conversion alone hides the revenue loss.

Q7 Test shows +2% on mobile and -2% on desktop, flat overall. Interpret.

Interpretation: opposite effects in the two segments cancel to a flat overall result, so the overall average hides two real but opposite stories, if they are real.

Check credibility: Was the platform split pre-planned? Are the sample sizes enough for each segment's confidence interval to exclude zero? Do the pre-period metrics look balanced? Is there a mechanism (for example a layout that helps small screens and breaks large ones)? Apply a multiple-comparison correction if you looked at many segments.

Action: If the heterogeneity is credible, **ship to mobile only** and fix or roll back desktop; if it was found by slicing post hoc, treat it as a hypothesis and confirm with a follow-up test specifically on that split. Also check the mix of mobile vs desktop users between arms (Simpson-type effects).

Q8 How do you test a change in delivery-fee pricing on a two-sided marketplace?

Randomising riders or customers individually is risky because the two groups **share supply and demand**, so treatment leaks into control (interference): a cheaper fee in treatment raises demand that competes with control users for the same riders.

Design: randomise at a **coarser unit** where spillover is contained: matched **cities or zones** (geo experiment), or a **switchback** design (alternate fee policies by time block, such as alternating days or hours within a city, and compare blocks). Choose matched pairs on historical demand, weather and festival calendars.

Metrics: primary: contribution margin per order or per customer (not only conversion), plus order volume, delivery time, rider utilisation, cancellations and customer retention as guardrails. **Analysis:** difference-in-differences or a paired comparison of zones; cluster-robust standard errors. **Risks:** small number of clusters (low power), carryover between time blocks, events in only some zones. **Duration:** long enough for behaviour to settle and to cover weekly cycles.

Q9 The test has run for 3 days and is significant. Should we ship?

No, not yet. Three days cannot cover a full weekly cycle, and **peeking** (stopping when the result is significant) inflates false positives: the test's error guarantees hold only at the planned sample size. Early effect sizes are also inflated (the winner's curse), and novelty effects may still be working.

What I would say to the PM: finish the pre-planned duration (at least one to two weeks), or use a pre-agreed **sequential testing** plan that permits early stopping with valid error control. Meanwhile monitor SRM and guardrails. If the change is cheap and reversible and waiting is costly, agree an explicit risk tolerance and a rollback plan.

Why not. Day-of-week effects, novelty and repeated peeking all inflate false positives. Decide the sample size and duration in advance and analyse once, or use a sequential method designed for early looks.

Q10 Only 5,000 users available per week; metric is revenue per user with heavy tails. Plan the test.

Constraint: only 5,000 eligible users per week and a **heavy-tailed** metric (revenue per user) means low power and outlier-driven noise.

Plan: (1) Consider a **more sensitive proxy** as the primary metric (conversion, orders per user, or capped revenue) and keep revenue as secondary. (2) **Cap (winsorise)** revenue at a high percentile (for example P99) or use a log transform; or use a **bootstrap** for inference. (3) Use **CUPED** with pre-experiment revenue to cut variance substantially. (4) **Compute the required sample:** with high variance the MDE achievable in a few weeks may be large, so be explicit about the smallest effect we can detect in N weeks and decide whether that is acceptable. (5) Randomise by user, run a fixed horizon (for example 4 to 6 weeks), analyse per user, and monitor guardrails. If even that is underpowered, test a bolder change, or use a Bayesian/expected-loss approach for a reversible decision.

Q11 Your experiment has SRM (51.5/48.5 at n = 500k). What next?

51.5% / 48.5% at n = 500,000 is a textbook SRM.

The expected deviation under chance is about 0.07 percentage points; this is 1.5 points off (about 21 standard deviations), a chi-square p-value around $1e-100$.

Action: treat the experiment as **invalid**; do not analyse or ship from it. Investigate causes: assignment bugs or caching, bot or internal-traffic filtering applied unevenly, a redirect or crash in one variant that loses users before logging, triggered-vs-assigned counting differences, different loading times causing drop-off before the assignment event. Fix the cause, verify with an **A/A test**, and rerun. Report the incident and add an automatic SRM alert to the experimentation platform.

What an SRM tells you. With $n = 500k$, a 51.5/48.5 split is about 4 standard errors from 50/50, essentially impossible by chance. Common causes: bot filtering applied unequally, a redirect that loses users in one arm, or a logging bug.

EXPERIMENT LIFECYCLE

1. **Hypothesis and decision:** what will we do differently if it wins?
 2. **Metrics:** one primary (the decision metric), a few secondary, **guardrails** that must not worsen (latency, refunds, unsubscribes).
 3. **Unit of randomisation:** usually the user; analyse at the same unit. Exclusions defined up front.
 4. **Sample size, duration** from baseline, minimum detectable effect (MDE), α and power.
 5. **Validate setup:** A/A test, sample ratio mismatch (SRM), balance on pre-period metrics.
 6. **Run** the planned duration, whole weeks, no peeking decisions.
 7. **Analyse** with the pre-registered test; check guardrails and segments (as hypotheses).
 8. **Decide, document, monitor** after launch (holdout, long-term effect).
- Write the decision rule beforehand: ship if lift $\geq$ MDE and guardrails flat; iterate if direction good but inconclusive; stop otherwise.

METRIC TYPES

Type	Example	Note
Primary / north star	conversion, revenue per user	one, sensitive, tied to decision
Driver / diagnostic	click-through, add-to-cart	explain the primary
Guardrail	latency, crash rate, refund rate, retention	must not degrade
Ratio	CTR = clicks/impressions	unit mismatch: use per-user or delta method

- Good metrics: measurable, sensitive, hard to game, aligned with long-term value. Optimising one alone gets gamed (clicks up, satisfaction down).

- Per-user metrics with heavy tails (revenue): winsorise, log-transform, or use a bootstrap.

SAMPLE SIZE (PER GROUP)

- Two proportions: $n = (z\alpha/2 \cdot \sqrt{p(1-p)} + z\beta \cdot \sqrt{p_1(1-p_1) + p_2(1-p_2)})^2 / \delta^2$, with p the pooled rate and $q = 1 - p$. Rule of thumb: $n \sim 16 \cdot p(1-p) / \delta^2$ ($\alpha = 0.05$, power 80%).
- Means: $n \sim 16\sigma^2 / \delta^2$ where δ is the absolute difference.
- z values: $\alpha = 0.05$ two-sided 1.96; power 80% 0.84; power 90% 1.28.

Baseline	+5% rel	+10% rel	+20% rel
2%	315k	81k	21k
5%	122k	31k	8.2k
10%	58k	15k	3.8k
20%	26k	6.5k	1.7k
50%	6.3k	1.6k	390

- Halving the MDE needs 4x the users. Duration = $(2n) / (\text{daily eligible users})$, rounded up to full weeks.
- Variance too high? Use CUPED, a more sensitive metric, a larger MDE, or longer duration.

RANDOMISATION AND EXPOSURE

- Assign by hashing the user id with an experiment salt: stable across visits, independent across experiments. Never assign by session if the metric is per user.
- **Ramp** 1% $\rightarrow$ 10% $\rightarrow$ 50% to catch bugs; the analysis uses the full-allocation period only.
- Log **exposure** (when a user actually sees the change) and analyse the triggered population to cut noise; keep the intent-to-treat view too.
- **Layers / mutually exclusive** experiments avoid interaction between concurrent tests. Keep a **global holdout** for long-term effects.
- Balance check: pre-experiment metrics and device mix should be equal across arms.

- Hypotheses: H0 no difference; H1 two-sided unless a one-sided direction is justified in advance.

ANALYSIS STEPS

1. Check SRM first: chi-square of observed vs expected split. 257.5k/242.5k out of 500k, expected 50/50: chi-square = 450, p tiny: a bug, do not trust the results.
2. Compute lift with a **confidence interval**, not just p.
3. Two proportions: $z = (p_2 - p_1) / \sqrt{p(1-p)(1/n_1 + 1/n_2)}$, p pooled. Example: 10% vs 11%, 14,400 each: $z = 2.77$, $p = 0.006$.
4. Means: Welch t-test; heavy tails: bootstrap CI.
5. Guardrails and secondary metrics: expect noise; correct for many metrics.
6. Segments (platform, new vs returning) only as hypotheses, or pre-planned.
7. Practical significance: is the CI above the minimum worth shipping?

```
import statsmodels.stats.proportion as sp
z, p = sp.proportions_ztest([c2, c1], [n2, n1])
es = sp.proportion_effectsize(0.11, 0.10)
from statsmodels.stats import power
power.NormalIndPower().solve_power(es, power=0.8, alpha=0.05)
```

VARIANCE REDUCTION AND RATIO METRICS

- **CUPED:** $Y_{\text{adj}} = Y - \theta(X - \text{mean } X)$, $\theta = \text{cov}(X, Y) / \text{var}(X)$, X = pre-experiment metric. Variance shrinks by $(1 - r^2)$: $r = 0.7$ halves it.
- **Stratification / blocking** by platform or country; **paired designs** where possible.
- **Ratio metrics** (CTR, revenue per session): analyse per user or use the delta method; the unit of analysis must match the unit of randomisation.
- Winsorise at the 99th percentile for revenue; report both.

PITFALLS AND FIXES

Pitfall	Fix
Peeking and stopping at first $p < 0.05$	fixed horizon or sequential testing
Multiple metrics / segments	one primary, correct (BH), hypotheses
SRM (split off)	find the bug (bots, redirects, logging) and rerun
Novelty / primacy effect	run 2+ weeks, look at later cohorts
Underpowered test	compute n ; report CI, not "no effect"
Interference (marketplaces, social)	cluster, geo or switchback designs
Wrong unit (session vs user)	analyse at the randomisation unit
Seasonality, launches	full weeks, avoid holidays, run concurrently
Survivorship (analyse only active users)	intent-to-treat analysis
Simpson's paradox	stratify, check allocation by segment

- Weekday effects: run whole weeks. Carry-over: do not reuse exposed users.
- A/A tests should reject ~5% of the time at $\alpha = 0.05$.

ADVANCED DESIGNS

- **Sequential testing** (alpha spending, mSPRT): valid early looks. **Bayesian A/B**: $P(B > A)$ and expected loss; choose a prior; still need a stopping plan.
- **Multi-armed bandits** (epsilon-greedy, Thompson sampling): shift traffic to winners, less regret, weaker inference. Good for short-lived choices (headlines).
- **Cluster randomisation**: randomise groups (cities, classrooms); effective sample size is smaller (design effect = $1 + (m - 1) \cdot ICC$).
- **Switchback**: alternate treatment by time block in one market (pricing, dispatch).
- **Quasi-experiments** when you cannot randomise: **difference-in-differences** (treated change – control change, needs parallel trends), **synthetic control**, regression discontinuity, matching / propensity scores, instrumental variables.

- **Interleaving** for ranking and search; **holdouts** for long-term effects; **multivariate** tests need much more traffic.
- **Uplift modelling**: who changes behaviour because of treatment, not who is likely to convert anyway.
- **Network effects**: randomise by graph cluster; expect bias if you do not.

DECISION MATRIX

Result	Action
CI above MDE, guardrails flat	ship (maybe ramp 10 → 50 → 100%)
CI includes 0, adequate power	no detectable effect: stop or iterate
CI includes 0, underpowered	extend or redesign, do not call it flat
Primary up, guardrail down	trade-off: quantify, product call
Segment-only win	follow-up test, do not ship on it
SRM or tracking bug	invalid: fix and rerun

- Always say: effect size, interval, cost of being wrong, next step.

CASE-ANSWER SKELETON

1. Clarify the goal and the decision.
 2. Primary metric, guardrails, unit of randomisation, exclusions.
 3. Sample size, duration, ramp plan.
 4. Risks: novelty, interference, seasonality, instrumentation, SRM.
 5. Analysis plan: test, CI, segments, corrections.
 6. Decision rule for each outcome; follow-up and monitoring.
- **Cannot A/B test?** Staged rollout with holdout, difference-in-differences, synthetic control, or before-and-after with caveats.
 - **Two-sided marketplace pricing**: randomise by market or time; track both sides (rider wait, driver earnings, cancellations).
 - **Only 5,000 users a week, heavy-tailed revenue**: CUPED, winsorise, a proxy metric, larger MDE, longer run.
 - **+0.3% conversion but AOV down**: judge revenue per user with its CI.

PRE-LAUNCH AND REVIEW CHECKLIST

- Hypothesis, decision rule and primary metric written down. Sample size, duration, MDE agreed.

- Instrumentation tested: events fire once, in both arms, with the experiment id.
- Eligibility and exclusions (bots, employees, repeat users) defined; exposure logged.
- Guardrails and alerts (auto-stop on crashes or revenue collapse).
- After: SRM, novelty check (effect by week), CI, segments flagged as exploratory, write-up with the decision and learning.
- Tell stakeholders: "We estimate +X% (95% CI a to b). Ship because ... Risk ...".

WHERE TESTS GO WRONG IN PRACTICE

- Tracking changes in one arm only; caching, redirects, or app versions that never see the variant.
- Treatment that changes who is counted in the metric (more users reach checkout, conversion falls): use a denominator that is unaffected.
- Mixed-up control (existing users) vs treatment (new users): not a randomised comparison.
- Effects that decay (promotions), or build slowly (habit features): use cohorts and a holdout.
- Testing many small variants at once: low power and multiple-testing risk.

QUICK NUMBERS

- 95% CI half-width for a proportion $\sim 2 \cdot \sqrt{p(1-p)/n}$: $n = 10,000$, $p = 10\%$: $\pm 0.6pp$.
- Detect 1pp on a 10% base needs ~14,400 per group; 0.5pp needs ~57,800.
- 20 metrics at $\alpha = 0.05$: 64% chance of at least one false positive.
- Dates: avoid holidays, launches; at least one full business cycle.

SAY IT IN 20 SECONDS

- "Randomise at the user, size for the effect that matters, run full weeks, no peeking."
- "A significant lift with a falling guardrail is not a win."
- "SRM means stop and debug; do not interpret."
- "No significance with low power is not 'no effect'."

7

CHAPTER 7 · HIGHEST PRIORITY · MODULE 7 IN THE APP

Machine Learning Fundamentals and Algorithms

The heart of the technical interview: how models learn from data, why they fail, and the intuition, assumptions and trade-offs of each core algorithm.

15 topics · about 6 h 45 min in the app · 30 questions answered · two-page cheat sheet at the end

- | | |
|--|--|
| 7.1 Core ML concepts | 7.9 Support vector machines |
| 7.2 Why L1 gives sparsity | 7.10 K-nearest neighbours |
| 7.3 Data leakage: classic examples | 7.11 Naive Bayes |
| 7.4 Linear regression | 7.12 Ensembling and the algorithm selection guide |
| 7.5 Logistic regression | 7.13 Optimisation: gradient descent and friends |
| 7.6 Decision trees | 7.14 Hyperparameter tuning and sklearn pipelines |
| 7.7 Random forest (bagging) | 7.15 ML Q&A and self-check |
| 7.8 Gradient boosting: GBM, XGBoost, LightGBM, CatBoost | |

Foundations

Machine learning sounds like a catalogue of algorithms, and candidates often study it that way, memorising a page per method. Interviewers are testing something simpler and deeper: do you understand what it means for a model to learn, and why it can go wrong? Four ideas carry the whole subject. The algorithms are variations on them.

Idea 1: learning is fitting a function to examples

In **supervised learning** you have examples with known answers and you want a rule that predicts the answer for new cases. If the answer is a number it is **regression**; if it is a category it is **classification**. The model has adjustable settings called **parameters**. A **loss function** measures how wrong its predictions are, and training means adjusting the parameters to make the loss small, most often by **gradient descent**: repeatedly nudging every parameter in the direction that reduces the loss.

Idea 2: the goal is generalisation, not memory

A model that scores perfectly on the data it was trained on has proved nothing. What matters is performance on data it has never seen. This is why we hold data back: a training set to fit, a validation set to make choices, and a test set touched once for an honest final number. **Cross-validation** rotates the validation role across several slices of the data for a steadier estimate.

Idea 3: the bias-variance trade-off

There are two opposite ways to fail.

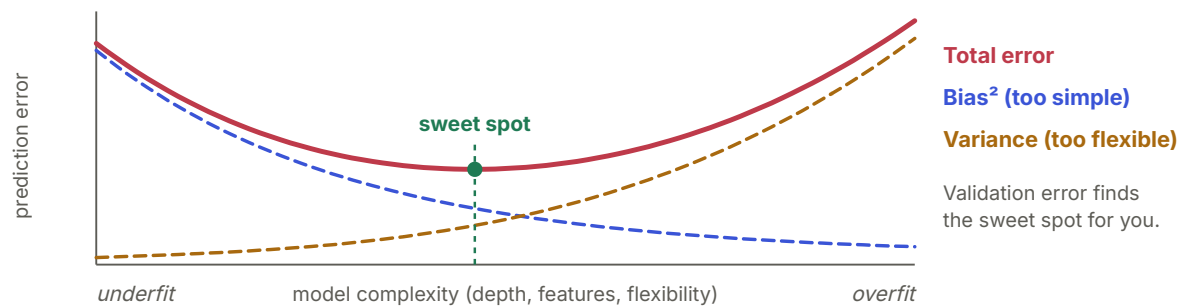


Figure 9. As a model becomes more flexible, error from bias falls and error from variance rises. Validation error locates the minimum.

A model that is too simple cannot capture the pattern; it is wrong in a consistent way. That is **bias**, or underfitting, and you see it as poor scores on both training and validation data. A model that is too flexible fits the noise in the particular sample it saw; it would change a lot with different data. That is **variance**, or overfitting, and you see it as an excellent training score with a much worse validation score. Nearly every practical question is a version of this diagnosis followed by its remedy. For high variance: more data, a simpler model, **regularisation** (a penalty on large parameters), or averaging many models. For high bias: better features or a richer model.

Idea 4: leakage, the silent killer

Data leakage means the model was given information during training that it will not have when making real predictions: a feature recorded after the outcome, statistics computed using the test set, or future rows in a time-ordered problem. Leaky models post spectacular scores and then fail in production. The defence is procedural: split the data first, fit every preprocessing step on the training portion only, and ask of each feature whether it would exist at the moment of prediction.

The algorithms as a family

With those ideas in place, the algorithms line up along the simple-to-flexible axis.

Linear and logistic regression predict with a weighted sum of the features. They are fast, stable and interpretable: each coefficient tells you the direction and size of a feature's effect. Logistic regression passes the sum through an S-shaped curve to produce a probability. They are the baseline every project should start with.

A **decision tree** asks a sequence of yes-or-no questions, choosing at each step the split that best separates the outcomes. Trees handle non-linear patterns and need no feature scaling, but a single deep tree overfits badly. The two great ensemble methods fix this in opposite ways. A **random forest** grows many deep trees on random samples of the data and averages them, which cancels their individual errors and lowers variance. **Gradient boosting** grows many shallow trees in sequence, each correcting the mistakes of the ones before, which lowers bias. Boosted trees are usually the strongest choice for tabular business data.

The remaining classics each embody one clear idea. **K-nearest neighbours** predicts from the most similar past cases. **Naive Bayes** multiplies simple probabilities and is a fine baseline for text. A **support vector machine** finds the boundary with the widest margin between classes.

How to choose

Start with a simple baseline. Move to gradient boosting if you need accuracy on tabular data. Prefer a linear model when you must explain decisions, as in lending. Use deep learning for images, text and audio. And always let validation, not preference, make the final choice.

Topic by topic

Each topic gives the idea in plain words, the fundamentals to know, a model spoken answer and three questions to test yourself. Open the same topic in the app for worked examples, runnable code and interactive demos.

7.1 Core ML concepts

IN PLAIN WORDS

A model is a student learning for an exam. A student who only memorises last year's answers (overfitting) does brilliantly on old questions and badly on new ones. A student who barely studied (underfitting) does badly on everything. The best student understands the ideas well enough to handle new questions. We keep a hidden exam (the test set) that nobody practises on, so we get an honest score. Regularisation is like a teacher telling the student "keep your answers simple", which stops memorising. Cross-validation is taking several mini-exams to get a steadier score.

Why it matters: Nearly every ML interview question is secretly about generalisation.

Takeaway: Judge models on data they have never seen; balance simplicity against flexibility.

WORKED EXAMPLE, STEP BY STEP

Diagnose three models from two numbers each

You trained three models to predict churn. Accuracy on training and validation data:

Model	Train	Validation	Gap
A: logistic regression, 3 features	68%	67%	1
B: depth-25 decision tree	99%	71%	28
C: gradient boosting, tuned	86%	83%	3

- 1. Model A: both scores low, small gap.** It is too simple to capture the pattern: **high bias (underfitting)**. Remedy: more informative features or a more flexible model.
- 2. Model B: near-perfect training, much worse validation.** It has memorised the training data, noise included: **high variance (overfitting)**. Remedy: limit the depth, require more samples per leaf, average many trees, or get more data.
- 3. Model C: high validation score, small gap.** The best of the three. A gap of 3 points is normal.
- 4. Choose by validation, never by training.** B's 99% means nothing for new customers.
- 5. Before reporting C,** score it once on the untouched **test set**, because tuning on validation data makes the validation score slightly optimistic.

What this shows. Two numbers, training and validation, diagnose most model problems. Low and low means bias; high and much lower means variance.

YOUR TURN (COVER THE SOLUTION FIRST)

1. A model scores 74% on training data and 73% on validation. Doubling the training data changes the scores to 74% and 74%. What is the diagnosis, and will more data help further?

Solution. Both scores are similar and modest, and adding data closed the small gap but did not raise the level: this is high bias, not variance. More data will not help much. Improve the model's capacity or its inputs instead: new features, interactions, or a more flexible model such as gradient boosting, then check whether a gap opens.

THE FUNDAMENTALS

Concept	What you need to say
Supervised / unsupervised / semi / self / RL	Labeled vs unlabeled vs mix vs labels derived from data vs reward-driven.
Classification vs regression	Discrete vs continuous target.
Bias-variance tradeoff	Error = bias ² + variance + irreducible noise. High bias = underfitting (too simple); high variance = overfitting (too flexible).
Overfitting	Low training error, high validation error. Remedies: more data, simpler model, regularization, dropout, early stopping, feature selection, cross-validation, ensembling.
Underfitting	High error on both. Remedies: richer model, more features, less regularization, train longer.
Regularization	Penalizes complexity. L1 (lasso) induces sparsity; L2 (ridge) shrinks weights; ElasticNet mixes both.
Train/val/test	Train fits parameters, validation tunes hyperparameters, test is touched once for final estimate.
Cross-validation	k-fold (k = 5 or 10), stratified for classification, TimeSeriesSplit for temporal data, GroupKFold when rows from the same entity must not leak across folds.
Data leakage	Information from outside the training context leaks into the model (see 7.2).
Parametric vs non-parametric	Fixed number of parameters (linear/logistic) vs complexity grows with data (KNN, trees).
Generative vs discriminative	Model P(x,y) (Naive Bayes) vs P(y
Curse of dimensionality	In high dimensions distances concentrate, data becomes sparse; hurts KNN/clustering.

SAY IT IN 60 SECONDS

"Machine learning learns a function from data. Supervised learning has labels: regression predicts a number, classification a category. Unsupervised learning finds structure without labels. The model's goal is to generalise to new data. Underfitting means too simple, with high bias and high error everywhere. Overfitting means memorising noise, with low training error and high test error. That is the bias-variance trade-off. Regularisation, more data, simpler models and cross-validation control it. We always split into train, validation and test so the final score is honest. The catch: a high training score alone means nothing, and I always compare it with a held-out score."

KEY TERMS bias-variance tradeoff · overfitting · underfitting · regularization · cross-validation · train validation test · curse of dimensionality · generative vs discriminative

BUILDS ON 5.1 Descriptive statistics

HANDS-ON PRACTICE Fit polynomials of degree 1, 3 and 9 to noisy data, compare train and test errors, and explain which is overfit.

COMMON MISTAKES Tuning on the test set · Random k-fold on time series or grouped data · Judging a model by training error

CHECK YOURSELF

1. Train accuracy 99%, validation 78%. Diagnosis and two fixes?

Overfitting (high variance). Add regularisation or reduce complexity, and get more data or use early stopping and cross-validation.

2. Both train and validation accuracy are 60%. What is wrong?

Underfitting (high bias): use a richer model or better features, reduce regularisation.

3. Why keep a test set separate from the validation set?

Tuning on validation overfits to it; the untouched test set gives an unbiased final estimate.

7.2 Why L1 gives sparsity

IN PLAIN WORDS

Imagine you have a budget for how big your weights can be. With the L2 rule the budget region is a round ball, so the best answer slides along the smooth edge and nothing is ever exactly zero. With the L1 rule the region is a diamond with sharp corners sitting on the axes; the best answer often lands right on a corner, where some weights are exactly zero.

Another way to feel it: L2 is a rubber band that pulls harder the further a weight is (so small weights barely feel it), while L1 pulls with a constant force (so small weights are pushed all the way to zero).

Why it matters: "Why does L1 give sparse solutions?" is one of the classic "reasoning" level questions.

Takeaway: L1 corners and constant push give exact zeros; L2 shrinks smoothly.

WORKED EXAMPLE, STEP BY STEP

L1 and L2 on a single weight, by numbers

Take one weight whose unpenalised best value is 0.3, and a penalty strength $\lambda = 1$.

- 1. The loss without a penalty** is $(w - 0.3)^2$, which is smallest at $w = 0.3$.
- 2. Add an L2 penalty:** $(w - 0.3)^2 + \lambda w^2$. Set the slope to zero: $2(w - 0.3) + 2\lambda w = 0$, so $w = 0.3 / (1 + \lambda) = 0.15$. The weight shrinks to half; it does not vanish.
- 3. Add an L1 penalty:** $(w - 0.3)^2 + \lambda|w|$. For $w > 0$ the slope is $2(w - 0.3) + \lambda = 0$, which gives $w = 0.3 - 0.5 = -0.2$. That is not positive, so the best point is the corner, **$w = 0$ exactly.**
- 4. Now a strong weight, unpenalised best 2.0.** L2 gives $2.0 / 2 = 1.0$. L1 gives $2.0 - 0.5 = 1.5$.

Unpenalised weight	L2 ($\lambda = 1$)	L1 ($\lambda = 1$)
0.3	0.15	0
2.0	1.0	1.5

5. Read the table. L1 subtracts a fixed amount ($\lambda/2$) and stops at zero, so small weights disappear and big ones survive almost intact. L2 divides by a fixed factor, so every weight shrinks and none reaches zero.

What this shows. L1 acts as a feature selector: weak features are switched off entirely. L2 is a gentle, proportional shrink. Scale the features first, or the penalty treats them unfairly.

YOUR TURN (COVER THE SOLUTION FIRST)

1. With $\lambda = 1$, what do L1 and L2 do to a weight whose unpenalised value is 0.8? And at what unpenalised value does L1 start setting weights to exactly zero?

Solution. L2: $0.8 / (1 + 1) = 0.4$. L1: $0.8 - 0.5 = 0.3$. L1 sets a weight to zero whenever its unpenalised value is at most $\lambda/2 = 0.5$ in size, so any weight between -0.5 and 0.5 is switched off. Raising λ widens that band and removes more features.

THE FUNDAMENTALS

The L1 constraint region is a diamond with corners on the axes; the loss contours are likely to touch at a corner, making some weights exactly zero. Equivalently, the L1 penalty's gradient has constant magnitude ($\text{sign}(w)$) pushing small weights to zero, while L2's gradient shrinks proportionally to w (never exactly zero).

SAY IT IN 60 SECONDS

"Regularisation adds a penalty on the size of the weights. L2, ridge, penalises squared weights and shrinks them smoothly towards zero but rarely to exactly zero. L1, lasso, penalises absolute values and drives some weights exactly to zero, giving feature selection. The geometry explains it: the L1 constraint region is a diamond with corners on the axes, so the loss contour tends to touch a corner, where some weights are zero. The L2 region is a circle with no corners. The gradient view agrees: L1's pull stays constant, so it can push a small weight all the way to zero, while L2's pull fades as the weight shrinks. The catch: with correlated features, lasso picks one arbitrarily."

KEY TERMS L1 regularization · L2 regularization · sparsity · lasso · ridge · constraint geometry · subgradient

BUILDS ON 7.1 Core ML concepts

HANDS-ON PRACTICE Fit lasso and ridge on data with 10 features and 3 true signals; count non-zero coefficients as the penalty grows.

COMMON MISTAKES Saying L1 is better without context · Forgetting to scale features before regularising · Explaining only one of the two arguments (geometry or gradient)

CHECK YOURSELF**1. You have 10,000 features, most irrelevant. Ridge or lasso, and why?**

Lasso (or elastic net): L1 sets many coefficients to exactly zero and selects features.

2. Why does L2 not give exactly zero weights?

Its gradient is proportional to the weight, so the pull vanishes as the weight nears zero.

3. What does elastic net combine and why?

L1 and L2 penalties: sparsity plus stability with groups of correlated features.

7.3 Data leakage: classic examples

IN PLAIN WORDS

Leakage is peeking at the answers before the exam. If any information that would not exist at prediction time sneaks into training, the model looks brilliant in testing and fails in the real world. The sneaky peeks include using next month's data to predict this month, letting the test set's averages influence your scaling, or using a column that only gets filled in after the outcome happens. A suspiciously perfect score is usually a clue.

Why it matters: Interviewers ask about leakage constantly because it is the most common reason a model works offline but not in production.

Takeaway: For every feature ask "would I have this at prediction time?" and always split before you fit anything.

WORKED EXAMPLE, STEP BY STEP**Spot the leaks in a feature list**

Goal: on 1 June, predict which subscribers will cancel during June. Candidate features:

Feature	When is it known?	Verdict
Logins in May	before 1 June	safe
Support tickets, last 90 days	before 1 June	safe
"Cancellation reason" field	only after cancelling	target leak
Days until renewal date	before 1 June	safe
Refund issued in June	after the prediction date	future leak
Average spend, computed over the <i>whole</i> year	includes June	future leak

1. Ask one question of every feature: "On 1 June, would I know this value?"

2. Target leak. The cancellation reason only exists for people who cancelled; the model learns "reason filled in means churn" and scores near 100% in testing, then fails in use.

3. Future leaks. A June refund and a full-year average contain information from the period you are predicting.

4. Fix them. Drop the reason field; compute every aggregate up to 31 May only.

5. Then check the process. Split the data before scaling or imputing, fit those steps on the training part, and split by time (train on earlier months, test on later ones).

6. The warning sign. If validation accuracy is suspiciously high, sort the features by importance and look at the top one first. It is often the leak.

What this shows. Leakage is a timing error. Draw the timeline, mark the prediction moment, and allow only information from before it.

YOUR TURN (COVER THE SOLUTION FIRST)

1. You forecast daily sales and evaluate with a random 80/20 split of days. The test error is excellent. Why is it misleading, and what should you do instead?

Solution. A random split puts later days in the training set and earlier days in the test set, so the model learns from the future (for example, the same week's pattern on neighbouring days) and the error is optimistic. Split by time: train on, say, January to September and test on October to December, or use several forward-chaining folds. Also make sure lag and rolling features use only past values.

THE FUNDAMENTALS

1. Scaling/imputing/encoding on the **entire dataset** before splitting (fit transformers only on train; use `Pipeline`).
2. Target encoding computed using the target of the same rows (use out-of-fold).
3. Features that contain future information (e.g., "total_refunds_after_purchase" predicting fraud; using next-month data to predict this month).
4. Random split for time series or repeated entities (same customer in train and test).
5. Duplicate rows across train/test.
6. Using a variable that is a proxy for the label (e.g., "call_duration" to predict "call_answered").

Signal: suspiciously high validation score (AUC 0.99) or one feature dominating importance.

SAY IT IN 60 SECONDS

"Data leakage means the model sees information in training that it will not have at prediction time, so scores look great and real performance collapses. Target leakage: a feature derived from the outcome, like 'refund issued' when predicting fraud. Train-test contamination: scaling or imputing on the whole dataset before splitting, or duplicates across the split. Temporal leakage: random splits of time-ordered data, so the model learns from the future. Group leakage: the same customer in both train and test. The fix is to split first, fit every preprocessing step inside a pipeline on training data only, split by time or group when needed, and be suspicious of near-perfect scores. The catch: ask of each feature, will this exist at prediction time?"

KEY TERMS data leakage · target leakage · train-test contamination · pipeline · out-of-fold target encoding · time-based split · group split

BUILDS ON 7.1 Core ML concepts

HANDS-ON PRACTICE Build a leaky and a leak-free pipeline on the same dataset and compare the scores.

COMMON MISTAKES Preprocessing before splitting · Target encoding without out-of-fold computation · Random splits for temporal or grouped data

CHECK YOURSELF**1. You standardise the whole dataset, then split. What is wrong?**

The mean and SD include test rows, leaking test information into training. Fit the scaler on train only, ideally in a Pipeline.

2. Predicting churn, a feature is 'days since account closed'. Problem?

Target leakage: it is only known after the churn happens.

3. Forecast sales with a random 80/20 split. Why bad?

Training rows come from after some test rows, so the model uses the future. Split chronologically.

7.4 Linear regression

IN PLAIN WORDS

Linear regression draws the best straight line (or flat plane) through your data so you can predict a number. "Best" means the total of the squared gaps between the line and the dots is as small as possible. Each slope tells you how much the prediction changes when that input goes up by one, keeping the others fixed. The line is easy to explain, which is why it remains the first model to try and a strong baseline.

Why it matters: Interviewers expect you to know its assumptions and diagnostics cold.

Takeaway: Minimise squared errors, check the residuals, beware correlated inputs.

WORKED EXAMPLE, STEP BY STEP**Read a fitted line: coefficients, residuals and R^2**

A model predicts monthly rent (thousands) from flat size (tens of square metres): **rent = 20 + 3 × size**. Four flats:

size	actual rent	predicted	residual (actual – predicted)
10	52	50	+2
20	78	80	-2
30	108	110	-2
40	142	140	+2

- The slope, 3:** each extra 10 square metres adds about 3 thousand to the rent.
- The intercept, 20:** the model's rent at size 0. It is not meaningful by itself (no flat has size 0); it positions the line.
- Residuals:** the vertical gaps between the points and the line. For a least-squares fit they add up to 0 (+2 – 2 – 2 + 2 = 0).
- Sum of squared residuals:** 4 + 4 + 4 + 4 = 16.
- R^2 .** The mean rent is 95. The total squared spread around that mean is $43^2 + 17^2 + 13^2 + 47^2 = 4,516$. $R^2 = 1 - 16 / 4,516 \approx 0.996$: the line explains 99.6% of the variation in rent.

6. Look at the residuals' pattern, not just R^2 . Here they alternate +, -, -, +, which is no pattern, so the line is a good fit. Residuals that curve, or that spread wider as size grows, would signal a missing curve or unequal variance.

What this shows. A coefficient is "change in the target per unit of the feature, holding the others fixed". Residuals are where you check whether to trust the line.

YOUR TURN (COVER THE SOLUTION FIRST)

1. A model of daily orders has a coefficient of +15 on `is_weekend` (1 on Saturday and Sunday, 0 otherwise) and +2 on `temperature_c`, with an intercept of 100. Predict orders for a Saturday at 30 °C and interpret the weekend coefficient.

Solution. Prediction = $100 + 15 \times 1 + 2 \times 30 = 175$ orders. Interpretation: holding temperature fixed, a weekend day has about 15 more orders than a weekday. It is an association learned from the data, and it is causal only if nothing else that differs between weekends and weekdays has been left out of the model.

THE FUNDAMENTALS

- **Model:** $y = Xw + b$; minimize squared error (OLS closed form: $w = (X^T X)^{-1} X^T y$).
- **Assumptions (LINE + no multicollinearity):** Linearity, Independence of errors, Normality of residuals (for inference), Equal variance (homoscedasticity), no perfect multicollinearity.
- **Diagnostics:** residual plots, QQ plot, Durbin-Watson, VIF (> 5-10 concerning), Breusch-Pagan.
- **Metrics:** $R^2 = 1 - SS_{res}/SS_{tot}$; adjusted R^2 penalizes extra features; RMSE/MAE.
- **Fixes:** log-transform target, polynomial terms, regularization (ridge for multicollinearity), remove correlated features.
- **Interpretation:** coefficient = change in y per unit change in x, holding others constant.
- **Pros:** interpretable, fast. **Cons:** linear only, outlier sensitive.

SAY IT IN 60 SECONDS

"Linear regression fits a line, or a plane, to predict a number as a weighted sum of features plus an intercept. It chooses the weights that minimise the sum of squared errors. Each coefficient is the expected change in the target for a one-unit change in that feature, holding the others fixed. The assumptions are a linear relationship, independent errors, constant error variance, and roughly normal errors for inference; multicollinearity makes coefficients unstable. R squared is the share of variance explained. Residual plots reveal violations. The catch: outliers can pull the line hard, and extrapolating beyond the range of the data is unsafe."

KEY TERMS ordinary least squares · assumptions · residuals · R-squared · adjusted R-squared · multicollinearity · VIF · homoscedasticity

BUILDS ON 7.1 Core ML concepts

HANDS-ON PRACTICE Fit a line by hand on five points, compute R-squared, then check against scikit-learn.

COMMON MISTAKES Ignoring residual plots · Interpreting coefficients causally · Comparing R-squared across models with different numbers of features

CHECK YOURSELF**1. Coefficient on 'square feet' is 150 in a price model. Interpretation?**

Holding other features fixed, each extra square foot is associated with a \$150 higher predicted price.

2. Residuals fan out as predictions grow. Which assumption fails and what can you do?

Constant variance (heteroscedasticity). Transform the target (for example log), use weighted least squares, or robust standard errors.

3. Two highly correlated features give wild, flipping coefficients. Why?

Multicollinearity: the model cannot separate their effects. Drop one, combine them, or use ridge.

7.5 Logistic regression

IN PLAIN WORDS

Logistic regression answers yes/no questions with a probability. It first works out a score from the inputs (just like linear regression), then squeezes that score through an S-shaped curve so the result is always between 0% and 100%. Big positive scores turn into "almost certainly yes", big negative ones into "almost certainly no". You then pick a cut-off: above it you say yes. The weights tell you how strongly each input pushes the odds of "yes" up or down.

Why it matters: It is the workhorse of credit risk, the baseline for classification everywhere, and a favourite interview topic.

Takeaway: A linear score, squeezed to a probability; coefficients act on the odds.

WORKED EXAMPLE, STEP BY STEP**From score to probability to decision**

A churn model: score $z = -3 + 0.8 \times (\text{support tickets in the last 90 days})$. Probability $p = 1 / (1 + e^{(-z)})$.

1. Compute for four customers.

Tickets	z	Probability of churn
0	-3.0	0.05
2	-1.4	0.20
4	0.2	0.55
6	1.8	0.86

2. The S-curve. The score is a straight line, but the probability bends: it moves slowly near 0 and 1 and fastest around $z = 0$, where $p = 0.5$.

3. Read the coefficient as odds. Each extra ticket multiplies the *odds* of churn by $e^{0.8} \approx 2.2$. It does not add a fixed amount to the probability; going from 2 to 4 tickets adds 35 points, going from 4 to 6 adds 31.

4. **Find the default cut-off.** $p = 0.5$ when $z = 0$, that is $0.8 \times \text{tickets} = 3$, so $\text{tickets} = 3.75$. With a 0.5 threshold, customers with 4 or more tickets are flagged.
5. **Choose the threshold from costs.** If a retention call is cheap and losing a customer is expensive, flag at $p = 0.15$, which means 2 or more tickets (2 tickets give 0.198; 1 ticket gives 0.10). The model stays the same; the decision changes.

What this shows. Logistic regression produces a linear score, turns it into a probability, and leaves the decision threshold to you.

YOUR TURN (COVER THE SOLUTION FIRST)

1. For a customer with $z = 0$, what is the churn probability? If this customer gets one more ticket, what are the new odds and the new probability?

Solution. At $z = 0$, $p = 1 / (1 + e^0) = 0.5$, so the odds are 1 (one to one). One more ticket multiplies the odds by $e^{0.8} \approx 2.23$, giving odds of 2.23, so $p = 2.23 / (1 + 2.23) \approx 0.69$. Equivalently, $z = 0.8$ and $p = 1 / (1 + e^{(-0.8)}) \approx 0.69$.

THE FUNDAMENTALS

- **Model:** $P(y=1|x) = \text{sigmoid}(w \cdot x + b)$; log-odds are linear in x .
- **Loss:** binary cross-entropy (log loss); trained via MLE/gradient descent (no closed form).
- **Coefficient meaning:** $\exp(w)$ = odds ratio per unit increase.
- **Threshold:** default 0.5, tune using business costs/PR curve.
- **Assumptions:** linearity of log-odds, independent observations, little multicollinearity.
- **Extensions:** multinomial (softmax), regularized ($C = 1/\text{lambda}$ in sklearn).
- **Why not linear regression for classification?** Unbounded outputs, not probabilities, sensitive to outliers, violates error assumptions.
- **Used heavily in credit risk (scorecards, WoE/IV) and as a strong baseline.**

SAY IT IN 60 SECONDS

"Logistic regression predicts a probability for a yes-or-no outcome. It computes a weighted sum like linear regression, then passes it through the sigmoid, which squeezes any number into zero to one. Equivalently it models the log-odds as linear. A coefficient is the change in log-odds per unit; exponentiate it to get an odds ratio. It is trained by maximising likelihood, using log loss, and the decision threshold of fifty per cent is a choice that I tune to the business cost. It gives a linear decision boundary. The catch: it is a classifier by name only in that the model outputs probabilities, and calibration and class imbalance affect them."

KEY TERMS sigmoid · log-odds · odds ratio · cross-entropy · decision threshold · regularization C · multinomial · scorecards

BUILDS ON 7.4 Linear regression

HANDS-ON PRACTICE Fit a logistic regression, convert each coefficient to an odds ratio, and explain one in plain language.

COMMON MISTAKES Interpreting coefficients as probability changes · Using 0.5 as the threshold by default · Forgetting to scale features when regularising

CHECK YOURSELF**1. Coefficient 0.69 on 'has_coupon'. Interpret.**

Odds ratio $e^{0.69} \approx 2$: having a coupon doubles the odds of the event, holding others fixed.

2. Why not use linear regression for a binary target?

It can predict outside 0–1, errors are not normal or constant, and squared error is a poor fit; logistic uses a bounded link and log loss.

3. The model outputs 0.3 for a fraud case. Is it 'not fraud'?

Only under a 0.5 threshold. For costly missed fraud, pick a lower threshold chosen from precision-recall and costs.

7.6 Decision trees

IN PLAIN WORDS

A decision tree plays "twenty questions" with your data. It asks the yes/no question that best separates the groups ("is income above 40,000?"), then asks the next best question inside each group, and keeps going until the groups are clean or it runs out of patience. The result is a flowchart that is easy to read and explain. The danger: if you let it ask too many questions it memorises every quirk of the training data.

Why it matters: Trees are the building block of random forests and gradient boosting, the best performers on tabular data.

Takeaway: Split to make groups purer; limit depth to avoid memorising.

WORKED EXAMPLE, STEP BY STEP**Choose between two splits with Gini**

10 customers: 5 churned (C) and 5 stayed (S). Gini impurity = $1 - \sum(\text{share}^2)$. The starting node: $1 - (0.5^2 + 0.5^2) = \mathbf{0.50}$.

1. Split A, by plan type. Monthly plan: 6 customers (5 C, 1 S). Annual plan: 4 customers (0 C, 4 S). $\text{Gini}(\text{monthly}) = 1 - ((5/6)^2 + (1/6)^2) = 1 - (25 + 1)/36 \approx 0.278$. $\text{Gini}(\text{annual}) = 1 - (0^2 + 1^2) = 0$, a pure node. Weighted: $(6/10) \times 0.278 + (4/10) \times 0 \approx \mathbf{0.167}$.

2. Split B, by age under 30. Under 30: 5 customers (3 C, 2 S). 30 and over: 5 (2 C, 3 S). Gini of each = $1 - (0.6^2 + 0.4^2) = 0.48$. Weighted: **0.48**.

3. Gain (drop in impurity). A: $0.50 - 0.167 = \mathbf{0.333}$. B: $0.50 - 0.48 = \mathbf{0.02}$.

4. The tree picks split A. Plan type separates churners from stayers far better than age.

5. Then it repeats inside the monthly-plan node (the annual node is already pure), until a stopping rule such as maximum depth or minimum samples per leaf.

What this shows. A tree is a sequence of greedy choices: at each node, take the split that leaves the children purest. Without a stopping rule it keeps splitting until it memorises the data.

YOUR TURN (COVER THE SOLUTION FIRST)

1. A node has 8 samples, 4 positive and 4 negative. A split produces a left child with 3 positive and 1 negative, and a right child with 1 positive and 3 negative. Compute the weighted Gini after the split and the gain.

Solution. Parent Gini = $1 - (0.5^2 + 0.5^2) = 0.5$. Each child has shares 0.75 and 0.25: Gini = $1 - (0.5625 + 0.0625) = 0.375$. Both children have 4 samples, so the weighted Gini = $0.5 \times 0.375 + 0.5 \times 0.375 = 0.375$. Gain = $0.5 - 0.375 = 0.125$. A useful split, though weaker than one producing a pure child.

THE FUNDAMENTALS

- Recursive splitting to maximize purity. **Gini** = $1 - \sum p_i^2$; **Entropy** = $-\sum p_i \log_2 p_i$; **information gain** = parent impurity - weighted child impurity. Regression trees minimize variance (MSE).
- **Hyperparameters:** `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_features`, `ccp_alpha` (pruning).
- **Pros:** interpretable, handles mixed types, no scaling needed, non-linear, captures interactions. **Cons:** high variance, overfits, unstable, step-wise predictions, cannot extrapolate.

SAY IT IN 60 SECONDS

"A decision tree asks a series of yes-or-no questions on features to split data into purer groups. At each node it picks the split that most reduces impurity, measured by Gini or entropy for classification and variance for regression. It stops at a depth limit or minimum samples, and the leaf predicts the majority class or mean. Trees are easy to read, need no scaling, and capture non-linear interactions. But a deep tree memorises the data, so it overfits, and a small change in data can change the tree. Controls are max depth, minimum leaf size and pruning. The catch: single trees are weak predictors, which is why we ensemble them."

KEY TERMS Gini impurity · entropy · information gain · pruning · `max_depth` · `min_samples_leaf` · variance of trees · regression trees

BUILDS ON 7.1 Core ML concepts

HANDS-ON PRACTICE Compute Gini and entropy for a 6-positive, 4-negative node by hand, then split it and compute the gain.

COMMON MISTAKES Letting trees grow unconstrained · Expecting trees to extrapolate · Thinking impurity-based importance is unbiased

CHECK YOURSELF

1. A node has 50 positive and 50 negative. Gini impurity?

$1 - (0.5^2 + 0.5^2) = 0.5$, the maximum for two classes.

2. Name two ways to stop a tree overfitting.

Limit max depth or require a minimum samples per leaf; or post-prune using a complexity penalty.

3. Do trees need feature scaling? Why or why not?

No: splits compare a feature to a threshold, so monotonic scaling does not change them.

7.7 Random forest (bagging)

IN PLAIN WORDS

One tree can be moody: change the data a little and it changes its mind. A random forest asks hundreds of trees and takes a vote. Each tree sees a slightly different random sample of the rows and only a random handful of the columns at each decision, so the trees make different mistakes, and mistakes that differ tend to cancel out in the average. It is like asking a large panel of slightly different experts instead of trusting one.

Why it matters: A robust, low-tuning default for tabular data, and the classic example of "reduce variance by averaging".

Takeaway: Many decorrelated, deep trees, averaged: lower variance without much extra bias.

WORKED EXAMPLE, STEP BY STEP

Why averaging trees helps

A house's true price is 305 (thousands). Three trees, each trained on a different bootstrap sample, predict 280, 320 and 330.

- 1. Each tree is noisy.** Their errors are -25, +15 and +25: each tree has latched onto quirks of its own sample.
- 2. Average them:** $(280 + 320 + 330) / 3 = \mathbf{310}$, an error of +5. The errors partly cancel because they point in different directions.
- 3. Why they differ.** Each tree sees a bootstrap sample (rows drawn with replacement), and each split considers only a random subset of features. Different data and different options give different mistakes.
- 4. The cancelling only works if the trees disagree.** If all three said 330, the average would be 330. Diversity is the point.
- 5. Out-of-bag rows.** A bootstrap sample of n rows misses each row with probability $(1 - 1/n)^n \approx 0.37$, so about 37% of rows were not seen by a given tree. Scoring each row with the trees that did not see it gives a free validation estimate, the OOB score.

What this shows. Averaging many independently wrong models lowers variance. A random forest makes its trees independent on purpose.

YOUR TURN (COVER THE SOLUTION FIRST)

- Five trees vote on whether a transaction is fraud: fraud, not fraud, fraud, fraud, not fraud. What does the forest predict, and what "probability" does it report? Why might that probability need calibrating?

Solution. Three of five vote fraud, so the forest predicts fraud, with a "probability" of $3/5 = 0.6$. The vote share measures agreement among the trees, not a calibrated probability: averaged votes tend to cluster away from 0 and 1. If the probability drives a decision, for example an expected-cost threshold, calibrate it on validation data with Platt scaling or isotonic regression.

- Train many deep trees on **bootstrap samples**, using **random feature subsets** at each split; average (regression) or vote (classification).
- Reduces **variance** without raising bias much; decorrelated trees give better averaging.
- **OOB error**: free validation using rows left out of each bootstrap.
- **Feature importance**: impurity-based (biased toward high-cardinality) vs permutation importance (preferred).
- **Hyperparameters**: `n_estimators`, `max_depth`, `max_features`, `min_samples_leaf`.
- **Pros**: robust, strong default, little tuning. **Cons**: large, slower inference, weaker at extrapolation, less interpretable.

SAY IT IN 60 SECONDS

"A random forest averages many decision trees. Each tree is trained on a bootstrap sample, a random draw with replacement of the rows, and at each split considers only a random subset of the features. That makes the trees different from each other, and averaging different, noisy-but-unbiased trees reduces variance without raising bias much. For classification the trees vote. Out-of-bag samples, the rows left out of each bootstrap, give a free validation score. Feature importance shows which features matter. It needs little tuning, handles non-linear patterns, and rarely overfits badly. The catch: it is less interpretable than one tree, slower to predict, and bad at extrapolating beyond the training range."

KEY TERMS bagging · bootstrap sample · random feature subsets · out-of-bag error · permutation importance · variance reduction

BUILDS ON 7.6 Decision trees

HANDS-ON PRACTICE Compare a single tree, a random forest and logistic regression on the same data with cross-validated AUC.

COMMON MISTAKES Thinking more trees can overfit · Using impurity importance to rank features with different cardinalities · Expecting random forests to extrapolate

CHECK YOURSELF**1. Why does averaging many trees help?**

Each tree overfits differently; averaging cancels their independent errors, cutting variance. Diversity from bootstrap and feature subsampling is essential.

2. What is out-of-bag error?

Each tree is scored on the ~37% of rows not in its bootstrap sample; the combined score is a built-in validation estimate.

3. Random forest trained on prices up to \$500 gets a \$900 item. What happens?

It predicts near the largest values it has seen; trees cannot extrapolate beyond the training range.

7.8 Gradient boosting: GBM, XGBoost, LightGBM, CatBoost

IN PLAIN WORDS

Gradient boosting is a team of small learners that fix each other's mistakes. The first learner makes a rough guess (say, the average). We look at where it was wrong (the errors), and train a second small learner specifically to predict those errors. Adding its corrections to the

guess makes the guess better. Then we look at the remaining errors and train a third learner, and so on. Each learner takes only a small step (the learning rate), so the team improves steadily instead of lurching.

Why it matters: XGBoost, LightGBM and CatBoost usually win on tabular data; interviewers expect you to explain the idea.

Takeaway: Add shallow trees one at a time, each fit to what the ensemble still gets wrong, with small steps.

WORKED EXAMPLE, STEP BY STEP

Learning rate versus number of trees, on paper

Suppose each new tree could perfectly fit the current error (residual), and we start with an error of 100. Each round removes a fraction η (the learning rate) of what is left.

- $\eta = 0.5$:** after each round the error is multiplied by 0.5: $100 \rightarrow 50 \rightarrow 25 \rightarrow 12.5$. Three rounds take it close to 10.
- $\eta = 0.1$:** each round keeps 90%: $100 \rightarrow 90 \rightarrow 81 \rightarrow 72.9$. It takes **22 rounds** to fall below 10.
- So why use small steps?** Real trees fit the residuals imperfectly, and part of what they fit is noise. Big steps lock in noise quickly; small steps, with many trees, let later trees correct earlier mistakes. Small learning rates usually generalise better.
- The cost** is more trees: more training time and slower predictions.
- The practical recipe:** set a small learning rate (0.05 to 0.1), a large maximum number of trees, and **early stopping**: stop adding trees when the validation score has not improved for, say, 50 rounds.

Learning rate	Error after 3 rounds	Rounds to get below 10
0.5	12.5	4
0.1	72.9	22

What this shows. Learning rate and the number of trees trade off: smaller steps need more trees and usually give a better model. Tune them together, with early stopping.

YOUR TURN (COVER THE SOLUTION FIRST)

- With the same idealised setup and $\eta = 0.3$, what error remains after 3 rounds, starting from 100? If validation loss is lowest at round 140 and rises after that, how many trees should the final model use?

Solution. Each round keeps 70%: $100 \times 0.7^3 = 34.3$. For the final model, keep the 140 trees where validation loss was lowest; this is exactly what early stopping does. Training to 500 trees would keep lowering training loss while the model overfits.

THE FUNDAMENTALS

- Boosting:** sequentially add shallow trees, each fitting the **residuals/gradient of the loss** of the current ensemble; reduces **bias** (and some variance via shrinkage).

- **Key hyperparameters:** `n_estimators`, `learning_rate` (trade-off with trees), `max_depth / num_leaves`, `subsample`, `colsample_bytree`, `min_child_weight`, `reg_lambda / reg_alpha`, early stopping.
- **XGBoost:** regularized objective, second-order gradients, handles missing values, column subsampling.
- **LightGBM:** histogram-based, leaf-wise growth (faster, may overfit small data), native categorical support.
- **CatBoost:** ordered boosting, strong native categorical handling, less tuning.
- **Bagging vs boosting:**

	Bagging (RF)	Boosting (GBM)
Training	Parallel, independent	Sequential, dependent
Primary effect	Reduces variance	Reduces bias
Base learners	Deep, high-variance trees	Shallow, weak trees
Overfitting risk	Low	Higher; needs tuning and early stopping
Outlier sensitivity	Lower	Higher

- **Tabular data reality:** GBMs (XGBoost/LightGBM) usually win; start here after a linear baseline.

SAY IT IN 60 SECONDS

"Gradient boosting builds trees one after another, each correcting the errors of the sum so far. The first tree makes a rough prediction, the next fits the residuals, meaning the gradient of the loss, and a small learning rate scales its contribution. Many shallow trees add up to a strong model. Random forest reduces variance by averaging independent deep trees; boosting reduces bias by sequentially fixing mistakes. XGBoost adds regularisation and speed, LightGBM grows leaf-wise and uses histograms for speed, CatBoost handles categories natively. The key knobs are learning rate, number of trees, depth, and subsampling. The catch: boosting can overfit noisy data, so I use early stopping on a validation set."

KEY TERMS boosting · residuals · gradient of the loss · learning rate · shrinkage · XGBoost · LightGBM · CatBoost · early stopping

BUILDS ON 7.7 Random forest (bagging)

HANDS-ON PRACTICE Hand-trace two boosting rounds on four data points with a learning rate of 0.5.

COMMON MISTAKES Not using early stopping · Tuning learning rate and number of trees independently · Using boosting without a validation set

CHECK YOURSELF**1. Bagging versus boosting in one line each?**

Bagging: train independent models in parallel and average to cut variance. Boosting: train sequentially, each fixing the previous errors, to cut bias.

2. Lower the learning rate: what else changes?

You need more trees; smaller steps usually generalise better but cost more compute.

3. Your boosted model's validation loss starts rising while training loss falls. Action?

Stop at the best iteration (early stopping), or reduce depth/learning rate and add regularisation or subsampling.

7.9 Support vector machines

IN PLAIN WORDS

Picture two groups of dots on paper. Many lines can separate them, but the best line is the one with the widest empty street between the groups. That widest-street line is what an SVM finds, and the dots touching the edges of the street are the "support vectors" that hold it in place. If the groups overlap a bit, you allow a few dots to cross the street (soft margin) and pay a penalty. If a straight line is not enough, the "kernel trick" lets you bend the paper so a straight cut becomes a curved boundary.

Why it matters: A classic algorithm that shows up in interviews to test margin, regularisation and kernels.

Takeaway: Widest margin, few support vectors, C trades margin against errors, kernels make curves.

WORKED EXAMPLE, STEP BY STEP**The kernel trick on a number line**

One feature, x . Class A sits at $-1, 0$ and 1 . Class B sits at -3 and 3 .

- 1. Try a single threshold** ("B if $x > t$ "). Any t puts some A points with some B points, because the B points are on *both* sides of A. No straight cut works in one dimension.
- 2. Add a new feature, x^2 .** A becomes $1, 0, 1$. B becomes $9, 9$.
- 3. Now one threshold separates them:** $x^2 > 5$ means B. The widest-margin cut sits halfway between the closest points, A at 1 and B at 9 .
- 4. Map it back.** $x^2 > 5$ means $|x| > 2.24$: a "middle versus outside" rule, which is curved in the original space.
- 5. The kernel trick** lets an SVM work as if it had added such features (even infinitely many, for the RBF kernel) without ever computing them, by working only with similarities between points.
- 6. The settings.** C sets how much misclassification is tolerated (small C : wider margin, more errors allowed). For the RBF kernel, γ sets how local each point's influence is (large γ : wiggly boundary, a risk of overfitting).

What this shows. Data that is not separable in its original features can become separable after a transformation. Kernels do that transformation implicitly.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Class A is at $x = -2$ and 2 ; class B is at $x = 0$ and 0.5 . Can one threshold on x separate them? Can one threshold on x^2 ? Which points would be the support vectors?

Solution. On x : no, because B lies between the two A points. On x^2 : A becomes 4 and 4 , B becomes 0 and 0.25 , so any threshold between 0.25 and 4 works; the widest margin is at the midpoint, 2.125 . The support vectors are the closest points from each class in that space: the B point at 0.5 ($x^2 = 0.25$) and the A points ($x^2 = 4$).

THE FUNDAMENTALS

- Find the hyperplane maximizing margin; support vectors define it. Soft margin uses C (high C = less tolerance for errors, risk overfitting).
- **Kernels:** linear, polynomial, RBF (γ controls reach: high γ = overfitting).
- Needs **feature scaling**. Scales poorly beyond $\sim 100k$ rows. No native probabilities.

SAY IT IN 60 SECONDS

"A support vector machine finds the line, or hyperplane, that separates two classes with the widest possible margin, the gap to the nearest points. Only those nearest points, the support vectors, define the boundary. If classes overlap, a soft margin allows some mistakes, controlled by C : large C punishes errors and risks overfitting, small C allows a wider margin. For curved boundaries, the kernel trick implicitly maps data into a higher-dimensional space; the RBF kernel is the common choice with a γ parameter for how local the boundary is. The catch: SVMs need feature scaling, and training is slow on very large datasets."

KEY TERMS margin · support vectors · soft margin · C parameter · kernel trick · RBF kernel · γ · feature scaling

BUILDS ON 7.1 Core ML concepts

HANDS-ON PRACTICE Fit an RBF SVM with a small and a large γ on two moons data and describe the decision boundaries.

COMMON MISTAKES Not scaling features · Using kernel SVMs on very large datasets · Expecting calibrated probabilities

CHECK YOURSELF

1. What are support vectors?

The training points closest to the boundary (on or inside the margin); only they determine the hyperplane.

2. Effect of large C versus small C ?

Large C : few violations allowed, narrow margin, risk of overfitting. Small C : wider margin, more tolerance, risk of underfitting.

3. Why scale features before an RBF SVM?

The kernel is based on distances; a large-scale feature would dominate.

7.10 K-nearest neighbours

IN PLAIN WORDS

To guess what a new person likes, ask the few people who live closest to them and go with what most of them say. That is KNN. There is nothing to "train": it just remembers everyone. If you ask only one neighbour ($k = 1$) you copy their quirks; if you ask the whole town (k huge) you just get the average opinion. And "closest" only makes sense if distances are fair, so all measures must be on comparable scales.

Why it matters: KNN is the simplest non-parametric model and a good way to discuss bias-variance and distance.

Takeaway: Vote among the k nearest points; choose k by cross-validation; scale features.

WORKED EXAMPLE, STEP BY STEP

The choice of k changes the answer

Customers on one feature (monthly spend). Labels: S = stays, L = leaves. Query: a new customer who spends 50.

Customer	Spend	Label	Distance to 50
1	48	L	2
2	55	S	5
3	44	S	6
4	58	S	8
5	40	L	10
6	63	L	13

- Sort by distance:** 1 (L), 2 (S), 3 (S), 4 (S), 5 (L), 6 (L).
- $k = 1$:** the nearest is customer 1, so predict **L**. One neighbour is one noisy opinion.
- $k = 3$:** L, S, S, so predict **S** (2 votes to 1).
- $k = 5$:** L, S, S, S, L, so predict **S** (3 to 2).
- Weighted voting** (weight = $1 / \text{distance}$) for $k = 3$: L gets $1/2 = 0.5$; S gets $1/5 + 1/6 \approx 0.37$. Predict **L**: the closest neighbour counts most.
- Choose k by cross-validation**, often odd for two classes to avoid ties. Small k follows noise (variance); large k blurs real local patterns (bias).

What this shows. KNN has no training step: the decision is made at query time, and k and the weighting decide it. With more than one feature, scale them first, or the feature with the largest units decides alone.

YOUR TURN (COVER THE SOLUTION FIRST)

- Two features: age in years and income in rupees. Customer P is (30, 50,000) and Q is (60, 51,000). Your query is (31, 60,000). Which is nearer without scaling, and why is that a problem?

Solution. Unscaled distance to P = $\sqrt{1^2 + 10,000^2} \approx 10,000$. To Q = $\sqrt{29^2 + 9,000^2} \approx 9,000$. So Q is "nearer", although Q is 29 years older and P is nearly the same age. Income, measured in large numbers, swamps age completely. Standardise both features (subtract the mean and divide by the standard deviation) so that each contributes on a comparable scale.

THE FUNDAMENTALS

- Lazy learner: predict via majority/mean of k closest points. Distance metrics: Euclidean, Manhattan, cosine.
- Choose k by CV (small k = high variance, large k = high bias). **Needs scaling.** Slow at inference; suffers in high dimensions.

SAY IT IN 60 SECONDS

"K-nearest neighbours predicts by looking at the k most similar training points and voting, or averaging for regression. There is no training, only storing the data. Similarity is a distance, usually Euclidean, so features must be scaled. Small k gives a jagged boundary and overfits; large k smooths too much. Choose k by cross-validation, often odd to avoid ties. Prediction is slow on big data because it compares against everything, and in high dimensions distances become similar for all points, the curse of dimensionality. The catch: KNN is a good baseline, and it is sensitive to irrelevant features and unscaled units."

KEY TERMS k-nearest neighbours · lazy learner · distance metrics · choice of k · feature scaling · curse of dimensionality

BUILDS ON 7.1 Core ML concepts

HANDS-ON PRACTICE Vary k from 1 to 50 and plot validation accuracy; find the sweet spot.

COMMON MISTAKES Forgetting to scale features · Using KNN in very high dimensions · Choosing k = 1 for noisy data

CHECK YOURSELF**1. Why does KNN need scaling?**

It uses distance, so a feature with a larger numeric range dominates the neighbours.

2. How does k affect bias and variance?

Small k: low bias, high variance (jagged). Large k: higher bias, lower variance (smooth).

3. Why does KNN struggle in 1,000 dimensions?

Distances concentrate, so nearest and farthest points look similar and neighbours stop being meaningful.

7.11 Naive Bayes

IN PLAIN WORDS

Naive Bayes is a detective who adds up clues. For each suspect (class), it asks "how likely is this kind of clue among this kind of email?" and multiplies the clue likelihoods together, pretending the clues do not influence each other (that is the "naive" part). The suspect with

the highest score wins. It is surprisingly good for text, because even if words are not really independent, the combined vote still points the right way.

Why it matters: A fast, strong baseline for text, and a clean application of Bayes' rule.

Takeaway: Multiply per-feature likelihoods by the prior, assume independence, and smooth the counts.

WORKED EXAMPLE, STEP BY STEP

Why naive Bayes needs smoothing

A spam filter has learned from 100 ham (normal) emails and 100 spam emails. The word "crypto" appeared in 30 spam emails and in 0 ham emails.

1. **Raw estimate:** $P(\text{"crypto"} \mid \text{ham}) = 0 / 100 = 0$.
2. **The problem.** Naive Bayes multiplies the probabilities of every word. A single zero makes the whole product for ham zero, so *any* email containing "crypto" can never be classified as ham, however strongly the other words say ham, even if it is your colleague's article about crypto regulation.
3. **Laplace (add-one) smoothing.** Pretend every word was seen once more in each class: $P(\text{word} \mid \text{class}) = (\text{count} + 1) / (\text{documents} + 2)$ for a present/absent word. For ham: $(0 + 1) / (100 + 2) \approx 0.0098$. Small, but not impossible.
4. **For spam:** $(30 + 1) / (100 + 2) \approx 0.30$.
5. **Its effect.** "crypto" now makes spam about 30 times more likely than ham ($0.30 / 0.0098$), a strong signal, but other words can still outweigh it.

What this shows. Multiplying probabilities is fragile: one unseen word can veto everything. Smoothing keeps every probability slightly above zero. It also explains why naive Bayes probabilities are poorly calibrated even when its rankings are good.

YOUR TURN (COVER THE SOLUTION FIRST)

1. With add-one smoothing on the same 100 ham and 100 spam emails, the word "meeting" appears in 40 ham emails and 2 spam emails. Compute $P(\text{"meeting"} \mid \text{ham})$ and $P(\text{"meeting"} \mid \text{spam})$, and the factor by which "meeting" shifts the odds.

Solution. $P(\text{meeting} \mid \text{ham}) = (40 + 1) / 102 \approx 0.40$. $P(\text{meeting} \mid \text{spam}) = (2 + 1) / 102 \approx 0.029$. The ratio ham to spam is about $0.40 / 0.029 \approx 14$, so seeing "meeting" multiplies the odds in favour of ham by about 14.

THE FUNDAMENTALS

- Applies Bayes with conditional independence assumption: $P(y|x)$ proportional to $P(y) \cdot \text{product } P(x_i|y)$. Variants: Gaussian, Multinomial (text counts), Bernoulli.
- Fast, good for text, works with little data; independence is unrealistic but often fine. Use Laplace smoothing for zero counts.

SAY IT IN 60 SECONDS

"Naive Bayes applies Bayes' theorem to classify: pick the class with the highest prior times the product of the feature likelihoods. The naive part is assuming features are independent given the class, which is rarely true, yet it often works well, especially for text. It is fast to train, needs little data, and handles many features. For word counts we use multinomial Naive Bayes, and Laplace smoothing adds one to every count so that a word never seen in a class does not make the whole probability zero. The catch: the probabilities it outputs are poorly calibrated, so it ranks well but its confidence is not trustworthy."

KEY TERMS Naive Bayes · conditional independence · Laplace smoothing · Gaussian NB · multinomial NB · text classification

BUILDS ON 5.3 Bayes' theorem: the worked example

HANDS-ON PRACTICE Classify an email with two words as spam or ham by hand using the counts in the worked example.

COMMON MISTAKES Forgetting smoothing so unseen words zero out the product · Trusting the probability outputs · Applying Gaussian NB to heavily non-Gaussian features

CHECK YOURSELF**1. What is the 'naive' assumption?**

Features are conditionally independent given the class, so likelihoods multiply.

2. Why use Laplace smoothing?

A word unseen in a class would get probability 0 and zero out the whole product; adding a pseudo-count avoids that.

3. Naive Bayes is accurate at ranking but its probabilities are off. Fix?

Calibrate the outputs, for example with Platt scaling or isotonic regression.

7.12 Ensembling and the algorithm selection guide

IN PLAIN WORDS

Ask one friend for advice and you get one opinion. Ask several friends who think differently and combine their answers, and you usually do better, because their mistakes do not all point the same way. That is an ensemble. You can average them (bagging), let each one correct the last one's mistakes (boosting), or train a final "manager" model that learns how much to trust each friend (stacking). The key word is *different*: five copies of the same friend add nothing.

Why it matters: Ensembles win most structured-data competitions and power most production systems.

Takeaway: Combine diverse models; for stacking, feed the manager only out-of-fold predictions.

WORKED EXAMPLE, STEP BY STEP**Three independent models vote**

Three models are each 70% accurate, and their mistakes are independent. The ensemble takes a majority vote.

1. **The majority is right if at least two of the three are right.**
2. **All three right:** $0.7^3 = 0.343$.
3. **Exactly two right:** there are 3 ways to choose which one is wrong: $3 \times 0.7^2 \times 0.3 = 0.441$.
4. **Ensemble accuracy:** $0.343 + 0.441 = 0.784$, against 0.70 for any single model.
5. **With five such models:** $P(\text{at least 3 of 5 right}) \approx 0.837$. More independent voters, better accuracy.
6. **The catch.** If the models make the *same* mistakes (all trained on the same features with the same method), voting changes nothing. Gains come from diversity: different algorithms, features or data samples.

What this shows. Ensembles work by cancelling uncorrelated errors. That is why random forests randomise their trees and why stacking combines different kinds of model.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Model A is 80% accurate and models B and C are each 60% accurate, all with independent errors. Is a majority vote of the three better than A alone?

Solution. The vote is right when at least two are right. All three right: $0.8 \times 0.6 \times 0.6 = 0.288$. A and B right, C wrong: $0.8 \times 0.6 \times 0.4 = 0.192$. A and C right, B wrong: 0.192 . B and C right, A wrong: $0.2 \times 0.6 \times 0.6 = 0.072$. Total = 0.744 , which is worse than A's 0.80. Weak voters can outvote a strong model. Weight the votes, use stacking, or just use A.

THE FUNDAMENTALS

7.3.9 Ensembling

- **Bagging, boosting, stacking (meta-model on out-of-fold predictions), blending, voting.**
- Diversity among models is what makes ensembles work.

7.3.10 Algorithm selection guide

Situation	Suggested approach
Need interpretability / regulated (credit)	Logistic/linear regression, shallow trees, scorecards, SHAP
Tabular data, best accuracy	LightGBM/XGBoost/CatBoost
Small data	Regularized linear models, RF, Naive Bayes
Text	TF-IDF + logistic regression/linear SVM, then transformers
Images	CNNs / pre-trained vision models
Very high cardinality categoricals	CatBoost, target encoding (out-of-fold), embeddings
Need probabilities	Logistic regression, calibrated tree models
Needs quick baseline	Logistic/linear regression, DummyClassifier

SAY IT IN 60 SECONDS

"Ensembling combines models for better predictions. Bagging trains parallel models on bootstrap samples and averages them, cutting variance, as in random forest. Boosting trains sequentially, cutting bias. Stacking trains a meta-model on the predictions of several different base models, using out-of-fold predictions to avoid leakage. Simple averaging of different models also works well. For choosing an algorithm: start with a simple baseline like logistic regression, use gradient boosting for tabular data, linear models when interpretability matters, KNN or SVM for small data, and deep learning for images, text and audio. The catch: ensembles cost more to serve and explain, so use them only if the gain justifies it."

KEY TERMS bagging · boosting · stacking · blending · voting · diversity · out-of-fold predictions · algorithm selection

BUILDS ON 7.8 Gradient boosting: GBM, XGBoost, LightGBM, CatBoost

HANDS-ON PRACTICE Build a voting ensemble of logistic regression, random forest and gradient boosting, and compare its cross-validated AUC with each member.

COMMON MISTAKES Stacking with in-sample predictions (leakage) · Ensembling very similar models and expecting gains · Choosing the algorithm before understanding the constraints

CHECK YOURSELF**1. Why does stacking use out-of-fold predictions for the meta-model?**

In-sample base predictions are overly good, so the meta-model would trust them too much; out-of-fold predictions mimic unseen data and prevent leakage.

2. Best default model for a tabular business dataset?

Start with a simple baseline, then gradient boosting (XGBoost or LightGBM), which usually wins on tabular data.

3. When would you skip an ensemble?

When latency, cost, or explainability matters more than a small accuracy gain.

7.13 Optimisation: gradient descent and friends

IN PLAIN WORDS

Gradient descent is walking downhill in fog. You cannot see the valley, but you can feel the slope under your feet, so you take a step downhill and check again. The length of your step is the learning rate: too small and it takes forever; too big and you leap across the valley and end up higher. Mini-batch versions estimate the slope from a handful of examples instead of all of them, which is much cheaper. Momentum is like a heavy ball rolling: it builds up speed in a consistent direction and smooths out wiggles.

Why it matters: It is how almost every modern model is trained, and learning-rate questions are common.

Takeaway: Step against the gradient; tune the learning rate; scale your features.

WORKED EXAMPLE, STEP BY STEP**Three learning rates on the simplest loss**

Minimise $f(w) = w^2$, whose slope (gradient) is $2w$, starting from $w = 1$. Update: $w \leftarrow w - \eta \times 2w$.

Step	$\eta = 0.1$	$\eta = 0.4$	$\eta = 1.1$
start	1	1	1
1	0.8	0.2	-1.2
2	0.64	0.04	1.44
3	0.512	0.008	-1.728

- $\eta = 0.1$, too cautious.** Each step keeps 80% of w . It gets there, slowly.
- $\eta = 0.4$, well chosen.** Each step keeps 20%. Close to the minimum in three steps.
- $\eta = 1.1$, too big.** Each step multiplies w by -1.2 : it jumps over the minimum and lands further away each time. The loss *grows*. This is what "the loss exploded to NaN" looks like.
- Why features need scaling.** With two weights whose features have very different scales, one direction is steep and the other is flat. A single learning rate is then too big for one and too small for the other, and descent zig-zags. Standardising the features makes the bowl round.
- Adam and momentum** adapt the step for each weight and smooth the path, which is why they are the default for neural networks.

What this shows. The learning rate is the most important setting in gradient-based training: too small wastes time, too large diverges.

YOUR TURN (COVER THE SOLUTION FIRST)

- For the same $f(w) = w^2$ starting at $w = 1$, what happens with $\eta = 0.5$? And with $\eta = 1.0$?

Solution. $\eta = 0.5$: $w \leftarrow w - 0.5 \times 2w = 0$, the exact minimum in one step (for this particular function). $\eta = 1.0$: $w \leftarrow w - 2w = -w$, so w flips between 1 and -1 forever and never converges. For this loss, any η between 0 and 1 converges and anything above 1 diverges.

THE FUNDAMENTALS

- Gradient descent:** $w := w - \eta r * dL/dw$. **Batch** (all data), **SGD** (one sample), **mini-batch** (most common).
- Learning rate:** too high diverges, too low slow; schedules/decay.
- Momentum, RMSprop, Adam** (adaptive; default in deep learning).
- Convex vs non-convex:** linear/logistic regression are convex (global minimum); neural nets are not.
- Feature scaling speeds convergence.**

SAY IT IN 60 SECONDS

"Training means minimising a loss. Gradient descent computes the slope of the loss with respect to each weight and steps downhill: new weight equals old weight minus learning rate times gradient. Too large a learning rate overshoots or diverges; too small is slow. Batch gradient descent uses all data per step; stochastic uses one example, noisy but fast; mini-batch is the practical middle. Momentum smooths the steps, and Adam adapts a learning rate per weight, which is the usual default for deep learning. Convex losses, like linear and logistic regression, have one global minimum; neural networks have many local ones, but good optima are usually found. The catch: feature scaling speeds up convergence a lot."

KEY TERMS gradient descent · learning rate · stochastic and mini-batch · momentum · RMSprop · Adam · convex vs non-convex · feature scaling

BUILDS ON 7.4 Linear regression

HANDS-ON PRACTICE Minimise $f(x) = x^2$ with learning rates 0.1, 0.9 and 1.1 and describe each trajectory.

COMMON MISTAKES Using a learning rate that is too large and calling it "no convergence" · Forgetting to scale features · Treating local minima as the main deep-learning problem

CHECK YOURSELF**1. Loss goes up and down wildly and then becomes NaN. First thing to try?**

Lower the learning rate (and check for unscaled inputs or exploding gradients).

2. Why mini-batch gradient descent rather than full-batch?

It gives frequent, cheaper updates that fit in memory, and the noise can help escape poor regions; it also uses hardware efficiently.

3. What does Adam add to plain SGD?

Momentum plus per-parameter adaptive learning rates from running averages of gradients and squared gradients.

7.14 Hyperparameter tuning and sklearn pipelines

IN PLAIN WORDS

Hyperparameters are the dials on a machine that you set before it starts (how deep trees may grow, how strong the penalty is). Tuning is turning the dials to find the best combination. Trying every combination (grid search) is thorough but wasteful; trying random combinations (random search) finds good settings faster because only a few dials really matter. Smarter methods learn from previous tries. And the golden rule: judge the final machine on data it never saw while you were turning dials.

Why it matters: Interviewers want to see you can tune without leaking or overfitting.

Takeaway: Random search in a pipeline, cross-validated, with an untouched test set at the end.

WORKED EXAMPLE, STEP BY STEP**Five-fold cross-validation, by numbers**

Compare two settings of a model's depth using 5-fold cross-validation. AUC on each validation fold:

Fold	depth 4	depth 10
1	0.81	0.84
2	0.79	0.76
3	0.82	0.85
4	0.80	0.75
5	0.81	0.83
mean	0.806	0.806
spread (max – min)	0.03	0.10

- 1. Each fold trains on 4/5 of the training data** and validates on the remaining 1/5, so every row is used for validation exactly once.
- 2. The means tie at 0.806.** On average neither setting is better.
- 3. Look at the spread.** Depth 10 swings between 0.75 and 0.85, so it is unstable: its result depends heavily on which rows it saw, a variance problem.
- 4. Choose depth 4:** the same average with far more consistency, and a simpler model.
- 5. Refit depth 4 on all the training data,** then score it **once** on the untouched test set for the number you report.
- 6. Keep preprocessing inside the folds.** Put the scaler and imputer in a pipeline so they are refit on each fold's training part; otherwise the validation folds leak into them.

What this shows. Cross-validation gives you a mean and a spread. When the means are close, prefer the simpler, more stable setting.

YOUR TURN (COVER THE SOLUTION FIRST)

- You have 12 months of data and want to tune a forecasting model with 3 time-series folds that always train on the past. Write the train and validation months for each fold.

Solution. One common scheme (expanding window, one month of validation): Fold 1 trains on months 1 to 8 and validates on month 9. Fold 2 trains on months 1 to 9 and validates on month 10. Fold 3 trains on months 1 to 10 and validates on month 11. Month 12 is kept as the final test set. Never validate on a month that comes before any training month.

THE FUNDAMENTALS

- Grid search (exhaustive), random search (more efficient in high dimensions), Bayesian optimization (Optuna, Hyperopt; model-guided), successive halving.
- Tune with **nested CV** or a clean validation split; avoid tuning on the test set.

```

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold

num_pipe = Pipeline([('imp', SimpleImputer(strategy='median')), ('sc', StandardScaler())])
cat_pipe = Pipeline([('imp', SimpleImputer(strategy='most_frequent')),
                     ('oh', OneHotEncoder(handle_unknown='ignore'))])
pre = ColumnTransformer([('num', num_pipe, num_cols), ('cat', cat_pipe, cat_cols)])

model = Pipeline([('pre', pre), ('clf', RandomForestClassifier(class_weight='balanced',
random_state=42))])

search = RandomizedSearchCV(
    model,
    {'clf__n_estimators': [200, 400, 800], 'clf__max_depth': [None, 8, 12, 20],
     'clf__min_samples_leaf': [1, 3, 5]},
    n_iter=15, scoring='average_precision',
    cv=StratifiedKFold(5, shuffle=True, random_state=42), n_jobs=-1, random_state=42)
search.fit(X_train, y_train)

```

SAY IT IN 60 SECONDS

"Hyperparameters are settings chosen before training, like tree depth or regularisation strength. I tune them with cross-validation: split training data into k folds, train on k minus one, score on the held-out fold, and average. Grid search tries every combination; random search samples combinations and is usually more efficient; Bayesian optimisation uses past results to choose the next try. A sklearn Pipeline chains preprocessing and the model, so that during cross-validation scaling and imputing are refit inside each fold, which prevents leakage. I tune on validation folds only and touch the test set once. The catch: tuning too much on one validation set overfits to it, so I use nested cross-validation if the final estimate matters."

KEY TERMS grid search · random search · Bayesian optimisation · nested cross-validation · Pipeline · ColumnTransformer · Optuna · successive halving

BUILDS ON 7.3 Data leakage: classic examples

HANDS-ON PRACTICE Tune a random forest with RandomizedSearchCV inside a Pipeline and report the score on an untouched test set.

COMMON MISTAKES Tuning on the test set · A huge grid over unimportant parameters · Reporting the best cross-validation score as the final estimate

CHECK YOURSELF

1. Why put the scaler inside a Pipeline when cross-validating?

So it is fit only on each training fold; fitting it on all data first leaks validation information.

2. Random search versus grid search?

Random search covers more distinct values of each important hyperparameter for the same budget and is typically more efficient in high dimensions.

3. Time series data: which CV scheme?

Forward-chaining (TimeSeriesSplit): always train on the past and validate on the future.

7.15 ML Q&A and self-check

IN PLAIN WORDS

This section is your ML spoken-answer drill. Most questions are one of a few shapes: compare two things (bagging versus boosting), diagnose a symptom (high train, low test score), choose a tool for a situation, or explain why something works. For each, a strong answer names the idea in a sentence, gives the reason, then states a trade-off or a trap. Practise saying them aloud with a timer. If a question exposes a gap, jump back to that algorithm and re-read its intuition before returning here.

Why it matters: ML interviews are conversations; clear comparisons and diagnoses win more than formulas.

Takeaway: Compare, diagnose, choose, justify, each with a trade-off.

THE FUNDAMENTALS

7.6 ML Q&A (short answers)

7.7 Self-check (Module 7)

- ☐ Explain each algorithm in 2 minutes: intuition, loss, key hyperparameters, pros/cons.
- ☐ Implement logistic regression and k-means from scratch.
- ☐ For a given business problem, justify the algorithm choice.

SAY IT IN 60 SECONDS

"For an ML question I use a four-part answer. First, name the concept in plain words. Second, give the reason it works or fails, the mechanism. Third, a concrete example with numbers or a situation. Fourth, the trade-off: what it costs and when I would not use it. For bias and variance: too-simple models miss patterns, too-flexible ones memorise noise; for example, a depth-two tree versus a depth-thirty tree; the trade-off is controlled with regularisation and validation. If I do not know, I reason from the basics and say what I would check. The catch: long theory without an example loses the listener."

KEY TERMS random forest variance · feature scaling · multicollinearity · regularisation trade-off · train-serve skew · k-means · precision-recall · SHAP · fit vs transform

BUILDS ON 7.8 Gradient boosting: GBM, XGBoost, LightGBM, CatBoost

HANDS-ON PRACTICE Answer each question aloud in under 60 seconds with one example, then reveal and compare.

COMMON MISTAKES Answers that list terms without a reason · Forgetting the train-serve skew causes · Not mentioning calibration or costs

CHECK YOURSELF**1. Give the four-part answer for "why does regularisation reduce overfitting?"**

Concept: it penalises large weights. Mechanism: it limits model flexibility so it cannot fit noise.

Example: ridge shrinks coefficients on noisy features. Trade-off: too much causes underfitting, tuned by validation.

2. An interviewer asks "which model would you use?" with no details. What do you do?

Ask about the data size, features, target, interpretability and latency needs; then propose a baseline and a stronger option with reasons.

3. How do you diagnose a model with strong offline metrics but poor production results?

Check leakage, train-serve skew, distribution drift, label delay and whether the offline metric matches the business objective.

Interview tips, tricks and hacks

- 1. Four beats for every algorithm question:** the idea in one sentence, why it works, an example, and the trade-off. Never finish an answer without the trade-off.
- 2. Diagnose from the two scores.** High training and low validation score means variance; both low means bias. State the diagnosis, then two remedies. This pattern answers a surprising share of ML questions.
- 3. Keep a comparison ready for the pairs.** Bagging versus boosting, L1 versus L2, random forest versus gradient boosting, logistic regression versus trees, bias versus variance. Interviewers love "versus" questions; have a crisp two-sentence contrast for each.
- 4. Always propose a baseline first.** "I would start with logistic regression to set a bar and check the pipeline, then try gradient boosting." It signals engineering maturity.
- 5. Treat a near-perfect score as a warning.** If you are told a model has 99 per cent accuracy, say you would check for leakage and class imbalance before celebrating.
- 6. Know which models need scaled features.** Distance-based and gradient-based models do (KNN, SVM, regularised linear models, neural networks). Trees and their ensembles do not. It is a frequent quick question.
- 7. Explain regularisation as a budget.** "The model pays a price for large weights, so it only keeps the ones that earn their place." L1 can set weights exactly to zero, which selects features.
- 8. Put preprocessing inside the pipeline.** Say "I fit the scaler and imputer on the training fold only, inside a pipeline, so cross-validation cannot leak." It is one of the clearest signals of real experience.
- 9. For "which model would you use?", ask before answering.** Data size, feature types, need for explanation, latency. Then give a baseline and a stronger alternative.
- 10. Tie the model to a decision.** A model is only useful if someone acts on its output. Mention the action and the cost of each type of error.

Frequently asked questions

30 questions interviewers ask on this chapter, with model answers. Cover the answer, say yours aloud, then compare.

CORE ML CONCEPTS

Q1 Training accuracy is 99% and validation accuracy is 71%. What would you try, in order?

A large train-validation gap signals **overfitting**. (1) Rule out bugs: leakage or duplicated rows can also make training look too good or validation look inconsistent, and check that the split mimics production (time or group splits). (2) Reduce complexity: limit tree depth, increase regularisation, prune features. (3) Get more or better data, add augmentation where applicable. (4) Use early stopping and ensembling (bagging). (5) Validate with cross-validation so the estimate is stable. Report validation, not training, performance.

WHY L1 GIVES SPARSITY

Q2 In a Bayesian view, what priors do L1 and L2 correspond to?

MAP estimation with a **Gaussian** prior on the weights gives an L2 penalty; with a **Laplace** prior (double exponential) it gives an L1 penalty. The Laplace density has a sharp peak at zero and heavier tails than the Gaussian, which expresses the belief "most weights are exactly zero, a few are large".

DATA LEAKAGE: CLASSIC EXAMPLES

Q3 You build a churn model with a feature "days since last login" computed at the time the data was extracted (today). The label is whether the customer churned in the last 30 days. What is wrong?

"Days since last login" at extraction time includes the period in which churn happened; customers who churned have not logged in for a long time *because* they churned. The feature must be computed **as of the prediction date** (before the label window begins), using only information available then. Rebuild the dataset with a clear **snapshot date**: features from before it, label from the window after it.

LINEAR REGRESSION

Q4 Your regression's R^2 is 0.92 on training data and 0.35 on test data. What happened, and what do you try?

A large drop from train to test signals **overfitting** (too many features, high-degree terms, small data) or **distribution differences** between train and test. Try ridge/lasso, fewer or simpler features, more data, and cross-validation; check for leakage in the training features and whether the test set comes from a different period or population. Always report the held-out metric.

LOGISTIC REGRESSION

Q5 A logistic regression coefficient for "monthly_spend" (in thousands of rupees) is -0.4. Interpret it.

$\exp(-0.4) \approx 0.67$: holding other features fixed, each additional ₹1,000 in monthly spend multiplies the **odds** of the positive class (say, churn) by 0.67, a 33% decrease in the odds. The change in probability depends on the starting probability. As with all coefficients, this describes association, not cause, and with regularisation or scaling the units matter.

DECISION TREES

Q6 Train accuracy is 100% and validation accuracy is 78% for your decision tree. What do you change?

A fully grown tree memorises the training set. Constrain it (`max_depth`, `min_samples_leaf`, `ccp_alpha`), select features, and validate with cross-validation. If you still need more accuracy, use an **ensemble**: a random forest averages many decorrelated deep trees to cut variance, and gradient boosting fits many shallow trees sequentially to cut bias. Keep the single tree when interpretability is the priority.

RANDOM FOREST (BAGGING)

Q7 Why does a random forest choose a random subset of features at each split, and what happens if `max_features` equals all features?

If one feature is very strong, every bootstrap tree would split on it first and the trees would be highly correlated, so averaging gains little variance reduction. Restricting each split to a random subset forces diversity. With `max_features = all` you get **bagged trees**, which still help but less. Smaller values reduce correlation but can increase bias because the best feature is sometimes unavailable; tune it.

GRADIENT BOOSTING: GBM, XGBOOST, LIGHTGBM, CATBOOST

Q8 You halve the learning rate of a boosting model. What else should you change and why?

`learning_rate` and `n_estimators` trade off: a smaller step needs more rounds to reach the same fit, but usually generalises better. Increase `n_estimators` (or set it large and use **early stopping** on a validation set), accept longer training, and re-check other regularisation (subsampling, `num_leaves`). The standard advice is to choose a small learning rate (0.01 to 0.1), set many trees, and let early stopping decide.

SUPPORT VECTOR MACHINES

Q9 An RBF SVM scores 100% on training data and 70% on validation. Which hyperparameters do you adjust?

High training and low validation accuracy means overfitting. A large `gamma` makes the decision boundary hug individual points and a large `C` punishes every training error; reduce both (search on a log scale with cross-validation), make sure features are scaled, and consider a linear kernel or more data.

K-NEAREST NEIGHBOURS

Q10 Your KNN model with $k = 1$ has 100% training accuracy. Is that impressive?

With $k = 1$ every training point's nearest neighbour is itself (distance zero), so training accuracy is always 100% and says nothing about generalisation. Evaluate on validation data or with cross-validation, and expect $k = 1$ to overfit noisy data.

NAIVE BAYES

Q11 Naive Bayes assumes conditional independence, which is clearly false for words. Why does it still work?

What matters for classification is which class gets the highest score, not the exact probability. Correlated features over-count evidence, making the probabilities extreme, but the **argmax** often stays correct. Also the model has low variance (few parameters), which is valuable with limited data. If calibrated probabilities are needed, recalibrate (Platt or isotonic), or use logistic regression.

ENSEMBLING AND THE ALGORITHM SELECTION GUIDE

Q12 You have 5,000 rows, 30 mostly numeric features, a regulator who requires explainable decisions, and a binary target. Which model and why?

Start with **regularised logistic regression** (possibly scorecard-style with WoE binning): interpretable coefficients, stable on 5,000 rows, easy to document. Compare with a constrained **gradient boosting** model (monotonic constraints, shallow trees) and explain it with **SHAP**; if the lift over logistic regression is small, prefer the simpler model. Validate with stratified cross-validation and report metrics tied to the business cost (for example KS, AUC, and expected loss at the approval threshold).

OPTIMISATION: GRADIENT DESCENT AND FRIENDS

Q13 Training loss goes to `nan` after a few steps. What do you check?

`nan` almost always means numerical blow-up. Check: (1) the **learning rate** (reduce it by 10x); (2) **feature scaling** and outliers in inputs or targets; (3) **exploding gradients** in deep nets (use gradient clipping, proper initialisation, normalisation layers); (4) invalid operations such as `log(0)` or division by zero in a custom loss; (5) corrupt data with NaNs or infinities.

HYPERPARAMETER TUNING AND SKLEARN PIPELINES

Q14 You ran a 500-configuration search and the best cross-validated AUC is 0.91. Your test AUC is 0.87. Is something wrong?

Choosing the maximum over hundreds of noisy estimates biases the winning score upward (a **selection effect**). A small gap like 0.91 to 0.87 is expected; a huge gap would suggest leakage or distribution shift between the training and test periods. Report the test number, reduce the search size or use nested CV, and check that the test set was used only once.

ML Q&A AND SELF-CHECK**Q15 How does Random Forest reduce variance?**

Averages many decorrelated high-variance trees (bootstrap + feature subsampling).

Q16 Why scale features?

Distance/gradient-based models (KNN, SVM, NN, regularized linear) are scale-sensitive; trees are not.

Q17 How to detect/handle multicollinearity?

Correlation matrix, VIF; drop/merge features, PCA, regularization.

Q18 What is regularization strength trade-off?

Higher lambda = simpler model, more bias, less variance.

Q19 Why does a model perform well in training and poorly in production?

Leakage, train-serve skew, distribution drift, different population, feature unavailability at inference, overfitting to validation.

Q20 How do you pick k in K-Means?

Elbow (inertia), silhouette score, gap statistic, domain needs.

Q21 Is accuracy always good?

No; with imbalance use precision/recall/PR-AUC/F1.

Q22 How do trees handle missing values?

Depends: XGBoost/LightGBM learn default direction; sklearn (recent versions) supports NaN in some tree models; otherwise impute.

Q23 Explain precision-recall tradeoff.

Raising threshold increases precision, decreases recall, and vice versa.

Q24 How to explain a black-box model?

Feature importance, permutation importance, PDP/ICE, SHAP, LIME, surrogate models.

Q25 Bagging vs boosting vs stacking?

See 7.3.5/7.3.9.

Q26 Difference between parameters and hyperparameters?

Parameters learned from data (weights, split points); hyperparameters set before training (depth, lr).

Q27 What is the difference between `fit`, `transform`, `fit_transform`?

`fit` learns statistics from data; `transform` applies them; `fit_transform` does both (only on train).

Q28 How do you handle missing values in tree-based vs linear models?

Gradient-boosting libraries such as XGBoost and LightGBM learn a **default branch** for missing values, and recent scikit-learn tree models also accept NaN; random forests in older setups need imputation. Linear and distance-based models need explicit **imputation** (median or group-wise), usually with a **missing indicator** feature, because the fact that a value is missing may itself be informative. Always fit imputers on training data only, and monitor the missing rate in production.

Q29 What is the difference between a parameter and a hyperparameter, with examples?

Parameters are learned by the training algorithm (regression weights, tree split thresholds, neural-network weights). **Hyperparameters** control the learning process or model capacity and are chosen by you or by search (tree depth, learning rate, regularisation strength, `k` in KNN). Hyperparameters are tuned on validation data, parameters on training data.

Q30 Your classification model has AUC 0.85. Can you tell whether it is good enough?

AUC summarises ranking quality independent of threshold, but "good enough" depends on the baseline (logistic regression, current rules), class balance (PR-AUC may be more informative), the **cost of each error**, calibration, and performance in important segments. Report precision/recall (or lift) at the **operating threshold** and the expected business value, and check stability over time.

CORE CONCEPTS

- **Supervised:** labels. Regression (number) / classification (category). **Unsupervised:** structure without labels (clustering, dimensionality reduction). **Reinforcement:** reward from actions.
- Goal: **generalise** to unseen data. Split: train (fit), validation (tune), test (touch once for the final score).
- **Bias** = error from too simple a model (underfit). **Variance** = sensitivity to the training sample (overfit). Total error = $\text{bias}^2 + \text{variance} + \text{irreducible noise}$.

Symptom	Diagnosis	Fix
train bad, val bad	high bias	richer model, features, less regularisation
train good, val bad	high variance	more data, regularise, simpler, bagging, early stop
both good, prod bad	leakage / drift / skew	audit features, monitor

- **Learning curve:** error vs training size. Curves that converge high: bias. A persistent gap: variance (more data helps).
- Parametric (fixed number of weights: linear, logistic) vs non-parametric (grows with data: KNN, trees).
- Loss: MSE (regression, outlier-sensitive), MAE (robust), Huber, log loss / cross-entropy (classification), hinge (SVM).

REGULARISATION

- Add a penalty on weights: **L2 (ridge)** $\lambda \sum w^2$ shrinks smoothly; **L1 (lasso)** $\lambda \sum |w|$ sets some weights exactly to 0 (feature selection); **elastic net** mixes both (handles correlated features).
- Why L1 is sparse: the diamond-shaped constraint has corners on the axes; L1's pull is constant, L2's pull fades as $w \rightarrow 0$.
- Larger λ = simpler model (more bias, less variance). Choose λ by cross-validation. **Scale features first.**
- Also: dropout, early stopping, max depth, pruning, data augmentation, more data.

DATA LEAKAGE

Model sees information at training that will not exist at prediction time. Scores look great, production fails.

- **Target leakage:** a feature derived from the outcome ("refund issued" predicting fraud; "days since account closed" predicting churn).
- **Train-test contamination:** scaling, imputing, encoding or feature selection fitted on all data; duplicates across the split; SMOTE before splitting.
- **Temporal:** random split on time-ordered data (use forward chaining); features computed with future data.
- **Group:** same customer or patient in train and test (use group k-fold).
- Fix: split first, fit every step in a Pipeline on training folds only; ask "will this exist at prediction time?"; distrust near-perfect scores.

LINEAR REGRESSION

- $y = Xw + \epsilon$, fit by least squares: $w = (X^T X)^{-1} X^T y$ (normal equation) or gradient descent. Coefficient = change in y per unit of x, other features fixed.
- Assumptions (**LINE**): Linear relationship, Independent errors, Normal errors (for inference), Equal variance (homoscedasticity). Check residual plots, Q-Q plot.
- $R^2 = 1 - \text{SSres}/\text{SStot}$ (rises with every feature; use adjusted R^2 or validation error).
- **Multicollinearity:** correlated features give unstable coefficients; check VIF > 5-10; drop, combine, or use ridge.
- Outliers and leverage points pull the line; extrapolation is unsafe. Heteroscedasticity: log target, weighted least squares, robust SEs.

LOGISTIC REGRESSION

- $p = 1/(1 + e^{-z})$, $z = w \cdot x + b$. It models **log-odds** linearly: $\log(p/(1-p)) = z$. $\exp(w) = \text{odds ratio}$ per unit.
- Loss: log loss = $-[y \log p + (1-y) \log(1-p)]$, trained by maximum likelihood. Linear decision boundary. Multiclass: softmax.
- 0.5 is a default threshold, not a law: choose from the cost of errors (precision-recall curve).
- Regularised by default in sklearn ($C = 1/\lambda$). Probabilities are usually well calibrated; scale features for faster convergence.
- Good baseline: interpretable, fast, stable; weak on interactions unless you add features.

DECISION TREES

- Greedy recursive splits: choose the feature and threshold with the largest impurity reduction. **Gini** = $1 - \sum p^2$; **entropy** = $-\sum p \log_2 p$; regression uses variance.
- Example: node 50/50 has Gini 0.5; a split into 45/5 and 5/45 gives 0.18 each, gain 0.32.
- Controls: max_depth, min_samples_leaf, ccp_alpha (pruning). Deep trees overfit; small data changes alter the tree.
- No scaling needed, handles mixed types, captures interactions; step-like, poor at extrapolation.

RANDOM FOREST (BAGGING)

- Many trees on **bootstrap samples**, each split on a random feature subset ($\text{max_features} \sim \sqrt{p}$); average (regression) or vote. Variance falls; bias unchanged.
- **OOB score:** each tree is tested on rows it did not see (~37%): a free validation estimate.
- Importance: impurity-based favours high-cardinality features; prefer **permutation importance** or SHAP.
- Few hyperparameters ($n_estimators$, max_depth, min_samples_leaf); robust; cannot extrapolate beyond training range; slow, large models.

GRADIENT BOOSTING

- Trees built in **sequence**: each fits the residuals (negative gradient of the loss) of the ensemble so far; prediction = $\sum \text{learning_rate} \times \text{tree}$. Reduces **bias**; shallow trees (depth 3-8).
- Key knobs: learning_rate (small + many trees), n_estimators, max_depth, subsample, colsample_bytree, min_child_weight, L1/L2 (reg_alpha/lambda). Use **early stopping** on a validation set.
- **XGBoost:** regularised, second-order gradients, handles missing values, parallel. **LightGBM:** histogram bins, leaf-wise growth, fast on big data. **CatBoost:** native categorical handling, ordered boosting (less leakage).
- Usually the strongest model on tabular data. Overfits noisy labels; needs tuning; less interpretable (use SHAP).

ALGORITHM CARDS

Model	Idea	Scale?	Watch out
SVM	maximum-margin hyperplane; support vectors only; kernels (RBF) for curves	yes	C large = overfit, gamma large = wiggly; slow on big n
KNN	vote of k nearest points; no training	yes	slow predict, curse of dimensionality; choose k by CV
Naive Bayes	P(class) × product of P(feature given class) assuming conditional independence	no	poor probabilities; Laplace smoothing; great text baseline
Linear / logistic	weighted sum	yes	non-linearity needs features
Tree / forest / boost	thresholds	no	trees extrapolate poorly

- SVM margin = $2/|w|$; soft margin trades errors for width. KNN example: unscaled income (dollars) swamps age in distance, so scale first.
- Naive Bayes example: $P(\text{spam} \mid \text{"free"}) = 0.4 \times 0.5 / (0.4 \times 0.5 + 0.6 \times 0.05) = 87\%$.

ENSEMBLING

- Bagging**: parallel models on bootstrap samples, average: cuts variance (random forest).
- Boosting**: sequential error correction: cuts bias (GBM, AdaBoost).
- Stacking**: a meta-model learns from base models' **out-of-fold** predictions (in-sample predictions leak).
- Voting/averaging**: combine diverse models. Gains need diversity; costs latency and explainability.
- Pick an algorithm**: start with a simple baseline (logistic/linear) → gradient boosting for tabular → linear when explanation or regulation matters → KNN/SVM for small data → deep learning for images, text, audio.

OPTIMISATION

- Gradient descent: $w \leftarrow w - \eta \text{ grad } L(w)$. Too large η diverges (NaN loss); too small is slow. Scale features.

- Batch** (all data), **stochastic** (one row, noisy), **mini-batch** (32-512, the default). **Momentum** smooths steps; **Adam** keeps per-parameter adaptive rates (default for deep nets); **RMSprop**, **AdaGrad**.
- Convex losses (linear, logistic) have one global minimum; neural nets have many local ones and saddle points. Use learning-rate schedules and early stopping.
- Normal equation $O(p^3)$ vs GD $O(n \cdot p)$ per step: GD for many features.

CROSS-VALIDATION AND TUNING

- k-fold** (k = 5): train on k - 1 folds, validate on one, average. **Stratified** for classes, **group k-fold** for entities, **TimeSeriesSplit** (forward chaining) for time.
- Grid search** (all combinations), **random search** (more efficient), **Bayesian/Optuna** (learns from trials). Tune on validation folds, test set once. **Nested CV** for an honest estimate when tuning heavily.

```
from sklearn.pipeline import
make_pipeline
from sklearn.preprocessing import
StandardScaler
from sklearn.linear_model import
LogisticRegression as LR
from sklearn.model_selection import
GridSearchCV
pipe = make_pipeline(StandardScaler(),
LR())
gs = GridSearchCV(pipe,
{"logisticregression__C": [.1, 1,
10]}),
cv=5, scoring="roc_auc")
gs.fit(X_train, y_train)
gs.best_params_
```

Scaler inside the pipeline = refit per fold = no leakage.

TYPICAL HYPERPARAMETERS (SKLEARN)

Model	Tune first	Typical range
Ridge / Lasso	alpha	1e-4 to 1e2, log scale
LogisticRegression	C (1/λ), penalty	1e-3 to 1e2
RandomForest	n_estimators, max_depth, min_samples_leaf, max_features	200-500, 5-30, 1-20, sqrt
GradientBoosting / XGB	learning_rate, n_estimators, max_depth, subsample	.01-.3, 100-2000, 3-8, 0.5-1
SVM (RBF)	C, gamma	1e-2 to 1e3, 1e-4 to 1
KNN	n_neighbors, weights	3-30, uniform/distance
DecisionTree	max_depth, min_samples_leaf, ccp_alpha	3-15

INTERPRETING MODELS

- Linear/logistic**: coefficients (scaled features) and odds ratios; check significance with statsmodels.
- Trees/forests**: impurity importance (biased); **permutation importance** (shuffle a feature, measure the score drop) is better.
- SHAP**: per-prediction additive contributions; global bar and beeswarm plots. **PDP/ICE**: average effect of a feature. **LIME**: local linear surrogate.
- Calibration: reliability curve; fix with Platt scaling or isotonic regression (CalibratedClassifierCV). Needed when probabilities drive decisions.
- Correlated features share importance; interpretation is not causation.

EVERYDAY SKLEARN

```
from sklearn.ensemble import (
RandomForestClassifier as RF)
m = RF(n_estimators=300, random_state=0)
m.fit(X_tr, y_tr)
p = m.predict_proba(X_te)[: , 1]
from sklearn.model_selection import (
cross_val_score)
cross_val_score(m, X, y, cv=5,
scoring="f1")
from sklearn.metrics import roc_auc_score
roc_auc_score(y_te, p)
```

fit on train only; transform test with train-fitted objects; set random_state.

WHEN IT BREAKS: SYMPTOM -> TRY

Symptom	Try
Overfits	regularise, prune, more data, bagging, early stop
Underfits	features, interactions, bigger model, lower λ
Slow training	subsample, LightGBM, simpler model, fewer features
Unbalanced	class weights, threshold, PR-AUC (see Module 8)
Unstable coefficients	drop collinear features, ridge
Offline great, online poor	leakage, train-serving skew, drift, wrong metric

SAY IT IN 20 SECONDS

- Bias-variance**: simple models underfit, flexible ones memorise noise.
- Bagging vs boosting**: average independent models vs fix errors in sequence.
- L1 vs L2**: L1 zeroes weights, L2 shrinks them.
- Leakage**: using information from the future or the label.
- Idea, reason, example, trade-off.

8

CHAPTER 8 · HIGH PRIORITY · MODULE 8 IN THE APP

Feature Engineering, Metrics, Imbalanced Data

Models are only as good as the features you feed them and the yardstick you judge them by. This chapter covers preparing data, choosing metrics from business cost, and handling rare classes.

7 topics · about 3 h 35 min in the app · 7 questions answered · two-page cheat sheet at the end

- | | |
|--|---|
| 8.1 Data preprocessing: missing values, outliers, encoding, scaling | 8.4 Classification metrics |
| 8.2 Feature engineering and feature selection | 8.5 Choosing the metric from business cost |
| 8.3 Dimensionality reduction: PCA, t-SNE, UMAP, LDA | 8.6 Regression, ranking and clustering metrics |
| | 8.7 Imbalanced data playbook and self-check |

Foundations

In practice, the algorithm is rarely what separates a good model from a poor one. The difference is made before training, in how the data is prepared, and after training, in how the result is measured. Interviewers know this, which is why questions about missing values, encoding and metrics are as common as questions about algorithms.

Preparing data: four recurring problems

Missing values. The first question is why the value is missing, not how to fill it. If people with high incomes tend to skip the income question, the gap itself carries information. Simple options are filling with the median or the most common value; a useful addition is a separate flag column that records that the value was missing.

Outliers. Decide whether an extreme value is an error or a genuine rarity. Errors are fixed or removed. Genuine extremes are kept, and handled by capping, taking logarithms, or using models that are not disturbed by them. Never delete points just because they hurt your score.

Categories. Models need numbers. With a few categories, create one yes-or-no column per category (**one-hot encoding**). With an order, such as small, medium, large, use ordered integers. With thousands of categories, use the category's frequency or its average outcome (**target encoding**), computed carefully so that a row's own answer never leaks into its feature.

Scale. If one feature ranges over thousands and another over fractions, any model based on distances or gradients will be dominated by the larger. **Standardising** puts features on a common scale. Tree-based models compare a feature only with a threshold and so do not need it.

One rule governs all four: every statistic used in preparation, whether a median for filling or a mean for scaling, is learned from the **training data only** and then applied unchanged to validation and test data.

Features carry the signal

Feature engineering is the craft of presenting the data so the pattern is easy to find. A timestamp becomes hour of day and day of week. A transaction history becomes "number of orders in the last 30 days" and "days since last order". A ratio such as spend per visit can say more than either number alone. The best features come from understanding the problem, which is why interviewers ask what features you would build before they ask which model you would use. One discipline applies: a feature may only use information available before the moment of prediction.

Measuring: start from the four outcomes

For a yes-or-no prediction, there are exactly four possibilities.

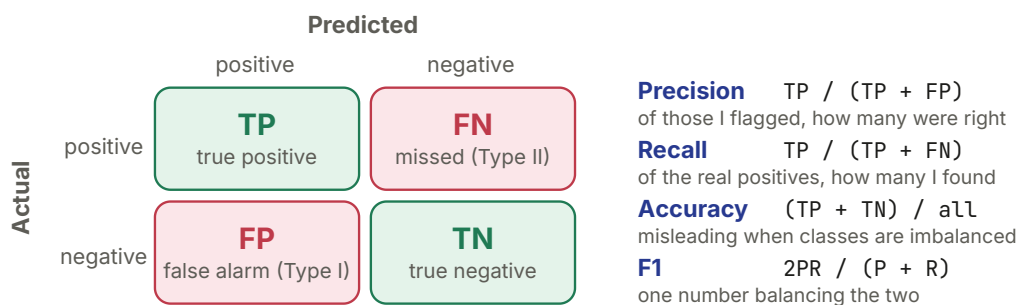


Figure 10. Every classification metric is a ratio of these four counts.

Accuracy, the share of predictions that are right, is natural but treacherous. If one transaction in a hundred is fraudulent, a model that never flags anything is 99 per cent accurate and completely useless. The two metrics that matter are **precision**, of the cases I flagged how many were real, and **recall**, of the real cases how many I caught. They pull against each other: flag more and you catch more but raise more false alarms. The model outputs a score, and the **threshold** you choose decides where on that trade-off you sit.

The metric comes from the cost of mistakes

This is the judgement interviewers most want to see. In cancer screening a missed case is far worse than a false alarm, so you favour recall. In a spam filter, blocking a real message is worse than letting spam through, so you favour precision. If costs can be put in money, compute the expected cost at each threshold and choose the cheapest. If only the top fifty cases can be reviewed each day, measure precision among the top fifty. Summary measures such as ROC-AUC and PR-AUC describe ranking quality across all thresholds, with PR-AUC the more honest when positives are rare.

For numerical predictions the same thinking applies. Mean absolute error treats all misses proportionally; root mean squared error punishes large misses more, so use it when big errors are disproportionately harmful.

Rare classes

When one class is rare, change the metric first, keep the class proportions the same in every data split, and then give the rare class more weight in training or rebalance the training data. Any rebalancing happens inside the training portion only. Finally, tune the threshold on validation data.

Topic by topic

Each topic gives the idea in plain words, the fundamentals to know, a model spoken answer and three questions to test yourself. Open the same topic in the app for worked examples, runnable code and interactive demos.

8.1 Data preprocessing: missing values, outliers, encoding, scaling

IN PLAIN WORDS

Raw data is messy: there are blanks, weird extremes, text labels and numbers on wildly different scales. Preprocessing is tidying up so a model can learn: fill the blanks sensibly (and remember which were blank), check whether the extreme values are real or typos, turn words like "Delhi" into numbers in a way that does not invent an order, and put numbers on similar scales so no single column shouts over the others. The golden rule is to learn all the tidy-up rules from the training data only.

Why it matters: Preprocessing choices move model performance as much as the algorithm, and they are a common source of leakage.

Takeaway: Understand why data is missing or extreme, encode categories carefully, scale when the model needs it, and fit everything on train only.

WORKED EXAMPLE, STEP BY STEP

Fit on training data, apply to test data

Training ages: 20, 30, 40 and one missing value. One test customer is 50.

- 1. Impute with a statistic learned from training only.** The training median is 30, so the missing age becomes 30. Training ages are now 20, 30, 40, 30.
- 2. Learn the scaling from training only.** Mean = 30; standard deviation = 7.07.
- 3. Scale the training rows:** $(20 - 30) / 7.07 = -1.41$; 30 gives 0; 40 gives +1.41; the imputed 30 gives 0.
- 4. Apply the same numbers to the test row:** $(50 - 30) / 7.07 = +2.83$. The test customer is clearly older than anyone in training, and the model sees that honestly.
- 5. The leaky version.** If you had scaled before splitting, using all five ages (mean 34, standard deviation 10.2), the test value would be $(50 - 34) / 10.2 = +1.57$. The test customer has quietly shaped the scaling, so evaluation is no longer a fair rehearsal of the future.
- 6. Make it automatic:** put the imputer and scaler in a `Pipeline`, so they are fitted inside each training fold and only applied to validation and test data.

What this shows. Every number used to prepare data (medians, means, category lists) is part of the model and must be learned from the training data alone.

YOUR TURN (COVER THE SOLUTION FIRST)

1. You one-hot encode `city` using the training data, which has Pune, Delhi and Mumbai. In production a customer from Jaipur arrives. What happens with a default encoder, and how do you handle it?

Solution. A strict encoder raises an error for an unseen category, or the columns no longer match the model's inputs. Use `OneHotEncoder(handle_unknown="ignore")`, so Jaipur becomes all zeros (none of the known cities), or group rare and unseen cities into an "other" bucket during training. Then monitor how often unseen categories appear: a sudden rise is a sign of drift.

THE FUNDAMENTALS**Missing values**

- Types: MCAR (random), MAR (depends on observed), MNAR (depends on the missing value itself).
- Options: drop rows/columns (if few), mean/median/mode, constant + **missing indicator flag**, group-wise imputation, KNN imputer, iterative (MICE), model-based, leave NaN for GBMs.
- Always fit imputers on train only.

Outliers

- Detect: IQR, z-score, robust z (MAD), isolation forest, domain rules.
- Treat: investigate first; cap/winsorize, transform (log), robust scaling/models, or separate model.

Encoding categoricals

Method	When	Caveat
One-hot	Low cardinality, linear/NN models	Dimension blowup; drop one to avoid dummy trap in linear models
Ordinal/label	Ordered categories; tree models for ids	Implies order for linear models
Frequency/count encoding	High cardinality	Collisions in meaning
Target/mean encoding	High cardinality	Leakage ; use out-of-fold + smoothing
Hashing	Very high cardinality, streaming	Collisions
Embeddings	Deep learning	Needs data
Weight of Evidence (WoE)	Credit scorecards	Needs binning and monotonic checks

Scaling

- **Standardization** (z-score): mean 0, SD 1; good for most models and PCA.
- **Min-Max**: [0,1]; for NN/bounded inputs, sensitive to outliers.
- **Robust scaler**: median/IQR; outlier-tolerant.
- Needed for: KNN, SVM, k-means, linear/logistic with regularization, NN, PCA. Not needed for: trees/forests/boosting.

Transformations: log/log1p (right skew), sqrt, Box-Cox/Yeo-Johnson, binning, clipping, polynomial/interactions.

SAY IT IN 60 SECONDS

"Preprocessing turns raw data into something a model can use. Missing values: first ask why they are missing; then drop, impute with the median or a model, and add a missing flag, since missingness can itself be informative. Outliers: check if they are errors or real, then cap, transform or leave them for robust models. Categorical encoding: one-hot for few categories, ordinal for ordered ones, target or frequency encoding for high cardinality, with care to avoid leakage. Scaling: standardise or min-max for distance and gradient-based models; trees do not need it. The catch: every statistic, such as the mean to impute or the scaler's parameters, must be learned on training data only."

KEY TERMS missing values · MCAR MAR MNAR · imputation · outliers · winsorizing · one-hot encoding · target encoding · scaling · log transform

BUILDS ON 7.3 Data leakage: classic examples

HANDS-ON PRACTICE Build a ColumnTransformer that imputes, scales numeric columns and one-hot encodes categoricals, and cross-validate it.

COMMON MISTAKES Fitting imputers and scalers on the full dataset · Target encoding without out-of-fold computation and smoothing · Deleting outliers without investigating them

CHECK YOURSELF

1. A 'city' column has 5,000 distinct values. How do you encode it?

Not one-hot (too wide). Use frequency or target encoding computed out-of-fold, hashing, or an embedding.

2. Which models need scaled features and which do not?

Need it: KNN, SVM, linear/logistic regression with regularisation, neural nets, PCA. Do not: decision trees, random forests, boosting.

3. Why add a 'was_missing' flag when imputing?

Missingness often carries signal (for example no income reported); imputing alone hides it.

8.2 Feature engineering and feature selection

IN PLAIN WORDS

Models only know the columns you give them, so good features are clues you craft. "Day of week" and "is it a festival?" may explain orders better than a raw timestamp. "Orders in the last 30 days" says more about a customer than their ID. Ratios such as "debt divided by income" compress two numbers into one meaningful one. Afterwards, feature selection removes clues that are useless or repeat each other. The safety question to ask of every clue: "Will I actually know this at the moment I have to predict?"

Why it matters: Feature engineering is where domain knowledge becomes model performance, and interviews test it with "what features would you build?".

Takeaway: Build informative, time-safe features; then prune the redundant ones.

WORKED EXAMPLE, STEP BY STEP**Build features from raw orders, as of a date**

Predict on 1 April whether a customer will order in April. Her orders: 3 January (800), 20 February (1,200), 10 March (600), 25 March (900).

- 1. Fix the as-of date: 1 April.** Use only orders before it.
- 2. Recency:** days since the last order = 1 April – 25 March = **7 days**.
- 3. Frequency:** orders in the last 90 days (since 2 January) = **4**; in the last 30 days (since 2 March) = **2**.
- 4. Monetary:** total spend in the last 90 days = 800 + 1,200 + 600 + 900 = **3,500**; average order value = **875**.
- 5. Trend:** 2 orders in the last 30 days against 4 in 90, so her monthly rate is roughly steady (2 per month against an average of 1.3), slightly rising.
- 6. The leak to avoid.** Do not compute "average order value over the whole year", because it would include April, the very period you are predicting.

Feature	Value	Plain meaning
recency_days	7	bought very recently
orders_90d	4	a regular customer
orders_30d	2	currently active
aov_90d	875	basket size

What this shows. Good features summarise behaviour up to the prediction moment: how recently, how often, how much and which way it is moving.

YOUR TURN (COVER THE SOLUTION FIRST)

1. You encode hour of day as $\sin(2\pi \cdot h/24)$ and $\cos(2\pi \cdot h/24)$ instead of the raw number 0 to 23. Why? Compare 23:00 with 00:00 in both encodings.

Solution. As raw numbers, 23 and 0 are 23 apart, although they are one hour apart in reality. With the cyclic encoding, 23:00 is about $(-0.26, 0.97)$ and 00:00 is $(0, 1)$, a distance of about 0.26, the same as any other pair of adjacent hours. 12:00 is $(0, -1)$, the farthest point from midnight. The two numbers place the hours on a clock face, so the model sees that the day wraps around.

THE FUNDAMENTALS**8.2 Feature engineering ideas (be ready with examples)**

- **Datetime:** hour, day of week, month, is_weekend, is_holiday (Indian holidays/festivals), days since event, cyclical encoding (sin/cos for hour/month).
- **Aggregates:** user-level count, sum, mean, std over last 7/30/90 days; recency, frequency, monetary.
- **Ratios:** debt-to-income, orders per month, cart-to-purchase ratio.
- **Text:** length, word counts, TF-IDF, embeddings, keyword flags.
- **Geo:** city tier, pin-code clusters, distance between points (haversine).
- **Interactions:** price x quantity; domain-based crosses.

- **Lag/rolling features** for time series (with strict time-awareness).
- **Target leakage check** for each engineered feature: "Would I have this at prediction time?"

8.3 Feature selection

- **Filter:** correlation, chi-square, mutual information, variance threshold, ANOVA F, IV (credit).
- **Wrapper:** RFE (recursive feature elimination), forward/backward selection.
- **Embedded:** L1 regularization, tree importances.
- **Dimensionality reduction:** PCA (unsupervised, loses interpretability).
- Watch: multicollinearity, stability across folds, permutation importance on validation.

SAY IT IN 60 SECONDS

"Feature engineering means creating inputs that make the pattern easier to learn. Examples are dates split into day of week and month, ratios like spend per visit, aggregates like a user's average order in the last thirty days, interaction terms, log transforms for skewed values and bins for thresholds. Domain knowledge is the best source. Feature selection then removes noise: filter methods score features by correlation or mutual information, wrapper methods like recursive elimination search subsets, and embedded methods like lasso or tree importance select during training. The catch: any aggregate must be computed using only data available before the prediction time, or it leaks the future."

KEY TERMS datetime features · cyclical encoding · aggregates · RFM · ratios · interactions · lag features · filter wrapper embedded selection · mutual information

BUILDS ON 8.1 Data preprocessing: missing values, outliers, encoding, scaling

HANDS-ON PRACTICE For a churn problem, list ten features across recency, frequency, monetary, support and product usage, and mark the prediction-time availability of each.

COMMON MISTAKES Creating features using future information · Selecting features on the full dataset before cross-validation · Adding hundreds of features without checking stability

CHECK YOURSELF

1. Predicting churn: suggest three engineered features from an events table.

Days since last activity, number of sessions in the last 7 vs 30 days (trend), and support tickets raised; all computed as of the prediction date.

2. Filter versus embedded feature selection?

Filter ranks features independently of the model (fast, ignores interactions); embedded selection happens inside training, as with L1 or tree importance.

3. Why log-transform a skewed feature like income?

It compresses the long tail so extreme values do not dominate linear models, and makes relationships more linear.

8.3 Dimensionality reduction: PCA, t-SNE, UMAP, LDA

IN PLAIN WORDS

Imagine photographing a long pencil. From the side you see its length; from the end you see a dot and lose almost everything. PCA finds the best angle to photograph your data from so that you keep as much of the "spread" as possible while using fewer dimensions. The first

direction captures the most variation, the second captures the most of what is left (at right angles), and so on. You keep only the first few and throw away the rest, which compresses the data with little loss.

Why it matters: High-dimensional data is slow, noisy and hard to visualise; PCA is the standard first tool.

Takeaway: PCA keeps the directions of maximum variance; standardise first; choose components by explained variance.

WORKED EXAMPLE, STEP BY STEP

How much does PCA compress two correlated features?

Two exam scores, both standardised, with correlation $r = 0.9$.

1. **The picture.** The points lie in a tight cigar shape along the diagonal, because students who score high on one test tend to score high on the other.
2. **PC1** points along the cigar (roughly "overall ability"); **PC2** points across it ("better at test A than test B").
3. **The share of variance on each.** For two standardised features, the components carry $(1 + r)$ and $(1 - r)$ units of variance out of 2. With $r = 0.9$: PC1 has $1.9 / 2 = 95\%$, PC2 has 5%.
4. **Decision:** keep PC1 only. One number now describes 95% of the variation in two.
5. **The other case, $r = 0.2$:** PC1 has $1.2 / 2 = 60\%$, PC2 has 40%. Dropping PC2 would throw away a lot; PCA does not help much when features are not correlated.
6. **Why standardise first.** If one score ran from 0 to 1,000 and the other from 0 to 10, PC1 would simply follow the large-scale feature, because PCA chases raw variance.

What this shows. PCA compresses well exactly when features are redundant (highly correlated). Read the explained-variance ratio before deciding how many components to keep.

YOUR TURN (COVER THE SOLUTION FIRST)

1. A PCA on 10 standardised features reports explained-variance ratios of 0.42, 0.21, 0.12, 0.08, 0.06, 0.04, 0.03, 0.02, 0.01, 0.01. How many components keep at least 90% of the variance, and what do you lose by using them in a model?

Solution. The cumulative shares are 0.42, 0.63, 0.75, 0.83, 0.89, 0.93, and so on, so 6 components reach 93% (5 give 89%). You go from 10 features to 6 and keep most of the information, but each component mixes all 10 original features, so you can no longer say "income matters" from the model. And PCA ignores the target, so the discarded 7% could, in principle, hold the predictive signal.

THE FUNDAMENTALS

- **PCA:** projects data to orthogonal directions of maximum variance (eigenvectors of covariance matrix). Standardize first. Choose components by cumulative explained variance (e.g., 95%) or scree plot. Components are linear combinations, less interpretable.

- **t-SNE / UMAP:** nonlinear, for visualization; do not interpret distances between clusters literally; t-SNE is not for feature inputs.
- **LDA (linear discriminant analysis):** supervised; maximizes class separation.

SAY IT IN 60 SECONDS

"Dimensionality reduction compresses many features into a few. PCA finds new axes, the principal components, along which the data varies most; each is a linear combination of the original features, and they are uncorrelated. I keep enough to explain, say, ninety-five per cent of variance, and I must scale features first. It helps with noise, speed and visualisation, at the cost of interpretability. t-SNE and UMAP are non-linear methods for plotting clusters in two dimensions; distances between clusters in a t-SNE plot are not meaningful, and they are not for feeding models. LDA is supervised and finds axes that best separate classes. The catch: PCA ignores the target, so top components may not be the predictive ones."

KEY TERMS PCA · principal components · explained variance · scree plot · t-SNE · UMAP · LDA · standardization

HANDS-ON PRACTICE Run PCA on a standardised dataset, plot cumulative explained variance, and choose components for 95%.

COMMON MISTAKES Not standardising before PCA · Interpreting distances between t-SNE clusters literally · Using PCA when interpretability is required

CHECK YOURSELF**1. Why standardise before PCA?**

PCA maximises variance, so a feature with a large scale would dominate the first component regardless of importance.

2. Can you read cluster sizes and distances off a t-SNE plot?

No: t-SNE preserves local neighbourhoods but not global distances or sizes; use it for exploration only.

3. PCA versus LDA?

PCA is unsupervised and maximises variance; LDA is supervised and maximises class separation.

8.4 Classification metrics

IN PLAIN WORDS

Suppose a smoke alarm has to spot real fires. **Recall** asks "of all the real fires, how many did it catch?". **Precision** asks "when it rings, how often is there really a fire?". A jumpy alarm catches every fire but cries wolf constantly (high recall, low precision); a sleepy one is almost always right when it rings but misses fires (high precision, low recall). Turning the sensitivity dial moves you along that trade-off. **Accuracy** can look great while the alarm is useless, because fires are rare.

Why it matters: Metric questions are in almost every ML interview, and choosing the wrong one is a classic red flag.

Takeaway: Understand the confusion matrix; choose metrics by business cost; beware accuracy on rare events.

WORKED EXAMPLE, STEP BY STEP**Move the threshold, watch precision and recall**

Ten transactions, sorted by the model's fraud score. F = fraud (positive), L = legitimate. There are 5 frauds.

score	0.95	0.90	0.80	0.70	0.60	0.45	0.40	0.30	0.20	0.10
truth	F	F	L	F	L	F	L	L	F	L

- Threshold 0.5** (flag anything above it): 5 flagged, of which 3 are fraud. TP = 3, FP = 2, FN = 2. Precision = $3/5 = 0.60$; recall = $3/5 = 0.60$.
- Threshold 0.35**: 7 flagged, 4 fraud. TP = 4, FP = 3, FN = 1. Precision = $4/7 = 0.57$; recall = $4/5 = 0.80$.
- Threshold 0.15**: 9 flagged, all 5 frauds among them. TP = 5, FP = 4, FN = 0. Precision = $5/9 = 0.56$; recall = **1.00**.

Threshold	Flagged	Precision	Recall
0.50	5	0.60	0.60
0.35	7	0.57	0.80
0.15	9	0.56	1.00

- The trade-off.** Lowering the threshold catches more fraud (recall rises) at the cost of more false alarms (precision falls).
- The model did not change, only the decision.** Precision and recall belong to a model *and* a threshold.

What this shows. A classifier gives scores; you choose the threshold. Plot precision and recall across thresholds and choose with the business.

YOUR TURN (COVER THE SOLUTION FIRST)

- Using the same ten transactions with a threshold of 0.75, compute TP, FP, FN, precision, recall and accuracy.

Solution. Above 0.75: the scores 0.95 (F), 0.90 (F) and 0.80 (L) are flagged. TP = 2, FP = 1, FN = 3 (the frauds at 0.70, 0.45 and 0.20), TN = 4. Precision = $2/3 \approx 0.67$; recall = $2/5 = 0.40$; accuracy = $(2 + 4) / 10 = 0.60$. A high threshold gives the most trustworthy alerts and misses the most fraud.

THE FUNDAMENTALS**Classification (confusion matrix)**

	Predicted +	Predicted -
Actual +	TP	FN
Actual -	FP	TN

- Accuracy** = $(TP+TN)/N$. Misleading under imbalance.
- Precision** = $TP/(TP+FP)$: "of those flagged, how many are truly positive?"
- Recall / Sensitivity / TPR** = $TP/(TP+FN)$: "of true positives, how many did we catch?"

- **Specificity** = $TN/(TN+FP)$. **FPR** = $1 - \text{specificity}$.
- **F1** = harmonic mean of precision and recall; **F-beta** weights recall more ($\beta > 1$).
- **ROC-AUC**: probability that a random positive ranks above a random negative; threshold-independent; can look optimistic under heavy imbalance.
- **PR-AUC / Average Precision**: better under heavy imbalance.
- **Log loss**: penalizes confident wrong probabilities; measures calibration and discrimination.
- **Brier score, calibration curve**: are predicted probabilities trustworthy?
- **KS statistic, Gini ($2 \cdot \text{AUC} - 1$)**: common in Indian BFSI credit modeling.
- **Lift / gain / decile charts**: marketing and targeting use cases.
- **MCC, Cohen's kappa**: balanced single-number summaries.
- **Multi-class**: macro (equal class weight), micro (global counts), weighted averages.

SAY IT IN 60 SECONDS

"Everything starts from the confusion matrix: true positives, false positives, true negatives, false negatives. Accuracy is the share correct, misleading with imbalance. Precision is of those I flagged, how many were right. Recall is of the real positives, how many I found. F1 is their harmonic mean. For example, with a thousand cases, one hundred fraud, and a model that finds sixty and flags a hundred, precision is sixty per cent and recall sixty per cent. ROC AUC measures ranking quality across thresholds; precision-recall AUC is more informative for rare positives. Log loss scores the probabilities themselves. The catch: each metric hides something, so I choose by the cost of each error type."

KEY TERMS confusion matrix · accuracy · precision · recall · specificity · F1 · F-beta · ROC-AUC · PR-AUC · log loss · calibration · KS · Gini · MCC

BUILDS ON 7.5 Logistic regression

HANDS-ON PRACTICE Compute a confusion matrix and all derived metrics for a fraud model on 1,000 transactions, then move the threshold and see what changes.

COMMON MISTAKES Reporting accuracy on imbalanced data · Using ROC-AUC alone with extreme imbalance · Comparing F1 scores at different thresholds

CHECK YOURSELF

1. **Model flags 100 as fraud; 60 are fraud; there are 120 frauds in total. Precision and recall?**

Precision $60/100 = 60\%$; recall $60/120 = 50\%$.

2. **99% of transactions are legitimate. A model predicting 'legit' for all has what accuracy, precision and recall for fraud?**

99% accuracy, undefined precision, 0% recall: accuracy is misleading on imbalanced data.

3. **Why does ROC AUC look optimistic on rare positives?**

The false positive rate divides by the huge negative class, so many false alarms still give a small rate; precision-recall exposes it.

8.5 Choosing the metric from business cost

IN PLAIN WORDS

A metric is only useful if it measures what the business actually cares about. A spam filter that blocks a real email is worse than one that lets a spam through, so precision matters. A cancer screening that misses a case is worse than one that raises a false alarm, so recall matters. So before you pick "F1" or "AUC", ask: what does each kind of mistake cost, in rupees, hours or trust? Then choose the threshold that makes the expected cost lowest.

Why it matters: "Which metric would you use, and why?" tests whether you think about the decision, not only the model.

Takeaway: Start from error costs and capacity, then pick the metric and the threshold.

WORKED EXAMPLE, STEP BY STEP

Choose the threshold that costs least

Same ten transactions as topic 8.4. A missed fraud costs 500; a false alarm (a blocked genuine payment) costs 5.

1. Count the errors at each threshold:

Threshold	FN	FP	Cost = $500 \times \text{FN} + 5 \times \text{FP}$
0.50	2	2	$1,000 + 10 = \mathbf{1,010}$
0.35	1	3	$500 + 15 = \mathbf{515}$
0.15	0	4	$0 + 20 = \mathbf{20}$

2. Choose the cheapest: 0.15. When misses are 100 times more expensive than false alarms, flag generously.

3. Change the costs, change the answer. If a false alarm cost 200 (an angry customer and a manual review) and a miss cost 100: at 0.50, $200 + 400 = 600$; at 0.35, $100 + 600 = 700$; at 0.15, $0 + 800 = 800$. Now **0.50** wins.

4. Explain it in business terms: "At this threshold we catch every fraud in the sample for the price of four extra reviews."

5. Do it on validation data, then report the chosen threshold's precision and recall on the test set once.

What this shows. The "best" metric and threshold come from what each mistake costs. Put the costs on the table, and the decision becomes arithmetic.

YOUR TURN (COVER THE SOLUTION FIRST)

1. A clinic can only follow up 3 flagged patients a week. A model ranks 10 patients, and the true positives are at ranks 1, 2, 5 and 9. Which metric should you report, and what is its value?

Solution. With capacity for 3, report precision at 3: of the top 3 (ranks 1, 2, 3), two are true positives, so $\text{precision@3} = 2/3 \approx 0.67$. $\text{Recall@3} = 2/4 = 0.5$ tells the clinic that half the positive patients are not followed up at this capacity, which is a useful argument for more resources.

THE FUNDAMENTALS

Problem	Costly error	Emphasize
Fraud detection	Missing fraud (FN)	Recall at acceptable precision; PR-AUC; review capacity

Problem	Costly error	Emphasize
Spam	Blocking a real email (FP)	Precision
Cancer screening	FN	Recall
Marketing campaign	Budget wasted (FP)	Precision/lift at top deciles
Credit default	Both (losses vs lost revenue)	KS, AUC, expected profit
Churn retention offers	FP cost (discount) vs FN (lost customer)	Expected value with threshold tuning

SAY IT IN 60 SECONDS

"I choose the metric from the cost of mistakes. Cancer screening: a missed case is terrible and a false alarm is cheap, so I maximise recall. Email spam filter: blocking a real email is worse than letting some spam through, so I favour precision. When both matter, F1 or a threshold chosen on the precision-recall curve. If errors have dollar costs, I compute expected cost directly: false negatives times cost plus false positives times cost, and pick the threshold that minimises it. For ranking tasks I use AUC or precision at K. The catch: I always say what the metric means for the business, and tune the threshold, not just the model."

KEY TERMS cost-sensitive metrics · expected profit · threshold selection · fraud · churn · credit default · marketing lift

BUILDS ON 8.4 Classification metrics

HANDS-ON PRACTICE For a churn model with a ₹500 retention offer and ₹2,000 lifetime value at risk, compute the expected value at three thresholds.

COMMON MISTAKES Choosing the metric first and the business goal second · Ignoring capacity constraints (review team size) · Using default thresholds

CHECK YOURSELF**1. Missing a fraud costs \$500, a false alarm costs \$5. How do you pick a threshold?**

Compute expected cost across thresholds, $500 \times \text{FN} + 5 \times \text{FP}$, and choose the minimiser; it will lean towards high recall.

2. Which metric for a spam filter, and why?

Precision (or a high-precision operating point): wrongly blocking real mail costs more than letting some spam through.

3. Support can review only 50 flagged cases a day. Metric?

Precision at $K = 50$, since only the top 50 are actioned.

8.6 Regression, ranking and clustering metrics

IN PLAIN WORDS

Different jobs need different yardsticks. For predicting a number (delivery time), ask "on average how many minutes off are we?" (MAE), and whether big misses hurt more (RMSE). For recommending things, what counts is whether the good items appear near the top of the list (precision@K, NDCG). For grouping without labels, ask whether points are closer to their own group than to others (silhouette) and whether the groups make business sense.

Why it matters: Wrong metric, wrong model choice; interviews often ask "how would you evaluate X?".

Takeaway: Match the metric to the output type: numbers, rankings or groups.

WORKED EXAMPLE, STEP BY STEP

MAE, RMSE, MAPE and WAPE on one forecast

Four days of demand:

Day	Actual	Forecast	Error	abs % error
1	100	110	+10	10%
2	120	115	-5	4.2%
3	80	90	+10	12.5%
4	10	20	+10	100%

- MAE** = $(10 + 5 + 10 + 10) / 4 = 8.75$ units: the typical miss, in units.
- RMSE** = $\sqrt{(100 + 25 + 100 + 100) / 4} = \sqrt{81.25} \approx 9.0$. A little higher than MAE because squaring weights bigger misses more.
- MAPE** = $(10\% + 4.2\% + 12.5\% + 100\%) / 4 \approx 31.7\%$. Day 4 dominates: a miss of 10 units on a tiny day counts as 100%.
- WAPE** = total absolute error / total actual = $35 / 310 \approx 11.3\%$, a percentage that is not hijacked by small days.
- Pick the one that matches the pain.** Stock-outs cost in units: MAE. Big misses are disproportionately bad: RMSE. Executives want a percentage: WAPE, not MAPE.

What this shows. Each error metric answers a slightly different question. MAPE misbehaves when actual values are near zero.

YOUR TURN (COVER THE SOLUTION FIRST)

- A search engine returns three results with relevance 1, 0, 1 (1 = relevant). The ideal order would be 1, 1, 0. Compute DCG, the ideal DCG and NDCG, using a discount of $1/\log_2(\text{position} + 1)$.

Solution. $DCG = 1/\log_2 2 + 0/\log_2 3 + 1/\log_2 4 = 1 + 0 + 0.5 = 1.5$. Ideal DCG = $1/\log_2 2 + 1/\log_2 3 + 0 = 1 + 0.631 = 1.631$. $NDCG = 1.5 / 1.631 \approx 0.92$. Putting the second relevant result at position 3 instead of 2 cost about 8%.

THE FUNDAMENTALS

Regression

- MAE:** average absolute error, robust, same units.
- MSE/RMSE:** penalizes large errors, RMSE in original units.
- R²:** variance explained; can be negative on test.
- MAPE / sMAPE / WMAPE:** percentage errors (MAPE breaks near zero actuals).
- Residual analysis always.

Ranking / recommenders

Precision@K, Recall@K, MAP, NDCG, MRR, hit rate, coverage, diversity.

Clustering

Silhouette, Davies-Bouldin, Calinski-Harabasz, and business interpretability.

SAY IT IN 60 SECONDS

"For regression, MAE is the average absolute error, in the target's units and robust to outliers. MSE squares errors so large misses hurt more; RMSE returns to the original units. R squared is the share of variance explained, and can be negative on new data. MAPE is a percentage but breaks near zero. For ranking and recommenders: precision at K, recall at K, MAP and NDCG, which rewards putting relevant items near the top. For clustering without labels: silhouette score, from minus one to one, compares closeness to own cluster against the nearest other. The catch: I match the metric to how errors are felt, like RMSE when big misses are disproportionately bad."

KEY TERMS MAE · RMSE · R-squared · MAPE · WAPE · precision at K · NDCG · MAP · MRR · silhouette

HANDS-ON PRACTICE Compute MAE, RMSE and WAPE for a small forecast table by hand.

COMMON MISTAKES Using MAPE when actuals can be near zero · Reporting R-squared alone · Judging a recommender by accuracy

CHECK YOURSELF

1. Errors of 1, 1, 1, 9. MAE and RMSE?

MAE = 3; RMSE = $\sqrt{((1+1+1+81)/4)} = \sqrt{21} \approx 4.6$. RMSE is higher because it punishes the large miss.

2. Why is MAPE risky?

It explodes when actual values are near zero and is asymmetric between over- and under-forecasting.

3. What does NDCG reward that precision at K does not?

Position and graded relevance: relevant items higher in the list score more.

8.7 Imbalanced data playbook and self-check

IN PLAIN WORDS

When only 1 in 100 cases is the thing you are hunting for, a lazy model that always says "no" is 99% accurate and completely useless. The fix has three parts: measure success with the right yardstick (not accuracy), tell the model that rare examples matter more (class weights) or show it more of them (resampling), and then choose the alert threshold carefully because the usual 50% cut-off no longer fits.

Why it matters: Fraud, churn, defaults and medical problems are all imbalanced; interviewers check you know the playbook.

Takeaway: Right metric, stratified splits, class weights first, resample only inside folds, tune the threshold.

WORKED EXAMPLE, STEP BY STEP**Class weights, by numbers**

1,000 training transactions: 990 legitimate, 10 fraud.

1. **The lazy model** predicts "legitimate" for everything: accuracy $990 / 1,000 = 99\%$, recall for fraud **0%**. Accuracy hides the failure completely.
2. **Why it happens.** Each legitimate row and each fraud row count equally in the loss, so getting the 990 right matters far more to the model than the 10.
3. **Balanced class weights** give each class equal total weight: $\text{weight} = n / (\text{number of classes} \times \text{class count})$. Legitimate: $1,000 / (2 \times 990) \approx 0.505$. Fraud: $1,000 / (2 \times 10) = 50$.
4. **The effect.** Each fraud now counts about 99 times as much as a legitimate row, and the two classes contribute equally to the loss. The model can no longer ignore fraud.
5. **Then measure properly:** precision, recall and PR-AUC, with a threshold chosen on validation data.
6. **If you resample instead** (oversampling or SMOTE), do it *inside* the training folds only, never before splitting.

What this shows. With rare positives, fix the metric first and then the training signal. Class weights are the simplest fix, with no data copied or invented.

YOUR TURN (COVER THE SOLUTION FIRST)

1. A colleague applies SMOTE to the full dataset (creating synthetic fraud cases) and then does a train/validation split. Validation recall jumps from 0.55 to 0.92. Why is that suspicious?

Solution. SMOTE creates synthetic frauds by interpolating between real fraud rows. After the split, many validation frauds have near-twins, made from the same originals, in the training set, so the model recognises them almost by memory. The jump is leakage, not skill. Split first, apply SMOTE only to the training part (for example with an `imblearn` pipeline inside cross-validation), and evaluate on untouched real data.

THE FUNDAMENTALS**8.6 Imbalanced data playbook**

1. Don't use accuracy. Use PR-AUC, recall/precision at threshold, F1, cost-based metrics.
2. Stratified splits and stratified CV.
3. **Class weights** (`class_weight='balanced'` , `scale_pos_weight` in XGBoost), often the first thing to try.
4. **Resampling:** random undersampling, oversampling, **SMOTE/ADASYN** (apply only inside training folds, never before the split).
5. **Threshold tuning** on validation using business costs.
6. **Anomaly-detection framing** for extreme imbalance.
7. **Better features / more data for the minority class.**
8. Calibrate probabilities afterward (Platt, isotonic) if resampling distorts them.

```
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE
pipe = ImbPipeline([('pre', pre), ('smote', SMOTE(random_state=42)), ('clf', clf)])
```

8.7 Self-check (Module 8)

- ☐ Pick a metric and threshold strategy for fraud, churn, and loan default and defend it.
- ☐ Explain why SMOTE must be applied inside cross-validation.
- ☐ Build a leakage-free sklearn Pipeline with ColumnTransformer.

SAY IT IN 60 SECONDS

"With imbalanced data, such as one per cent fraud, a model can get high accuracy by always predicting the majority. My playbook: first change the metric to precision, recall, PR-AUC. Use stratified splits so every fold has positives. Then try class weights, the simplest fix. Resample the training set only: undersample the majority, or oversample the minority, with SMOTE creating synthetic points. Tune the decision threshold. Consider anomaly detection if positives are extremely rare. The catch: resampling must happen after the split and only on training folds, otherwise duplicated rows leak into validation and inflate scores."

KEY TERMS class weights · SMOTE · undersampling · resampling inside cross-validation · threshold tuning · PR-AUC · calibration · anomaly detection

BUILDS ON 8.4 Classification metrics

HANDS-ON PRACTICE Train a logistic regression with and without class weights on 2%-positive data and compare recall, precision and PR-AUC at several thresholds.

COMMON MISTAKES Oversampling before the train-test split · Reporting accuracy · Keeping the 0.5 threshold after rebalancing

CHECK YOURSELF

1. Why must SMOTE be applied only inside the training fold?

Synthetic points derived from rows that also appear in validation leak information and make validation scores unrealistically high.

2. Cheapest first fix for imbalance?

Class weights (or adjusting the threshold) with a proper metric; no data changes needed.

3. After oversampling, predicted probabilities are inflated. How do you fix it?

Choose the threshold on an untouched validation set or recalibrate to the true base rate.

Interview tips, tricks and hacks

- 1. For any metric question, start with the cost of each error.** "A missed fraud costs 500, a false alarm costs 5 of review time, so I would favour recall and choose the threshold that minimises expected cost."
- 2. Never report accuracy alone on imbalanced data.** Mention the trivial baseline ("always predict the majority") to show why.
- 3. Draw the confusion matrix.** In a metrics question, sketch the four cells with the given numbers before computing anything. It prevents mixing up precision and recall.

4. **Keep the plain-language definitions ready.** Precision: "when I raise an alarm, how often am I right?" Recall: "of all real cases, how many did I catch?"
5. **Tune the threshold, not just the model.** Saying "0.5 is only a default" is a mark of maturity.
6. **For missing values, ask why before how.** Then give the simple recipe: median or mode, plus a was-missing flag, fitted on training data.
7. **Match the encoding to the cardinality.** One-hot for a handful of categories; frequency or out-of-fold target encoding for thousands.
8. **Say where every transformation is fitted.** "On the training fold only, inside the pipeline." Repeat it for scaling, imputation, encoding and resampling.
9. **Propose features from the domain.** For churn: recency, frequency, trend in usage, support contacts, all computed as of the prediction date. Interviewers value two thoughtful features over twenty generic ones.
10. **For imbalance, give the order of moves.** Right metric, stratified split, class weights, resampling inside training folds, threshold tuning.

Frequently asked questions

7 questions interviewers ask on this chapter, with model answers. Cover the answer, say yours aloud, then compare.

DATA PREPROCESSING: MISSING VALUES, OUTLIERS, ENCODING, SCALING

Q1 A feature `city` has 5,000 distinct values and you want to use a linear model. What do you do?

One-hot would create 5,000 sparse columns (overfitting, memory). Options: (1) **group rare levels** into "other", or roll cities up into regions or tiers; (2) **out-of-fold smoothed target encoding** (or weight of evidence) so each city becomes one numeric value without leaking the label; (3) **frequency encoding** if popularity is predictive; (4) **hashing** if the vocabulary is unbounded; (5) learned **embeddings** in a neural model; or use **CatBoost**, which handles high-cardinality categoricals natively. Validate that the encoding is computed inside cross-validation.

FEATURE ENGINEERING AND FEATURE SELECTION

Q2 Design features for predicting whether a customer will churn in the next 30 days.

Fix a **snapshot date**; features use only data before it, the label looks 30 days ahead. Features: **recency** (days since last order), **frequency** (orders in the last 7/30/90 days) and its **trend** (last 30 days vs the previous 30), **monetary** (average order value, discount dependence), **service quality** (late deliveries, cancellations, refunds, support tickets and their sentiment), **engagement** (app opens, search-to-order ratio, cart abandonment), tenure, city tier, and competitor/season signals. Validate with a time-based split and check each feature's availability at scoring time.

DIMENSIONALITY REDUCTION: PCA, T-SNE, UMAP, LDA

Q3 After PCA on standardised data, the first component explains 73% of the variance. Does that mean it is the best feature for predicting your target?

Variance is not predictive power. The top component captures the direction with the most spread; a low-variance direction could separate your classes perfectly. Evaluate components with the downstream model (cross-validated), consider supervised reduction (LDA, PLS, feature selection), and do not assume high-variance means high-signal.

CLASSIFICATION METRICS

Q4 Your fraud model has 99.5% accuracy. The fraud rate is 0.5%. Should the team celebrate?

With a 0.5% positive rate, a model that never flags anything scores 99.5% accuracy and catches zero fraud. Report **recall** (fraud caught), **precision** (share of flags that are real), **PR-AUC** and the confusion matrix at the chosen threshold, together with the operational cost (review capacity, loss per missed fraud). Compare with a baseline, and choose the threshold from costs.

CHOOSING THE METRIC FROM BUSINESS COST

Q5 Two fraud models: Model A has higher ROC-AUC; Model B has higher precision at the top 200 alerts per day (the review capacity). Which do you deploy?

The team can only act on about 200 alerts a day, so the relevant quantity is how many true frauds sit in the top 200 (precision@200 or recall@200) and the money saved. Global AUC averages performance over thresholds you will never use. Check that B's advantage is stable (confidence intervals, different time periods) and that it is well-calibrated; then deploy B, while keeping A as a challenger in shadow mode.

REGRESSION, RANKING AND CLUSTERING METRICS

Q6 Your demand forecast for thousands of slow-moving products has a MAPE of 400%, but the business says the forecasts look fine. What is going on?

MAPE divides by the actual, so products with actuals near zero produce huge percentage errors that dominate the average, even if the absolute errors are tiny. Use **WAPE** (total absolute error over total actuals), **MAE** and bias, evaluate at a meaningful aggregation (category, store), and weight by value. Also compare with a naive baseline to see the actual skill.

IMBALANCED DATA PLAYBOOK AND SELF-CHECK

Q7 After using SMOTE the model's predicted probabilities average 0.4 while the real fraud rate is 1%. What do you do?

Resampling changes the class prior the model sees, so predicted probabilities no longer match real-world frequencies. If only the ranking matters (top-K review queues) this is fine. If you need probabilities (expected-loss calculations), **recalibrate** on a validation set drawn from the real distribution using Platt scaling or isotonic regression, or apply the prior correction $\text{odds_real} = \text{odds_model} \times (\pi_{\text{real}} / (1 - \pi_{\text{real}})) / (\pi_{\text{train}} / (1 - \pi_{\text{train}}))$.

MISSING VALUES

- First ask **why**: MCAR (random), MAR (depends on other variables), MNAR (depends on the missing value itself, e.g. high earners skip income).

Option	When
drop rows / columns	few missing, MCAR; column > ~60% empty
median / mode impute	simple, robust baseline
group-wise impute	missingness differs by segment
KNN / iterative (MICE)	MAR with strong correlates
add a <code>was_missing</code> flag	missingness itself is signal
tree models (XGB/LightGBM)	handle NaN natively

- Fit imputers on **training data only** (inside the pipeline).

OUTLIERS

- Detect: IQR fences (Q1 – 1.5·IQR, Q3 + 1.5·IQR), z-score > 3 (inflated by the outlier itself), robust z with median and MAD, Isolation Forest.
- Decide: error (fix/drop), real but extreme (keep). Treat: winsorise (cap at 1st/99th percentile), log-transform, robust scaler, robust models (trees, Huber).
- Never delete points only because they hurt the model.

ENCODING CATEGORICALS

Method	Use	Caution
one-hot	few levels, linear/NN	wide with many levels; drop one for linear
ordinal	natural order (S < M < L)	do not order unordered labels
frequency / count	high cardinality	collisions of equal counts
target / mean encoding	high cardinality	leaks: compute out-of-fold, smooth
hashing	huge, streaming	collisions
embeddings	very high cardinality, NN	needs data
native (CatBoost, LightGBM)	boosting	built-in handling

- Unseen categories at serving: `handle_unknown="ignore"`, an "other" bucket.

SCALING AND TRANSFORMS

Scaler	Formula	Use
StandardScaler	$(x - \text{mean})/\text{SD}$	linear/logistic, SVM, KNN, PCA, NN
MinMaxScaler	$(x - \text{min})/(\text{max} - \text{min})$	bounded inputs, NN, images
RobustScaler	$(x - \text{median})/\text{IQR}$	outliers

- Trees, forests and boosting **do not need scaling**. Fit the scaler on train only.
- Log / log1p** for right-skewed positives (income, counts); Box-Cox / Yeo-Johnson for general skew; binning for non-linear thresholds (KBinsDiscretizer).

FEATURE IDEAS

- Dates**: hour, day of week, month, `is_weekend`, holiday, days since event; cyclic encoding ($\sin/\cos$ of $2\pi \cdot h/24$).
- Aggregates**: user mean/sum/count over 7/30/90 days, recency-frequency-monetary (RFM), ratios (spend per visit), trends (7-day vs 30-day).
- Interactions** (x1·x2), polynomials, domain features (BMI, distance), text length, TF-IDF, embeddings, geo distance.
- Window features** must use data **before** the prediction time only.
- Selection: **filter** (correlation, mutual information, chi-square, variance threshold), **wrapper** (recursive feature elimination), **embedded** (L1, tree importance, permutation importance). Remove constant, duplicate and leaky features; check stability over time.

FEATURES BY DATA TYPE

- Text**: length, counts, TF-IDF n-grams, embeddings; lower-case, strip, handle rare words.
- Time series**: lags, rolling mean/std, differences, expanding stats, calendar and holiday flags.
- Geo**: distance to points, clusters, region; **images**: pretrained embeddings.
- Counts/rates**: per-user averages, ratios with smoothing (add a prior) for small denominators.

- Target (mean) encoding recipe**: out-of-fold category means, smoothed: $(n \cdot \text{cat_mean} + m \cdot \text{global_mean})/(n + m)$.

DIMENSIONALITY REDUCTION

- PCA**: centre and **scale**, take eigenvectors of the covariance matrix; PC1 has maximal variance; components are orthogonal linear mixes. Keep enough for 90-95% explained variance (`explained_variance_ratio_`, scree plot). Unsupervised; loses interpretability; helps noise, speed, multicollinearity.
- t-SNE / UMAP**: non-linear embeddings for 2-D visualisation of clusters; distances between clusters and cluster sizes are **not** meaningful; not for feeding models (UMAP is better for that).
- LDA**: supervised, finds axes that best separate classes (at most classes – 1). **Autoencoders**: non-linear compression. **SVD**: matrix factorisation behind PCA and recommenders.

```
from sklearn.compose import
ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import (
    OneHotEncoder, StandardScaler)
from sklearn.pipeline import
make_pipeline
pre = ColumnTransformer([
    ("num", make_pipeline(
        SimpleImputer(strategy="median"),
        StandardScaler()), num_cols),
    ("cat", OneHotEncoder(
        handle_unknown="ignore"), cat_cols)])
```

CONFUSION MATRIX AND CLASSIFICATION METRICS

	Predicted +	Predicted -
Actual +	TP	FN
Actual -	FP	TN

Metric	Formula	Question answered
Accuracy	$(TP + TN) / \text{all}$	share correct (misleads when imbalanced)
Precision	$TP / (TP + FP)$	of flagged, how many are real
Recall (TPR, sensitivity)	$TP / (TP + FN)$	of real, how many found
Specificity	$TN / (TN + FP)$	of negatives, how many cleared
FPR	$FP / (FP + TN)$	false alarm rate
F1	$2PR / (P + R)$	harmonic mean; F β weights recall $\beta \times$ more
Balanced accuracy	$(TPR + TNR) / 2$	imbalance-aware accuracy
MCC	all four cells	single robust score

- Example: 100 flagged, 60 real, 120 real in total: precision 60%, recall 50%, F1 0.545.
- A model that always says "no fraud" on 99:1 data: accuracy 99%, recall 0%.
- Raise the threshold $\rightarrow$ precision up, recall down; lower it $\rightarrow$ the reverse.

WORKED EXAMPLE

1,000 cases, 50 fraud. Model flags 80: 40 are fraud (TP = 40), so FP = 40, FN = 10, TN = 910.

- Accuracy = $950 / 1000 = 95\%$. Precision = $40 / 80 = 50\%$. Recall = $40 / 50 = 80\%$. F1 = 0.615. FPR = $40 / 950 = 4.2\%$.
- A "never fraud" model: accuracy 95%, recall 0%. Same accuracy, useless.
- Cost view: FN costs \$500, FP costs \$5: cost = $10 \cdot 500 + 40 \cdot 5 = \$5,200$. Lowering the threshold trades FNs for cheap FPs until cost stops falling.

CURVES AND PROBABILITIES

- **ROC-AUC:** P(random positive ranks above random negative); 0.5 = chance, 1 = perfect. Threshold-free, ranking quality; looks optimistic with rare positives.
- **PR-AUC / average precision:** focuses on the positive class; baseline = prevalence. Prefer for imbalance.

- **Log loss** (cross-entropy) punishes confident wrong probabilities; **Brier score** = mean squared probability error. **Calibration:** of cases scored 0.7, ~70% are positive.
- **KS statistic** (max gap between cumulative good/bad), **Gini** = $2 \cdot \text{AUC} - 1$ (credit scoring), **lift/gain** at top decile.

CHOOSING THE METRIC

Situation	Metric
Missing a case is costly (cancer, fraud)	recall at acceptable precision
False alarms are costly (spam block, ad fatigue)	precision
Both matter	F1 or the PR curve
Known costs	expected cost = $c_{\text{FN}} \cdot \text{FN} + c_{\text{FP}} \cdot \text{FP}$, pick the threshold that minimises it
Limited review capacity (50 cases/day)	precision@K
Balanced, symmetric costs	accuracy / ROC-AUC
Probabilities are used (pricing, risk)	log loss, calibration

- Say what the metric means for the business, then **tune the threshold**, not only the model.

REGRESSION, RANKING, CLUSTERING

- **MAE** mean |error| (robust, in units). **MSE** mean squared (punishes big misses). **RMSE** $\sqrt{\text{MSE}}$ (units). Errors 1, 1, 1, 9: MAE 3, RMSE 4.6.
- **R²** = $1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$ (can be negative out of sample). **MAPE** = mean |error|/|actual| (explodes near 0). **SMAPE**, **RMSLE** (relative errors).
- **Ranking:** precision@K, recall@K, MRR (1/rank of first hit), MAP, **NDCG** (graded relevance, discounts by log2 position).
- **Clustering:** silhouette (-1 to 1: closeness to own vs nearest cluster), Davies-Bouldin (lower better), inertia/elbow; with labels: adjusted Rand index, NMI. Judge usefulness by actionability.

IMBALANCED DATA PLAYBOOK

1. **Metrics first:** precision, recall, PR-AUC; never plain accuracy.
2. **Stratified** splits so every fold has positives.
3. **Class weights** (`class_weight="balanced"`, `scale_pos_weight`): the simplest fix.

4. **Resample training folds only:** undersample the majority, oversample the minority, **SMOTE** (synthetic neighbours), ADASYN.
5. **Tune the threshold** on a validation set; recalculate probabilities if resampled.
6. Try anomaly detection when positives are extremely rare; collect more positives; cost-sensitive learning.

TRAP Resampling before the split duplicates rows into validation and inflates scores. Do it inside the training fold (`imblearn.Pipeline`).

```
from sklearn.metrics import (
    precision_recall_curve,
    classification_report,
    average_precision_score)
p, r, t = precision_recall_curve(y,
    score)
print(classification_report(y, pred))
```

THRESHOLD AND CALIBRATION RECIPE

1. Get scores on a **validation** set (never the training set).
2. Compute precision/recall (or cost) for each threshold; plot the PR curve.
3. Choose the threshold that meets the business constraint (recall $\geq 80\%$, review capacity, minimum expected cost).
4. Fix it, then report the test-set metrics once at that threshold.
5. Re-check calibration after resampling or drift; recalibrate (Platt, isotonic).

METRIC TRAPS

TRAP Optimising AUC when only the top 1% is actioned; use precision@K. Comparing models by different thresholds or on different data. Reporting accuracy for imbalanced classes, R² as accuracy, or RMSE without a baseline (always compare with predicting the mean/last value). Tuning on the test set; macro vs micro vs weighted averaging in multiclass (macro treats classes equally).

SAY IT IN 20 SECONDS

- "Accuracy lies on imbalanced data; use precision, recall and PR-AUC."
- "Pick the threshold from the cost of each kind of error."
- "ROC-AUC is about ranking, not about calibrated probabilities."
- "Scale, impute and encode inside the pipeline, fit on train only."

9

CHAPTER 9 · MEDIUM-HIGH PRIORITY · MODULE 9 IN THE APP

Time Series, Clustering, Recommenders, Anomaly Detection

Four families that appear in case rounds and on the job: forecasting over time, grouping without labels, recommending items, and finding the unusual.

4 topics · about 2 h 5 min in the app · 8 questions answered · two-page cheat sheet at the end

9.1 Time series forecasting

9.3 Recommender systems

9.2 Clustering

9.4 Anomaly detection

Foundations

The models in Chapter 7 assume rows are independent and each has a known label. Much real data breaks one of those assumptions. Sales arrive in time order. Customers come with no labels saying which segment they belong to. A recommender has millions of items and mostly missing feedback. Fraud is so rare that you may have almost no examples. Each situation has its own family of methods, and for interviews you need the framing of each, its standard methods, how it is evaluated, and its characteristic trap.

Time series: order is information

In a time series, yesterday helps predict today. Start by seeing its parts.

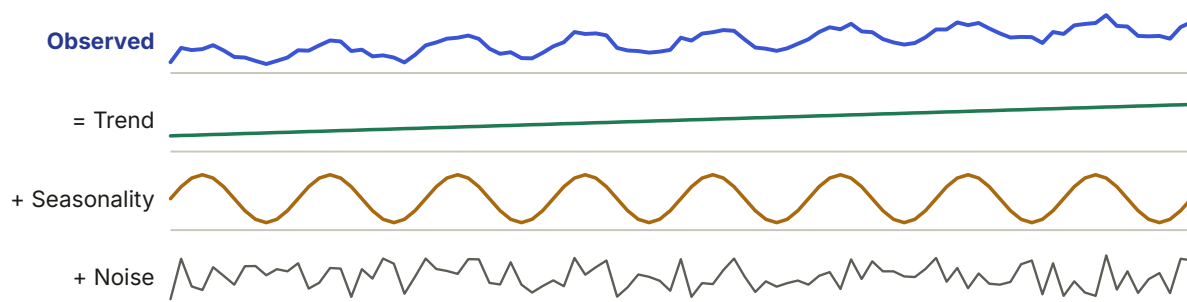


Figure 11. A series is usually a slow trend, a repeating seasonal pattern and irregular noise added together.

The method matters less than three disciplines. First, **beat a naive baseline**: "tomorrow equals today" or "next Monday equals last Monday" is surprisingly strong, and a model that cannot beat it is worthless. Second, **never shuffle**: train on the past and test on the future, moving the cut-off forward several times for a reliable estimate. A random split lets the model peek at the future. Third, **build features only from the past**: lagged values, rolling averages, and calendar

information such as weekday and holidays. Classical models such as ARIMA describe a series through its own past values and past errors; modern practice often feeds lag features into gradient boosting.

Clustering: finding groups nobody labelled

Clustering groups similar items when there is no target to predict, for example to segment customers. **K-means** is the standard: choose k centres, assign each point to its nearest centre, move each centre to the mean of its points, and repeat until stable. It is fast and easy to explain, but it assumes roughly round clusters of similar size, needs you to choose k , and depends on distances, so features must be scaled. **DBSCAN** groups dense regions and marks sparse points as noise, so it handles irregular shapes and finds outliers. **Hierarchical clustering** builds a tree of merges that you can cut at any level. Because there is no correct answer to compare with, a clustering is judged by whether the groups are distinct, stable and above all useful: can the business act differently towards each?

Recommenders: predicting preference

A recommender predicts what a user will like. **Content-based** methods recommend items similar to those the user already liked, using item descriptions. **Collaborative filtering** uses behaviour: people who agreed with you in the past are a guide to your future taste. Its modern form, matrix factorisation, learns a short list of numbers for each user and each item such that their match predicts preference. Real systems work in two stages: a fast step retrieves a few hundred candidates from millions, then a careful model ranks them. The characteristic difficulty is the **cold start**: a new user or new item has no history, so you fall back on popularity and content. Evaluation uses ranking metrics offline and A/B tests online.

Anomaly detection: finding the rare and strange

When the events you care about are extremely rare and often unlabelled, you model what is normal and flag what departs from it. Simple rules flag values far from the centre. A subtlety worth knowing: the common rule based on mean and standard deviation can miss an obvious outlier, because the outlier itself inflates both; rules based on the median and quartiles are more robust. Among learned methods, an **isolation forest** uses the fact that unusual points are easy to separate from the rest with a few random splits. With almost no labels, evaluation means having experts review the top flagged cases, and the practical challenge is keeping false alarms low enough that people keep reading the alerts.

Topic by topic

Each topic gives the idea in plain words, the fundamentals to know, a model spoken answer and three questions to test yourself. Open the same topic in the app for worked examples, runnable code and interactive demos.

9.1 Time series forecasting

IN PLAIN WORDS

Forecasting is predicting tomorrow from a diary of yesterdays. The diary has a slow drift up or down (trend), a repeating rhythm like busier weekends or festival spikes (seasonality), and random wobble (noise). The simplest forecast, "next week will look like this week", is surprisingly hard to beat, so every model must prove itself against it. Most important rule: never peek into the future when training. That means you test on the most recent period, not on random days scattered through the diary.

Why it matters: Demand, ETA, electricity load and UPI volume forecasting are common Indian use cases, and time-awareness is a favourite interview check.

Takeaway: Decompose, make the series stationary if needed, beat the seasonal naive baseline, validate forward in time.

WORKED EXAMPLE, STEP BY STEP

Two baselines every forecast must beat

Daily orders show a weekly pattern. Last week, Monday to Sunday: 100, 90, 95, 105, 130, 160, 150. This week's actuals: 104, 92, 99, 108, 135, 165, 148.

- 1. Naive forecast:** tomorrow equals today. Monday's forecast is last Sunday's 150; Tuesday's is Monday's 104, and so on.
- 2. Seasonal naive forecast:** each day equals the same day last week: Monday 100, Tuesday 90, and so on.
- 3. Errors (mean absolute error over the week):**

Day	Actual	Naive	Seasonal naive
Mon	104	150	100
Tue	92	104	90
Wed	99	92	95
Thu	108	99	105
Fri	135	108	130
Sat	165	135	160
Sun	148	165	150
MAE		21.1	3.6

- 4. Read it.** The naive forecast is poor because it ignores the weekly rhythm: it always forecasts the weekend from Friday. The seasonal naive forecast is already within 4 orders a day.

5. The bar for any model: an MAE well below 3.6 on several future weeks, not just one.

What this shows. Before building a model, compute the dumb forecasts. If the series is seasonal, "same day last week" is a surprisingly strong opponent.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Your gradient-boosting forecast has an MAE of 4.5 on the same week, against 3.6 for seasonal naive. The scaled error (MASE) is your MAE divided by the naive benchmark's MAE. What is it, and what do you conclude?

Solution. $MASE = 4.5 / 3.6 \approx 1.25$. A value above 1 means the model is worse than the simple benchmark, so it is not ready, however sophisticated it is. Check the features (does it know the weekday and holidays?), the validation set-up and possible leakage, and compare over several weeks, not one.

THE FUNDAMENTALS

Concepts

- Components: trend, seasonality, cyclic, residual/noise. Additive vs multiplicative.
- **Stationarity:** constant mean, variance, autocovariance. Test with **ADF** (H0: non-stationary) and **KPSS** (H0: stationary). Achieve via differencing, log transform, detrending.
- **ACF/PACF:** identify AR (PACF cuts off) and MA (ACF cuts off) orders.
- **Models:** naive/seasonal naive baselines, moving average, exponential smoothing (Holt-Winters), **ARIMA(p,d,q) / SARIMA**, **Prophet** (trend + seasonality + holidays), ML with lag features (LightGBM), deep models (LSTM, N-BEATS, TFT) for big data.
- **Validation:** never random split. Use **rolling-origin / expanding-window (TimeSeriesSplit)**. Forecast horizon should match production use.
- **Metrics:** MAE, RMSE, MAPE/sMAPE, MASE, WAPE; compare vs naive baseline.
- **Feature engineering for ML-based forecasting:** lags, rolling mean/std, day of week, holidays/festivals (Diwali, IPL, sale events), price, promotions, weather, hierarchy (store, SKU).
- **Common Indian use cases:** demand forecasting for quick commerce/retail, ETA prediction, ride demand, electricity load, UPI transaction volumes, inventory planning.

```
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.statespace.sarimax import SARIMAX
from sklearn.model_selection import TimeSeriesSplit

adf_p = adfuller(series.dropna())[1]          # p < 0.05 → stationary
model = SARIMAX(series, order=(1,1,1), seasonal_order=(1,1,1,7)).fit(dispatch=False)
forecast = model.forecast(14)

# Lag features for ML
df['lag_1'] = df['y'].shift(1)
df['lag_7'] = df['y'].shift(7)
df['roll_7_mean'] = df['y'].shift(1).rolling(7).mean()  # shift(1) prevents leakage
```

SAY IT IN 60 SECONDS

"Time series data is ordered in time, so order matters and random splits are wrong. I look at the trend, seasonality, and noise. Stationarity, a stable mean and variance, matters for classical models; I difference to remove trend. ARIMA models the series by its own past values and past errors; SARIMA adds seasonality. Modern approaches build lag features, rolling means and calendar features, and use gradient boosting; Prophet handles holidays well. Baselines come first: the last value, or the same day last week. Validation is forward-chaining. The catch: leakage through future information, and judging models by one split rather than several time windows."

KEY TERMS trend · seasonality · stationarity · ADF test · ACF and PACF · ARIMA · SARIMA · Prophet · rolling-origin validation · seasonal naive baseline · lag features

BUILDS ON 7.3 Data leakage: classic examples

HANDS-ON PRACTICE Build naive, seasonal naive and SARIMA forecasts on a synthetic weekly-seasonal series and compare MAE.

COMMON MISTAKES Random train-test split on time series · Lag or rolling features that include the current value (leakage) · Not comparing against a seasonal naive baseline

CHECK YOURSELF**1. What baseline should any forecast beat?**

Naive forecasts: yesterday's value, or the value from the same weekday last week (seasonal naive).

2. How do you make a trending series stationary?

Difference it (subtract the previous value), possibly after a log transform to stabilise variance; test with ADF.

3. Why use rolling-origin cross-validation?

Several train-past, test-future windows give a more reliable error estimate and respect time order.

9.2 Clustering

IN PLAIN WORDS

Clustering sorts things into groups without being told the groups in advance, like sorting a pile of mixed buttons by similarity. K-means drops k flags and keeps moving them until each flag sits at the centre of the buttons nearest to it. DBSCAN instead finds crowded neighbourhoods and treats lonely buttons as noise, so it can find odd shapes. The hard parts are deciding how many groups there are and checking that they are useful, not just mathematically neat.

Why it matters: Customer segmentation (RFM), store clustering and content grouping are everyday tasks.

Takeaway: Scale first, pick k by several signals plus business sense, and always profile the clusters.

WORKED EXAMPLE, STEP BY STEP**Inertia and silhouette by hand**

Five customers on one feature (orders per month): 1, 2, 3, 8, 9.

1. Inertia (the sum of squared distances to each point's cluster centre) for different k:

k	Clusters	Inertia
1	{1, 2, 3, 8, 9}	53.2
2	{1, 2, 3}, {8, 9}	2.5
3	{1, 2, 3}, {8}, {9}	2.0

2. The elbow. Going from 1 to 2 clusters removes almost all the inertia (53.2 to 2.5); going to 3 barely helps. Choose $k = 2$.

3. Silhouette of the point 3: a = its mean distance to its own cluster mates (1 and 2) = $(2 + 1) / 2 = 1.5$. b = its mean distance to the nearest other cluster (8 and 9) = $(5 + 6) / 2 = 5.5$. $s = (b - a) / \max(a, b) = 4 / 5.5 \approx \mathbf{0.73}$.

4. Silhouette of the point 1: $a = 1.5$, $b = 7.5$, $s = 0.80$. It sits more firmly inside its cluster than 3 does.

5. Read the scale. Close to 1: clearly in the right cluster. Close to 0: on the border.
Negative: probably in the wrong cluster.

What this shows. Inertia always falls as k grows, so look for the elbow. Silhouette checks each point's fit and penalises clusters that are close together.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Compute the silhouette of the point 8 in the same two-cluster solution ($\{1, 2, 3\}$ and $\{8, 9\}$).

Solution. a = distance to its own cluster mate 9 = 1. b = mean distance to 1, 2 and 3 = $(7 + 6 + 5) / 3 = 6$. $s = (6 - 1) / 6 \approx 0.83$. Point 8 is clearly in the right cluster.

THE FUNDAMENTALS

Algorithm	Idea	Pros	Cons
K-Means	Assign to nearest centroid; update centroids; repeat	Fast, simple	Needs k , assumes spherical equal-size clusters, sensitive to scale/outliers/init (use k-means++)
Hierarchical (agglomerative)	Merge closest clusters; dendrogram	No k upfront, structure view	$O(n^2)$ memory/time
DBSCAN	Density-based: core points within <code>eps</code> and <code>min_samples</code>	Finds arbitrary shapes, detects noise	Struggles with varying density; eps tuning
GMM	Soft clustering with Gaussians via EM	Soft assignments, elliptical clusters	Needs k , sensitive to init

- **Choosing k :** elbow, silhouette, gap statistic, business usability.
- **Always scale features;** consider PCA before clustering in high dimensions.
- **Validate:** silhouette, stability across runs, profiling clusters on business variables.
- **Use cases:** customer segmentation (RFM), store clustering, content grouping.

SAY IT IN 60 SECONDS

"Clustering groups similar points without labels. K-means picks k centres, assigns each point to the nearest, moves each centre to its group's mean, and repeats. It is fast but assumes round, similar-sized clusters, needs k up front, and is sensitive to scale and starting points. I choose k with the elbow of inertia or the silhouette score. Hierarchical clustering builds a tree you can cut at any level. DBSCAN groups dense regions, finds odd shapes and labels noise, but struggles with varying densities. The catch: there is no ground truth, so the cluster usefulness is judged by interpretability and business action, not only a score."

KEY TERMS k-means · k-means++ · hierarchical clustering · DBSCAN · Gaussian mixture · elbow · silhouette · RFM segmentation · scaling

BUILDS ON 8.1 Data preprocessing: missing values, outliers, encoding, scaling

HANDS-ON PRACTICE Cluster customers on RFM features, choose k with silhouette, and write a one-line persona for each cluster.

COMMON MISTAKES Not scaling features · Choosing k only by the elbow · Treating clusters as ground truth instead of a useful segmentation

CHECK YOURSELF**1. Customers with very different shapes of groups; K-means gives odd results. Alternative?**

DBSCAN or a Gaussian mixture model, which handle non-spherical, differently sized clusters.

2. How do you pick k for K-means?

Elbow in the inertia curve, silhouette score, plus whether the segments are actionable and stable.

3. Why scale features before clustering?

Distances drive the grouping; a large-range feature such as income would dominate over age.

9.3 Recommender systems

IN PLAIN WORDS

A recommender guesses what you will like. The easiest guess: "most people like the popular things" (a surprisingly tough baseline). Better: "people who liked what you liked also liked this" (collaborative filtering) or "this item is similar to things you liked" (content-based). Matrix factorisation learns a short secret profile for every user and item so that matching profiles predict a high rating. Large companies use two stages: a quick first pass picks a few hundred candidates, then a richer model ranks them.

Why it matters: Flipkart, Swiggy and Netflix-style problems appear in case rounds and projects.

Takeaway: Start with popularity, then collaborative filtering or factorisation, add features, evaluate with ranking metrics and A/B tests.

WORKED EXAMPLE, STEP BY STEP**A recommendation as a dot product**

Matrix factorisation describes each user and each dish with a few learned numbers, here two: "spicy" and "sweet".

	spicy	sweet
User Arjun	0.9	0.1
User Kavya	0.2	0.8
Biryani	1.0	0.1
Gulab jamun	0.1	1.0

1. Predicted liking = the dot product of the user and dish vectors.

2. Arjun and biryani: $0.9 \times 1.0 + 0.1 \times 0.1 = \mathbf{0.91}$. Arjun and gulab jamun: $0.9 \times 0.1 + 0.1 \times 1.0 = \mathbf{0.19}$. Recommend biryani.

3. Kavya: biryani $0.2 \times 1.0 + 0.8 \times 0.1 = 0.28$; gulab jamun $0.2 \times 0.1 + 0.8 \times 1.0 = \mathbf{0.82}$. Recommend dessert.

4. Where the numbers come from. Nobody types them in. The model learns them so that the dot products match the ratings or orders we have already seen; the two dimensions only resemble "spicy" and "sweet" here to make the idea readable.

5. Cold start. A brand-new user has no learned vector. Start with popular dishes, or with a vector built from their first few clicks or from onboarding answers, and switch to their learned vector as data arrives.

What this shows. Collaborative filtering compresses everyone's history into short vectors, and the match between a user and an item is a dot product.

YOUR TURN (COVER THE SOLUTION FIRST)

1. For one user, the model recommends 5 items in order; the user later bought items 2 and 5 from that list, plus one item that was not recommended. Compute precision@5 and recall@5.

Solution. Precision@5 = relevant items in the top 5 / 5 = $2/5 = 0.4$. Recall@5 = relevant items found / all relevant items = $2/3 \approx 0.67$. A ranking metric such as NDCG would also reward putting the bought items higher than positions 2 and 5.

THE FUNDAMENTALS

Approach	How	Notes
Popularity baseline	Recommend top-selling	Strong baseline; always include
Content-based	Item features and user profile similarity (TF-IDF, embeddings, cosine)	No cold-start for items; limited novelty
Collaborative filtering (user-user / item-item)	Similarity from interaction matrix	Sparse, cold start, scaling issues
Matrix factorization (SVD, ALS)	Learn latent user/item vectors; score = dot product	Handles sparsity; implicit feedback via ALS
Hybrid / two-tower / deep	Combine features and embeddings	Industry standard at scale
Learning-to-rank	Retrieval (candidate gen) then ranking (GBM/NN)	How Flipkart/Swiggy-scale systems work

- **Cold start:** new user (popularity, onboarding questions, context/geo), new item (content features).
- **Implicit vs explicit feedback:** clicks/views/purchases vs ratings.

- **Metrics:** offline Precision@K, Recall@K, MAP, NDCG, coverage, diversity; online CTR, conversion, GMV, A/B test.
- **Pitfalls:** popularity bias, feedback loops, leakage via random splits (use time-based splits).

SAY IT IN 60 SECONDS

"A recommender suggests items a user will like. Content-based methods recommend items similar to what the user liked, using item features. Collaborative filtering uses behaviour: users who liked what you liked also liked this item. Matrix factorisation learns a small vector per user and per item so that their dot product predicts the rating. Real systems use two stages: candidate generation to pick hundreds out of millions, then a ranking model. Cold start, new users or items with no history, is handled with popularity or content features. Metrics are precision at K, recall at K and NDCG offline, then A/B tests. The catch: popularity bias, and feedback loops that narrow what users see."

KEY TERMS popularity baseline · content-based · collaborative filtering · matrix factorization · implicit feedback · cold start · two-tower · learning to rank · NDCG

HANDS-ON PRACTICE Build a popularity baseline, an item-item collaborative filter and a matrix factorisation model on a small ratings dataset and compare Precision@K.

COMMON MISTAKES Random train-test splits that leak time · Ignoring popularity bias and feedback loops · Forgetting a popularity baseline

CHECK YOURSELF**1. A new user has no history. What do you recommend?**

Popular or trending items, items by context (location, device), then ask for preferences; switch to personalised as signals arrive.

2. Content-based versus collaborative filtering?

Content-based uses item attributes (no other users needed, narrower); collaborative uses behaviour patterns (discovers surprising items, needs data and has cold start).

3. Why two stages in large recommenders?

Scoring millions of items with a heavy model is too slow; a cheap retrieval step narrows to hundreds for the precise ranker.

9.4 Anomaly detection

IN PLAIN WORDS

An anomaly is something that does not look like the rest. The simplest detectors ask "is this number far from the average?" (z-score). Isolation Forest uses a clever trick: to isolate a weird point you need very few random cuts (it is alone in a far corner), while a normal point in the crowd needs many cuts. In practice you also have to decide how many alerts a human team can actually review, so anomaly detection is as much about capacity as about maths.

Why it matters: Fraud, system monitoring and data quality all need it, usually with very few labels.

Takeaway: Start with simple statistics and rules, add Isolation Forest, and measure by precision within a review budget.

WORKED EXAMPLE, STEP BY STEP**Flag an odd day against its own weekday**

Orders on the last four Mondays: 100, 104, 98, 102. Today, a Monday, there are 80. Is that unusual?

1. **Compare like with like.** Mondays have their own level, so compare today with recent Mondays, not with the whole week.
2. **Robust centre:** the median of 100, 104, 98, 102 is **101**.
3. **Robust spread:** the median absolute deviation. Distances from 101 are 1, 3, 3, 1, with median **2**.
4. **Robust z-score:** $0.6745 \times (80 - 101) / 2 \approx -7.1$. A common rule flags values beyond ± 3.5 , so today is a strong anomaly.
5. **Before alerting, check the cheap explanations:** a holiday, an app outage, a tracking change, a city-level event. Many "anomalies" are data problems.
6. **Tune the alert.** If this rule fires five times a week and four are false, people stop reading it. Measure precision of the alerts over a month and adjust the threshold.

What this shows. Anomaly detection is "compare with the right normal, using robust statistics, and keep false alarms rare".

YOUR TURN (COVER THE SOLUTION FIRST)

1. A fraud team can review 20 cases a day. An isolation forest flags 200 transactions per day. What should you do, and what metric tells you whether the model is good?

Solution. Rank by anomaly score and send only the top 20 a day for review (capacity sets the cut-off), with the threshold set accordingly. Measure precision@20: the share of reviewed cases that turn out to be real fraud. Track it over weeks, and use the confirmed cases to improve the model, for example by moving to a supervised model once enough labels exist.

THE FUNDAMENTALS

- Statistical (z-score, IQR, Grubbs), **Isolation Forest** (anomalies isolated with fewer splits), One-Class SVM, Local Outlier Factor, autoencoders (high reconstruction error), time-series residual methods, rule-based + ML hybrids.
- Evaluate with precision@K, recall at fixed review capacity, PR-AUC; labels are scarce and delayed (chargebacks).

SAY IT IN 60 SECONDS

"Anomaly detection finds rare, unusual points like fraud or sensor faults. Simple statistical rules come first: a z-score flags values more than three standard deviations from the mean, and the IQR rule flags values beyond one and a half interquartile ranges from the quartiles, which is robust to outliers. Isolation Forest isolates points with random splits; anomalies need fewer splits. Autoencoders learn to reconstruct normal data, and high reconstruction error signals oddness. With few labels, I evaluate by precision at K and by review of flagged cases. The catch: define what 'anomalous' means for the business, and expect drift to change what is normal."

KEY TERMS z-score · IQR · Isolation Forest · One-Class SVM · Local Outlier Factor · autoencoder · precision at K · delayed labels

HANDS-ON PRACTICE Inject 10 outliers into 500 normal points, and compare z-score, IQR and Isolation Forest.

COMMON MISTAKES Using accuracy · Ignoring that labels are delayed (chargebacks) · Flagging too many alerts for the review team

CHECK YOURSELF

1. Why can the z-score miss an obvious outlier?

The outlier itself inflates the mean and standard deviation, shrinking its own z-score; use robust statistics like median and MAD or IQR.

2. Why does Isolation Forest isolate anomalies quickly?

Anomalies are few and different, so random splits separate them in fewer steps (shorter path length).

3. Labels are almost nonexistent. How do you evaluate an anomaly detector?

Have experts review the top-K flagged cases (precision at K), inject synthetic anomalies, and track findings and false alarms over time.

Interview tips, tricks and hacks

- 1. Open every forecasting answer with the baseline.** "First I would see how seasonal naive performs, because any model must beat that." It is the single most reliable signal of experience in time series.
- 2. Say "time-based split" immediately.** For anything ordered in time, mention forward-chaining validation before the interviewer has to ask.
- 3. Describe a series by its parts.** Trend, seasonality, holidays or events, noise. It structures both your exploration and your feature list.
- 4. For clustering, justify k two ways.** A statistical check (elbow or silhouette) and a business check (are the segments distinct and actionable?).
- 5. Always scale before clustering or KNN.** State it without being asked; forgetting it is the classic mistake.
- 6. Explain collaborative filtering in one sentence.** "People who behaved like you in the past liked this." Then mention cold start before the interviewer does.
- 7. Describe recommenders as retrieve then rank.** It shows you know how real systems cope with millions of items.
- 8. For anomalies, ask what happens to a flagged case.** If a human reviews it, capacity limits how many you can flag, which sets the metric: precision among the top K.
- 9. Prefer robust rules for outliers.** Median and interquartile range over mean and standard deviation, and say why.
- 10. Name the feedback loop.** Recommenders and fraud models change the data they later learn from. Mentioning it shows product awareness.

Frequently asked questions

8 questions interviewers ask on this chapter, with model answers. Cover the answer, say yours aloud, then compare.

TIME SERIES FORECASTING

Q1 Why can't we shuffle time-series data?

Breaks temporal order and leaks future into past.

Q2 What if the series has multiple seasonalities?

TBATS, Prophet, Fourier terms, ML with calendar features.

Q3 How do you forecast new products with no history (cold start)?

Use category-level/similar-item priors, hierarchical models.

Q4 How do you forecast demand for a newly launched product with no history (cold start)?

Use **analogous items** (products with similar attributes, price and launch patterns) and **category-level forecasts** scaled by expected share; train a global ML model with product attributes (category, brand, price, pack size) as features so new items get predictions from their attributes; use **hierarchical forecasting** to reconcile levels; apply **Bayesian updating** or a fast-adapting smoother as the first observations arrive; and plan with wider uncertainty ranges and conservative inventory while learning.

Q5 Your series has both weekly and yearly seasonality. Which approaches handle multiple seasonalities?

SARIMA handles one seasonal period. For several seasonalities use **Fourier terms** (sine/cosine pairs for each period) in a regression or ARIMA-with-regressors, **Prophet** (additive seasonal components), **TBATS**, or **ML/deep models** with calendar features (day of week, day of year, holiday flags) and lags.

CLUSTERING

Q6 Your k-means segments change every time you rerun the notebook. Is the method broken?

K-means converges to a **local optimum** that depends on the random start. Use `init="k-means++"`, a large `n_init` (for example 20) and a fixed `random_state`; check **stability** by comparing labels across runs (adjusted Rand index) or subsamples. If the segments are genuinely unstable, the data may lack natural cluster structure, so use fewer clusters, a different algorithm or features, or treat the result as a soft segmentation.

RECOMMENDER SYSTEMS

Q7 How would you build "restaurants you might like" for a food-delivery app?

Retrieval (hundreds of candidates): restaurants within delivery range, popular and newly opened ones, collaborative filtering/embeddings from past orders, same-cuisine lookalikes. **Ranking**: a gradient-boosted or neural model predicting order probability from user history (cuisine affinity, price band, time of day), restaurant features (rating, prep time, discounts), and context (time, weather, distance, ETA). **Business layers**: diversity across cuisines, exploration for new restaurants, rules for closed or overloaded restaurants. **Evaluation**: offline NDCG@10 with a time split, then an A/B test on conversion, GMV, repeat rate and delivery-time guardrails.

ANOMALY DETECTION

Q8 An Isolation Forest flags 5% of transactions but the fraud team can review only 0.5%. What do you do?

The threshold should come from **capacity and cost**, not a fixed contamination rate: sort by anomaly score and send only the top 0.5% to reviewers (measure precision@K on confirmed cases), automate low-risk handling (step-up authentication for the next tier), and improve the model with reviewer feedback and additional features (velocity, device, geography). Track precision, loss prevented and customer friction.

Time Series, Clustering, Recommenders, Anomaly Detection

Forecasting, segmentation, recommendation and outlier detection: methods, parameters, metrics and the traps in each.

DS Prep
INTERVIEW CHEAT
SHEET

TIME SERIES BASICS

- Components: **trend**, **seasonality** (fixed period), **cycle**, **noise**. Additive: $y = T + S + e$; multiplicative: $y = T \times S \times e$ (seasonal swing grows with level; take logs).
- Stationary**: constant mean, variance and autocovariance. Make stationary by differencing ($y_t - y_{t-1}$), seasonal differencing (lag m), log transform. Test with **ADF** (null: non-stationary) / **KPSS**.
- ACF/PACF** plots: ACF cuts off at $q \rightarrow$ MA(q); PACF cuts off at $p \rightarrow$ AR(p). Autocorrelation at lag 7 = weekly pattern.
- Always beat a baseline**: naive (last value), seasonal naive (same weekday last week), moving average.

SMOOTHING FORMULAS AND WORKFLOW

- Simple exponential smoothing**: $\text{level}_t = \alpha y_t + (1 - \alpha) \text{level}_{t-1}$; α near 1 reacts fast, near 0 is smooth. Holt adds a trend term, Holt-Winters a seasonal term.
- Moving average of window w : lags the series by about $w/2$.
- Workflow: plot $\rightarrow$ decompose (STL) $\rightarrow$ baseline $\rightarrow$ stationarity check $\rightarrow$ model $\rightarrow$ **residuals should look like white noise** (Ljung-Box test, no ACF spikes) $\rightarrow$ forecast with intervals $\rightarrow$ backtest.
- Missing timestamps: reindex to a regular calendar, fill (interpolate for sensors, 0 for counts), flag outliers, mind time zones and daylight saving.
- Many series: global model (one GBM across series with series-id features) often beats one ARIMA per series. Intermittent demand (many zeros): Croston, or count models.

FORECASTING MODELS

Model	Idea	Use
Moving average / exponential smoothing	weighted recent history	short horizon, level only
Holt-Winters (ETS)	level + trend + season	seasonal business series
ARIMA(p, d, q)	AR(p) past values, d differences, MA(q) past errors	single stationary-ish series
SARIMA	ARIMA + seasonal (P, D, Q, m)	seasonality
SARIMAX / regression	add external regressors	promotions, price, weather
Prophet	trend + seasonal + holidays	business series with gaps, holidays
GBM / LightGBM on lag features	lags, rolling stats, calendar	many series, rich features
LSTM / Transformer	sequence nets	long histories, many series

- Feature recipe: lags (1, 7, 28), rolling mean/std, expanding stats, differences, day of week, month, holiday, price/promo. Use only information known at forecast time.

VALIDATION AND METRICS

- Never random-split**. Use a temporal holdout and **rolling-origin / forward-chaining** CV (`TimeSeriesSplit`); gap between train and test if features look ahead.
- Metrics: **MAE**, **RMSE**, **MAPE** (breaks near 0), **sMAPE**, **MASE** (vs naive; < 1 beats naive). Evaluate by horizon; add prediction intervals.

- Pitfalls: leakage from future data (rolling windows computed centred), ignoring holidays/structural breaks, using ARIMA on non-stationary data, forecasting too far ahead, tuning on one split.

```
from statsmodels.tsa.arima.model import ARIMA
m = ARIMA(y, order=(1, 1, 1)).fit()
m.forecast(steps=7)
from statsmodels.tsa.stattools import adfuller
adfuller(y)[1] # p < 0.05: stationary
from sklearn.model_selection import TimeSeriesSplit
```

CLUSTERING

- Unsupervised grouping; **scale features first**; no ground truth, so judge by metrics and usefulness.

Method	Idea and limits
K-means	assign to nearest centre, recompute means, repeat. Fast; needs k , assumes round similar-size clusters, sensitive to scale, outliers and init (use <code>k-means++</code>)
Hierarchical	merge closest groups, cut the dendrogram. No k up front; $O(n^2)$; linkage choice (Ward, single, complete)
DBSCAN	dense regions via <code>eps</code> , <code>min_samples</code> ; noise = -1. Arbitrary shapes, finds outliers; struggles with varying density
GMM	soft assignments with Gaussians (EM). Elliptical clusters, probabilities; needs k

- Choose k** : elbow of inertia (WCSS), **silhouette** score (-1 to 1), gap statistic, stability across seeds, business sense.
- Mixed data: Gower distance / k -prototypes. High dimensions: PCA first. Evaluate with labels (ARI, NMI) only if available.
- Profile clusters (means per feature, sizes), name them, test they are stable and actionable. K-means distance is Euclidean: standardise.

RECOMMENDER SYSTEMS

- **Content-based:** recommend items similar to what the user liked (item features, TF-IDF, embeddings, cosine similarity). No cold start for items; narrow, filter-bubble risk.
- **Collaborative filtering:** use behaviour. **User-user / item-item** neighbourhoods (cosine, Pearson); **matrix factorisation** (ALS, SVD): $R \sim U \text{ times } V\text{-transpose}$, learn k -dim vectors so the dot product predicts the rating; implicit feedback (clicks, views) needs confidence weighting.
- **Hybrid** and **deep** models (two-tower, neural CF, sequence models) combine signals.
- **Two stages at scale:** candidate generation (retrieve hundreds from millions with approximate nearest neighbours) then a **ranking** model (GBM / NN using user, item, context features), then re-ranking (diversity, business rules).
- **Cold start:** new user: popularity, context (location, device), onboarding questions; new item: content features, exploration.
- **Metrics:** offline precision@K, recall@K, MAP, **NDCG@K**, hit rate, coverage, diversity, novelty; RMSE only for explicit ratings. Online: A/B on CTR, conversion, retention, revenue.
- **Pitfalls:** popularity bias, feedback loops, position bias in clicks, offline-online mismatch, sparsity, leakage (use time-based splits), implicit data has no negatives (sample them).

ANOMALY DETECTION

- Define "anomalous" for the business first: point (one odd value), contextual (odd for the time/segment), collective (odd pattern).

Method	Idea	Note
z-score	$\text{abs}(x - \text{mean}) / \text{SD} > 3$	assumes normal; outliers inflate mean and SD
IQR rule	outside Q1 – 1.5 IQR, Q3 + 1.5 IQR	robust, simple
Median/MAD	robust $z = 0.6745 (x - \text{median}) / \text{MAD}$	skewed data
Isolation Forest	random splits isolate outliers in few steps	high dimensions, fast
LOF	density versus neighbours	local anomalies
One-class SVM	boundary around normal data	small data, scale needed
Autoencoder	high reconstruction error = odd	complex data, NN
Time series	residuals from a forecast, seasonal decomposition (STL)	check against the pattern

- Example: orders 98-103 normal, one day 180: mean 108.9, SD 26.7 $\rightarrow z = 2.7$ (missed); IQR fence 108.5 $\rightarrow$ flagged.
- Evaluation with few labels: precision@K by expert review, injected synthetic anomalies, alert fatigue (false alarms per day). Handle drift: what is normal changes; retrain, use rolling baselines.
- Supervised fraud: imbalance playbook (Module 8) + rules + model.

```
from sklearn.ensemble import
IsolationForest
m = IsolationForest(contamination=0.01,

random_state=0).fit(X)
flag = m.predict(X) == -1      #
anomalies
score = -m.score_samples(X)   # higher
= odder
```

SIMILARITY AND ASSOCIATION RULES

Measure	Use
Cosine similarity	direction of vectors (text, ratings); ignores magnitude
Pearson correlation	rating similarity after centring users
Jaccard	overlap of sets (items bought)
Euclidean	dense numeric features (scale first)

- **Market basket / association rules:** support = $P(A \text{ and } B)$; confidence = $P(B | A)$; **lift** = confidence/ $P(B)$ (> 1 means positive association). Apriori / FP-growth. Beware: popular items give high confidence but lift ~ 1 .
- Evaluate recommenders offline with a **time-based split** (train on the past, test on later interactions), then confirm with an A/B test: offline gains often vanish online.
- **Exploration:** bandits (epsilon-greedy, Thompson) to learn about new items; avoid showing only what is already popular.

TIME SERIES ANOMALIES AND MONITORING

- Forecast, then flag residuals beyond ~ 3 SD or beyond the prediction interval. Use STL (seasonal-trend-loess) to remove trend and seasonality, then apply z/IQR rules to the remainder.
- Alerting: tune the threshold for false alarms per day; use rolling windows; compare with the same weekday last week.
- Drift in what is "normal" (new product launch, holiday) causes alert storms: allow manual overrides and retrain.

METHOD CHOOSER

Task	Start with
Forecast a daily metric	seasonal naive $\rightarrow$ Holt-Winters/Prophet $\rightarrow$ GBM with lags
Customer segments	K-means on scaled RFM features; check silhouette
Odd-shaped clusters / noise	DBSCAN
"Customers like you also bought"	item-item CF, then factorisation
New users, little data	popularity + content
Detect fraud with few labels	rules + Isolation Forest, review top-K

SAY IT IN 20 SECONDS

- "Time series: respect order, beat a naive baseline, validate forward in time."
- "K-means needs scaled features and a chosen k ; DBSCAN finds odd shapes and noise."
- "Recommenders retrieve, then rank; cold start needs content and popularity."
- "A z-score can miss an outlier because the outlier inflates the SD."

10

CHAPTER 10 · MEDIUM-HIGH PRIORITY · MODULE 10 IN THE APP

Deep Learning, NLP, GenAI/LLMs

Neural networks, language models and generative AI. Know how they work at a conceptual level, when they are the right tool, and how a retrieval-based application is put together.

7 topics · about 3 h 50 min in the app · 7 questions answered · two-page cheat sheet at the end

10.1 Deep learning fundamentals

10.2 NLP: classical pipeline, TF-IDF and embeddings

10.3 Transformers and attention

10.4 LLM fundamentals and prompting

10.5 RAG end to end

10.6 RAG vs fine-tuning, agents and project ideas

10.7 Computer vision and self-check

Foundations

Deep learning powers the systems that read text, recognise images and generate language. For a beginner-level data science interview you are not expected to derive the mathematics, but you are expected to explain the ideas clearly, to know when deep learning is and is not appropriate, and increasingly to describe how applications built on large language models work.

A neural network is a stack of simple steps

A **neuron** takes its inputs, multiplies each by a weight, adds them up, and passes the result through a simple non-linear function called an **activation**. A **layer** is many neurons side by side, and a network is layers stacked so that each layer's output feeds the next. Early layers learn simple patterns and later layers combine them into complex ones. The non-linear activation is essential: without it, any stack of layers would collapse into a single linear model, no more powerful than Chapter 7's regression.

Training is the loop from Chapter 7 at scale. Make predictions (the forward pass), measure the error with a loss function, work out how much each weight contributed to the error (**backpropagation**, which is the chain rule of calculus applied layer by layer), and adjust every weight slightly to reduce the error. Because networks have enormous capacity, they overfit readily, and the standard defences are dropout, weight penalties, stopping when validation error stops improving, and more data.

Architectures match the shape of the data

Convolutional networks suit images: they slide small pattern detectors across the picture, reusing the same detector everywhere, which is efficient and matches the fact that an edge is an edge wherever it appears. **Transformers** suit sequences such as text, and have largely replaced older recurrent networks. In practice you rarely train these from nothing. **Transfer learning** takes a model already trained on a huge dataset and adapts it to your task with a small amount of your own data.

A point of judgement that interviewers appreciate: for ordinary tables of business data, gradient-boosted trees usually match or beat deep learning with far less effort.

Turning text into numbers

Classical language processing counts words. **TF-IDF** scores a word highly in a document if it is frequent there but rare elsewhere, so distinctive words stand out. It ignores meaning and order, yet combined with a linear model it remains a strong, cheap baseline. **Embeddings** go further by representing each word or sentence as a list of numbers arranged so that similar meanings are close together, which lets a system recognise that "excellent" and "great" are related.

Attention and transformers

The idea behind transformers is **attention**: when processing one word, the model looks at every other word in the passage and decides how much each one matters for understanding this one. In "the animal did not cross the street because it was tired", attention links "it" to "animal". Because every word can look at every other word directly, long-range relationships are captured, and because all positions are processed at once, training is fast on modern hardware.

Large language models

A **large language model** is a very large transformer trained on vast amounts of text to predict the next piece of text, called a token. From that single objective it learns grammar, facts and patterns of reasoning. Three properties matter in practice. It works within a limited **context window** of text it can see at once. Its output is sampled, with a **temperature** setting that trades consistency for variety. And it can **hallucinate**: produce fluent, confident statements that are false, because it is trained to produce plausible text, not verified truth.

RAG: giving the model the right pages

Retrieval-augmented generation is the standard way to make a language model answer from your own documents.

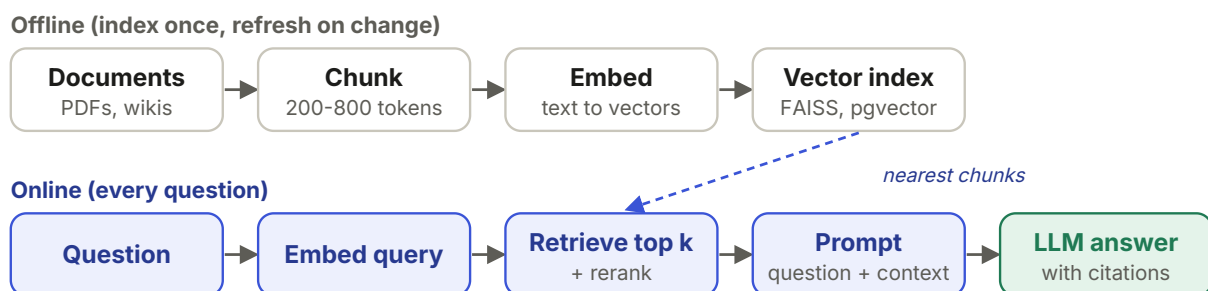


Figure 12. Documents are indexed once. At question time the most relevant passages are retrieved and placed in the prompt.

Documents are split into passages, each passage is converted into an embedding, and the embeddings are stored. When a question arrives, it is embedded the same way, the closest passages are retrieved, and they are given to the model with an instruction to answer only from them and cite its sources. This reduces hallucination and keeps knowledge current without

retraining. The rule of thumb for choosing techniques: use **RAG to add knowledge**, **fine-tuning to change behaviour** such as tone or format, and good prompting before either. An **agent** goes one step further, letting the model plan and call tools such as search or a database in a loop.

Topic by topic

Each topic gives the idea in plain words, the fundamentals to know, a model spoken answer and three questions to test yourself. Open the same topic in the app for worked examples, runnable code and interactive demos.

10.1 Deep learning fundamentals

IN PLAIN WORDS

A neural network is a stack of simple "voting" units. Each unit adds up its inputs with weights (like scoring a job applicant on several criteria), then decides how strongly to fire. Stack many layers and the network builds understanding step by step: edges, then shapes, then faces. Training is trial and error at scale: show an example, see how wrong the answer was, and nudge every weight slightly in the direction that would have made it less wrong. Working out which weights to nudge and by how much, layer by layer backwards, is backpropagation.

Why it matters: Even if you mostly use trees at work, interviews check you understand the basics of how neural nets learn.

Takeaway: Weighted sums plus non-linearities, trained by gradient descent with gradients from backprop.

WORKED EXAMPLE, STEP BY STEP

One neuron learns, step by step

Inputs $x = [1, 2]$, weights $w = [0.5, -0.25]$, bias $b = 0$, sigmoid output. The true label is 1. Learning rate 0.5.

- 1. Forward pass:** $z = 0.5 \times 1 + (-0.25) \times 2 + 0 = 0$. $p = \text{sigmoid}(0) = 0.5$.
- 2. Loss** (log loss for label 1): $-\ln(0.5) \approx 0.693$.
- 3. Backward pass.** For sigmoid with log loss, the error signal at z is simply $p - y = 0.5 - 1 = -0.5$. Each weight's gradient is that signal times its input: $[-0.5 \times 1, -0.5 \times 2] = [-0.5, -1.0]$. The bias gradient is -0.5 .
- 4. Update:** $w \leftarrow w - 0.5 \times \text{gradient} = [0.5 + 0.25, -0.25 + 0.5] = [0.75, 0.25]$; $b \leftarrow 0 + 0.25 = 0.25$.
- 5. Forward again:** $z = 0.75 + 0.5 + 0.25 = 1.5$, $p \approx 0.82$, loss ≈ 0.20 . One step moved the prediction from 0.5 towards the true label.
- 6. A network repeats this** for millions of weights. Backpropagation is the chain rule passing the error signal backwards through each layer, so every weight learns its share of the blame.

What this shows. Training is forward pass, loss, gradient, small step, repeated. The input with the larger value ($x_2 = 2$) received the larger correction.

YOUR TURN (COVER THE SOLUTION FIRST)

1. A two-layer network uses no activation function: hidden = $2x + 1$, output = $3 \times \text{hidden} - 2$. Simplify the output as a function of x . What does this show about why activations are needed?

Solution. Output = $3(2x + 1) - 2 = 6x + 1$, which is just another straight line. Stacking linear layers always collapses to a single linear function, however many layers you add. A non-linear activation such as ReLU, $\max(0, z)$, between the layers is what lets a network bend and represent curved patterns.

THE FUNDAMENTALS

- **Neuron:** output = activation($w \cdot x + b$). **MLP:** stacked layers.
- **Activations:** sigmoid (binary output, vanishing gradient), tanh, **ReLU** (default hidden), Leaky ReLU, GELU, **softmax** (multi-class output).
- **Loss:** MSE (regression), binary cross-entropy, categorical cross-entropy.
- **Backpropagation:** chain rule to compute gradients layer by layer; then optimizer updates weights.
- **Optimizers:** SGD (+momentum), RMSprop, **Adam/AdamW**.
- **Training issues and fixes:**

Problem	Fix
Overfitting	Dropout, weight decay, early stopping, data augmentation, smaller model, more data
Vanishing/exploding gradients	ReLU-family, proper init (He/Xavier), batch/layer norm, residual connections, gradient clipping
Slow/unstable training	Tune learning rate, scheduler, normalize inputs, batch size
Class imbalance	Weighted loss, focal loss, resampling

- **Batch norm vs layer norm:** normalize across batch vs across features (layer norm used in transformers).
- **Architectures (conceptual):**
 - **CNN:** convolutions, pooling; spatial hierarchies; images.
 - **RNN/LSTM/GRU:** sequences; gating handles long dependencies.
 - **Autoencoders:** compression/denoising/anomaly detection.
 - **Transformers:** self-attention, parallelizable, dominate NLP/vision now.
- **Transfer learning:** reuse pre-trained model; freeze early layers, fine-tune later ones.
- **Frameworks:** PyTorch (preferred in industry/research), TensorFlow/Keras.

```
import torch, torch.nn as nn

class MLP(nn.Module):
    def __init__(self, d_in, d_hidden=64):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(d_in, d_hidden), nn.ReLU(), nn.Dropout(0.2),
                                nn.Linear(d_hidden, 1))
    def forward(self, x): return self.net(x).squeeze(-1)

model = MLP(d_in=20)
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.BCEWithLogitsLoss()
for epoch in range(20):
    model.train()
    for xb, yb in train_loader:
        opt.zero_grad()
        loss = loss_fn(model(xb), yb)
        loss.backward()
        opt.step()
```

SAY IT IN 60 SECONDS

"A neural network is layers of simple units. Each unit takes a weighted sum of its inputs plus a bias and passes it through an activation function like ReLU, which adds the non-linearity that lets the network learn curved patterns. The forward pass computes predictions; a loss measures the error; backpropagation uses the chain rule to get the gradient of the loss for every weight; an optimiser such as Adam updates them. Overfitting is controlled with dropout, weight decay, early stopping and more data. Vanishing gradients, where signals shrink through many layers, are eased by ReLU, good initialisation, batch normalisation and residual connections. The catch: for small tabular data, gradient boosting usually beats deep learning."

KEY TERMS neuron · activation functions · backpropagation · loss · Adam · dropout · batch norm · vanishing gradients · CNN · RNN · LSTM · autoencoder · transfer learning · PyTorch

BUILDS ON 7.13 Optimisation: gradient descent and friends

HANDS-ON PRACTICE Train a small MLP on a tabular dataset, plot train and validation loss, and apply early stopping and dropout.

COMMON MISTAKES Forgetting `model.train()` and `model.eval()` · Using softmax before `CrossEntropyLoss` in PyTorch · Training without normalised inputs

CHECK YOURSELF**1. Why do networks need non-linear activations?**

Without them, stacked linear layers collapse into a single linear function and cannot model complex patterns.

2. What does backpropagation compute?

The gradient of the loss with respect to every weight, efficiently reusing intermediate results via the chain rule.

3. Training loss falls, validation loss rises. Three remedies?

Dropout or weight decay, early stopping, and more data or augmentation (or a smaller network).

10.2 NLP: classical pipeline, TF-IDF and embeddings

IN PLAIN WORDS

To let a computer read text, first chop it into pieces (words or sub-words), then turn the pieces into numbers. The oldest trick is counting words. TF-IDF improves it by giving more weight to words that are frequent in this document but rare in the others ("refund" is informative; "the" is not). Modern methods use embeddings: each word or sentence becomes a point in space where similar meanings sit close together, so "cheap" and "affordable" are neighbours even though they share no letters.

Why it matters: Reviews, tickets and resumes are text; a TF-IDF baseline is often surprisingly hard to beat.

Takeaway: Start simple with TF-IDF and a linear model; move to embeddings when meaning matters more than keywords.

WORKED EXAMPLE, STEP BY STEP

Turn sentences into vectors and compare them

Two reviews: A = "good food fast delivery", B = "fast delivery good service".

1. **Vocabulary:** good, food, fast, delivery, service.
2. **Bag-of-words counts:** A = [1, 1, 1, 1, 0]; B = [1, 0, 1, 1, 1].
3. **Dot product:** $1 + 0 + 1 + 1 + 0 = 3$ (the shared words: good, fast, delivery).
4. **Lengths:** $|A| = \sqrt{4} = 2$ and $|B| = \sqrt{4} = 2$.
5. **Cosine similarity** = $3 / (2 \times 2) = 0.75$. Quite similar, which is fair.
6. **Where counting fails.** C = "not good food". Adding "not" to the vocabulary, A = [good 1, food 1, fast 1, delivery 1, not 0] and C = [1, 1, 0, 0, 1]. The dot product is 2, $|C| = \sqrt{3}$, so cosine = $2 / (2 \times 1.73) \approx 0.58$, a fairly high similarity to a positive review, although C is negative.

What this shows. Bag-of-words compares shared words and ignores order and meaning. That is why it is a fine, cheap baseline, and why contextual embeddings that understand "not good" do better.

YOUR TURN (COVER THE SOLUTION FIRST)

1. In 100 reviews, "delivery" appears in 50 and "soggy" in 2. In one review, each word appears once among 10 words. Using $tf = \text{count} / \text{words}$ and $idf = \log_{10}(100 / \text{document count})$, compute TF-IDF for both words. Which is more informative?

Solution. "delivery": $tf = 0.1$, $idf = \log_{10}(100/50) = \log_{10} 2 \approx 0.30$, TF-IDF ≈ 0.03 . "soggy": $tf = 0.1$, $idf = \log_{10}(100/2) = \log_{10} 50 \approx 1.70$, TF-IDF ≈ 0.17 . "soggy" scores about six times higher: it is rare across reviews, so it says much more about this particular one.

THE FUNDAMENTALS

Classical pipeline: lowercase → tokenize → remove stopwords/punctuation → stem (crude chop) or lemmatize (dictionary-based) → vectorize.

Representation	Notes
Bag of Words / CountVectorizer	Word counts; sparse; ignores order
TF-IDF	$tf \times \log(N/df)$; downweights common words; strong baseline with logistic regression/linear SVM
N-grams	Capture short phrases
Word2Vec / GloVe / fastText	Static dense embeddings; fastText handles subwords/typos
Contextual embeddings (BERT etc.)	Word meaning depends on context
Sentence embeddings	For semantic search, clustering (sentence-transformers)

Tasks: sentiment/text classification, NER, topic modeling (LDA, BERTopic), summarization, QA, semantic search, translation, intent detection.

Indian context: code-mixed text (Hinglish), transliteration, multilingual models (IndicBERT, MuRIL, mBERT, XLM-R), and lack of labeled data for Indic languages.

SAY IT IN 60 SECONDS

"Classical NLP turns text into numbers. The pipeline is clean the text, tokenise, optionally remove stop words and stem, then vectorise. Bag of words counts words; TF-IDF weights a word by how often it appears in a document times how rare it is across documents, so common words get low weight and distinctive ones high. These are sparse and ignore order and meaning. Embeddings such as word2vec place words in a dense space where similar meanings are close. Contextual embeddings from transformers give a word a different vector depending on the sentence, so 'bank' in river bank and bank loan differ. The catch: TF-IDF with logistic regression is a strong, cheap baseline, so I start there."

KEY TERMS tokenization · stemming · lemmatization · bag of words · TF-IDF · n-grams · word embeddings · sentence embeddings · code-mixed text · Hinglish

HANDS-ON PRACTICE Build a TF-IDF + logistic regression sentiment classifier and compare it with a sentence-embedding version.

COMMON MISTAKES Removing stopwords when they matter (for example in sentiment with negation) · Fitting the vectoriser on all data before splitting · Using a heavy model without a TF-IDF baseline

CHECK YOURSELF

1. Why does TF-IDF down-weight words like 'the'?

Their inverse document frequency is near zero because they appear in almost every document, so they carry no discriminating power.

2. What can embeddings do that bag of words cannot?

Capture similarity: 'great' and 'excellent' get nearby vectors, and contextual models handle word sense and order.

3. What is a sensible first model for classifying support tickets?

TF-IDF features plus logistic regression; evaluate it, then upgrade to fine-tuned transformers only if needed.

10.3 Transformers and attention

IN PLAIN WORDS

Reading a sentence, you understand each word by looking at the other words around it: in "The animal didn't cross the street because it was too tired", "it" means the animal. Self-attention lets every word look at every other word and decide how much to borrow from each. Each word asks a question (query), every word advertises what it offers (key), and the best matches pass along their content (value). Because all words look at each other at the same time, transformers process a sentence in parallel, unlike older models that read one word at a time.

Why it matters: Transformers power BERT, GPT, and every modern LLM; "explain attention" is a standard question.

Takeaway: attention weights = $\text{softmax}(QK^T/\sqrt{d})$; output = weighted sum of values.

WORKED EXAMPLE, STEP BY STEP

Attention for one word, with tiny vectors

The word "it" decides which earlier words to pay attention to. Its query is $q = [1, 0]$. The keys are: "animal" $[2, 0]$, "street" $[0, 2]$, "it" $[1, 1]$. The values are: "animal" $[1, 0]$, "street" $[0, 1]$, "it" $[0.5, 0.5]$.

- Scores = query · key:** animal 2, street 0, it 1.
- Scale by $\sqrt{d}$** (here $d = 2$, $\sqrt{2} \approx 1.41$): 1.41, 0, 0.71. Scaling stops large dot products from making softmax too sharp.
- Softmax turns scores into weights that sum to 1:** $e^{1.41} \approx 4.11$, $e^0 = 1$, $e^{0.71} \approx 2.03$; total 7.14. Weights: animal **0.58**, street **0.14**, it **0.28**.
- Output = weighted average of the values:** $0.58 \times [1, 0] + 0.14 \times [0, 1] + 0.28 \times [0.5, 0.5] = [0.72, 0.28]$.
- Read it.** The new representation of "it" is mostly "animal": attention has resolved what "it" refers to.
- In a real model** the queries, keys and values are learned projections of each word's embedding, every word does this for every other word at once, and several heads run in parallel.

What this shows. Attention is a weighted average, where the weights come from how well one word's query matches the others' keys.

YOUR TURN (COVER THE SOLUTION FIRST)

- Repeat the calculation with the query $q = [0, 1]$ (the same keys and values). Which word gets the most weight, and what is the output vector?

Solution. Scores: animal 0, street 2, it 1. Scaled: 0, 1.41, 0.71. Softmax weights: animal 0.14, street 0.58, it 0.28. Output = $0.14 \times [1, 0] + 0.58 \times [0, 1] + 0.28 \times [0.5, 0.5] = [0.28, 0.72]$. Now "street" dominates. The same keys and values give a different result for a different query, which is how each word gathers the context it needs.

THE FUNDAMENTALS

Transformers (be able to explain in 2 minutes):

- **Self-attention:** each token computes queries, keys, values; attention weights = $\text{softmax}(QK^T / \sqrt{d_k})$; output = weighted sum of values. Captures long-range dependencies in parallel (vs sequential RNNs).
- **Positional encoding** injects order. **Multi-head** attention learns different relations.
- **Encoder-only (BERT):** understanding tasks. **Decoder-only (GPT):** generation. **Encoder-decoder (T5):** seq2seq.
- **Pre-train then fine-tune;** masked language modeling vs next-token prediction.

```
from transformers import pipeline
clf = pipeline("sentiment-analysis")
clf("The delivery was super fast but the packaging was poor.")
```

SAY IT IN 60 SECONDS

"A transformer processes a whole sequence at once using self-attention. For each token, the model forms a query, key and value. The score between two tokens is the dot product of one's query with the other's key, scaled and passed through softmax to get weights; the token's new representation is the weighted sum of the values. So each token looks at the tokens most relevant to it, however far away. Multi-head attention runs several of these in parallel, and positional encodings add order information. Unlike RNNs, it parallelises on GPUs and handles long-range dependencies. The catch: attention cost grows with the square of the sequence length, which limits context size."

KEY TERMS self-attention · queries keys values · softmax · scaled dot product · multi-head attention · positional encoding · BERT · GPT · encoder-decoder · pre-training · fine-tuning

BUILDS ON 10.1 Deep learning fundamentals · 10.2 NLP: classical pipeline, TF-IDF and embeddings

HANDS-ON PRACTICE Compute attention weights by hand for three tokens with 2-dimensional vectors, then verify with numpy.

COMMON MISTAKES Saying attention "looks at every word equally" · Forgetting positional encoding · Mixing up encoder-only and decoder-only models

CHECK YOURSELF**1. In one sentence, what does self-attention compute?**

For each token, a weighted average of all tokens' value vectors, with weights from how well its query matches their keys.

2. Why do transformers need positional encodings?

Attention itself is order-agnostic; position information must be added so 'dog bites man' differs from 'man bites dog'.

3. Main advantage over RNNs?

Parallel training across the sequence and direct long-range connections, avoiding sequential bottlenecks and vanishing gradients.

10.4 LLM fundamentals and prompting

IN PLAIN WORDS

A large language model is a very well-read autocomplete. It has read a huge amount of text and learned to predict the next small piece of text (a token) that would most plausibly come next. Give it a prompt, and it keeps predicting the next piece, one at a time. It does not look facts up; it writes what sounds right, which is why it can confidently make things up (hallucinate). You steer it with clear instructions, examples and by giving it the facts it needs, and control how adventurous it is with the temperature setting.

Why it matters: GenAI questions are increasingly common in data science interviews.

Takeaway: Tokens in, next-token probabilities out; steer with prompts and context; ground facts with retrieval.

WORKED EXAMPLE, STEP BY STEP

What temperature does to the next word

After "The capital of France is", a model's raw scores (logits) are: Paris 5, Lyon 2, "a" 1. Probabilities come from $\text{softmax}(\text{logits} / \text{temperature})$.

- 1. Temperature 1:** $e^5 \approx 148.4$, $e^2 \approx 7.4$, $e^1 \approx 2.7$. Probabilities: Paris **0.94**, Lyon 0.05, "a" 0.02.
- 2. Temperature 0.5** (the logits double to 10, 4, 2): Paris **0.997**. Nearly deterministic: the top word almost always wins.
- 3. Temperature 2** (the logits halve to 2.5, 1, 0.5): Paris **0.74**, Lyon 0.16, "a" 0.10. More variety, and more chances to pick something wrong.

Temperature	Paris	Lyon	"a"
0.5	0.997	0.002	0.000
1	0.94	0.05	0.02
2	0.74	0.16	0.10

- 4. Choose by task.** Extracting facts, writing SQL or classifying: low temperature (around 0 to 0.3). Brainstorming names or marketing copy: higher (0.7 to 1).
- 5. Hallucination link.** At any temperature, the model picks *plausible* text. If the right fact is not well represented, a fluent wrong answer can still have the highest probability. Grounding with retrieved sources (RAG) addresses that; temperature does not.

What this shows. The model produces a probability for every next token; temperature sharpens or flattens that distribution before sampling.

YOUR TURN (COVER THE SOLUTION FIRST)

1. A support bot will answer 2,000 questions a day. Each call sends a 1,200-token prompt (instructions plus retrieved passages) and receives a 300-token answer. How many tokens a day is that, and name two ways to cut it.

Solution. $2,000 \times (1,200 + 300) = 3,000,000$ tokens a day. To cut it: retrieve fewer or shorter chunks (for example the top 3 instead of the top 6) and shorten the instructions; cache answers to frequent questions; route simple questions to a smaller, cheaper model; and cap the answer length. Measure answer quality on an evaluation set after each cut.

THE FUNDAMENTALS**Fundamentals**

- Tokens and tokenization (BPE), context window, embeddings, next-token prediction, sampling parameters (**temperature**, top-p, top-k, max tokens).
- **Training stages:** pre-training → supervised fine-tuning → preference tuning (RLHF/DPO).
- **Hallucination:** fluent but wrong output; mitigations: grounding with retrieval, citations, constrained outputs, verification, lower temperature.

Prompting

- Zero-shot, few-shot, chain-of-thought, role/system prompts, structured outputs (JSON schema), delimiters, iterative prompt evaluation.

SAY IT IN 60 SECONDS

"A large language model is a transformer trained to predict the next token on huge text. Text is split into tokens, pieces of words. At generation, the model outputs a probability over the next token, and sampling settings control creativity: temperature near zero is near-deterministic, higher is more varied; top-p limits sampling to the most likely tokens. After pretraining, instruction tuning and human feedback make it follow instructions. The context window is the amount of text it can see at once. Prompting works better with a clear role, explicit task, examples, and the output format. The catch: LLMs hallucinate, producing fluent but false statements, so I ground them with retrieved sources and validate outputs."

KEY TERMS tokens · tokenization · context window · embeddings · temperature · top-p · pre-training · supervised fine-tuning · RLHF · hallucination · few-shot · chain-of-thought

BUILDS ON 10.3 Transformers and attention

HANDS-ON PRACTICE Write a prompt that extracts three fields from a support email as JSON, test it on five tricky inputs and refine it.

COMMON MISTAKES Using a high temperature for factual tasks · Assuming the model knows up-to-date or private facts · Prompts without clear format or examples

CHECK YOURSELF**1. Temperature 0 versus temperature 1?**

Near 0 picks the most likely token each time (stable, repetitive); 1 samples more freely (varied, riskier).

2. Why do LLMs hallucinate?

They are trained to produce plausible next tokens, not verified facts; without grounding they will fill gaps convincingly.

3. Three prompt-quality tips?

Specify the role and task clearly, give examples (few-shot), and define the output format; ask for reasoning or citations when useful.

10.5 RAG end to end**IN PLAIN WORDS**

RAG is an open-book exam for an AI. Instead of asking the model to answer from memory, you first look up the most relevant pages from your own documents, then hand those pages to the model and say "answer using only these". To make the lookup work, you cut documents into chunks, turn each chunk into a vector of numbers that captures its meaning, and store them. When a question arrives, you turn it into a vector too and fetch the closest chunks. Most mistakes come from the lookup (wrong pages found), not from the writing.

Why it matters: It is the dominant pattern for company GenAI apps (policies, reports, knowledge bases) and a top project idea.

Takeaway: Good RAG is good retrieval; evaluate retrieval and answers separately.

WORKED EXAMPLE, STEP BY STEP**Retrieval by cosine similarity, then the prompt**

Question embedding $q = [0.9, 0.1, 0.4]$. Three chunks in the index:

Chunk	Embedding	Cosine with q
Refund policy	[0.8, 0.2, 0.5]	0.985
Shipping times	[0.1, 0.9, 0.3]	0.318
Warranty terms	[0.6, 0.1, 0.8]	0.874

- 1. Embed the question** with the *same* model that embedded the chunks.
- 2. Score every chunk** by cosine similarity (an approximate-nearest-neighbour index does this quickly for millions of chunks).
- 3. Take the top $k = 2$:** Refund policy and Warranty terms.
- 4. Build the prompt:** *"Answer using only the sources below. If the answer is not in them, say you do not know. Cite the source title. [Refund policy] ... [Warranty terms] ... Question: Can I return a phone after 20 days?"*
- 5. Generate** with a low temperature, then check that the answer cites a source.

6. Evaluate both halves separately. Retrieval: was the right chunk in the top k (recall@k)?

Generation: is every claim supported by the retrieved text (faithfulness)?

What this shows. RAG is a search step followed by a constrained writing step. When an answer is wrong, first check whether the right chunk was retrieved.

YOUR TURN (COVER THE SOLUTION FIRST)

1. For 50 test questions, the chunk containing the answer ranks 1st for 30, 2nd or 3rd for 10, between 4th and 10th for 6, and is not found for 4. With $k = 3$, what is recall@3? Name two fixes for the misses.

Solution. $\text{Recall@3} = (30 + 10) / 50 = 0.8$. Ten questions never reach the model with the right context. Fixes: raise k and add a reranker so that the relevant chunk moves up (it helps the 6 at ranks 4 to 10); improve chunking, for example smaller chunks or keeping headings with their text; and add hybrid keyword search for exact terms such as product codes, which often catches the 4 that were not found.

THE FUNDAMENTALS**RAG (Retrieval-Augmented Generation): know end to end**

Documents → clean/parse → chunk (size 300-800 tokens, overlap) → embed → vector DB
 Query → embed → retrieve top-k (hybrid: BM25 + dense) → (optional) rerank → build prompt with context → LLM → answer with citations

- **Design decisions:** chunking strategy, embedding model, vector store (FAISS, Chroma, Pinecone, Qdrant, pgvector), top-k, hybrid search, reranking, metadata filters, prompt template, guardrails.
- **Evaluation:** retrieval (recall@k, MRR, hit rate), generation (faithfulness/groundedness, answer relevance, context precision/recall; frameworks like RAGAS, LLM-as-judge with human spot checks), latency and cost.
- **Failure modes:** bad chunking, irrelevant retrieval, lost-in-the-middle, stale index, prompt injection from documents.

SAY IT IN 60 SECONDS

"Retrieval-augmented generation gives an LLM relevant documents at question time. Offline: split documents into chunks, turn each into an embedding vector, and store them in a vector database. Online: embed the question, retrieve the nearest chunks by similarity, put them in the prompt with an instruction to answer only from them, and generate with citations. It reduces hallucination, uses fresh private data, and needs no retraining. Quality hinges on chunk size, good embeddings, hybrid search with keywords, and reranking. I evaluate retrieval separately, with recall at K, and the answer for faithfulness to the sources. The catch: if retrieval fails, the answer is wrong, so I debug retrieval first."

KEY TERMS chunking · embeddings · vector database · hybrid search · BM25 · reranking · faithfulness · recall at k · prompt injection · lost in the middle

BUILDS ON 10.4 LLM fundamentals and prompting

HANDS-ON PRACTICE Build a small RAG over five PDFs with a golden set of 30 questions and measure retrieval recall@k and answer faithfulness.

COMMON MISTAKES Evaluating only the final answer and not the retrieval step · Chunks that cut through sentences or tables · Trusting retrieved text blindly (prompt injection)

CHECK YOURSELF

1. The RAG bot gives a wrong answer. What do you check first?

Whether the right chunk was retrieved (retrieval recall). If so, check the prompt and generation; if not, fix chunking, embeddings, or add hybrid search and reranking.

2. Why chunk documents?

Embeddings and prompts have limits and a whole document dilutes meaning; smaller focused chunks retrieve precisely and fit the context window.

3. Name two benefits of RAG over relying on the model alone.

Up-to-date or private knowledge without retraining, and answers traceable to sources, which reduces hallucination.

10.6 RAG vs fine-tuning, agents and project ideas

IN PLAIN WORDS

There are two main ways to make an AI know your stuff. You can hand it a reference book each time (RAG: great for facts that change and need citations), or you can send it to school on your examples so its habits change (fine-tuning: great for a consistent style, format or specialist behaviour). Agents go one step further: the model can decide to use tools, like running a database query or a calculator, look at the result, and continue. The more power you give it, the more you must test, limit and monitor it.

Why it matters: Interviewers ask about trade-offs (RAG vs fine-tuning), safety and evaluation, not about API syntax.

Takeaway: Facts via retrieval, behaviour via fine-tuning, actions via tools with guardrails, all measured on a golden set.

WORKED EXAMPLE, STEP BY STEP

Prompt, RAG or fine-tune: decide case by case

Four requests from the business.

- "Summarise customer calls in our standard three-bullet format."** The knowledge is in the call itself; only the format matters. **Prompting** with two or three examples. Fine-tune only if the format still drifts across thousands of calls.
- "Answer employee questions from the HR policy, which changes every quarter."** The knowledge is private and changes. **RAG**: update the documents, and the answers change the same day.
- "Write product descriptions in our brand voice, at scale, cheaply."** Behaviour and style, needed many times. **Fine-tune** a smaller model (for example with LoRA) on a few thousand approved descriptions, which also lowers cost per call.

4. **"Tell me last month's revenue against the month before."** The facts live in a database. An **agent with a SQL tool**: it writes a query, runs it, checks the result and explains it, within read-only permissions.
5. **For every case, write the evaluation first:** 30 to 100 real examples with the expected output, scored before and after each change.

What this shows. Knowledge goes in through retrieval or tools; behaviour goes in through prompts or fine-tuning. Start with the cheapest option that passes your evaluation set.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Trace an agent answering "Which city had the biggest drop in orders last week?" List the steps it should take, and two guardrails.

Solution. Steps: (1) plan: compare orders per city for last week and the week before; (2) call the SQL tool with a query grouping orders by city and week; (3) compute the change per city from the result; (4) check the result (for example, that the totals match a known dashboard figure); (5) answer with the city, the size of the drop and the query used. Guardrails: read-only database credentials with a row limit and a timeout; a maximum number of tool calls per question; and logging each step so a human can audit it.

THE FUNDAMENTALS

RAG vs fine-tuning

Need	Prefer
Fresh/private knowledge, citations, frequent updates	RAG
Style, format, domain behavior, latency/cost at scale	Fine-tuning (LoRA/QLoRA, parameter-efficient)
Both	RAG + light fine-tuning

Other topics: agents and tool/function calling, embeddings and cosine similarity, LangChain/LlamaIndex basics, evaluation datasets ("golden sets"), guardrails/PII handling, cost and latency optimization (caching, smaller models, batching), prompt injection and safety.

Project-ready ideas: document Q&A over policies/annual reports, text-to-SQL assistant, resume screener, support-ticket triage, review summarizer.

SAY IT IN 60 SECONDS

"When should I use RAG and when fine-tuning? RAG adds knowledge: facts that change or are private. Fine-tuning changes behaviour: tone, format, or a specialised skill. They combine well. Start with prompting, add RAG for knowledge, and fine-tune last, when needed for consistent style or latency and cost. An agent is an LLM that can plan and call tools, like search, a database or code, in a loop until the task is done. It is powerful but less predictable, so I add limits, logging and human checks. Good project ideas: a document question-answering bot, a support-ticket triage tool, or a SQL assistant. The catch: evaluate with a test set of real questions, not impressions."

KEY TERMS RAG vs fine-tuning · LoRA · agents · tool calling · function calling · guardrails · cost and latency · golden sets · project ideas

BUILDS ON 10.5 RAG end to end

HANDS-ON PRACTICE Design a text-to-SQL assistant: list components, guardrails, evaluation and failure modes.

COMMON MISTAKES Fine-tuning to inject facts that change often · Giving agents broad permissions · Shipping without an evaluation set

CHECK YOURSELF

1. Company policies change weekly and the bot must answer from them. RAG or fine-tuning?

RAG: update the document store and answers change immediately; fine-tuning would need retraining for each change.

2. What makes an agent riskier than a plain prompt?

It takes actions and chains steps, so errors compound and side effects occur; mitigate with tool limits, validation, logging and approvals.

3. How do you evaluate an LLM application?

Build a labelled set of realistic questions, score retrieval and answer faithfulness/correctness (with human or model graders), and track regressions on each change.

10.7 Computer vision and self-check

IN PLAIN WORDS

Teaching a computer to see is like teaching a child who has already learned to recognise edges, shapes and textures. You do not start from zero: take a model that has already learned from millions of images and teach it your specific task (a few hundred photos of damaged parcels) by retraining only its last layers. To spot objects rather than label whole images, detection models draw boxes, and we score them by how much the box overlaps the real one (IoU).

Why it matters: Only needed if the role involves images or documents, but transfer learning and OCR questions are common for KYC and insurance use cases in India.

Takeaway: Transfer learning plus augmentation; detection is judged by IoU and mAP; OCR pipelines need validation and privacy care.

WORKED EXAMPLE, STEP BY STEP

Count a convolution's outputs and parameters

Input: a 32×32 colour image (3 channels). Layer: 16 filters of size 3×3 , stride 1, padding 1.

- 1. Output size:** $(W - K + 2P) / S + 1 = (32 - 3 + 2) / 1 + 1 = 32$. With padding 1 the spatial size is kept: the output is $32 \times 32 \times 16$.
- 2. Parameters per filter:** $3 \times 3 \times 3 = 27$ weights (the filter spans all 3 input channels) plus 1 bias = 28.
- 3. Parameters for the layer:** $16 \times 28 = 448$.
- 4. Compare a dense layer** from the same image ($32 \times 32 \times 3 = 3,072$ inputs) to just 16 units: $3,072 \times 16 + 16 = 49,168$ parameters, over 100 times more, with no sense of "nearby pixels".
- 5. Why so few?** Each filter is small and is *reused at every position* (weight sharing): an edge detector works the same at the top left and the bottom right.

6. Then pooling: a 2×2 max-pool halves width and height: $16 \times 16 \times 16$.

What this shows. Convolutions exploit two facts about images: nearby pixels matter together, and a pattern means the same thing wherever it appears. That is why CNNs need far fewer parameters than dense networks.

YOUR TURN (COVER THE SOLUTION FIRST)

1. A grey-scale 28×28 image goes through 8 filters of size 5×5 , stride 1, no padding. What is the output shape, and how many parameters does the layer have?

Solution. Output size = $(28 - 5 + 0) / 1 + 1 = 24$, so $24 \times 24 \times 8$. Parameters: each filter has $5 \times 5 \times 1 = 25$ weights plus 1 bias = 26; 8 filters give 208 parameters.

THE FUNDAMENTALS

10.4 Computer vision (only if role requires)

Image classification with transfer learning (ResNet/EfficientNet), augmentation, object detection (YOLO, Faster R-CNN) basics, IoU, mAP, OCR (Tesseract, PaddleOCR, cloud APIs), especially for Indian documents (Aadhaar/PAN KYC workflows).

10.5 Self-check

- ☐ Explain attention and why transformers replaced RNNs.
- ☐ Draw a RAG architecture and name three failure modes and how you'd evaluate it.
- ☐ Explain TF-IDF and when it beats embeddings.

SAY IT IN 60 SECONDS

"Computer vision teaches models to understand images. Convolutional networks slide small filters over an image to detect edges, then textures, then parts, then objects, sharing weights so they need far fewer parameters than dense layers. Pooling shrinks the map. Tasks: classification, one label per image; detection, boxes plus labels; segmentation, a label per pixel. In practice I use transfer learning: take a model pretrained on a large dataset such as ImageNet, replace the final layer and fine-tune on my data, which works with small datasets. Data augmentation, like flips and crops, adds variety. Detection is scored by IoU and mean average precision. The catch: check that train and production images look alike."

KEY TERMS transfer learning · data augmentation · object detection · YOLO · IoU · mAP · OCR · KYC documents

HANDS-ON PRACTICE Fine-tune a pre-trained ResNet on a small two-class image dataset with augmentation.

COMMON MISTAKES Training a CNN from scratch on a tiny dataset · Evaluating detection with accuracy · Handling identity documents without privacy controls

CHECK YOURSELF**1. Why do CNNs use fewer parameters than dense layers on images?**

Small filters are shared across all positions (weight sharing) and connect locally.

2. Only 500 labelled images. Approach?

Transfer learning from a pretrained model with augmentation; freeze early layers, fine-tune the last ones.

3. Two boxes of area 16 overlap in area 8. IoU?

$8 / (16 + 16 - 8) = 0.33$.

Interview tips, tricks and hacks

- 1. Explain a network as "many simple functions stacked, trained by gradient descent".** Then add that backpropagation is how the error is shared out among the weights. Simple and correct beats detailed and shaky.
- 2. Say when not to use deep learning.** "For a few thousand rows of tabular data I would start with gradient boosting." This judgement is often worth more than architecture knowledge.
- 3. Give attention in one sentence.** "Each word takes a weighted average of all the other words, with weights showing how relevant each is." Follow with the "it refers to the animal" example.
- 4. Describe RAG as two phases.** Offline: chunk, embed, store. Online: embed the question, retrieve, build the prompt, generate with citations. Drawing it takes thirty seconds and answers most GenAI questions.
- 5. When a RAG answer is wrong, check retrieval first.** Was the right passage retrieved? If not, fix chunking, embeddings or search. If yes, fix the prompt. This is the standard debugging question.
- 6. Keep the RAG versus fine-tuning rule ready.** Knowledge that changes: RAG. Behaviour, tone or format: fine-tuning. Start with prompting.
- 7. Always mention evaluation.** A test set of real questions with expected answers, measured again after every change. Many candidates describe how to build but not how to know it works.
- 8. Define hallucination and one mitigation.** Grounding in retrieved sources with citations, and allowing "I do not know".
- 9. Use transfer learning as your default for images and text.** "Pretrained model, replace the final layer, fine-tune on my data, with augmentation."
- 10. Be honest about depth.** If you have built with an API but not trained a model, say so and describe precisely what you did. Interviewers probe GenAI claims hard.

Frequently asked questions

7 questions interviewers ask on this chapter, with model answers. Cover the answer, say yours aloud, then compare.

DEEP LEARNING FUNDAMENTALS

Q1 Why can't a neural network with only linear activations learn a non-linear function?

If every layer computes $Wx + b$, then two layers give $W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$, which is still linear. Non-linear activations (ReLU, tanh) between layers let the network approximate arbitrary non-linear functions (the universal approximation idea). This is why the choice of activation matters, and why a network without them is just a linear or logistic regression.

NLP: CLASSICAL PIPELINE, TF-IDF AND EMBEDDINGS

Q2 Customer-support tickets must be routed to one of 12 teams. You have 5,000 labelled tickets. What is your approach?

Start with a **TF-IDF (word and character n-grams) + logistic regression or linear SVM** baseline: fast, strong, interpretable. Evaluate macro-F1 and a per-class confusion matrix (classes may be imbalanced). Then try **sentence embeddings + logistic regression**, and, if the gain justifies the cost, a **fine-tuned multilingual transformer**. Handle Hinglish and typos (character n-grams or fastText), add metadata features (channel, product), set a confidence threshold to route uncertain tickets to humans, and monitor drift in ticket topics.

TRANSFORMERS AND ATTENTION

Q3 Why does self-attention divide the scores by $\sqrt{d_k}$?

If query and key components have unit variance, their dot product has variance d_k , so for large d_k the scores become large in magnitude. The softmax then outputs near one-hot distributions with tiny gradients, making training unstable. Scaling by $1/\sqrt{d_k}$ keeps the variance near 1.

LLM FUNDAMENTALS AND PROMPTING

Q4 An LLM-powered assistant confidently quotes a refund policy that does not exist. How do you fix it?

Put the real policy documents behind a **retrieval step** (RAG), instruct the model to answer **only from the provided context** and cite the passage (or say it does not know), keep temperature low, and validate that cited text actually supports the answer. Add an evaluation set of policy questions (including unanswerable ones) to measure **faithfulness** and refusal behaviour, and log failures for continuous improvement.

RAG END TO END

Q5 Your RAG bot answers incorrectly on 20% of test questions. How do you find the cause?

For each failed question, inspect the retrieved chunks. If the **needed passage was not retrieved**, it is a retrieval problem: improve chunking, add hybrid search or reranking, tune top-k, rewrite queries, fix metadata filters. If the passage **was retrieved but the answer was wrong**, it is a generation problem: improve the prompt (answer only from context, cite), reduce context noise, use a stronger model, or add a verification step. Fix the larger bucket first and track recall@k and faithfulness separately so you can see which change helped.

RAG VS FINE-TUNING, AGENTS AND PROJECT IDEAS

Q6 Design a text-to-SQL assistant for business users. What can go wrong and how do you protect against it?

Components: retrieve the relevant table and column descriptions (schema RAG), give a few examples, generate SQL, **validate** it (parse, allow only **SELECT**, whitelisted tables, row and time limits), execute with a **read-only** user and timeouts, and explain the result. Risks: **incorrect logic** that looks plausible (mitigate with an evaluation set of question-to-SQL pairs, result sanity checks, showing the SQL to the user), **ambiguous questions** (ask clarifying questions), **data leakage across permissions** (row-level security), **prompt injection** via column comments or data, and **cost**. Measure execution accuracy on the golden set and log user corrections.

COMPUTER VISION AND SELF-CHECK

Q7 You have 400 labelled photos of product defects. What is your plan?

Use **transfer learning** (pre-trained ResNet/EfficientNet), freeze most layers, fine-tune the head, and **augment** images to mimic real variations (lighting, angle, blur). Hold out a validation set that reflects production conditions (new days and cameras), handle class imbalance (weights), and choose metrics by defect cost (recall on critical defects). Inspect misclassified images for label noise, use Grad-CAM to check that the model looks at the defect and not at background artefacts, and collect more data where it fails.

NEURAL NETWORK MECHANICS

- A **neuron**: output = activation($w \cdot x + b$). Layers of neurons; the **non-linear activation** is what lets the network learn curves (stacked linear layers collapse to one linear map).
- Training loop**: forward pass → loss → **backpropagation** (chain rule gives $d\text{Loss}/dw$ for every weight) → optimiser step (Adam/SGD) → repeat per mini-batch; one pass over the data = epoch.

Activation	Formula / range	Note
ReLU	$\max(0, x)$	default hidden; "dying ReLU"
Leaky ReLU / GELU	small slope for $x < 0$ / smooth	transformers use GELU
Sigmoid	$1/(1 + e^{-(x)})$, (0, 1)	binary output; vanishing gradients
Tanh	(-1, 1)	zero-centred; RNNs
Softmax	$e^{(z_i)} / \sum e^{(z_j)}$	multiclass probabilities

- Output + loss pairs**: regression: linear + MSE; binary: sigmoid + binary cross-entropy; multiclass: softmax + cross-entropy.
- Optimisers**: SGD + momentum; **Adam** (default); learning-rate schedules, warm-up; typical Adam lr $1e-3$ ($1e-4$ for fine-tuning).
- Initialisation**: Xavier/Glorot (sigmoid/tanh), He (ReLU). **Vanishing/exploding gradients**: ReLU, good init, **batch/layer norm**, residual connections, gradient clipping.
- Regularise**: dropout (randomly zero units while training), weight decay (L2), early stopping, data augmentation, more data, smaller net.
- Batch norm** normalises activations per batch (faster, stabler); **layer norm** per example (transformers).
- For small tabular data, gradient boosting usually beats deep nets.

ARCHITECTURES AT A GLANCE

Type	Good for	Core idea
MLP	tabular, simple	dense layers
CNN	images, audio	local filters with shared weights, pooling
RNN / LSTM / GRU	sequences	hidden state; LSTM gates fight vanishing gradients
Transformer	text, now vision	self-attention, parallel
Autoencoder / VAE	compression, anomalies, generation	encoder → latent → decoder
GAN / diffusion	image generation	adversarial / denoising

- CNN output size**: $(W - K + 2P)/S + 1$. Parameters of a conv layer: $K \cdot K \cdot C_{in} \cdot C_{out} + C_{out}$.
- Framework skeleton (PyTorch): `loss = crit(model(x), y); opt.zero_grad(); loss.backward(); opt.step(); model.eval()` and `torch.no_grad()` for inference.

BACKPROP AND SEQUENCE MODELS BY HAND

- One neuron, squared loss: $y_{\text{hat}} = \sigma(wx + b)$, $L = (y_{\text{hat}} - y)^2$; $dL/dw = 2(y_{\text{hat}} - y) \cdot \sigma'(z) \cdot x$; update $w \leftarrow w - \eta \cdot dL/dw$. Backprop reuses each layer's gradient going backwards.
- RNN**: $h_t = \tanh(W \cdot h_{t-1} + U \cdot x_t)$; gradients shrink over long sequences. **LSTM** adds forget, input and output gates and a cell state; **GRU** is a lighter two-gate version.
- Parameter count of a dense layer: $n_{in} \cdot n_{out} + n_{out}$.
- Hyperparameters**: learning rate (most important), batch size, depth/width, dropout rate, weight decay, epochs with early stopping.
- Debug order: data and labels → loss at start ($\sim \ln(K)$ for K classes) → overfit one batch → learning rate → architecture.

COMPUTER VISION

- Convolution** slides small filters (edges → textures → parts → objects); pooling shrinks maps. Far fewer parameters than dense layers.
- Tasks: **classification** (label), **detection** (boxes + labels: YOLO, Faster R-CNN), **segmentation** (label per pixel: U-Net, Mask R-CNN).
- Transfer learning**: take a model pretrained on ImageNet, replace the head, freeze early layers, fine-tune the rest. Works with small datasets. **Augmentation**: flips, crops, colour jitter.
- IoU** = overlap area/union area; 2 boxes of area 16 overlapping 8: $8/24 = 0.33$. A detection counts correct at $\text{IoU} \geq 0.5$. **AP** = area under the precision-recall curve per class; **mAP** = mean over classes (and IoU thresholds).
- Check train and production images look alike (lighting, devices).

CLASSICAL NLP PIPELINE

- Clean → tokenise → (lowercase, stop words, stem/lemmatise) → vectorise → model.
- Bag of words**: counts. **TF-IDF** = $\text{tf} \times \log(N/\text{df})$: frequent in this document, rare overall; sparse, ignores order and meaning. n-grams capture short phrases.
- Embeddings**: dense vectors where similar meanings are close (word2vec, GloVe, fastText); **contextual** embeddings (BERT) give "bank" different vectors by sentence. Cosine similarity compares vectors.
- Baselines: TF-IDF + logistic regression / linear SVM / Naive Bayes are strong and cheap; then fine-tune a transformer.
- Tasks and metrics: classification (F1), NER, summarisation (ROUGE), translation (BLEU), retrieval (recall@K, MRR), language modelling (perplexity).

```
from sklearn.feature_extraction.text
import (
    TfidfVectorizer)
v = TfidfVectorizer(ngram_range=(1, 2),
    min_df=2, stop_words="english")
X = v.fit_transform(texts)
```


TRANSFORMERS AND ATTENTION

- **Self-attention:** each token makes a query Q, key K and value V. Attention = $\text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$: every token takes a weighted average of all tokens' values, weights from query-key match. The $\sqrt{d_k}$ scaling keeps softmax from saturating.
- **Multi-head:** several attentions in parallel learn different relations. **Positional encodings** inject order. **Residual + layer norm**, feed-forward block per layer.
- **Encoder** (BERT: understanding, classification, embeddings), **decoder** (GPT: next-token generation, causal mask), encoder-decoder (T5: translation, summarisation).
- vs RNNs: parallel training, direct long-range links. **Cost grows as n^2 with sequence length**, which limits context.
- Pretrain on huge text → **fine-tune** for a task.

LLM FUNDAMENTALS

- **Tokens:** sub-word pieces (~0.75 words each); context window = max tokens in prompt + output. Cost and latency scale with tokens.
- Generation: the model outputs a probability over the next token. **Temperature 0** = near-deterministic, higher = more varied; **top-p** (nucleus) samples from the most likely mass; **top-k**.
- Training stages: pretraining (next token) → **instruction tuning (SFT)** → **RLHF / preference tuning (DPO)**.
- **Prompting:** clear role, task and format; **few-shot** examples; ask for step-by-step reasoning (chain of thought); delimit inputs; request JSON and validate it; keep prompts versioned and tested.
- **Hallucination:** fluent but false output. Reduce with retrieval (RAG) and citations, lower temperature, constrained formats, verification/tool calls, "say you do not know".
- **Evaluation:** a golden set of real questions; automatic checks, LLM-as-judge (with human calibration); track regressions per prompt/model change.
- Safety: prompt injection (untrusted text containing instructions), PII leakage, jailbreaks: filter, isolate untrusted content, least-privilege tools.

RAG END TO END

- **Offline:** load → clean → **chunk** (200-800 tokens, overlap 10-20%, respect headings) → **embed** → store in a vector index (FAISS, pgvector, Pinecone) with metadata.
- **Online:** embed query → retrieve top-k (similarity: cosine/dot; ANN indexes: HNSW, IVF) → optional **rerank** (cross-encoder) → build prompt with sources and "answer only from the context" → generate with citations.
- Improve: **hybrid search** (BM25 keyword + vectors), metadata filters, query rewriting, parent-document retrieval, good chunking.
- **Evaluate separately:** retrieval (recall@K, hit rate, MRR) and generation (faithfulness/groundedness, answer relevance, correctness). If the answer is wrong, **check retrieval first**.
- Failure modes: wrong chunk size, stale index, missing documents, embedding model mismatch, context overflow, lost-in-the-middle.

RAG VS FINE-TUNING, AGENTS

Need	Use
Fresh, private or changing facts, citations	RAG
Style, tone, format, specialised behaviour, lower latency	Fine-tune (LoRA / QLoRA: train small low-rank adapters)
Both	RAG + a lightly fine-tuned model

- Order of effort: prompting → RAG → fine-tuning.
- **Agent:** an LLM that plans and calls tools (search, SQL, code, APIs) in a loop. ReAct pattern: reason → act → observe. Add step limits, logging, validation, human approval for risky actions; errors compound over steps.
- **Embedding models** for search vs **generation models:** different jobs. **Function calling / structured outputs** make tool use reliable.
- Project ideas: document Q&A bot, support-ticket triage, text-to-SQL assistant, resume screener, meeting summariser (with evaluation set and cost report).
- Cost levers: smaller model, caching, shorter prompts, batching, streaming.

TOKENISATION, EMBEDDINGS, PROMPTS

- **BPE / WordPiece** tokenisers split rare words into pieces: no out-of-vocabulary words; counts differ by language (non-English costs more tokens).
- Sentence embeddings (e.g. SBERT) → cosine similarity for semantic search, clustering, dedupe; normalise vectors so dot product = cosine.
- Prompt skeleton: **Role** (you are ...), **Task** (do ...), **Context** (use only ...), **Format** (JSON with keys ...), **Examples** (2-3), **Constraints** (if unknown say so).
- Eval table: question | expected | retrieved | answer | grounded? | correct? → aggregate; re-run after every prompt, chunking or model change.
- Rough sizes: context 8k-200k tokens; model parameter count ~ memory (7B params ~ 14 GB in fp16, ~4 GB with 4-bit quantisation).

TRAINING TIPS AND PITFALLS

- Overfit a tiny batch first to check the pipeline; watch train vs validation loss; stop when validation stalls.
- Loss NaN: lower the learning rate, check inputs, clip gradients. Class imbalance: weights or sampling.
- Data leakage via near-duplicates between train and test (images, text).
- GPU memory: smaller batch, mixed precision (fp16/bf16), gradient accumulation.
- Reproducibility: seeds, pinned versions, logged configs.

SAY IT IN 20 SECONDS

- "Attention lets each token take a weighted average of all tokens."
- "RAG adds knowledge at question time; fine-tuning changes behaviour."
- "If a RAG answer is wrong, check retrieval before the prompt."
- "ReLU gives non-linearity; backprop is the chain rule."
- "Transfer learning: pretrained backbone, new head, fine-tune."

11

CHAPTER 11 · MEDIUM PRIORITY · MODULE 11 IN THE APP

Data Engineering and Tooling

The plumbing and tools around the models: how data reaches you, how it is processed at scale, and the everyday tools every data team expects you to use.

5 topics · about 2 h 40 min in the app · 9 questions answered · two-page cheat sheet at the end

11.1 Data engineering concepts

11.2 PySpark

11.3 Databases and cloud platforms

11.4 Developer tools: Git, Linux, environments, Docker

11.5 BI tools, Excel and self-check

Foundations

A data scientist does not work alone with a clean file. The data arrives through pipelines that engineers build, sits in systems with particular strengths, and is too large for one laptop. The code lives in a shared repository and must run on other people's machines. Interviews for junior roles test this area at conversational depth: you should be able to explain each concept in a few sentences, say when you would use it, and describe honestly what you have used.

The journey of data

Follow one click from a phone to a dashboard.

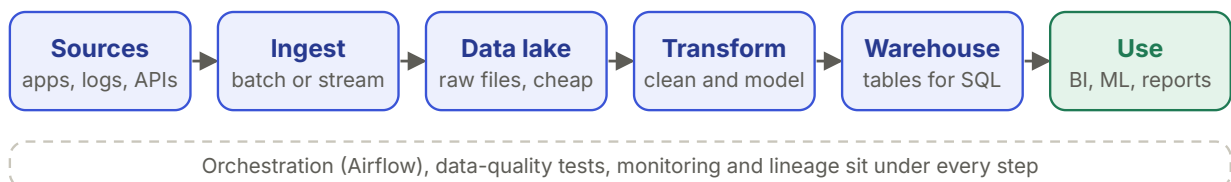


Figure 13. Data moves from operational systems through storage and transformation to the people and models that use it.

The application writes to an operational database. A pipeline copies the data into cheap bulk storage, the **data lake**, in raw form. Transformations clean it, join it and reshape it into well-defined tables in a **data warehouse**, where analysts can query it with SQL. The classic approach is **ETL**: extract, transform, then load. The modern approach in cloud warehouses is **ELT**: load the raw data first and transform it inside the warehouse, which keeps the raw copy for reprocessing. Work happens either in **batches** on a schedule or as a **stream**, event by event; batch is simpler and is the right default unless the business truly needs answers within seconds. An **orchestrator** such as Airflow runs the steps in the right order, retries failures and raises alerts.

Choosing a store by how it is used

A **relational database** stores rows and guarantees that a transaction, such as a payment, either happens completely or not at all. It is built for many small reads and writes. A **data warehouse** stores each column together, which makes summing a billion values of one column fast. It is built for large analytical questions. Various **NoSQL** stores trade some guarantees for scale or flexibility: key-value stores for instant lookups, document stores for flexible records. The practical lesson for interviews is to ask what the workload is before naming a technology.

When data outgrows one machine

Spark processes large data by splitting it into partitions spread across many machines. Two ideas explain most questions about it. It is **lazy**: describing a transformation does nothing until you ask for a result, which lets Spark plan the whole job efficiently. And the expensive step is the **shuffle**, when rows must move between machines to be grouped or joined. Good Spark code filters early, avoids unnecessary shuffles, and tells Spark when one side of a join is small enough to copy to every machine.

The everyday toolkit

Git records the history of your code and lets a team work in parallel: you create a branch, commit small changes, and propose them for review. **Linux commands** let you inspect files and run jobs on servers. **Virtual environments** give each project its own set of library versions, so that code which runs today still runs next year. **Docker** goes further, packaging the code together with its whole environment into an image that runs identically on any machine. The thread connecting all four is **reproducibility**: someone else, or you in six months, must be able to run the work and get the same result.

The last mile: showing results

Business intelligence tools such as Tableau and Power BI turn warehouse tables into dashboards, and spreadsheets remain everywhere. A pivot table is a `GROUP BY` and a lookup is a left join. A useful dashboard starts from a decision someone needs to make, shows the key number first, and uses definitions everyone has agreed on.

Topic by topic

Each topic gives the idea in plain words, the fundamentals to know, a model spoken answer and three questions to test yourself. Open the same topic in the app for worked examples, runnable code and interactive demos.

11.1 Data engineering concepts

IN PLAIN WORDS

Data engineering is the plumbing that brings raw data from where it is created (apps, payments, sensors) to where analysts and models can use it, clean and on time. Think of a city water system: pipes (pipelines) carry water from reservoirs (sources) to treatment plants (transformations) to taps (dashboards and models). Good plumbing is reliable (it runs every day), repeatable (running it twice does not double the water), and monitored (alarms go off when something leaks).

Why it matters: Data scientists who understand pipelines ship models faster and debug data problems sooner.

Takeaway: Move, clean, store and check data reliably, and make every step safe to rerun.

WORKED EXAMPLE, STEP BY STEP

Why a daily load must be idempotent

A pipeline copies yesterday's orders (1,200 rows) into the warehouse table `fact_orders` every night at 02:00.

- Version 1 appends:** `INSERT INTO fact_orders SELECT * FROM source WHERE order_date = yesterday .`
- The job fails halfway** after writing 700 rows. The scheduler retries it, and the retry inserts all 1,200 again.
- Count the damage:** $700 + 1,200 = 1,900$ rows for that day. 700 orders are now double-counted, and the next morning's revenue dashboard is about 58% too high for that date.
- Version 2 replaces the day's slice:** delete that date's rows, then insert, inside one transaction (or overwrite the date partition). A retry gives 1,200 rows however many times it runs.
- Version 3 upserts by key:** `MERGE INTO fact_orders USING staging ON order_id WHEN MATCHED THEN UPDATE ... WHEN NOT MATCHED THEN INSERT ...` . Late-arriving updates, such as a cancellation recorded the next day, are also handled.
- Add a check after the load:** the row count for the date equals the source count, and no `order_id` appears twice. Fail loudly if not.

What this shows. Pipelines fail and are retried. "Idempotent" means a rerun produces the same result as a single successful run, which is what makes retries and backfills safe.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Batch or streaming? (a) A finance team's daily revenue report; (b) blocking a card payment that looks fraudulent; (c) a model retrained weekly on clickstream data; (d) a live map of delivery partners.

Solution. (a) Batch: a daily schedule is enough, and simpler. (b) Streaming, or at least real-time scoring: the decision must be made within a second or two, while the payment is happening. (c) Batch: weekly training does not need second-level freshness; collect the stream into storage and process it on a schedule. (d) Streaming: positions change every few seconds. Choose batch unless the business decision genuinely needs freshness.

THE FUNDAMENTALS

- **ETL vs ELT:** transform before loading vs load raw then transform in the warehouse (modern, e.g., dbt).
- **Batch vs streaming** (Kafka, Spark Structured Streaming, Flink).
- **Data warehouse** (structured, schema-on-write: BigQuery, Snowflake, Redshift), **data lake** (raw, schema-on-read: S3/ADLS), **lakehouse** (Delta Lake, Iceberg; Databricks).
- **Modeling:** star schema (fact + dimension tables), snowflake schema, slowly changing dimensions (SCD 1/2), grain, surrogate keys.
- **File formats:** CSV/JSON (row, text) vs **Parquet/ORC** (columnar, compressed, predicate pushdown) vs Avro (row, schema evolution, streaming).
- **Data quality:** null checks, uniqueness, referential integrity, freshness, range checks, schema validation (Great Expectations, dbt tests), lineage, idempotent pipelines.
- **Orchestration: Airflow** (DAGs, operators, schedules, retries, backfills), Prefect, Dagster.
- **APIs and scraping:** requests, pagination, rate limits, BeautifulSoup/Selenium, respect robots.txt and ToS.

SAY IT IN 60 SECONDS

"Data engineering moves data from sources to where analysis can use it. ETL extracts, transforms, then loads into a warehouse; ELT loads raw data first and transforms inside the warehouse, which is common with cloud warehouses. A data lake stores raw files cheaply; a warehouse stores structured, modelled tables for analytics; a lakehouse combines both. Batch processing handles data in chunks, say nightly, and streaming handles events as they arrive, for example with Kafka. Orchestrators like Airflow schedule dependent tasks as a DAG. Good pipelines are idempotent, so rerunning gives the same result, and have data-quality checks. The catch: I choose batch unless the business truly needs low latency, as streaming is more complex."

KEY TERMS ETL vs ELT · batch vs streaming · data warehouse · data lake · lakehouse · star schema · slowly changing dimensions · Parquet · Airflow · data quality

HANDS-ON PRACTICE Sketch a daily pipeline that loads orders from an API to a warehouse and list where you would put five data quality checks.

COMMON MISTAKES Non-idempotent pipelines that duplicate data on rerun · Skipping data quality checks until the dashboard breaks · Treating the data lake as a dumping ground without schema or ownership

CHECK YOURSELF**1. ETL versus ELT?**

ETL transforms before loading; ELT loads raw data first and transforms in the warehouse, using its compute and keeping raw data for reprocessing.

2. What does idempotent mean for a pipeline, and why does it matter?

Re-running a job produces the same result without duplicates, so retries and backfills are safe.

3. Daily sales dashboard: batch or streaming?

Batch: daily freshness is enough, and it is simpler and cheaper.

11.2 PySpark

IN PLAIN WORDS

When data is too big for one computer, Spark splits it into pieces and lets many computers work on their pieces at the same time. A "driver" plans the work and "executors" do it. Spark is lazy: it does not do anything when you describe steps, only when you ask for a result (an action), which lets it plan the cheapest route. The expensive part is moving data between computers to regroup it (a shuffle), like asking everyone to swap papers to sit by the right group. Most Spark tuning is about avoiding or shrinking shuffles.

Why it matters: PySpark is asked frequently at Indian analytics and big-data firms.

Takeaway: Lazy plans, narrow vs wide steps, avoid shuffles with broadcast and pre-filtering, and never collect big data to the driver.

WORKED EXAMPLE, STEP BY STEP**Follow a Spark job and count the data it moves**

`events` is 400 GB across 4,000 partitions; `users` is 50 MB. The job: filter March events, join to users, count events per city.

- 1. Reading and filtering (narrow).** Each executor reads its own partitions and drops non-March rows. No data crosses the network. If the data is partitioned by date on disk, Spark skips the other months entirely (partition pruning).
- 2. Laziness.** Until you call an action such as `.count()` or `.write`, Spark only builds a plan. Then it optimises the whole plan, for example pushing the filter down to the read.
- 3. The join, done naively (shuffle join).** Both tables are redistributed by `user_id` so that matching keys meet on the same machine. Most of the large table travels over the network: the March slice of hundreds of gigabytes.
- 4. The join, done well (broadcast join).** `F.broadcast(users)` copies the 50 MB table to every executor. With 100 executors that is about 5 GB of copies, and the big table does not move at all.
- 5. The group by city (wide).** Rows for the same city must meet, which needs a shuffle, but after a partial count on each machine only a few numbers per city per partition move.
- 6. Write the result** as Parquet, partitioned by month.

What this shows. Spark's cost is the network. Filter early, broadcast small tables, and know which steps shuffle (joins, group by, distinct, order by).

YOUR TURN (COVER THE SOLUTION FIRST)

1. One task in a join stage takes 40 minutes while the other 199 tasks finish in 2 minutes. 60% of the rows have `country = 'IN'`, and you join on country. What is happening, and name two fixes.

Solution. Data skew: all the "IN" rows hash to the same partition, so one task does most of the work. Fixes: broadcast the other side if it is small, so no shuffle is needed; "salt" the hot key by adding a random suffix 0 to 9 on the big side and duplicating the small side for each suffix, spreading the hot key over 10 tasks; or turn on adaptive query execution, which splits skewed partitions automatically.

THE FUNDAMENTALS

Concepts

- Driver and executors, cluster manager, **RDD vs DataFrame vs Dataset**, **lazy evaluation**, **transformations** (narrow: map/filter; wide: groupBy/join, causing **shuffle**) vs **actions** (count, collect, show, write).
- Partitions, `repartition` vs `coalesce`, skew, **broadcast joins** for small tables, caching/persisting, avoiding Python UDFs (use built-in functions or pandas UDFs), predicate pushdown, partitioned Parquet/Delta writes.
- Spark SQL, window functions, Catalyst optimizer, Adaptive Query Execution.

```
from pyspark.sql import SparkSession, functions as F, Window

spark = SparkSession.builder.appName("demo").getOrCreate()
df = spark.read.parquet("s3://bucket/orders/")

res = (df.filter(F.col("status") == "delivered")
      .groupBy("city", F.date_trunc("month", "order_date").alias("mth"))
      .agg(F.sum("amount").alias("rev"), F.countDistinct("user_id").alias("users")))

w = Window.partitionBy("city").orderBy(F.desc("rev"))
top = res.withColumn("rk", F.dense_rank().over(w)).filter("rk <= 3")

from pyspark.sql.functions import broadcast
joined = df.join(broadcast(small_df), "product_id", "left")
```

SAY IT IN 60 SECONDS

"Spark processes data too big for one machine by splitting it into partitions across a cluster. PySpark is its Python API. DataFrame operations are lazy: transformations like filter, select and groupBy only build a plan, and nothing runs until an action like count, show or write. Narrow transformations work within a partition; wide ones like joins and groupBy need a shuffle, moving data over the network, which is the main cost. To speed up, I filter early, avoid unnecessary shuffles, broadcast the small table in a join, cache reused data, and watch skew, where one key has most rows. The catch: collect brings everything to the driver, so it can crash on big data."

KEY TERMS driver and executors · lazy evaluation · transformations and actions · shuffle · broadcast join · partitions · repartition vs coalesce · skew · caching · Catalyst · adaptive query execution

BUILDS ON 11.1 Data engineering concepts

HANDS-ON PRACTICE Write a PySpark query with a filter, groupBy, window rank and broadcast join and mark which steps cause a shuffle.

COMMON MISTAKES Using Python UDFs where built-in functions exist · Calling collect() on large data · Too few or too many partitions

CHECK YOURSELF

1. What does 'lazy evaluation' mean in Spark?

Transformations only build a plan; computation runs when an action is called, letting Spark optimise the whole plan.

2. Joining a 1 TB table with a 10 MB table. Optimisation?

Broadcast join: send the small table to every executor, avoiding a shuffle of the big one.

3. One task runs far longer than the others. Likely cause?

Data skew: one key holds most of the data; fix with salting, broadcast, or repartitioning.

11.3 Databases and cloud platforms

IN PLAIN WORDS

Different databases are different kinds of storage. A relational database is a tidy set of spreadsheets that follow strict rules and can be combined with joins. A document store keeps flexible records (like JSON) that do not need a fixed shape. A key-value store such as Redis is a very fast sticky-note board for things you look up by name. A vector database stores meaning as numbers so you can ask "find things similar to this". Cloud providers sell all of these as services, and the main skill is knowing which service plays which role.

Why it matters: Interviewers check you can place tools in an architecture, not recite product brochures.

Takeaway: Pick the store by access pattern; know one cloud's equivalents for storage, warehouse, Spark, ML and orchestration.

WORKED EXAMPLE, STEP BY STEP

Row store against column store: count the bytes read

Table: 100 million orders, 20 columns of 8 bytes each, about 16 GB. Query: `SELECT SUM(amount) FROM orders WHERE order_date ≥ '2025-03-01' AND order_date < '2025-04-01'`.

- 1. Row store with no useful index** (a typical transactional database): rows are stored whole, so it reads every row, all 20 columns: about **16 GB**.
- 2. Column store** (a warehouse such as BigQuery or Snowflake): each column is stored separately, so it reads only `order_date` and `amount`: $100\text{M} \times 2 \times 8 \text{ bytes} = \mathbf{1.6 \text{ GB}}$, and compression usually shrinks that a lot further.
- 3. Column store, partitioned by month:** it reads only March's files, about one twelfth: roughly **133 MB**.

4. Why not use the warehouse for everything? Inserting one order means touching 20 separate column files. The row store writes one row in one place and guarantees the transaction (payment and stock change together). Transactions belong in the row store.

5. The usual architecture: the app writes to Postgres; a nightly or streaming pipeline copies data to the warehouse; analysts and dashboards query the warehouse.

What this shows. Choose the store by how it is read and written: many small writes need a row store; big scans of a few columns need a column store.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Pick a store for each: (a) the current contents of each user's shopping cart, read on every page view; (b) five years of clickstream for model training; (c) monthly revenue by category for a dashboard; (d) placing an order that must decrement stock and record the payment together.

Solution. (a) A key-value store or cache, such as Redis: very fast lookups by user id. (b) Object storage (a data lake) in Parquet: cheap, scalable, readable by Spark. (c) A columnar warehouse: large aggregations over a few columns. (d) A relational database with ACID transactions, so the order, stock and payment change together or not at all.

THE FUNDAMENTALS

11.3 Databases

- **Relational:** PostgreSQL, MySQL (ACID, joins).
- **NoSQL:** MongoDB (document), Redis (in-memory key-value, caching), Cassandra (wide-column, write-heavy), Neo4j (graph), Elasticsearch (search).
- **CAP theorem:** in a partition choose consistency or availability.
- **Vector databases** for embeddings (FAISS, Pinecone, Qdrant, Weaviate, pgvector).

11.4 Cloud (know one platform reasonably well)

Need	AWS	Azure	GCP
Object storage	S3	Blob/ADLS	Cloud Storage
Warehouse	Redshift, Athena	Synapse	BigQuery
Managed Spark	EMR, Glue	Databricks, Synapse	Dataproc, Dataflow
ML platform	SageMaker	Azure ML	Vertex AI
Serverless	Lambda	Functions	Cloud Functions/Run
Orchestration	MWAA, Step Functions	Data Factory	Composer

Indian job market note: AWS and Azure are most common; Databricks + Azure is big in services/analytics firms; GCP/BigQuery is common in startups.

SAY IT IN 60 SECONDS

"Pick the database by access pattern. Relational databases like Postgres give tables, joins and ACID transactions, the default for structured data. NoSQL types: key-value for fast lookups, document stores for flexible JSON, wide-column for huge write loads, graph for relationships. Warehouses such as BigQuery and Snowflake are columnar, built for large analytical scans. On the cloud, the pieces map: object storage like S3 for files, a warehouse for analytics, a managed Spark service for big processing, a model serving service for ML. The CAP theorem says a distributed system under a network partition must choose between consistency and availability. The catch: say what the workload is, transactions or analytics, before naming a technology."

KEY TERMS relational · NoSQL · MongoDB · Redis · Cassandra · CAP theorem · vector database · S3 · BigQuery · Redshift · SageMaker

HANDS-ON PRACTICE Pick one cloud and draw the architecture for a daily batch pipeline and a real-time inference API using its services.

COMMON MISTAKES Citing CAP as "pick any two" without context · Using a NoSQL store without knowing the access pattern · Listing cloud services without describing what you built

CHECK YOURSELF**1. Why not run the heavy dashboard queries on the production Postgres?**

They would slow down live transactions; analytics belongs in a separate columnar warehouse optimised for scans.

2. Row store versus column store?

Row stores keep whole records together (good for transactions); column stores keep each column together (fast aggregations and compression for analytics).

3. What does CAP say in practice?

During a network partition you must sacrifice either consistency or availability; systems choose by use case.

11.4 Developer tools: Git, Linux, environments, Docker

IN PLAIN WORDS

Git is a time machine and a teamwork tool for code: every save point (commit) is remembered, and branches let you try ideas without breaking the main version, merging them back when they work. The Linux command line is the quickest way to poke at files and servers. Virtual environments and Docker make sure "it works on my machine" also means "it works on yours": a container is a sealed box holding your code and everything it needs.

Why it matters: These are table stakes for working with engineers, and interviewers check the basics quickly.

Takeaway: Commit small, branch freely, never commit secrets, and package environments for reproducibility.

WORKED EXAMPLE, STEP BY STEP**A branch, a pull request and a merge conflict**

You and a teammate both edit `features.py` on separate branches.

- 1. Start a branch from up-to-date main:** `git switch main && git pull`, then `git switch -c feature/recency`.
- 2. Commit small, meaningful changes:** `git add -p` (review each change), then `git commit -m "Add recency_days feature"`.
- 3. Push and open a pull request:** `git push -u origin feature/recency`. A reviewer comments; you push fixes to the same branch.
- 4. Meanwhile your teammate's branch is merged,** and it changed the same line. GitHub reports a conflict.
- 5. Resolve it locally:** `git switch feature/recency && git merge main`. The file now contains:

```
<<<<<< HEAD
WINDOW_DAYS = 30
=====
WINDOW_DAYS = 28
>>>>>> main
```

Decide with your teammate which value is right (or combine them), delete the markers, run the tests, then `git add features.py` and `git commit`. **6. Push.** The pull request is now mergeable.

What this shows. A conflict is Git asking a human to choose between two edits of the same lines. Small, frequent merges keep conflicts small.

YOUR TURN (COVER THE SOLUTION FIRST)

1. This Dockerfile rebuilds slowly every time you change one line of Python. Why, and how do you reorder it?

```
FROM python:3.11-slim
COPY . /app
RUN pip install -r /app/requirements.txt
CMD ["python", "/app/serve.py"]
```

Solution. Docker caches each step and reuses a cached step only if nothing before it changed. `COPY . /app` copies all your code, so any code edit invalidates that step and every step after it, including the slow `pip install`. Copy the requirements file first, install, then copy the code: `COPY requirements.txt /app/`, `RUN pip install -r /app/requirements.txt`, then `COPY . /app`. Code edits now reuse the cached install.

THE FUNDAMENTALS**Git/GitHub**

- `clone`, `add`, `commit`, `push`, `pull`, `branch`, `checkout/switch`, `merge` vs `rebase`, `stash`, `reset` vs `revert`, resolving conflicts, pull requests, `.gitignore`, meaningful commit messages.

- Never commit secrets/data files; use `.env` and `.gitignore`.

Linux and shell: `ls`, `cd`, `grep`, `find`, `head`, `tail`, `awk`, `sed`, `wc`, `sort`, `uniq`, `chmod`, `ps`, `top`, `cron`, pipes, SSH.

Environments: `venv` / `conda` / `poetry`, `requirements.txt`, reproducibility (pin versions, set seeds).

Docker: image vs container, Dockerfile, layers, volumes, ports, `docker-compose`.

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

SAY IT IN 60 SECONDS

"Four tools come up. Git tracks changes: I make a branch, commit small logical changes, push, and open a pull request; merge combines histories, rebase replays my commits on top of another branch for a linear history; I never rebase shared branches. Linux basics: `ls`, `grep`, `find`, `head`, `tail`, `chmod`, and pipes to chain commands. Environments isolate dependencies: a virtual environment with a pinned requirements file, so the project runs the same on any machine. Docker packages the code and its environment into an image that runs identically anywhere. The catch: reproducibility is the common thread, so I pin versions, set random seeds, and keep secrets out of the repository."

KEY TERMS git branch merge rebase · reset vs revert · stash · gitignore · Linux commands · pipes · cron · virtual environments · Docker image vs container · Dockerfile

HANDS-ON PRACTICE Create a branch, make conflicting edits, merge, resolve the conflict, and write a proper commit message.

COMMON MISTAKES Committing secrets or large data files · Rewriting shared history · Not pinning dependency versions

CHECK YOURSELF

1. Merge versus rebase?

Merge keeps both histories with a merge commit; rebase rewrites your commits on top of the target for a linear history. Never rebase shared branches.

2. Why use Docker for a model service?

The image bundles code, libraries and system dependencies, so it runs the same in development and production.

3. You committed an API key. What now?

Revoke and rotate the key immediately (history keeps it), then remove it from the code and use environment variables or a secrets manager.

11.5 BI tools, Excel and self-check

IN PLAIN WORDS

BI tools (Tableau, Power BI) turn tables into interactive dashboards for people who will never write SQL. The two building blocks are dimensions (the labels you slice by: city, month, product) and measures (the numbers you add up: revenue, orders). A good dashboard is like a car dashboard: a few clear numbers at the top, a trend, a breakdown, and nothing that does not help you make a decision. Excel remains everywhere in analytics firms, so lookups, pivot tables and conditional sums are expected.

Why it matters: Analytics and services companies test these tools heavily, and communication through dashboards is part of the job.

Takeaway: Start from the decision and audience, keep definitions consistent, and show few well-chosen KPIs.

WORKED EXAMPLE, STEP BY STEP

A lookup and a pivot, mapped to SQL

Sheet `Orders` (columns A to C: `order_id`, `user_id`, `amount`) and sheet `Users` (columns A and B: `user_id`, `city`).

- 1. Bring the city into Orders:** in D2, `=XLOOKUP(B2, Users!A:A, Users!B:B, "not found")`. For each order it finds the user's row and returns the city. This is a **LEFT JOIN** of orders to users; "not found" plays the role of NULL.
- 2. Older equivalent:** `=VLOOKUP(B2, Users!A:B, 2, FALSE)`. The FALSE means exact match; forgetting it returns wrong cities silently.
- 3. Insert a pivot table** with City as rows and Sum of amount as values. That is `SELECT city, SUM(amount) FROM ... GROUP BY city`.
- 4. Add Count of order_id** to the values area: `COUNT(order_id)`.
- 5. Check the total** against `=SUM(Orders!C:C)`. If they differ, some cities are blank or "not found", which is an unmatched join.
- 6. The usual #N/A causes:** an id stored as text in one sheet and as a number in the other, or a trailing space (`=TRIM()` fixes it).

What this shows. Spreadsheet skills are SQL ideas with a different interface: a lookup is a join, a pivot is a GROUP BY.

YOUR TURN (COVER THE SOLUTION FIRST)

- Write the Excel formula for "total amount of delivered orders in Pune", with amounts in C, city in D and status in E. What is the SQL equivalent?

Solution. `=SUMIFS(C:C, D:D, "Pune", E:E, "delivered")`. SUMIFS adds column C only where every condition holds. SQL: `SELECT SUM(amount) FROM orders_with_city WHERE city = 'Pune' AND status = 'delivered'`.

THE FUNDAMENTALS

11.6 BI tools and Excel (essential at analytics and services companies)

- Tableau/Power BI:** data connections, dimensions vs measures, calculated fields, LOD expressions (Tableau) / DAX (`CALCULATE`, `FILTER`, time intelligence), filters, parameters, drill-downs, dashboard design (KPI cards on top, trends, breakdowns).

- **Excel:** VLOOKUP/XLOOKUP/INDEX-MATCH , pivot tables, SUMIFS/COUNTIFS , IF , text and date functions, conditional formatting, data validation, Power Query basics.
- **Dashboard principles:** define audience and decisions first; fewer, well-chosen KPIs; consistent definitions; performance.

11.7 Self-check

- ☐ Explain a typical ELT pipeline and where data quality checks belong.
- ☐ Write a PySpark window + join query and explain which steps shuffle.
- ☐ Explain branching and merge conflict resolution in Git.

SAY IT IN 60 SECONDS

"BI tools such as Tableau and Power BI connect to data and let people explore dashboards. A good dashboard answers specific questions, with the key metric first, simple charts, filters, and clear definitions of every metric. Excel remains common: know VLOOKUP or better XLOOKUP, pivot tables, SUMIFS and conditional formatting. A pivot table is a groupby: rows are groups, values are aggregates. The SQL mapping holds here too, since a lookup is a left join. Interviewers check whether I can turn a business question into a clear view. The catch: a dashboard nobody uses is waste, so I ask who will decide what from it, and agree the metric definitions first."

KEY TERMS Tableau · Power BI · DAX CALCULATE · LOD expressions · measures vs dimensions · XLOOKUP · pivot tables · SUMIFS · dashboard design

HANDS-ON PRACTICE Build a one-page sales dashboard with KPI cards, a trend and a breakdown, and write the decision it supports.

COMMON MISTAKES Putting too many charts and KPIs on one page · Inconsistent metric definitions across dashboards · Using VLOOKUP with approximate match by accident

CHECK YOURSELF

1. Excel pivot table equals which SQL operation?

GROUP BY with aggregates, with the column field making it a pivot (conditional aggregation).

2. What makes a dashboard useful?

It answers a specific decision, shows the key metric first, uses consistent definitions and simple charts, and has the right filters.

3. VLOOKUP returns #N/A for some rows. Likely reasons?

The key is missing in the lookup table, or type and whitespace mismatches (text vs number, trailing space); also an exact-match flag omitted.

Interview tips, tricks and hacks

1. **Answer "have you used X?" honestly and specifically.** "I have not used Airflow in production; I understand it schedules dependent tasks as a graph, and I built a small pipeline with it in a project." Honest and precise beats vague and inflated.
2. **Explain each tool by the problem it solves.** Docker: "it works on my machine" problems. Git: several people changing the same code. Spark: data too big for one machine. Airflow: steps that must run in order, reliably.

3. **Give the ETL versus ELT difference in one breath.** Where the transformation happens, and why keeping raw data in the warehouse is useful.
4. **Default to batch.** If asked batch or streaming, choose batch unless there is a real need for seconds-level freshness, and say that streaming costs complexity.
5. **For Spark, remember three phrases.** Lazy evaluation, shuffles are expensive, broadcast the small table.
6. **Choose databases by access pattern.** Transactions: relational. Large aggregations: columnar warehouse. Fast lookups: key-value. State the workload first.
7. **Know the Git flow you actually use.** Branch, commit, push, pull request, review, merge. Be able to explain how you would resolve a merge conflict.
8. **Mention reproducibility unprompted.** Pinned dependency versions, fixed random seeds, a README with run instructions.
9. **If you committed a secret, rotate it.** A classic question: deleting the file is not enough, because history keeps it.
10. **Translate spreadsheet questions into SQL.** Pivot table equals group by; lookup equals left join. It shows the concepts transfer.

Frequently asked questions

9 questions interviewers ask on this chapter, with model answers. Cover the answer, say yours aloud, then compare.

DATA ENGINEERING CONCEPTS

Q1 Your daily load job failed halfway and someone reran it. The next day the revenue dashboard shows double counts. What went wrong and how do you prevent it?

The first run wrote part of the day's data; the rerun appended everything again, duplicating rows. Make loads **idempotent**: load into a staging table, then `MERGE /upsert` on a unique key, or **overwrite the partition** for the run date in one atomic step. Add a **uniqueness check** on the primary key and a row-count reconciliation against the source, and alert when checks fail.

PYSPARK

Q2 Why is `collect()` dangerous?

Pulls all data to the driver; OOM.

Q3 Reduce shuffle?

Broadcast small tables, pre-filter, proper partition keys, avoid unnecessary `groupByKey`.

Q4 Data skew fix?

Salting, AQE skew join, broadcast.

Q5 repartition vs coalesce ?

Repartition does a full shuffle (increase/decrease); coalesce reduces partitions without a full shuffle.

Q6 A Spark job has 199 tasks finishing in seconds and one task running for an hour. What is happening and what do you try?

One partition holds a disproportionate share of rows, usually because a join or group-by key is highly skewed (for example a default `null` or one giant merchant). Look at the Spark UI's task-time distribution and key counts; then enable **AQE skew join, broadcast** the smaller table if it fits, **salt** the hot key, filter or pre-aggregate the skewed key separately, or change the partitioning key.

DATABASES AND CLOUD PLATFORMS

Q7 You need to serve real-time features (a user's order count in the last hour) to a fraud model with a 50 ms latency budget. Where do you store them?

A warehouse or relational database cannot reliably answer within 50 ms at high throughput, so keep **precomputed** features in an in-memory **key-value store** (Redis) or a feature store's online store, updated by a streaming job (Kafka + Flink/Spark Structured Streaming). Keep the same feature definitions for training (offline store) to avoid **train-serve skew**, set TTLs, and monitor freshness and latency.

DEVELOPER TOOLS: GIT, LINUX, ENVIRONMENTS, DOCKER

Q8 You accidentally committed a file containing an API key and pushed it to GitHub. What do you do?

Treat the key as compromised the moment it is pushed: **revoke/rotate it first** (bots scan public repositories within minutes), then remove the secret from the repository and its history (for example with `git filter-repo`), add the file to `.gitignore`, move secrets to environment variables or a secrets manager, and consider a pre-commit secret scanner. Deleting the file in a new commit is not enough because history still contains it.

BI TOOLS, EXCEL AND SELF-CHECK

Q9 A stakeholder asks for "all the numbers" on one dashboard. What do you do?

Interview the stakeholder about the **decisions** the dashboard should enable and how often they will look at it. Propose a one-page summary with a few KPIs, a trend and the top breakdowns, each with a precise definition, plus secondary pages for detail. A dashboard that shows everything supports no decision; document the metric definitions and owners so numbers agree across reports.

DATA ENGINEERING CONCEPTS

- **ETL**: extract, transform, then load. **ELT**: load raw data first, transform inside the warehouse (common with cloud warehouses); keeps raw data for reprocessing.

Store	What	Use
Data lake	raw files in cheap object storage (S3, GCS)	all data, ML, schema-on-read
Data warehouse	modelled, columnar tables (BigQuery, Snowflake, Redshift)	BI, SQL analytics, schema-on-write
Lakehouse	lake + warehouse features (Delta, Iceberg, Hudi)	both

- **Batch** (nightly/hourly) vs **streaming** (events as they arrive: Kafka, Kinesis, Flink, Spark Structured Streaming). Choose batch unless latency truly matters.
- **Orchestration**: Airflow/Dagster/Prefect schedule tasks as a DAG with retries and alerts. **dbt** for SQL transformations with tests.
- **Good pipelines: idempotent** (re-running gives the same result), incremental loads, backfills, schema checks, data-quality tests (nulls, duplicates, freshness, row counts), monitoring and lineage.
- **File formats**: CSV/JSON (row text, slow), **Parquet/ORC** (columnar, compressed, fast scans, predicate pushdown), Avro (row, schema evolution, streaming). Partition by date; avoid tiny files.
- **Modelling**: star schema = fact table (events, measures) + dimension tables (who, what, when); **SCD type 2** keeps history of changing dimensions; normalised for OLTP, denormalised for analytics.
- **CDC** (change data capture) streams database changes; **data contracts** define schemas between teams.

PYSPARK

- Distributed processing: data split into **partitions** across executors; the driver plans work. **Lazy evaluation**: transformations (select, filter, join, groupBy) build a plan; **actions** (count, show, collect, write) run it.
- **Narrow** transformations stay within a partition; **wide** ones (join, groupBy, distinct, orderBy) cause a **shuffle** (network + disk): the main cost.

```
from pyspark.sql import SparkSession, functions as F
spark = SparkSession.builder.getOrCreate()
df = spark.read.parquet("s3://b/events/")
out = (df.filter(F.col("dt") >= "2025-01-01")
      .groupBy("user_id")
      .agg(F.count("*").alias("n"),
           F.sum("amt").alias("rev"))
      .join(F.broadcast(users), "user_id", "left"))
out.write.mode("overwrite").parquet("o/")
```

- **Speed**: filter and select early, **broadcast** the small side of a join, avoid wide ops you do not need, use built-in functions (not Python UDFs), cache reused data (df.cache()), repartition sensibly (coalesce to reduce), fix **skew** (one huge key: salting, broadcast, AQE).
- **collect()** pulls all data to the driver: can crash. DataFrames are optimised by Catalyst; RDDs are lower level.
- **Window**:

```
F.row_number().over(Window.partitionBy("g").orderBy("d"))
```

DATABASES

Type	Examples	Good for
Relational (SQL)	Postgres, MySQL	transactions, joins, ACID
Key-value	Redis, DynamoDB	fast lookups, caching, sessions
Document	MongoDB	flexible JSON
Wide-column	Cassandra, HBase	huge write volume
Graph	Neo4j	relationships
Columnar warehouse	BigQuery, Snowflake	analytic scans
Vector	pgvector, Pinecone, FAISS	embedding search

- **OLTP** many small transactions (row store); **OLAP** big aggregations (column store).
- **CAP**: under a network partition a distributed system must choose consistency or availability. **ACID** vs **BASE** (eventual consistency). Sharding splits data across machines; replication copies it for reliability.
- Choose by access pattern: transactions → relational; scans → warehouse; fast lookup → key-value; files → object storage.

CLOUD SERVICE MAP

- Format: need = **AWS / GCP / Azure**.
- Object storage = S3 / Cloud Storage / Blob Storage.
- Warehouse = Redshift / BigQuery / Synapse.
- Managed Spark = EMR, Glue / Dataproc / Databricks.
- ML platform = SageMaker / Vertex AI / Azure ML.
- Serverless functions = Lambda / Cloud Functions / Functions.
- Streaming = Kinesis, MSK / Pub/Sub / Event Hubs.
- Orchestration = MWAA, Step Functions / Composer / Data Factory.
- IAM = identities and permissions (least privilege). Cost levers: right-size, spot instances, partition and prune, storage tiers, turn clusters off.

GIT

```
git clone URL; git status; git diff
git add -p; git commit -m "msg"
git switch -c feature/x
git push -u origin feature/x
git pull --rebase; git fetch
git merge main; git rebase main
git log --oneline --graph; git blame f
git stash; git stash pop
git restore f; git reset --soft HEAD~1
git revert SHA # safe undo on
shared branch
git cherry-pick SHA; git tag v1.0
```

- Branch → small commits → push → pull request → review → merge. **Merge** keeps history with a merge commit; **rebase** replays your commits for a linear history: never rebase shared branches.
- Conflict: edit markers <<<<<<, pick the right content, git add, continue.
.gitignore for data, secrets, .venv. A committed secret is leaked: **rotate it**, then remove it.

LINUX AND SHELL

```
ls -lah; cd; pwd; mkdir -p d; rm -r d
head -n 5 f; tail -f log; less f; wc -l f
grep -rn "err" . | sort | uniq -c | sort -nr
cut -d, -f1,3 f.csv; awk -F, '{s+=$2} END{print s}' f
find . -name "*.py"; du -sh *; df -h
chmod +x run.sh; ps aux | grep py; kill -9 PID
cat a.txt | tr a-z A-Z > b.txt; command >> log 2>&1
ssh user@host; scp f host:/path; curl -O URL
export VAR=1; env; which python; cron: 0 2 * * *
```

Pipes | chain commands; > overwrites, >> appends, 2>&1 merges errors.

ENVIRONMENTS AND DOCKER

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt; pip freeze > requirements.txt
conda create -n x python=3.11; conda activate x
docker build -t app .; docker run -p 8000:8000 app
docker ps; docker logs -f ID; docker exec -it ID sh
docker compose up -d
```

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0"]
```

- Image** = blueprint; **container** = running instance. Copy requirements first to use layer caching. Same image runs everywhere: reproducibility. Keep secrets out of images (env vars/secret managers). Pin versions, set random seeds.

BI TOOLS AND DASHBOARDS

- Tableau / Power BI / Looker** connect to data, build charts, filters, calculated fields, and dashboards. Know: measures vs dimensions, extracts vs live connections, LOD expressions/DAX basics.
- A good dashboard answers specific questions: key metric first, trend, breakdown, filters, consistent definitions, last-refreshed date, few colours.
- Ask who decides what from it; agree metric definitions; avoid vanity metrics.

EXCEL ESSENTIALS

- Lookups: XLOOKUP(key, keys, vals) (or VLOOKUP(key, table, col, FALSE); INDEX/MATCH). #N/A = key missing, type mismatch or stray space (TRIM).
- Aggregation: SUMIFS, COUNTIFS, AVERAGEIFS; logic IF, IFS, IFERROR; text LEFT/MID/RIGHT, TEXTJOIN, SPLIT via Text-to-columns; dates EOMONTH, DATEDIF, TEXT(d, "yyyy-mm").
- Pivot tables** = GROUP BY (rows = groups, values = aggregates, columns = pivot); slicers filter; refresh after data changes. Lookup = left join.
- Tips: absolute references \$A\$1, named ranges, Ctrl+Shift+arrows, data validation, conditional formatting, Power Query for repeatable cleaning.

PIPELINES IN PRACTICE

- Incremental load / upsert:** MERGE INTO tgt USING src ON key WHEN MATCHED THEN UPDATE ... WHEN NOT MATCHED THEN INSERT ...; keep a **watermark** (last processed timestamp); make reruns safe with dedupe on a key.

- Streaming:** event time vs processing time; **windows** (tumbling, sliding, session); **watermarks** for late data; delivery semantics at-most-once, at-least-once, **exactly-once** (idempotent writes + transactions). Kafka: topics, partitions, consumer groups, offsets.
- Data quality checks:** not-null, unique keys, accepted values, referential integrity, freshness, row-count and distribution drift (Great Expectations, dbt tests).

```
SELECT COUNT(*) - COUNT(DISTINCT id) AS
dups,
SUM(CASE WHEN amt IS NULL THEN 1
ELSE 0 END) AS nulls,
MAX(loaded_at) AS freshness
FROM fact_orders;
```

- APIs and JSON:** REST verbs GET/POST/PUT/DELETE, status codes (200 ok, 400 client error, 401/403 auth, 404, 429 rate limit, 500 server error); requests.get(url, timeout=10).json(); paginate and retry with backoff.
- CI/CD:** run tests and linters on every push (GitHub Actions); build the image; deploy automatically to staging, then production.
- Testing:** unit tests for transformations, sample-data integration tests, schema tests.

COST AND PERFORMANCE HABITS

- Store as Parquet, partition by date, prune partitions in queries, avoid SELECT * on columnar warehouses (pay per bytes scanned).
- Right-size clusters, auto-terminate idle ones, use spot capacity for batch, cache hot results.
- Monitor job duration, failures, data freshness, and cost per run.

SAY IT IN 20 SECONDS

- "ELT loads raw first and transforms in the warehouse."
- "Spark is lazy; shuffles are the cost; broadcast the small table."
- "Choose the database by access pattern: transactions, scans, lookups."
- "Merge keeps history, rebase rewrites it; never rebase shared branches."
- "Docker bundles code and environment so it runs the same everywhere."

12

CHAPTER 12 · MEDIUM PRIORITY · MODULE 12 IN THE APP

MLOps and Deployment Basics

A model that lives only in a notebook creates no value. This chapter is about getting a model into use and keeping it healthy once it is there.

3 topics · about 1 h 7 min in the app · 6 questions answered · two-page cheat sheet at the end

12.1 The ML lifecycle

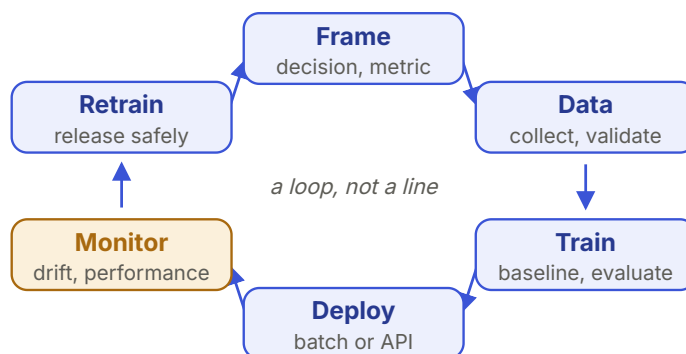
12.3 MLOps Q&A

12.2 MLOps key concepts: serving, drift, monitoring

Foundations

Most machine learning projects that fail do not fail because the model was inaccurate. They fail in the gap between a promising notebook and a system that people rely on. **MLOps** is the set of practices that closes that gap. For a junior interview you need the shape of the lifecycle, the vocabulary for what goes wrong in production, and a sensible answer to "your model's performance dropped: what do you do?".

A loop, not a line



Most projects fail at the hand-over from notebook to production.

Monitoring sends you back to data and features: plan for it from day one.

Figure 14. Deployment is the middle of the story. Monitoring leads back to data.

Training a model is one step in a cycle. You frame the problem, prepare data, train and evaluate, deploy, monitor, and then retrain when the world changes. The world always changes: customers behave differently, products are launched, fraudsters adapt. A model is a snapshot of the past and it slowly goes out of date, which is why the work is never finished at deployment.

Two ways to serve predictions

In **batch** serving, the model scores many records on a schedule, perhaps every night, and stores the results for other systems to read. It is simple, cheap and easy to debug, and it is the right choice whenever predictions do not need to be fresh to the second, as with a weekly list of customers likely to leave. In **online** serving, the model sits behind an interface that answers one

request at a time within milliseconds, as fraud checks at payment must. Online serving brings engineering concerns: response time, traffic spikes, and what to do when the model is unavailable.

The mismatch that quietly breaks models

In training, features are usually computed with one set of code over historical data. In production, they are computed by different code on live data. If the two disagree, even slightly, the model receives inputs unlike those it learned from, and accuracy falls with no error message. This is **training-serving skew**. The remedy is to define each feature once and use that definition in both places, which is the purpose of a **feature store**.

Drift: when the world moves

Data drift means the inputs have changed: the customers arriving now are older, or come from a new region. **Concept drift** means the relationship itself has changed: the same customer behaviour now signals something different, as when a new payment method changes what fraud looks like. Data drift can be detected by comparing the current distribution of each feature with the training distribution. Concept drift shows up as falling performance, but only once the true outcomes are known, and that can take weeks: you do not learn whether a loan defaulted for months. So in the meantime you watch the inputs, the distribution of predictions, and business indicators.

Releasing without breaking things

New models are introduced cautiously. In **shadow mode** the new model makes predictions on real traffic that nobody acts on, so it can be compared with the current one. In a **canary release** a small share of traffic goes to the new model first. An **A/B test** measures the effect on the business metric. In every case the previous model stays ready, so that rolling back takes minutes.

Reproducibility and responsibility

You must be able to say exactly which code, data and settings produced the model that made a given prediction. That requires versioning all three and recording every experiment. And a deployed model affects people, so teams check that error rates are similar across groups, that decisions can be explained where the law requires it, and that personal data is protected.

Topic by topic

Each topic gives the idea in plain words, the fundamentals to know, a model spoken answer and three questions to test yourself. Open the same topic in the app for worked examples, runnable code and interactive demos.

12.1 The ML lifecycle

IN PLAIN WORDS

A machine learning project is not "build a model"; it is a loop. You decide what problem is worth solving, collect and clean data, build and test models, package the best one, put it to work, watch it in the real world, and retrain when the world changes. Most of the effort is in the parts before and after the modelling, and most failures happen there too.

Why it matters: Interviewers for beginner roles expect you to understand the path a model takes to production, even if you do not build it yourself.

Takeaway: Frame, collect, explore, engineer, model, package, deploy, monitor, retrain.

WORKED EXAMPLE, STEP BY STEP

From notebook to a weekly churn list

A churn model scores 0.81 AUC in a notebook. Make it useful.

1. **Agree the action.** Every Monday the retention team calls the 500 customers with the highest risk. That makes it a **batch** job: no live API is needed.
2. **Turn the notebook into scripts:** `features.py` (computes features as of a date), `train.py`, `score.py`. Same feature code for training and scoring.
3. **Version everything:** code in Git, the training data snapshot (date and query), the model file with its metrics, and the environment (requirements file or Docker image).
4. **Schedule it:** each Monday at 05:00, compute features as of Sunday, score all active customers, write the top 500 with scores to a table the team's tool reads.
5. **Monitor it:** row counts, missing-value rates, the score distribution compared with training, and, once outcomes are known four weeks later, precision among the called customers.
6. **Close the loop:** compare churn among called customers with a small random holdout who were not called. That measures the *business* impact, not just the model's accuracy.
7. **Retrain** monthly, or when monitoring shows drift, and release the new model only if it beats the current one on recent data.

What this shows. Deployment is mostly plumbing and agreements: an action, a schedule, the same features in training and scoring, monitoring and a holdout.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Six months later a teammate asks: "Which data and code produced the scores on 3 March?"
List what you must have recorded to answer exactly.

Solution. The Git commit of the scoring and feature code; the model version or file hash used that day, with its training data snapshot and training date; the environment (image tag or pinned requirements); the as-of date and input snapshot used for features; and the run log (when it ran and row counts). With these you can rerun 3 March and get identical scores.

THE FUNDAMENTALS

Problem framing → Data collection → EDA/cleaning → Feature engineering →
Modeling/validation
→ Packaging → Deployment (batch/real-time) → Monitoring → Retraining

SAY IT IN 60 SECONDS

"The ML lifecycle goes from business problem to ongoing operation. Define the problem and the success metric. Collect and clean data. Build features and a baseline. Train and evaluate with proper validation. Deploy, as a batch job or an online service. Monitor performance and data drift. Retrain when things change. It is a loop, not a line: monitoring sends you back to data and features. Most effort goes to data and operations, not to model code. Version data, code and models so any result can be reproduced, and write tests for the pipeline. The catch: a model in a notebook has no value until it is deployed and monitored, and many projects fail at that handover."

KEY TERMS ML lifecycle · problem framing · deployment · monitoring · retraining

HANDS-ON PRACTICE Draw the lifecycle for a churn model and annotate each stage with one risk and one check.

COMMON MISTAKES Treating deployment as the end of the project · Not defining how success will be measured before building

CHECK YOURSELF**1. Why is the lifecycle a loop?**

Monitoring reveals drift or degradation, which triggers new data, features and retraining.

2. What would you version besides code?

Data, features, model artefacts and configuration, so any prediction is reproducible.

3. Why start with a baseline?

It sets the bar to beat, checks the pipeline end to end, and shows whether complexity is justified.

12.2 MLOps key concepts: serving, drift, monitoring**IN PLAIN WORDS**

Once a model leaves the notebook, four things matter: how it is served (a nightly list, or an instant answer while the customer waits), whether the world is changing underneath it (new customers behave differently), whether the code computes inputs the same way in

production as in training, and whether you can roll back quickly. Monitoring is like a health check-up for the model: are the inputs still looking normal, are the predictions still sensible, and when the truth finally arrives, is the model still right?

Why it matters: "How do you know a model still works in production?" is a classic mid-level question.

Takeaway: Serve appropriately, track everything, watch for drift and skew, and roll out gradually.

WORKED EXAMPLE, STEP BY STEP

Measure drift with PSI

A model was trained when customers' ages split evenly across four buckets. This month's share per bucket has moved.

Age bucket	Training share (expected)	This month (actual)	(actual – expected) × ln(actual / expected)
18 to 25	25%	15%	$(-0.10) \times \ln(0.60) = 0.051$
26 to 35	25%	20%	$(-0.05) \times \ln(0.80) = 0.011$
36 to 45	25%	30%	$(+0.05) \times \ln(1.20) = 0.009$
46+	25%	35%	$(+0.10) \times \ln(1.40) = 0.034$
		PSI	0.105

- 1. Bucket the feature** using the training distribution, so each bucket has a known expected share.
- 2. Compare this month's shares** with the expected ones.
- 3. Each term is positive** whenever a share has moved, in either direction. The sum is the population stability index.
- 4. Rule of thumb:** below 0.1 is stable; 0.1 to 0.25 is a moderate shift; above 0.25 is a major shift. Here **0.105**: moderate. The customer base is getting older.
- 5. What to do:** check whether performance has dropped on recent labelled data, look for a cause (a marketing campaign aimed at older customers?), and plan a retrain if the shift continues.

What this shows. Drift monitoring compares today's inputs with what the model learned from. It warns you before labels arrive to confirm a performance drop.

YOUR TURN (COVER THE SOLUTION FIRST)

1. A delivery-time model performs well offline but badly in production. You find that in training `distance` was in kilometres, while the live app sends metres. What is this problem called, and how do you prevent it in future?

Solution. Training-serving skew: the feature is computed differently in training and in serving, so the model receives values about 1,000 times larger than any it saw. Prevent it by computing features with one shared piece of code (or a feature store) for both paths, validating the schema and value ranges of live inputs, and logging served features to compare their distributions with training.

THE FUNDAMENTALS

Topic	What to know
Batch vs real-time inference	Batch: scheduled scoring (daily churn list). Real-time: API with latency SLA (fraud at payment).
Serving a model	Save with <code>joblib</code> / <code>pickle</code> / ONNX; wrap in FastAPI/Flask ; containerize with Docker; deploy on cloud (Cloud Run, ECS, Lambda, SageMaker endpoints, Azure ML).
Experiment tracking	MLflow, Weights and Biases: log params, metrics, artifacts, model versions.
Reproducibility	Seeds, pinned dependencies, data versioning (DVC), config files.
Feature stores	Consistent features across training/serving (Feast, Tecton).
Train-serve skew	Differences between training and production feature computation.
Monitoring	Latency, errors, data drift (input distribution change: PSI, KS test), concept drift (relationship X-to-y changes), performance decay (needs labels; use proxies when delayed).
Retraining	Scheduled vs triggered by drift/performance drops.
CI/CD	Tests (unit tests for data/feature code), automated pipelines, model registry, staged rollout (shadow, canary, A/B).
Responsible ML	Fairness, bias audits, explainability (SHAP), privacy (PII masking, India's DPDP Act awareness), model cards.

PSI rule of thumb: < 0.1 stable, 0.1-0.25 moderate shift, > 0.25 significant shift.

```
# Minimal model API
from fastapi import FastAPI
from pydantic import BaseModel
import joblib, pandas as pd

app = FastAPI()
model = joblib.load("model.joblib")

class Features(BaseModel):
    age: float
    tenure: float
    monthly_spend: float

@app.post("/predict")
def predict(f: Features):
    X = pd.DataFrame([f.dict()])
    return {"churn_probability": float(model.predict_proba(X)[0, 1])}
```

SAY IT IN 60 SECONDS

"Serving can be batch, where predictions are computed on a schedule and stored, or online, where an API answers per request in milliseconds. Batch is simpler; online is for real-time needs. Training-serving skew happens when features are computed differently at training and serving, and a feature store helps keep them identical. Monitoring tracks system health like latency, data quality, input drift where feature distributions shift, concept drift where the input-output relationship changes, and the model's performance when labels arrive. Safe rollouts use shadow mode, canary releases and A/B tests, with rollback ready. The catch: labels often arrive late, so I monitor input drift and proxy metrics in the meantime."

KEY TERMS batch vs real-time inference · FastAPI · joblib · MLflow · DVC · feature store · train-serve skew · data drift · concept drift · PSI · shadow canary rollout · model card

BUILDS ON 12.1 The ML lifecycle

HANDS-ON PRACTICE Compute the PSI between a training distribution and a live distribution and decide whether to retrain.

COMMON MISTAKES Computing features differently in training and serving · Monitoring only accuracy (which needs labels) · Deploying without a rollback plan

CHECK YOURSELF

1. Data drift versus concept drift?

Data drift: input distribution changes. Concept drift: the relationship between inputs and target changes, so the same inputs now mean something different.

2. Labels arrive 30 days late. How do you monitor now?

Track input and prediction distributions (PSI, KS test), data quality, and proxy business metrics; evaluate on real labels when they arrive.

3. What is a canary release?

Send a small share of traffic to the new model, watch metrics, and expand gradually or roll back.

12.3 MLOps Q&A

IN PLAIN WORDS

This page practises the questions about getting models into the real world. They revolve around four ideas: how predictions are served, how you notice a model going bad, what changed when it did, and how you release changes safely. A strong answer names the failure mode, how you would detect it, and what you would do. Think of the model as a product with a life after training, and answer like someone who has been paged at night.

Why it matters: Employers want models that keep working, and MLOps questions test that maturity.

Takeaway: For each issue: failure mode, detection, action.

SAY IT IN 60 SECONDS

"For an MLOps question I answer in three steps: what can go wrong, how I would detect it, and what I would do. For example, accuracy drops in production. Causes: data drift, concept drift, a broken upstream feature, training-serving skew. Detection: dashboards on feature distributions, missing-value rates and prediction distributions, plus performance on late labels. Action: fix the data pipeline if it is a bug, retrain on recent data if it is drift, roll back to the previous model if it is urgent. Then I add alerts so I find out sooner next time. The catch: I always mention rollback and monitoring as part of the first deployment, not after."

KEY TERMS offline vs online gap · delayed labels · safe rollout · shadow mode · canary

BUILDS ON 12.2 MLOps key concepts: serving, drift, monitoring

HANDS-ON PRACTICE Answer each question in under a minute, naming at least three concrete causes or steps.

COMMON MISTAKES Blaming "the model" without checking the data and the serving path · No rollback plan

CHECK YOURSELF**1. Model performance fell after a week in production. First three checks?**

Input data quality and schema (broken or missing features), feature and prediction distribution shifts versus training, and recent upstream or code changes.

2. How do you retrain safely?

Retrain on a schedule or trigger, validate against the current model on a holdout, release via shadow or canary, and keep rollback available.

3. What is training-serving skew and how do you prevent it?

Features differ between training and serving; share feature code or a feature store and log served features for comparison.

Interview tips, tricks and hacks

- 1. Answer production questions in three parts.** What can go wrong, how I would detect it, what I would do. It works for almost every MLOps question.
- 2. For "performance dropped", start with the data, not the model.** Is the metric computed correctly? Did an upstream field change or go missing? Only then consider drift and retraining.
- 3. Choose batch unless you need real time.** State the reason: simpler, cheaper, easier to debug. Then name the cases that do need online serving.
- 4. Distinguish the two drifts with an example.** Data drift: more users from a new city. Concept drift: the same behaviour now means something else. Interviewers ask this constantly.
- 5. Mention delayed labels.** "True outcomes arrive weeks later, so I would monitor input and prediction distributions in the meantime." It shows practical awareness.
- 6. Always include a rollback.** Any deployment answer should end with shadow or canary release and the ability to revert.
- 7. Say "monitoring" in your project stories.** Even for a course project, describe what you would track after launch. It separates you from candidates who stop at accuracy.
- 8. Version three things.** Code, data, model. And pin the environment.
- 9. Explain training-serving skew simply.** "The model was trained on features computed one way and served features computed another way." Then offer the fix: one shared definition.

Frequently asked questions

6 questions interviewers ask on this chapter, with model answers. Cover the answer, say yours aloud, then compare.

THE ML LIFECYCLE

Q1 Before writing any code for a "reduce churn" request, what do you clarify?

What **decision** will the model change (who gets what offer, when?), the precise **definition of churn** and prediction horizon, the **cost of each type of error**, what data exists **at prediction time**, the capacity to act (how many customers can the retention team contact), how success will be **measured** (A/B test of the intervention, not only model AUC), and deployment constraints (batch weekly or real-time). This framing determines the target, the features, the metric and the validation scheme.

MLOPS KEY CONCEPTS: SERVING, DRIFT, MONITORING

Q2 PSI for your most important feature is 0.31 this week. What do you do?

0.31 is a significant shift. First rule out a **pipeline or data bug** (units, a new default value, missing data, a changed upstream definition). If the change is real (new customer mix, seasonality), check whether **model performance** is affected (recent labelled samples, proxy metrics, calibration); if yes, **retrain** on recent data (or recalibrate), possibly with new features, and roll out gradually. Document the incident and set the alert thresholds accordingly.

MLOPS Q&A

Q3 Model works offline but not online: why?

Train-serve skew, leakage, drift, population mismatch, latency forcing simpler features, feedback loops.

Q4 How do you monitor a model with delayed labels?

Input drift, prediction distribution, proxy metrics, periodic labeled audits.

Q5 How would you roll out a new model safely?

Shadow mode → canary → A/B test → full rollout with rollback.

Q6 You deploy a credit model and, six months later, discover the average score has crept upward. No defaults have matured yet. What do you check?

Input drift (PSI per feature: is the applicant mix changing? a new channel or city?), **data pipeline changes** (a bureau field definition or a new default value), the **score distribution** and approval rate versus expectations, and **early-delinquency proxies** (first-payment default, days past due buckets). Compare with the validation-time distribution, segment by channel, and if the drift is real, plan a recalibration or rebuild using recent data while tightening policy cut-offs until the evidence matures.

THE ML LIFECYCLE

1. **Frame** the problem: decision, metric, cost of errors, constraints (latency, explainability).
 2. **Data**: collect, label, validate, version.
 3. **Features** and a **baseline**; train and evaluate with the right split.
 4. **Package and deploy**: batch job or online service.
 5. **Monitor**: system health, data quality, drift, model performance, business impact.
 6. **Retrain / improve** and release safely; **retire** old models.
- It is a loop: monitoring sends you back to data and features. Most effort is data and operations, not model code. A model in a notebook has no value until it is deployed and monitored.
 - **Version everything**: code (Git), data (DVC, lakeFS, snapshots), features, model artefacts and config, environment (Docker, pinned deps) so any prediction can be reproduced.

SERVING PATTERNS

Pattern	How	Use when
Batch	score on a schedule, store results	recommendations refreshed daily, risk lists; simplest
Online (real-time)	API returns a prediction in ms	fraud at checkout, search ranking
Streaming	score events as they arrive	monitoring, alerts
Edge / on-device	model runs on phone or device	offline, privacy, latency

- Online serving: wrap the model in an API (FastAPI/Flask/BentoML/TorchServe), containerise with Docker, run behind a load balancer on Kubernetes or a managed endpoint (SageMaker, Vertex AI). Concerns: **latency** (p50/p95/p99), throughput, autoscaling, cold starts, timeouts, fallbacks, caching, batching requests, cost.
- Reduce latency: smaller model, distillation, quantisation, ONNX, precomputed features, GPU only if needed.

```
from fastapi import FastAPI
import joblib
app = FastAPI(); model =
joblib.load("m.pkl")
@app.post("/predict")
def predict(x: dict):
    p = model.predict_proba([[x["a"],
x["b"]]])
    return {"score": float(p[0][1])}
```

TRAINING-SERVING SKEW AND FEATURE STORES

- **Skew**: features computed differently in training (SQL on history) and serving (service code), or with data not available at serving time. Causes silent accuracy loss.
- Prevent: **one shared feature definition (feature store)**: Feast, Tecton, Databricks) with an offline store for training and an online store for low-latency serving; point-in-time-correct joins; log served features and compare with training; same preprocessing code in a versioned pipeline artefact.
- Training data must reflect what was known **at prediction time** (no future data), including late-arriving labels.

DRIFT AND DECAY

Type	Meaning	Detect
Data (covariate) drift	input distribution P(X) changes	PSI, KS test, chi-square, summary stats
Label (prior) drift	outcome rate changes	label rate over time
Concept drift	relationship P(y given X) changes	performance drop with labels; error by segment
Upstream change	schema, units, missing rates, new categories	data-quality checks

- **PSI** = $\sum (\text{actual}\% - \text{expected}\%) \times \ln(\text{actual}\% / \text{expected}\%)$ per bucket. Rule of thumb: < 0.1 stable; 0.1-0.25 moderate shift; > 0.25 major shift. Example: shares 50/30/20 → 40/30/30 gives PSI = 0.063.
- Causes: seasonality, product changes, new users, fraud adapting, pipeline bugs. Models decay: plan for it.

MONITORING LAYERS

Layer	What to track
System	latency, errors, throughput, CPU/GPU, memory, cost
Data	schema, missing/null rate, ranges, new categories, freshness, volume
Model	prediction distribution, drift (PSI/KS), confidence, performance on labels
Business	conversion, revenue, complaints, fairness by group

- **Delayed labels** (fraud chargebacks, churn): monitor input and prediction drift and proxy metrics now; compute true performance when labels arrive (evaluate on matured cohorts).
- Alerts with owners and runbooks; dashboards; log inputs, outputs, model version and request id (with privacy controls); sample for human review.

SAFE RELEASE STRATEGIES

- **Shadow mode:** new model scores live traffic silently; compare with current.
- **Canary:** route a small share (1-5%) to the new model; watch metrics; ramp or **roll back**.
- **A/B test:** randomise users, measure the business metric; **interleaving** for ranking. Bandits for fast learning.
- **Blue/green:** two environments, switch traffic, instant rollback. Feature flags for kill switches.
- Always keep the previous model deployable. Validate a new model against the champion on a holdout **before** release (champion/challenger).

RETRAINING

- Triggers: schedule (weekly), drift alert, performance drop, new data volume, business change.
- Retrain pipeline: fetch data → validate - > features → train → evaluate vs champion → register → deploy via canary. Automated but gated.
- **Continuous training (CT)** with CI/CD: tests on code and data, build image, deploy staging, promote.
- Online learning is rarer; beware feedback loops (the model changes the data it later trains on) and label bias.

CI/CD FOR ML AND TESTING

- **CI:** unit tests (feature code), data-validation tests (schema, ranges, no leakage), model tests (beats baseline, no regression on slices, invariance/directional tests), integration test of the API.
- **CD:** build the image, run smoke tests, deploy to staging, promote to production with approval; infrastructure as code.
- Registry stages: **None → Staging → Production → Archived**; each model links to code commit, data version, metrics and owner (**MLflow Model Registry**, SageMaker, Vertex).

EXPERIMENT TRACKING AND REPRODUCIBILITY

```
import mlflow
with mlflow.start_run():
    mlflow.log_params({"lr": 0.1,
"depth": 6})
    mlflow.log_metric("auc", 0.87)
    mlflow.sklearn.log_model(model,
"model")
```

- Track parameters, metrics, artefacts, code version, data version, environment. Fix random seeds; pin dependencies; containerise.
- **Model card:** purpose, data, metrics by segment, limits, risks, owner. Data card for datasets.

KUBERNETES AND SCALING BASICS

- **Pod** = one or more containers; **Deployment** keeps N replicas and does rolling updates; **Service** gives a stable address; **Ingress** routes external traffic; **HPA** autoscales on CPU/QPS; **readiness/liveness probes** gate traffic and restart failures.
- Resource requests/limits (CPU, memory, GPU); node pools for GPU; model files from object storage or baked into the image.
- SLOs: availability (99.9%), latency (p95 < 200 ms), error budget. Load-test before launch; add timeouts, retries with backoff, circuit breakers, and a rule-based fallback.

LLMOPS (GENAI IN PRODUCTION)

- Version **prompts**, retrieval settings and model names like code; run an **evaluation set** on every change (faithfulness, correctness, format validity, safety).
- Monitor token usage, cost per request, latency, refusal and error rates, user feedback (thumbs), hallucination reports; log prompts and outputs with PII controls.

- **Guardrails:** input and output filters, schema validation, tool allow-lists, rate limits, prompt-injection defences; cache frequent answers; route easy queries to a smaller model.
- Vector index freshness and re-embedding when the embedding model changes.

RESPONSIBLE ML AND SECURITY

- **Fairness:** compare error rates and outcomes across groups; remove proxies carefully; document trade-offs. **Explainability:** SHAP, reason codes (regulated lending). **Privacy:** minimise data, anonymise, access control, consent; **adversarial** inputs and model extraction; prompt injection for LLMs.
- Human in the loop for high-stakes decisions; fallback rules when the model is down or uncertain.

INCIDENT RUNBOOK: ACCURACY DROPPED

1. Confirm it is real (metric definition, delayed labels, volume).
2. Check data quality and upstream changes (schema, null rates, units).
3. Compare feature and prediction distributions with training; find the shifted features.
4. Check recent releases (code, model, config) and segments.
5. Mitigate: roll back, fall back to rules, retrain; then fix the root cause and add the missing alert.

SAY IT IN 20 SECONDS

- "Batch is simple; online is for real-time needs."
- "Skew means training and serving compute features differently; a feature store fixes it."
- "Data drift is inputs changing; concept drift is the relationship changing."
- "Ship with shadow or canary and a rollback."
- "Labels arrive late: watch drift and proxies now, performance later."

13

CHAPTER 13 · HIGH PRIORITY · MODULE 13 IN THE APP

Business Sense, Product Metrics, Case Studies

Turning a vague business question into a structured analysis. This is where hiring managers decide whether you think like a data scientist.

6 topics · about 2 h 30 min in the app · 9 questions answered

13.1 Metric vocabulary

13.2 Framework: diagnosing a metric drop

13.3 Framework: measuring the success of a feature

13.4 ML problem framing: from business ask to DS problem

13.5 Guesstimates

13.6 Communication rules for data scientists

Foundations

Technical rounds test whether you can use the tools. The case round tests whether you know which tool the situation calls for, and whether you can reason clearly when the problem arrives as a sentence, not a dataset: "orders dropped ten per cent", "how would you measure this feature?", "should we raise the delivery fee?". There is no single right answer, which unsettles many candidates. The interviewer is not grading your conclusion. They are grading your **structure**: whether you clarify before answering, break the problem into parts, name what data you need, and finish with a recommendation.

Metrics are how a business sees itself

Every case eventually becomes a discussion of metrics, so begin with a clear vocabulary. A company usually has one **north-star metric** that captures the value it delivers, such as weekly active buyers. Beneath it sit **driver metrics**, the levers that move it. Alongside are **guardrail metrics** that must not be damaged while you chase the main one. A good metric is measurable, sensitive enough to move when things change, hard to manipulate, and tied to a decision. Any metric optimised alone will be gamed: push for clicks and you get misleading headlines, which is why a metric is always paired with a guardrail.

The reusable frameworks

Three frameworks cover most cases. They are not scripts to recite; they are orders of thought that stop you from jumping to a guess.

Diagnosing a change in a metric.

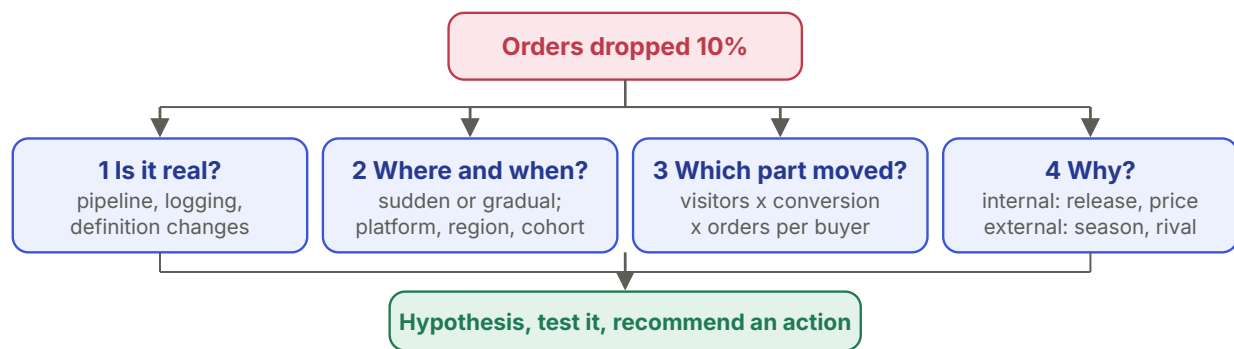


Figure 15. Work from the cheapest explanation to the most expensive: data problems first, then where, then what, then why.

First ask whether the change is real, because a broken pipeline or a changed definition is the most common cause and the cheapest to check. Then locate it in time (sudden or gradual) and in the user base (which platform, region or cohort). Then decompose the metric into its components to see which one moved. Only then look for causes, internal ones such as a release or a price change before external ones such as a holiday or a competitor.

Measuring the success of a feature. State the feature's purpose, choose a primary metric tied to that purpose, add adoption metrics and guardrails, and decide the method, ideally an experiment. Say in advance what result would lead to shipping, iterating or stopping. The key insight is that people using a feature is not the same as the feature succeeding; success means the goal metric moved without harm elsewhere.

Framing a machine learning problem. Start from the decision the business wants to make, not from the model. "Reduce churn" becomes: predict which active customers will leave in the next thirty days, so that the retention team can contact the top tenth, judged by revenue retained. That sentence contains the target, the timing, the action and the measure of success. Sometimes the honest conclusion is that a simple rule or a dashboard would serve better than a model.

Estimation

Guesstimates such as "how many delivery orders are placed in this city each day?" test the same skill with numbers. Break the quantity into parts you can estimate, state each assumption, compute with round numbers, and check the result against something you know. The structure and the sanity check are what earn the marks.

Communication is part of the answer

Lead with the conclusion, support it with the two or three strongest pieces of evidence, state the uncertainty honestly, and end with a recommended action. An analysis that does not change a decision has not finished its job.

Topic by topic

Each topic gives the idea in plain words, the fundamentals to know, a model spoken answer and three questions to test yourself. Open the same topic in the app for worked examples, runnable code and interactive demos.

13.1 Metric vocabulary

IN PLAIN WORDS

Metrics are the company's dashboard lights. "Are people using it?" (DAU, MAU), "do they come back?" (retention), "are they buying?" (conversion, AOV), "is it worth what we pay to get them?" (CAC vs LTV), "is the service good?" (delivery time, NPS). The trick is to define each number exactly: what is counted, out of what, over what time, for which users. And remember: when a number becomes a target, people (and algorithms) start gaming it.

Why it matters: Case rounds start by asking for the right metrics; vague definitions are an immediate red flag.

Takeaway: Always state numerator, denominator, window and unit, and pair every target metric with a guardrail.

WORKED EXAMPLE, STEP BY STEP

Build a metric tree for a food-delivery app

- 1. North star: weekly completed orders.** It reflects value for customers (they got food), restaurants (they sold) and the company (it earned).
- 2. Break it into a product of drivers:** weekly orders = weekly active users × orders per active user.
- 3. Go one level deeper:** weekly active users = new users + returning users; orders per active user depends on app conversion (sessions that end in an order) and on how often people come back.
- 4. Put numbers on it:** 200,000 active users × 1.5 orders each = 300,000 orders. A 10% rise in conversion with no change in users gives about 330,000.
- 5. Add guardrails** that must not get worse while the drivers improve: delivery time, cancellation rate, restaurant ratings and contribution margin per order.
- 6. Give each team a driver.** Growth owns new users; the search team owns conversion; operations owns delivery time. Everyone can see how their number feeds the north star.

Level	Metric	Why
North star	Weekly completed orders	the value delivered
Drivers	Active users, orders per user, conversion	levers teams can move
Guardrails	Delivery time, cancellations, margin	stop harmful growth

What this shows. A metric tree turns one goal into levers people can act on, and the guardrails stop a team from moving its lever in a way that damages the whole.

YOUR TURN (COVER THE SOLUTION FIRST)

1. An app has 1,200,000 monthly active users and 300,000 average daily active users. Compute DAU/MAU and explain what it means in plain words.

Solution. $DAU/MAU = 300,000 / 1,200,000 = 0.25$. On an average day a quarter of the month's users show up; equivalently, a typical monthly user is active on about $0.25 \times 30 \approx 7.5$ days a month. It measures stickiness: a messaging app might be above 0.5, a travel-booking app far lower.

THE FUNDAMENTALS

Area	Metrics
Growth/engagement	DAU, WAU, MAU, stickiness (DAU/MAU), sessions per user, time spent, activation rate
Retention	D1/D7/D30 retention, cohort retention, churn rate, resurrection rate
E-commerce/quick-commerce	GMV, orders, AOV, conversion rate, cart abandonment, repeat rate, fill rate, delivery time (P50/P90), cancellation rate, NPS, take rate
Marketing	CAC, ROAS, CTR, CPC, CPA, conversion funnel, attribution
Revenue/SaaS	ARPU, LTV, LTV:CAC, MRR/ARR, net revenue retention
Fintech/lending	Approval rate, default rate, NPA, delinquency buckets (DPD), collection efficiency, TPV, fraud rate, chargebacks
Marketplace	Supply-demand balance, liquidity, utilization, match rate
Ops	SLA adherence, cost per order, first-contact resolution

Metric hygiene: define numerator/denominator precisely, time window, unit (user/order), inclusion rules, and watch for gaming and Goodhart's law.

SAY IT IN 60 SECONDS

"A metric turns behaviour into a number. I separate three kinds. The north star is the single metric that captures delivered value, like weekly active buyers. Input or driver metrics are the levers that move it, such as search-to-click rate. Guardrails are what must not break, like latency or refund rate. Common ones: DAU and MAU, retention, conversion, ARPU, CAC and LTV, churn, NPS. Good metrics are measurable, sensitive, hard to game and tied to a decision. Ratios need both numerator and denominator thought through. The catch: a metric optimised alone gets gamed, such as clicks rising while satisfaction falls, so I always pair it with a guardrail."

KEY TERMS DAU MAU stickiness · retention · churn · GMV · AOV · conversion · CAC · LTV · NPS · default rate · Goodhart's law

HANDS-ON PRACTICE Pick an app you use and define five metrics (one per area) with numerator, denominator, window and unit.

COMMON MISTAKES Vague metric definitions (no denominator, no time window) · Confusing GMV with revenue · Ignoring Goodhart's law

CHECK YOURSELF**1. A video app optimises 'watch time'. What guardrail would you add and why?**

User satisfaction or return rate, or complaint/skip rates, because watch time can rise from addictive or low-quality content that hurts retention.

2. DAU/MAU of 0.2 means what?

On an average day, 20% of monthly users are active (stickiness): they use the product about 6 days a month.

3. Why is a ratio metric like conversion rate tricky in an experiment?

Both numerator and denominator can change; the unit of analysis must match the unit of randomisation, and variance needs the delta method or per-user metrics.

13.2 Framework: diagnosing a metric drop**IN PLAIN WORDS**

When a key number suddenly drops, act like a detective, not a fortune teller. First make sure the drop is real and not a broken chart. Then split the number into its parts to find where it fell (fewer visitors? fewer of them buying?). Then slice by groups (which phones, cities, new vs old customers) to find where it is concentrated. Only then list the possible causes (something we changed, something outside our control), check the evidence for each, and recommend what to do.

Why it matters: This is the single most common product-analytics case question.

Takeaway: Verify, decompose, segment, hypothesise from the evidence, prioritise, act, monitor.

WORKED EXAMPLE, STEP BY STEP**Decompose a 10% drop in orders**

Orders fell from 10,000 last week to 9,000 this week.

1. Is it real? The dashboard query is unchanged, the pipeline ran, and the payment logs show the same drop. It is real.

2. Write orders as a product: orders = sessions × conversion.

	Last week	This week	Change
Sessions	50,000	49,500	-1%
Conversion	20.0%	18.2%	-9%
Orders	10,000	9,000	-10%

3. Read it. Traffic barely moved; conversion fell. People are arriving but not completing orders. The problem is inside the app, not in marketing.

4. Segment conversion by platform:

	Last week	This week
iOS (20,000 sessions)	20.0%	20.0%

	Last week	This week
Android (about 30,000 sessions)	20.0%	16.9%

5. Narrow it further. Only Android fell. Check the release calendar: Android version 4.2 shipped on Monday. Look at the Android funnel: is the drop at payment?

6. Recommend: roll back or hotfix 4.2's payment screen, add an alert on conversion by platform, and estimate the lost orders (about 1,000 for the week) for the incident report.

What this shows. Verify, decompose, segment, find the cause, recommend. Writing the metric as a product points straight at the component that moved.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Revenue fell 8% week on week. Visitors are flat, conversion is flat, and orders are flat. Where is the problem, and what would you check next?

Solution. Revenue = visitors × conversion × average order value. With the first two flat (and orders flat), the drop must be in average order value. Check: a price or discount change, a promotion pushing cheaper items, a shift in product or category mix, a change in delivery fees counted in revenue, or a change in how revenue is calculated (taxes or refunds). Then segment order value by category and city.

THE FUNDAMENTALS

Question: "Orders dropped 10% week over week. How do you investigate?"

- 1. Clarify and verify:** Is it real? Check the data pipeline, logging, definitions, dashboards lagging. Size and timing (sudden vs gradual).
- 2. Decompose the metric:** Orders = Users x Sessions/User x Conversion x Orders/Converter. Find which component moved.
- 3. Segment:** platform (Android/iOS/web), city/region, new vs returning users, category, acquisition channel, time of day, app version.
- 4. Internal causes:** releases/bugs, pricing or fee changes, promotions ended, supply issues (stockouts, fewer delivery partners), ranking/recommendation changes, experiments.
- 5. External causes:** seasonality, festivals, holidays, weather, competitor offers, cricket matches/events, policy/regulatory changes, payment gateway outages.
- 6. Funnel analysis:** where did drop-off increase (search → product → cart → payment → success)?
- 7. Form hypotheses, test them with data, prioritize by impact, recommend actions and monitoring.**

SAY IT IN 60 SECONDS

"When a metric drops, I do not guess; I follow a framework. First, verify: is the data real, or a logging, pipeline or definition bug? Second, scope it: when did it start, is it gradual or a step, and is it everywhere? Third, decompose the metric into parts, like orders equal visitors times conversion, and see which part moved. Fourth, segment by platform, region, new versus returning, channel and product. Fifth, check internal causes such as releases, experiments, price changes and outages, and external ones such as seasonality, holidays, competitors. Then form hypotheses, test the likeliest and recommend an action. The catch: I start with the cheapest checks, data issues and recent releases."

KEY TERMS metric decomposition · segmentation · funnel analysis · internal vs external causes · hypothesis prioritisation · data validation

BUILDS ON 13.1 Metric vocabulary

HANDS-ON PRACTICE Answer the "orders dropped 10%" question aloud in three minutes using the seven steps, then compare with the worked example.

COMMON MISTAKES Jumping to causes without verifying the data · Listing ideas without structure or prioritisation · Forgetting external factors (festivals, weather, matches)

CHECK YOURSELF**1. Orders dropped 10% yesterday. What do you check first?**

Whether the data is correct (pipeline, tracking, definitions), then whether it is a step change at a specific time, which suggests a release or outage.

2. How do decomposition and segmentation differ?

Decomposition splits the metric into components (traffic × conversion × basket); segmentation splits by user or context (platform, country) to find where it moved.

3. The drop is only on Android after the 4.2 release. Next step?

Check release notes and crash or funnel data for Android 4.2, reproduce the bug, and roll back or hotfix.

13.3 Framework: measuring the success of a feature

IN PLAIN WORDS

"How would you measure whether this feature is a success?" has a recipe. First say what the feature is for and who it is for. Then choose one main number that shows the feature is working, a few supporting numbers that explain why, and some "do no harm" numbers. Decide how you will test it (usually an A/B test) and what you will compare. Think about what it might steal from elsewhere (cannibalisation), and decide beforehand what result means "keep it".

Why it matters: Product and analytics interviews use this prompt constantly, with different features.

Takeaway: Goal, primary / secondary / guardrail metrics, leading vs lagging, experiment design, segments and cannibalisation, decision criteria.

WORKED EXAMPLE, STEP BY STEP**Why "users of the feature buy more" proves little**

A "reorder in one tap" button launched to everyone. Data: customers who used it place 6 orders a month; those who did not place 3.

1. **The tempting conclusion:** "The button doubled orders." Wrong.
2. **Ask who chose to use it.** Frequent customers are the ones most likely to reorder, so they were always heavier buyers. That is **selection bias**.
3. **Look at the same two groups before launch:** future users ordered 5.8 a month; future non-users ordered 3.0.
4. **Difference in differences:** users went from 5.8 to 6.0 (+0.2); non-users from 3.0 to 3.0 (0). The effect is about **+0.2 orders a month** per user, not +3.
5. **Is +0.2 worth it?** Multiply by the number of users and the margin per order, and compare with the cost of maintaining the feature.
6. **Next time, plan a holdout:** launch to 95% of users and keep 5% without the button for a month. That gives a clean causal estimate.

What this shows. Adoption shows who likes a feature, not what it changed. Compare against a baseline: an experiment, a holdout, or at least the same people before the launch.

YOUR TURN (COVER THE SOLUTION FIRST)

1. A loyalty programme's members spend 4,000 a month and non-members 1,500. Before joining, the members spent 3,500 and the non-members 1,400 over the same months. Estimate the programme's effect and name the assumption you rely on.

Solution. Members changed by $4,000 - 3,500 = +500$; non-members by $1,500 - 1,400 = +100$. Difference in differences: $500 - 100 = +400$ a month per member, far less than the raw gap of 2,500. The assumption is parallel trends: without the programme, members' spending would have grown by the same 100 as non-members'. Check it with several months of pre-programme data.

THE FUNDAMENTALS

1. Goal of the feature and users targeted.
2. **Primary metric** (north star link), **secondary**, **guardrails**.
3. Leading vs lagging indicators.
4. Experiment design (A/B or quasi-experiment).
5. Segments to inspect; potential cannibalization.
6. Decision criteria and next steps.

SAY IT IN 60 SECONDS

"To measure the success of a feature, I start with its purpose: what user problem and business goal does it serve? Then metrics. A primary success metric tied to that goal, adoption metrics like the share of users who try it and repeat, input metrics showing it is used well, and guardrails for side effects, such as cannibalisation or slower performance. Then the method: an A/B test if I can randomise; otherwise a staged rollout compared against holdouts, or a before-and-after with caution. I define in advance what result would make us ship, iterate or kill it. The catch: usage of the feature alone is not success; it must move the goal metric without harming others."

KEY TERMS success metrics · north star · leading and lagging indicators · guardrails · cannibalisation · quasi-experiments · difference-in-differences

BUILDS ON 13.1 Metric vocabulary · 6.1 The experiment lifecycle

HANDS-ON PRACTICE Answer each of the four prompts aloud using the six-step framework, then compare with the model answers.

COMMON MISTAKES Choosing only engagement metrics and ignoring revenue and quality · Ignoring cannibalisation and guardrails · Forgetting how to evaluate without an experiment

CHECK YOURSELF**1. Feature adoption is 40%, but revenue per user is flat. What might it mean?**

Users are using it but it is not creating new value: it may substitute an existing behaviour (cannibalisation) or be a novelty. Check against a control group.

2. What is a guardrail for a new recommendation feature?

Page load time, bounce rate, and revenue per session, so that engagement gains do not hide other harm.

3. You cannot A/B test a feature (shipped to all). How do you estimate impact?

Use a staggered rollout or holdout if possible; otherwise difference-in-differences or an interrupted time series, stating the assumptions and limits.

13.4 ML problem framing: from business ask to DS problem

IN PLAIN WORDS

A business person says "reduce churn" or "cut fraud". That is a wish, not a machine learning problem. Framing means turning it into a precise question a model can answer and a decision someone can act on: "Among customers active today, who will stop ordering in the next 30 days, and which of them should we send an offer to?" The four questions that do the work: what decision will this change, what does each kind of mistake cost, what data will I really have when I must predict, and how will we know it worked?

Why it matters: Interviewers use vague asks to see whether you start with the business, not the algorithm.

Takeaway: Decision, cost of errors, data at prediction time, success measure.

WORKED EXAMPLE, STEP BY STEP**Frame "use AI to reduce delivery delays"**

- 1. Find the decision.** Delays can be reduced by (a) giving customers more honest time promises, (b) assigning riders better, or (c) helping kitchens prepare on time. Ask which lever the team controls. Suppose (b): the dispatch team chooses which rider takes each order.
- 2. Define the target precisely.** For each candidate rider at assignment time, predict the minutes from assignment to delivery.
- 3. Define the prediction moment and the features available then:** rider location and current load, restaurant preparation-time history, distance, time of day, weather. Nothing recorded after assignment.
- 4. Choose metrics on two levels.** Model: mean absolute error in minutes. Business: the share of orders delivered later than promised, and average delivery time.
- 5. Name the baseline:** the current rule, "nearest free rider".
- 6. Plan the test:** a switchback design (alternate the new and old assignment logic by time block within each city), because riders are shared between control and treatment.
- 7. Check whether ML is even needed:** if most delay comes from a few slow kitchens, a simple rule or a conversation with those restaurants may beat any model.

What this shows. Framing turns a slogan into a decision, a target, a prediction moment, metrics, a baseline and a test. The model comes last.

YOUR TURN (COVER THE SOLUTION FIRST)

1. A manager asks you to "predict which new restaurants will become popular". Write the framing in four lines: decision, target, prediction moment, metric.

Solution. Decision: which new restaurants get promotional placement in their first month. Target: whether a restaurant reaches the top 30% of its city by orders in months 2 and 3. Prediction moment: day 14 after onboarding, using menu, prices, cuisine, location, ratings and orders from the first two weeks only. Metric: precision among the top 20 restaurants chosen each month, plus the order lift for promoted restaurants measured against a randomly held-out group.

THE FUNDAMENTALS

Business ask	Framing	Target	Key features	Metric	Action
"Reduce churn"	Binary classification (churn in next 30 days)	churn flag	recency, frequency, support tickets, delivery delays	PR-AUC, lift at top 10%, expected profit	Targeted retention offers
"Reduce fraud"	Classification/anomaly detection	fraud flag	velocity, device, geo, amount patterns	Recall at fixed FPR, PR-AUC	Block/step-up auth
"Forecast demand"	Time-series regression	units per SKU-store-day	lags, promo, festival, price	WAPE, bias	Inventory planning

Business ask	Framing	Target	Key features	Metric	Action
"Better ETAs"	Regression	delivery time	distance, traffic, restaurant prep time, weather	MAE, P90 error, % late	Customer promise
"Segment customers"	Clustering	none	RFM, categories, behavior	Silhouette + business profiling	Marketing strategy
"Approve loans"	Credit scoring	default flag	bureau, income, behavior	KS, AUC, approval-vs-loss tradeoff	Approve/decline/limit
"Recommend items"	Ranking	click/purchase	user/item embeddings, context	NDCG, CTR in A/B	Personalized feed

Always ask: *What decision will this model change? What is the cost of each kind of error? What data is available at prediction time? How will success be measured?*

SAY IT IN 60 SECONDS

"Framing turns a vague business ask into a data science problem. I ask: what decision will this support, and who acts on it? What is the target and how is it defined? What is the metric of success and the cost of each error type? What data exists at prediction time? Is it a prediction, a causal question, an optimisation or just a report? For example, 'reduce churn' becomes: predict which active customers will cancel in the next thirty days, so that retention can target the top ten per cent with an offer, evaluated on retained revenue. Sometimes no ML is needed; a rule or a dashboard is better. The catch: a model with no action attached has no value."

KEY TERMS problem framing · target definition · prediction time · features available at prediction time · cost of errors · action · success metric

BUILDS ON 8.5 Choosing the metric from business cost

HANDS-ON PRACTICE Frame "reduce delivery delays" as an ML problem: target, prediction time, features, metric, action.

COMMON MISTAKES Starting with the algorithm · Vague target definitions (what is churn exactly?) · Optimising AUC instead of the action's value

CHECK YOURSELF**1. "Increase sales with AI." What clarifying questions do you ask?**

Which decision, which sales (segment, channel), what lever can change (price, recommendations, outreach), what data exists, how success is measured, and the constraints.

2. Predict churn or estimate the effect of a discount: what is the difference?

Prediction ranks who will churn; the discount question is causal (uplift): who changes behaviour because of the offer. Targeting the likely churners can waste money on those who would stay anyway.

3. When would you recommend not using ML?

When a simple rule or report meets the need, there is little data or signal, or errors are too costly to risk without explanation.

13.5 Guesstimates

IN PLAIN WORDS

A guesstimate is a structured way to make a good guess when you do not have the data, like estimating how many balloons fit in a room by measuring one balloon and the room. You break the big unknown into smaller pieces you can reason about, put a sensible number on each, multiply, and then check whether the answer feels right. Interviewers are not testing whether you know the exact number; they are testing how you think and whether you state assumptions out loud.

Why it matters: Common at consulting-style analytics firms.

Takeaway: Clarify, segment the population, assume with numbers, compute, sanity-check.

WORKED EXAMPLE, STEP BY STEP**Estimate a neighbourhood gym's monthly revenue, two ways**

- 1. Clarify:** a mid-range gym in a large Indian city; revenue from memberships only.
- 2. Supply side (capacity).** About 80 machines and stations. At the peak hour, 7 to 8 pm, maybe 80% are in use, so 64 people. Members come about 3 times a week and spread over roughly 6 busy hours, with the peak taking about a quarter of a day's visits. Visits a day $\approx 64 / 0.25 \approx 250$.
- 3. Visits to members.** 250 visits a day $\times 7$ days = 1,750 visits a week. At 3 visits per member: about **580 members**.
- 4. Revenue:** $580 \times 2,000$ rupees a month (an average after discounts on annual plans) $\approx$ **11.6 lakh a month**.
- 5. Sanity check from the demand side.** The catchment has perhaps 40,000 adults, of whom about 3% use some gym, and this gym has half the local market: $40,000 \times 3\% \times 0.5 = 600$ members. Close to 580.
- 6. State the weakest assumption:** the peak-hour share. If the peak holds a third of daily visits, not a quarter, visits fall to about 190 a day, members to about 450 and revenue to about 9 lakh.

What this shows. Break the quantity into parts you can estimate, keep numbers round, check with an independent method, and say which assumption moves the answer most.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Estimate the number of orders a single cloud kitchen in a busy area handles per day. Give your structure and a sanity check.

Solution. One structure, by capacity: 2 cooking lines $\times$ 4 orders in parallel per line $\times$ an average cooking time of 12 minutes gives $2 \times 4 \times 5 = 40$ orders an hour at full speed. Busy periods total about 5 hours a day (lunch and dinner) at about 70% utilisation: $40 \times 5 \times 0.7 \approx 140$ orders, plus perhaps 30 in quieter hours, so about 170 a day. Sanity check by revenue: at an average of 350 rupees, that is about 60,000 rupees a day, or 18 lakh a month, which is plausible for a busy kitchen. The weakest assumption is the number of parallel orders per line.

THE FUNDAMENTALS

Structure: clarify scope $\rightarrow$ segment the population $\rightarrow$ apply assumptions with rough numbers $\rightarrow$ compute $\rightarrow$ sanity-check.

Example: "How many food deliveries happen per day in Bengaluru?" Population 13M $\times$ share ordering regularly (15%) $\sim$ 2M people $\times$ $\sim$ 2 orders/week $/ 7 = \sim$ 570k orders/day across all platforms. Sanity check vs known platform volumes; state assumptions explicitly.

SAY IT IN 60 SECONDS

"A guesstimate is a structured estimate from reasonable assumptions. I restate the question, choose an approach, split into parts I can estimate, state my assumptions with numbers, calculate, then sanity-check against something known. For example, how many coffee cups are sold in a city per day? Population one million, say forty per cent adults who drink coffee, two cups a day on average, but only half buy outside, so about 400,000 cups a day. I round numbers to make arithmetic easy and keep units. The catch: the interviewer is grading the structure and the sanity check, not the exact answer, so I talk through each assumption and say which I am least sure of."

KEY TERMS guesstimate · market sizing · top-down · bottom-up · assumptions · sanity check

HANDS-ON PRACTICE Do three guesstimates under five minutes each and compare with a real reference you look up afterwards.

COMMON MISTAKES Silent arithmetic without stating assumptions · Unrealistic round numbers with no justification · Skipping the sanity check

CHECK YOURSELF

1. Estimate the number of petrol stations in a city of 2 million.

About 1 car per 4 people gives 500,000 cars; each fills up about once a week = 70,000 fills/day; one station serves roughly 500/day, so about 140 stations. Sanity-check with local knowledge.

2. Why state assumptions explicitly?

It shows structure, lets the interviewer correct a number, and makes the answer traceable and adjustable.

3. What is a sanity check for a market-size estimate?

Compare with a top-down or known benchmark (revenue of a known company, per-capita spend) to see if the order of magnitude is plausible.

13.6 Communication rules for data scientists

IN PLAIN WORDS

Good analysis that nobody understands changes nothing. The rule is to put the answer first, then the evidence: "We should fix the Android payment bug; it cost us 40,000 orders this week", then a chart that shows it, then the details for anyone who wants them. Talk in rupees, users and hours, not p-values and AUCs, unless your listener is another data scientist. And always say how sure you are and what you did not check.

Why it matters: Communication is half of the job and a major factor in hiring decisions.

Takeaway: Answer first, impact in business terms, one chart per message, assumptions and limits stated.

WORKED EXAMPLE, STEP BY STEP

Rewrite an analysis update for an executive

Before: *"We pulled six months of data from the events table, cleaned the duplicates, built cohorts by signup week and ran a logistic regression with 14 features. AUC was 0.78. Interestingly, the onboarding tutorial completion variable had the largest coefficient. We also tried XGBoost."*

- 1. Find the answer the reader needs:** what should we do, and why?
- 2. Lead with it:** "Users who finish the onboarding tutorial are about twice as likely to still be active after 30 days. I recommend making the tutorial shorter and testing it as the default."
- 3. Give the two or three strongest supporting facts, in business terms:** "Only 35% finish it today. Among users who joined in the same week and city, finishers retain at 48% against 24%."
- 4. State the caution honestly:** "This is an association: motivated users may simply be more likely to finish. The A/B test will tell us the true effect."
- 5. End with the ask and the next step:** "Approve a two-week test from Monday; we will report the effect on 30-day retention by the 20th."
- 6. Move the method to an appendix:** the data, the model and the AUC are there for anyone who wants them.

What this shows. Answer first, then evidence, then caveat, then the ask. The analysis is not finished until someone knows what to decide.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Rewrite for a sales director: "The model's precision at the 0.6 threshold is 0.42 with recall 0.71, and the PR-AUC improved by 0.08 over baseline."

Solution. "Of the accounts the model flags as likely to leave, about 4 in 10 really are at risk, and it finds about 7 in 10 of all at-risk accounts. That is clearly better than our current rule. If the team calls the flagged accounts each week, we expect to reach most at-risk customers while about 6 in 10 calls go to customers who were fine." A number they can act on, in their terms, with the trade-off stated.

THE FUNDAMENTALS

- **Lead with the answer** (BLUF), then evidence.
- Translate metrics to business impact (Rs, users, hours).
- State assumptions, limitations, and confidence.
- Use one chart per message; annotate the takeaway.
- Tailor depth to audience: executives (decision + impact), product managers (trade-offs), engineers (details).

SAY IT IN 60 SECONDS

"Communication is a core data science skill. I lead with the answer or recommendation, then the supporting evidence, then the caveats. I tailor to the audience: executives want the decision and impact; engineers want the details. I use one idea per slide, a clear chart with a takeaway title, and plain language, defining terms. I quantify uncertainty honestly with ranges rather than false precision. When asked something I do not know, I say so and explain how I would find out. In an interview I structure answers, such as situation, approach, result, and check in with the listener. The catch: the model's accuracy is not the message; what changes because of it is."

KEY TERMS BLUF · bottom line up front · tailoring to audience · one chart per message · assumptions · limitations

HANDS-ON PRACTICE Rewrite one of your project summaries as a three-sentence message for an executive.

COMMON MISTAKES Starting with methodology · Reporting statistical output without business meaning · Overloading slides with charts

CHECK YOURSELF**1. How do you present a model result to a non-technical executive?**

Start with the decision and business impact, one clear chart, the main risk, and the recommended next step; keep metrics in business terms such as dollars saved.

2. Stakeholders challenge your result. How do you respond?

Listen, restate their concern, show the evidence and assumptions, acknowledge limits, and offer a check that could settle the point.

3. Why lead with the conclusion?

It respects the listener's time and attention, and everything after is read as support for a known point.

Interview tips, tricks and hacks

- 1. Never answer in the first thirty seconds.** Restate the problem, ask two or three clarifying questions (which metric exactly, over what period, for which users), and outline your approach. This alone separates strong candidates from weak ones.
- 2. For any metric drop, say "first I would check whether it is real".** Data and logging issues, definition changes, a dashboard lag. It is correct, cheap and expected.
- 3. Decompose with multiplication.** Orders equal visitors times conversion times orders per buyer. Writing the metric as a product shows instantly where to look.
- 4. Segment in a fixed order.** Time, platform, geography, new versus returning users, channel. A standard list prevents you from forgetting the obvious.
- 5. Internal causes before external.** Releases, experiments, pricing and outages are more common and easier to verify than competitors or seasonality.
- 6. Always pair a metric with a guardrail.** Whenever you propose a success metric, name what it could damage.
- 7. Finish with a recommendation and a next step.** "I would roll back the Android release and add an alert on checkout failures." A list of possibilities is an unfinished answer.
- 8. In guesstimates, round aggressively and say your weakest assumption.** "The share who order weekly is my least certain number; if it is half that, the answer halves."
- 9. Think aloud and check in.** "I will start with data quality, then segments: does that match what you want to explore?" The interviewer will often steer you towards the intended path.
- 10. Ask whether ML is needed at all.** Suggesting a rule or a report when it fits shows judgement, not weakness.

Frequently asked questions

9 questions interviewers ask on this chapter, with model answers. Cover the answer, say yours aloud, then compare.

METRIC VOCABULARY

Q1 DAU is 1.5M and MAU is 6M. The CEO says "stickiness is too low". Is 25% low?

Stickiness = $DAU/MAU = 1.5 / 6 = 25\%$, meaning the average monthly user is active about 7.5 days a month. Whether that is low depends on the **natural usage frequency**: a messaging or news app should target 50% or more; a fashion or grocery-restock app can be healthy at 15 to 30%. Compare with your own trend and peers, look at cohorts and the distribution of days active (power users vs occasionals), and define what "active" means before judging.

FRAMEWORK: DIAGNOSING A METRIC DROP

Q2 Weekly active users of a fitness app dropped 15% in the first week of January compared with the last week of December. What do you think?

Verify the data and **compare with the same week in previous years**: late December can show unusual usage (holiday campaigns, New Year's resolutions starting later) and January often has its own pattern, so a week-over-week comparison across a holiday boundary is misleading. Segment by new vs returning users and by acquisition channel, check campaigns that ended, app releases and notification changes. If it is seasonal, the right comparison is year over year or against an expected baseline; if not, follow the decomposition and segmentation steps.

FRAMEWORK: MEASURING THE SUCCESS OF A FEATURE**Q3 How would you measure the success of "Swiggy One" / UPI cashback / a new "reorder" button?**

Goal and users: make repeat ordering effortless for returning customers who order similar items.

Primary metric: reorders per active customer per week (or share of orders that are reorders); link to north star: orders per customer per month.

Secondary: reorder-button click-through, time from app open to order, AOV, conversion of sessions using the button. **Guardrails:** cancellation and complaint rate (wrong or unavailable items reordered), delivery time, new-user conversion (is the button distracting?), long-run search/discovery engagement (cannibalisation).

Design: randomise eligible returning users (user-level) 50/50 for 2+ weeks; analyse orders per user (not sessions). **Leading vs lagging:** 7-day reorder rate (leading) and 30/90-day retention and order frequency (lagging). **Segments:** frequency tiers, cities, categories (groceries vs restaurants).

Decision: ship if orders per customer rise by a meaningful margin (for example +2%), guardrails hold, and the gain does not come at the cost of new-customer discovery.

Q4 Should Zomato increase delivery fees in rainy weather? What data do you need?

Clarify the goal: manage supply-demand balance in rain (rider availability drops and delivery times rise), protect service quality and profitability, without losing too many orders. **Data needed:** rider supply and demand by zone and hour, historical rain impact on delivery time and cancellations, price elasticity of demand (from past fee changes or experiments), customer segments and sensitivity, competitor pricing, and the cost of delays (refunds, churn).

Approach: test with a **geo or switchback experiment** (randomising individual customers is risky because demand and supply are shared between groups): raise the fee in matched zones on rainy hours, compare orders, delivery time, cancellations, rider earnings and retention against control zones. **Metrics:** contribution margin per order, completed orders, delivery time P90, customer complaints and repeat rate over following weeks.

Risks: perceived unfairness and churn, regulatory and PR concerns, fee changes that only shift demand. **Decision:** adopt a dynamic fee if margin and service quality improve without harmful repeat-rate decline; consider alternatives such as rider incentives or capped surge.

Data needed. Weather and demand by area and hour, rider supply and acceptance rates, delivery times, price sensitivity from past fee changes, and competitor pricing. Compare the extra fee revenue to lost orders and long-term customer churn.

Q5 How do you evaluate a loyalty program's impact if you cannot run an A/B test?

Without an A/B test, build a credible counterfactual. Options: (1) **Staggered rollout** as a natural experiment (compare cities or cohorts joining at different times, difference-in-differences). (2)

Matching or propensity scores to compare members with similar non-members on pre-programme behaviour (recency, frequency, spend, city, tenure), remembering **self-selection**: people who join loyalty programmes were already more engaged, so naive comparisons overstate impact. (3) **Regression discontinuity** if eligibility has a threshold (for example a spend cutoff). (4) **Synthetic control** at a market level.

Metrics: incremental orders and margin per customer (net of benefits cost), retention, and cannibalisation of full-price purchases. Validate with **pre-period parallel trends** and placebo tests, report uncertainty, and be explicit that observational estimates are weaker than randomised ones. If possible, run a **small holdout** (a randomly withheld group) going forward to get a clean measurement.

Why selection matters. People who join a loyalty program are already heavier users, so a naive comparison overstates its effect. Matching on pre-program behaviour or difference-in-differences controls for that.

Q6 Define a health metric for a ride-hailing driver marketplace.

A marketplace's health depends on both sides, so define a **health metric for drivers** linked to supply quality: (1) **Earnings per online hour** (and its distribution: median and lower percentile, not only the average), (2) **utilisation** (share of online time spent on trips), (3) **retention and churn** of active drivers (weekly active drivers, cohort retention), (4) **acceptance and cancellation rates**, (5) **driver satisfaction** (NPS, complaints, safety incidents).

Composite view: track earnings per hour, utilisation and 4-week retention together; a rise in average earnings with falling retention signals a problem.

Balance with rider side: wait times, ETAs and price; over-incentivising drivers raises costs, too few drivers raises wait times. **Guardrails**: fairness across segments and cities, safety and compliance. Monitor leading indicators (acceptance, time between trips) for early warning, and run experiments (incentive changes) with marketplace-aware designs.

ML PROBLEM FRAMING: FROM BUSINESS ASK TO DS PROBLEM**Q7 The head of operations says "predict late deliveries". How do you frame it and what do you ask?**

Ask: **when** must the prediction be made (at order placement, at pickup?) and therefore which features exist; **what action** follows (extend the promised time, assign a different rider, alert the customer, compensate); what "late" means (promised vs actual, tolerance); and the **costs** (a padded ETA lowers conversion; missed lateness raises complaints). Frame as **regression** on delivery time (with prediction intervals or the P90 to set promises) or **classification** of late deliveries. Features: distance, restaurant prep-time history, rider availability, traffic, weather, order size, time of day. Metrics: MAE, P90 error, % of orders late, and the business effect in an experiment.

GUESSTIMATES**Q8 Estimate the number of smartphones sold in India in a year.**

A structured try: India $\approx$ 1.4 billion people, about 55% ($\approx$ 770 million) own a smartphone, replacement every $\sim$ 4 years $\rightarrow$ $\sim$ 190 million a year. *Sanity check*: published market shipments are often quoted around 150 to 160 million a year; the estimate is in the right ballpark. The key uncertainty is the **replacement cycle** (3 to 5 years) and ownership share; say so, and show the range (155 to 260 million). Good guesstimates report a **range** and the main driver of uncertainty.

COMMUNICATION RULES FOR DATA SCIENTISTS**Q9 Explain a random forest model's churn predictions to the marketing head in three sentences.**

"The model reviews each customer's recent ordering, delivery experience and support history and gives a churn risk score. The top 10% by score contain about three times more future churners than average, so focusing offers there is three times as efficient as contacting customers at random. The biggest warning signs are fewer orders over the last 30 days, repeated late deliveries and unresolved support tickets, so fixing those experiences also matters." This leads with the use, gives impact in plain terms and surfaces drivers without jargon.

14

CHAPTER 14 · HIGHEST PRIORITY · MODULE 14 IN THE APP

Projects: What to Build and How to Present

Your projects are the evidence that you can do the job. Build a few with depth, and learn to defend every decision in them.

6 topics · about 2 h 7 min in the app · 21 questions answered

- | | |
|---|---|
| 14.1 What a strong project demonstrates | 14.4 Turning a project into resume bullets |
| 14.2 Recommended portfolio and project ideas | 14.5 The project deep-dive questions |
| 14.3 Project template and workflow checklist | 14.6 Red flags to avoid |

Foundations

An interviewer cannot watch you work for a month, so they use your projects as a substitute. A project lets them ask the questions that matter: can this person take a vague problem, handle untidy data, make sensible decisions, and explain them? This is why the project discussion is often the longest part of a technical round, and why two or three projects you understand completely are worth more than ten you cannot defend.

What a strong project demonstrates



Interviewers probe every arrow: why this metric, this split, this model, and what you would change.

Figure 16. A strong project shows the whole path from a real question to something another person can use.

A weak project starts with a clean, famous dataset, tries several models, and reports the best accuracy. A strong project starts with a **question someone would care about**, uses **real and imperfect data**, establishes a **simple baseline**, validates **properly**, studies **where the model fails**, and ends with something usable: a small application, an interface, or a clear recommendation. It also states its limitations. None of this requires a sophisticated model. A careful logistic regression with honest validation and a thoughtful error analysis impresses more than a deep network with an unexplained score.

A portfolio that shows range

Two to four projects are enough if they differ. A good set includes an analytics project that answers a business question with SQL and a dashboard; a predictive project with a full modelling pipeline; and one that shows a further skill, such as an experiment analysis, a forecasting problem, or an application built on a language model. Choose subjects you find interesting, because your energy when describing the work is itself a signal.

The deep dive

Interviewers follow a predictable path through a project, and you can prepare for every stop. Why this problem, and who would use the result? Where did the data come from, and what was wrong with it? How did you define the target? How did you split the data, and why that way? Which metric, and why not accuracy? Why this model, and what did the simpler one score? How did you prevent leakage? Where does the model perform worst? How would you deploy and monitor it? What would you do differently?

For each, the best answers have the same shape: **the choice, the reason, and the trade-off**. "I split by time, not at random, because the model will be used to predict future months; a random split gave a higher score but would have leaked future information."

From project to resume line

A resume bullet should state what you did, how, and what resulted, with a number: the size of the data, the improvement over the baseline, the time saved. "Built a churn model" says little. "Built a gradient-boosted churn model on 200,000 customers, raising recall in the top decile from 18 to 31 per cent over a logistic baseline" tells the reader the scale, the method and the gain, and gives the interviewer a good first question. Every number must be one you can explain and reproduce.

What worries interviewers

A few patterns raise doubts quickly: being unable to explain your own choices; a near-perfect score with no mention of leakage checks; no baseline; a model with no connection to any decision; a list of tools you cannot discuss; and taking credit for a team's work. The remedy for all of them is the same: know the project thoroughly, and be honest about what you did and what you did not.

Topic by topic

Each topic gives the idea in plain words, the fundamentals to know, a model spoken answer and three questions to test yourself. Open the same topic in the app for worked examples, runnable code and interactive demos.

14.1 What a strong project demonstrates

IN PLAIN WORDS

A strong project is a small, complete story: a real question, messy real data, a fair test of ideas, an honest result, and a way for someone to use it. Interviewers spend a large part of a technical round on your projects because they are the one place where you are the expert and they can probe how you think. A few deep projects that you can explain line by line beat many shallow ones.

Why it matters: "Walk me through your best project" decides a lot of interviews.

Takeaway: Two or three projects, each a complete story, each one you can defend end to end.

WORKED EXAMPLE, STEP BY STEP

Upgrade a tutorial project into an interview project

Starting point: "Titanic survival prediction with a random forest, 84% accuracy."

- 1. Ask a real question that someone owns.** Swap the dataset for a decision: "Which customers of a subscription business are likely to cancel next month, so that retention can call the top 10%?" Public telecom or e-commerce churn data works.
- 2. Make the data honest:** document the source, the grain, missing values and leakage risks, and compute features as of a prediction date.
- 3. Set a baseline:** "customers with three or more support tickets" as a simple rule, then logistic regression. Report how much the final model beats each.
- 4. Validate the way it would be used:** train on earlier months, test on a later month.
- 5. Choose a business metric:** precision among the top 10% (call capacity), with an estimated money value per correct call.
- 6. Do an error analysis:** where does the model fail? For example, customers in their first month with no history.
- 7. Deliver something usable:** a small app or notebook that takes a customer file and returns the call list, plus a one-page summary with limitations.

What this shows. The algorithm hardly changed. The project became strong because it now has a decision, honest validation, a baseline, a business metric and something usable at the end.

YOUR TURN (COVER THE SOLUTION FIRST)

1. A candidate's project reads: "Sentiment analysis on movie reviews using BERT, 93% accuracy." Name three upgrades that would make it stronger in an interview.

Solution. (1) Tie it to a decision: for example, route negative product reviews or support tickets to an urgent queue, measured by how quickly they get answered. (2) Add a baseline and a comparison: TF-IDF with logistic regression against BERT, with the accuracy gain weighed against the extra cost and latency. (3) Add error analysis on sarcasm, mixed reviews or Hinglish, and evaluate on data from a different source or time, not only the benchmark test split. Deploying a small demo endpoint is a fourth.

THE FUNDAMENTALS

1. **Problem framing:** a real question with a clear user/decision.
2. **Data work:** messy real data (scraped, API, public datasets), cleaning, joins, quality checks.
3. **EDA with insights** (not just plots).
4. **Sound modeling:** baseline first, proper validation, no leakage, tuning, error analysis.
5. **Right metrics** aligned to business cost.
6. **Explainability:** feature importance/SHAP, sanity checks.
7. **Delivery:** a dashboard, API, or app, plus a recommendation.
8. **Engineering hygiene:** GitHub repo, README, requirements, reproducible code.
9. **Reflection:** limitations, what you would do next.

SAY IT IN 60 SECONDS

"A strong project shows the whole path, not just a model. It starts from a real problem and a clear question, uses real, messy data, and describes the cleaning and the choices. It sets a baseline and compares models fairly with proper validation. It chooses a metric tied to the problem and interprets errors. It ends with a result that someone could act on, plus limitations and next steps. Even better, it is deployed or at least reproducible, with a readable repository. Interviewers use projects to judge depth, so I should be able to explain every decision and what I would change. The catch: a Kaggle-style accuracy chase on a clean dataset says less than a modest, well-reasoned end-to-end project."

KEY TERMS problem framing · baseline · leakage · metrics · explainability · deployment · README · reflection

HANDS-ON PRACTICE Score one of your projects from 0 to 2 on each of the nine items and pick the two weakest to fix.

COMMON MISTAKES Copying tutorials without original thinking · Reporting 99% accuracy without questioning it · Notebook dumps with no narrative

CHECK YOURSELF**1. What separates a strong project from a tutorial copy?**

An original question, messy real data, justified choices, a baseline and proper validation, error analysis, and a clear actionable conclusion.

2. Why include a baseline?

It shows whether the complex model adds value, and proves you evaluate rather than assume.

3. Name two signals of reproducibility.

A README with setup steps and pinned dependencies, and fixed random seeds with a runnable script or notebook order.

14.2 Recommended portfolio and project ideas

IN PLAIN WORDS

Pick projects like you would pick dishes for a tasting menu: variety, and each one shows a different skill. One business classification problem (churn or credit risk), one forecasting or regression, one text or GenAI project, and one piece that shows you can take something to production (an API or an app). Choose a real dataset, a real decision, and scope it small enough to finish and polish.

Why it matters: A portfolio with a clear spread tells the interviewer what you can do at a glance.

Takeaway: Three or four finished projects across categories beat ten half-done ones.

WORKED EXAMPLE, STEP BY STEP**Choose three projects for a product-analyst role**

Target: product and analytics roles at consumer apps.

- 1. Read the target jobs:** SQL, A/B testing, metrics, dashboards and some modelling.
- 2. Project 1 (SQL and BI):** "Where do users drop out of the checkout funnel?" Public e-commerce event data: cohort retention and funnel queries in SQL, plus a dashboard with three charts and one recommendation.
- 3. Project 2 (experimentation):** analyse a public A/B test dataset (for example a mobile game's gate placement): sample-size check, a sample ratio mismatch check, the effect with a confidence interval, guardrails and a one-page decision memo.
- 4. Project 3 (modelling):** a churn or conversion model with a time-based split, a baseline, a business metric and error analysis.
- 5. Make each one quick to scan:** a README with the question, the answer in two sentences, one chart and a "how to run" section. A recruiter spends about a minute on each.
- 6. Check the coverage:** SQL depth (1), statistics and product judgement (2), ML rigour (3). No two projects show the same skill.

What this shows. A portfolio is evidence for a specific job. Three projects that each prove a different required skill beat ten similar notebooks.

YOUR TURN (COVER THE SOLUTION FIRST)

1. You are targeting a fintech lender. Propose three projects, one sentence each, saying which skill each one proves.

Solution. (1) A credit default model with logistic regression and a scorecard (weight of evidence, KS and Gini), with reason codes for each decision: proves interpretable risk modelling. (2) An SQL analysis of repayment cohorts by month of disbursal (vintage curves) with a small dashboard: proves SQL and portfolio monitoring. (3) A fraud or anomaly detection project on transaction data, with a precision-at-capacity metric and a cost-based threshold: proves handling of imbalance and business-cost thinking.

THE FUNDAMENTALS

#	Type	Project ideas (India-relevant)	Skills shown
1	Classification (business)	Customer churn (telecom/OTT/SaaS); loan default/credit risk with WoE/IV scorecard; UPI/credit-card fraud detection	Imbalance, feature engineering, metric choice, explainability
2	Forecasting / regression	Demand forecasting for retail/quick commerce (Kaggle M5, Rossmann); delivery-time (ETA) prediction; used-car price (Cars24/CarDekho data)	Time-aware CV, lag features, error analysis
3	NLP	Sentiment/aspect analysis of Zomato/Amazon/Play Store reviews; support-ticket classification; resume-JD matching; Hinglish sentiment	Text preprocessing, TF-IDF baseline vs transformer
4	Recommender	Movie/restaurant/product recommender with popularity, item-item CF, and matrix factorization; evaluate Precision@K/NDCG	Cold start, ranking metrics
5	GenAI/LLM	RAG chatbot on RBI/SEBI circulars, insurance policy PDFs or company filings; text-to-SQL assistant; report summarizer	Chunking, embeddings, vector DB, evaluation
6	Analytics / experimentation	Cohort/retention + funnel analysis in SQL; A/B test analysis on a public dataset (Cookie Cats, Criteo uplift); RFM segmentation; Power BI/Tableau dashboard	Business storytelling, SQL, stats
7	Deployment / engineering	Serve any of the above with FastAPI + Docker; Streamlit app; deploy to Render/Hugging Face Spaces/AWS/GCP free tier; add MLflow tracking	Production readiness
8	Big data (optional)	PySpark pipeline on large NYC taxi or e-commerce events dataset; Airflow DAG	Scale and pipelines

Data sources: Kaggle, UCI, Google Dataset Search, data.gov.in, RBI/NSO/NSE datasets, Open-Meteo, public APIs, Hugging Face datasets, your own scraped data (respect ToS).

Avoid: Titanic/Iris/MNIST as headline projects, tutorial copies without original thinking, notebook dumps with no narrative.

SAY IT IN 60 SECONDS

"A good portfolio has two to four projects that show range. Typical set: an analytics project with SQL and a dashboard answering a business question; a predictive ML project with proper validation and error analysis; an experimentation or causal-inference project; and, if relevant, a text or LLM project such as a RAG bot. Each has a short README with the problem, approach, result and how to run. Choose data you find interesting, so you can talk about it with energy. Quality beats quantity. The catch: avoid the most overused datasets like Titanic and Iris unless you add something genuinely new, since interviewers have seen hundreds."

KEY TERMS churn · credit risk · fraud · forecasting · NLP · recommender · RAG · A/B test analysis · deployment · datasets

BUILDS ON 14.1 What a strong project demonstrates

HANDS-ON PRACTICE Choose three projects from different rows of the table, write a one-paragraph problem statement for each and pick the datasets.

COMMON MISTAKES Picking too many similar projects · Choosing a dataset with no real decision attached · Overscoping and finishing nothing

CHECK YOURSELF**1. You can do only three projects. Which mix?**

One SQL/analytics case with a dashboard, one end-to-end ML model with validation and error analysis, and one experimentation or LLM/RAG project; this shows range.

2. How do you make a common dataset stand out?

Ask a new question, enrich it with another source, deploy it, or add rigorous evaluation and business framing.

3. What should a README contain?

Problem, data, approach, key results, limitations, and how to run it.

14.3 Project template and workflow checklist

IN PLAIN WORDS

A project folder is like a tidy lab: data in one place, exploration notebooks in another, reusable code in a third, and a front door (the README) that tells visitors what this is, how well it worked, and how to run it. A recruiter gives a repository about two minutes; the README decides whether they keep reading. The checklist is a flight checklist: every modelling project should tick the same boxes so you never forget the important ones.

Why it matters: Repository quality signals engineering hygiene, which many candidates lack.

Takeaway: Standard structure, a clear README with a results table, and the twelve-step checklist.

WORKED EXAMPLE, STEP BY STEP**A four-week project plan with a decision log**

Week	Milestone	Decision log entries
1	Question, data, EDA	why this target; why this prediction date

Week	Milestone	Decision log entries
2	Baseline and features	why this split; which features were dropped for leakage
3	Models and error analysis	why model B over model A; where it fails
4	Delivery and write-up	what you would do with more time

- Week 1:** write the question and the decision in two sentences, get the data, and record its grain, size and quality issues.
- Week 2:** build a simple baseline end to end (data in, metric out) before any tuning. It catches pipeline bugs early.
- Week 3:** try two or three models with the same validation; analyse errors by segment.
- Week 4:** package it (an app, an API or a clean notebook), write the README, and record the limitations.
- Keep the decision log as you go.** One line each: date, decision, reason, alternative rejected. For example: "12 May: split by month, not at random, because the model predicts future months; a random split scored 0.05 higher, a sign of leakage."
- The log becomes your interview answers.** Every deep-dive question ("why this metric?") has a written answer with a date.

What this shows. A plan with weekly milestones keeps the project finished and honest, and the decision log turns your work into ready answers.

YOUR TURN (COVER THE SOLUTION FIRST)

- Write three decision-log lines for a house-price project: the target, the metric and the validation split.

Solution. Examples: "Target: log of sale price, because prices are right-skewed and errors should be relative; the alternative, raw price, was dominated by a few mansions." "Metric: RMSE on the log price (about a percentage error), also reported as the median absolute error in rupees for stakeholders." "Split: train on sales before 2024, test on 2024, because the model will price future listings; a random split leaked neighbourhood trends from the future."

THE FUNDAMENTALS

14.3 Project build template (follow for each)

```

project-name/
  README.md          ← problem, data, approach, results, how to run, limitations
  data/              ← (small samples or download script; not large files)
  notebooks/         ← 01_eda.ipynb, 02_baseline.ipynb, 03_modeling.ipynb
  src/               ← reusable python modules (features.py, train.py, predict.py)
  app/               ← FastAPI/Streamlit code
  tests/             ← a few unit tests
  requirements.txt
  Dockerfile
  reports/           ← figures, final summary

```

README structure: 1) Problem & business value, 2) Data (source, size, grain), 3) Approach, 4) Results (table with baseline vs final), 5) Key insights and recommendations, 6) Limitations and next steps, 7) How to run, 8) Demo link/screenshots.

14.4 Workflow checklist for each modeling project

- ☐ Define target, prediction time, and what data is available then
- ☐ Split correctly (time-based or group-based where needed) **before** any fitting
- ☐ Baseline (dummy, simple linear/logistic, seasonal naive)
- ☐ Pipeline for preprocessing (no leakage)
- ☐ Cross-validation and appropriate metrics
- ☐ Hyperparameter tuning and comparison of 2-3 model families
- ☐ Error analysis by segment; look at worst predictions
- ☐ Explainability and sanity checks
- ☐ Threshold selection tied to business cost
- ☐ Final test-set evaluation (once)
- ☐ Deployment + monitoring plan
- ☐ Clear business recommendation with quantified impact estimate

SAY IT IN 60 SECONDS

"I use a repeatable workflow for projects. One: define the problem, the user and the metric. Two: collect data and document its source and limits. Three: explore and clean, recording decisions. Four: set a baseline, then engineer features and try a few models with cross-validation. Five: evaluate on a held-out test set, with error analysis and a threshold chosen from costs. Six: interpret, using feature importance or SHAP, and check fairness and robustness. Seven: package it as a script, an API or an app. Eight: write the story. Keeping a checklist stops me from skipping the unglamorous steps. The catch: write down why I made each choice while I make it, since that becomes the interview answer."

KEY TERMS project structure · README · reproducibility · baseline · pipeline · threshold · explainability · deployment plan

BUILDS ON 14.2 Recommended portfolio and project ideas

HANDS-ON PRACTICE Create a repository skeleton and write a README with the eight sections for a project you have done.

COMMON MISTAKES Committing large data files or secrets · No instructions to run the project · No results table comparing to a baseline

CHECK YOURSELF**1. Why keep a decision log during a project?**

It records why each choice was made, giving ready, honest answers in a deep-dive interview.

2. Where does the test set first get used?

Once, after all modelling choices are final, to report an unbiased estimate.

3. Which step do beginners most often skip?

Baselines and error analysis; they jump to complex models and report a single score.

14.4 Turning a project into resume bullets

IN PLAIN WORDS

A resume bullet is a tiny advertisement for one thing you did. It works best when it says what you built, how, and what it achieved, in one line, with a number you can defend. "Used machine learning for churn" says nothing; "Built a churn model that found 3x more at-risk customers in the top 10% than random, enabling targeted offers" tells a story. And every number must survive the question "how did you get that?".

Why it matters: Interviewers pick bullets to interrogate; good bullets steer the conversation to your strengths.

Takeaway: Verb + what + technique/tools + quantified result + context. No fabricated numbers.

WORKED EXAMPLE, STEP BY STEP

Rewrite a bullet in four passes

Start: "Worked on a machine learning model for churn."

1. **Use a strong verb and say what you built:** "Built a churn prediction model."
2. **Add the method and the scale:** "Built a gradient-boosted churn model on 200,000 telecom customers using 30 behavioural features."
3. **Add the result against a baseline:** "...raising recall in the top 10% riskiest customers from 18% (rule-based) to 31%."
4. **Add the business meaning, honestly estimated:** "...enough to target about 1,300 more at-risk customers a month at the same calling cost."
5. **Final:** "Built a gradient-boosted churn model on 200,000 telecom customers, raising top-decile recall from 18% to 31% over the rule-based baseline, enough to reach about 1,300 more at-risk customers a month at the same cost."
6. **Check that you can defend every number:** where it came from, how it was measured and on which data. If you cannot, remove it.

What this shows. Verb, what, how, scale, result against a baseline, impact. Each number invites a good interview question.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Rewrite this bullet: "Made dashboards in Power BI for the sales team."

Solution. For example: "Built a Power BI sales dashboard (pipeline, conversion by region, weekly targets) used by 25 sales managers, replacing a weekly manual Excel report and saving about 6 analyst hours a week." Use only numbers you can confirm, such as the user count and the hours saved; if you cannot confirm them, describe the decision the dashboard supports instead.

THE FUNDAMENTALS

Formula: Action verb + what you built + technique/tools + quantified result + context.

- "Built a churn prediction model (LightGBM, SMOTE-free class weighting) on 7k telecom customers; improved PR-AUC from 0.46 (logistic baseline) to 0.63; top-decile lift of 3.1x, enabling targeted retention."
- "Designed a RAG assistant over 200 pages of insurance policy PDFs (FAISS + sentence-transformers + LLM API); reached 86% faithfulness on a 60-question golden set; deployed via FastAPI + Docker."

- "Analyzed 1.2M e-commerce events in SQL/pandas; identified 38% drop-off at payment step on Android, suggesting a fix projected to add ~4% conversion."

Do not fabricate numbers. Every figure must be reproducible from your repo.

SAY IT IN 60 SECONDS

"A good resume bullet has an action verb, what I did, how, and the measurable result. A formula: accomplished X, measured by Y, by doing Z. Instead of 'built a churn model', I write 'built a gradient boosting churn model on 200k customers, lifting recall at top decile from 18 to 31 per cent, projected to save about 120k dollars a year in retention offers'. Numbers show impact; tools show skills, so I name them once. I keep each bullet to one or two lines and put the strongest first. If I do not have business numbers, I use scale, speed or accuracy. The catch: never claim a number I cannot defend, since I will be asked how I measured it."

KEY TERMS action verb · technique · quantified result · context · reproducible numbers

BUILDS ON 14.1 What a strong project demonstrates

HANDS-ON PRACTICE Rewrite three of your current resume bullets with the formula and confirm every number is reproducible from your repo.

COMMON MISTAKES Listing tools with no outcomes · Fabricating or rounding up numbers · Vague verbs like "worked on"

CHECK YOURSELF

1. Rewrite: "Worked on a recommendation system."

"Built a collaborative-filtering recommender on 1M interactions in PySpark, improving offline NDCG@10 from 0.21 to 0.29 over a popularity baseline" (use numbers you can defend).

2. Course project with no business number. What metric can you quote?

Model performance versus a baseline, data size, runtime reduction, or time saved by automation.

3. Why avoid inflated numbers?

Interviewers probe them; if you cannot explain how it was measured, credibility drops across the whole resume.

14.5 The project deep-dive questions

IN PLAIN WORDS

These sixteen questions are what a curious interviewer will ask about any project, in roughly this order: why, where did the data come from, how did you prepare it, how did you test it, how well did it work, and how would it survive in the real world. The best preparation is to write out your real answers in advance, with numbers, then practise saying them out loud in two minutes or less.

Why it matters: Interviewers often spend thirty to fifty per cent of a technical round on your projects.

Takeaway: Prepare specific answers with real numbers for all sixteen.

WORKED EXAMPLE, STEP BY STEP**Answer "Why did you use that metric?" in three parts**

Project: a fraud model. The interviewer asks why you used PR-AUC and not accuracy.

- 1. The choice:** "I optimised PR-AUC during model selection and reported precision at the review team's capacity of 200 cases a day."
- 2. The reason, with a number:** "Fraud was 0.4% of transactions, so a model that never flags anything is 99.6% accurate. Accuracy could not tell good models from useless ones. PR-AUC focuses on how well the model ranks the rare positive class."
- 3. The trade-off you accepted:** "PR-AUC is harder to explain to the business, so for them I translated it into precision at 200: 38% of reviewed cases were fraud, against 9% under the old rules."
- 4. The follow-up you invite:** "If they could review more cases, I would move the threshold and report the cost per extra fraud caught."
- 5. Keep it short:** about 30 seconds, then stop and let the interviewer choose where to go next.

What this shows. Choice, reason with a number, trade-off. The same three parts answer "why this split?", "why this model?" and "why these features?".

YOUR TURN (COVER THE SOLUTION FIRST)

1. Answer "How did you validate your model?" in the choice, reason and trade-off format, for a demand-forecasting project.

Solution. Choice: "I used rolling-origin validation: train on all data up to a month, forecast the next month, and repeat over the last six months." Reason: "The model is used to forecast the future, so every test had to be in the future relative to training; a random split would leak information about the same weeks." Trade-off: "Six folds of one month each give a noisier estimate than random folds would, so I reported the mean and the range of MAE across the six months, and kept the final quarter as an untouched test set."

THE FUNDAMENTALS**Problem and data****Preparation****Modeling****Evaluation and impact****Production thinking**

Work-experience candidates (1-2 years): prepare a story with situation, your specific contribution, stakeholder dynamics, a technical obstacle, how you measured impact, and a lesson learned. Interviewers check ownership: "What did *you* do versus the team?"

SAY IT IN 60 SECONDS

"In a project deep dive, the interviewer probes each decision. I prepare a two-minute story: the problem, the data, my approach, the result, and what I learned. Then I expect questions on why this metric, how I validated, how I handled missing data and leakage, why this model over simpler ones, what the errors looked like, how I would deploy and monitor it, and what I would do differently. I answer with the choice, the reason, and the trade-off. If I did not do something, I say so and describe how I would. The catch: honest ownership beats bluffing; I distinguish what I did from what the team did."

KEY TERMS problem and data · preparation · modelling · evaluation and impact · production thinking · ownership

BUILDS ON 14.1 What a strong project demonstrates

HANDS-ON PRACTICE Answer all sixteen questions for your best project in writing, then record yourself answering five aloud.

COMMON MISTAKES Answering with generalities instead of specifics from your project · Claiming team work as your own · Not knowing the numbers in your own README

CHECK YOURSELF**1. "Why did you choose XGBoost?" Give a strong answer shape.**

Baseline logistic regression scored X; boosting captured non-linear interactions and improved Y; I tuned with CV, and accepted lower interpretability, mitigated with SHAP.

2. How do you answer "what would you do differently?"

Name a real limitation and a specific improvement, for example collecting more recent data, better validation by time, or deploying with monitoring.

3. You did only part of a team project. How do you describe it?

State clearly what you owned and decided, and describe the rest as team context, without claiming credit.

14.6 Red flags to avoid

IN PLAIN WORDS

Interviewers have seen the same project mistakes hundreds of times, so they recognise them fast. The seven red flags are all about honesty and understanding: results that are too good to be true, shortcuts in how the model was tested, and claims that do not match reality. Fixing them usually takes an afternoon and raises your credibility a lot.

Why it matters: A single red flag can overshadow a good project.

Takeaway: Question your own results, test properly, and only claim what you did.

WORKED EXAMPLE, STEP BY STEP**Spot the red flags in a project pitch**

A candidate says: *"I built a fraud model with 99.7% accuracy using deep learning. We used all the columns from the transactions table. The team used my model, and I improved revenue by 30%."*

1. **"99.7% accuracy"** on fraud, where positives are well under 1%: probably meaningless, since predicting "no fraud" every time is about as accurate. The interviewer will ask for recall and precision.
2. **"Deep learning"** with no baseline: why not logistic regression or gradient boosting? Without a comparison, the choice looks like fashion.
3. **"All the columns"**: did that include chargeback or dispute fields recorded *after* the fraud? A strong smell of leakage.
4. **"We" and "I" mixed**, and a claimed 30% revenue gain with no way to measure it: credibility drops.
5. **A better version**: "Fraud was 0.5% of transactions. A logistic baseline reached 0.31 PR-AUC; gradient boosting reached 0.46. I removed three post-event fields that leaked the outcome. At the review team's capacity, 40% of flagged cases were fraud, against 12% under the old rules. Rolling out the model was a team effort; I built the features and the evaluation."

What this shows. Every red flag has the same cure: real metrics, a baseline, a leakage check, and honest ownership.

YOUR TURN (COVER THE SOLUTION FIRST)

1. A resume line says "Expert in Python, R, SQL, Spark, Hadoop, TensorFlow, PyTorch, Kubernetes, Tableau and Power BI." What is the risk, and how would you fix it?

Solution. Every tool listed is fair game for questions, and a shallow answer on any of them undermines the claim of expertise everywhere. Keep only the tools you can discuss for five minutes with an example, group them (for example Languages: Python, SQL; ML: scikit-learn, XGBoost; BI: Power BI), and drop the word "expert". Show depth through project bullets that use the tools instead.

THE FUNDAMENTALS

- Cannot explain the math or assumptions behind the model in your own project
- 99% accuracy without questioning leakage or imbalance
- Tuning on the test set
- Random split for time-dependent data
- No baseline
- Claiming production deployment when it was a notebook
- Copy-pasted GitHub code you cannot explain line by line

SAY IT IN 60 SECONDS

"Interviewers get worried by a few patterns. Not being able to explain your own project's choices. Claiming results without a baseline or proper validation. Leakage you did not notice, such as near-perfect accuracy. Using a model for its own sake where a simple method would do. Ignoring the business context and costs. Listing tools you cannot discuss. Blaming others, or bluffing when you do not know. Copy-paste tutorial projects with no original thought. The fix for each is preparation and honesty: know your projects deeply, show a baseline, state limitations, and say 'I am not sure, but here is how I would reason'. The catch: a suspiciously perfect score invites a leakage question, so I check first and mention the checks."

KEY TERMS leakage · tuning on test · random split for time · no baseline · fake deployment · copy-paste code

HANDS-ON PRACTICE Audit one of your projects against the seven red flags and fix every one you find.

COMMON MISTAKES Overclaiming deployment · Not being able to explain code in your own repository

CHECK YOURSELF

1. Your model has 99.5% accuracy on a hard problem. What will an interviewer suspect?

Leakage or a trivial baseline on imbalanced data; be ready to describe the checks you ran.

2. What is the best response to a question you cannot answer?

Admit it briefly, reason from fundamentals aloud, and say how you would verify.

3. Why is listing 30 tools on a resume risky?

Each is fair game for questions; shallow knowledge shows. List what you can discuss in depth.

Interview tips, tricks and hacks

- 1. Prepare a two-minute and a ten-minute version of each project.** The short one: problem, data, approach, result, lesson. The long one adds decisions and trade-offs. Rehearse both aloud.
- 2. Open with the problem and the result, not the method.** "I predicted which customers would leave, and the model found twice as many as the existing rule." Then explain how.
- 3. Write a decision log while you work.** One line for each choice and its reason. It becomes your interview answers.
- 4. Always name the baseline and the gain over it.** A score without a baseline cannot be judged.
- 5. Prepare the ten deep-dive answers in writing.** Target, split, metric, model choice, leakage, features, errors, deployment, impact, what you would change.
- 6. Volunteer a limitation.** "The data covers only six months, so seasonal effects are not captured." It builds trust and steers the conversation to ground you have prepared.
- 7. Use "I" for your work and "we" for the team's.** If asked about a part you did not do, say so and describe what you understand of it.
- 8. Verify every number on your resume against your repository.** You will be asked how it was measured.
- 9. Make the repository readable.** A README with the problem, the result and how to run it; pinned dependencies; no notebooks named "final_v3".
- 10. Add something to a common dataset if you must use one.** A new question, extra data, a deployed demo or a rigorous evaluation. Otherwise choose another.

Frequently asked questions

21 questions interviewers ask on this chapter, with model answers. Cover the answer, say yours aloud, then compare.

WHAT A STRONG PROJECT DEMONSTRATES

Q1 You have 99% accuracy on your fraud detection project. What is the first thing an interviewer will ask and how should you answer?

They will ask about class balance and leakage. A good answer: "Fraud was 0.6% of rows, so accuracy is meaningless (a model that always says 'legit' scores 99.4%). I evaluated with PR-AUC and recall at a fixed precision; the baseline logistic regression got PR-AUC 0.31 and the final LightGBM 0.47. I checked leakage by making sure features used only information before the transaction time and by splitting on time." Show you already asked the tough questions yourself.

RECOMMENDED PORTFOLIO AND PROJECT IDEAS

Q2 You are targeting BFSI roles. Which three projects do you build?

(1) A **credit-risk scorecard**: logistic regression with WoE binning and IV feature selection, KS, AUC/Gini, PSI for stability, and a clear interpretation of each variable. (2) A **fraud or default prediction** project emphasising class imbalance, cost-based threshold selection and leakage control. (3) A **delivery piece**: a FastAPI scoring service or a Power BI/Tableau dashboard of portfolio risk (vintages, delinquency buckets), showing you can communicate to risk teams. Mention SQL/SAS skills in the README.

PROJECT TEMPLATE AND WORKFLOW CHECKLIST

Q3 What goes in the "Limitations" section of a README, and why include it?

Data limits (size, time span, label noise, geography), modelling assumptions (for example stationary behaviour), known failure modes found in error analysis, what was *not* tested (other segments, longer horizons), and what would be needed for production (monitoring, retraining, fairness checks). Including it shows maturity and honesty, and it pre-empts the interviewer's hardest questions.

TURNING A PROJECT INTO RESUME BULLETS

Q4 An interviewer points at "improved PR-AUC from 0.46 to 0.63" and asks "how did you validate that?". What should you be ready to explain?

The split (time-based or stratified CV), the size and class balance of the evaluation set, how the baseline was tuned (fairly), whether the improvement is statistically meaningful (confidence intervals or multiple folds), the absence of leakage, and what 0.63 means operationally (for example recall at the contact budget). Have the evaluation notebook ready to show.

THE PROJECT DEEP-DIVE QUESTIONS

Q5 Why this problem? Who would use the output and what decision changes?

Strong answer: name the user and the decision, and the cost of the current approach. *Example:*

"Retention managers were giving blanket discounts to everyone who lapsed, costing about 18% of margin. The model ranks customers by churn risk so they can target only the top decile, which is the decision it changes."

Q6 Where did the data come from? Size, time span, grain, quality issues?

Strong answer: source, size, time span, grain, known quality issues and licence. *Example:* "A public telecom dataset with 7,043 customers, one row per customer at a single snapshot; 26.5% churn; some `TotalCharges` were blank strings for new customers which I converted and imputed with zero tenure logic."

Q7 How did you define the target? Any ambiguity or label noise?

Strong answer: the exact definition, why that window, and acknowledged ambiguity or noise.

Example: "Churn = cancelled within the observation period as recorded in the source. I noted that the dataset gives no time dimension, so I could not define a prediction horizon; in production I would define churn as no order in the next 30 days from a snapshot date."

Q8 How did you handle missing values, outliers, duplicates, and categorical variables, and why?**Strong answer:** each choice with the reason.*Example:* "Missing `TotalCharges` (11 rows) were new customers, so zero was right; I used median imputation inside a pipeline for other numerics, one-hot for low-cardinality categoricals, and checked duplicates. I capped monthly charges at the 99th percentile after confirming they were genuine but extreme."**Q9 Which features mattered? Which did you engineer and what was the logic?****Strong answer:** top features with business logic and any engineered ones. *Example:* "Contract type, tenure and monthly charges dominated. I engineered `charges_per_tenure_month` and a flag for customers with multiple add-ons but no tech support; SHAP confirmed month-to-month contracts and short tenure as the strongest drivers, which matches domain intuition."**Q10 How did you avoid leakage?****Strong answer:** a concrete leakage audit. *Example:* "I fit all transformers inside a scikit-learn Pipeline so cross-validation refit them per fold, removed a column recording the cancellation reason since it only exists after churn, and checked that no feature was derived from the target. A suspiciously perfect feature would have been a red flag; none appeared."**Checklist to quote.** Features available at prediction time; preprocessing fit on training folds only; no duplicate or same-entity rows across split; time-ordered split for temporal data; suspicion of near-perfect scores.**Q11 Why this algorithm? What else did you try and why did it lose?****Strong answer:** compare at least two families with reasons. *Example:* "Logistic regression was the interpretable baseline; LightGBM beat it on PR-AUC (0.63 vs 0.46) by capturing interactions between contract type and tenure. A random forest was close but slower to tune; a neural net was overkill for 7k rows. I kept the logistic model as a documented fallback."**Q12 How did you split and validate? Why that strategy?****Strong answer:** the split and why it fits the problem. *Example:* "Stratified 5-fold CV for model selection and a held-out 20% test set used once. With real data I would split by time (train on earlier months, test on later) because customers' behaviour drifts."**Q13 Which metric did you optimize and why? Why not accuracy?****Strong answer:** a metric tied to cost, and why not accuracy. *Example:* "PR-AUC for ranking, recall within the 30% of customers the retention team can contact, and expected profit at the chosen threshold. Accuracy would be 73% by always predicting 'no churn'."**Example.** For fraud at 0.5% prevalence, say: "accuracy would be 99.5% for a model that finds nothing; I optimised recall at a precision of at least 40%, because reviewers can handle limited alerts."**Q14 How did you tune hyperparameters? What was the final configuration?****Strong answer:** the method, the search space, and the final values with discipline about the test set.*Example:* "Randomised search, 40 trials, on `num_leaves` (8 to 64), `learning_rate` (0.01 to 0.2, log scale), `min_child_samples` and subsampling, using 5-fold stratified CV on the training data with early stopping; final: 31 leaves, 0.05 learning rate, about 350 trees. The test set was touched once."**Q15 How did you handle class imbalance?****Strong answer:** what you tried and what worked.*Example:* "Churn was 26.5% (mild imbalance), so I used stratified splits and `class_weight` first, then tuned the threshold on validation for F1 and cost rather than using 0.5. SMOTE gave no gain in PR-AUC, so I dropped it."**Order of tries.** Class weights and threshold tuning first (no data changes), then undersampling or SMOTE inside the training folds, and PR-AUC as the metric throughout.**Q16 What are the model's failure modes? Where does it perform worst?****Strong answer:** specific segments and examples where it fails. *Example:* "Error analysis showed lower recall for customers with tenure under three months (very little history) and for fibre-optic customers on month-to-month contracts, where churn is driven by service issues not in the data. I documented these as limitations."**How to find failure modes.** Group predictions by segment (region, device, new versus old users) and by feature bins; compare error metrics; read 20 mistakes by hand to find patterns.

Q17 How do you explain predictions to stakeholders?

Strong answer: global and local explanations in plain words. *Example:* "SHAP summary plots for global drivers and per-customer waterfall plots for the retention agent: 'high risk because month-to-month contract, 4 months tenure, and two support tickets'. I avoid causal language since drivers are associations."

Q18 What is the estimated business impact or cost/benefit?

Strong answer: a transparent, assumption-labelled estimate. *Example:* "Per 1,000 customers with 26.5% churn there are about 265 churners. Contacting the top 30% (300 customers) captures 71% of them, about 188 churners (63% precision). With an offer costing ₹300, a 30% save rate and a ₹6,000 customer value, that is $56 \text{ saved customers} \times ₹6,000 = ₹3.4 \text{ lakh}$ of retained value against ₹0.9 lakh of offer cost, a net of roughly ₹2.5 lakh per 1,000 customers. It is a projection that depends on the save-rate assumption, which an A/B test should confirm."

Q19 How would you deploy, monitor, and retrain it? What could go wrong (drift, latency, feature availability)?

Strong answer: batch vs real-time choice, serving, monitoring and retraining. *Example:* "Weekly batch scoring in a scheduled job, model packaged in a Docker image with a FastAPI endpoint for ad-hoc scoring. Monitor PSI on inputs and score distribution, track offer conversion, and retrain monthly or when PSI exceeds 0.25. Risks: drift, feature availability and feedback loops from offers."

Say all four. Serving mode (batch or online); monitoring (data quality, drift, performance with delayed labels); retraining trigger (schedule or drift alert); rollback and canary release.

Q20 What would you do differently with more time/data?

Strong answer: realistic, prioritised improvements. *Example:* "Get time-stamped data for a proper time-based split; add service-quality features (delivery delays, complaints); try an uplift model to target customers who will respond to an offer, not just those who might churn; and run a live A/B test of the retention policy."

RED FLAGS TO AVOID**Q21 You realise during an interview that your project has a leakage bug you had not noticed. What do you do?**

Say so, calmly: explain how you spotted it, what it means for the reported numbers, and how you would fix and re-evaluate (for example re-run with the feature removed or the split corrected). Honest self-correction under pressure is a strong signal; hiding or defending a flawed result is a red flag.

15

CHAPTER 15 · HIGH PRIORITY · MODULE 15 IN THE APP

Resume, Behavioral and HR Round

The parts of the process that are about you: the resume that earns the interview, the stories that show how you work, and the conversation about pay.

5 topics · about 1 h 40 min in the app · 14 questions answered

15.1 Resume checklist

15.2 "Tell me about yourself"

15.3 Behavioural questions and STAR stories

15.4 HR and logistics questions, including salary

15.5 Interview-day tips

Foundations

Candidates often treat the non-technical rounds as formalities. They are not. A resume decides whether you are interviewed at all. Behavioural questions decide whether the team wants you. The final conversation decides the offer. These rounds reward preparation exactly as technical rounds do, and they are easier to improve, because the questions are almost entirely predictable.

The resume has one job

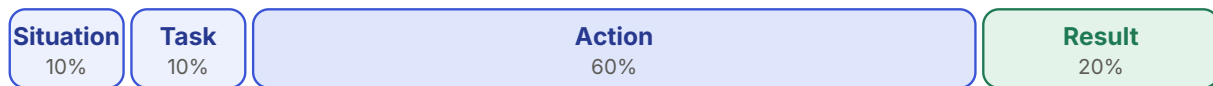
A resume exists to get you an interview. A reviewer may give it thirty seconds, so it must be easy to scan: one page for an early-career candidate, a plain layout that automated screening systems can read, the most relevant material in the top half, and achievements written with numbers. Each line is also a promise: anything you list, you may be asked about for five minutes. Remove what you cannot discuss.

Your introduction sets the agenda

"Tell me about yourself" opens most interviews, and a good answer takes about ninety seconds in three parts: the **present** (who you are and your strongest relevant skill), the **past** (two or three experiences that built that skill, with one concrete result), and the **future** (why this role is the next step). It is not your life story. Done well, it also plants the topics you would like to be asked about, because interviewers usually follow up on what you mention.

Behavioural questions want evidence

"Tell me about a time when..." questions rest on the idea that past behaviour predicts future behaviour. They are answered with a specific story, and the standard structure is **STAR**.



Spend most of a two-minute answer on Action and Result; end with a measurable outcome.

Figure 17. Keep the situation and task brief. The action and the result are what the interviewer is assessing.

Describe the situation and your task in a sentence or two, spend most of your time on what you personally did and why, and end with the result, a number if possible, and what you learned. You do not need a story for every possible question. Five or six well-chosen stories can be adapted to almost any prompt: a project you drove from start to finish, a failure and what you changed afterwards, a disagreement you resolved, a problem that was vague until you gave it structure, something you learned quickly, and a time you persuaded someone with data.

The story about failure deserves care. Choose a real one of modest size, own your part plainly, and spend most of the answer on what you did about it and what you do differently now. The interviewer is looking for self-awareness, not perfection.

The conversation about pay and logistics

Questions about salary, notice period and location are part of the assessment. Research the usual range for the role, level and city beforehand. If asked for your expectation early, it is reasonable to say you would like to understand the role and the range for it first; if pressed, give a researched range, and say you are flexible for the right role. Consider the whole offer, including learning and growth. Be honest about other offers and about your current pay: an offer made on false information can be withdrawn.

On the day

Check the practical details in advance: the link, your camera and microphone, a quiet place. During the interview, listen to the whole question, take a moment to think, and structure your answer. If you do not know something, say what you do know and how you would find out. Afterwards, write down every question while you still remember it.

Topic by topic

Each topic gives the idea in plain words, the fundamentals to know, a model spoken answer and three questions to test yourself. Open the same topic in the app for worked examples, runnable code and interactive demos.

15.1 Resume checklist

IN PLAIN WORDS

A resume is a one-page menu for the interviewer: they will order from it, so only list what you can cook well. Put your contact details and links at the top, a short tailored summary if you like, your skills grouped clearly, your best experience and projects with results, and your education. Keep the format plain so the computer that scans resumes (the ATS) can read it, and use words from the job description where they are honestly true. Every skill you list is an invitation to be grilled on it.

Why it matters: The resume decides who gets an interview, then steers the questions you will face.

Takeaway: One page, results-focused, honest, plain formatting, tailored keywords.

WORKED EXAMPLE, STEP BY STEP

The 30-second scan test

A recruiter gives your resume about 30 seconds and reads mostly the top third. Check what that top third says.

- 1. Cover everything below the top third** and read only what is visible.
- 2. Does it say who you are and what you want?** A one-line headline such as "Data analyst moving into data science; SQL, Python, experimentation" beats a paragraph of adjectives.
- 3. Is your strongest evidence there?** The best project or job bullet, with a number, should be visible in that first glance, not on line 25.
- 4. Is it machine-readable?** A single column, standard headings (Experience, Projects, Skills, Education), no tables or text inside images. Applicant tracking systems read text, not layout.
- 5. Cut anything you cannot defend:** tools you used once, vague phrases such as "hard-working team player", and generic coursework.
- 6. Repeat the test with a friend:** give them 30 seconds, take the resume away, and ask what they remember. If it is not your top skill and your best result, reorder.

What this shows. A resume is an advertisement for an interview, read in seconds. Put the strongest evidence where the eye lands first.

YOUR TURN (COVER THE SOLUTION FIRST)

1. A fresher's resume opens with an objective ("Seeking a challenging role in a reputed organisation..."), then education, then a skills list of 20 tools, and puts projects on page 2. What three changes would you make?

Solution. (1) Replace the objective with a one-line headline naming the target role and two or three core skills. (2) Move the two strongest projects, with outcome numbers, directly under the headline, above education, keeping the page to one. (3) Cut the skills list to the tools actually used in those projects, grouped by type, so every listed tool has evidence behind it.

THE FUNDAMENTALS

- ☐ Header with email, phone, LinkedIn, GitHub, portfolio
- ☐ 2-line summary (optional), tailored to the role
- ☐ Skills grouped: Languages (Python, SQL), Libraries (pandas, scikit-learn, PyTorch), Tools (Git, Docker, Tableau, Airflow), Cloud (AWS/Azure/GCP)
- ☐ Experience/internships with quantified impact bullets
- ☐ 3-4 projects with links and one-line results
- ☐ Education, relevant certifications (only meaningful ones), Kaggle rank, publications, hackathons
- ☐ ATS-friendly: simple formatting, keywords from the JD, PDF, no photos/tables/graphics
- ☐ Every skill listed is one you can be grilled on

SAY IT IN 60 SECONDS

"A data science resume is one page for early career, scannable in thirty seconds. At the top: contact and links to GitHub or portfolio. Then a short skills section grouped by area, projects or experience as impact-driven bullets with numbers, and education. Keep layout plain, one font, no photos or tables that applicant tracking systems cannot read. Tailor keywords to the job description honestly. Every line should be something I can discuss for five minutes. Proofread for typos. Save as a PDF named with my name. The catch: the resume's job is to earn the interview, so I put the most relevant and strongest items on the first half of the page."

KEY TERMS one-page resume · ATS · keywords · quantified bullets · skills grouping · projects section

HANDS-ON PRACTICE Rewrite your resume to one page using the checklist, then have a friend mark every line they would ask about.

COMMON MISTAKES Listing skills you cannot defend · Two pages of tools with no outcomes · Fancy formatting that breaks parsers

CHECK YOURSELF**1. Why use a plain layout?**

Applicant tracking systems parse text; tables, columns and graphics can scramble it, and humans scan simple layouts faster.

2. How do you tailor a resume to a job description?

Mirror honest keywords, reorder bullets so the most relevant projects come first, and highlight matching skills.

3. How long should an early-career resume be?

One page.

15.2 "Tell me about yourself"

IN PLAIN WORDS

"Tell me about yourself" is not a request for your life story; it is your 90-second trailer. Say who you are now (your present focus), how you got here (one or two highlights with a result), and why you are excited about this role (the future). Practise it until it is smooth but not robotic, and change the last sentence for every company.

Why it matters: It is the first question in almost every interview and sets the tone.

Takeaway: Present, past, future: sixty to ninety seconds, tailored ending.

WORKED EXAMPLE, STEP BY STEP

Write a 90-second introduction

About 200 words, spoken. Present, then past, then future.

- 1. Present (about 20 seconds):** "I'm a data analyst at a logistics company, where I build SQL reporting and forecasting for the operations team. My strongest skill is turning messy operational data into decisions."
- 2. Past, the evidence (about 45 seconds):** "Over the last two years I automated our weekly delivery reports, cutting a day of manual work to minutes, and I built a demand forecast that reduced rider shortages at peak hours by about 15%. To go deeper into modelling, I completed a churn project end to end: features as of a prediction date, a time-based split and a cost-based threshold."
- 3. Future, why this role (about 20 seconds):** "I'm looking for a data science role where forecasting and experimentation drive the product, which is why your team's work on delivery-time prediction interests me."
- 4. Stop.** No life story, no hobbies unless asked.
- 5. Plant hooks:** the forecast and the churn project are the two things you *want* to be asked about. You have just chosen the next question.
- 6. Rehearse aloud with a timer** until it lands between 75 and 100 seconds and sounds conversational.

What this shows. The introduction sets the agenda. Present, past and future, with one concrete result, steer the interview towards your best material.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Write only the "future" sentence for a candidate applying to a fintech company's risk analytics team, who has built a credit-default project.

Solution. For example: "I want to work where models directly shape lending decisions and have to be explainable to risk and compliance teams, which is what drew me to your risk analytics team; my credit-default project, where I built a scorecard with reason codes, is the closest I have come so far, and I'd like to do it with real portfolio data." Specific to the team, linked to evidence, and forward-looking.

Structure: **Present** (who you are, current focus) → **Past** (education/experience highlights, key project with result) → **Future** (why this role/company).

"I'm a data science graduate with a year of experience at [X], where I built a demand-forecasting model that cut stock-outs by 12%. I'm strongest in SQL, Python, and classical ML, and I enjoy turning messy data into decisions. Recently I built a RAG assistant over policy documents and deployed it with FastAPI. I'm excited about [Company] because of [specific data problem], and I'd like to contribute to [team area] while growing in [skill]."

SAY IT IN 60 SECONDS

"'Tell me about yourself' is a ninety-second pitch. I use present, past, future. Present: who I am now, my role or studies and my strongest relevant skill. Past: two or three experiences or projects that built that skill, with one concrete result. Future: why this role at this company fits my direction. I tailor it to the job and do not recite my whole resume or personal history. I end with something that invites a follow-up, like a project I am proud of. I rehearse it aloud until it sounds natural, not memorised. The catch: it is the first impression and sets the questions I will get, so I plant hooks for topics I want to discuss."

KEY TERMS present past future · 60 to 90 seconds · key project with result · why this company

HANDS-ON PRACTICE Write and record a 75-second answer; listen once for clarity and once for energy, then trim.

COMMON MISTAKES Reciting the resume chronologically · Going over two minutes · Ending without a link to the role

CHECK YOURSELF

1. What are the three parts of the answer?

Present (who I am and my core strength), past (relevant experience with a result), future (why this role).

2. How long should it be?

About 60 to 90 seconds.

3. Why mention one concrete project result?

It makes the pitch credible and gives the interviewer an easy, favourable follow-up question.

15.3 Behavioural questions and STAR stories

IN PLAIN WORDS

Behavioural questions ("tell me about a time when...") are checking how you actually behave at work. Use STAR: explain the **S**ituation, your **T**ask, the **A**ctions *you* took (the longest part) and the **R**esult, plus what you learned. Prepare five or six real stories in advance; each story can answer several different questions if you tilt it slightly. Keep each to about two minutes.

Why it matters: Hiring managers and GCCs (Amazon-style leadership principles) weight these rounds heavily.

Takeaway: Five or six real stories, told as Situation, Task, Action, Result, learning.

WORKED EXAMPLE, STEP BY STEP**Turn raw notes into a STAR story**

Raw notes: *"Dashboard numbers did not match finance. Big argument. I found the issue. Fixed. Everyone happy."*

- 1. Situation (1 or 2 sentences):** "Our weekly revenue dashboard showed 8% more revenue than finance's report, and leadership had started to distrust both."
- 2. Task (1 sentence):** "As the analyst who owned the dashboard, I had to find the cause before the quarterly review in a week."
- 3. Action (the core, said with "I"):** "I compared the two at order level for one week and found two differences: finance excluded cancelled orders and recorded refunds on the refund date, while the dashboard used the order date. I documented both definitions, met the finance lead to agree one definition per metric, changed the dashboard query, and added a reconciliation check that flags any gap over 1%."
- 4. Result (a number and a lesson):** "The two reports matched within 0.2% from the following week, and the reconciliation check has caught two pipeline issues since. I now agree metric definitions in writing before I build any report."
- 5. Timing:** about 10% situation, 10% task, 60% action, 20% result. Around 90 seconds.

What this shows. STAR turns "I fixed it" into evidence: what you did, how you decided, what changed, and what you learned.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Rewrite this action sentence so the interviewer can assess *you*: "We cleaned the data, built a model and presented it to the client."

Solution. For example: "I wrote the data validation checks that removed 4% duplicate records, I built and compared the baseline and gradient-boosting models, and I prepared and delivered the five-slide summary of the results to the client's operations head; my teammate handled the data extraction." Specific actions with "I", and honest credit for the parts others did.

THE FUNDAMENTALS

STAR: Situation, Task, Action, Result (plus learning).

Prepare **5-6 stories** that can be adapted to many questions.

SAY IT IN 60 SECONDS

"Behavioural questions ask for past examples. I answer with STAR: Situation, Task, Action, Result. Briefly set the context, say what my responsibility was, describe what I specifically did, the 'I' not the 'we', and end with the result, quantified if possible, and what I learned. I keep five to six prepared stories that cover conflict, failure, tight deadlines, leadership, ambiguity and influencing without authority, and reuse them flexibly. Most of the time should be on Action and Result. For failure questions, choose a real, modest failure and show the learning. The catch: a story with no clear result, or one that blames others, fails; I end every story with the outcome."

KEY TERMS STAR method · ownership · failure · conflict · ambiguity · influence · deadline · feedback · impact · story bank

HANDS-ON PRACTICE Write a STAR outline for each of your six stories, then map each story to at least three themes.

COMMON MISTAKES Describing the team's work instead of your own actions · Stories with no result or learning · Reusing one story for everything without adapting

CHECK YOURSELF

1. What does STAR stand for, and where should most of the time go?

Situation, Task, Action, Result; spend most time on Action and Result.

2. "Tell me about a failure." What makes a good answer?

A real, owned failure with limited stakes, what you did to fix it, and a concrete change in how you work since.

3. How many stories should you prepare?

Five or six flexible ones that can be adapted to different questions.

15.4 HR and logistics questions, including salary

IN PLAIN WORDS

The HR round checks that you will be a good, reliable colleague and that the practical details (salary, notice period, location) line up. Answer honestly and positively: show real motivation for data science and for this company, name a real weakness you are actively improving, and talk about why you are moving *towards* something, not running away. For salary, do your research, give a range, and focus on the total package and growth, not one number.

Why it matters: Offers can be lost or left unnegotiated here.

Takeaway: Be prepared, be honest, be positive, and research before you talk numbers.

WORKED EXAMPLE, STEP BY STEP

Compare two offers by total value, not by the headline

	Offer A	Offer B
Fixed pay	14.0 lakh	12.5 lakh
Variable (target)	10% of fixed	20% of fixed
Joining bonus	none	1.0 lakh (one-time)
Stock (ESOPs)	none	2.0 lakh a year, vesting over 4 years
Learning	small team, no mentor	senior data scientists, structured reviews

1. Expected cash in year 1. A: $14.0 + 1.4$ (if the variable is paid in full) = 15.4 lakh. B: $12.5 + 2.5 + 1.0 = 16.0$ lakh.

2. Discount what is uncertain. If variable pay usually comes in at about 70% of target: A becomes $14.0 + 0.98 = 14.98$; B becomes $12.5 + 1.75 + 1.0 = 15.25$.

3. Year 2 onwards (no joining bonus): A ≈ 14.98 , B ≈ 14.25 in cash, plus B's stock, whose value depends on the company and on liquidity. Treat it as a possible upside, not as salary.

4. **Add what does not fit in a table:** B offers mentorship and harder problems, which compound into your next salary.
5. **Negotiate with data:** "I'm excited about B. Given A's higher fixed pay, could the fixed component move to 13.5 lakh?" Calm, specific and honest about the existence of the other offer.

What this shows. Compare offers on expected, risk-adjusted value over two or three years, plus learning, not on the headline number.

YOUR TURN (COVER THE SOLUTION FIRST)

1. A recruiter asks for your expected salary in the first call. You have researched that the range for the role is 12 to 16 lakh. What do you say?

Solution. For example: "I'd like to understand the role and its scope first. Could you share the budgeted range?" If pressed: "Based on my research for this role and level in this city, I'm looking at the 14 to 16 lakh range for fixed pay, and I'm flexible depending on the overall package and the learning the role offers." This anchors with research, stays in the upper part of the market range, and leaves room to negotiate.

THE FUNDAMENTALS

- Why data science? Why this company/role? Where do you see yourself in 3-5 years?
- Strengths and weaknesses (real weakness + mitigation).
- Why are you leaving your current job (positive framing, no badmouthing)?
- **Notice period, current CTC, expected CTC:** research ranges on AmbitionBox, Glassdoor, Levels.fyi, LinkedIn; give a researched range, emphasize flexibility based on total package and growth. Know fixed vs variable, joining bonus, ESOPs.
- Relocation, work mode (remote/hybrid), bond/service agreements (read carefully).
- **Questions to ask them:** What does success look like in 6 months? What does the data stack/team structure look like? How are models deployed and monitored? How do data scientists work with product/engineering? What are current biggest data challenges?

SAY IT IN 60 SECONDS

"HR and logistics rounds cover motivation, notice period, current and expected compensation, location and timeline. I prepare honest, concise answers. 'Why this company?' needs specific research on its product and data challenges. For salary, I research the market range first and I try to let them name a range first; if pushed, I give a researched range, with my target in the lower half, based on role and market rather than my current pay, where permitted. I also consider total compensation: bonus, equity, growth and learning. I am polite and flexible but know my floor. The catch: negotiating happens after an offer, so I stay positive and collaborative and never lie about other offers or my current pay."

KEY TERMS why data science · strengths and weaknesses · leaving your job · notice period · CTC · expected CTC · ESOPs · questions to ask

HANDS-ON PRACTICE Write your answers to "why data science?" and "why this company?" and research the salary range for one target role on two sites.

COMMON MISTAKES Naming a number before researching the range · Badmouthing the current employer · Having no questions for the interviewer

CHECK YOURSELF**1. How do you answer "What are your salary expectations?" early on?**

Say you would like to learn the range for the role; if pressed, give a researched market range for the role and level, and say you are flexible for the right fit.

2. What should a "why this company" answer include?

Specific product or data problems that interest you, a link to your skills, and evidence you researched.

3. Besides base pay, what else do you compare in offers?

Bonus, equity, benefits, growth, team and learning opportunities, and flexibility.

15.5 Interview-day tips

IN PLAIN WORDS

On the day, how you think matters as much as what you know. Say your thoughts aloud, ask questions before jumping in, start with a simple version, and test it on a tiny example including awkward cases. If you are stuck, say what you do know, propose an approach, and ask a targeted question: silence hurts more than a partial answer. If you genuinely do not know something, say so and explain how you would find out.

Why it matters: Interviewers decide from your reasoning, not just your final answer.

Takeaway: Clarify, think aloud, start simple, test, and be honest.

WORKED EXAMPLE, STEP BY STEP**The day, hour by hour**

- 1. The day before:** confirm the time zone and the link, test camera, microphone and screen sharing, and charge the laptop. Reread your resume, project notes and the job description. Stop studying by early evening and sleep properly.
- 2. One hour before:** a light meal and water nearby; a pen and paper; your three questions to ask. Close notifications.
- 3. The first five minutes:** smile, give your 90-second introduction, and listen closely to how the interviewer describes the round.
- 4. For each question:** restate it, ask one clarifying question, think aloud with a structure, check in ("does that answer what you were asking?").
- 5. If you go blank:** say "Let me think about this for a moment", write down what you know, and solve a smaller version aloud. Silence for 20 seconds with a visible plan is fine; panic is not.
- 6. The last five minutes:** ask your questions about the team's problems and how success is measured.
- 7. Within an hour of finishing:** write down every question you were asked and how you did; send a two-line thank-you note.

What this shows. Most interview-day failures are avoidable: logistics, nerves, and not using the last five minutes. A routine handles all three.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Halfway through a SQL question you realise your join has been wrong for five minutes. What exactly do you say and do?

Solution. Say it plainly: "I've just noticed my join multiplies rows, because order_items has several rows per order. Let me fix that by aggregating the items first." Then fix it, and test the corrected query on two or three rows aloud. Catching and fixing your own bug is a positive signal; interviewers watch how you debug, not just whether the first draft was perfect.

THE FUNDAMENTALS

- Think aloud, clarify requirements before coding, state assumptions.
- For SQL/coding: start simple, test with a small example and edge cases (NULLs, ties, empty input).
- If stuck: say what you know, propose an approach, ask a targeted question. Silence hurts more than partial progress.
- Be honest about what you don't know; show how you would find out.

SAY IT IN 60 SECONDS

"On interview day, preparation is mostly logistics and mindset. Before: test the video link, camera, mic and a quiet space; have water, a notepad and the job description; review my project stories. During: listen fully, clarify the question, think aloud, structure the answer, and check in. If stuck, state what I know and reason from basics; silence while thinking is fine if I say so. In coding, I talk through the approach before typing and test with examples. After: send a short thank-you note and write down what was asked while it is fresh. The catch: nerves are normal, so I slow down and breathe, since a calm, structured answer scores higher than a rushed correct one."

KEY TERMS think aloud · clarify · edge cases · honesty · partial progress · small examples

HANDS-ON PRACTICE Do one mock interview where a friend interrupts you with "what if the input is empty?" and "what is the complexity?" and notice how you respond.

COMMON MISTAKES Silence when stuck · Coding before clarifying · Bluffing instead of saying what you do know

CHECK YOURSELF

1. You blank on a SQL question. What do you do?

Restate the problem, write the output columns and a small example, break it into steps (join, filter, aggregate), and talk through each; progress aloud earns credit.

2. What should you do in the first minute of a case question?

Clarify the goal and context, restate the problem, and outline your approach before diving in.

3. What do you do after each round?

Note the questions and your weak spots, and send a brief thank-you message.

Interview tips, tricks and hacks

- 1. Write and rehearse a ninety-second introduction.** Present, past, future, tailored to the role. Say it aloud until it sounds like you talking, not reciting.
- 2. Build a story bank of six.** For each story write four lines: situation, task, action, result. Before an interview, map them to likely questions.
- 3. Spend sixty per cent of a story on your actions.** Use "I". Interviewers cannot assess "we".
- 4. End every story with a number and a lesson.** "Turnaround fell from three days to one, and I now agree the definition of the metric before starting any analysis."
- 5. Choose a real failure.** Not a disguised strength. Own it in one sentence, then describe the fix and the lasting change.
- 6. Keep the resume to what you can defend.** For each line, ask: could I talk about this for five minutes? If not, cut it.
- 7. Research "why this company" with one specific data problem of theirs.** Generic praise is forgettable.
- 8. Do not name a salary number first if you can avoid it.** When you must, give a researched range for the role and market.
- 9. Prepare three questions to ask.** About the team's current problems, how success is measured in the role, and what the first three months look like. Asking nothing reads as low interest.
- 10. Pause before answering.** "Let me think for a moment" is a sign of composure. Then structure the answer.
- 11. Send a short thank-you note and log the questions.** Each interview should make the next one easier.

Frequently asked questions

14 questions interviewers ask on this chapter, with model answers. Cover the answer, say yours aloud, then compare.

RESUME CHECKLIST

Q1 Your resume lists "Spark" but you have only run a tutorial. The interviewer asks about it. What do you say?

Be honest about depth: "I completed a PySpark tutorial and used it to process a 5 GB dataset in a personal project; I understand lazy evaluation, shuffles and broadcast joins conceptually but have not run production jobs." Then show the conceptual knowledge. Better still, remove or qualify it ("Spark (basic)") before the interview.

"TELL ME ABOUT YOURSELF"

Q2 What do you do if the interviewer follows up with "Why are you changing roles?"

Give a positive, forward-looking answer: what you want more of (modelling, experimentation, ownership) rather than what you dislike. "I've learned a lot in analytics and I want to work on the prediction and experimentation side more deeply; this role gives me that." Do not criticise your current employer.

BEHAVIOURAL QUESTIONS AND STAR STORIES**Q3 Tell me about a project you drove end to end.**

Ownership. Choose a project where you drove the scope, not just the code. Cover: how you framed the problem with stakeholders, key decisions and trade-offs (metric, data, model), obstacles you unblocked, how you measured impact, and what you would change. Make the **result** concrete (a metric moved, a decision changed).

Q4 A time your model/analysis was wrong or a project failed.

Failure. Pick a real but not catastrophic mistake. Explain what went wrong (your role in it, not blame), how you noticed, what you did to contain and fix it, and, most importantly, the **specific process change** you made afterwards (for example adding a leakage check, a data validation step or a code review). Interviewers assess accountability and learning.

Structure. Situation (one sentence), what went wrong and your part in it, how you detected it, the fix, and the new habit: for example adding a data-quality check or reviewing assumptions with a colleague.

Q5 Disagreement with a teammate/stakeholder.

Conflict. Show respectful, data-driven handling: what the disagreement was about, how you sought to understand the other view, what evidence or experiment you proposed, how you reached a decision (even if you were overruled) and what the relationship looked like afterwards. Avoid villains and "I was right" endings.

Q6 A vague problem statement you had to structure.

Ambiguity. Describe a vague request ("improve engagement") and how you structured it: clarifying questions, defining metrics, breaking it into testable pieces, prioritising by impact and effort, aligning with stakeholders on scope, and delivering an iterative result. The hallmark is turning fog into a plan.

Q7 Something technical you learned quickly and applied.

Learning. Show a fast, deliberate learning loop: what you needed to learn and why, how you learned it (docs, a small prototype, a mentor), how you applied it within a deadline, and the outcome (for example picking up PySpark in two weeks to process a 50 GB file). Emphasise application, not just study.

Q8 Convincing a non-technical stakeholder with data.

Influence. Explain how you tailored the message to a non-technical stakeholder: led with the answer and impact, used one clear chart, addressed their concerns and risks, and gave options. State the outcome (they approved, changed a decision). Mention how you handled pushback.

Example. "I had to convince marketing to stop a discount. I showed one chart: customers who got the discount did not buy more than a matched group, and it cost 8% margin. They agreed to a smaller test."

Q9 Tight deadline with trade-offs.

Deadline. Show prioritisation under pressure: what was essential vs nice-to-have, the trade-offs you made and communicated (for example a simpler model with a documented limitation), how you protected quality where it mattered (validation, no leakage) and what you delivered on time. Mention the follow-up to pay down shortcuts.

Q10 Critical feedback you received and acted on.

Feedback. Pick genuinely critical feedback (for example unclear communication or underdocumented code), show you listened without defensiveness, asked for examples, made a concrete change (a README template, summary-first emails) and the improvement that followed, ideally confirmed by the person who gave it.

Q11 Your biggest measurable contribution.

Impact. Choose the contribution with the clearest, most defensible number (revenue, hours saved, error reduced) and explain **your** part versus the team's, how the number was measured, and why it mattered to the business. Be honest about attribution.

Choose carefully. Prefer a result with a measure (time saved, revenue, accuracy over baseline) and a clear link between your action and the outcome. Be ready for "how did you measure it?".

Q12 The interviewer says "Tell me about a time you disagreed with your manager." You only have a mild example. Is that a problem?

No: a mild, real example told well is better than a dramatic invented one. Show how you raised your view with evidence (for example a quick analysis), listened to their constraints, and how you committed to the decision (or how it was resolved). Interviewers look for professionalism and collaboration, not drama.

HR AND LOGISTICS QUESTIONS, INCLUDING SALARY**Q13 The recruiter asks for your current CTC and expected CTC. How do you respond?**

Give your current CTC truthfully if asked directly (it is often verified) with the breakup (fixed vs variable), then steer to the role: "For this role I'm expecting ₹X to ₹Y based on market research and the scope; I'm flexible on the structure and more interested in the overall opportunity. What is the band you have in mind?" Keep it calm and collaborative, and never invent figures.

INTERVIEW-DAY TIPS**Q14 You are asked about a technique you have never used (say, survival analysis). What do you say?**

"I haven't used it directly, but here's my understanding: it models time until an event and handles censored observations (customers who haven't churned yet). I'd use it for churn timing questions, with Kaplan-Meier curves for non-parametric estimates and Cox regression for covariate effects. I'd start with a simple baseline and validate on a time split." Show what you know, connect to a related concept, and say how you would learn the rest.

16

CHAPTER 16 · MODULE 16 IN THE APP

Master Question Bank with Answers

Rapid-fire recall across statistics, machine learning, SQL, Python and cases. Use this chapter for speed, not for first learning.

6 topics · about 2 h 45 min in the app · 59 questions answered

16.1 Question bank: statistics and probability

16.2 Question bank: machine learning I

16.3 Question bank: machine learning II

16.4 Question bank: SQL and Python rapid-fire

16.5 Question bank: case and product

16.6 Self-test: answer aloud in under two minutes

Foundations

By this point you have met every idea in the book. This chapter exists for a different purpose: **retrieval under time pressure**. In an interview you rarely have minutes to assemble an answer. A question arrives and you have a few seconds to begin a clear reply. That fluency is a separate skill from understanding, and it is built only by practising recall, repeatedly, with the answer hidden.

Why testing yourself works

Re-reading notes feels productive because the material looks familiar, but familiarity is not the ability to produce an answer. Each time you force yourself to recall something without looking, the memory becomes easier to reach next time. So the method for this chapter is strict: cover the answer, say yours aloud, then compare. If you only read the questions and answers, you will gain very little from them.

One formula for concept questions



Thirty to sixty seconds, then stop and let the interviewer steer.

Figure 18. A four-beat answer is complete, short and easy to follow.

Almost every short conceptual question, in any subject, can be answered in four beats. **Define** the idea in one plain sentence. **Explain why** it works or matters. Give a small **example**, ideally with numbers. Add a **caution**: the trade-off, the assumption, or the common misunderstanding. For "what is a p-value?": it is the probability of data at least this extreme if there were no real

effect; it tells us how surprising the result is under that assumption; for example, 0.03 means about three such results in a hundred no-effect experiments; it is not the probability that the hypothesis is true. That takes under thirty seconds and leaves nothing important out.

The shapes questions come in

Recognising the shape of a question tells you how to structure the answer before you know what you will say.

- **"What is X?"** Use the four beats.
- **"X versus Y?"** One sentence on what they share, one on the key difference, one on when to choose each.
- **"Why does X happen?"** Give the mechanism, then a concrete illustration.
- **"What would you do if...?"** Name the diagnosis, then two or three remedies in order of effort.
- **"How would you...?"** State the goal, then steps, then how you would check the result.

How to work through the bank

Take a section at a time. Answer each question aloud with a timer, thirty seconds for a definition and up to two minutes for a case. Mark each one: answered well, hesitant, or missed. For every hesitant or missed question, go back to the matching chapter, then write the answer in your own words. Two days later, attempt only the marked questions again. The list shrinks each round, and what remains in the final week is precisely what you need to revise.

The final self-test

The chapter ends with five questions to answer aloud in under two minutes each, without notes. They test explanation, storytelling, judgement, product sense and clear thinking. Record yourself. Listening back is uncomfortable and extremely effective: you will hear the jargon, the missing example and the rambling that you cannot notice while speaking.

Topic by topic

Each topic gives the idea in plain words, the fundamentals to know, a model spoken answer and three questions to test yourself. Open the same topic in the app for worked examples, runnable code and interactive demos.

16.1 Question bank: statistics and probability

IN PLAIN WORDS

This bank trains the thirty-second statistics answer. Every good answer has the same skeleton: one plain sentence that defines the idea, one tiny example with numbers, and one caution about how people misuse it. For instance, a p-value is how surprising your data would be if nothing were going on; example: $p = 0.03$ means a gap this big appears in about 3 of 100 no-effect experiments; caution: it is not the probability the effect is real. Say each answer aloud before revealing, then compare.

Why it matters: Statistics questions are short, and interviewers judge clarity and correctness within the first two sentences.

Takeaway: Define, example, caution, in under thirty seconds.

SAY IT IN 60 SECONDS

"For statistics questions I use a thirty-second formula. A plain definition first, then a small numeric example, then the common misunderstanding. A p-value: the probability of data at least this extreme if the null is true; with a conversion test, p of 0.03 means a gap like this would happen about three per cent of the time by chance; it is not the chance the null is true. Type one is a false alarm, type two a miss. The central limit theorem: averages become normal as n grows. Outliers: investigate before removing. Then I stop and let the interviewer follow up. The catch: answers without an example sound memorised."

KEY TERMS p-value · Type I and II error · CLT · t-test vs z-test · confidence interval · outliers · covariance · KS test · expected flips · Bayes · multicollinearity

HANDS-ON PRACTICE Cover the answers, say each aloud in 30 seconds, reveal, and rate yourself honestly.

COMMON MISTAKES Giving definitions with no example · Mixing up confidence intervals and probabilities

CHECK YOURSELF

1. Give the example and the caution for "confidence interval".

Example: a 95% CI of $10\% \pm 1\%$ for conversion from 10,000 users. Caution: it does not mean there is a 95% probability that the truth lies in this particular interval.

2. Why can a result be significant but useless?

With huge samples a tiny, practically worthless difference becomes significant; always report effect size and its business value.

3. Expected flips to see two heads in a row: the answer and the idea?

Six flips. Set up expected values with states (no head yet, one head so far) and solve the two linear equations.

16.2 Question bank: machine learning I

IN PLAIN WORDS

These twelve questions are the classic machine learning screen. Behind them sit just a few ideas: models can be too simple or too flexible (bias and variance); there are two ways to combine models (bagging and boosting); penalties shrink weights (L1 and L2); scores can lie (accuracy on imbalanced data, leakage); and thresholds trade precision against recall. A strong answer gives the idea, the reason, an example and the trade-off, in about a minute.

Why it matters: Interviewers use these questions to see whether your fundamentals are solid before going deeper.

Takeaway: Idea, reason, example, trade-off.

SAY IT IN 60 SECONDS

"For the classic ML questions I answer in four beats: the idea, the reason, an example and the trade-off. Bias-variance: simple models underfit, flexible ones overfit; example depth two versus depth thirty trees; control with regularisation and validation. Bagging versus boosting: bagging averages independent models to cut variance; boosting corrects errors in sequence to cut bias. Accuracy on ninety-nine to one data is meaningless, since always predicting the majority scores ninety-nine; use precision, recall and PR-AUC. Trees do not need scaling because they only compare against thresholds. The catch: trade-offs show maturity, so I never leave an answer without one."

KEY TERMS bias-variance · bagging vs boosting · L1 sparsity · XGBoost vs GBM · data leakage · accuracy on imbalanced data · precision recall · ROC vs PR · k-means · decision tree splits · tree scaling · overfitting in neural nets

HANDS-ON PRACTICE Answer each question in under 60 seconds including one example.

COMMON MISTAKES Answering "see the notes" instead of having the answer ready · Forgetting trade-offs

CHECK YOURSELF

1. Answer "bias-variance trade-off" in four beats.

Idea: error splits into bias (too simple) and variance (too sensitive). Reason: flexibility lowers one and raises the other. Example: depth-2 versus depth-30 tree. Trade-off: tune complexity with validation and regularise.

2. Why is accuracy misleading at 99:1 and what do you report?

Predicting always-majority scores 99%. Report precision, recall, PR-AUC and a cost-based threshold.

3. Why does L1 give sparse weights and L2 does not?

L1's constant-size pull (and diamond-shaped constraint) can push weights exactly to zero; L2's pull shrinks with the weight, so weights get small but not zero.

16.3 Question bank: machine learning II

IN PLAIN WORDS

These thirteen questions go one level deeper than the classics: validation schemes, regularisation strength, categorical features with many levels, probability calibration, explaining predictions, and what to do when the offline score does not survive production. The common thread is practical judgement. For each, ask yourself: what would I actually do, what could go wrong, and how would I check? That is the answer.

Why it matters: These questions separate people who have only read about ML from those who have shipped it.

Takeaway: Answer with what you would do, the risk, and the check.

SAY IT IN 60 SECONDS

"These questions are about judgement, so I answer with action, risk and check. Cross-validation estimates generalisation by rotating validation folds; I avoid plain k-fold for time series and grouped data, where I use forward-chaining and group splits. I choose the regularisation strength by validation. For a categorical column with a thousand levels I use target or frequency encoding done out-of-fold, hashing, or embeddings. If a model works offline but not online, I check leakage, training-serving skew, drift and metric mismatch. For small data I use simple models, strong regularisation and cross-validation. The catch: every answer should state how I would verify it."

KEY TERMS cross-validation · regularisation and lambda · gradient descent · high-cardinality categoricals · calibration · SHAP · PCA vs feature selection · RF importance bias · logistic regression to a business person · small data · missing values in production · concept drift

HANDS-ON PRACTICE Answer each in under 60 seconds and add one example from your projects.

COMMON MISTAKES Forgetting time or group structure in cross-validation · Treating importance as causation

CHECK YOURSELF

1. A model's offline AUC is 0.92 but online conversion shows no gain. Name three causes.

Leakage or skew between offline and online features; the metric does not match the business goal (AUC versus revenue); distribution drift or feedback effects in production.

2. Why not use plain k-fold on user-level data with many rows per user?

Rows from the same user fall in both train and validation, leaking identity; use group k-fold by user.

3. What does calibration mean and when does it matter?

Predicted probabilities match observed frequencies (of cases scored 0.7, about 70% are positive); it matters when probabilities drive decisions, such as pricing or risk.

16.4 Question bank: SQL and Python rapid-fire

IN PLAIN WORDS

These are the rapid-fire coding questions: ten that interviewers use to test fluency, not cleverness. For SQL: second-highest without window functions, consecutive-month activity, UNION versus JOIN, percentage change. For Python and pandas: swapping variables, counting words, list versus generator, NaN handling, ranking within groups, and the copy-versus-view trap. Practise writing each from a blank page in under three minutes, then compare with the answer.

Why it matters: Fluency on small problems frees your attention for the hard ones.

Takeaway: Write it blind, run it mentally on a tiny input, then check.

SAY IT IN 60 SECONDS

"Rapid-fire coding questions test fluency. For the second highest salary without windows, I take the maximum salary below the overall maximum, or order descending and offset one with a distinct. Users active in consecutive months: truncate to month, then join the table to itself on the next month, or use LAG. UNION stacks result rows vertically, JOIN combines columns horizontally by a key. In Python, a list comprehension builds the whole list in memory and a generator expression computes lazily, one item at a time. In pandas, dropna with how equal to any or all, groupby rank, and the chained-assignment trap where I should use loc. The catch: I test each on a tiny input by hand."

KEY TERMS second highest salary · consecutive months · UNION vs JOIN · percent change by category · swap variables · Counter · generator vs list comprehension · dropna · rank within groups · chained assignment

HANDS-ON PRACTICE Write each solution in a scratch file before revealing it, then run it on the sample data.

COMMON MISTAKES Leaving out NULL and ties handling · Forgetting sort before a window function

CHECK YOURSELF

1. Second highest salary without window functions?

`SELECT MAX(salary) FROM emp WHERE salary < (SELECT MAX(salary) FROM emp);` returns NULL if there is no second value.

2. `[i for i in range(3)]` versus `(i for i in range(3))`?

The first is the list `[0, 1, 2]`; the second is a generator object that yields values lazily and can be consumed once.

3. `df[df.a > 1]['b'] = 0` does not change df. Why, and the fix?

It assigns to a temporary copy from the chained indexing; use `df.loc[df.a > 1, 'b'] = 0`.

16.5 Question bank: case and product

IN PLAIN WORDS

Case questions have no single right answer, so the interviewer is watching your structure. A good case answer follows a repeatable path: clarify the goal, pick the metrics, break the problem into parts, say what data you need, state the risks, and finish with a recommendation. The seven cases here (metric drop, feature success, A/B test design, churn, fraud, recommendations, data bug) reuse the same few frameworks, so learn the frameworks once and apply them seven times.

Why it matters: The case round is where hiring managers decide whether you think like a data scientist.

Takeaway: Clarify, metric, structure, data, risks, recommendation.

SAY IT IN 60 SECONDS

"For any case I use one skeleton. Clarify the goal and the decision. Define the metric and guardrails. Break the problem into parts. Say what data I need. Name the risks and biases. Finish with a recommendation and a next step. For a ten per cent drop in orders: verify the data, scope the timing, decompose orders into visitors times conversion times basket, segment by platform and region, check releases and external factors. For a churn model: define churn and the prediction window, build features as of the prediction date, split by time, choose the metric from costs, and plan deployment and monitoring. The catch: finish with a clear recommendation, not a list of ideas."

KEY TERMS metric drop · feature success · A/B test design · churn model · fraud detection · recommendations · data bug

HANDS-ON PRACTICE Answer each in three minutes aloud using the frameworks, then reveal and compare.

COMMON MISTAKES Unstructured answers · Skipping the decision and the action

CHECK YOURSELF**1. You discover a data bug affecting last month's reports. What do you do?**

Quantify the impact and scope, tell stakeholders immediately and plainly, fix the pipeline and backfill corrected numbers, add checks and tests to prevent recurrence, and document what happened.

2. Walk through the skeleton for "detect fraud at a payments company".

Goal and cost of errors; labels (chargebacks arrive late); features such as velocity, device, amount versus history; time-based split; metric precision/recall at a review budget; real-time scoring with rules plus model; monitoring and feedback loop.

3. Why should a case answer end with a recommendation?

It shows you can turn analysis into a decision; a list of ideas leaves the interviewer to do that work.

16.6 Self-test: answer aloud in under two minutes**IN PLAIN WORDS**

This is your final check: five questions you must be able to answer aloud in under two minutes, without notes. They test explanation (gradient boosting to a beginner), storytelling (your best project), judgement (logistic regression versus XGBoost), product sense (is the feature worth shipping) and clear thinking (correlation versus causation). Record yourself, play it back, and listen for jargon, missing examples and rambling. If you can do all five calmly, you are close to ready.

Why it matters: Being able to explain simply is the strongest sign that you understand.

Takeaway: Record, listen back, cut the jargon, repeat.

SAY IT IN 60 SECONDS

"The self-test is about explaining under time pressure. I aim for under two minutes, structured: a one-sentence answer, the reason, one example, one caveat. Gradient boosting to a beginner: many small decision trees, each one trained to fix the mistakes of the ones before, like a team where each new member focuses on what the others got wrong. Logistic regression over XGBoost when I need interpretability, a stable baseline, small data or a regulatory explanation. Correlation versus causation: ice cream sales and drownings rise together because of summer; only randomisation or careful design shows cause. The catch: I record myself, because rambling is invisible from the inside."

KEY TERMS explain gradient boosting · walk through your best project · logistic regression vs XGBoost · check whether a feature is worth shipping · correlation vs causation

HANDS-ON PRACTICE Record yourself answering each question in under two minutes, listen back, and trim filler words.

COMMON MISTAKES Going over two minutes · Using jargon with a non-technical audience

CHECK YOURSELF**1. Give a 20-second beginner explanation of gradient boosting.**

Start with a rough guess, measure the mistakes, train a small tree to predict those mistakes, add it with a small weight, and repeat many times until the errors are small.

2. Three reasons to choose logistic regression over XGBoost?

Interpretability (coefficients and odds ratios), a strong, fast baseline with little tuning, and regulatory or small-data settings; its probabilities are also usually well calibrated.

3. Give a domain example of correlation without causation and the way to test causation.

Ads shown more to high-intent users correlate with sales; randomise exposure in an A/B test (or use an instrument or holdout) to measure the true effect.

Interview tips, tricks and hacks

- 1. Cover, say, compare.** Never read a question and its answer together. The effort of recall is the practice.
- 2. Use a timer.** Thirty seconds for definitions, sixty for comparisons, two minutes for cases. Speed without rambling is the goal.
- 3. Mark honestly in three grades.** Solid, shaky, missed. Shaky counts as missed for revision purposes.
- 4. Revisit marked questions on a schedule.** After two days, then a week. Spaced repetition beats one long session.
- 5. Put every answer in your own words.** A memorised sentence collapses at the first follow-up question; an understood idea does not.
- 6. Add the example.** If your answer has no example, it is half an answer. Keep one small numeric example for each core concept.
- 7. Stop when you have answered.** Say the four beats and pause. Let the interviewer choose the follow-up.

- 8. Practise the "I am not sure" answer.** "I have not used that directly. My understanding is... and I would verify it by..." Calm honesty with reasoning scores well.
- 9. Record the final five.** Listen for jargon, filler and missing structure, then record again.

Frequently asked questions

59 questions interviewers ask on this chapter, with model answers. Cover the answer, say yours aloud, then compare.

QUESTION BANK: STATISTICS AND PROBABILITY

Q1 What is a p-value?

Probability of data at least as extreme as observed, assuming H_0 is true. Not $P(H_0 \text{ true})$.

Say it with numbers. In an A/B test of 10.0% versus 10.5% conversion, $p = 0.03$ says: if the true rates were equal, a gap this large would occur in about 3% of repeated experiments. Combine it with the confidence interval for the size of the lift.

Q2 Type I vs Type II error?

False positive (alpha) vs false negative (beta). Power = $1 - \beta$.

Memory aid. Type I is the boy who cried wolf (false alarm); Type II is the wolf arriving unnoticed (a miss). Lowering α increases β unless you increase the sample size.

Q3 Explain CLT.

Sample means approach normal with $SE = \sigma/\sqrt{n}$, regardless of population shape, for large n .

Why it matters in practice. CLT is why we can build confidence intervals and t-tests for means of skewed metrics like order value once the sample is large. It does not make individual values normal.

Q4 When to use t-test vs z-test?

t when sigma unknown/small n ; z when sigma known or n large (they converge).

In practice. With n above about 30 and unknown σ , t and z give nearly identical results; the t-test is the safe default because the difference vanishes as n grows.

Q5 Confidence interval meaning?

Procedure that captures the true parameter in 95% of repeated samples.

Careful wording. "If we repeated the sampling and interval procedure many times, about 95% of the intervals would contain the true value." Informally, people say "we are 95% confident"; avoid saying "there is a 95% probability the true value is in this particular interval" in a strict frequentist setting.

Q6 How to handle outliers?

Investigate; cap, transform, robust stats, or separate modeling.

A decision path. Is it a typo or sensor fault? Fix or drop it. Is it real but extreme (a whale customer)? Keep it, use medians or a log transform, or report with and without. Never delete points only because they hurt the model.

Q7 Covariance vs correlation?

Correlation = covariance scaled to $[-1,1]$.

Example. Height in cm and weight in kg have a covariance of 60 (hard to interpret); changing to metres shrinks it by 100. The correlation of 0.7 is unit-free.

Q8 How to test if two distributions differ?

KS test, Mann-Whitney, chi-square on binned data, permutation tests.

Choose by data type. Two continuous samples: two-sample KS test (any difference in shape) or a t-test for the mean only. Categorical counts: chi-square. Always add a histogram or ECDF plot.

Q9 Why might a test be significant but useless?

Tiny effect with large n ; check effect size/CI and business impact.

Example. With 10 million users, a 0.01% lift is statistically significant yet worth less than the engineering cost; compare the interval to the minimum effect worth shipping.

Q10 Expected number of coin flips for two consecutive heads?

6.

Derivation. Let E be the expected flips from scratch and H the expected flips remaining after a first head. $E = 1 + \frac{1}{2}H + \frac{1}{2}E$ and $H = 1 + \frac{1}{2}E$ (with probability $\frac{1}{2}$ the next flip is heads and you finish). Solving gives $E = 6$. (For HT it is 4.)

Q11 Bayes on medical test?

Use prior, sensitivity, false-positive rate; post-test probability may be low (see 5.2).

Numbers. With 1% prevalence, 99% sensitivity, 5% false-positive rate: $P(D \mid +) = 0.0099 / 0.0594 = 16.7\%$. With 10% prevalence it would be 68.8%. See Bayes' theorem.

Q12 Multicollinearity effect?

Unstable coefficients/inflated SEs, not reduced predictive power; use VIF/regularization.

Detect and fix. Variance inflation factor above about 5 to 10 is a warning. Drop or combine correlated features, use PCA, or ridge regression. Predictions are fine; interpretation of individual coefficients is not.

QUESTION BANK: MACHINE LEARNING I**Q13 Bias-variance tradeoff?**

Simpler models high bias/low variance; complex reverse; balance for minimum generalization error.

Picture it. Fit a straight line to curved data: wrong in a consistent way (bias). Fit a 15-degree polynomial to ten points: perfect on those points, wild on new ones (variance). The best model sits in between; validation error shows where.

Q14 Bagging vs boosting?

Bagging trains many independent models in parallel on bootstrap samples and averages them, mainly reducing **variance** (random forest: deep trees plus random feature subsets). **Boosting** trains models sequentially, each one fitting the errors (residuals or gradients) of the ensemble so far, mainly reducing **bias** (shallow trees, learning rate). Bagging is robust and hard to overfit; boosting is usually more accurate when tuned but needs early stopping and is more sensitive to noise. Stacking is a third option: a meta-model learns to combine different base models. See gradient boosting for details.

Why each works. Averaging models that make different errors cancels noise, so bagging helps unstable, deep trees. Boosting uses weak learners (shallow trees) and adds capacity step by step, so it lowers bias but can overfit noise; use early stopping.

Q15 Why L1 sparse, L2 not?

Diamond vs circle constraint; constant vs proportional gradient.

Geometry. Minimising loss within $|w_1| + |w_2| \leq t$: the diamond has corners on the axes, and the loss contours usually touch a corner, where one weight is zero. With $w_1^2 + w_2^2 \leq t$, the circle has no corners.

Q16 How does XGBoost differ from GBM?

Regularization, second-order gradient, sparsity-aware splits, parallelism, handling missing values.

Practical differences. XGBoost is regularised (L1/L2 on leaf weights), handles missing values natively, and is much faster; LightGBM and CatBoost are further variants for speed and categorical data.

Q17 What is data leakage?

Data leakage is information from outside the training context leaking into the model, so offline scores are inflated and production performance disappoints. Examples: scaling or imputing on the whole dataset before splitting; target encoding computed on the same rows; features containing future information (refunds after purchase); random splits for time series or repeated customers; duplicates across train and test; a proxy for the label. Signal: suspiciously high AUC or one dominant feature. Prevention: split first, use a `Pipeline`, time- or group-aware splits, and audit each feature for availability at prediction time. See leakage.

Checklist. Does every feature exist at prediction time? Was each transformer fit on training data only? Are duplicates or the same customer split across train and test? Is the split by time when forecasting?

Q18 Accuracy on 99:1 data?

A trivial "always negative" model scores 99%; use PR-AUC/recall/precision/F1, class weights, threshold tuning.

Numbers. With 1% positives, "always negative" gets 99% accuracy and 0% recall. Use PR-AUC, recall and precision at a business-driven threshold, `class_weight`, and stratified splits.

Q19 Precision vs recall tradeoff?

Threshold slides along the PR curve; choose by cost.

Numbers. Threshold 0.5 gives precision 80%, recall 50%. Lowering to 0.3 may give precision 60%, recall 75%. Choose where expected cost ($\text{cost_FN} \times \text{FN} + \text{cost_FP} \times \text{FP}$) is lowest.

Q20 ROC-AUC vs PR-AUC?

ROC can be misleading under heavy imbalance since TN dominates FPR; PR focuses on positive class.

Why ROC misleads. $\text{FPR} = \text{FP} / (\text{FP} + \text{TN})$; with a huge number of negatives the TN term is enormous, so even many false positives barely move FPR and the ROC curve looks excellent. $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$ uses FP directly, so PR curves expose the problem. On a problem with 2% positives the same scores gave ROC-AUC 0.86 but average precision 0.20.

Q21 How to choose K in K-Means?

Elbow, silhouette, business constraints.

Procedure. Run K-means for $k = 2 \dots 10$, plot inertia (elbow) and silhouette. Prefer the k where silhouette is high and the segments make business sense and are stable across seeds.

Q22 How does a decision tree choose splits?

Maximizes impurity reduction (Gini/entropy/variance).

Example. A node with 50 spam and 50 ham has Gini 0.5. A split sending 45 spam and 5 ham left, 5 spam and 45 ham right gives each child Gini 0.18, so the gain is 0.32; the tree picks the split with the largest gain.

Q23 Why do tree models not need scaling?

Splits depend on order, not magnitude.

Why. A split like "income > 40,000" depends only on the ordering of income values. Any monotonic rescaling (standardising, log) leaves the ordering, and therefore every possible split, unchanged.

Q24 How to reduce overfitting in neural nets?

Dropout, weight decay, early stopping, augmentation, smaller model.

Priority order. More or augmented data first, then dropout and weight decay, early stopping on validation loss, then reduce capacity.

QUESTION BANK: MACHINE LEARNING II**Q25 What is cross-validation and when not to use vanilla k-fold?**

Rotating validation folds; not for time series or grouped data.

Example. For a churn model with multiple rows per customer, plain k-fold puts the same customer in train and validation folds, leaking customer-specific patterns: use `GroupKFold`. For monthly forecasting, use `TimeSeriesSplit` so you always validate on the future.

Q26 What is regularization and how to choose lambda?

Penalty on complexity; choose via CV.

Choosing λ . Search on a log scale (for example $1e-4$ to $1e2$) using cross-validation. Pick the value with the best validation score, or the largest λ within one standard error of the best for a simpler model.

Q27 Explain gradient descent and learning rate.

Iteratively move against gradient; LR too high diverges, too low is slow.

Intuition. Imagine walking downhill in fog: the gradient is the slope under your feet, and the learning rate is your stride. Too long a stride and you overshoot the valley; too short and you take forever.

Q28 How do you handle categorical variables with 1000+ levels?

Target encoding (out-of-fold), frequency encoding, hashing, embeddings, CatBoost.

Code sketch. Out-of-fold smoothed target encoding: for each fold compute category means on the other folds, blend with the global mean using a pseudo-count, and apply to the held-out fold. CatBoost implements an ordered version natively.

Q29 What is calibration?

Predicted probabilities match observed frequencies; check reliability curve; fix with Platt/isotonic.

Check. Bin predicted probabilities (for example deciles) and compare with the observed positive rate per bin: a calibrated model lies on the diagonal. Platt scaling fits a logistic regression on the scores; isotonic regression fits a monotone step function (needs more data).

Q30 How to explain model predictions?

SHAP (global and local), permutation importance, PDP, LIME.

Pick by audience. Executives: top three drivers in plain words. Analysts: SHAP summary plot. Customers (for a decision such as a loan): reason codes for that single prediction.

Q31 Model performs well offline, poorly online?

A model that scores well offline but poorly online usually points to one of: **leakage** in offline evaluation; **train-serve skew** (features computed differently in production: units, joins, time windows, missing handling); a **distribution mismatch** between the offline test set and live traffic (time, geography, user mix); **drift** since training; **feature unavailability or latency** at inference forcing simpler inputs; **feedback loops**; or an overfit validation set after heavy tuning. Debug by logging production features and comparing them with offline features for the same entities. See MLOps Q&A.

Debug order. First compare feature values between offline and online for the same entity (skew). Then check label definitions and the time window. Then compare population distributions, then the metric alignment.

Q32 Difference between PCA and feature selection?

PCA creates new combined features; selection keeps a subset of originals.

Example. PCA turns 100 correlated features into 10 components that are blends of all 100; feature selection keeps, say, 12 of the original columns, so you can still say "income" matters.

Q33 How is Random Forest feature importance biased?

Impurity importance favors high-cardinality/continuous features; use permutation importance.

Fix. Use permutation importance on a validation set (shuffle one feature and measure the score drop) or SHAP values, which are less biased towards high-cardinality features.

Q34 Explain logistic regression to a business person.

It estimates the probability of an event from a weighted combination of factors; each factor raises or lowers the odds.

Plain-language version. "It gives each customer a score for how likely they are to do something, say cancel. Each factor, like late deliveries or months as a customer, pushes the score up or down by a fixed amount, and we add them up. So we can tell you exactly why a customer has a high score."

Q35 What if training data is small?

Simpler/regularized models, CV, augmentation, transfer learning, domain priors, collect more.

Playbook. Strong regularisation, simpler model, repeated cross-validation, pretrained features or embeddings, data augmentation, and a plan to collect more labels where the model is most uncertain.

Q36 Handling missing values in production?

Same imputation as training, missing flags, monitoring for new missingness patterns.

Production recipe. Save imputation values with the model, add a was-missing flag, log the missing rate per feature, alert on jumps, and choose a safe default behaviour when a critical feature is missing.

Q37 How would you detect concept drift?

Monitor performance, PSI on inputs/outputs, statistical tests, retrain triggers.

Practical monitoring. Track rolling performance once labels arrive, PSI on inputs and on the score, the positive rate of predictions, and calibration drift. Concept drift often shows first as calibration or rank-performance decay on recent labelled data, while input drift can be present without hurting performance.

QUESTION BANK: SQL AND PYTHON RAPID-FIRE**Q38 Second highest salary without window functions?**

```
SELECT MAX(salary) FROM e WHERE salary < (SELECT MAX(salary) FROM e);
```

Edge cases. With ties at the top (two people at the maximum), this returns the next *distinct* salary, which is usually what "second highest" means. If there is only one distinct salary it returns NULL. With a window function: `SELECT DISTINCT salary FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) r FROM e) t WHERE r = 2.`

Q39 Find users active in consecutive months?

Use LAG on month or gaps-and-islands.

SQL. Users with consecutive months of activity:

```
WITH m AS (SELECT DISTINCT user_id,
DATE_TRUNC('month', order_date) AS mth FROM
orders)
SELECT DISTINCT a.user_id
FROM m a JOIN m b ON a.user_id = b.user_id AND
b.mth = a.mth + INTERVAL '1 month';
```

For longer streaks, number the months and use the gaps-and-islands subtraction trick.

Q40 Difference between UNION and JOIN ?

UNION stacks rows; JOIN combines columns based on keys.

Example. `SELECT a FROM t1 UNION SELECT a FROM t2` stacks rows (same columns); `SELECT * FROM t1 JOIN t2 ON t1.id = t2.id` adds columns from matching rows. UNION changes the number of rows, JOIN changes the number of columns.

Q41 How to find the percent change between two periods per category?

Window LAG per category or self-join.

SQL.

```
WITH m AS (SELECT category, DATE_TRUNC('month',
order_date) AS mth, SUM(amount) AS rev FROM
sales GROUP BY 1, 2)
SELECT category, mth, rev,
100.0 * (rev - LAG(rev) OVER (PARTITION
BY category ORDER BY mth)) / NULLIF(LAG(rev)
OVER (PARTITION BY category ORDER BY mth), 0) AS
pct_change
FROM m;
```

The `PARTITION BY category` makes each category compare only with its own previous month.

Q42 Python: swap two variables without a temp?

```
a, b = b, a.
```

Why it works. The right side builds a tuple (b, a) first, then it is unpacked into a and b, so no temporary variable is needed.

Q43 Python: count word frequency in text?

```
Counter(text.lower().split()).
```

Code.

```
from collections import Counter
Counter("the cat and the
hat".split()).most_common(2) # [('the', 2),
('cat', 1)]
```

Normalise first (`lower()` , strip punctuation) if the question needs it.

Q44 Python: what's the output of `[i for i in range(3)]` vs `(i for i in range(3))` ?

List `[0, 1, 2]` vs generator object.

Memory. `[i for i in range(3)]` builds the list `[0, 1, 2]` immediately. `(i for i in range(3))` prints as `<generator object <genexpr> at 0x...>` and produces values one at a time; wrap it in `list()` to see them, and note it can be consumed only once.

Q45 pandas: drop rows where any/all values NaN?

```
dropna(how='any'/'all').
```

Options. `dropna(subset=['col'])` considers only some columns; `thresh=k` keeps rows with at least k non-null values; `axis=1` drops columns.

Q46 pandas: rank within groups?

```
groupby(...)[ 'col' ].rank(method='dense',
ascending=False).
```

Example. `df["rank"] = df.groupby("dept")["salary"].rank(method="dense", ascending=False)` (method `"min"` is RANK, `"first"` is ROW_NUMBER).

Q47 pandas: why `df[df.a > 1]['b'] = 0` fails?

Chained assignment on a copy; use `df.loc[df.a > 1, 'b'] = 0`.

Fix. `df.loc[df.a > 1, "b"] = 0`: a single `.loc` call with the row mask and column performs the assignment on the original frame. The chained form assigns into a temporary object.

QUESTION BANK: CASE AND PRODUCT**Q48 Orders dropped 10%. Investigate.**

Structure (see diagnosing a metric drop): (1) Verify the drop is real (pipeline, logging, definitions, dashboard lag). (2) Decompose: orders = users × sessions per user × conversion × orders per converter; find which factor moved. (3) Segment by platform, app version, city, new vs returning, channel, category. (4) Internal causes: releases, pricing, promotions ending, supply issues, experiments. (5) External causes: festivals, weather, competitors, events, outages. (6) Funnel: where did drop-off rise? (7) Prioritise hypotheses by evidence and impact, recommend action and monitoring. Example: traffic steady, conversion down only on Android 4.2, payment success fell from 94% to 81%: a payment-SDK bug, so hotfix or roll back.

Example decomposition. Orders = sessions × conversion × orders per converter. If sessions are flat and conversion fell only on Android after release 4.2, the investigation is narrow: check the app version funnel and crash logs.

Q49 Measure success of a new feature.

Structure (see measuring feature success): goal and target users; one primary metric linked to the north star, secondary diagnostics, guardrails; leading vs lagging indicators; an A/B test (or a quasi-experiment if randomisation is impossible); segments and cannibalisation; decision criteria set in advance (ship, iterate, kill). Always address what the feature might steal from elsewhere.

Example. For a reorder button: primary metric is repeat orders per user in 14 days; adoption is taps per eligible user; guardrails are order value and support contacts; decide with an A/B test.

Q50 Design an A/B test for checkout redesign.

Structure (see the experiment lifecycle): hypothesis ("a shorter form will raise checkout conversion"); primary metric checkout conversion, secondary time to complete and payment errors, guardrails latency, refunds and support contacts; randomise by user; sample size from baseline and MDE (for 10% to 11%, about 14,700 per group); run at least one or two weekly cycles; check SRM; analyse per user with a confidence interval; decision rule written in advance. Mention peeking, novelty, and segment analysis as hypothesis generation.

Design. Baseline conversion 3%, detect +0.3pp: $n \approx 16 \times 0.029 / 0.003^2 \approx 51,000$ per group. At 10,000 eligible users a day split in two, that is about 10 days; run two full weeks.

Q51 Build a churn model: walk through target definition, features, split, metrics, deployment, action.

Structure (see ML problem framing): decision (who gets a retention offer) and cost of errors; target (no order in the next 30 days from a snapshot date); features as of the snapshot (recency, frequency and trend, service quality, support, engagement); time-based split and baseline then gradient boosting; metrics PR-AUC, lift at the contact budget and expected profit; threshold from costs; deployment as weekly batch scoring with monitoring (PSI) and a holdout to measure real retention impact (consider uplift modelling).

Details. Churn = no order in the next 30 days among customers active in the last 90. Features use only data before the snapshot date. Evaluate precision at the top decile of risk because the retention budget is limited.

Q52 How would you detect fraud at a payments company?

Rules + supervised model + anomaly detection, velocity/device/graph features, recall at fixed precision, feedback loop from disputes, latency constraints.

Fuller outline. *Data and labels:* transaction, device, geography and customer history; labels arrive late (chargebacks) and are biased toward cases that were flagged. *Features:* velocity counts per card/device/IP, amount relative to the customer's norm, new device or location, merchant risk, graph features (shared devices). *Models:* rules for known patterns, a supervised model (gradient boosting) for recall, an anomaly model for novelty. *Metric:* recall at a fixed false-positive rate, PR-AUC, precision@review-capacity. *Serving:* real-time under tens of milliseconds with online features. *Loop:* analyst feedback and disputes become labels; monitor drift and adversarial adaptation.

Q53 How would you build a recommendation for a food delivery app?

Retrieval (popularity, CF, embeddings) + ranking (GBM/NN with context like time, location, weather) + diversity/business rules; metrics: CTR, conversion, GMV, repeat rate; A/B test.

Fuller outline. Retrieval (popularity, collaborative filtering, cuisine and location filters, open restaurants only), then a ranking model on user, restaurant and context features predicting order probability, then re-ranking for diversity and new-restaurant exploration. Cold start through popularity and content; offline NDCG with a time split; online A/B test on conversion, GMV and repeat rate, with delivery-time guardrails.

Q54 You discover a data bug affecting last month's reports. What do you do?

Quantify impact, inform stakeholders quickly, fix and backfill, add tests/monitoring, document postmortem.

Fuller outline. Quantify the impact first (which reports, which decisions, how large); notify stakeholders promptly and plainly; fix the root cause and **backfill** corrected data; version and annotate the corrected reports; add tests and monitoring (row-count and freshness checks, reconciliation with source systems) so it cannot recur silently; write a short postmortem (what happened, impact, detection, fix, prevention). Transparency and speed matter more than perfection.

SELF-TEST: ANSWER ALOUD IN UNDER TWO MINUTES**Q55 Explain how gradient boosting works to a beginner.**

"Imagine guessing house prices. Start with the average price for everyone. That guess is wrong by different amounts for different houses. Now train a tiny decision tree whose only job is to predict *those errors*: maybe 'big houses in this area are underestimated by ₹20 lakh'. Add a fraction of its correction to the guess. Some errors remain, so train another tiny tree on the remaining errors, and add a fraction again. After hundreds of small corrections, the guess is very accurate. The 'fraction' is the learning rate, and small steps stop us from overfitting. The 'gradient' in the name means we correct the errors in the direction that most reduces the loss. Caution: because each tree depends on the previous one, we need early stopping on a validation set, or it will start fitting noise."

A beginner script. "Imagine predicting house prices. First guess the average. Look at where you were wrong. Train a small decision tree to predict those errors. Add a fraction of it to your guess. Repeat a hundred times. Each tree fixes a little of what remains."

Q56 Walk me through your best project as if I am a hiring manager.

Use this outline (90 to 120 seconds): **Problem and decision** (who uses it, what changes) → **Data** (source, size, grain, key issues) → **Approach** (split strategy, baseline, features, models tried and why) → **Results** (metric vs baseline, what the number means in business terms) → **Insights and failure modes** (error analysis, explainability) → **Production thinking** (how you would deploy and monitor) → **What I would do next**. Start with the headline: "I built X that does Y, improving Z from A to B," then go through the layers, inviting the interviewer to dig into any part.

Structure. 20 seconds on the problem and why it mattered, 20 on the data, 40 on the approach and a key decision, 20 on the result with a number, 20 on what you would improve.

Q57 Why would you choose logistic regression over XGBoost?

"I would choose logistic regression when **interpretability, stability and regulation** matter, when data is small or mostly linear, when I need well-calibrated probabilities, or when I want a robust baseline and easy monitoring: each coefficient gives an odds ratio I can explain to a risk or compliance team. XGBoost usually wins on raw accuracy with non-linearities and interactions and many features, but it is harder to explain and to monitor, and needs tuning and early stopping. In practice I start with logistic regression as the baseline; if XGBoost's lift is meaningful for the business (measured on a time-based validation set, at the operating threshold) and I can explain it with SHAP, I use it, otherwise I ship the simpler model."

Add the caveat. If the data has many non-linear interactions and accuracy matters most, XGBoost may be better; show you would test both, then explain the trade-off with SHAP.

Q58 How would you check whether a new feature is worth shipping?

"I would run an A/B test. Define the hypothesis, one primary metric linked to the goal, secondary metrics and guardrails (latency, errors, refunds). Randomise by user, compute the sample size from the baseline and minimum detectable effect, run for whole weekly cycles, check for sample ratio mismatch, and analyse per user with a confidence interval. Decide using effect size and cost, not just significance, and look at segments as hypotheses. If an experiment is not possible, use a quasi-experiment such as difference-in-differences and be upfront about the weaker evidence."

Sketch. Primary metric and guardrails, test unit, sample size, duration, pre-set decision rule, and who will act on the result.

Q59 What is the difference between correlation and causation, with an example from your domain?

"Correlation means two things move together; causation means changing one *causes* the other to change. Example from delivery: orders with more items have longer delivery times, but adding items does not make the rider slower: the confounder is **order size and restaurant prep time**. Ice-cream sales and drownings correlate because of summer. To establish causation we randomise (an A/B test) so groups differ only by the treatment, or use careful causal methods such as difference-in-differences when we cannot. Correlation is still useful for prediction, but dangerous for decisions about what to change."

Own-domain example. In e-commerce, users who see more recommendations buy more, but heavy buyers also browse more, so the correlation overstates the effect. A randomised holdout reveals the true lift.

17

CHAPTER 17 · MODULE 17 IN THE APP

12-Week Study Plan, Checklists, Resources

Turn everything in this book into a schedule you can keep, a checklist you can test yourself against, and a final week that leaves you ready.

4 topics · about 47 min in the app · 4 questions answered

17.1 The 12-week plan

17.2 Master readiness checklist

17.3 Company-wise emphasis cheat sheet

17.4 Resources and final advice

Foundations

Knowing what to study is half of preparation. The other half is doing it, steadily, in a sensible order, for long enough. This chapter is about that second half: the plan, the honest self-check, and the last few days.

The order matters

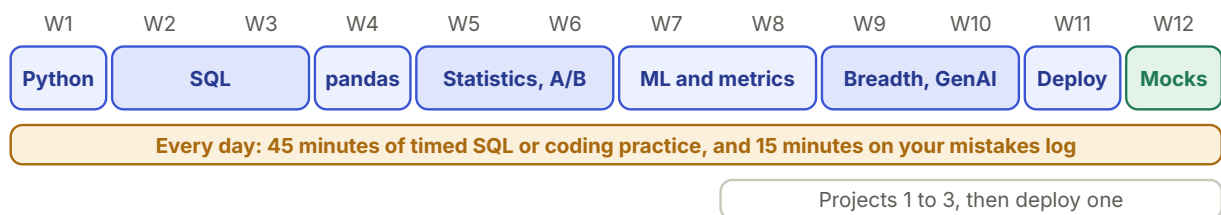


Figure 19. The twelve-week plan: coding and SQL first and every day, theory in the middle, projects and practice interviews at the end.

The plan is ordered deliberately. Python and SQL come first because they are the first filter and because everything later uses them. Statistics comes before machine learning because model evaluation is statistics. Features and metrics follow the algorithms they serve. Breadth topics come later because they are tested less deeply. Projects begin once you can build something end to end, and practice interviews come last, when there is something to practise. One thing runs through all twelve weeks: a short daily session of timed SQL or coding, because fluency fades within days if it is not exercised.

A day that works

A sustainable study day of two to three hours has four parts: about forty-five minutes of timed practice problems, an hour on one concept that you finish by explaining it aloud, forty-five minutes on your current project, and fifteen minutes reviewing your mistakes log. The review is the part people skip and the part that makes the rest stick.

When you have less time

If you have eight weeks or four, keep the order and reduce the breadth. The core is never cut: SQL, statistics with experimentation, machine learning fundamentals with metrics, and one project you can discuss in depth. What gets compressed is the depth of time series, deep learning details and tooling, where the cheat sheets can stand in for full study. Appendix A sets out all three plans.

Readiness is something you measure

Feeling ready and being ready are different. The readiness checklist in this chapter is useful only if each item is verified by an actual attempt. "I can solve a medium SQL problem" means you did so, against a timer, this week. "I can explain a confidence interval simply" means you recorded yourself doing it and the recording made sense. Tick only what passed. The items left unticked a week before the interview are your plan for that week, and for most people they are the same three: SQL speed, simple explanations of statistics, and project deep dives.

Tailoring to the company

The core is common, and the extras depend on who is interviewing you. A product company adds harder SQL, experimentation and metrics. A consulting firm adds case structure and communication. A bank adds logistic regression and risk vocabulary. A technology centre adds data structures and algorithms. A company building with language models adds retrieval, evaluation and stronger coding. Add these in the order of your interview dates.

The final week

Do not start anything new. Work through your mistakes log, your marked questions and the cheat sheets. Rehearse your introduction and your project stories aloud. Do two full practice interviews. Confirm the logistics. Sleep properly the night before, since a rested mind retrieves far better than a crammed one.

After each interview

Whatever the outcome, write down every question you were asked, mark the ones you handled badly, and close those gaps within two days. Hiring involves chance as well as merit, and a rejection is information about what to improve, not a verdict on your ability. Candidates who treat each interview as practice for the next almost always get there.

Topic by topic

Each topic gives the idea in plain words, the fundamentals to know, a model spoken answer and three questions to test yourself. Open the same topic in the app for worked examples, runnable code and interactive demos.

17.1 The 12-week plan

IN PLAIN WORDS

A plan is a promise to your future self about what you will do and when. The twelve-week plan keeps a sensible order: coding and SQL first and every day, statistics before machine learning, projects once the basics are in place, and mock interviews at the end. If time is short, compress by cutting the exotic parts, never the core. Use the Planner in the app so that sessions land on real dates, and when life interferes, reschedule rather than abandon.

Why it matters: Steady, ordered practice beats last-minute cramming, and a visible plan reduces anxiety.

Takeaway: Keep the order, protect the daily rhythm, adjust the pace not the priorities.

WORKED EXAMPLE, STEP BY STEP

Turn "12 hours a week" into a calendar

- 1. Fix the hours you really have:** weekdays 7 to 8:30 pm ($1.5 \text{ hours} \times 5 = 7.5$), Saturday 9 to 11:30 am (2.5), Sunday 4 to 6 pm (2). Total: 12 hours.
- 2. Give each block a job:**

Block	Length	What
Mon to Fri, first 45 min	45 min	timed SQL or coding problems
Mon to Fri, last 45 min	45 min	one topic: read in the app, then "explain it in 60 seconds"
Saturday	2.5 h	project work
Sunday	2 h	review the mistakes log, re-solve missed problems, plan the next week

- 3. Put this week's topics in the evening slots,** following the module order: SQL first, then statistics, and so on.
- 4. Protect the 45-minute practice slot above everything.** On a bad day, do only that.
- 5. Every Sunday, check two numbers:** topics done against the plan, and timed-problem speed. If you are behind, move sessions forward rather than cramming.

What this shows. A plan works when every block has a time, a length and a job. "Study more this week" is a wish, not a plan.

YOUR TURN (COVER THE SOLUTION FIRST)

1. You have 6 weeks, not 12, at the same 12 hours a week. What do you cut and what do you keep? Give the module order.

Solution. Keep the core at full depth: SQL (Module 2), statistics and A/B testing (5 and 6), ML fundamentals and metrics (7 and 8), and one strong project (14) with the behavioural preparation (15). Cover Python and pandas (1 and 3) quickly through practice problems; read Modules 9 to 12 via their guides and cheat sheets only; keep data structures light unless a target company tests it. A sequence: weeks 1 and 2 SQL plus Python practice; week 3 statistics and A/B testing; week 4 ML and metrics; week 5 project and breadth cheat sheets; week 6 question bank, cases and mock interviews.

THE FUNDAMENTALS

Week	Focus	Deliverables
1	Python fundamentals + NumPy	30 easy coding problems; implement mean/median/std, train-test split
2	SQL basics to joins, aggregation, subqueries	40 SQL problems (LeetCode/DataLemur)
3	SQL window functions, CTEs, cohort/retention patterns	40 more SQL, including all problems in Module 2
4	pandas + visualization + EDA	Full EDA on a messy dataset; SQL-to-pandas translations
5	Statistics and probability	Notes on all tests; solve 30 probability puzzles; Bayes drills
6	A/B testing + hypothesis testing in Python	Analyze a public A/B dataset; write a one-page experiment design
7	ML core: regression, logistic, trees, ensembles	Implement logistic regression and k-means from scratch; sklearn pipelines
8	Feature engineering, metrics, imbalance, tuning	Project 1 (classification) end to end
9	Time series, recommenders, clustering	Project 2 (forecasting or recommender)
10	DL basics, NLP, GenAI/RAG	Project 3 (NLP or RAG)
11	Data engineering/PySpark/cloud/Git/Docker + deployment	Deploy one project with FastAPI + Docker; BI dashboard
12	Case studies, DSA practice, resume, mock interviews	5+ mock interviews; polished GitHub and resume

Daily rhythm: 45 min SQL/coding practice + 60 min concept study + 45 min project work + 15 min revision of mistakes log.

SAY IT IN 60 SECONDS

"My preparation plan has an order and a rhythm. The order: Python and SQL first and daily, then statistics, then machine learning, projects from week eight, and mocks at the end. The rhythm for two to three hours a day: forty-five minutes of timed practice, an hour on one concept that I explain aloud, forty-five minutes on a project, and fifteen minutes of review of my mistakes log. If I have fewer weeks I keep SQL, statistics and ML core and cut breadth like deep learning details. I use the Planner to put sessions on dates and reschedule missed ones. The catch: skipping the review block is the commonest mistake; it is where retention happens."

KEY TERMS 12-week plan · daily rhythm · compressing the plan · milestones

HANDS-ON PRACTICE Open the Planner, accept a preset, and adjust weekly hours and days off to match your real calendar.

COMMON MISTAKES Planning without milestones · Dropping SQL and statistics in favour of exciting topics

CHECK YOURSELF

1. You have four weeks. What stays and what goes?

Stays: daily SQL and Python practice, statistics and A/B testing, ML core and metrics, the question bank, one project story, two mocks. Goes: depth in time series, deep learning details and exotic topics.

2. What is the daily rhythm and why include review?

Practice 45 min, concepts 60, project 45, review 15. Review of mistakes and spaced recall is what turns study into retention.

3. You miss three days. What do you do?

Use reschedule to move missed sessions forward, drop the least important ones if needed, and keep the daily core practice going without guilt.

17.2 Master readiness checklist

IN PLAIN WORDS

Readiness is not a feeling; it is a list of things you have tested. Each line on the checklist should be backed by an actual attempt: a timer for coding and SQL, a recording for explanations, a blank page for designs. Tick only what passed. The unticked lines are your plan for the final week, and usually they are the same three: SQL speed, simple explanations, and project deep-dives.

Why it matters: It turns vague anxiety into a short, specific list of gaps.

Takeaway: Test, then tick; spend the final week on the unticked three.

WORKED EXAMPLE, STEP BY STEP

Test one checklist item honestly

Item: *"I can explain a confidence interval simply."*

- 1. Set the pass criteria before testing:** under 60 seconds; correct (it describes the procedure, not "a 95% chance the truth is inside"); one example with numbers; no jargon a non-technical listener would not understand.
- 2. Record yourself** answering "What is a confidence interval?" with your phone.
- 3. Play it back against the criteria:** 75 seconds, correct, example present, used the term "sampling distribution" without explaining it. Result: **fail** (too long, one piece of jargon).
- 4. Fix:** cut the jargon, lead with the example ("We measured a 10% conversion rate from 10,000 users, give or take about 0.6 points"), then the meaning.
- 5. Retest tomorrow:** 50 seconds, all criteria met. Tick it, and schedule one more test a week later.
- 6. Do the same for every item.** A timer for coding and SQL, a recording for explanations, a blank page for designs.

What this shows. A checklist is only as honest as its test. Decide the pass criteria first, test with evidence, and tick only what passed.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Write pass criteria for the item "I can solve a medium SQL problem in 15 minutes".

Solution. For example: a problem not seen before (from a list you have not used), a visible timer, at most 15 minutes from reading to a final query, correct on the expected output including ties and NULLs, the grain stated aloud at the start, and the result checked on two or three rows by hand. Pass two such problems in a row in the same week before ticking it.

THE FUNDAMENTALS

Coding and SQL

- ☐ Solve LeetCode easy in under 15 min, medium in under 30
- ☐ Solve SQL medium in 15 min, hard in 25 min
- ☐ Translate any window-function query between SQL and pandas

Statistics and experimentation

- ☐ Explain p-value, CI, CLT, power, Type I/II errors simply
- ☐ Pick correct hypothesis test for any scenario
- ☐ Design an A/B test incl. sample size, guardrails, pitfalls

ML

- ☐ Explain 10 core algorithms with assumptions and hyperparameters
- ☐ Choose metrics and thresholds for imbalanced business problems
- ☐ Build leakage-free pipelines and time-aware validation
- ☐ Explain model behavior with SHAP/importance

Breadth

- ☐ Basics of DL, NLP, transformers, RAG
- ☐ PySpark/cloud/BI/Git/Docker at conversational level
- ☐ Deployment and monitoring concepts

Projects and presentation

- ☐ 3-4 polished projects on GitHub with READMEs
- ☐ Each project explainable in 2 and 10 minutes
- ☐ Resume bullets quantified and verifiable
- ☐ 6 STAR stories prepared
- ☐ Salary research done; questions for interviewer ready

SAY IT IN 60 SECONDS

"The readiness checklist works only if every tick is earned. For coding and SQL, a timed set. For statistics, a sixty-second recorded explanation of each core concept. For ML, ten algorithms explained with assumptions and hyperparameters, plus a leakage-free pipeline. For breadth, conversation-level knowledge of transformers, RAG, Spark, cloud and Docker. For projects, two-minute and ten-minute versions of each story and every resume number verified. One week before the interview, I take the three least-ticked areas and work them daily, then do two mocks. The catch: the checklist is for finding gaps, not for feeling good."

KEY TERMS readiness checklist · coding · SQL · statistics · ML · breadth · projects · behavioural

HANDS-ON PRACTICE Test each item with a timed attempt or a 60-second explanation; only tick it if it passes.

COMMON MISTAKES Ticking boxes without testing yourself · Skipping the timed items

CHECK YOURSELF**1. How do you test "I can explain a p-value simply"?**

Record a 60-second explanation for a non-technical listener, play it back, and tick only if it is correct, jargon-free and has an example.

2. One week left and ten items are unticked. What do you do?

Pick the three with the highest interview impact, usually SQL timing, statistics explanations and project deep-dives; drill them daily and do two mocks; accept the rest.

3. Why verify every resume number?

Interviewers probe them; unverifiable or wrong numbers damage trust in everything else.

17.3 Company-wise emphasis cheat sheet

IN PLAIN WORDS

Different companies ask for different things, so preparation should not be identical. Consulting firms test structured thinking and SQL. IT services want solid Python and ML basics. Product startups ask hard SQL, A/B testing and metrics. Global capability centres test DSA, SQL and statistics. Banks and lenders look for logistic regression and risk vocabulary. AI-first startups want LLM, RAG and strong coding. Study the shared core first, then add the extras for your actual interview dates.

Why it matters: Spending hours where they count is half of preparation.

Takeaway: Shared core first, then company-specific extras, ordered by interview date.

WORKED EXAMPLE, STEP BY STEP**Merge priorities for three target companies**

Interviews: a product startup on the 10th, an analytics consulting firm on the 18th, a global technology centre on the 28th.

- 1. List each company type's priorities** (topic 17.3): startup: hard SQL, A/B testing, metrics. Consulting: SQL, statistics, cases and communication. Technology centre: data structures and algorithms, SQL, statistics and ML theory.

- 2. Find the shared core:** SQL and statistics appear in all three. Do them first and keep them going daily.
- 3. Order the extras by date:** A/B testing and metrics before the 10th; case frameworks, guesstimates and communication before the 18th; data structures and ML theory before the 28th.
- 4. Turn it into weeks:** week 1, SQL depth plus A/B testing and metrics; week 2, cases and statistics explanations; weeks 3 and 4, data structures and ML theory, with SQL kept warm daily.
- 5. Use the early interviews:** the startup round on the 10th is also practice for the other two. Log every question.

What this shows. Prepare the shared core first, then add each company's extras in the order of interview dates.

YOUR TURN (COVER THE SOLUTION FIRST)

1. Your targets are an IT services company (interview in 5 days) and an AI-first startup (interview in 3 weeks). What is the shared core, and how do you order the extras?

Solution. Shared: Python and SQL fundamentals, plus basic ML. First (next 5 days): Python practice, SQL basics, ML definitions and a clear project walk-through for the IT services interview. Then (the following 2 to 3 weeks): stronger coding, LLM fundamentals, RAG, evaluation and API deployment for the AI-first startup, ideally with a small RAG project you can demonstrate.

THE FUNDAMENTALS

Company type	Prioritize
Analytics consulting (Fractal, Tiger, LatentView, ZS, Tredence)	SQL, stats, case studies, communication, Python/pandas, basic ML, Excel/BI
IT services (TCS, Infosys, Wipro, Cognizant)	Python, SQL, ML basics, projects, cloud basics, communication
Product/startups (Flipkart, Swiggy, Zomato, Meesho, PhonePe, Razorpay)	Hard SQL, A/B testing, metrics/product sense, ML depth, project rigor
GCCs (Amazon, Walmart, Microsoft, Google, Goldman)	DSA + SQL + stats/ML theory + behavioral (leadership principles at Amazon)
BFSI/credit (HDFC, ICICI, Bajaj, Amex, Jio Financial)	Logistic regression/scorecards, WoE/IV, KS/Gini, SQL/SAS, risk concepts, regulations
AI-first/GenAI startups	Python, LLMs/RAG/agents, evaluation, deployment, APIs, strong coding

SAY IT IN 60 SECONDS

"The core is the same everywhere: SQL, Python, statistics and ML basics. On top of it, I add by company type. Consulting: case studies, guesstimates, communication. IT services: Python, ML basics, project stories. Product companies: hard SQL, A/B testing, metrics and product sense. Global capability centres: data structures and algorithms, plus statistics and ML theory. Banks and credit: logistic regression, scorecards, weight of evidence, KS and Gini. AI-first startups: LLMs, RAG, evaluation and deployment. With several targets I study the union in the order of interview dates. The catch: do not over-invest in one company's extras before the shared core is solid."

KEY TERMS company-type emphasis · analytics consulting · IT services · product startups · GCC · BFSI · AI-first

HANDS-ON PRACTICE List your top five target companies, tag each by type and compile the union of their priorities.

COMMON MISTAKES Using a single generic preparation for all targets

CHECK YOURSELF

1. **You apply to a fintech lender and a consulting firm. What is the shared core and what are the extras?**

Shared: SQL, Python, statistics and ML basics. Lender extras: logistic regression, scorecards, KS/Gini. Consulting extras: case structure, guesstimates, communication.

2. **Which extras for a product startup?**

Hard SQL (cohorts, funnels), A/B testing, metric design, and product sense.

3. **How do you order preparation across several interviews?**

Build the shared core first, then add extras in the order of interview dates, soonest first.

17.4 Resources and final advice

IN PLAIN WORDS

You do not need more resources; you need a few good ones used well. Pick one for practice problems, one for theory depth where this course is not enough, and one for mock interviews. Then stop collecting. In the final days, the best use of time is not new material but repetition: your weak topics, your project stories, your mistakes log, and sleep. Walk in knowing you prepared with a plan.

Why it matters: More resources usually means more procrastination; focused repetition wins.

Takeaway: Few resources, deep use, then rest.

WORKED EXAMPLE, STEP BY STEP

Build a small resource stack and a weekly loop

1. **One practice source per skill:** a SQL problem site, a coding problem site, and one public dataset for your project.
2. **One theory source for depth,** used only when the app is not enough on a topic: a single textbook or video series, not five.
3. **One way to practise interviews:** a friend, a peer group or recordings of yourself, once a week.
4. **The weekly loop:** Monday to Friday, timed problems and one topic a day; Saturday, project; Sunday, review the mistakes log, re-solve missed problems and plan the next week.
5. **The stop rule:** no new resource until the current one is finished or has clearly stopped helping. Collecting courses feels productive and usually is not.
6. **After every real interview:** add the questions to your log and fix the gaps within 48 hours.

What this shows. Results come from repetition on a few good sources, not from the number of sources.

YOUR TURN (COVER THE SOLUTION FIRST)

1. You have bookmarked six courses, three books and four YouTube playlists, and have finished none. What do you do this week?

Solution. Pick one item per need: a SQL practice site, the app as the main theory path, and one book only for depth in your weakest area. Archive the rest out of sight. Set a weekly target (for example 10 timed SQL problems and 5 topics explained aloud) and one Sunday review. Measure progress by problems solved and topics explained, not by resources opened.

THE FUNDAMENTALS

17.4 Resources

Courses and videos

- StatQuest with Josh Starmer (stats/ML intuition)
- Andrew Ng: Machine Learning Specialization; Deep Learning Specialization
- CampusX (100 Days of ML/DL), Krish Naik, Codebasics (Indian-context interview series)
- fast.ai Practical Deep Learning; Hugging Face NLP/LLM course
- Kaggle Learn micro-courses (pandas, SQL, ML, feature engineering)
- 3Blue1Brown (linear algebra, neural nets intuition)
- Stanford CS229 / CS224N lectures (for depth)

Books

- *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow* (Aurelien Geron)
- *An Introduction to Statistical Learning* (ISLR/ISLP; free)
- *Practical Statistics for Data Scientists* (Bruce, Bruce, Gedeck)
- *Ace the Data Science Interview* (Kevin Huo, Nick Singh)
- *Trustworthy Online Controlled Experiments* (Kohavi et al.) for A/B testing
- *Cracking the PM Interview / Case in Point* (for case structuring, optional)
- *Designing Machine Learning Systems* (Chip Huyen) for ML production

Practice platforms

- SQL: LeetCode SQL 50, DataLemur, StrataScratch, HackerRank, Mode
- Coding: LeetCode, HackerRank, GeeksforGeeks
- ML/data: Kaggle (datasets, competitions, notebooks), DrivenData, Analytics Vidhya hackathons, Zindi
- Interview experiences: AmbitionBox, Glassdoor, LeetCode Discuss, GeeksforGeeks interview experiences, Reddit (r/datascience, r/developersIndia, r/cscareerquestionsIN)
- Mock interviews: Pramp, Interviewing.io, peers, recording yourself

Communities and upskilling in India

- Analytics Vidhya, Kaggle India, LinkedIn data science groups, local meetups, company hackathons (Flipkart GRiD, Amazon ML Challenge, Walmart Sparkathon, Myntra HackerRamp, etc.) which also lead to interviews

17.5 Final advice

1. **Depth over breadth for fundamentals:** SQL, statistics, and core ML explanations win more offers than exotic deep-learning knowledge.
2. **Be project-fluent:** every number on your resume should have a story.
3. **Practice out loud:** interviews test communication as much as knowledge.
4. **Learn from rejections:** log every question you missed and close the gap within 48 hours.
5. **Apply broadly and early:** use referrals, off-campus drives, LinkedIn outreach, and company career pages; aim for volume with tailored resumes for priority companies.
6. **Stay current but grounded:** know the GenAI landscape at a practical level, but do not let it displace fundamentals.

Good luck. Revisit this document weekly, tick off the checklists, and iterate based on mock interview feedback.

SAY IT IN 60 SECONDS

"My final advice is to choose few resources and use them deeply. One source of practice problems, one for theory when I need more depth, one way to do mocks with a real person. In the last week I do not start new topics; I repeat my weak topics, rehearse my project stories and tell the pitch out loud. I check the logistics, sleep well the night before and eat. In the interview I stay curious: I ask clarifying questions, think aloud, and treat it as a conversation. After each interview I write down what I was asked, so my next one is better. The catch: the process is a numbers game in part, so each interview is practice, not a verdict."

KEY TERMS StatQuest · Andrew Ng · CampusX · fast.ai · Kaggle Learn · ISLP · Practical Statistics · mock interviews · hackathons

HANDS-ON PRACTICE Pick one resource per area (statistics, ML, SQL, coding, mocks) and commit to finishing only those.

COMMON MISTAKES Collecting resources instead of practising · Watching videos without practising

CHECK YOURSELF

1. Three days before the interview, should you start a new topic?

Generally no: repeat weak topics and project stories, do a mock, and rest; new material rarely sticks and raises anxiety.

2. How many resources should you use?

Few: one for practice problems, one for theory depth, one for mocks; depth of use matters more than count.

3. What do you do after a rejection?

Write down the questions you struggled with, fix those gaps, and ask for feedback if possible; treat it as data for the next interview.

Interview tips, tricks and hacks

- 1. Put sessions on real dates.** A plan that says "Tuesday 7 pm: window functions, 45 minutes" gets done. A plan that says "study SQL this week" does not. Use the app's planner.
- 2. Protect the daily practice block above everything.** If a day goes wrong, do the forty-five minutes of timed practice and drop the rest.
- 3. Reschedule, do not abandon.** Missing three days is normal. Move the sessions forward, cut the least important, and carry on.
- 4. Cut breadth, never the core.** With limited time keep SQL, statistics, ML fundamentals and one project.
- 5. Verify the checklist with a timer and a recording.** Tick only what you have demonstrated this week.
- 6. Use few resources, deeply.** One source of practice problems, one for theory, one way to do practice interviews. Collecting resources is a form of procrastination.
- 7. Spend the last week on your weak three.** For most candidates: SQL timing, plain-language statistics, project deep dives.
- 8. Do two full mock interviews before the first real one.** Spoken, timed, with someone who interrupts.
- 9. Apply early and broadly.** Early interviews at companies you care less about are the best practice for the ones you care about most.
- 10. Log every interview.** Questions asked, what went well, what did not. Within three interviews you will see the pattern.

Frequently asked questions

4 questions interviewers ask on this chapter, with model answers. Cover the answer, say yours aloud, then compare.

THE 12-WEEK PLAN

Q1 You have only 3 weeks before your first interview. What do you prioritise?

Follow the **4-week crash** logic: SQL (daily timed problems, windows and cohorts), statistics and A/B testing (definitions, test selection, sample size), ML core (algorithms, metrics, leakage) and the question bank, practised with reveal-answer drills in Intuitive mode. Add one project walk-through and two mocks. Skip exotic topics; depth on fundamentals gives the most offers per hour.

MASTER READINESS CHECKLIST

Q2 You can do everything on the list except explaining concepts simply. How do you fix that in a week?

Practise daily with the 60-second rule: pick one concept, record yourself explaining it to a non-technical listener, play it back, and trim jargon. Use the Intuitive mode for each topic as a model, then rewrite it in your own words. Do two live mocks with a friend who interrupts with "why?". Clarity is a trainable skill and improves quickly with feedback.

COMPANY-WISE EMPHASIS CHEAT SHEET

Q3 You have interviews at a bank's analytics team in 3 weeks and at a startup in 6 weeks. How do you sequence?

Weeks 1 to 3: core SQL/Python/statistics/ML plus bank extras (scorecards, WoE/IV, KS/Gini, credit-risk concepts, SQL/SAS). Weeks 4 to 6: add product-sense extras (A/B testing, metrics frameworks, case rounds) and a polished project. Keep daily timed SQL throughout since both test it.

RESOURCES AND FINAL ADVICE

Q4 After a rejection, what is the most useful thing to do in the next 48 hours?

Write down every question you remember, mark the ones you missed or answered weakly, and close each gap (study, redo the problem, rehearse the explanation). Add them to your mistakes log and the Review drill, and ask the recruiter for feedback if possible. Rejections are data; the fastest improvement comes from processing them while they are fresh.

Appendices

Appendix A. Study plans

All three plans keep the same order: coding and SQL first, statistics before machine learning, projects once you can build end to end, practice interviews last. The app's Planner turns each plan into dated sessions that you can move around.

The twelve-week plan (about 12 hours a week)

Week	Focus	By the end of the week
1	Orientation, Python fundamentals, NumPy	30 easy coding problems; mean, median and a train-test split written from scratch
2	SQL basics: joins, aggregation, subqueries	40 SQL problems
3	SQL window functions, CTEs, cohorts, retention	Retention and streak queries written from memory
4	pandas, visualisation, EDA	A full exploration of a messy dataset
5	Statistics and probability	Bayes drills; choosing the test for ten scenarios
6	A/B testing and hypothesis testing in Python	A one-page experiment design
7	ML core: regression, logistic, trees, ensembles	Logistic regression and k-means from scratch
8	Feature engineering, metrics, imbalance; Project 1	A classification project, end to end
9	Time series, recommenders, clustering; Project 2	A forecasting or recommender project
10	Deep learning, NLP, GenAI and RAG; Project 3	An NLP or RAG project
11	Data engineering, PySpark, cloud, Git, Docker, MLOps	One project deployed behind an API in a container
12	Business cases, DSA, resume, behavioural, question bank	Five or more practice interviews

The eight-week plan (about 16 hours a week, some prior exposure)

Week	Focus	By the end of the week
1	Orientation, Python, NumPy, SQL basics	20 easy coding and 40 SQL problems
2	Advanced SQL, pandas, visualisation, EDA	Retention and streak queries from memory; one full EDA
3	Statistics and probability	Bayes drills; test-selection practice
4	A/B testing and experimentation	A one-page experiment design
5	ML core	Logistic regression and k-means from scratch
6	Features, metrics, imbalance; the project	One classification project, end to end

Week	Focus	By the end of the week
7	Essentials of time series, recommenders, deep learning, GenAI, MLOps	Attention and RAG explained aloud
8	Cases, DSA, resume, behavioural, question bank	Five or more practice interviews

The four-week plan (about 20 hours a week, interview soon, basics in place)

Week	Focus	By the end of the week
1	SQL, the biggest filter	Retention and streak queries from memory; three timed problems a day
2	Statistics, probability, A/B testing	CLT, p-value and confidence interval explained aloud; one experiment designed
3	ML core and metrics	Eight algorithms explained; metrics chosen for fraud, churn and default
4	Cases, projects, behavioural, practice interviews	Two practice interviews; readiness checklist complete

The daily rhythm (two to three hours)

Block	Time	What
Practice	45 min	SQL or coding problems against a timer
Concepts	60 min	One topic: study it, then explain it aloud in 60 seconds
Project	45 min	The current project milestone
Review	15 min	Your mistakes log and the app's review drill

Appendix B. Chapters, priorities and time

Chapter	Title	Priority	Topics	Time in the app	Cheat sheet
0	Orientation	Read first / use throughout	3	28 min	
1	Python Programming	High	10	3 h 49 min	yes
2	SQL	Highest	14	5 h 45 min	yes
3	NumPy, pandas, Visualization	High	7	3 h	yes
4	Data Structures and Algorithms (light)	Medium	3	1 h 5 min	yes
5	Statistics and Probability	Highest	11	5 h 30 min	yes
6	A/B Testing and Experimentation	High	5	2 h 35 min	yes
7	Machine Learning Fundamentals and Algorithms	Highest	15	6 h 45 min	yes
8	Feature Engineering, Metrics, Imbalanced Data	High	7	3 h 35 min	yes
9	Time Series, Clustering, Recommenders, Anomaly Detection	Medium-High	4	2 h 5 min	yes
10	Deep Learning, NLP, GenAI/LLMs	Medium-High	7	3 h 50 min	yes

Chapter	Title	Priority	Topics	Time in the app	Cheat sheet
11	Data Engineering and Tooling	Medium	5	2 h 40 min	yes
12	MLOps and Deployment Basics	Medium	3	1 h 7 min	yes
13	Business Sense, Product Metrics, Case Studies	High	6	2 h 30 min	
14	Projects: What to Build and How to Present	Highest	6	2 h 7 min	
15	Resume, Behavioral and HR Round	High	5	1 h 40 min	
16	Master Question Bank with Answers	Read first / use throughout	6	2 h 45 min	
17	12-Week Study Plan, Checklists, Resources	Read first / use throughout	4	47 min	

Priority reflects how often and how deeply the area is tested for entry-level roles. "Time in the app" is reading and practice time for the topics, not including external problem sets or projects.

Appendix C. Formula sheet

Statistics

Quantity	Formula
Sample variance	sum of $(x - \text{mean})^2$ divided by $(n - 1)$
Standard deviation	square root of the variance
z-score	$(x - \text{mean}) / \text{SD}$
Correlation	$\text{covariance}(x, y) / (\text{SD of } x \text{ times SD of } y)$
Standard error of a mean	$\text{SD} / \sqrt{n}$
Standard error of a proportion	$\sqrt{p(1 - p) / n}$
95% confidence interval	estimate $\pm 1.96 \times \text{standard error}$
Bayes' theorem	$P(H \text{ given } E) = P(E \text{ given } H) \times P(H) / P(E)$
At least one success in n tries	$1 - (1 - p)^n$
Binomial mean and variance	np and $np(1 - p)$
Sample size per group (rule of thumb)	$16 \times \text{variance} / (\text{difference})^2$, at 5% significance and 80% power
False positives across m tests	$1 - 0.95^m$ chance of at least one, at the 5% level

Machine learning and metrics

Quantity	Formula
Linear model	$\text{prediction} = w \cdot x + b$
Logistic (sigmoid)	$p = 1 / (1 + e^{(-z)})$, where $z = w \cdot x + b$
Odds ratio for a coefficient w	e^w
Gradient descent step	w becomes $w - \text{learning rate} \times \text{gradient of the loss}$

Quantity	Formula
Ridge (L2) and lasso (L1) penalties	$\lambda \times \text{sum of } w^2 \text{ and } \lambda \times \text{sum of absolute } w$
Gini impurity	$1 - \text{sum of (class share)}^2$
Precision	$TP / (TP + FP)$
Recall (sensitivity)	$TP / (TP + FN)$
Specificity	$TN / (TN + FP)$
F1	$2 \times \text{precision} \times \text{recall} / (\text{precision} + \text{recall})$
MAE and RMSE	mean of absolute errors; square root of the mean squared error
R-squared	$1 - (\text{residual sum of squares} / \text{total sum of squares})$
TF-IDF	term frequency $\times$ log(number of documents / documents containing the term)
Intersection over union	area of overlap / area of union
PSI	sum over buckets of $(\text{actual share} - \text{expected share}) \times \ln(\text{actual share} / \text{expected share})$

Appendix D. Decision tables

Which statistical test?

Situation	Test
One mean against a known value	One-sample t-test
Two independent group means	Welch's t-test
The same units before and after	Paired t-test
Three or more group means	ANOVA, then pairwise comparisons
Two proportions (conversion rates)	Two-proportion z-test or chi-square
Two categorical variables	Chi-square test of independence
Small or clearly non-normal samples	Mann-Whitney U (independent), Wilcoxon (paired)
Strength of a linear relationship	Pearson correlation; Spearman for ranks

Which metric?

Situation	Metric
Missing a positive is very costly (disease, fraud)	Recall, at an acceptable precision
False alarms are very costly (blocking real email)	Precision
Both matter	F1, or a point chosen on the precision-recall curve
Costs are known in money	Expected cost at each threshold
Only the top K cases can be acted on	Precision at K
Rare positives, comparing models	PR-AUC
Numerical prediction, large errors especially harmful	RMSE
Numerical prediction, all errors proportional	MAE

Situation	Metric
Ranking and recommendation	NDCG, precision at K

Which model first?

Situation	Start with
Any new problem	A simple baseline: a rule, or linear or logistic regression
Tabular data, accuracy matters	Gradient-boosted trees
Decisions must be explained (lending, healthcare)	Logistic or linear regression, or a shallow tree
Images, audio, free text	A pretrained deep model, fine-tuned
Text classification on a budget	TF-IDF with logistic regression
Forecasting a daily business metric	Seasonal naive, then a model with lag features
Customer segmentation	K-means on scaled features
Very rare events with few labels	Rules plus an isolation forest

Which SQL pattern?

The question asks for	Reach for
Rows with no match	LEFT JOIN with IS NULL, or NOT EXISTS
Duplicates	GROUP BY with HAVING COUNT(*) > 1
Top N per group, Nth highest	A ranking window function in an inner query
Running total, moving average	SUM or AVG OVER an ordered window
Change from the previous period	LAG
Consecutive days	Date minus ROW_NUMBER, then group
Retention	First date per user, then a LEFT JOIN on the later date
Funnel	COUNT(DISTINCT user) per step
Rows into columns	SUM(CASE WHEN ...)
Hierarchy	A recursive CTE

Which chart?

Purpose	Chart
Compare categories	Bar chart, starting at zero
Change over time	Line chart
Distribution of one variable	Histogram or box plot
Relationship between two numbers	Scatter plot
Many pairwise relationships	Correlation heatmap

Appendix E. Checklists

Before you call yourself ready

- I solved an easy coding problem in 15 minutes and a medium one in 30, this week, against a timer.
- I solved a medium SQL problem in 15 minutes and a hard one in 25, and wrote a retention query from memory.
- I recorded 60-second explanations of the p-value, confidence interval, CLT, power and Type I and II errors, and they made sense on playback.
- I chose the right test for five scenarios and computed a sample size by hand.
- I explained ten algorithms with their assumptions, strengths and main settings.
- I chose metrics and thresholds for a fraud, a churn and a loan-default problem, and justified them by cost.
- I can describe attention, TF-IDF and a RAG system in a few sentences each.
- I have a two-minute and a ten-minute version of each project, and I verified every number on my resume.
- I have six STAR stories and a 90-second introduction, rehearsed aloud.
- I have done at least two full practice interviews.

The experiment design checklist

- The hypothesis, and the decision it will inform.
- One primary metric, guardrail metrics, and the unit of randomisation.
- Sample size, duration in whole weeks, and the minimum effect worth detecting.
- Risks: novelty, interference, seasonality, tracking errors.
- The analysis: which test, the confidence interval, how segments will be treated.
- The decision rule for each possible result.

The project deep-dive checklist

- The problem, the user and the decision that changes.
- The data: source, size, grain, quality problems and what you did about them.
- The target definition and the prediction moment.
- The split, and why it matches how the model would be used.
- The metric, and why not accuracy.
- The baseline, the final model and the gain.
- How leakage was prevented.
- Where the model fails.
- How you would deploy and monitor it.
- What you would do differently.

Interview day

- The day before: confirm the time and the link, test camera and microphone, reread your resume and project notes, stop studying early and sleep.

- Just before: water, pen and paper, the job description, your three questions to ask.
- During: listen to the whole question, clarify, think aloud, structure the answer, check in with the interviewer.
- When stuck: say what you know, reason from a small example, state what you would verify.
- Afterwards: write down every question, note what to improve, send a short thank-you message.

Appendix F. Where to practise and read further

Practice

- **SQL:** LeetCode SQL 50, DataLemur, StrataScratch, HackerRank, Mode.
- **Coding:** LeetCode, HackerRank, GeeksforGeeks.
- **Data and ML:** Kaggle datasets, competitions and Learn micro-courses; DrivenData; Analytics Vidhya hackathons.
- **Practice interviews:** Pramp, Interviewing.io, peers, or recording yourself.
- **Interview experiences:** AmbitionBox, Glassdoor, LeetCode Discuss, GeeksforGeeks.

Courses and videos

- StatQuest with Josh Starmer, for statistics and ML intuition.
- Andrew Ng's Machine Learning and Deep Learning specialisations.
- CampusX, Krish Naik and Codebasics, for interview series with an Indian context.
- fast.ai Practical Deep Learning; the Hugging Face NLP and LLM course.
- 3Blue1Brown, for linear algebra and neural network intuition.

Books

- *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow* by Aurélien Géron.
- *An Introduction to Statistical Learning* (free online).
- *Practical Statistics for Data Scientists* by Bruce, Bruce and Gedeck.
- *Ace the Data Science Interview* by Kevin Huo and Nick Singh.
- *Trustworthy Online Controlled Experiments* by Kohavi, Tang and Xu, for A/B testing.
- *Designing Machine Learning Systems* by Chip Huyen, for production ML.

Final advice

Depth on the fundamentals wins more offers than breadth on the exotic. Know every number on your resume. Practise aloud. Log what you miss and close each gap within two days. Apply early and widely. Keep up with new developments, but never at the expense of SQL, statistics and core machine learning.

Glossary

280 terms used in this book and in data science interviews, in plain language.

A

A/A test An experiment where both groups get the same experience, run to check that the testing system itself is sound; it should find a "significant" difference only about 5% of the time.

A/B test A randomised experiment that compares a control version with a changed version to measure the effect of the change.

Accuracy The share of predictions that are correct. Misleading when one class is rare.

ACID Four guarantees of a database transaction: atomic (all or nothing), consistent, isolated and durable.

Activation function The non-linear function applied to a neuron's weighted sum, such as ReLU or sigmoid. It lets networks learn curved patterns.

Adam A widely used optimiser for neural networks that adapts the step size for each weight.

Agent (LLM) A language model that plans and calls tools such as search or a database in a loop until a task is finished.

Aggregation Reducing many rows to a summary value such as a count, sum or average.

Alias (SQL) A temporary name given to a column or table within a query.

Alpha (significance level) The false-positive rate you accept in a test, commonly 5%.

Anomaly detection Finding rare observations that differ markedly from normal behaviour.

ANOVA A test that compares the means of three or more groups at once.

Anti-join A query that returns rows from one table that have no match in another, for example customers with no orders.

API A defined way for one program to request something from another, such as asking a model service for a prediction.

ARIMA A classical forecasting model that predicts a series from its own past values, differences and past errors.

Attention The mechanism in transformers by which each token weighs every other token when building its representation.

AUC (ROC-AUC) The probability that a model ranks a random positive above a random negative; 0.5 is chance, 1 is perfect.

Autoencoder A neural network trained to compress and then reconstruct its input; large reconstruction error can signal an anomaly.

B

Backpropagation The method for computing how much each weight in a network contributed to the error, using the chain rule.

Bagging Training many models on random resamples of the data and averaging them to reduce variance.

Base rate How common something is in the population before any evidence is considered.

Baseline A simple reference model or rule that any serious model must beat.

Batch processing Processing data in scheduled chunks, as opposed to continuously.

Bayes' theorem The rule for updating a probability in the light of evidence: posterior is proportional to likelihood times prior.

Bias (model) Error from a model being too simple to capture the pattern; it leads to underfitting.

Bias-variance trade-off The tension between models that are too simple (bias) and too flexible (variance).

Big-O notation A description of how an algorithm's running time or memory grows with input size.

Binary search Finding a value in sorted data by repeatedly halving the search range; $O(\log n)$.

Binomial distribution The distribution of the number of successes in a fixed number of independent yes-or-no trials.

Bonferroni correction Dividing the significance level by the number of tests to limit false positives when testing many hypotheses.

Boosting Building models in sequence, each one correcting the errors of those before it.

Bootstrap Resampling the data with replacement many times to estimate the uncertainty of a statistic.

Breadth-first search (BFS) Exploring a graph level by level with a queue; finds the shortest path when edges are unweighted.

Broadcast join A Spark join that copies a small table to every machine so the large table does not have to move across the network.

Broadcasting NumPy's rule for combining arrays of different shapes by stretching dimensions of size one.

C

Calibration How well predicted probabilities match observed frequencies.

Canary release Sending a small share of traffic to a new model or version before a full rollout.

Cardinality The number of distinct values in a column.

Categorical dtype A pandas column type that stores each distinct value once and small integer codes per row, saving memory for columns with few distinct values.

Central Limit Theorem (CLT) The average of many independent observations is approximately normal, whatever the original distribution.

Chi-square test A test of whether two categorical variables are related, or whether counts fit an expected pattern.

Churn Customers ceasing to use a product or service.

Class imbalance A classification problem in which one class is much rarer than the other.

Class weights Weights that make errors on a rare class count more in the training loss, so the model cannot ignore it.

Classification Predicting a category, such as fraud or not fraud.

Clustering Grouping similar items without predefined labels.

Coerce To convert values to a type while turning the ones that cannot be converted into missing values (`errors="coerce"`) instead of raising an error.

Cohort A group of users who share a starting event, such as signing up in the same month.

Cold start The difficulty of making recommendations for new users or items with no history.

Collaborative filtering Recommending items based on the behaviour of similar users or the co-occurrence of items.

Comprehension (Python) A compact expression that builds a list, set or dictionary from a loop.

Concept drift A change over time in the relationship between inputs and the outcome.

Confidence interval A range produced by a method that captures the true value in a stated share (such as 95%) of repeated samples.

Confounder A third variable that influences both the supposed cause and the effect, creating a misleading association.

Confusion matrix A table of true positives, false positives, true negatives and false negatives.

Context manager An object used in a `with` block that sets something up and guarantees the clean-up, such as closing a file, even if an error occurs.

Context window The maximum amount of text a language model can consider at once.

Control group The group in an experiment that does not receive the change.

Conversion rate The share of users who complete a desired action.

Correlation A measure from -1 to 1 of how closely two variables follow a straight-line relationship.

Cosine similarity The similarity of two vectors measured by the angle between them: 1 means same direction, 0 means unrelated.

Covariance A measure of how two variables vary together; its size depends on their units.

Cross-validation Estimating performance by training and validating on several different splits of the data.

CTE (common table expression) A named temporary result defined with `WITH` and used within a query.

CUPED A technique that reduces variance in experiments by adjusting for each user's pre-experiment behaviour.

D

Data drift A change over time in the distribution of a model's inputs.

Data lake Cheap storage for raw files of any type, structured later when read.

Data leakage A model being trained with information it will not have at prediction time.

Data skew (Spark) Uneven data across partitions, often one very common key, which makes one task much slower than the rest.

Data warehouse A database of cleaned, structured tables organised for analysis.

DataFrame A table of labelled columns in pandas (or Spark).

DAU / MAU Daily and monthly active users. Their ratio measures how habitually a product is used.

Decision tree A model that predicts by asking a sequence of yes-or-no questions about the features.

Decorator A Python function that wraps another function to add behaviour.

Deep learning Machine learning with neural networks of many layers.

Dense rank A ranking in which ties share a rank and no numbers are skipped.

Depth-first search (DFS) Exploring a graph by going as deep as possible along each path before backtracking.

Difference-in-differences Estimating an effect by comparing the change in a treated group with the change in a comparable untreated group.

Dimensionality reduction Compressing many features into fewer while keeping most of the information.

Distribution A description of which values a variable takes and how often.

Docker A tool that packages code with its environment into an image that runs the same anywhere.

Dropout Randomly switching off units during neural network training to reduce overfitting.

Dynamic programming Solving a problem by storing and reusing the answers to overlapping sub-problems.

E

Early stopping Halting training when validation performance stops improving.

EDA (exploratory data analysis) Systematically inspecting a dataset to understand its structure, quality and patterns before modelling.

Effect size The magnitude of a difference or relationship, as distinct from its statistical significance.

Elastic net A regularisation that combines L1 and L2 penalties.

ELT Extract, load, then transform inside the warehouse.

Embedding A list of numbers representing an item, such as a word or sentence, so that similar items are close together.

Ensemble A combination of several models that predicts better than its members.

Entropy A measure of impurity or uncertainty, used to choose splits in decision trees.

Epoch One complete pass through the training data.

ETL Extract, transform, then load into a data store.

Expected value The long-run average of a random quantity.

F

F1 score The harmonic mean of precision and recall.

False negative A real positive case that the model missed.

False positive A negative case that the model wrongly flagged.

Fan-out Row multiplication caused by joining a table to one with several matching rows.

Feature An input variable used by a model.

Feature engineering Creating or transforming input variables so that patterns are easier for a model to learn.

Feature store A system that defines features once and serves the same values for training and for live prediction.

Fine-tuning Further training of a pretrained model on a smaller, task-specific dataset.

Foreign key A column that refers to the primary key of another table.

Funnel A sequence of steps users pass through, with the share continuing at each step.

G

Generalisation A model's ability to perform well on data it has not seen.

Generator (Python) A function that yields values one at a time, producing them only when needed.

GIL (global interpreter lock) The mechanism in standard Python that lets only one thread run Python code at a time.

Gini impurity A measure of how mixed the classes are in a group, used to choose decision tree splits.

Git A version control system that records the history of code and supports collaboration.

Gradient The direction and rate of steepest increase of a function; in training, it shows how to change each weight.

Gradient boosting An ensemble that adds shallow trees in sequence, each fitted to the remaining errors.

Gradient descent Minimising a loss by repeatedly stepping in the direction that reduces it.

Grain What one row of a table represents.

Guardrail metric A metric that must not get worse while another is being improved.

Guesstimate A structured estimate of an unknown quantity built from stated assumptions.

H

Hallucination A language model producing fluent but false or unsupported content.

Hash map A structure that stores key-value pairs with constant-time lookup on average; a Python dictionary.

Hashable Able to be used as a dictionary key or set member; in practice, immutable.

HAVING The SQL clause that filters groups after aggregation.

Heap A structure that gives fast access to the smallest (or largest) item; used for top-k problems.

Holdout set Data kept aside and not used in training, for honest evaluation.

Hyperparameter A setting chosen before training, such as tree depth or learning rate.

Hypothesis test A procedure for deciding whether data are consistent with a default assumption of no effect.

I

Idempotent Producing the same result however many times it is run; a desirable property of data pipelines.

Immutable Unable to be changed after creation, like Python strings and tuples.

Imputation Filling in missing values.

Index (database) A sorted structure that speeds up lookups and joins at the cost of slower writes.

Inertia (k-means) The sum of squared distances from each point to its cluster centre; it always falls as k grows.

Inner join A join that keeps only rows with a match in both tables.

Interference When the treatment given to one user affects the outcome of another, breaking a standard A/B test.

Interquartile range (IQR) The spread of the middle 50% of values: the third quartile minus the first.

IoU (intersection over union) The overlap between a predicted and a true region divided by their combined area; used in object detection.

Isolation forest An anomaly detector based on how few random splits are needed to isolate a point.

Iterator An object that hands out one item each time `next()` is called and signals the end with `StopIteration`; it is what a `for` loop consumes.

K

K-fold Cross-validation that splits the data into k parts and uses each in turn for validation.

K-means A clustering method that assigns points to the nearest of k centres and updates the centres repeatedly.

K-nearest neighbours (KNN) Predicting from the labels of the k most similar training examples.

Kernel trick Letting a model such as an SVM behave as if features were transformed into a richer space, without computing the transformation.

L

L1 regularisation (lasso) A penalty on the absolute size of weights; it can set weights to exactly zero.

L2 regularisation (ridge) A penalty on the squared size of weights; it shrinks them smoothly.

LAG A window function that returns a value from a previous row.

Laplace smoothing Adding a small count to every category so that an unseen word or value never gets a probability of zero.

Law of large numbers As the sample grows, its average converges to the true mean.

Lazy evaluation Deferring computation until a result is actually required, as in Spark and Python generators.

Learning curve A plot of training and validation error against training-set size, used to diagnose bias and variance.

Learning rate The size of each step in gradient descent.

Left join A join that keeps every row of the left table, with NULLs where the right has no match.

Lift The improvement of a treatment over control, or of a model over random selection.

Linear regression Predicting a number as a weighted sum of features.

LLM (large language model) A very large transformer trained to predict the next token of text.

Log loss A loss for classification that heavily penalises confident wrong probabilities.

Logistic regression A linear model that outputs a probability for a yes-or-no outcome.

Long and wide format Two layouts of the same data: long has one row per observation, wide has one column per category. `pivot` and `melt` convert between them.

LoRA A fine-tuning method that trains small added matrices instead of all of a model's weights.

Loss function The quantity a model is trained to minimise; a measure of how wrong its predictions are.

LTV (lifetime value) The total value a customer is expected to bring over the relationship.

M

MAD (median absolute deviation) The median of the distances from the median; a measure of spread that resists outliers.

MAE (mean absolute error) The average absolute difference between predictions and actual values.

MAPE Mean absolute percentage error. Unreliable when actual values are near zero.

MASE Mean absolute scaled error: a forecast's MAE divided by the MAE of a naive benchmark; below 1 beats the benchmark.

Matrix factorisation Representing users and items by short vectors whose match predicts preference.

Maximum likelihood (MLE) Choosing the parameter values that make the observed data most probable.

Mean The arithmetic average.

Median The middle value when the data are sorted; robust to outliers.

Merge (pandas) Combining two tables on key columns, like a SQL join.

Method chaining Writing a pandas transformation as one sequence of calls, each returning a new DataFrame, so the steps read top to bottom.

Minimum detectable effect (MDE) The smallest effect an experiment is designed to detect reliably.

MLOps The practices for deploying, monitoring and maintaining machine learning models in production.

Model registry A catalogue of trained model versions with their stage, metrics and origin.

Multicollinearity Strong correlation among features, which makes linear model coefficients unstable.

Multiple testing Running many tests, which raises the chance of at least one false positive.

Mutable Able to be changed in place, like Python lists and dictionaries.

N

Naive Bayes A classifier that applies Bayes' theorem assuming the features are independent given the class.

NDCG A ranking metric that rewards placing the most relevant items nearest the top.

Neural network A model made of layers of simple connected units with adjustable weights.

Normal distribution The symmetric bell-shaped distribution described by a mean and a standard deviation.

Normal equations The linear system $(X.T @ X) \beta = X.T @ y$ whose solution gives the least-squares regression coefficients.

Normalisation (database) Organising tables to remove redundancy.

North-star metric The single metric that best captures the value a product delivers to its users.

Novelty effect A temporary change in behaviour caused by newness, which fades with time.

NULL A missing or unknown value in a database.

Null hypothesis The default assumption in a test, usually that there is no effect or no difference.

O

Odds ratio The factor by which the odds of an outcome change for a one-unit change in a feature.

OLAP Analytical workloads involving large scans and aggregations.

OLTP Transactional workloads involving many small reads and writes.

One-hot encoding Representing a category as a set of yes-or-no columns, one per category.

Outlier An observation far from the bulk of the data.

Overfitting Fitting noise in the training data, so that performance on new data is worse.

P

p-value The probability of data at least as extreme as observed, assuming the null hypothesis is true.

Paired test A test for two measurements on the same units, such as before and after.

Parameter A value a model learns from data, such as a weight.

Parquet A compressed, column-oriented file format suited to analytics.

Partition (window) The group of rows over which a window function is computed.

PCA (principal component analysis) A method that finds new uncorrelated axes capturing the most variance in the data.

Peeking Checking experiment results repeatedly and stopping when they look significant, which inflates false positives.

Percentile The value below which a given percentage of observations fall.

Pipeline (ML) A chain of preprocessing and modelling steps fitted and applied together.

Pivot Reshaping data so that the values of one column become separate columns.

Poisson distribution The distribution of the number of independent events in a fixed interval.

Posterior The updated probability after evidence has been taken into account.

Power The probability that a test detects a real effect of a given size.

PR-AUC The area under the precision-recall curve; more informative than ROC-AUC when positives are rare.

Precision Of the cases flagged positive, the share that really are positive.

Primary key A column, or set of columns, that uniquely identifies each row of a table.

Prior The probability assigned before seeing the evidence.

Prompt The text given to a language model to instruct it.

PSI (population stability index) A number summarising how much a variable's distribution has shifted between two periods.

R

R-squared The share of variance in the outcome explained by a regression model.

RAG (retrieval-augmented generation) Retrieving relevant passages and giving them to a language model so that it answers from them.

Random forest An ensemble of decision trees trained on random samples and feature subsets, then averaged.

Randomisation Assigning units to groups by chance, so that the groups are comparable.

Recall Of the real positive cases, the share the model found.

Recall@k The share of all relevant items that appear in the top k results.

Recursive CTE A query that refers to itself to walk a hierarchy or generate a sequence.

Regression Predicting a numerical value.

Regularisation Penalising model complexity to reduce overfitting.

ReLU The activation function $\max(0, x)$.

Reproducibility The ability to rerun work and obtain the same result.

Retention The share of users who return after a given period.

RMSE Root mean squared error; it penalises large errors more than MAE does.

ROC curve A plot of true positive rate against false positive rate across all thresholds.

S

Sample ratio mismatch (SRM) An experiment split that differs from the planned ratio, indicating a bug.

Sampling bias A sample that does not represent the population it is meant to describe.

Scaling (features) Putting features on a comparable numerical range.

Schema The structure of a database: its tables, columns and types.

Seasonality A pattern that repeats at a fixed interval, such as weekly or yearly.

Selection bias A distortion that arises when the people or rows being compared chose, or were chosen, in a way related to the outcome.

Self join Joining a table to itself, for example to compare employees with their managers.

Sensitivity Another name for recall, or the true positive rate.

Series (pandas) A single labelled column of data.

Shadow deployment Running a new model on live traffic without acting on its predictions.

SHAP A method that attributes a prediction to the contribution of each feature.

Shuffle (Spark) Moving data between machines to group or join it; the main cost in distributed processing.

Sigmoid An S-shaped function that maps any number to a value between 0 and 1.

Silhouette score A measure from -1 to 1 of how well separated clusters are.

Simpson's paradox A trend that appears in several groups but reverses when the groups are combined.

Sliding window A technique that maintains a moving range over a sequence to solve substring and subarray problems efficiently.

SMOTE An oversampling method that creates synthetic examples of the minority class.

Softmax A function that turns a list of scores into probabilities that sum to one.

Specificity Of the real negative cases, the share correctly identified.

Stable sort A sort that keeps items with equal keys in their original order; Python's sort is stable, which allows sorting by several keys in turn.

Standard deviation The typical distance of values from the mean; the square root of the variance.

Standard error The standard deviation of an estimate, such as a sample mean; it shrinks with the square root of the sample size.

STAR Situation, Task, Action, Result: a structure for answering behavioural questions.

Stationarity A time series whose statistical properties do not change over time.

Stratified sampling Sampling so that each subgroup is represented in proportion.

Streaming Processing data continuously as events arrive.

Supervised learning Learning from examples that include the correct answer.

Support vector machine (SVM) A classifier that finds the boundary with the widest margin between classes.

Switchback test An experiment that alternates treatment and control over time blocks in the same market, used when users influence each other.

T

t-test A test comparing means when the population standard deviation is estimated from the sample.

Target encoding Replacing a category with the average outcome for that category, computed without leakage.

Temperature A setting that controls how random a language model's output is.

Test set Data used once, at the end, to estimate real-world performance.

TF-IDF A weighting that scores a word highly if it is frequent in a document but rare across documents.

Threshold The score above which a classifier predicts the positive class.

Token A piece of text, often part of a word, that a language model reads or produces.

Tracing (code) Following a program by hand, line by line, while recording the value of each variable.

Training-serving skew A difference between how features are computed in training and in production.

Transfer learning Reusing a model trained on one task as the starting point for another.

Transformer A neural network architecture based on attention, used in modern language models.

Treatment group The group in an experiment that receives the change.

Truthy and falsy How Python treats a value in an `if`: `None`, zero and anything empty are falsy; everything else, including the string "False", is truthy.

Type I error Rejecting a true null hypothesis; a false positive.

Type II error Failing to reject a false null hypothesis; a miss.

U

Underfitting A model too simple to capture the pattern; it performs poorly even on training data.

Unsupervised learning Finding structure in data that has no labels.

Upsert (MERGE) Updating rows that already exist and inserting rows that do not, matched on a key; it makes repeated loads safe.

V

Validation set Data used to tune and compare models during development.

Variance (model) Error from a model being too sensitive to the particular training sample; it leads to overfitting.

Variance (statistics) The average squared distance of values from the mean.

Vector database A store that finds the embeddings most similar to a query embedding.

Vectorisation Applying an operation to a whole array at once instead of looping over elements.

View (NumPy) An array that shares memory with another, so that changes to one affect the other.

Virtual environment An isolated set of Python packages for one project.

W

WAPE Weighted absolute percentage error: total absolute error divided by total actual value.

Weight sharing Reusing the same weights at every position, as a convolution filter does across an image.

Window function A SQL function that computes across related rows while keeping every row.

Worked example A problem solved step by step, showing every intermediate result, used to learn a method before trying it yourself.

Z

z-score The number of standard deviations a value lies from the mean.



Now go and practise.

Reading builds the frame; speaking and solving fill it in. Open the app, pick the topic you find hardest, explain it aloud in sixty seconds, and then do it again tomorrow. Interviews reward the person who has practised saying it, far more than the person who has read it twice.

DS Prep · Created by Rahul Kapil